Ambarella®

www.ambarella.com VERSION 1.5 | AUGUST 1, 2025

CV22FS Hardware Programming Reference Manual - Preliminary

SUMMARY DESCRIPTION

The CV22FS hardware programming reference manual (PRM) lists software-programmable registers accessible from the CV22FS CPU core, providing comprehensive information on each field of each register, as well as step-by-step programming instructions.

The PRM document also includes an overview of the system memory map, power-on configuration options, and available Arm® interrupts.

KEY TOPICS

- Register programming information
- Power on configuration (POC) detail
- Boot mode summary
- DSP subsystem overview
- Detailed peripheral interface information
- Memory allocation summary

CONTENTS

1. Introduction 1
2. System Boot, Clocking and Configuration 6
3. DSP Subsystem 14
4. DMA Engine Subsystem 16
5. Graphics DMA (GDMA) 35
6. Video / Sensor Input Interface 50
7. Video Output (VOUT) 77
8. Delayed VSync Generator 121
9. I2S Interface 123
10. Gigabit Ethernet Controller 144
11. USB Device Controller 201
12. USB Host Controller 237
13. SPI NAND Flash 269
14. SD Controllers 302
15. Scratchpad 352
16. Security 390
17. AXI Configuration 400
18. Generic Interrupt Controller (GIC) 414
19. GPIO 432
20. SSI / SPI 453
21. UART 499
22. I2C Two-Wire 520
23. Controller Area Network (CAN) Bus 538
24. RTC 563
25. Pulse Width Modulator (PWM) 565
26. Interval Timers 568
27. Watchdog Timer (WDT) 576
28. Chip Error Handling Unit (CEHU) 580
29. CPU Error Monitoring Unit (CEMU) 590
30. AMBA ECRU 595
31. DRAM 601
32. ECC Handler (ECCH) 612
33. Temperature Sensor 614
34. Important Notice 621
35. Revision History 622

Confidential
For Mekra Lang only
1.5 (08012025)

The material in this document is for information only. Ambarella assumes no responsibility for errors or omissions and reserves the right to change, without notice, product specifications, operating characteristics, packaging, ordering, etc. Ambarella assumes no liability for damage resulting from the use of information contained in this document. All brands, product names and company names are trademarks of their respective owners. Further information, including additional disclaimers, appears in the Important Notice at the end of this document.

 Copyright © 2025, Ambarella International LP.

1. INTRODUCTION

This is the preliminary hardware programming reference manual for the CV22FS camera processor from Ambarella. Please note that the information described in this hardware programming reference manual is subject to change.

The document starts with a brief overview of the chip (this chapter) and follows with essentials for system boot / configuration (Chapter 2) and the DSP subsystem (Chapter 3). Registers and programming details for CV22FS peripherals are subsequently provided (Chapter 4 +).

This overview continues with:

- (Section 1.1) Introduction: Functional Block Diagram
- (Section 1.2) Introduction: Memory Map

Applicable third-party reference documents include:

- Samsung 10 nm LPP 40Kb OTP Datasheet
- Samsung 10 nm LPP 40Kb OTP User Guide
- Samsung LPDDR4 Datasheet
- Micron LPDDR4 Datasheet
- JEDEC Standard for LPDDR4
- Winbond SPI NAND Flash W25N01GV RevA Datasheet
- Micron SPI NAND Flash MT29F4G01AAADD Datasheet
- Toshiba Serial Interface NAND TC58CVG2S0H*AI* Technical Datasheet
- ISO 11898-1-2003
- ISO 11898-2-2003
- ISO 11898-4-2004
- ISO 26262-2018
- BOSCH CAN with Flexible Data-Rate (CAN FD) white paper version 1.1

For Mekra Lang only Confidential (08.12.2025)

1.1 Introduction: Functional Block Diagram

The following figure shows the CV22FS processors and peripherals described in this document according to system address. Please contact an Ambarella representative for more information.

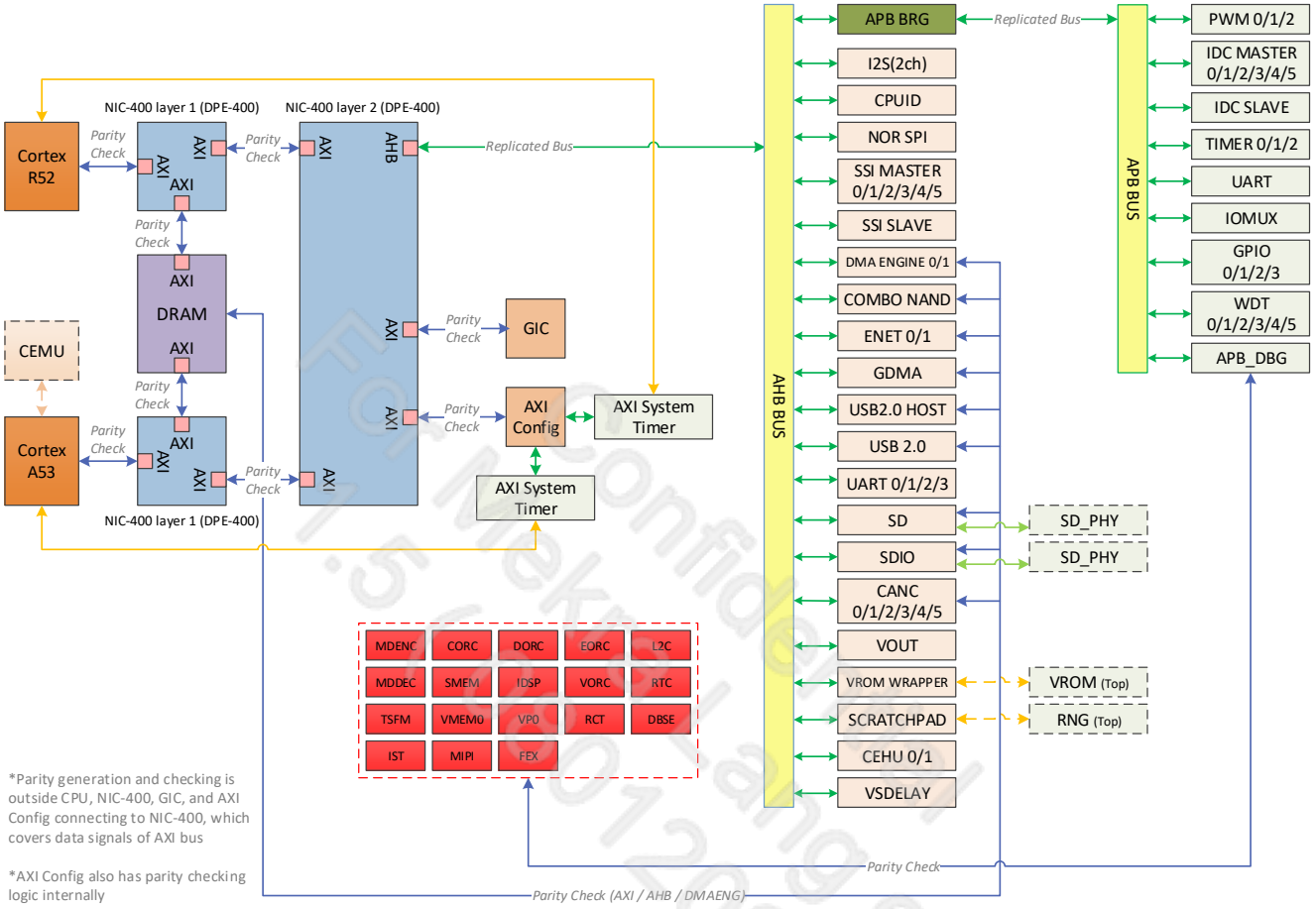


Figure 1-1. Functional Block Diagram for the CV22FS Chip.

- The AHB bus provides a high-performance interconnect for high-performance peripherals such as the DMA engine and I2S controllers. The AHB bus connects to an APB bus via bridge logic.
- The APB bus connects many lower-performance peripherals, including timers and the UART controllers.
- One AXI bus connects the Arm® Cortex®-A53 cores and interfaces to the SDRAM controller.
- Another AXI bus connects to Arm® Cortex®-R52 core and interfaces to the SDRAM controller.
- A config / debug bus connects to the APB by a bridge and allows configuration of the RCT / DSP / VP / DRAM / MIPI and more.
- The video DSP connects to video input (VIN) and video output (VOUT) modules and the SDRAM controller via proprietary hardware interfaces. The VIN and VOUT modules are programmed by DSP APIs.

For Ambarella Confidential only (1.5 Rev 01 2025)

1.2 Introduction: Memory Map

This chapter includes the following sections:

- (Section 1.2.1) Memory Map: AMBA Address Mapping
- (Section 1.2.2) Memory Map: AHB Bus
- (Section 1.2.3) Memory Map: APB Bus

1.2.1 Memory Map: AMBA Address Mapping

Description	Memory Range	Size
DRAM Space	0000.00000-7FFE.FFFF 7FFF.0000-7FFF.FFFF (not allowed for R52). 8000.0000-DFFD.FFFF DFFE.0000-DFFE.7FFF (R52 only) DFFE.8000-DFFF.FFFF	3.5 GB
AHB Bus	E000.0000 - E3FF.FFFF	64 MB
APB Bus	E400.0000 - E7FF.FFFF	64 MB
GPV NIC400_A		
(each for R52 and A53)	F000.0000 - F00F.FFFF	1 MB
GPV NIC400_B (R52 only)	F100.0000 – F10F.FFFF	1 MB
AXI Configuration (R52 only)	F200.0000 – F200.FFFF	64 KB
GIC	F300.0000 – F3FF.FFFF	16 MB

Table 1-1. AMBA Address Map.

Note:

Minimum External DDR SDRAM recommended is 512 MB.

For Confidential Lang only (1.5 Feb 01 2025)

1.2.2 Memory Map: AHB Bus

Instance	Address Mapping	Size	Notes
CPU ID	E000.0000 - E000.0FFF	4 KB	
SPI NOR	E000.1000 - E000.1FFF	4 KB	
Flash CPU Bus	E000.2000 - E000.2FFF	4 KB	Flash controller DMA
SDHC	E000.4000 - E000.4FFF	4 KB	
SDXC	E000.5000 - E000.5FFF	4 KB	
USB Device Controller	E000.6000 - E000.7FFF	8 KB	
VOUT	E000.8000 - E000.AFFF	12 KB	
GDMA	E000.C000 - E000.CFFF	4 KB	
ENET MAC	E000.E000 - E000.EFFF	4 KB	
ENET DMA	E000.F000 - E000.FFFF	4 KB	
SSIS 0	E001.0000 - E001.0FFF	4 KB	
SSI 0	E001.1000 - E001.1FFF	4 KB	
SSI 1	E001.2000 - E001.2FFF	4 KB	
SSI 2	E001.3000 - E001.3FFF	4 KB	
SSI 3	E001.4000 - E001.4FFF	4 KB	
UART 0	E001.7000 - E001.7FFF	4 KB	
UART 1	E001.8000 - E001.8FFF	4 KB	
UART 2	E001.9000 - E001.9FFF	4 KB	
UART 3	E001.A000 - E001.AFFF	4 KB	
DAI	E001.D000 - E001.DFFF	4 KB	
OHCI	E001.E000 - E001.EFFF	4 KB	USB Host OHCI
EHCI	E001.F000 - E001.FFFF	4 KB	USB Host EHCI
DMA	E002.0000 - E002.0FFF	4 KB	
DMA1	E002.1000 - E002.1FFF	4 KB	
SCRATCHPAD_NSEC	E002.2000 - E002.2FFF	4 KB	
FIO8K_LO	E002.3000 - E002.3FFF	4 KB	Flash controller 8K FIFO low bytes
FIO8K_HI	E002.4000 - E002.4FFF	4 KB	Flash controller 8K FIFO high bytes
CEHU0	E002.5000 - E002.5FFF	4 KB	
CEHU1	E002.6000 - E002.6FFF	4 KB	
AMBA_ECRU	E002.7000 - E002.7FFF	4 KB	
CAN0	E002.8000 - E002.8FFF	4 KB	
CAN1	E002.9000 - E002.9FFF	4 KB	
CAN2	E002.A000 - E002.AFFF	4 KB	
CAN3	E002.B000 - E002.BFFF	4 KB	
CAN4	E002.C000 - E002.CFFF	4 KB	
CAN5	E002.D000 - E002.DFFF	4 KB	
SCRATCHPAD_SEC	E002.F000 - E002.FFFF	4 KB	
SCRATCHPAD_RAM	E003.0000 - E003.0FFF	4 KB	8 KB RAM low bytes
SCRATCHPAD_RAM	E003.1000 - E003.1FFF	4 KB	8 KB RAM high bytes
ENET1 MAC	E003.2000 - E003.2FFF	4 KB	
ENET1 DMA	E003.3000 - E003.3FFF	4 KB	
SSI4	E003.4000 - E003.4FFF	4 KB	
SSI5	E003.5000 - E003.5FFF	4 KB	

Table 1-2. AHB Address Ranges.

Proprietary and Confidential Information Language only 2025

1.2.3 Memory Map: APB Bus

Instance	Address Mapping	Size	Notes
UART 0	E400.0000 - E400.0FFF	4 KB	
PWM 0	E400.3000 - E400.3FFF	4 KB	
Timer 0	E400.4000 - E400.4FFF	4 KB	
Timer 1	E400.5000 - E400.5FFF	4 KB	
Timer 2	E400.6000 - E400.6FFF	4 KB	
IDCS	E400.7000 - E400.7FFF	4 KB	I2C slave instance
IDC 0	E400.8000 - E400.8FFF	4 KB	I2C master 0
IDC 1	E400.9000 - E400.9FFF	4 KB	I2C master 1
IDC 2	E400.A000 - E400.AFFF	4 KB	I2C master 2
IDC 3	E400.B000 - E400.BFFF	4 KB	I2C master 3
PWM 1	E400.C000 - E400.CFFF	4 KB	
PWM 2	E400.D000 - E400.DFFF	4 KB	
IOMUX	E400.E000 - E400.EFFF	4 KB	
WDT	E400.F000 - E400.FFFF	4 KB	
GPIO0	E401.0000 - E401.0FFF	4 KB	
GPIO1	E401.1000 - E401.1FFF	4 KB	
GPIO2	E401.2000 - E401.2FFF	4 KB	
GPIO3	E401.3000 - E401.3FFF	4 KB	
WDT_CORE0	E401.4000 - E401.4FFF	4 KB	
WDT_CORE1	E401.5000 - E401.5FFF	4 KB	
WDT_CORE2	E401.6000 - E401.6FFF	4 KB	
WDT_CORE3	E401.7000 - E401.7FFF	4 KB	
WDT_R52	E401.8000 - E401.8FFF	4 KB	
IDC 4	E401.9000 - E401.9FFF	4 KB	I2C master 4
IDC 5	E401.A000 - E401.AFFF	4 KB	I2C master 5

Table 1-3. APB Address Ranges.

For Evaluation Only
(Jan 2025)

2. SYSTEM BOOT, CLOCKING AND CONFIGURATION

2.1 Boot: Overview

This chapter includes the following sections:

- (Section 2.1) Boot: Overview
- (Section 2.2) Boot: Power On Configuration

2.2 Boot: Power On Configuration

The CV22FS chip uses a 32-bit power on configuration with strapping functions on external pins. After deassertion of the **POR_L** external reset pin, the chip generally latches the initial hardware settings on the pins.

The table in this section lists the power on configuration bits POC[0:31] and corresponding strapping pins. However, if POC[31] is set to 1, some (or all) of the configuration data may be obtained from OTP ROM (Section 2.2.9).

Notes:

- Unused bit—Although external pin is assigned to the bit, it is not used for a power-on configuration function. these pins should be connected to ground using the pull-down resistor.
- Reserved bit—No external pin is assigned to the bit, and it is not used for a power-on configuration function.

Additional configuration information is listed in the following subsections:

- (Section 2.2.1) POC: Pin Assignments
- (Section 2.2.2) POC: Clock Configuration
- (Section 2.2.3) POC: Boot Mode Summary
- (Section 2.2.4) POC: Force USB Boot
- (Section 2.2.5) POC: SPI NOR Boot
- (Section 2.2.6) POC: SPI NAND Boot
- (Section 2.2.7) POC: MMC Boot
- (Section 2.2.8) POC: Boot Bypass
- (Section 2.2.9) POC: OTP ROM

For Mediatek Lang only (08012025) Confidential 1.5

2.2.1 POC: Pin Assignments

Ball	Bits	Pin	Function	Setting
G17	0	**ENET_TXD_0**	Unused	-
H18, F17, E17	3:1	**ENET_TXD_[3:1]**	IDSP / Core / DDR / Vision clock configuration	Refer to Section 2.2.2 below
E14,D14	5:4	**ENET1_TXD_[1:0]**	Boot Mode[1:0] Select	Select boot mode (SPI NOR is the CV22FS default) 2'b00 SPI NOR 2'b01 SPI NAND 2'b10 eMMC
E16	6	**ENET1_TXD_2**	Secure boot	0 Regular boot 1 Secure boot
-	7	-	Reserved	-
W20	8	**CLK_AU**	Boot bypass	0 Disable boot bypass 1 Enable boot bypass
-	9	-	Reserved	-
L17	10	**ENET_PTP_PPS**	Force USB boot	Setting this pin to high overrides the current boot mode setting and enables USB boot. This bit is always derived from the pin.
-	12:11	-	Reserved	-
Y16	13	**UART0_AHB_TX**	Boot option 0 select	Refer to Table 2-4
V12	14	**UART1_AHB_TX**	Boot option 1 select	
U14	15	**UART_APB_TX**	Boot option 2 select	
W18	16	**I2S_SO_0**	Boot option 3 select	
R19	17	**GPIO25**	Boot option 4 select	
V18	18	**I2S_WS**	Boot option 5 select	
V15	19	**SVSYNC**	Boot option 6 select	
W15	20	**SHSYNC**	Boot option 7 select	
A13	21	**ENET_GTX_CLK**	Unused	-
-	22	-	Reserved	-
-	23	-	CLOCK_SRC_MODE0	Refer to CLOCK_SRC_MODE1 This bit can only be set from OTP
-	24	-	CLOCK_SRC_MODE1	CLOCK_SRC_MODE1 Select clock sources for core and DDR clocks 00 Normal sources 01 Reference clock 10 Reserved 11 Reserved This bit can only be set from OTP
-	25	-	Reserved	
-	26	-	PERIPH_CLOCK_MODE	Controls clock to USBHOST, SD, FLASH controller. Set this bit to 1 when GCLK_CORE is < 466 MHz 0 Peripherals use GCLK_CORE/2 1 Peripherals use GCLK_CORE This bit can only be set from OTP
-	29:27	-	Reserved	-
W16	30	**PWM0**	Unused	-

For Internal Use Only Confidential

Ball	Bits	Pin	Function	Setting
Y19	31	**PWM1**	System configuration data source	Source of sys-config data 0 Config data is set by pins 1 Config data is read from on-chip OTP (except Bit[31]) This bit is always derived from the pin

Table 2-1. Power On Configuration Pin Assignments.

2.2.2 POC: Clock Configuration

The POC[3:1] bits enable the user to select from the pre-configured PLL settings in the table below. These frequencies are enabled during the power on process only, and the settings may be overridden by system software for operation.

POC[3:1]	VDSP (gclk_core)	IDSP (gclk_idsp)	DRAM (gclk_dram)	Vision (gclk_vision)	Cortex0 (gclk_cortex0)	Cortex1 (gclk_cortex1)
000	696	696	1824	792	456	1008
001	672	432	396	792	456	864
010	840	1008	2160	1320	552	1488
011	840	792	2160	744	456	1200
100	504	792	2160	1008	528	1320
101	504	408	1584	744	480	864
110	696	768	1920	720	480	1152
111	216	216	396	504	396	396

Table 2-2. Power On Clock Rates (MHz).

2.2.3 POC: Boot Mode Summary

Several bits of the power on configuration pins are decoded to determine the CV22FS boot mode. The following table shows the available boot-mode settings, including which pins must be controlled (with pull-up / pull-down) and which can be left unconnected (floating). Table 2-4 outlines the CV22FS shared boot options.

POC Bit	POC Description	Pin	SPI NOR Boot	SPI NAND Boot	MMC Boot	USB Boot	Boot Bypass (SPI NAND)	Boot Bypass (MMC)
0	Unused	**ENET_TXD_0**	0	0	0	0	0	0
3:1	clk_freq	**ENET_TXD_3, ENET_TXD_2, ENET_TXD_1**	0-7	0-7	0-7	0-7	0-7	0-7
4	boot_modes[0]	**ENET1_TXD_0**	0	1	0	x	1	0
5	boot_modes[1]	**ENET1_TXD_1**	0	0	1	x	0	1
6	secure_boot	**ENET1_TXD_2**	0/1	0/1	0/1	0/1	0/1	0/1
7	Reserved	-	-	-	-	-	-	-
8	boot_bypass	**CLK_AU**	x	0	0	x	1	1
9	Reserved	-	-	-	-	-	-	-

For Ambarella Confidential Use Only (20120201)

POC Bit	POC Description	Pin	SPI NOR Boot	SPI NAND Boot	MMC Boot	USB Boot	Boot Bypass (SPI NAND)	Boot Bypass (MMC)
10	force_usb_boot	**ENET_PTP_PPS**	0	0	0	1	0	0
12:11	Reserved	-	-	-	-	-	-	-
13	boot_options[0]	**UART0_AHB_TX**	x	0/1	0/1	x	0/1	0/1
14	boot_options[1]	**UART1_AHB_TX**	0/1	0/1	0/1	x	0/1	0/1
15	boot_options[2]	**UART_APB_TX**	0/1	0/1	0/1	x	0/1	0/1
16	boot_options[3]	**I2S_SO_0**	0/1	0/1	0/1	x	0/1	0/1
17	boot_options[4]	**GPIO25**	0/1	0/1	0/1	x	0/1	0/1
18	boot_options[5]	**I2S_WS**	0/1	0/1	0/1	x	0/1	0/1
19	boot_options[6]	**SVSYNC**	x	x	0/1	x	x	0/1
20	boot_options[7]	**SHSYNC**	0/1	0/1	0/1	x	0/1	0/1
21	Unused	**ENET_GTX_CLK**	0	0	0	0	0	0
22	Reserved	-	-	-	-	-	-	-
23	clk_mode[0]	**OTP ONLY**	0/1	0/1	0/1	0/1	0/1	0/1
24	clk_mode[1]	**OTP ONLY**	0/1	0/1	0/1	0/1	0/1	0/1
25	Reserved	-	-	-	-	-	-	-
26	periph_clock_mode	**OTP ONLY**	0/1	0/1	0/1	0/1	0/1	0/1
29:27	Reserved	-	-	-	-	-	-	-
30	Unused	**PWM0**	0	0	0	0	0	0
31	Sysconfig-src	**PWM1**	0/1	0/1	0/1	0/1	0/1	0/1

Table 2-3. Power On Boot Mode.

Notes:

- Boot bypass is supported only for SPI NAND and eMMC boot, and is used for debugging purposes.
- The boot_options value will be ignored during boot bypass; however, this value cannot be unknown—it should be the same as the boot option for SPI NAND / MMC boot.
- The maximum clock that peripheral controllers (USB, flash, and SD) can run is 500 MHz. For applications which require a > 500 MHz core clock, bit 26 of POC must be set to 0 to enable peripherals to run at half of the core clock. If the core clock is ≤ 500 MHz, bit 26 of POC can be set to 1 to enable peripherals to run at the same frequency.
- For MMC boot, use eMMC with a specification higher than or equal to Version 4.4, which supports boot function. A generic SD card or an eMMC 4.1 *should not* be used.
- For the hardware connection, use SD0 pins to connect to eMMC (4.4 or later).

Boot Option	SPI NOR	SPI NAND	eMMC	Force USB
boot_option[7]	8K signed image size in secure boot	8K BST code size select	emmc_bst_8k	
boot_option[6]			trsca_sel_boot	
boot_option[5]	Reset command	SCK mode	trsca_en_boot	

Confidential For Ambarella Lang Only 1.5 080 Logo

Boot Option	SPI NOR	SPI NAND	eMMC	Force USB
boot_option[4]	Use 2-byte address	4K-page select 2K-page select	emmc_reset_default_on	
boot_option[3]	Use alternative clock	Skip ECC check for blank pages	emmc_boot_sd_ pull	
boot_option[2]	SPI mode selection	SPI NAND ECC BCH enable	sd4_boot	
boot_option[1]	LSB first	SPI NAND spare cell 2X	sd8_boot	
boot_option[0]		Flash fast boot		

Table 2-4. Shared Boot Options Matrix.

2.2.4 POC: Force USB Boot

The USB boot option is enabled using POC bit 10. Setting this pin to high overrides the current boot mode setting and forces the chip into USB boot mode.

2.2.5 POC: SPI NOR Boot

SPI NOR boot options are listed below. Five boot option pins are required.

Boot Option	Description	POC Pin
Signed image size	Image size for secure boot 0 4K 1 8K In non-secure boot, this bit is reserved	20
Reset command	Reset command for the external SPI NOR device 0 Send 0x66 - 0x99 1 Send 0xFF - 0xF0	18
2-byte address	Number of bytes in SPI NOR command address phase 0 3 bytes 1 2 bytes	17
Use alternative clock	SPI NOR reference clock selection 0 Use GCLK_ENET as SPI NOR controller reference clock 1 Use GCLK_CORE as SPI NOR controller reference clock	16
SPI mode selection	SPI mode selection 0 SCPOL = 0 (SPI mode 00) Always set to 0	15
LSB first	Controls TX / RX data swap 0 MSB first (do not swap TX send data and RX receive data) 1 LSB first (swap TX send data and RX receive data)	14

Table 2-5. SPI NOR Boot Options.

Note:

Boot code reads RCT system configuration register and configures SPI NOR.

For Mekra Lang only (08012025) Confidential

2.2.6 POC: SPI NAND Boot

SPI NAND boot options are listed below. Seven boot option pins are required.

Boot Option	Description	POC Pin
bst_8k	0 4K BST code size 1 8K BST code size	20
SCK mode	0 Mode 0 (CPOL = 0. CPHA = 0) 1 Mode 3 (CPOL = 1. CPHA = 1)	18
2k_pg	0 4K-page 1 2K-page	17
skip_bnk_ecc	0 Do not skip ECC check for blank pages 1 Skip ECC check for blank pages	16
ecc_bch	Enable ECC BCH function or not	15
sp_2x	0 Support 1X SPI NAND spare area When SPI NAND page size is 0: Spare area = 128 B When SPI NAND page size is 1: Spare area = 64 B 1 Support 2X SPI NAND spare area When SPI NAND page size is 0: Spare area = 256 B When SPI NAND page size is 1: Spare area = 128 B	14
flash_fastboot	0 Normal SPI NAND flash component read cycle time 1 Short SPI NAND flash component read cycle time	13

Table 2-6. SPI NAND Boot Options.

Note:

RCT system configuration register drives SPI NAND controller input.

2.2.7 POC: MMC Boot

MMC boot options are listed below. Seven boot option pins are required.

Boot Option	Description	POC Pin
emmc_bst_8k	0 4K BST code size 1 8K BST code size	20
trsca_sel_boot	0 For tRSCA=75 SD clock cycles 1 For tRSCA=10000 SD clock cycles	19
trsca_en_boot	Set this value to 1 to delay the timing of boot initialization	18
emmc_reset_default_on	When EMMC boot = 1 and emmc_reset_default_on = 1, the SD controller drives sd_reset signal SMIO_35	17
emmc_boot_sd_pull	0 No pull on sd_cmd, and sd_d[7:0] 1 Pull on sd_cmd, and sd_d[7:0]	16

For Confidential Preliminary Lang only (20250801)

Boot Option	Description	POC Pin
sd4_boot	Use 4 SD data pins for booting	15
sd8_boot	Use 8 SD data pins for booting	14

Table 2-7. MMC Boot Options.

The following table lists the MMC boot options for pull enable.

Pad Name	Boot Mode Setting	boot_sd_pull setting	boot_data_bit setting
	boot_mode (POC[5:4])	emmc_boot_sd_pull (boot_option[3])	{sd4_boot, sd8_boot} (boot_option[2:1])
smio_3(sd_cmd)	2'b10	1'b1	2'b00(1-bit boot) 2'b10(4-bit boot) 2'bx1(8-bit boot)
smio_18(sd_d0)			2'b00(1-bit boot) 2'b10(4-bit boot) 2'bx1(8-bit boot)
smio_19(sd_d1)			2'b10(4-bit boot) 2'bx1(8-bit boot)
smio_20(sd_d2)			2'b10(4-bit boot) 2'bx1(8-bit boot)
smio_21(sd_d3)			2'b10(4-bit boot) 2'bx1(8-bit boot)
smio_22(sd_d4)			2'bx1(8-bit boot)
smio_23(sd_d5)			2'bx1(8-bit boot)
smio_24(sd_d6)			2'bx1(8-bit boot)
smio_25(sd_d7)			2'bx1(8-bit boot)

Table 2-8. MMC Boot Options for Pull Enable.

Notes:

- Pads pull_en and pull_sel are decided by boot_mode, emmc_boot_sd_pull, {sd4_boot, and sd8_boot}
- eMMC 1-bit boot sd_cmd and sd_d0 must pull up
- eMMC 4-bit boot sd_cmd, sd_d0, sd_d1, sd_d2, sd_d3, sd_d4 must pull up
- eMMC 8-bit boot sd_cmd, sd_d0, sd_d1, sd_d2, sd_d3, sd_d4, sd_d5, sd_d6, sd_d7 must pull up

Confidential For Mekra Lang only 1.5 (08012025)

The following lists the MMC boot modes.

EMMC Boot Mode	Boot Mode Setting	boot_sd_pull setting	boot_data_bit setting
	boot_mode (POC[5:4])	emmc_boot_sd_pull (boot_option[3])	{sd4_boot, sd8_boot} (boot_option[2:1])
eMMC 1-bit boot	2'b10	1'b1	2'b00 (1-bit boot)
eMMC 4-bit boot			2'b10 (4-bit boot)
eMMC 8-bit boot			2'bx1 (8-bit boot)

Table 2-9. MMC Boot Mode.

Notes:

- The RCT system configuration register drives SD / SDIO controller input.
- When eMMC boot mode is chosen, the **SD_RESET** pin must be pulled low by the external resistor.

2.2.8 POC: Boot Bypass

The boot bypass option is enabled using POC[8]. Boot bypass can be used when the boot process from media (SPI NAND and eMMC) will be bypassed, and also to enable system debugging.

2.2.9 POC: OTP ROM

The CV22FS chip enables users to select which power on configuration bits are loaded from OTP ROM or from strapping pins.

The CV22FS OTP ROM region for POC function is 64 bits; i.e., the lower 32 bits store system configuration data and the upper 32 bits handle selection.

If strapping pin 31 (**PWM1**) is pulled down, all configuration values are loaded from the pins. If it is pulled up, all power on configuration bits are loaded from OTP, with the exception of POC bits 10 and 31, which are always loaded from POC pins.

The contents of the OTP ROM module are set at the factory and are not user-accessible.

For Mekra Lang only
1.5 (08012025)
Confidential

3. DSP SUBSYSTEM

This chapter includes the following sections:

- (Section 3.1) DSP: System Overview
- (Section 3.2) DSP: Control of Video Input (VIN) and Output (VOUT)

3.1 DSP: System Overview

The DSP (digital signal processing) subsystem interfaces with the video input module, the video output, and the DRAM controller. The subsystem runs DSP microcode loaded by the Arm, and performs tasks per command control from the Arm such as video processing, image processing, video post-processing (for output), and video encode / decode.

The following shows the DSP subsystem and its interface to the remainder of the chip.

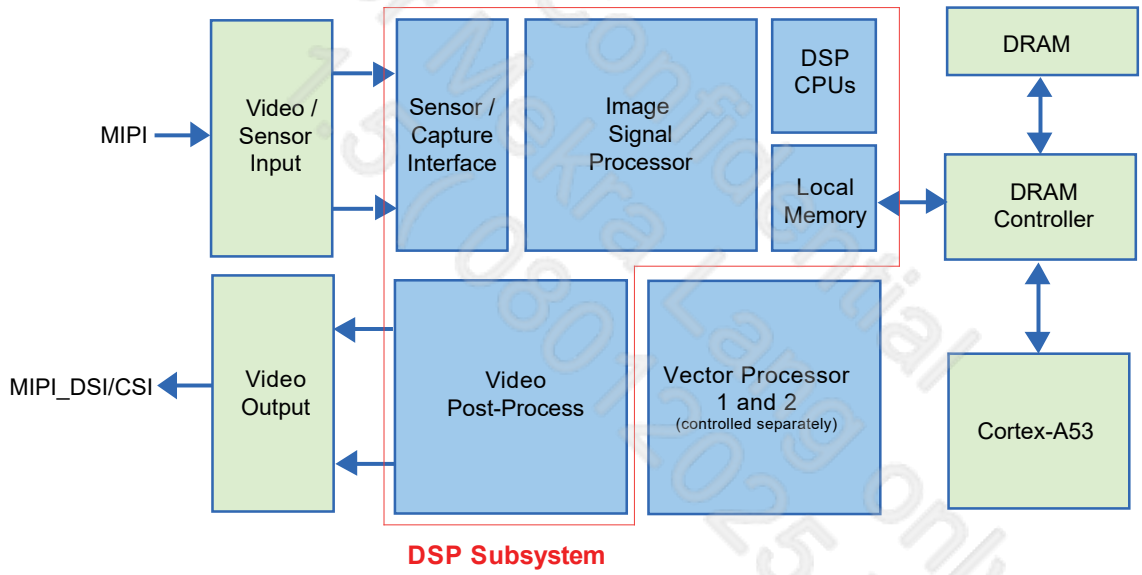


Figure 3-1. The DSP Subsystem of the CV22FS Chip and its Relationship with the System Modules.

There is no direct hardware access to the DSP subsystem from memory-mapped registers. The Arm uses a dedicated DRAM memory location to send commands to or receive status updates from the DSP subsystem.

The CV22FS DSP subsystem accepts YUV OR RAW data from CV22FS sensor interfaces or other external sources.

Image processing in the CFA domain includes bad pixel and vignetting corrections, chromatic aberration removal, noise and leakage filtering, as well as local exposure adjustment. In the RGB domain, demosaicing, color remapping, and color space transformations are performed.

YUV domain processes include sharpening, dewarping, chroma noise filtering, 3D motion compensated temporal filtering (MCTF), and resampling. Stream-out from the YUV processor is rescaled and used for preview / overlay.

The rescaler supplies data to the analytics engine. AAA statistics are sampled from the RGB and YUV domains.

3.2 DSP: Control of Video Input (VIN) and Output (VOUT)

The DSP subsystem handles all register programming for VIN. Refer to Chapter 7 “Video Output (VOUT)” for details on VOUT. See the CV22FS chip datasheet for interface pins and settings.

Confidential
For Mekra Lang only
1.5 (08012025)

4. DMA ENGINE SUBSYSTEM

This chapter provides programming information for the CV22FS direct memory access (DMA) engine subsystem, and includes the following sections:

- (Section 4.1) DMA: Features
- (Section 4.2) DMA: Engines and Channel Assignment
- (Section 4.3) DMA: Operation Overview
- (Section 4.4) DMA: Operation Detail
- (Section 4.5) DMA: Transaction Non-Descriptor Mode
- (Section 4.6) DMA: Transaction Descriptor Mode
- (Section 4.7) DMA: Descriptor Format
- (Section 4.8) DMA: Programming DMA Registers
- (Section 4.9) DMA: Registers
- (Section 4.10) DMA: Errors
- (Section 4.11) DMA: Interrupts

Note:

In this chapter, the terms DMA transaction and DMA operation are used interchangeably.

4.1 DMA: Features

The DMA engine subsystem provides direct memory access (DMA) functionality for AMBA peripherals. It moves blocks of data between the main memory and AHB-bus peripheral devices, as well as between areas within the main memory itself. Features of the DMA engine subsystem include the following:

- Sixteen DMA channels
  - For I2S, SPI NOR, UART, and SSI interfaces.
- Memory-to-memory transfers including support for transfers between memory and peripherals with DMA between:
  - AHB bus devices and RAM memory
  - Two memory areas in RAM memory
- Physical DMA
- Programmable address, transaction size (8 to 1024 bytes), and transfer size (1, 2, 4, or 8 bytes)
- Programmable transfer count (up to 4 MB)
- Memory interface transaction size optimized for RAM burst operations
- Direct software writes or descriptor information from memory to program DMA control

Confidential For Mekra Lang only (08/12/23)

- DMA scatter / gather via chained descriptor list in memory with DMA control information source
- Host processor initiates the DMA operation
- Interrupts signal the end of DMA operations
- Recognition and logging of error types that end DMA operations
- Sharing of the AHB bus master interface (and the memory interface) with round-robin arbitration
- Fully pipelined for highest throughput
- AHB bus slave interface for register reads and writes

4.2 DMA: Engines and Channel Assignment

The CV22FS DMA subsystem provides two DMA Engines (E002.0000 and E002.1000) and sixteen channels.

For DMA channel multi-function detail, refer to the register information (AHB_SCRATCHPAD_NSEC, E002.2000) below.

Offset	Register	Bits	Field	Access	Reset	Description
0x50	**AHBSP_dma0_channel_sel0**	31:30	**Reserved**	-	X	Refer to Table 4-3
		29:24	**dma0_channel3_sel**	RW	0x0	
		23:22	**Reserved**	RW	X	
		21:16	**dma0_channel2_sel**	RW	0x0	
		15:14	**Reserved**	RW	X	
		13:8	**dma0_channel1_sel**	RW	0x0	
		7:6	**Reserved**	RW	X	
		5:0	**dma0_channel0_sel**	RW	0x0	
0x54	**AHBSP_dma0_channel_sel1**	31:30	**Reserved**	RW	X	Refer to Table 4-3
		29:24	**dma0_channel7_sel**	RW	0x0	
		23:22	**Reserved**	RW	X	
		21:16	**dma0_channel6_sel**	RW	0x0	
		15:14	**Reserved**	RW	X	
		13:8	**dma0_channel5_sel**	RW	0x0	
		7:6	**Reserved**	RW	X	
		5:0	**dma0_channel4_sel**	RW	0x0	
0x58	**AHBSP_dma1_channel_sel0**	31:30	**Reserved**	RW	X	Refer to Table 4-3
		29:25	**dma1_channel3_sel**	RW	0x0	
		23:22	**Reserved**	RW	X	
		21:16	**dma1_channel2_sel**	RW	0x0	
		15:14	**Reserved**	RW	X	
		13:8	**dma1_channel1_sel**	RW	0x0	
		7:6	**Reserved**	RW	X	
		5:0	**dma1_channel0_sel**	RW	0x0	

For Mellanox Confidential Internal Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x5C	**AHBSP_dma1_channel_sel1**	31:30	**Reserved**	RW	X	Refer to Table 4-3
		29:24	**dma1_channel7_sel**	RW	0x0	
		23:22	**Reserved**	RW	X	
		21:16	**dma1_channel6_sel**	RW	0x0	
		15:14	**Reserved**	RW	X	
		13:8	**dma1_channel5_sel**	RW	0x0	
		7:6	**Reserved**	RW	X	
		5:0	**dma1_channel4_sel**	RW	0x0	

Table 4-1. AHB Scratchpad DMA Channel Selection Registers.

Instance	Channel	Source
dma_engine_amba0	0	dma0_channel0_sel[4:0]
	1	dma0_channel1_sel[4:0]
	2	dma0_channel2_sel[4:0]
	3	dma0_channel3_sel[4:0]
	4	dma0_channel4_sel[4:0]
	5	dma0_channel5_sel[4:0]
	6	dma0_channel6_sel[4:0]
	7	dma0_channel7_sel[4:0]
dma_engine_amba1	0	dma1_channel0_sel[4:0]
	1	dma1_channel1_sel[4:0]
	2	dma1_channel2_sel[4:0]
	3	dma1_channel3_sel[4:0]
	4	dma1_channel4_sel[4:0]
	5	dma1_channel5_sel[4:0]
	6	dma1_channel6_sel[4:0]
	7	dma1_channel7_sel[4:0]

Table 4-2. AHB Scratchpad General Control Register: DMA Channel Assignment Detail.

dma*_channel*_sel[4:0]	Selected Device
5'd1	ssi0_tx_dma_req
5'd2	ssi0_rx_dma_req
5'd3	ssi1_tx_dma_req
5'd4	ssi1_rx_dma_req
5'd5	nor_spi_tx_dma_req
5'd6	nor_spi_rx_dma_req
5'd7	ssis0_tx_dma_req
5'd8	ssis0_rx_dma_req
5'd9	uart0_tx_dma_req
5'd10	uart0_rx_dma_req
5'd11	uart1_tx_dma_req
5'd12	uart1_rx_dma_req
5'd13	uart2_tx_dma_req
5'd14	uart2_rx_dma_req

For Mentor Confidential 1.5 (08/25/25) Only

dma*_channel*_sel[4:0]	Selected Device
5'd15	uart3_tx_dma_req
5'd16	uart3_rx_dma_req
5'd17	ssi2_tx_dma_req
5'd18	ssi2_rx_dma_req
5'd19	ssi3_tx_dma_req
5'd20	ssi3_rx_dma_req
5'd21	i2s_tx_dma_req
5'd22	i2s_rx_dma_req
5'd23	ssi4_tx_dma_req
5'd24	ssi4_rx_dma_req
5'd25	ssi5_tx_dma_req
5'd26	ssi5_rx_dma_req

Table 4-3. AHB Scratchpad General Control Register: DMA Channel Assignment Detail 2.

4.3 DMA: Operation Overview

The DMA engine subsystem supports block transfers of sizes up to 4 MB. The exact transfer count of each DMA operation is set by software. DMA operations are performed as a series of main-memory and AHB-bus transactions to move the data block. The length of each AHB bus transaction is software-programmable so that it can be optimized according to system characteristics. Each individual AHB bus transaction is from 8 bytes to 1024 bytes, according to a value programmed into a DMA Channel Control register (**DMA_ch[n]_control**). Additionally, the data size of each data transfer on the AHB bus is software programmable to be 1, 2, 4 or 8 bytes.

The source and destination starting addresses are set by software. These addresses are incremented by the DMA engine subsystem to form the request addresses at the memory- and AHB-bus interfaces of the DMA engine subsystem. All are physical addresses.

The DMA control information determined by software (for example, source and destination starting addresses, transfer count, bus transaction size, data transfer size, and bus address increment mode) can be set in two ways: The first is by direct software writes to the DMA engine subsystem registers with DMA control information; the second is through descriptors in memory that hold the DMA control information and are initialized by software. To support DMA configuration from descriptors, the DMA engine subsystem contains logic to read the descriptors from tables in memory and load the appropriate registers with the DMA control information.

A DMA operation begins when a DMA channel is enabled and after its source and destination starting addresses, transfer count, bus transaction size, data transfer size, and bus address increment mode are configured. The DMA engine subsystem moves the data block, and the DMA operation ends naturally when the number of bytes specified by the transfer count have been moved. A DMA operation can also end early due to an error or a request from software.

When a DMA channel is disabled by hardware for any reason, an interrupt informs the host processor. Each DMA channel includes a register that identifies the cause of the interrupt. The cause can be an event such as a natural DMA operation end, a natural end of a descriptor chain, or any among several types of errors.

DMA channels share a single AHB bus master at the DMA engine subsystem bus interface. The subsystem has a single interface to memory that is shared by all DMA channels; it uses round-robin arbitration to choose the DMA channel whose request is serviced on the bus and memory interfaces.

The DMA engine subsystem contains several registers. These registers are accessed using single transactions initiated by host processor reads and writes and include DMA channel control and status information, DMA chan-

nel source and destination starting addresses, DMA channel descriptor addresses, and DMA engine subsystem control and status information.

For further details, refer to Section 4.4.

4.4 DMA: Operation Detail

DMA transactions move a block of data from a source data area to a destination data area. The data is transferred via a series of reads from the source data area and writes to the destination data area. When performing the reads and writes, the DMA engine subsystem begins by reading data at the lowest source location, writing this data to the lowest destination location, and then incrementing the source and destination addresses for the next data transfer.

Source and destination addresses are continually incremented for all subsequent data transfers until the entire data block has been moved or an error occurs. Either of these events ends the DMA transaction, and the DMA engine subsystem informs the host processor that the DMA transaction has ended with an interrupt.

The DMA operation can be summarized by the following:

1. The DMA channel is set up.
2. The DMA channel is enabled to transfer data. This starts the DMA transaction.
3. A series of reads and writes are performed to transfer the data from its source to the destination.
4. Data transfers end when the entire block of data has been moved or an error occurs.
5. DMA channel data transfer is automatically disabled by hardware.
6. Completion status for the DMA transaction is logged in the Channel Status register (**DMA_ch[n]_status**).

The DMA channel must be configured prior to the commencement of a DMA operation. Each DMA channel has three types of registers that hold channel configuration information for the DMA operation:

1. Channel Control (**DMA_ch[n]_control**)
2. Channel Source Address (**DMA_ch[n]_source_addr**)
3. Channel Destination Address (**DMA_ch[n]_dest_addr**)

The starting address for the reads from the source data area is written to the Channel Source Address register; the Channel Destination Address register is written with the starting address of the writes to the destination data area; and the Channel Control register is written to indicate the number of bytes to be moved, the AHB bus transaction size, the AHB bus data transfer size, the AHB bus address increment mode, and whether the source and destination data areas are in memory or in AHB bus peripheral devices.

Up to 4 MB can be moved by a DMA operation. The AHB bus transaction size can be set from 8 bytes to 1024 bytes. Data transfers on the AHB bus can be 1, 2, 4, or 8 bytes. DMA operations can be memory-to-memory, AHB-bus-to-memory, or memory-to-AHB-bus. Moves between AHB bus devices are not supported.

All information is programmed into the Channel Control register. The register map and set details for the DMA engine subsystem are provided in Section 4.8.

Perform the following steps to initiate and configure a DMA transaction:

1. Write the **Channel Source Address** and **Channel Destination Address** registers.
2. Write the **Channel Control** register with the remaining configuration information.
3. Set the data transfer enable flag to 1.

Once enabled, the DMA channel issues read requests to the data source and write requests to the data destination. Upon receiving read data, the DMA channel temporarily buffers it before writing it to the destination. The buffering accumulates enough data to issue requests to a DRAM controller, which will interpret these requests as burst requests.

Data reads and writes continue until the number of bytes shown in the count field of the Channel Control register, have been moved or until an error is reported. When either of these events occurs, the DMA transaction ends. Section 4.10 details the DMA error descriptions.

When the DMA transaction ends, DMA channel data transfer is automatically disabled. This occurs when the hardware resets the data transfer enable flag in the Channel Control register to 0 after the last data has been written to the destination.

The cause of the DMA transaction completion is logged in the Channel Status register. The Channel Status register also shows the number of bytes that were transferred.

Note that when an error ends the DMA transaction, the number of bytes shown may differ from the number of bytes that were intended to be transferred (for example, the value programmed into the count field of the Channel Control register).

4.5 DMA: Transaction Non-Descriptor Mode

The host processor sets up each DMA transaction in non-descriptor mode. A DMA channel is in non-descriptor mode when the Channel Control register descriptor mode bit is 0.

Before a DMA operation, software running on the host processor selects one of the DMA channels for the DMA operation. For details on Channel Assignment see Section 4.2. After selecting the channel, the software then configures the chosen channel by directly writing to the three registers that hold channel transaction-configuration information:

1. Channel Control register (**DMA_ch[n]_control**)
2. Channel Source Address (**DMA_ch[n]_source_addr**)
3. Channel Destination Address (**DMA_ch[n]_dest_addr**)

Note that the DMA channel must be disabled (Channel Control register-enable bit is 0) before it can be configured. Refer to the DMA registers in Section 4.8 for details.

The DMA transaction begins when the enable bit is written to 1 by the host processor, as the data transfer enable flag is the Channel Control register-enable bit in non-descriptor mode. Typically, the host processor writes the Channel Source Address and Channel Destination Address registers first; then, the host processor writes the Channel Control register with the remainder of the configuration information and sets the enable bit to 1.

When data transfer ends (either naturally, when the number of set bytes in the Channel Control register count field have been moved, or when an error occurs) the DMA channel generates an interrupt. This interrupt informs the host processor that the DMA transaction has completed.
The host processor recognizes this interrupt, and branches to a routine to handle it. The DMA channel signaling the interrupt is identified by reading the DMA Engine Interrupt register. The host processor can also read the Channel Status register to determine how the DMA transaction ended.

Confidential
For Meikang only
Feb 2015

4.6 DMA: Transaction Descriptor Mode

In descriptor mode, the host processor sets up several transactions in DMA descriptors located in memory. Each DMA descriptor contains:

- The DMA transaction configuration information described above, for one DMA transaction.
- A pointer to a memory location where the DMA transaction completion status will be written once the DMA transaction ends.
- A pointer to the descriptor for the next DMA transaction.

When the host processor enables the DMA channel by setting the Channel Control register-enable bit to 1, the DMA Engine Subsystem processes the DMA descriptors and DMA transactions occur for all descriptors without additional host processor intervention. A DMA channel is in descriptor mode when the Channel Control register descriptor mode bit is 1.

Prior to the execution of a DMA operation, software running on the host processor selects one of the DMA channels for the DMA operation. Subsequently, it writes descriptors to memory for several DMA transactions, and loads the Channel Descriptor Address (**DMA_ch[n]_descriptor**) register with the address of the descriptor for the first transaction.

The DMA channel must be disabled (set the Channel Control register-enable bit to 0) when setting up the descriptors for the first DMA transactions. When extending the DMA descriptor chain in memory, the DMA channel can be either enabled or disabled.

DMA descriptor processing begins when the host processor enables the DMA channel by writing the Channel Control register-enable bit to 1. When the DMA channel is enabled, the descriptor-processing logic fetches the descriptor for the first DMA transaction from memory at the location shown in the DMA Channel Descriptor Address register.

Using the contents of the fetched descriptor, the descriptor-processing logic loads the DMA channel transaction information registers, saves the DMA transaction status report address, and saves the next descriptor address. To do this, the descriptor-processing logic writes the following DMA channel registers:

- Channel Control (**DMA_ch[n]_control)**
- Channel Source Address (**DMA_ch[n]_source_addr)**
- Channel Destination Address (**DMA_ch[n]_dest_addr)**
- Channel Descriptor Address (**DMA_ch[n]_desc_addr)**

In descriptor mode, the data transfer enable flag is maintained by the descriptor-processing logic. After the DMA channel transaction information registers are loaded by the descriptor-processing logic, the descriptor-processing logic sets the data transfer enable flag to 1, and the transaction begins.

The data block is moved from its source to the destination. When the data transfer ends (either naturally, when the number of bytes shown in the count field of the DMA descriptor have been moved, or when an error occurs) the data transfer enable flag is reset to 0 by the descriptor-processing logic. The descriptor-processing logic then writes the transaction completion status found in the Channel Status register to memory at the report address location shown in the descriptor.

If the DMA descriptor end-of-chain bit is not set to 1, the descriptor-processing logic then fetches the next descriptor from memory, loads the channel transaction information registers, and starts the next transaction data transfer. This process continues until a DMA descriptor with an end-of-chain bit set to 1 is encountered, or until an error is encountered for cases in which the Channel Control register stop-on-error bit is set to 1.

Confidential
For Maker Lab only
(15 Oct 2025)

After the transaction described by a descriptor with its end-of-chain bit set to 1, or when an error is encountered and the Channel Control register stop-on-error bit is 1, the descriptor processing concludes and the DMA channel is automatically disabled. The channel then generates an interrupt, which informs the host processor that the transactions have been completed.

The host processor recognizes this interrupt, and branches to a routine to handle it. The DMA channel which signaled the interrupt is identified by reading the DMA Engine Interrupt register.

4.7 DMA: Descriptor Format

Descriptor fields, which are read-only, are described in detail in Section 4.4 above.

Memory Address	Descriptor	Comments
Descriptor address	Source starting address [31:0]	
Descriptor address + 4	Destination starting address [31:0]	
Descriptor address + 8	Next descriptor address [31:0]	
Descriptor address + 12	Report address [31:0]	
Descriptor address + 16	Transfer count	Refer to Table 4-5.
Descriptor address + 20	Control information	

Table 4-4. A DMA Descriptor is Composed of Six 32-bit Words.

Bits	Name	Access	Reset	Description
31:22	-	-	-	Reserved
21:0	**transfer_count**	R	-	Number of bytes to transfer See **DMA_ch[n]_control** description. The size of each descriptor memory block must be a multiple of eight bytes. The transfer count in each descriptor must also be a multiple of eight.

Table 4-5. DMA Descriptor Transfer Count.

Bits	Name	Access	Reset	Description
31:25	-	-	-	Reserved
24	**eoc**	R	-	End of chain flag DMA will stop after this descriptor. Note that if the **eoc** bit is set, the DMA channel will generate an interrupt when this descriptor's DMA operation ends without error.
23	**wm**	R	-	Write memory See **DMA_ch[n]_control** description.
22	**rm**	R	-	Read memory See **DMA_ch[n]_control** description.

For Ambarella Confidential Use Only
Ambarella Confidential 2025

Bits	Name	Access	Reset	Description
21	**ni**	R	-	No bus address increment flag 1 - A bus starting address written to **DMA_ch[n]_source** or **DMA_ch[n]_destination** will not be incremented during the DMA operation. Therefore, addresses on the AHB bus will only increment up to the amount shown in the bus transaction block size field of this **DMA_ch[n]_control** and then return to the value written to **DMA_ch[n]_source** or **DMA_ch[n]_destination** to begin incrementing again when this bit is 1. 0 - The AHB bus address is incremented continually throughout the DMA operation. (This no-increment mode affects only peripheral addressing, but not main memory addressing, mainly used for peripherals that have a FIFO-type DMA interface such as the I2S unit.)
20:19	**ts**	R	-	Transfer size See **DMA_ch[n]_control** description
18:16	**blk**	R	-	Bus transaction block size See **DMA_ch[n]_control** description
15:3	-	-	-	Reserved
2	**id**	R	-	Interrupt on done 1 indicates that when this descriptor's DMA operation ends without an error, the DMA channel will signal an interrupt. No interrupt will be signaled when this bit is 0.
1	**ie**	R	-	Interrupt on error When the DMA channel continues running upon an error, 1 indicates that the DMA channel will signal an interrupt when an error occurs. No interrupt will be signaled as long as the DMA channel continues to run when this bit is 0. An interrupt is always signaled when the DMA channel stops. Therefore, when the DMA channel stops on errors in descriptor mode, this bit is ignored.
0	**st**	R	-	Stop on error 1 indicates that the DMA channel will be disabled when an error occurs. The DMA channel will continue running when an error occurs in descriptor mode if this bit is 0.

Table 4-6. DMA Descriptor Control Information.

For Mass Production Only (1.5 / 08-Dec-15)

4.8 DMA: Programming DMA Registers

The DMA Engine Subsystem contains registers that hold DMA addresses, control information and results. Each DMA channel is associated with one register that contains:

- Channel control information (**DMA_ch[n]_control**)
- Source address (**DMA_ch[n]_source_addr**)
- Destination address (**DMA_ch[n]_dest_addr**)
- Descriptor address (**DMA_ch[n]_desc_addr**)
- Channel results including interrupts and transfer count (**DMA_ch[n]_status**)

The registers are used by each DMA channel to control its DMA transactions and to report completion status for these transactions.

Additionally the DMA subsystem provides a single global register (**DMA_intr_sts**) that identifies DMA channels with outstanding interrupts.

The registers are accessed with single AHB bus transactions that have a data size of 4 bytes. During an AHB bus transaction, AMBA Address[9:0] is the register index that indicates the register being accessed

4.9 DMA: Registers

4.9.1 DMA Registers: Map

Offset	Register	Description
0x200 - 0x2FC	-	Reserved
0x300	**DMA_ch0_control**	Channel 0 control
0x304	**DMA_ch0_source_addr**	Channel 0 main source address
0x308	**DMA_ch0_dest_addr**	Channel 0 main destination address
0x30C	**DMA_ch0_status**	Channel 0 main status
0x310	**DMA_ch1_control**	Channel 1 control
0x314	**DMA_ch1_source_addr**	Channel 1 main source address
0x318	**DMA_ch1_dest_addr**	Channel 1 main destination address
0x31C	**DMA_ch1_status**	Channel 1 main status
0x320	**DMA_ch2_control**	Channel 2 control
0x324	**DMA_ch2_source_addr**	Channel 2 main source address
0x328	**DMA_ch2_dest_addr**	Channel 2 main destination address
0x32C	**DMA_ch2_status**	Channel 2 main status
0x330	**DMA_ch3_control**	Channel 3 enable and control
0x334	**DMA_ch3_source_addr**	Channel 3 main source address
0x338	**DMA_ch3_dest_addr**	Channel 3 main destination address
0x33C	**DMA_ch3_status**	Channel 3 main status
0x340	**DMA_ch4_control**	Channel 4 control
0x344	**DMA_ch4_source_addr**	Channel 4 main source address
0x348	**DMA_ch4_dest_addr**	Channel 4 main destination address
0x34C	**DMA_ch4_status**	Channel 4 main status
0x350	**DMA_ch5_control**	Channel 5 control

Confidential For Hekra Use Only 1.5 (08012013)

Offset	Register	Description
0x354	**DMA_ch5_source_addr**	Channel 5 main source address
0x358	**DMA_ch5_dest_addr**	Channel 5 main destination address
0x35C	**DMA_ch5_status**	Channel 5 main status
0x360	**DMA_ch6_control**	Channel 6 control
0x364	**DMA_ch6_source_addr**	Channel 6 main source address
0x368	**DMA_ch6_dest_addr**	Channel 6 main destination address
0x36C	**DMA_ch6_status**	Channel 6 main status
0x370	**DMA_ch7_control**	Channel 7 control
0x374	**DMA_ch7_source_addr**	Channel 7 main source address
0x378	**DMA_ch7_dest_addr**	Channel 7 main destination address
0x37C	**DMA_ch7_status**	Channel 7 main status
0x380	**DMA_ch0_desc_addr**	Channel 0 main descriptor address
0x384	**DMA_ch1_desc_addr**	Channel 1 main descriptor address
0x388	**DMA_ch2_desc_addr**	Channel 2 main descriptor address
0x38C	**DMA_ch3_desc_addr**	Channel 3 main descriptor address
0x390	**DMA_ch4_desc_addr**	Channel 4 main descriptor address
0x394	**DMA_ch5_desc_addr**	Channel 5 main descriptor address
0x398	**DMA_ch6_desc_addr**	Channel 6 main descriptor address
0x39C	**DMA_ch7_desc_addr**	Channel 7 main descriptor address
0x3A0 - 0x3E8	-	Reserved
0x3EC	**DMA_fault_inject**	Inject fault for test
0x3F0	**DMA_intr_sts**	Global interrupt register
0x3F4	**DMA_pause_set**	Set pause bit of dma channel
0x3F8	**DMA_pause_clr**	Clear pause bit of dma channel
0x3FC	**DMA_early_end_req**	Send early end request to dma channel

Table 4-7. DMA Engine Subsystem Registers.

Confidential
Ambarella Liang only
(Expire 08012025)

4.9.2 DMA Registers: Details

4.9.2.1 DMA_ch[n]_control Register

Each DMA Channel Control register (**DMA_ch[n]_control**) holds information that controls DMA operations for that channel. Software writes to this register are used to configure the channel and then to enable the channel to initiate the transaction.

Bits	Name	Access	Reset	Description
31	**en**	RW	0	DMA channel enable 0 - DMA channel is idle 1 - DMA operation is in progress Software sets this bit to 1 to start the DMA operation. When the DMA operation is finished due to the successful transfer of the number of bytes shown in the count field or to an error, this bit is automatically reset to 0. This bit can only be written when all interrupts for this channel have been cleared by writing the interrupt flags in **DMA_ch[n]_status** to 0.
30	**d**	RW	0	Descriptor mode 0 - DMA channel is configured by software writes to the DMA Engine Subsystem 1 - DMA channel is configured from a descriptor
29	**wm**	RW	1	Write memory 0 - DMA writes are to the AHB bus 1 - DMA writes are to memory
28	**rm**	RW	1	Read memory 0 - DMA reads are from the AHB bus 1 - DMA reads are from memory
27	**ni**	RW	1	No bus address increment flag 0 - The AHB bus address is incremented continually throughout the DMA operation 1 - Bus starting address written to **DMA_ch[n]_source_addr** or **DMA_ch[n]_dest_addr** will not be incremented during the DMA operation. Therefore, addresses on the AHB bus will only increment up to the amount shown in the bus transaction block size field of this **DMA_ch[n]_control**, and then return to the value written to the **DMA_ch[n]_source_addr** or **DMA_ch[n]_dest_addr** to begin incrementing again when this bit is 1. (This no-increment mode affects only peripheral AMBA addressing, but not main memory addressing, mainly used as a workaround for peripherals that have a FIFO-type DMA interface such as the I2S unit.)
26:24	**blk**	RW	0	Bus transaction block size Each request to the AHB bus master in the DMA Engine Subsystem is a request to transfer the number of bytes indicated by this bus transaction block size field: The AHB bus master uses this information and chooses the optimal AHB bus burst type. Bus characteristics and usage should be considered to set this block size for efficient bus usage without starving other bus masters. This field can only be written when the DMA channel is idle and the Channel Control register-enable bit is 0.
23:22	**ts**	RW	0	Transfer size Indicates the data size for each data transfer on the AHB bus
21:0	**count**	RW	0	DMA transaction byte count Number of bytes moved by the DMA operation if the DMA operation ends without an error. When an error occurs, the DMA operation ends regardless of the number of bytes that have been transferred before the error.

Table 4-8. DMA Channel[n] Enable and Control Register.

blk	Bus Request Size	Comments
0	8 bytes	
1	16 bytes	
2	32 bytes	For best performance, use Bus Request Size of 32 bytes or less
3	64 bytes	
4	128 bytes	
5	256 bytes	
6	512 bytes	
7	1024 bytes	

*Table 4-9. Definition of **blk** in **DMA_ch[n]_control**.*

ts[1:0]	Bus Transfer Size
0	Byte
1	Half word—2 bytes
2	Word—4 bytes
3	Double word—8 bytes

*Table 4-10. Definition of ts in **DMA_ch[n]_control**.*

Ambarella Confidential Only
For Hekra Testing Only
1.5 (Oct 2023)

4.9.2.2 DMA_ch[n]_source_addr Register

Each DMA Channel Main Area Source Address register (**DMA_ch[n]_source_addr**) holds the address of the next read request of the DMA operation. Software writes to this register are used to set up the starting source address for the channel, prior to enabling the channel. This register is automatically updated during the DMA operation after each read request, to show the location of the next read request. This register can only be written when the DMA channel is idle and the Channel Control register-enable bit is 0.

Bits	Name	Access	Reset	Description
31:0	**source_addr**	RW	NR	Byte address of the next read request

Table 4-11. DMA Channel[n] Main Source Address.

4.9.2.3 DMA_ch[n]_dest_addr Register

Each DMA Channel Main Area Destination Address register (**DMA_ch[n]_dest_addr**) holds the address of the next write request of the DMA operation. Software writes to this register are used to set up the starting destination address for the channel, prior to enabling the channel. This register is automatically updated during the DMA operation after each write request, to show the location of the next write request. This register can only be written when the DMA channel is idle and the Channel Control register-enable bit is 0.

Bits	Name	Access	Reset	Description
31:0	**dest_addr**	RW	NR	Byte address of the next write request

Table 4-12. DMA Channel[n] Main Destination Address.

4.9.2.4 DMA_ch[n]_status Register

Each DMA Channel Main Area Status register (**DMA_ch[n]_status**) captures completion status when a DMA operation concludes. This includes the interrupt flags that show the reason for the termination of the operation, and the number of bytes transferred by the operation. Software should write this register to 0 before starting each DMA operation. Except for the descriptor operation-error bit and the descriptor operation-done bit, this register can only be written when the channel is idle and the Channel Control register-enable bit is 0. The descriptor operation-error bit and the descriptor operation-done bit can be written at any time. DMA engine logic prevents writes to bits other than the descriptor operation-error bit and the descriptor operation-done bit, when it is not safe to write these other bits.

Bits	Name	Access	Reset	Description
31	**dm**	RW	0	Descriptor memory error This bit is set to 1 when an error occurs on a descriptor memory read
30	**oe**	RW	0	Descriptor DMA operation error This bit is set to 1 in descriptor mode when a DMA operation ends due to an error (**DMA_ch[n]_status[26:24]** != 0) or due to an early end request (**DMA_ch[n]_status[26:24]** = 0).
29	-	-	-	Reserved
28	**dd**	RW	0	Descriptor chain done This bit is set to 1 when the DMA operation of the last descriptor of a descriptor chain ends
27	**od**	RW	0	Descriptor DMA operation done This bit is set to 1 in descriptor mode when a DMA operation ends without an error
26	**me**	RW	0	DMA memory error This bit is set to 1 when an error occurs on a DMA data memory read
25	**be**	RW	0	DMA bus error This bit is set to 1 when an error occurs on an AHB bus read
24	**rwe**	RW	0	DMA bus read / write error This bit is set to 1 when a DMA operation between two AHB bus devices is attempted by setting both the Channel Control register (**DMA_ch[n]_control**) bits **rm** and **wm** to 0
23	-	-	-	Reserved
22	**dn**	RW	0	DMA operation done This bit is set to 1 when a DMA operation ends with the number of bytes shown in **DMA_ch[n]_control** count field having been transferred. If a DMA operation ends due to an error before all data has been transferred, this bit is unchanged (0) if the register is cleared before DMA operation starts
21:0	**count**	RW	0	DMA transfer byte count Number of bytes moved by the DMA operation. When an error occurs, the DMA operation ends regardless of the number of bytes that have been transferred before the error. This field shows the number of good bytes moved when data is read from memory and when data is read from the AHB bus without an error. When data is read from the AHB bus and an error ends the DMA operation, this field can show a transfer count that is up to eight bytes more than the number of bytes transferred before the error. This field is automatically reset to 0 when a DMA operation starts.

Table 4-13. DMA Channel[n] Main Status Register.

For Confidential Use Only

4.9.2.5 DMA_ch[n]_desc_addr Register

Each DMA Channel Main Area Descriptor Address (**DMA_ch[n]_desc_addr** register) holds the address of the next descriptor that will be used to configure the channel if it is in descriptor mode as indicated by the Channel Control register descriptor mode bit. This register can only be written when the channel is idle and the Channel Control register-enable bit is 0.

Bits	Name	Access	Reset	Description
31:0	**desc_addr**	RW	0	Byte address of the next descriptor that will be used to configure **DMA_ch[n]_control** if it is in descriptor mode.

Table 4-14. DMA Channel[n] Main Descriptor Address Register.

4.9.2.6 DMA_fault_inject Register

The DMA Fault Inject register (**DMA_fault_inject**) asserts dma_sm_error for latent fault detection of ECRU.

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**fault_inject**	RW	0	Assert dma_sm_error to ECRU

Table 4-15. DMA Fault Inject Register.

4.9.2.7 DMA_int_sts Register

The DMA interrupt register (**DMA_interrupt**) shows the channels that have outstanding interrupts.

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7	**i7**	R	0	DMA channel 7 interrupt 1 indicates that DMA channel 7 has an outstanding interrupt. The Channel 7 Status register (**DMA_ch[n]_status**) should be read for the details of this channel 7 interrupt. This bit is cleared by writing the Channel 7 Status register to 0.
6	**i6**	R	0	DMA channel 6 interrupt 1 indicates that DMA channel 6 has an outstanding interrupt. The Channel 6 Status register (**DMA_ch[n]_status**) should be read for the details of this channel 6 interrupt. This bit is cleared by writing the Channel 6 Status register to 0.
5	**i5**	R	0	DMA channel 5 interrupt 1 indicates that DMA channel 5 has an outstanding interrupt. The Channel 5 Status register (**DMA_ch[n]_status**) should be read for the details of this channel 5 interrupt. This bit is cleared by writing the Channel 5 Status register to 0.

Confidential
For Meeka Lang only
(08012025)

Bits	Name	Access	Reset	Description
4	**i4**	R	0	DMA channel 4 interrupt 1 indicates that DMA channel 4 has an outstanding interrupt. The Channel 4 Status register (**DMA_ch[n]_status**) should be read for the details of this channel 4 interrupt. This bit is cleared by writing the Channel 4 Status register to 0.
3	**i3**	R	0	DMA channel 3 interrupt 1 indicates that DMA channel 3 has an outstanding interrupt. The Channel 3 Status register (**DMA_ch[n]_status**) should be read for the details of this channel 3 interrupt. This bit is cleared by writing the Channel 3 Status register to 0.
2	**i2**	R	0	DMA channel 2 interrupt 1 indicates that DMA channel 2 has an outstanding interrupt. **DMA_ch2_status** should be read for the details of this channel 2 interrupt. This bit is cleared by writing **DMA_ch2_status** to 0.
1	**i1**	R	0	DMA channel 1 interrupt 1 indicates that DMA channel 1 has an outstanding interrupt. **DMA_ch1_status** should be read for the details of this channel 1 interrupt. This bit is cleared by writing **DMA_ch1_status** to 0.
0	**i0**	R	0	DMA channel 0 interrupt

Table 4-16. DMA Interrupt Register.

4.9.2.8 DMA_pause_set Register

The DMA pause register (**DMA_pause**) shows the channels whose operations have been halted.

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7	**pause_7**	RW	0	DMA channel 7 pause 1 indicates that DMA channel 7 is paused. Write 1 to set this bit, and write 1 to **DMA_pause_clr[7]** (Section 4.9.2.9) to clear this bit.
6	**pause_6**	RW	0	DMA channel 6 pause; refer to **pause_7** above for details
5	**pause_5**	RW	0	DMA channel 5 pause; refer to **pause_7** above for details
4	**pause_4**	RW	0	DMA channel 4 pause; refer to **pause_7** above for details
3	**pause_3**	RW	0	DMA channel 3 pause; refer to **pause_7** above for details
2	**pause_2**	RW	0	DMA channel 2 pause; refer to **pause_7** above for details
1	**pause_1**	RW	0	DMA channel 1 pause; refer to **pause_7** above for details
0	**pause_0**	RW	0	DMA channel 0 pause; refer to **pause_7** above for details

Table 4-17. DMA Pause Set Register.

4.9.2.9 DMA_pause_clr Register

The DMA Pause Clear register (**DMA_pause_clr**) is used to clear a specified pause bit.

For Mekra Lang (080) Confidential 1.5

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7	**pause_clr_7**	W	0	DMA channel 7 pause clear Write 1 to clear **DMA_pause[7]** (Section 4.9.2.8).
6	**pause_clr_6**	W	0	DMA channel 6 pause clear; refer to **pause_clr_7** above for details
5	**pause_clr_5**	W	0	DMA channel 5 pause clear; refer to **pause_clr_7** above for details
4	**pause_clr_4**	W	0	DMA channel 4 pause clear; refer to **pause_clr_7** above for details
3	**pause_clr_3**	W	0	DMA channel 3 pause clear; refer to **pause_clr_7** above for details
2	**pause_clr_2**	W	0	DMA channel 2 pause clear; refer to **pause_clr_7** above for details
1	**pause_clr_1**	W	0	DMA channel 1 pause clear; refer to **pause_clr_7** above for details
0	**pause_clr_0**	W	0	DMA channel 0 pause clear; refer to **pause_clr_7** above for details

Table 4-18. DMA Pause Clear Register.

4.9.2.10 DMA_early_end_req Register

The DMA Early End Request register (**DMA_early_end_req**) is used to send a request to terminate the operations of a working channel early.

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7	**early_end_7**	RW	0	DMA channel 7 early end request Write 1 to send an early termination request to DMA channel 7.
6	**early_end_6**	RW	0	DMA channel 6 early end request; refer to **early_end_7** above for details
5	**early_end_5**	RW	0	DMA channel 5 early end request; refer to **early_end_7** above for details
4	**early_end_4**	RW	0	DMA channel 4 early end request; refer to **early_end_7** above for details
3	**early_end_3**	RW	0	DMA channel 3 early end request; refer to **early_end_7** above for details
2	**early_end_2**	RW	0	DMA channel 2 early end request; refer to **early_end_7** above for details
1	**early_end_1**	RW	0	DMA channel 1 early end request; refer to **early_end_7** above for details
0	**early_end_0**	RW	0	DMA channel 0 early end request; refer to **early_end_7** above for details

Table 4-19. DMA Early End Request Register.

4.10 DMA: Errors

Each DMA channel detects five error conditions:

1. Descriptor memory error
2. Descriptor address error
3. DMA memory error
4. DMA bus error

5. DMA bus read/write error

When an error is detected by a DMA channel, the operation ends, the channel is disabled, and an interrupt is signaled, if any of the following conditions are true:

- The DMA channel is in non-descriptor mode (the **DMA_ch[n]_control** descriptor mode bit is 0).
- The DMA channel is in descriptor mode (the **DMA_ch[n]_control** descriptor mode bit is 1) and it is configured to stop upon an error (the **DMA_ch[n]_control** stop-on-error bit is 1).
- The DMA channel is in descriptor mode (the **DMA_ch[n]_control** descriptor mode bit is 1) and a descriptor memory error or descriptor address error occurs.

When a DMA channel detects an error, the operation ends, the channel advances to the next operation and continues to run, and optionally, may signal an interrupt under the following circumstance:

- The DMA channel is in descriptor mode (the **DMA_ch[n]_control** descriptor mode bit is 1), it is configured to continue upon an error (the **DMA_ch[n]_control** stop-on-error bit is 0), and a DMA memory error, bus error, or bus read/write error occurs.

Refer to Section 4.11 for a detailed description of the DMA Engine Subsystem interrupt mechanism.

4.10.1 DMA Error: Descriptor Address

A descriptor address error occurs when the address in the Channel Descriptor Address register (**DMA_ch[n]_descriptor**) is not double word aligned. A descriptor address error has the following effects:

- No data is transferred for the descriptor with the address error.
- The Channel Status register (**DMA_ch[n]_status**) address error bit is 1.
- The descriptor processing ends in the channel with the descriptor address error.

4.10.2 DMA Error: Bus

A bus error occurs on an AHB bus-read when the channel's AHB bus master receives an AHB bus ERROR response from the AHB bus slave that it was attempting to read. In the DMA engine subsystem, AHB bus-writes fail silently. The cause of a bus error depends on the AHB bus device that reports the error.

Note that errors reported by devices may or may not indicate bad data. Therefore, the data associated with a bus error is written to its destination regardless. It is the responsibility of the device and host processor to communicate the data status through other device-specific registers or previously agreed-upon specifications. A bus error results in the following:

- The DMA engine subsystem writes the data associated with the error and all earlier data to its destination.
- The Channel Status register (**DMA_ch[n]_status**) DMA bus error bit is set to 1.
- The operation ends in the channel with the bus error.

Confidential
For Mekra Lang only
1.5 (08/12/2025)

4.10.3 DMA Error: Bus Read/Write

A bus read / write error occurs when both the source and destination are AHB bus devices. This combination is not permitted. Specifically, this occurs when both the Channel Control register (**DMA_ch[n]_control**) read memory and write memory bits are set to 0. A bus read / write error results in the following:

- No data is transferred.
- The Channel Status register (**DMA_ch[n]_status**) DMA bus read/write error bit is set to 1.
- The operation ends in the channel with the bus read / write error.

4.11 DMA: Interrupts

Each DMA channel independently signals an interrupt when one of the following eight conditions occur for the channel. In addition, the channel must be either disabled or in descriptor mode and configured to send an interrupt upon an error or completed DMA operation:

1. Descriptor memory error
2. Descriptor address error
3. Completed descriptor processing (EOC encountered)
4. DMA memory error
5. DMA bus error
6. DMA bus read/write error
7. DMA address error
8. Completed DMA transfer

The DMA engine subsystem interrupt is signaled by assertion (1) of the DMA interrupt signal DMA_INT, which is output to the VIC. Channels with active interrupts have their corresponding bits set to 1 in the DMA Interrupt register (**DMA_interrupt**). Each channel's active interrupts are shown in the Channel Status register (**DMA_ch[n]_status**). Refer to Section 4.8 above for more information.

An interrupt remains active until the software has cleared the interrupt by clearing the corresponding bit (write to 0) in one of the Channel Status registers (**DMA_ch[n]_status**). The global DMA engine subsystem interrupt, signaled at the DMA_INT output port, is deasserted when all interrupts are cleared.

Confidential
For Makra Labs only
1.5 (08/15/2017)

5. GRAPHICS DMA (GDMA)

This chapter overviews the graphics DMA (GDMA) co-processor including:

- (Section 5.1) GDMA: Overview
- (Section 5.2) GDMA: Registers
- (Section 5.3) GDMA: Support for Pending Operations
- (Section 5.4) GDMA: Assumptions
- (Section 5.5) GDMA: Use
- (Section 5.6) GDMA: Operations
- (Section 5.7) GDMA: Pixel Formats

5.1 GDMA: Overview

The graphics DMA co-processor (GDMA) performs six image operations on one or two source area(s) in DRAM and writes the resulting image to a destination area in DRAM. The source and destination areas are each defined by a base address, row pitch (in bytes), width (in pixels), height (in rows), and pixel format (bits per pixel). The six operations (Section 5.6) are as follows:

1. Linear copy
2. Two-dimensional copy
3. Composite
4. Alpha Blend (where source 2 is pre-multiplied by the alpha value)
5. Alpha Blend (where source 2 is not pre-multiplied by the alpha value)
6. Run-length decode

The source 1, source 2, and destination areas may have different pixel formats, provided the destination format is equal to or larger than the source format(s). The supported pixel formats (Section 5.7) from smallest to largest are as follows:

- 4-bit (lookup)
- 8-bit (lookup)
- 16-bit (1555)
- 16-bit (565)
- 24-bit
- 32-bit

The source 1 area may overlap arbitrarily (in DRAM) with the destination area. The source 2 area must not overlap with the destination area unless the source 2 area is identical to the destination area.

There are no byte-alignment restrictions for programming the GDMA registers that define the source and destina-

tion areas. Any integer value may be used to program the registers.

Up to eight outstanding operations can be issued to the GDMA by programming the appropriate GDMA registers via stores to the corresponding AHB addresses as described in Section 5.3. When the GDMA completes all pending operations, it sends a completion interrupt to the VIC (see Chapter 16). The user also can check the number of operations that have not been completed by loading the AHB address and reading the register **GDMA numpendingops** (Section 5.2.2.11).

5.2 GDMA: Registers

5.2.1 GDMA Registers: Map

All GDMA registers below are four-byte aligned.

Address Offset	Register Name	Description
0x000	**GDMA_source1base**	Base address of Source 1
0x004	**GDMA_source1pitch**	Row pitch of Source 1
0x008	**GDMA_source2base**	Base address of Source 2
0x00C	**GDMA_source2pitch**	Row pitch of Source 2
0x010	**GDMA_destinationbase**	Base address of destination
0x014	**GDMA_destinationpitch**	Row pitch of destination
0x018	**GDMA_width**	Source / destination width
0x01C	**GDMA_height**	Source / destination height
0x020	**GDMA_transparent**	Transparent value
0x024	**GDMA_opcode**	Operation code
0x028	**GDMA_numpendingops**	Number of pending operations
0x02C	**GDMA_pixelformat**	Pixel format
0x030	**GDMA_alpha**	Alpha value for blend
0x034	**GDMA_fatalerror**	Fatal error source / type
0x038	**GDMA_fatalerrorGDMA_error-detected**	Force / clear errors detected
0x400	**GDMA_lookuptable**	Color lookup table

Table 5-1. Graphics DMA Register Map.

5.2.2 GDMA Registers: Details

5.2.2.1 GDMA_source1base Register

Bits	Name	Access	Reset	Description
31:0	**source1base**	RW	NR	Base address of source 1 area

Table 5-2. Graphics DMA Source 1 Base Register.

5.2.2.2 GDMA_source1pitch Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**source1pitch**	RW	NR	Row pitch of source 1 area (bytes)

Table 5-3. Graphics DMA Source 1 Pitch Register.

5.2.2.3 GDMA_source2base Register

Bits	Name	Access	Reset	Description
31:0	**source2base**	RW	NR	Base address of source 2 area

Table 5-4. Graphics DMA Source 2 Base Register.

5.2.2.4 GDMA_source2pitch Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**source2pitch**	RW	NR	Row pitch of source 2 area (bytes)

Table 5-5. Graphics DMA Source 2 Pitch Register.

5.2.2.5 GDMA_destinationbase Register

Bits	Name	Access	Reset	Description
31:0	**destinationbase**	RW	NR	Base address of destination area

Table 5-6. Graphics DMA Destination Base Register.

For Confidential Mekrot Lang only 1.5 (Oct 01 2025)

5.2.2.6 GDMA_destinationpitch Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**destinationpitch**	RW	NR	Row pitch of destination area (bytes)

Table 5-7. Graphics DMA Destination Pitch Register.

5.2.2.7 GDMA_width Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15	**InterruptAfterAll-PendingOpsCom-pleted**	RW	NR	InterruptAfterAllPendingOpsCompleted When InterruptAfterAllPendingOpsCompleted == 0, the gDMA sends a completion interrupt after each operation has com-pleted. When InterruptAfterAllPendingOpsCompleted ==1, the gDMA sends a completion interrupt after all pending operations have completed.
14:13				Reserved
12:0	**width**	RW	NR	Number of pixels minus 1 per row in source/destination area

Table 5-8. Graphics DMA Width Register.

5.2.2.8 GDMA_height Register

Bits	Name	Access	Reset	Description
31:12				Reserved
11:0	**height**	RW	NR	Number of rows minus 1 in source / destination area

Table 5-9. Graphics DMA Height Register.

5.2.2.9 GDMA_transparent Register

Bits	Name	Access	Reset	Description
31:0	**transparent**	RW	NR	Transparent value (for composite operation) The valid width of this register depends on the Source 2 pixel format. 8 bits - Source 2 at 8 bits 4 bits - Source 2 at 4 bits 15 bits - Source 2 at 16 bits (1555). MSB is ignored. 16 bits - Source 2 at 16 bits (565) 24 bits - Source 2 at 24 bits 32 bits - Source 2 at 32 bits

Table 5-10. Graphics DMA Transparent Value Register.

For Mekra Lang only (1.5 - 08012025)

5.2.2.10 GDMA_opcode Register

Bits	Name	Access	Reset	Description
31:3				Reserved
2:0	**op**	RW	0	Operation Code A write to this register initiates the corresponding operation. All other registers for this operation should be written before writing to this register. 0 - Linear copy 1 - Two-dimensional copy 2 - Composite 3 - Alpha blend (source 2 colors are pre-multiplied with **alpha**) 4 - Run length decode 5 - Alpha blend (source 2 colors are not pre-multiplied with **alpha**) 6 - No operation 7 - No operation

Table 5-11. Graphics DMA Operation Code Register.

5.2.2.11 GDMA_numpendingops Register

Bits	Name	Access	Reset	Description
31:4				Reserved
3:0	**numpendingops**	R	0	Number of operations remaining to be completed. Value of zero indicates GDMA is idle.

Table 5-12. Graphics DMA Number of Pending Operations Register.

5.2.2.12 GDMA_pixelformat Register

Bits	Name	Access	Reset	Description
31:12				Reserved
11	**throttledram**	RW	NR	ThrottleDRAM When **throttledram** is 1, the maximum number of outstanding DRAM requests issued by the GDMA is 1. Otherwise, the maximum number of outstanding DRAM requests issued is 4.
10:8	**source1format**	RW	NR	Source 1 Pixel Format 0 - 8 bit 1 - 4 bit 2 - 16 bit (1555) 3 - 16 bit (565) 4 - 24 bit 5 - 32 bit
7				Reserved

Confidential
For Meeting only
1 5 08 / 2025

Bits	Name	Access	Reset	Description
6:4	**source2format**	RW	NR	Source 2 Pixel Format 0 - 8 bit 1 - 4 bit 2 - 16 bit (1555) 3 - 16 bit (565) 4 - 24 bit 5 - 32 bit
3				Reserved
2:0	**destinationfor-mat**	RW	NR	Destination Pixel Format 0 - 8 bit 1 - 4 bit 2 - 16 bit (1555) 3 - 16 bit (565) 4 - 24 bit 5 - 32 bit

Table 5-13. Graphics DMA Pixel Format Register.

5.2.2.13 GDMA_alpha Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**alpha**	RW	NR	Alpha value for Alpha Blend operations

Table 5-14. Graphics DMA Alpha Value Register.

5.2.2.14 GDMA_fatalerror Register

Bits	Name	Access	Reset	Description
7:4	**fatalerrorsource**	R		Sub-block from which fatal error originates. 1 - input signal 2 - amba_slave 3 - gdma_regfile 4 - gdma_address 5 - gdma_copy 6 - gdma_compblend 7 - gdma_rldecode 8 - gdma_dram_read 9 - gdma_dram_write 10 - gdma_dramintf 11 - gdma_buffers 12 - gdma_read2cache This register is read-only

Confidential For Melvin Lang only 1.5 (Oct 2012)

Bits	Name	Access	Reset	Description
3:0	**fatalerrortype**	R		Type of Fatal Error 1 - Input signal 2 - Flip-flop 3 - Configuration SRAM 4 - Data SRAM 5 - Watchdog Timeout This register is read-only

Table 5-15. Graphics DMA Fatal Error Register.

5.2.2.15 GDMA_errordetected Register

Bits	Name	Access	Reset	Description
0	**forceerrordetected**	W		Writing a 1 to this register causes the GDMA to signal an error to the CEHU. This register is write-only.
1	**clearerrordetected**	W		Write a 1 to this register causes the GDMA to clear the error signal to the CEHU. This register is write-only.

Table 5-16. Graphics DMA Error Detected Register.

5.2.2.16 GDMA_lookuptable Register

Bits	Name	Access	Reset	Description
31:0	**lookup**	W	NR	8b / 4b Lookup Table Color lookup table (256 entries) indexed by source color values when the source format is 8 bit or 4 bit. The valid width of the lookup table output depends on the destination pixel format. 16 bits - Destination format is 16 bits (1555) 16 bits - Destination format is 16 bits (565) 24 bits - Destination format is 24 bits 32 bits - Destination format is 32 bits

Table 5-17. Graphics DMA Color Lookup Table Register.

5.3 GDMA: Support for Pending Operations

There are eight instances (0 through 7) of all registers except (**GDMA_numpendingops** and **GDMA_lookuptable).** This facilitates support of up to eight pending operations on the chip.

When writing to these registers (0 through 7), addresses should be exactly as specified in Section 5.2.1. The instance need not be specified in the address because the GDMA manages which instance of the register is written. See below for details on multiple operations.

When reading these registers, the instance (0 through 7) must be specified in bits [8:6] of the address. For example, the address for reading instance six of **GDMA_destinationpitch** is 0x194.

The instances sequence is as follows. The GDMA contains an internal three-bit wide instance pointer that is incremented by 1 each time **GDMA_opcode** is written (i.e. each time an operation is initiated). It is not incremented when other registers are written and is not reset to 0 when **GDMA_numpendingops** becomes 0.

The following sequence illustrates which register instances are updated.

EVENT	INSTANCE POINTER
Store(s) to register(s) other than **GDMA_opcode**	0
Store to **GDMA_opcode**	1
Store(s) to register(s) other than **GDMA_opcode**	1
Store to **GDMA_opcode**	2
Wait until TWO operations complete (i.e. **GDMA_numpendingops** becomes 0)	2
Store(s) to register(s) other than **GDMA_opcode**	2
Store to **GDMA_opcode**	3
Store(s) to register(s) other than **GDMA_opcode**	3
Store to **GDMA_opcode**	4
Store(s) to register(s) other than **GDMA_opcode**	4
Store to **GDMA_opcode**	5
Store(s) to register(s) other than **GDMA_opcode**	5
Store to **GDMA_opcode**	6
Store(s) to register(s) other than **GDMA_opcode**	6
Store to **GDMA_opcode**	7
Store(s) to register(s) other than **GDMA_opcode**	7
Store to **GDMA_opcode**	0
Store(s) to register(s) other than **GDMA_opcode**	0
Store to **GDMA_opcode**	1
Store(s) to register(s) other than **GDMA_opcode**	1
Store to **GDMA_opcode**	2
Wait until EIGHT operations complete (i.e. **GDMA_numpendingops** becomes 0)	2

The order of register instances updated for this group of eight operations IS: 2, 3, 4, 5, 6, 7, 0, 1 (*NOT 0, 1, 2, 3, 4, 5, 6, 7*).

For Mekra Lang only (08012025) Confidential 1.5

5.4 GDMA: Assumptions

The proper functioning of the GDMA is contingent on the following assumptions:

- Pitch (which is in units of bytes) must be greater than or equal to the number of bytes per row (i.e. width + 1 multiplied by the number of bytes per pixel)
  - This applies to the **Source1Pitch**, **Source2Pitch**, and **DestinationPitch**
- When two sources are used (the **GDMA_opcode** bits **op** = 2, 3 or 5), the source 2 area cannot overlap with the destination area unless the two areas exactly match.

- For the source 2 area to exactly match the destination area, **source2base** must equal **destinationbase**; **source2pitch** must equal **destinationpitch**; and **GDMA_pixelformat** bits, **source2format** and **destinationformat** must be equal.

- For run length decoding (**op** = 4), the source 1 area (i.e. the encoded stream) and the destination area cannot overlap.
- When any **GDMA_pixelformat** bits **source1format**, **source2format** or **destinationformat** is 4 bits, the number of pixels per row must be even (i.e. **GDMA_width** must be an odd number).
- When either **GDMA_pixelformat** bits **source1format** or **source2format** is 16 bits (1555 or 565) or 24 bits, and **destinationformat** is 32 bits, **GDMA_alpha** must be programmed.
  - Source data that is 16 bits (1555 or 565) or 24 bits have no intrinsic alpha value, whereas destination data of 32 bits contain an alpha value per pixel, and **GDMA_alpha** is used to convert from the source to destination format.
- For run length decoding (**op** = 4), **GDMA_pixelformat** must be programmed as 0x0 (for four outstanding DRAM requests) or 0x800 (for one outstanding DRAM request).

5.5 GDMA: Use

Ambarella recommends using the GDMA as follows:

1. Program all necessary GDMA registers for the operation by issuing stores to the corresponding AHB addresses.

   The following table indicates which registers to program for each operation (X = must be programmed).

Register	Linear Copy	2D Copy	Composite	Alpha Blend (op = 3 or op = 5)	Run Length Decode
GDMA_source1base	X	X	X	X	X
GDMA_source1pitch	-	X	X	X	-
GDMA_source2base	-	-	X	X	-
GDMA_source2pitch	-	-	X	X	-
GDMA_destinationbase	X	X	X	X	X
GDMA_destinationpitch	-	X	X	X	X
GDMA_width	X	X	X	X	X
GDMA_height	-	X	X	X	X
GDMA_transparent	-	-	X	-	-
GDMA_pixelformat	X	X	X	X	X
GDMA_alpha	X	X	X	X	-

Table 5-18. Recommended use of the Graphics DMA.

2. Initiate the operation by issuing a store to the **GDMA_opcode** AHB address.
3. Repeat Steps 1 and 2 (in order) up to seven more times (for up to eight total operations issued).
4. Optionally execute other unrelated Arm code.
5. Check the number of uncompleted operations by issuing a load to the **GDMA_numpendingops** AHB address or wait for the GDMA's completion interrupt.
6. Repeat Steps 1 through 5 to perform more operations as desired.

Confidential
For Meera Lang only
10/08/2025

5.6 GDMA: Operations

5.6.1 GDMA Operations: Linear Copy and 2D Copy

Linear copy (**op** = 0) and **two-dimension**al copy (**op** = 1) both transfer the pixels from the source 1 area to the destination area.

Linear copy operates on only one row. (i.e., **GDMA_source1pitch**, **GDMA_destinationpitch** and **GDMA_height** settings are irrelevant) whereas two-dimensional copy operates on one or more rows.

If **source1format** = **destinationformat**, the source 1 pixels from DRAM are written to one of six source buffers (buffer 0, 1, 2, 3, 4 or 5), read from the same buffer, and then sent to DRAM. Note that when the **source1format** is 16 bits (1555) and the **destinationformat** is 16 bits (565), the GDMA considers **source1format** = **destinationformat**.

If **source1format** != **destinationformat**, the source 1 pixels from DRAM are written to one of two source buffers (buffer 0 or 1), read from the source buffer, converted to the destination format and written to one of four destination buffers (buffer 2, 3, 4 or 5), and finally read from the destination buffer and sent to DRAM.

The supported pixel format combinations include the following:

Destination Format	Source 1 Format
4 bits	4 bits
8 bits	8 bits
16 bits (1555)	4 bits, 8 bits, 16 bits (1555)
16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)
24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits
32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits

Table 5-19. Graphics DMA Pixel Format Combinations for Linear Copy and 2D Copy.

For Confidential Planning only 1.5 (Oct 2025)

5.6.2 GDMA Operations: Composite

Composite operates on pixels from both source 1 and source 2 areas based on the following equation:

destination = (source2 == **transparent**) ? ConvertToDestFormat(source1.color) :
ConvertToDestFormat(source2.color) ;

where **transparent** is in **source2format**.

The source 1 pixels from DRAM are written to one of two source 1 buffers (buffer 0 or 1), while the source 2 pixels from DRAM are written to one of two source 2 buffers (buffer 2 or 3). The source 1 and 2 pixels are read from their respective source buffers, converted to the destination format and operated upon. The result is written to one of two destination buffers (buffer 4 or 5) and finally read from the destination buffer and sent to DRAM.
The supported pixel format combinations include the following:

Destination Format	Source 1 Format	Source 2 Format
4 bits	4 bits	4 bits
8 bits	8 bits	8 bits
16 bits (1555)	4 bits, 8 bits, 16 bits (1555)	4 bits, 8 bits, 16 bits (1555)
16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)

Destination Format	Source 1 Format	Source 2 Format
24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits
32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits

Table 5-20. Graphics DMA Pixel Format Combinations for Composite Operation.

5.6.3 GDMA Operations: Alpha Blend (Source 2 Pre-Multiplied by Alpha)

Alpha blend operates on pixels from both source 1 and source 2 areas based on the following equations. Source 1 pixels are assumed to be pre-multiplied by their alpha values. Source 2 pixels also are assumed to be pre-multiplied by their alpha values (**op** = 3).

```
adjustedSource2Alpha = (source2.alpha == 0) ? 0 : (source2.alpha + 1) ;
destination.color = clip( source2.color + round( source1.color * (256 – adjustedSource2Alpha) ) );
```

where source2.alpha is 8 bits wide, and the color is red, green, blue or alpha;
round(value) = (value + 128) >> 8; and
clip(value) = (value > 255) ? 255 : value.

When **source2format** is 32 bits, source2.alpha is the most significant byte of the pixel, otherwise source2.alpha is taken from **GDMA_alpha**. When **destinationformat** is 16 bits (1555 or 565), the destination.color is further reduced to 5 bits ((destination.color[7:0] + 4) >> 3) or 6 bits ((destination.color[7:0]+ 2) >> 2).

The source 1 pixels from DRAM are written into one of two source 1 buffers (buffer 0 or 1), while the source 2 pixels from DRAM are written into one of two source 2 buffers (buffer 2 or 3). The source 1 and source 2 pixels are read from their respective source buffers, converted to the destination format and operated upon. The result is written to one of two destination buffers (buffer 4 or 5), and finally read from the destination buffer and sent to DRAM.

The supported pixel format combinations include the following:

Destination Format	Source 1 Format	Source 2 Format
16 bits (1555)	4 bits, 8 bits, 16 bits (1555)	4 bits, 8 bits, 16 bits (1555)
16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)
24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits
32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits

Table 5-21. Graphics DMA Pixel Format Combinations for Alpha Blend (Source 2 is Pre-Multiplied by Alpha).

Confidential
For Meku Liang only
Nov 06 2025

5.6.4 GDMA Operations: Alpha Blend (Source 2 Not Pre-Multiplied by Alpha)

Alpha blend operates on pixels from both source 1 and source 2 areas based on the following equations. Source 1 pixels are assumed to be pre-multiplied by their alpha values (**op** = 5).

```
adjustedSource2Alpha = (source2.alpha == 0) ? 0 : (source2.alpha + 1) ;
destination.color = clip( round( source2.color * adjustedSource2Alpha +
                                 source1.color * (256 – adjustedSource2Alpha) ) );
```

where source2.alpha is 8 bits wide, and the color can be red, green, blue or alpha;
round(value) = (value + 128) >> 8; and
clip(value) = (value > 255) ? 255 : value.

When **source2format** is 32 bits, source2.alpha is the most significant byte of the pixel, otherwise source2.alpha is taken from **GDMA_alpha**. When **destinationformat** is 16 bits (1555 or 565), the destination.color is further reduced to 5 bits ((destination.color[7:0] + 4) >> 3) or 6 bits ((destination.color[7:0] + 2) >> 2).

The source 1 pixels from DRAM are written into one of two source 1 buffers (buffer 0 or 1), while the source 2 pixels from DRAM are written into one of two source 2 buffers (buffer 2 or 3). The source 1 and source 2 pixels are read from their respective source buffers, converted to the destination format and operated upon. The result is written into one of two destination buffers (buffer 4 or 5), and finally read from the destination buffer and sent to DRAM.

The supported pixel format combinations include the following:

Destination Format	Source 1 Format	Source 2 Format
16 bits (1555)	4 bits, 8 bits, 16 bits (1555)	4 bits, 8 bits, 16 bits (1555)
16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)	4 bits, 8 bits, 16 bits (1555), 16 bits (565)
24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits
32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits	4 bits, 8 bits, 16 bits (1555), 16 bits (565), 24 bits, 32 bits

Table 5-22. Graphics DMA Pixel Format Combinations for Alpha Blend (Source 2 Not Pre-Multiplied by Alpha).

5.6.5 GDMA Operations: Run Length Decode

Run length decode (**op** = 4) operates on pixels from source 1 (the encoded stream) and writes the decoded stream to the destination area. The pixel format always is 8 bits. The encoded stream is assumed to use big endian byte ordering, although it is stored in DRAM with little endian byte ordering.

The following tables demonstrate the concepts:

Byte 3 (addr N + 3)								Byte 2 (addr N + 2)								Byte 1 (addr N + 1)								Byte 0 (addr N)							
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Table 5-23. Four-Byte Data Fetched from DRAM (Little Endian).

Type	Byte 0								Byte 1								Byte 2								Byte 3							
									1	1	1	1	1	1			2	2	2	2	1	1	1	1	3	3	2	2	2	2	2	2
	7	6	5	4	3	2	1	0	5	4	3	2	1	0	9	8	3	2	1	0	9	8	7	6	1	0	9	8	7	6	5	4
Pixel Code	V	V	V	V	V	V	V	V																								
End of Line Signal	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
Run Length Zero (1 - 63)	0	0	0	0	0	0	0	0	0	0	N	N	N	N	N	N																
Run Length Zero (64 - 16 K)	0	0	0	0	0	0	0	0	0	1	N	N	N	N	N	N	N	N	N	N	N	N	N	N								
Run Length (3 - 63)	0	0	0	0	0	0	0	0	1	0	N	N	N	N	N	N	V	V	V	V	V	V	V	V								
Run Length (64 - 16 K)	0	0	0	0	0	0	0	0	1	1	N	N	N	N	N	N	N	N	N	N	N	N	N	N	V	V	V	V	V	V	V	V

Table 5-24. Decoding of Encoded Stream (Big Endian).

If the GDMA has decoded (**width** + 1) bytes before encountering an End of Line Signal, it will continue reading and decoding the encoded stream but discard the corresponding decoded data until it either encounters an End of Line Signal or has discarded approximately (**width** + 1)/2 bytes of decoded data, whichever comes first. This mechanism prevents the GDMA from hanging for an encoded stream that never contains an End of Line Signal.

If the GDMA encounters an End of Line Signal before it has decoded (**width** + 1) bytes, it will write 0xFF to the destination until writing a total of (**width** + 1) bytes to the destination for the current line.

5.7 GDMA: Pixel Formats

The following diagrams are layouts of the different pixel formats using source 1 as an example. Each box represents 1 byte of memory.

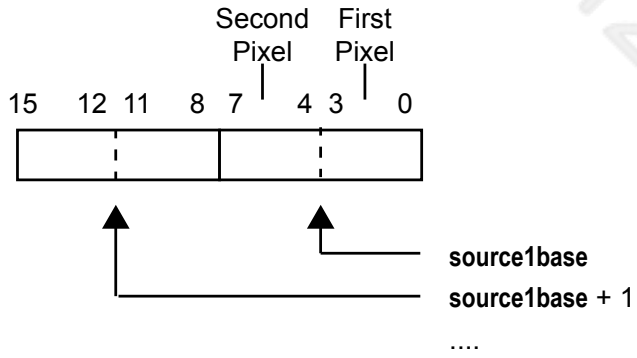


Figure 5-1. GDMA 4-bit Pixel Format.

Confidential For Ambarella Lang only (08012025)

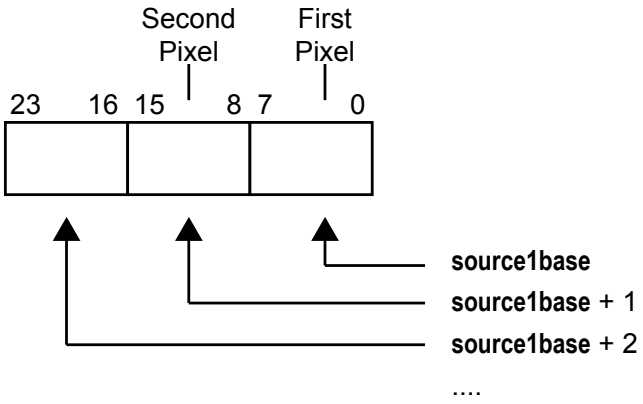


Figure 5-2. GDMA 8-bit Pixel Format.

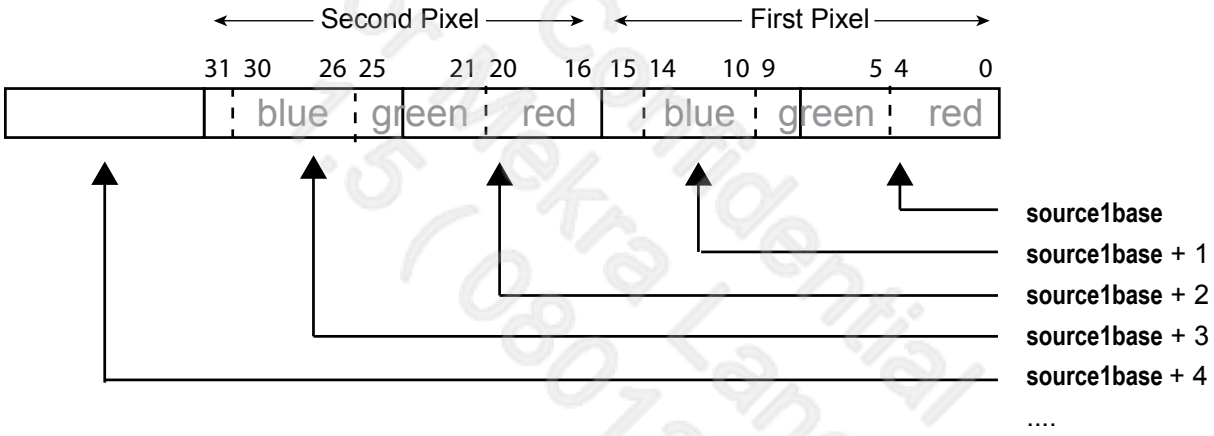


Figure 5-3. GDMA 16-bit Pixel Format (1555).

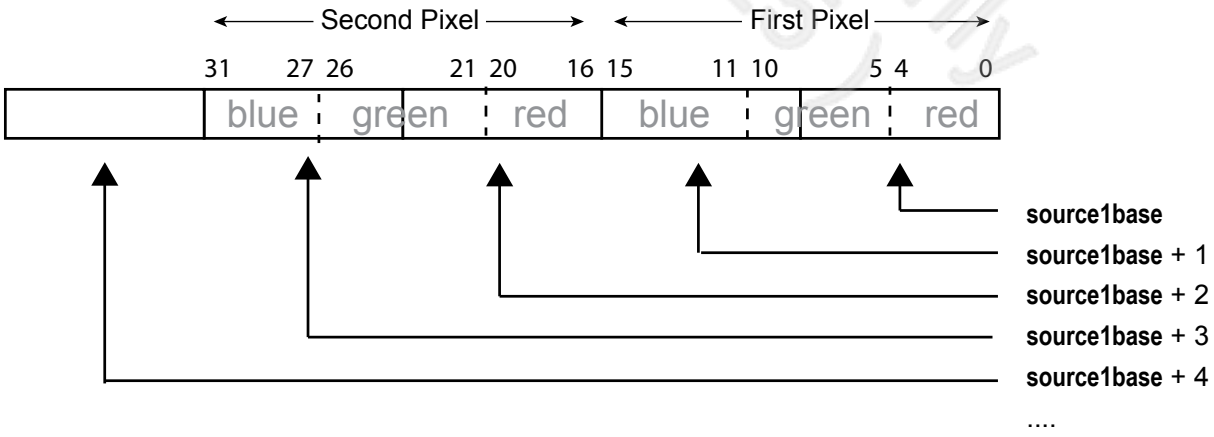


Figure 5-4. GDMA 16-bit Pixel Format (565).

For Ambarella Confidential Use Only (08012025)

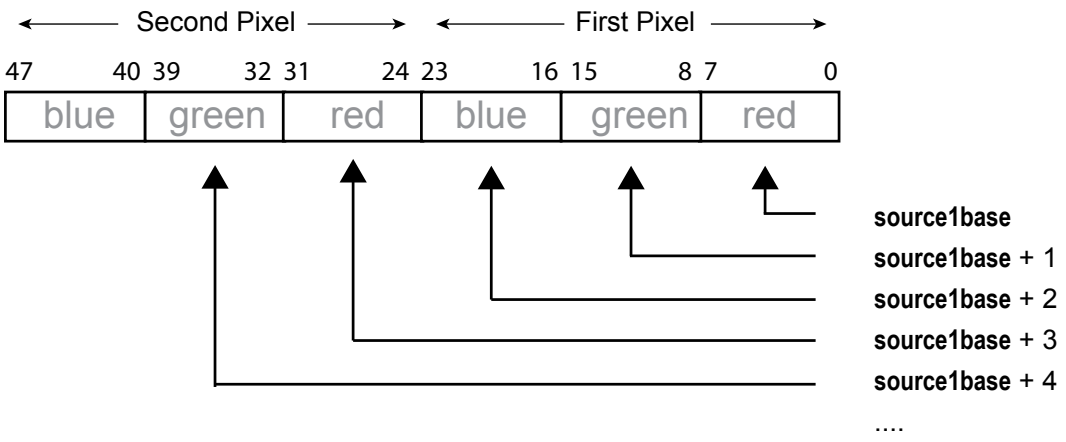


Figure 5-5. GDMA 24-bit Pixel Format.

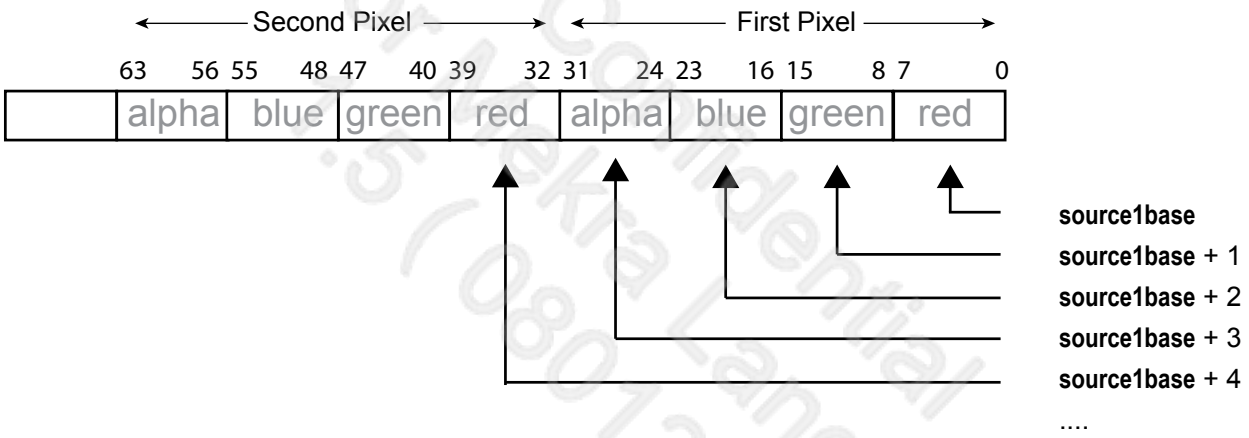


Figure 5-6. GDMA 32-bit Pixel Format

For Ambarella Confidential Use Only (0801 2025)

6. VIDEO / SENSOR INPUT INTERFACE

This chapter includes the following sections:

- (Section 6.1) VIN: Overview
- (Section 6.2) VIN: Programming Offsets
- (Section 6.3) VIN: Resets
- (Section 6.4) VIN: Interrupts
- (Section 6.5) VIN: Programming Details
- (Section 6.6) VIN: Master Sync Generation
- (Section 6.7) VIN: Registers

6.1 VIN: Overview

Features of the CV22FS VIN module include the following:

- Three VIN instances–one main VIN and two secondary VINs
- Operating modes:
  - Primary VIN (VIN0):
    - 1 / 2 / 3 / 4 / 8 lane MIPI CSI-2, up to four virtual channels (Note: When virtual channel is used, only 1 / 2 / 3 / 4 lane MIPI CSI-2 is supported)
  - Secondary VINs (VIN1 / VIN8):
    - 1–4 lane MIPI CSI-2
  - Virtual channel only VINs: VIN2 / VIN3 / VIN4

The following table provides a summary comparison between the CV22FS VIN design and previous Ambarella VIN designs.

Feature	CV1	CV2	CV22FS
Design Base	A9A	CV1	CV2
MIPI	Yes	Yes	Yes
SLVS Lanes	24	24	N/A
Parallel LVDS	Yes (12-bit)	Yes (12-bit)	N/A

Confidential For Mekra Lang only (Oct 2023)

Feature	CV1	CV2	CV22FS
Special features	- HDR line splitting - Deskew - 4x4 mode (4 sensors x 4 lanes) - Two 12-lane SLVS	- HDR line splitting - Deskew - 2 x 12 mode - 6 x 4 mode - MIPI virtual channel (limited) - 8 lane MIPI - MIPI lane remapping - 16 bit MIPI - Black level offset at output - Add mipi_fwd_ecc register - Add non_block register so that VIN can be test alone	-3x4 mode -All CV2 features -Full MIPI virtual channel support -MIPI embedded data packets support -ISO 26262 safety in MIPI mode
Config interface	Standard IDSP config I/F	Standard IDSP config I/F	Standard IDSP config I/F
Max width	4096	4096	4096

Table 6-1. Comparison Between VIN Designs.

The CV22FS VIN module is part of the DSP cluster. Similar to other modules in the DSP cluster, it is configured using a set of APIs. Contact the Ambarella support team for information regarding VIN module configuration, and additional lane sharing combinations.

- The video / sensor input module is implemented as part of the DSP subsystem on the CV22FS chip and is configured using a set of APIs. Chapter 3 of this manual provides an overview of the DSP subsystem.
- The VIN clock GCLK_SO_VIN is configured by CV22FS system software.
- The primary and secondary interface details are provided in the *"CV22FS Datasheet."*

6.2 VIN: Programming Offsets

There are three main configuration sections in all VINs:

1. Global / static config
   - Includes unchanging configuration bits, such as the sensor type, and is programmed via the following:
     - Special IDSP controller command
     - Debug bus
2. VIN main config
   - Includes the main VIN configuration, and is programmed via:
     - IDSP Controller section command
     - Debug bus (for debug only)
3. Master sync generator config
   - Includes configuration for the master sync generator, and is programmed via:
     - Special IDSP controller command
     - Debug bus

Confidential For Meta Lang only (ISO 12025)

6.2.1 Debug Bus Offsets and Controller Command Addresses

Region	Debug Bus Offset (within filter addr space)	Controller Command Address
VIN0 primary configuration	0	<section 1>*
VIN0 global / static	512 (0x200)	0x80
VIN0 master sync gen	256 (0x100)	0x88-0x8A
VIN1 primary configuration	0	<section 10>*
VIN1 global / static	512 (0x200)	0xC0
VIN1 master sync gen	256 (0x100)	0xC8-0xCA
VIN2 primary configuration	0	<section 11>*
VIN2 global / static	512 (0x200)	0x90
VIN3 primary configuration	0	<section 12>*
VIN3 global / static	512 (0x200)	0xA0
VIN4 primary configuration	0	<section 13>*
VIN4 global / static	512 (0x200)	0xB0
VIN8 primary configuration	0	<section 19>*
VIN8 global / static	512 (0x200)	0xD0

Table 6-2. Debug Bus Offsets and Controller Command Addresses.

* VIN global / master sync configuration address location. (VIN* primary configuration is the main VIN configuration—via the standard configuration bus.)

Confidential For Mekra Lang only 1.5 (08012025)

6.3 VIN: Resets

Ucode provides a reset cmd for the system to call. The reset cmd from the system triggers the DSP VIN thread to issue reset for the intended VIN along with the VIN's internal ucode state's reset.

Ucode also provides a time out mechanism. Ucode waits for the VIN completion interrupt for a period of time (typically determined by the system). If Ucode detects a timeout, it reports the timeout to the system. The system then resets the intended VIN.

The following includes a list of the resets and their descriptions:

- IDSP reset (GRST_IDSP)
  - Resets ALL VIN logic
- VIN / section reset
  - From the IDSP controller
  - Main reset
  - Resets all VIN logic and config, EXCEPT global / static config
  - Does not reset master sync logic
- VIN master sync reset
  - From the IDSP controller
  - Resets master sync generator unit
- VIN soft reset
  - Register in VIN config / programming
  - Resets the VIN pipeline only
  - Does not reset VIN config registers
  - Can be used to reset VIN while a new config is loaded

For Mekra Lang only Confidential (08012025)

6.4 VIN: Interrupts

The section includes a diagram and table describing interrupts.

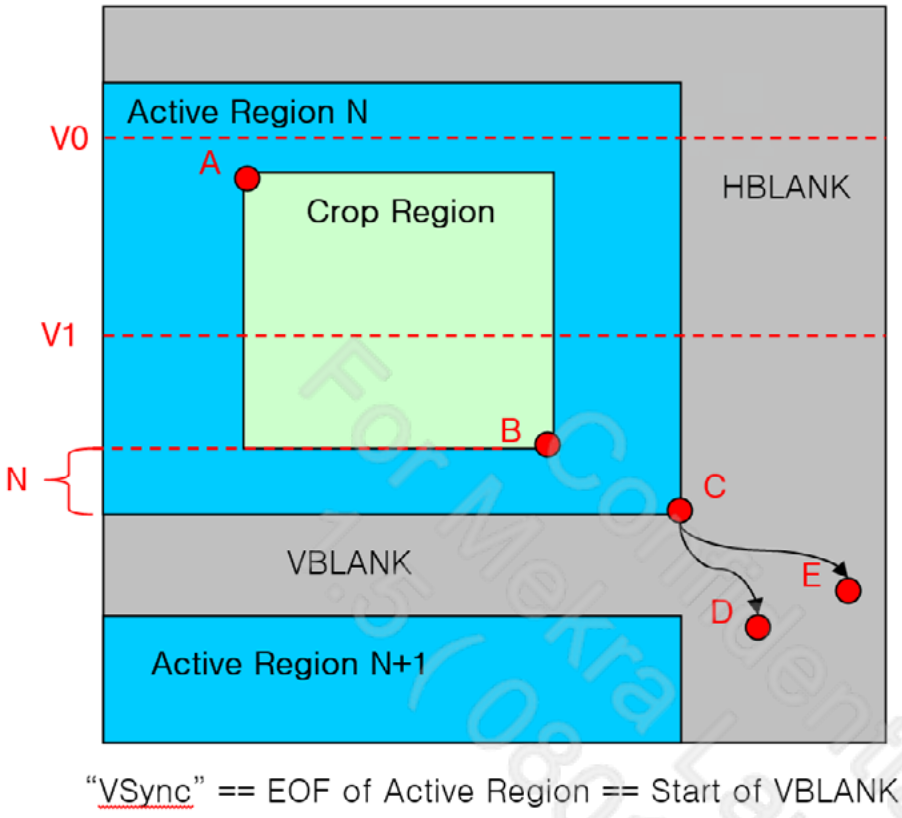


Figure 6-1. Interrupts Diagram.

Label	Description
A	Start of frame (start of crop region) interrupt to Arm
B	End of frame (end of crop region) interrupt to Arm
C	VSync sync to controller; VSync interrupt to Arm and ORC; triggers SCLK cycle counters to generate D and E; buffered VIN config is also loaded at this point
D	Delayed VSync sync to controller
E	Delayed VSync interrupt to Arm
N	"No-interrupt" signal to ORC Asserted between EOF of crop region and VSync
V0, V1	VOUT syncs Asserted for the duration of active line N, in odd, even, or both fields.

Table 6-3. Interrupts Table.

For Mekra Lang only (08012025) Confidential

6.5 VIN: Programming Details

The general VIN programming sequence is as follows:

1. Program the VIN global / static config region to select sensor type / source (MIPI) and the correct clock source for VIN.
2. In MIPI mode, set the appropriate MIPI_VC_PAT and MIPI_VC_MASK to capture intended virtual channels.
3. Ensure that the clock from the sensor is valid (PHY should be reset before VIN).
4. Reset VIN (via the IDSP controller) to prepare it for data.

   VIN is now ready for programming (via the IDSP controller section command).

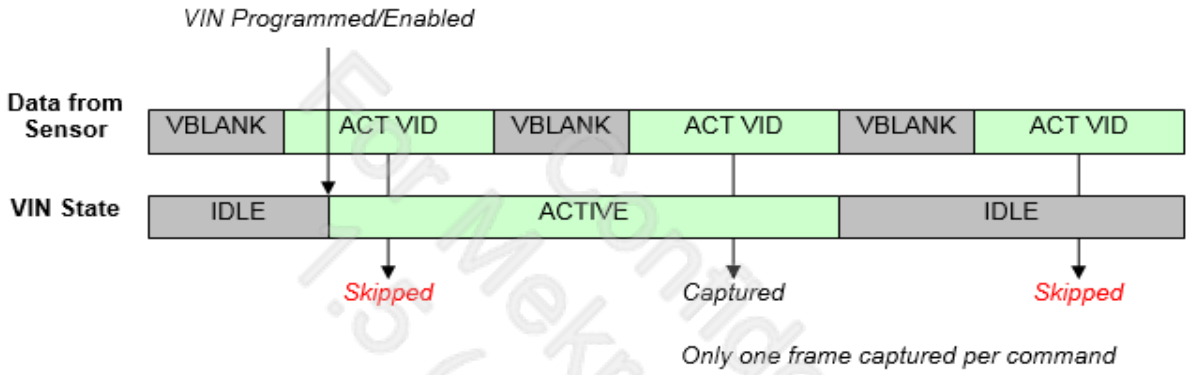


Figure 6-2. Programming Details.

Notes:

- After enabled, VIN waits for the first full frame.
- VIN captures one frame only per section command.

6.5.1 Safety-Related Programming

Each VIN section includes its own error collection and reporting (ECR) module which reports random hardware errors in VIN as well as sensor input errors detected in VIN. The VIN top-level reset and IDSP section reset can set a VIN section into a safe state. SW must then clear any VIN errors to bring the system out of the safe-state.

The following lists the status errors each VIN can detect:

- Watchdog_timeout_error
  - Occurs when one frame takes longer than the programmed watchdog threshold value cycles to complete
- Short frame
- Short line
- SFIFO overflow
- AFIFO overflow

Confidential For Mekra Lang only 1.5 (08012025)

- Err_fs
  - Occurs when there is an incorrect wrong frame sync packet sequence in MIPI stream
- CRC_ERR
  - Occurs when the detected crc error number is larger than CRC_ERR_THRESHOLD
- ECC correctable error
  - Occurs when the detected 1-bit ecc error number is larger than ECC_CORR_ERR_ THRESHOLD
- ECC_UNCORR_ERR

All errors are reported both through error valid to CEHU and through interrupts to core. This function is configurable, enabling users to select how error notification is performed.

For example, to disable SHORT_ERROR reported through interrupt, and keep it in error valid to CEHU, set WATCHDOG_TIMEOUT_INTR_MASK=0, and WATCHDOG_TIMEOUT_ECR_MASK = 1.

6.5.2 Sensor-Specific Programming

The following lists information related to sensor-specific programming:

- Set WORD_SIZE to correspond to sensor pixel / data width (for example, 10-bit, 12-bit)
- If programming MIPI sensors, set LANE_ENABLE accordingly
- For programming sync code registers (for non-MIPI sensors)
  - Use SYNC_INTERLEAVING if using HiSPi sensor with interleaved sync codes.
  - Specify SYNC_TYPE (0=ITU-656 compatible; 1=custom).
  - If using custom sync codes, enable / program the relevant sync code masks, enables, and patterns:

- PAT_MASK_ALIGNMENT specifies whether the sync codes are MSB aligned or LSB aligned, and should always be set to 1 (MSB aligned) except for Panasonic sensors, which use LSB-aligned codes.
- SYNC_DETECT_MASK / SYNC_DETECT_PAT: This pattern and mask is used to improve sync detection. It should be set such that for ALL sync codes:
  - SYNC_CODE & SYNC_DETECT MASK == SYNC_DETECT_PAT
- Because the MSB for most sync code formats is always 1, these registers would be set to the following:
  - SYNC_DETECT_MASK = 0x8000
  - SYNC_DETECT_PAT = 0x8000
- SYNC_COMPARE_MASK should be set to select the bits to compare in the sync codes. For example, if the 8 MSBs of the sync code are valid, set to 0xFF00.
- Depending on the sync code protocol, enable the various SOL / EOL codes with the <code>_EN registers.
- For the enabled sync codes, program the relevant pattern into the <code>_PAT registers as follows:
  - SYNC_CODE & SYNC_COMPARE_MASK == <code>_PAT
  - By default, disable sync code error tolerance as follows:
- SYNC_TOLERANCE=0
- SYNC_ALL_LANES=1

Confidential For Meeta Lang only Not Subject to Export Control

- ALLOW_PARTIAL_CODES=0
- LOCK_SYNC_PHASE=0
- SYNC_CORRECTION=0

Note: If the sensor interface is noisy and VIN is losing lock, these registers can be used to adjust performance.

  - Program the SYNC_TIMEOUT value to specify the maximum number of cycles between sync codes before the VIN sync state machine unlocks and resets:

- Two line-times should be sufficient.
- This value is specified in SENSOR_CLOCK x 2 cycles; therefore, if the desired timeout is 4000 sensor clock cycles, then set SYNC_TIMEOUT = 2000.
  - Set ACTIVE_WIDTH and ACTIVE_HEIGHT according to the frame format output by the sensor (not including hblank / vblank /sync codes).

• For MIPI sensors, program MIPI registers as follows:

  - To support four virtual channels in lane 0-3, program VIN0 / 2 / 3 / 4 VC_PAT to 0 / 1 / 2 / 3, and each VIN extracts only the programmed stream from the mixed data stream.
  - To extract embedded data from embedded packets in any VIN section, set MIPI_ED_ENABLE=1 and ED_DT_PAT to the expected data type pattern.
  - All VIN data type patterns include a mask function to provide an inexact match potential.

6.5.3 Examples of Sync Code Programming for Various Sensor Protocols

The following table lists examples of sync code programming for various sensor protocols.

Register	HiSPi Streaming-S	HiSPi Streaming-SP	HiSPi Packetized-SP	Panasonic (MN34070PLH)	Sony (ITU-656) (IMX118, IMX117, etc)
SYNC_INTERLEAVING	0, 1 or 2 **	0	0	0	0
SYNC_TYPE	1	1	1	1	0 or 1
PAT_MASK ALIGNMENT	1	1	1	0	1
SYNC_DETECT_MASK	0x8000	0x8000	0x8000	0xFC00	0x8000
SYNC_DETECT_PAT	0x8000	0x8000	0x8000	0x0000	0x8000
SYNC_COMPARE_MASK	0xFF00	0xF000	0xF000	0x0003	0xFF00
SOL_PAT*	0x8000	0x8000	0x8000	0x0	0x8000
EOL_PAT*	X	X	0xA000	0x1	0x9D00
SOF_PAT*	X	0xC000	0xC000	0x2	X

For Ambarella Confidential use only (1.5 / 08.12.2025)

Register	HiSPi Streaming-S	HiSPi Streaming-SP	HiSPi Packetized-SP	Panasonic (MN34070PLH)	Sony (ITU-656) (IMX118, IMX117, etc)
EOF_PAT*	X	0	0xE000	0x3	X
SOV_PAT*	0xAB00	0x9000	X	X	0xAB00
EOV_PAT*	X	X	X	X	0xB600

Table 6-4. Examples of Sync Code Programming for Various Sensor Protocols.

*: If <code>_PAT is not X, it implies that <code>_EN is set to 1, else set <code>_EN=0
**: Streaming-S sync code interleaving depends on the sensor configuration (see HiSPi / sensor spec)

6.5.4 Example Programming

The figures in this section show example of lane mapping programming for common sensors.

The following provides a description of the lane mapping programming.

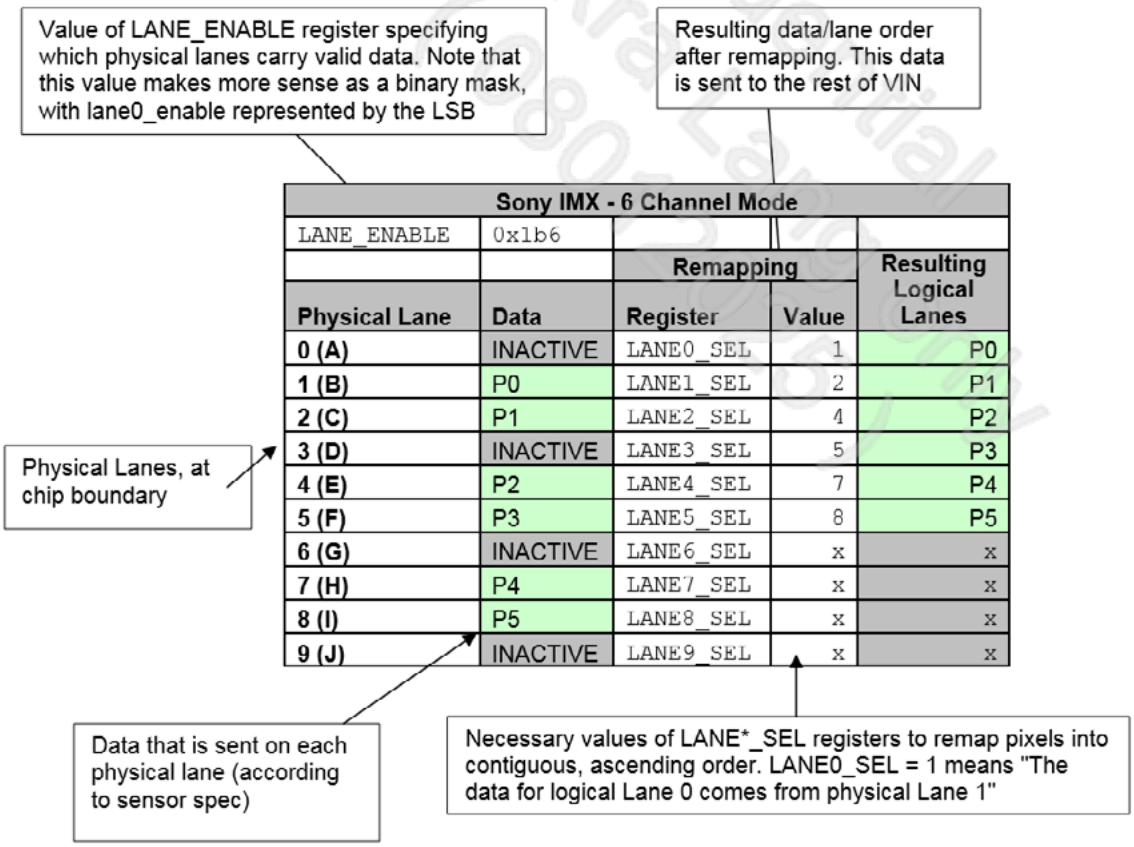


Figure 6-3. Description of Lane Mapping.

6.5.4.1 Sony IMX - 1 Channel Mode

Sony IMX - 1 Channel Mode				
LANE_ENABLE	0x20			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	INACTIVE	LANE0_SEL	4	P0
1 (B)	INACTIVE	LANE1_SEL	x	x
2 (C)	INACTIVE	LANE2_SEL	x	x
3 (D)	INACTIVE	LANE3_SEL	x	x
4 (E)	P0	LANE4_SEL	x	x
5 (F)	INACTIVE	LANE5_SEL	x	x
6 (G)	INACTIVE	LANE6_SEL	x	x
7 (H)	INACTIVE	LANE7_SEL	x	x
8 (I)	INACTIVE	LANE8_SEL	x	x
9 (J)	INACTIVE	LANE9_SEL	x	x

Figure 6-4. Sony IMX - 1 Channel Mode.

6.5.4.2 Sony IMX - 2 Channel Mode

Sony IMX - 2 Channel Mode				
LANE_ENABLE	0x30			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	INACTIVE	LANE0_SEL	4	P0
1 (B)	INACTIVE	LANE1_SEL	5	P1
2 (C)	INACTIVE	LANE2_SEL	x	x
3 (D)	INACTIVE	LANE3_SEL	x	x
4 (E)	P0	LANE4_SEL	x	x
5 (F)	P1	LANE5_SEL	x	x
6 (G)	INACTIVE	LANE6_SEL	x	x
7 (H)	INACTIVE	LANE7_SEL	x	x
8 (I)	INACTIVE	LANE8_SEL	x	x
9 (J)	INACTIVE	LANE9_SEL	x	x

Figure 6-5. Sony IMX - 2 Channel Mode.

Ambarella Confidential for Mekra Training only 1.5 (Jan 2025)

6.5.4.3 Sony IMX - 4 Channel Mode

Sony IMX - 4 Channel Mode				
LANE_ENABLE	0xb4			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	INACTIVE	LANE0_SEL	2	P0
1 (B)	INACTIVE	LANE1_SEL	4	P1
2 (C)	P0	LANE2_SEL	5	P2
3 (D)	INACTIVE	LANE3_SEL	7	P3
4 (E)	P1	LANE4_SEL	x	x
5 (F)	P2	LANE5_SEL	x	x
6 (G)	INACTIVE	LANE6_SEL	x	x
7 (H)	P3	LANE7_SEL	x	x
8 (I)	INACTIVE	LANE8_SEL	x	x
9 (J)	INACTIVE	LANE9_SEL	x	x

Figure 6-6. Sony IMX - 4 Channel Mode.

6.5.4.4 Sony IMX - 6 Channel Mode

Sony IMX - 6 Channel Mode				
LANE_ENABLE	0x1b6			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	INACTIVE	LANE0_SEL	1	P0
1 (B)	P0	LANE1_SEL	2	P1
2 (C)	P1	LANE2_SEL	4	P2
3 (D)	INACTIVE	LANE3_SEL	5	P3
4 (E)	P2	LANE4_SEL	7	P4
5 (F)	P3	LANE5_SEL	8	P5
6 (G)	INACTIVE	LANE6_SEL	x	x
7 (H)	P4	LANE7_SEL	x	x
8 (I)	P5	LANE8_SEL	x	x
9 (J)	INACTIVE	LANE9_SEL	x	x

Figure 6-7. Sony IMX - 6 Channel Mode.

Confidential for Meeting only (1.5 (08/12/2025))

6.5.4.5 Sony IMX - 8 Channel Mode

Sony IMX - 8 Channel Mode				
LANE_ENABLE	0x3b7			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	P0	LANE0_SEL	0	P0
1 (B)	P1	LANE1_SEL	1	P1
2 (C)	P2	LANE2_SEL	2	P2
3 (D)	INACTIVE	LANE3_SEL	4	P3
4 (E)	P3	LANE4_SEL	5	P4
5 (F)	P4	LANE5_SEL	7	P5
6 (G)	INACTIVE	LANE6_SEL	8	P6
7 (H)	P5	LANE7_SEL	9	P7
8 (I)	P6	LANE8_SEL	x	x
9 (J)	P7	LANE9_SEL	x	x

Figure 6-8. Sony IMX - 8 Channel Mode.

6.5.4.6 Sony IMX - 10 Channel Mode

Sony IMX - 10 Channel Mode				
LANE_ENABLE	0x3ff			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (A)	P0	LANE0_SEL	0	P0
1 (B)	P1	LANE1_SEL	1	P1
2 (C)	P2	LANE2_SEL	2	P2
3 (D)	P3	LANE3_SEL	3	P3
4 (E)	P4	LANE4_SEL	4	P4
5 (F)	P5	LANE5_SEL	5	P5
6 (G)	P6	LANE6_SEL	6	P6
7 (H)	P7	LANE7_SEL	7	P7
8 (I)	P8	LANE8_SEL	8	P8
9 (J)	P9	LANE9_SEL	9	P9

Figure 6-9. Sony IMX - 10 Channel Mode.

For Confidential Mekra Lang only (Sept 2025)

6.5.4.7 Panasonic - (MN34070) - 2 Channel 4 Port

Panasonic (MN34070) - "2 Channel 4 Port" (i.e. 8 Lanes)				
LANE_ENABLE 0xff				
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (CH1 Port0)	P0	LANE0_SEL	0	P0
1 (CH1 Port1)	P2	LANE1_SEL	4	P1
2 (CH1 Port2)	P4	LANE2_SEL	1	P2
3 (CH1 Port3)	P6	LANE3_SEL	5	P3
4 (CH2 Port0)	P1	LANE4_SEL	2	P4
5 (CH2 Port1)	P3	LANE5_SEL	6	P5
6 (CH2 Port2)	P5	LANE6_SEL	3	P6
7 (CH2 Port3)	P7	LANE7_SEL	7	P7

Figure 6-10. Panasonic - (MN34070) - 2 Channel 4 Port.

6.5.4.8 Panasonic (MN34070) - 2 Channel 3 Port

Panasonic (MN34070) - "2 Channel 3 Port" (i.e. 6 Lanes)				
LANE_ENABLE 0x77				
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (CH1 Port0)	P0	LANE0_SEL	0	P0
1 (CH1 Port1)	P2	LANE1_SEL	4	P1
2 (CH1 Port2)	P4	LANE2_SEL	1	P2
3 (CH1 Port3)	INACTIVE	LANE3_SEL	5	P3
4 (CH2 Port0)	P1	LANE4_SEL	2	P4
5 (CH2 Port1)	P3	LANE5_SEL	6	P5
6 (CH2 Port2)	P5	LANE6_SEL	x	x
7 (CH2 Port3)	INACTIVE	LANE7_SEL	x	x

Figure 6-11. Panasonic (MN34070) - 2 Channel 3 Port.

For Confidential Mating Eng only (15 Aug 2025)

6.5.4.9 Panasonic (MN34070) - 2 Channel 2 Port

Panasonic (MN34070) - "2 Channel 2 Port" (i.e. 4 Lanes)				
LANE_ENABLE 0x33				
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (CH1 Port0)	P0	LANE0_SEL	0	P0
1 (CH1 Port1)	P2	LANE1_SEL	4	P1
2 (CH1 Port2)	INACTIVE	LANE2_SEL	1	P2
3 (CH1 Port3)	INACTIVE	LANE3_SEL	5	P3
4 (CH2 Port0)	P1	LANE4_SEL	x	x
5 (CH2 Port1)	P3	LANE5_SEL	x	x
6 (CH2 Port2)	INACTIVE	LANE6_SEL	x	x
7 (CH2 Port3)	INACTIVE	LANE7_SEL	x	x

Figure 6-12. Panasonic (MN34070) - 2 Channel 2 Port.

6.5.4.10 HiSpi 2 PHYs - 2 Active Lanes Per PHY

HiSPi 2 PHYs, 2 Active Lanes Per PHY				
LANE_ENABLE 0x33				
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (PHY0 Lane0)	P0	LANE0_SEL	0	P0
1 (PHY0 Lane1)	P1	LANE1_SEL	1	P1
2 (PHY0 Lane2)	INACTIVE	LANE2_SEL	4	P2
3 (PHY0 Lane3)	INACTIVE	LANE3_SEL	5	P3
4 (PHY1 Lane0)	P2	LANE4_SEL	x	x
5 (PHY1 Lane1)	P3	LANE5_SEL	x	x
6 (PHY1 Lane2)	INACTIVE	LANE6_SEL	x	x
7 (PHY1 Lane3)	INACTIVE	LANE7_SEL	x	x

Figure 6-13. HiSpi 2 PHYs - 2 Active Lanes Per PHY.

For Makra Leng only Confidential (2025)

6.5.4.11 HiSpi 2 PHYs - 3 Active Lanes Per PHY

HiSPi 2 PHYs, 3 Active Lanes Per PHY				
LANE_ENABLE	0x77			
		Remapping		Resulting Logical Lanes
Physical Lane	Data	Register	Value	
0 (PHY0 Lane0)	P0	LANE0_SEL	0	P0
1 (PHY0 Lane1)	P1	LANE1_SEL	1	P1
2 (PHY0 Lane2)	P2	LANE2_SEL	2	P2
3 (PHY0 Lane3)	INACTIVE	LANE3_SEL	4	P3
4 (PHY1 Lane0)	P3	LANE4_SEL	5	P4
5 (PHY1 Lane1)	P4	LANE5_SEL	6	P5
6 (PHY1 Lane2)	P5	LANE6_SEL	x	x
7 (PHY1 Lane3)	INACTIVE	LANE7_SEL	x	x

Figure 6-14. HiSpi 2 PHYs - 3 Active Lanes Per PHY.

6.6 VIN: Master Sync Generation

The VIN unit includes a standalone module that generates VSync and HSync output signals for the sensor.

HSync and VSync generation is controlled by programming a period, pulse width, offset, and polarity for each.

The reference clock is GCLK_SO_VIN from RCT (and GCLK_SO_PIP for the PIP instantiation).

.

The following figure shows how the output waveform will be generated.

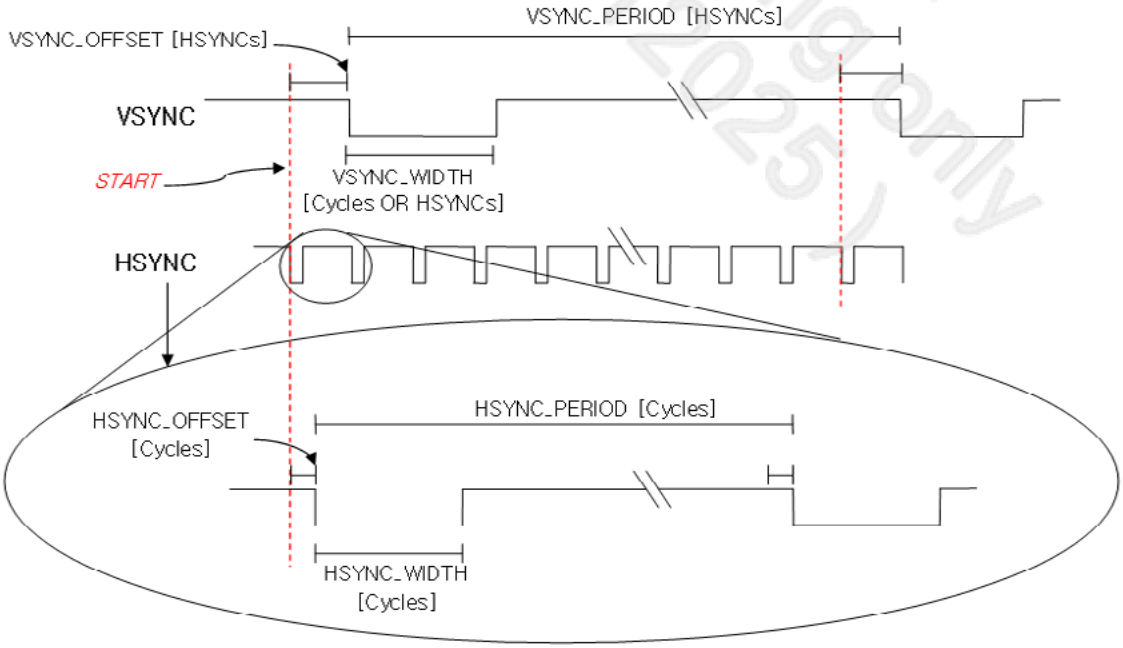


Figure 6-15. Master Sync Output Waveform.

In addition to parameters that affect the sync timing, the command also includes fields that specify how long to generate the sync and how / when to transition to the next command.

The programming is double-buffered; for example, as soon as the first programming / command completes, the next is immediately loaded and activated (provided that a second configuration has been buffered before the current one finishes). This enables seamless transition from one sync format to another.

Register	Bits	Description
HSYNC_PERIOD	32	Period of master HSync signal, in cycles
HSYNC_WIDTH	16	Width of HSync pulse, in cycles
HSYNC_OFFSET	16	Offset / delay of HSync pulse edge, in cycles
VSYNC_PERIOD	32	Period of master VSync signal, in HSyncs
VSYNC_WIDTH	16	Width of VSync pulse, in cycles OR HSyncs (see VSYNC_WIDTH_UNIT)
VSYNC_OFFSET	16	Offset / delay of HSync pulse edge, in HSync
HSYNC_POLARITY	1	0=Active Low; 1=Active High
VSYNC_POLARITY	1	0=Active Low; 1=Active High
NO_VBLANK_HSYNC	1	Do not toggle HSync during VBLANK
VSYNC_WIDTH_UNIT	1	0=VSYNC_WIDTH measured in cycles; 1=VSYNC_WIDTH measured in HSYNCS
NUM_VSYNCS	8	Number of VSync cycles to generate before allowing a new command to be loaded (if present)
CONTINUOUS	1	If no new command is available after sending NUM_VSYNCS: 0=Stop; 1=Continue until new command is received
PREEMPT	1	Load this command on the next VSync even if the current command hasn't reached NUM_VSYNCS
INTERRUPT_MODE	1	VSync interrupt at 0=End of VBLANK period; 1=End of VBLANK Pulse

Table 6-5. Master Sync Command Fields.

The following flowchart shows the details of how a master sync channel operates according to the types of commands / programming.

Confidential For Alexa Lang only 1.5 (03/12/2019)

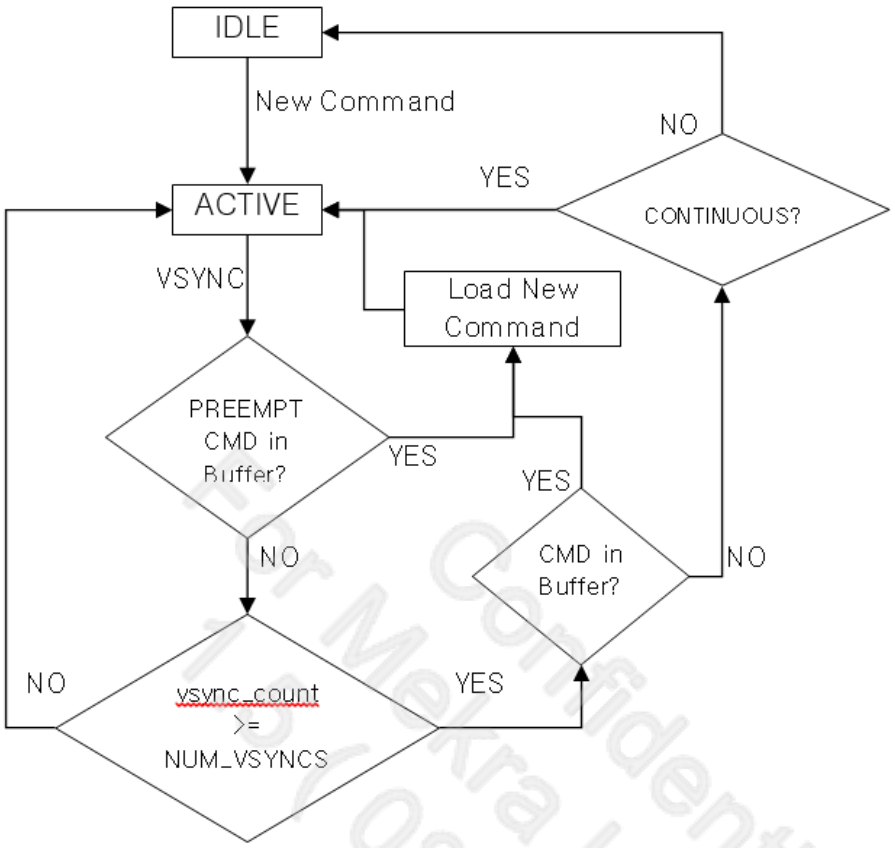


Figure 6-16. Flowchart of Master Sync Channel Operation.

6.6.1 Turning off Sync Generation

Sending a command with NUM_VSYNCS=0 creates the effect of disabling the sync generator after the command is loaded (for example, generate NO VSYNCS).

The following section show examples of continuous sync generation.

6.6.2 Example of Continuous Sync Generation

The figure below shows an example of continuous sync generation.

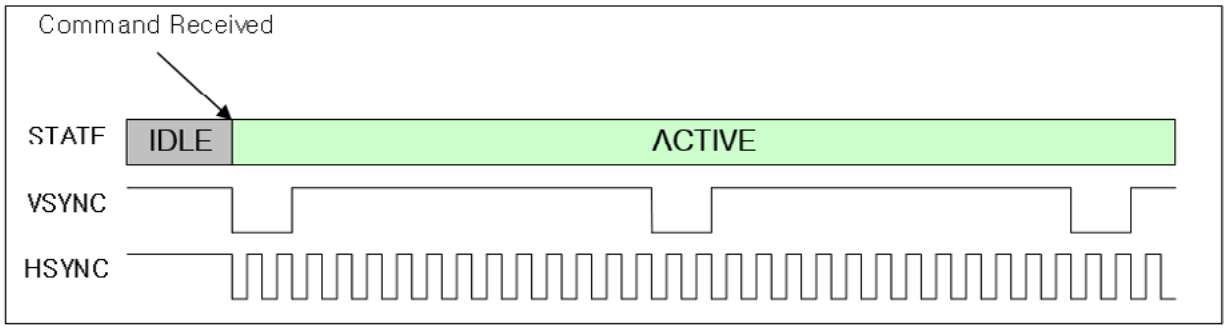


Figure 6-17. Continuous Sync Generation Example.

The procedure for programming the sync generator to generate a continuous VSync / HYSNC output is as follows:

- Starting with the sync generator in IDLE state, send a command with CONTINUOUS = 1.

  VSync and HSync begin toggling according to the programmed PERIOD and WIDTH values. (Sync generation continues until another command is sent.)

6.6.3 Example of Sync Timing Change

The figure below shows an example of sync timing change.

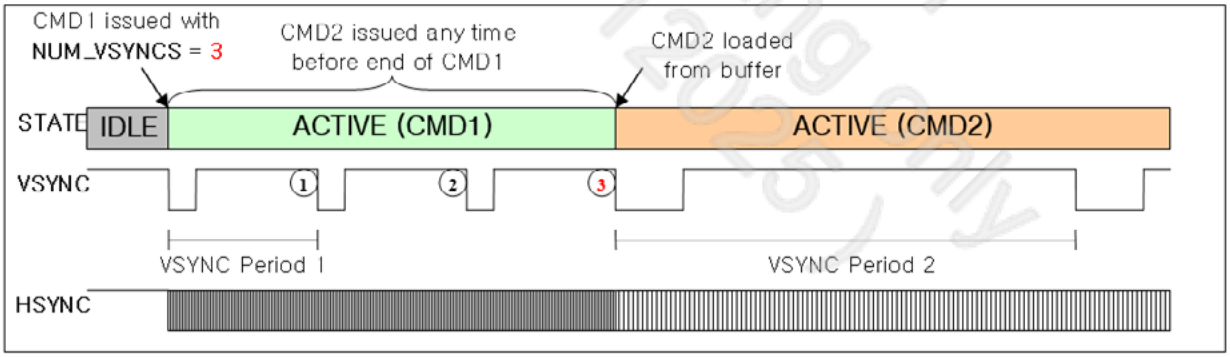


Figure 6-18. Sync Timing Change Example.

The procedure for programming the sync generator to generate N VSync periods using one particular timing, followed by M VSync periods of a new timing is as follows:

1. Starting with the sync generator in IDLE state, send a command with NUM_VSYNCS = N.

   VSync and HSync begin toggling according to the programmed PERIOD and WIDTH values.
2. Send a second command with NUM_VSYNCS = M (and new PERIOD and WIDTH values).

   The sync generator outputs N VSync periods using the first PERIOD/WIDTH values, and outputs the M VSync periods using the second PERIOD / WIDTH values.

For Nikra Lang only (Oct 2015)
Confidential

6.6.4 Example of Command Preemption

The following figure shows an example of command preemption (special case / debug feature) and how to interrupt / preempt an active command with a new command.

Typically, the new command is loaded after the active command has generated the number of VSyncs set in NUM_VSYNCS, but if PREEMPT is set in the new command, it becomes active at the next VSync, regardless of whether or not the previous command has finished.

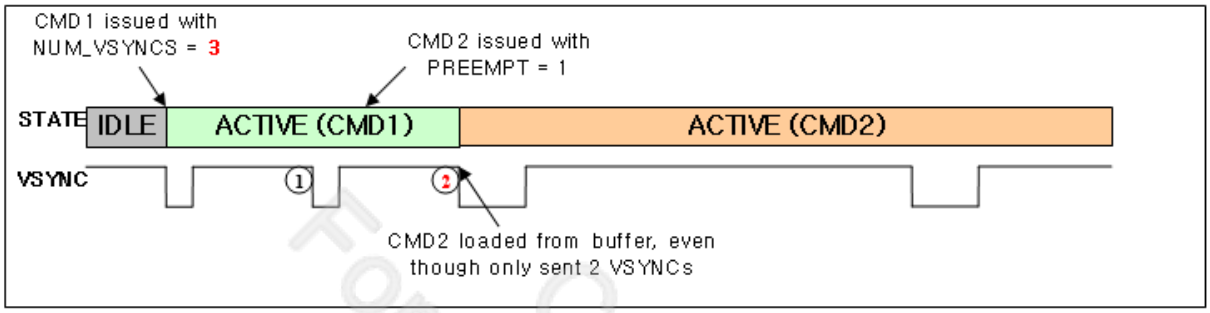


Figure 6-19. Command Preemption Example.

The procedure for programming the sync generator for command preemption is as follows:

1. Starting with the sync generator in IDLE state, send a command with NUM_VSYNCS = N.

   VSync and HSync begin toggling according to the programmed PERIOD and WIDTH values.
2. To generate a new master sync timing without having to wait for the first command to send N VSyncS, send a second command with PREEMPT = 1.

   As soon as the generator comes to the next VSync, it loads the new command and begins generating HSync / VSync using the new parameters.

Confidential
For Mediatek-ang only
(08/01/2025)

6.7 VIN: Registers

The following table includes the VIN register definitions.

Offset	Register	Bits	Access	Reset	Description
0	**SWRESET**	0	RW	1	Reset VIN logic (except config regs) when config is loaded; self-clearing
0	**ENABLE**	1	RW	0	Enable Sensor Interface logic (and interrupts, syncs etc.)
0	**OUTPUT_ENABLE**	2	RW	0	Enable output of VIN (!self-cleared after each frame!)
0	**WORD_SIZE**	5:3	RW	0	Sensor data width 0=8-bit 1=10-bit 2=12-bit 3=14-bit 4=16-bit
1	**LANE_ENABLE**	15:0	RW	0	MIPI Lane enables (physical index). Set bits 15:12 as 0
2	**Reserved**	0	RW	0	Reserved
2	**PARALLEL_RATE**	1	RW	0	0=SDR 1=DDR
2	**PARALLEL_WIDTH**	2	RW	0	0=1-Pixel wide input 1=2-Pixel wide input
2	**PARALLEL_EDGE**	3	RW	0	Data valid on 0=posedge 1=negedge
2	**PARALLEL_SYNC**	4	RW	0	0=Embedded Sync 1=External Sync
2	**PARALLEL_EMB_SYNC_LOC**	6:5	RW	0	For 2-pix width input, sync is on 0=Lower pix 1=Upper pix 2/3=Interleaved across both
2	**RGB_YUV_MODE**	7	RW	0	0=RGB 1=YUV
2	**PIX_REORDER**	9:8	RW	0	Reorder pixels {3,2,1,0} --> 0={3,2,1,0} (passthrough) 1={0,3,2,1} 2={2,3,0,1} 3={1,2,3,0}
2	**VSYNC_POLARITY**	10	RW	0	External/601 VSync 0=Active High 1=Active Low
2	**HSYNC_POLARITY**	11	RW	0	External/601 HSync 0=Active High 1=Active Low
2	**FIELD_POLARITY**	12	RW	0	External/601 FIELD 0=Non-inverted 1=Inverted
2	**FIELD_MODE**	13	RW	0	0=Field sampled at active edge of VSync 1=Field sampled continuously

For Meta Lang 1.5 (08/12/2025) Confidential

Offset	Register	Bits	Access	Reset	Description
2	DISABLE_AFTER_VSYNC	14	RW	1	Deactivate VIN front end after VSync (delayed VSync keeps counting)
2	DOUBLE_BUFF_ENABLE	15	RW	0	Enable double-buffering on config data
3	FIELD_PIN_SEL	4:0	RW	0	Typically set to 2. Uses pin SPCLK_P_1. To use a different pin, consult the Ambarella support team
3	VSYNC_PIN_SEL	9:5	RW	0	Typically set to 1. Uses pin SPCLK_N_1. To use a different pin, consult the Ambarella support team
3	HSYNC_PIN_SEL	14:10	RW	0	Typically set to 0. Uses pin SPCLK_N_0. To use a different pin, consult the Ambarella support team
4	LANE_SELECT0	3:0	RW	0	
4	LANE_SELECT1	7:4	RW	1	
4	LANE_SELECT2	11:8	RW	2	
4	LANE_SELECT3	15:12	RW	3	
5	LANE_SELECT4	3:0	RW	4	
5	LANE_SELECT5	7:4	RW	5	
5	LANE_SELECT6	11:8	RW	6	
5	LANE_SELECT7	15:12	RW	7	
6	LANE_SELECT8	3:0	RW	8	
6	LANE_SELECT9	7:4	RW	9	
6	LANE_SELECT10	11:8	RW	10	
6	LANE_SELECT11	15:12	RW	11	
7	LANE_SELECT12	3:0	RW	12	Not used. Set to default reset value
7	LANE_SELECT13	7:4	RW	13	Not used. Set to default reset value
7	LANE_SELECT14	11:8	RW	14	Not used. Set to default reset value
7	LANE_SELECT15	15:12	RW	15	Not used. Set to default reset value
8	CROP_DISABLE	0	RW	0	Disable crop region; if it is 1, ignore the crop_start/end_col/row
8	DBG_NON_BLOCKING	1	RW	0	Only set to 1 when bring up VIN to check sof/eof interrupts
8	DBG_EOL_HOLD	2	RW	0	Reserved bit, only set to 1 when requested by designer
8	MIPI_MAX_SKEW_CYCLE	5:3	RW	0	Only set it to non-zero value in 8-lane MIPI sensor; its unit is mipi_byteclk cycle; or 4*mipi_sensor_clk cycle
9	ACTIVE_WIDTH	15:0	RW	0	0=Auto-detect (only supported for protocols with EOL/EAV codes)
10	ACTIVE_HEIGHT	15:0	RW	0	0=Auto-detect
11	SPLIT_WIDTH	15:0	RW	0	For readouts that pack multiple exposure lines into a single readout line 0=Disabled
12	CROP_START_COL	15:0	RW	0	
13	CROP_START_ROW	15:0	RW	0	
14	CROP_END_COL	15:0	RW	0	
15	CROP_END_ROW	15:0	RW	0	

For Ambarella Confidential Issued Dec 12 2025

Offset	Register	Bits	Access	Reset	Description
16	SYNC_INTERLEAVING	1:0	RW	0	0=None 1=2-lane interleaving 2=4-lane interleaving (For HiSPi™ Streaming-S)
16	SYNC_TOLERANCE	3:2	RW	0	0=No error tolerance 1=Allow a corrupted sync every other code 2=Always allow corruption
16	SYNC_ALL_LANES	4	RW	0	0=Majority of active lanes 1=Sync code must be detected on all active lanes in each group
17	ALLOW_PARTIAL_CODES	0	RW	0	1=Allow one of the all-0/all-1 words to be corrupted and still be a valid sync
17	LOCK_SYNC_PHASE	1	RW	0	0=Constantly re-align phase 1=Lock/coast sync phase to first detected code
17	SYNC_CORRECTION	2	RW	1	Enable correction of sync code corruption by examining the codes across multiple lanes
17	DESKEW_ENABLE	3	RW	1	0=Disable/bypass deskew 1=enable
17	SYNC_TYPE	4	RW	0	0=ITU-656 1=Custom; field status bit will only toggle in ITU-656
17	ENABLE_656_ECC	5	RW	0	Enable ITU-656 sync error correction/detection
17	PAT_MASK_ALIGNMENT	6	RW	1	Sync code patterns/masks 0=LSB aligned 1=MSB aligned
17	UNLOCK_ON_TIMEOUT	7	RW	1	Force sync unlock on sync timeout
17	UNLOCK_ON_DESKEW_ERROR	8	RW	1	Force sync unlock on deskew error
17	LINE_REORDER	9	RW	0	Enable line reordering for SSLL --> SLSL conversion
18	SOL_EN	0	RW	0	Enable Detection of SOL sync codes
18	EOL_EN	1	RW	0	Enable Detection of EOL sync codes
18	SOF_EN	2	RW	0	Enable Detection of SOF sync codes
18	EOF_EN	3	RW	0	Enable Detection of EOF sync codes
18	SOV_EN	4	RW	0	Enable Detection of SOV sync codes
18	EOV_EN	5	RW	0	Enable Detection of EOV sync codes
19	WATCHDOG_TIMEOUT_L	15:0	RW	0	0=Disabled Else=Timeout in unit IDSP cycles
20	WATCHDOG_TIMEOUT_H	15:0	RW	0	
21	SYNC_DETECT_MASK	15:0	RW	0	VIN compares (SYNC_DETECT_MASK & SYNC_CODE) against SYNC_DETECT_PAT
22	SYNC_DETECT_PAT	15:0	RW	0	
23	SYNC_COMPARE_MASK	15:0	RW	0	Mask to apply to sync code before comparing to patterns
24	SOL_PAT	15:0	RW	0	Pattern to match to indicate SOL code
25	EOL_PAT	15:0	RW	0	

For Meta Confidential 1.5 Oct 2025

Offset	Register	Bits	Access	Reset	Description
26	**SOF_PAT**	15:0	RW	0	
27	**EOF_PAT**	15:0	RW	0	
28	**SOV_PAT**	15:0	RW	0	
29	**EOV_PAT**	15:0	RW	0	
30	**MIPI_VC_MASK**	1:0	RW	0	2-bit value to mask the corresponding virtual channel ID bit. Set it to 11 to capture all virtual channels.
30	**MIPI_VC_PAT**	3:2	RW	0	2-bit value to indicate the pattern of the virtual channel ID a VIN is supposed to capture
30	**MIPI_DT_MASK**	9:4	RW	0	6-bit value to mask the corresponding data type ID bit. Set to 0x3f to capture all data types.
30	**MIPI_DT_PAT**	15:10	RW	0	6-bit value to indicate the pattern of the data type ID a VIN is supposed to capture
31	**MIPI_ED_DT_MASK**	5:0	RW	0	6-bit value to mask the corresponding embedded data ID bit. Set to 0x3f to capture all image data (embedded and video).
31	**MIPI_ED_DT_PAT**	11:6	RW	0	6-bit value to indicate the pattern of the embedded data ID a VIN is supposed to capture
31	**MIPI_BYTE_SWAP**	12	RW	0	
31	**MIPI_ECC_ENABLE**	13	RW	0	
31	**MIPI_FWD_ECC**	14	RW	1	0=Do not forward errored packets 1=Forward errored packets (default mode)
31	**MIPI_ED_ENABLE**	15	RW	0	Enable embedded data capturing
32	**VOUT_SYNC0_MODE**	1:0	RW	0	0=Disabled 1=Even Fields 2=Odd Fields 3=All Fields Usage example: If mode is MIPI, mode must be set to 0 or 3.
32	**VOUT_SYNC0_LINE**	15:2	RW	0	VOUT SYNC0 is asserted for the duration of the specified active line
33	**VOUT_SYNC1_MODE**	1:0	RW	0	0=Disabled 1=Even Fields 2=Odd Fields 3=All Fields
33	**VOUT_SYNC1_LINE**	15:2	RW	0	VOUT SYNC1 is asserted for the duration of the specified active line
34	**STRIG0_ENABLE**	0	RW	0	
34	**STRIG0_POLARITY**	1	RW	0	
34	**STRIG0_START_LINE**	15:2	RW	0	Active line number to start asserting STRIG0
35	**STRIG0_END_LINE**	15:0	RW	0	Active line number to stop asserting STRIG0
36	**STRIG1_ENABLE**	0	RW	0	
36	**STRIG1_POLARITY**	1	RW	0	
36	**STRIG1_START_LINE**	15:2	RW	0	Active line number to start asserting strig1
37	**STRIG1_END_LINE**	15:0	RW	0	Active line number to stop asserting strig1

For Nekta Langue Confidential 1.5 Dec 12 2025

Offset	Register	Bits	Access	Reset	Description
38	DELAYED_VSYNC_SYNC_DELAY_L	15:0	RW	0	Delay in cycles after VSync before assertion of 'delayed VSync' signal to controller (>0) **Note**: Cycles refers to the delay in the number of sensor clock cycles
39	DELAYED_VSYNC_SYNC_DELAY_H	15:0	RW	0	
40	DELAYED_VSYNC_INTR_DELAY_L	15:0	RW	0	Delay in cycles after VSync before assertion of delayed VSync interrupt to Arm (Lower 16-bits)
41	DELAYED_VSYNC_INTR_DELAY_H	15:0	RW	0	Delay in cycles after VSync before assertion of delayed VSync interrupt to Arm (Upper 16-bits)
42	BLACK_LEVEL_OFFSET_SIGN_BIT	0	RW	0	
43	BLACK_LEVEL_OFFSET_16LSB	15:0	RW	0	
44	WATCHDOG_TIMEOUT_INTR_MASK	0	RW	1	Error *will not* be reported to IDSP INTR when set to zero
44	SHORT_FRAME_INTR_MASK	1	RW	1	Bit to mask short frame error interrupt to Arm
44	SHORT_LINE_INTR_MASK	2	RW	1	Bit to mask short line error interrupt to Arm
44	SFIFO_OVERFLOW_INTR_MASK	3	RW	1	Bit to mask SFIFO overflow error interrupt to Arm
44	AFIFO_OVERFLOW_INTR_MASK	4	RW	1	Bit to mask AFIFO overflow error interrupt to Arm
44	Reserved	5	RW	1	Reserved
44	UNKNOWN_SYNC_CODE_INTR_MASK	6	RW	1	Bit to mask unknown sync code error interrupt to Arm
44	UNCORRECTABLE_656_ERR_INTR_MASK	7	RW	1	Bit to mask uncorrectable BT656 error interrupt to Arm
44	Reserved	8	RW	1	Reserved
44	ERR_FS_INTR_MASK	9	RW	1	Bit to mask error interrupt to Arm when there is an incorrect frame sync packet sequence in MIPI stream
44	CRC_ERR_INTR_MASK	10	RW	1	Bit to mask error interrupt to Arm when the detected CRC error number is larger than CRC_ERR_THRESHOLD
44	ECC_CORR_ERR_INTR_MASK	11	RW	1	Bit to mask error interrupt to Arm when the detected 1-bit ECC error number is larger than ECC_CORR_ERR_THRESHOLD
44	ECC_UNCORR_ERR_INTR_MASK	12	RW	1	Bit to mask uncorrectable ECC error
45	WATCHDOG_TIMEOUT_ECR_MASK	0	RW	1	Error *will not* be reported to sm ERR_VLD when set to zero
45	SHORT_FRAME_ECR_MASK	1	RW	1	
45	SHORT_LINE_ECR_MASK	2	RW	1	
45	SFIFO_OVERFLOW_ECR_MASK	3	RW	1	
45	AFIFO_OVERFLOW_ECR_MASK	4	RW	1	

Ambarella Confidential Use Only 12/25/2025

Offset	Register	Bits	Access	Reset	Description
45	LOST_LOCK_AFTER_SOF_ECR_MASK	5	RW	1	
45	UNKNOWN_SYNC_CODE_ECR_MASK	6	RW	1	
45	UNCORRECTABLE_656_ERR_ECR_MASK	7	RW	1	
45	SERIAL_DESKEW_ERR_ECR_MASK	8	RW	1	
45	ERR_FS_ECR_MASK	9	RW	1	
45	CRC_ERR_ECR_MASK	10	RW	1	
45	ECC_CORR_ERR_ECR_MASK	11	RW	1	
45	ECC_UNCORR_ERR_ECR_MASK	12	RW	1	
46	GOT_ACT_SOF	0	RWC	0	Detected start of active region (write 1 to clear)
46	GOT_WIN_SOF	1	RWC	0	Reached start of crop region (write 1 to clear)
46	GOT_WIN_EOF	2	RWC	0	Reached end of crop region (write 1 to clear)
46	GOT_VSYNC	3	RWC	0	Reached end of active region (start of VBLANK) (write 1 to clear)
46	AFIFO_OVERFLOW	4	RWC	0	AFIFO overflow (must reset VIN)
46	SFIFO_OVERFLOW	5	RWC	0	SFIFO has overflowed (must reset VIN)
46	SHORT_LINE	6	RWC	0	EOL received before reaching end of programmed crop width (write 1 to clear)
46	SHORT_FRAME	7	RWC	0	EOF received before reaching end of programmed crop region (write 1 to clear)
46	FIELD	8	R	0	Field bit value
46	RESERVED	9	RW	0	Unused
46	SENT_MASTER_VSYNC	10	RWC	0	Sent master VSync (from master sync generator unit) (write 1 to clear)
46	UNCORRECTABLE_656_ERROR	11	RWC	0	ITU-656 sync code was corrupted and uncorrectable (write 1 to clear)
46	SYNC_LOCKED	12	R	0	VIN front-end sync is locked
46	LOST_LOCK_AFTER_SOF	13	RWC	0	Lost lock after detecting start of frame (must reset VIN)
46	PARTIAL_SYNC_DETECTED	14	RWC	0	Partial/corrupted sync was detected (write 1 to clear)
46	UNKNOWN_SYNC_CODE	15	RWC	0	Sync code did not match any patterns (write 1 to clear)
47	SYNC_STATE	1:0	R	0	State of front-end sync FSM 0=IDLE 1=ACTIVE 2=HBLANK 3=VBLANK)
47	Reserved	2	RWC	0	Reserved
47	WATCHDOG_TIMEOUT	3	RWC	0	
47	SFIFO_COUNT	14:4	R	0	
48	DETECTED_ACTIVE_WIDTH	15:0	R	0	

For Media Lane Confidential
1.5 Oct 2025

Offset	Register	Bits	Access	Reset	Description
49	DETECTED_ACTIVE_HEIGHT	15:0	R	0	
50	SYNC_CODE	15:0	R	0	
51	ACT_V_COUNT	15:0	R	0	
52	ACT_H_COUNT	15:0	R	0	
53	WIN_V_COUNT	15:0	R	0	
54	WIN_H_COUNT	15:0	R	0	
55	MIPI_ECC_ERR_2BIT	0	RWC	0	2-bit uncorrectable ECC error
55	MIPI_ECC_ERR_1BIT	1	RWC	0	1-bit ECC error corrected
55	MIPI_CRC_ERROR	2	RWC	0	Long packet CRC error
55	MIPI_ERR_FS	3	RWC	0	Frame sync error (received unpaired SOF or EOF)
55	MIPI_PROT_STATE	5:4	R	0	State of FSM in MIPI protocol module
55	MIPI_SOF_PACKET_RCVD	6	RWC	0	SOF short packet received
55	MIPI_EOF_PACKET_RCVD	7	RWC	0	EOF short packet received
56	MIPI_LONG_PACKET_COUNT	15:0	RWC	0	Number of long packets received
57	MIPI_SHORT_PACKET_COUNT	15:0	RWC	0	Number of short packets received
58	MIPI_LONG_PACKET_SIZE	15:0	R	0	Number of bytes in last received long packet
59	MIPI_NUM_CRC_ERR_PACKET	15:0	RWC	0	Number of packets with CRC errors
60	MIPI_FRAME_NUMBER	15:0	R	0	Frame number (from short-packet / header info)
61	MIPI_CRC_ERR_THRESHOLD	15:0	RW	0	Number of CRC err threshold that when crossed, error will be reported
62	MIPI_CORR_ECC_ERR_THRESHOLD	15:0	RW	64	Number of correctable threshold that when crossed, error will be reported
63	MIPI_NUM_CORR_ECC_ERR_PACKET	15:0	RWC	0	Number of packets with correctable errors
128	DEBUG_COMMAND_SHIFT	0	W	0	To manually shift VIN programming via debug bus
65520	ERROR_STATUS_0_COUNT	0	RW	0	Fatal error status error count
65520	ERROR_STATUS_0_COUNTERROR_STATUS_0_MODULE	14:10	RW		Fatal error status module ID. Can be used to debug which internal RTL module generated the error
65522	ERROR_STATUS_1_COUNTERROR_STATUS_0_COUNT	9:0	RW	0	Non-fatal error status error count
65522	ERROR_STATUS_0_COUNTERROR_STATUS_1_MODULE	14:10	RW		Non-fatal error status module ID. Can be used to debug which internal RTL module generated the error.
65524	ERROR_INSERT_FATALERROR_STATUS_0_COUNT	0	RW	0	Insert fatal error
65524	ERROR_INSERT_NON_FATALERROR_STATUS_0_COUNT	1	RW	0	Insert non-fatal error
65525	ERROR_THRESHOLDERROR_STATUS_0_COUNT	9:0	RW	127	Threshold for non-fatal error
256	HSYNC_PERIOD_L	15:0	RW	0	HSync period in cycles (low bits)
257	HSYNC_PERIOD_H	15:0	RW	0	HSync period in cycles (high bits)

For Beta Confidential Use Only 12/25/2025

Offset	Register	Bits	Access	Reset	Description
258	**HSYNC_WIDTH**	15:0	RW	0	Width of HSync pulse in cycles
259	**HSYNC_OFFSET**	15:0	RW	0	Offset of HSync pulse in cycles
260	**VSYNC_PERIOD_L**	15:0	RW	0	Number of HSync between VSyncs
261	**VSYNC_PERIOD_H**	15:0	RW	0	Number of HSYNCS between VSyncS
262	**VSYNC_WIDTH**	15:0	RW	0	Width of VSync pulse in cycles OR HSyncs (see VSYNC_WIDTH_UNIT)
263	**VSYNC_OFFSET**	15:0	RW	0	Offset between HSync and VSync edges (in cycles); VSync trails HSync
264	**HSYNC_POLARITY**	0	RW	0	0=Active Low 1=Active High Example: Polarity for master HSync
264	**VSYNC_POLARITY**	1	RW	0	0=Active Low 1=Active High Example: Polarity for master VSync
264	**NO_VBLANK_HSYNC**	2	RW	0	Do not toggle HSync during VBLANK
264	**INTERRUPT_MODE**	3	RW	0	VSync interrupt at 0=End of VBLANK period 1=End of VBLANK Pulse
264	**VSYNC_WIDTH_UNIT**	4	RW	0	VSYNC_WIDTH is measured in 0=Cycles 1=HSYNCS
264	**NUM_VSYNCS**	12:5	RW	0	Number of VSync cycles to send before loading next command (if present) 0=Turn off sync
264	**CONTINUOUS**	13	RW	1	If no new command is available after sending NUM_VSYNCS 0=Stop 1=Continue until new command is received
264	**PREEMPT**	14	RW	0	Load this command on the next VSync even if the current command hasn't finished
512	**SENSOR_TYPE**	1:0	RW	0	0=Reserved 1=Reserved 2=Reserved 3=MIPI
512	**CLK_MODE**	2	RW	0	0=Reserved 1=Every 4 lanes latched by group / phy clock
512	**Reserved**	3	RW	0	Reserved
512	**Reserved**	4	RW	0	Reserved
512	**MIPI_VC_ENABLE**	5	RW	0	Set to 1 if virtual channels are present

Table 6-6. VIN Registers.

Confidential
For Ambarella Engineering
1.5 CV22 01/2025

7. VIDEO OUTPUT (VOUT)

This chapter includes the following sections:

- (Section 7.1) VOUT: Overview
- (Section 7.2) VOUT: Address Space Overview
- (Section 7.3) VOUT: Top Registers—Map
- (Section 7.4) VOUT: Top Registers—Detail
- (Section 7.5) VOUT: Mixer Section Registers—Overview
- (Section 7.6) VOUT: Mixer Section Registers—Map
- (Section 7.7) VOUT: Mixer Section Registers—Detail
- (Section 7.8) VOUT: Display Section Registers—Map
- (Section 7.9) VOUT: Display Section Registers—Detail
- (Section 7.10) VOUT: OSD Rescale Registers—Map
- (Section 7.11) VOUT OSD Rescale Registers—Detail
- (Section 7.12) VOUT: Safety-Related Registers in VOUT ECR—Map
- (Section 7.13) VOUT: Safety Notes
- (Section 7.14) VOUT: BTA Programming Flow
- (Section 7.15) VOUT: MIPI DSI-PHY Protocol Timing

7.1 VOUT: Overview

The video output module drives data to on-chip MIPI PHY. There are two functionally identical Display sections driving two streams to two PHYs. Each Display section supports MIPI DSI, MIPI CSI-2.

One OSD rescale unit is included which can be used by Mixer A or Mixer B (but not both) at any given time. At a basic level, the Mixer sections combine the OSD and video data. The Display sections create the output frame timing and convert the data into the format needed for the particular display in use.

Note:

The VOUT interface and pin and package details are provided in the *"CV22FS Datasheet."*

7.2 VOUT: Address Space Overview

The VOUT address space is logically divided into a number of sections according to the following table. Offsets listed are from the VOUT base address. Note that some tables are mapped to a separate part of the AHB address space due to the number of registers. When accessing from AHB, adjust the offsets for the Gamma and CLUT tables according to the AHB VOUT section offset.

Confidential For Meka Lang only (2021-08-12)

Address Range	Section	Description
0x200 - 0x2FC	**Mixer A**	Mixer section A control
0x300 - 0x4FC	**Display A**	Display section A control
0x500 - 0x5FC	**Mixer B**	Mixer section B control
0x600 - 0x7FC	**Display B**	Display section B control
0x800 - 0x9FC	**OSD Rescale**	Controls OSD rescaler
0xA00- 0xAFC	**VOUT TOP**	Top level reset, clock enables and ECR
0xB00-0xB14	**MIPI COMMAND A**	Display A MIPI command
0xE00-0xE14	**MIPI COMMAND B**	Display B MIPI command
0x1000-0x1BFC	**GAMMA TABLE A**	Gamma table for display A
0x1C00-0x27FC	**GAMMA TABLE B**	Gamma table for display B
0x2800 - 0x2BFC	**Mixer A CLUT**	Color-lookup-table for the OSD
0x2C00 - 0x2FFF	**Mixer B CLUT**	Color-lookup-table for the OSD

Table 7-1. VOUT Address Space Overview.

7.3 VOUT: Top Registers—Map

7.3.1 Clock and Reset

Offset	Register	Description
0xA00	**VOUT_RESET**	Soft reset entire VOUT
0xA04	**VOUT_ENABLE_CLOCK_A**	Enable clock for DISPLAY_A and MIXER_A
0xA08	**VOUT_ENABLE_CLOCK_B**	Enable clock for DISPLAY_B and MIXER_B
0xA0C	**VOUT_ENABLE_CLOCK_OSD_RESCALE**	Enable clock for OSD_RESCALER
0xA10	**VOUT_SMEMIF_CLOCK_GATING**	Clock gating SMEM interface
0xA14	**VOUT_CONFIG_CLOCK_GATING**	Clock gating configuration section
0xA18	**VOUT_MIXER_CLOCK_GATING**	Clock gating mixer sections

Table 7-2. VOUT Address Space Overview.

7.3.2 VOUT ECR

Offset	Register	Description
0xA40	**VOUT_ECR_CTRL**	Final error valid and clear error
0xA44	**VOUT_ECR_ERR**	ECR error bits
0xA48	**VOUT_ECR_MASK**	Mask bits to disable a specific section error valid

Table 7-3. VOUT Address Space Overview.

Confidential For Mekra Use Only 1.5 (Feb 25)

7.4 VOUT: Top Registers—Detail

By default, the registers' reset values will not disable functions. The following can be used to disable select functions and save power.

7.4.1 Vout_Reset

Bits	Name	Access	Reset	Description
31	**reset_vout**	RW	0	Reset entire VOUT
30:0	-	-	-	Reserved

Table 7-4. Vout_Reset.

7.4.2 Vout_Enable_Clock_A

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**Enable_clock_a**	RW	1	Enable clocks for DISPLAY_A and MIXER_A

Table 7-5. Vout_Enable_Clock_A.

7.4.3 Vout_Enable_Clock_B

Bits	Name	Access	Reset	Description
31:0	-	-	-	Reserved
0	**Enable_clock_b**	RW	1	Enable clocks for DISPLAY_B and MIXER_B

Table 7-6. Vout_Enable_Clock_B.

7.4.4 Vout_Enable_Clock_Osdr

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**Enable_clock_ Osdr**	RW	1	Enable clocks for OSD_RESCALER

Table 7-7. Vout_Enable_Clock_Osdr.

7.4.5 Vout_Enable_SmemIf_Clock_gating

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved

For Mekra Lang only Confidential (08.2025)

Bits	Name	Access	Reset	Description
0	**Enable_clock_smemif**	RW	1	Enable clocks for SMEM interface

Table 7-8. Vout_Enable_SmemIf_Clock_gating.

7.4.6 Vout_Enable_Config_Clock_gating

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**Enable_config_gating**	RW	0	Set to 1 when clock gating is needed for configuration

Table 7-9. Vout_Enable_Config_Clock_gating.

7.4.7 Vout_Enable_Mixer_Clock_gating

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**Enable_mixer_gating**	RW	0	Set to 1 when clock gating is needed for mixer sections

Table 7-10. Vout_Enable_Mixer_Clock_gating.

7.4.8 Vout_Ecr_Ctrl

Bits	Name	Access	Reset	Description
31:2	-	-	-	Reserved
1	**Inj_err**	RW	0	Write 1 will assert VOUT safety error valid to CEHU
0	**Clr_status**	RW	0	Write 1 will clear VOUT safety error valid

Table 7-11. Vout_Ecr_Ctrl.

7.4.9 Vout_Ecr_Error

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6	**smemif_err**	RW	0	Error detected in SMEM interface module
5	**display_b_err**	RW	0	Error detected in DISPLAY_B module
4	**display_a_err**	RW	0	Error detected in DISPLAY_A module
3	**mixer_b_err**	RW	0	Error detected in MIXER_B module
2	**mixer_a_err**	RW	0	Error detected in MIXER_A module
1	**osd_rescale_err**	RW	0	Error detected in OSD_RESCALE module

Confidential For Media Lang only for 08012022

Bits	Name	Access	Reset	Description
0	**top_misc_err**	RW	0	Error detected in other top level modules, like sync counter interface, reset controller, AHB_INTERFACE

Table 7-12. Vout_Ecr_Error.

7.4.10 Vout_Ecr_Mask

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6	**smemif_err_mask**	RW	1	Error mask in SMEM interface module 0: Do not report error 1: Report error when detected
5	**display_b_err_mask**	RW	1	Error mask in DISPLAY_B module 0: Do not report error 1: Report error when detected
4	**display_a_err_mask**	RW	1	Error mask in DISPLAY_A module 0: Do not report error 1: Report error when detected
3	**mixer_b_err_mask**	RW	1	Error mask in MIXER_B module 0: Do not report error 1: Report error when detected
2	**mixer_a_err_mask**	RW	1	Error mask in MIXER_A module 0: Do not report error 1: Report error when detected
1	**osd_rescale_err_mask**	RW	1	Error mask in OSD_RESCALE module 0: Do not report error 1: Report error when detected
0	**top_misc_err_mask**	RW	1	Error mask in other top level modules, like sync counter interface, reset controller, AHB_INTERFACE 0: Do not report error 1: Report error when detected

Table 7-13. Vout_Ecr_Mask.

7.5 VOUT: Mixer Section Registers—Overview

The Mixer section must first be configured before it can be enabled. A basic configuration consists of active region programming with optional video and / or OSD regions as well. The Mixer section can be completely reconfigured between frames. If **osd_rescale** is used to function with a specific Mixer section, program **osd_rescale** before enabling Mixer **frame_enable**.

CV22FS includes double command buffer support in two Mixer sections in VOUT, providing the software with a large timing window for programming the VOUT Mixer section. This feature also enables users to potentially change the Mixer programming between frames. However, Ambarella cannot currently support full double-buffering in all mixer-related registers, such as the configurations that are stored in SRAM.

Because the RTL design is backward compatible, users can choose to use the double-buffering feature or the previous feature, “program next frame when current frame finishes.”

The diagram on the following page shows an example of two approaches to programming: the left side of the figure provides an example of the older method (no double-buffering); the right side of the figure provides an example of when the double-buffering is used.

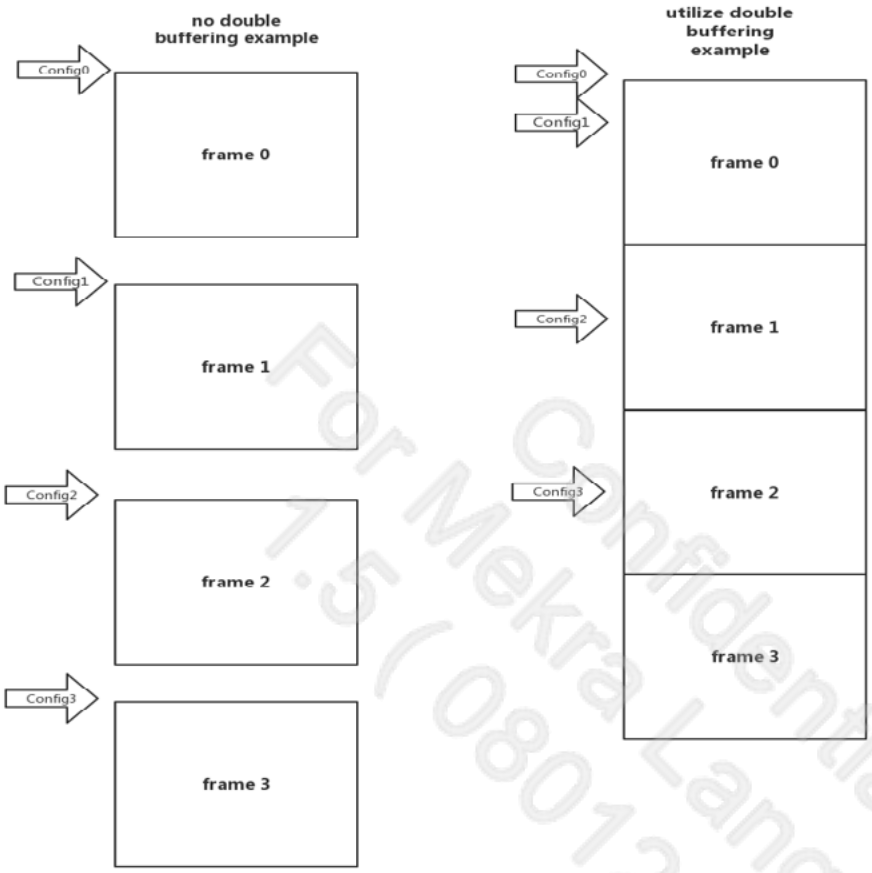


Figure 7-1. Example of Mixer Section Programming Approaches.

After the first configuration, it is unnecessary to write all mixer addresses. All unwritten registers maintain the previous frame's number. Therefore, if users do not want to make changes between frames, they need only to write frame enable.

7.6 VOUT: Mixer Section Registers—Map

7.6.1 Mixer Section 0 Registers

Offset	Register	Description
0x200	**VOUTM_control**	Mixer section control
0x204	**VOUTM_status**	Mixer status
0x208	**VOUTM_active_size**	Total size of active region
0x20C	**VOUTM_video_start**	Start location of video region
0x210	**VOUTM_video_end**	End location of video region
0x214 -	**VOUTM_dummy_access**	For SW to test reset

For Mekra Lang only Confidential 1.5 (08012025)

Offset	Register	Description
0x218	**Reserved**	-
0x21C	**VOUTM_osd_start**	Start location of OSD region
0x220	**VOUTM_osd_end**	End location of OSD region
0x224	**VOUTM_background**	Background color
0x228	**VOUTM_threshold**	Highlight threshold and color
0x22C	**VOUTM_osd_control**	OSD control
0x230	**VOUTM_osd_csc_param_0**	Color-space-conversion setting for OSD
0x234	**VOUTM_osd_csc_param_1**	Color-space-conversion setting for OSD
0x238	**VOUTM_osd_csc_param_2**	Color-space-conversion setting for OSD
0x23C	**VOUTM_osd_csc_param_3**	Color-space-conversion setting for OSD
0x240	**VOUTM_osd_csc_param_4**	Color-space-conversion setting for OSD
0x244	**VOUTM_osd_csc_param_5**	Color-space-conversion setting for OSD
0x248	**VOUTM_osd_csc_param_6**	Color-space-conversion setting for OSD
0x24C	**VOUTM_osd_csc_param_7**	Color-space-conversion setting for OSD
0x250	**VOUTM_osd_csc_param_8**	Color-space-conversion setting for OSD
0x254	**VOUTM_vertical_upsampler_control**	Chroma vertical upsampler control
0x258	**VOUTM_vertical_upsampler_out_height**	Output height
0x25C	**VOUTM_vertical_upsampler_phase_inc**	Phase increment
0x260	**VOUTM_vertical_upsampler_phase_init**	Initial phase
0x264	**VOUTM_vertical_upsampler_dither_ctrl**	Dither control
0x268	**VOUTM_vertical_upsampler_dither_seed**	Dither write seed
0x26C-0x2A8	**VOUTM_vertical_upsampler_coeff**	Coefficients for upsampler
0x2AC	**VOUTM_stream_enables**	Enables inputs and outputs
0x2B0	**VOUTM_input_sync_counter_control**	Controls input synchronization
0x2B4	**Reserved**	-
0x2B8	**VOUTM_output_sync_counter_control**	Controls output synchronization
0x2BC	**VOUTM_stream_in_control0**	Input SMEM channels SETUP0
0x2C0	**OUTM_stream_in_contorl1**	Input SMEM channels SETUP1
0x2C4	**VOUTM_stream_out_control**	Output SMEM channel setup
0x2C8	**VOUTM_frame_enable**	ENABLE_FRAME
0x2800 - 0x2BFC	**VOUTM_clut**	Color-lookup-table for the OSD

Table 7-14. Mixer Section 0 Registers Map.

7.6.2 Mixer Section 1 Registers

Offset	Register	Description
0x500	**VOUTM_control**	Mixer section control
0x504	**VOUTM_status**	Mixer status
0x508	**VOUTM_active_size**	Total size of active region
0x50C	**VOUTM_video_start**	Start location of video region
0x510	**VOUTM_video_end**	End location of video region
0x514	**VOUTM_dummy_access**	For SW to test reset
0x518	**Reserved**	-
0x51C	**VOUTM_osd_start**	Start location of OSD region
0x520	**VOUTM_osd_end**	End location of OSD region
0x524	**VOUTM_background**	Background color
0x528	**VOUTM_threshold**	Highlight threshold and color
0x52C	**VOUTM_osd_control**	OSD control
0x530	**VOUTM_csc_param_0**	Color-space-conversion setting for OSD
0x534	**VOUTM_csc_param_1**	Color-space-conversion setting for OSD
0x538	**VOUTM_csc_param_2**	Color-space-conversion setting for OSD
0x53C	**VOUTM_csc_param_3**	Color-space-conversion setting for OSD
0x540	**VOUTM_csc_param_4**	Color-space-conversion setting for OSD
0x544	**VOUTM_csc_param_5**	Color-space-conversion setting for OSD
0x548	**VOUTM_csc_param_6**	Color-space-conversion setting for OSD
0x54C	**VOUTM_csc_param_7**	Color-space-conversion setting for OSD
0x550	**VOUTM_csc_param_8**	Color-space-conversion setting for OSD
0x554	**VOUTM_vertical_upsampler_control**	Chroma vertical upsampler control
0x558	**VOUTM_vertical_upsampler_out_height**	Output height
0x55C	**VOUTM_vertical_upsampler_phase_inc**	Phase increment
0x560	**VOUTM_vertical_upsampler_phase_init**	Initial phase
0x564	**VOUTM_vertical_upsampler_dither_ctrl**	Dither control
0x568	**VOUTM_vertical_upsampler_dither_seed**	Dither write seed
0x56C-0x5A8	**VOUTM_vertical_upsampler_coeff**	Coefficients for upsampler
0x5AC	**VOUTM_stream_enables**	Enables inputs and outputs
0x5B0	**VOUTM_input_sync_counter_control**	Controls input synchronization
0x5B4	**Reserved**	-
0x5B8	**VOUTM_output_sync_counter_control**	Controls output synchronization
0x5BC	**VOUTM_stream_in_control0**	Input SMEM channels SETUP0
0x5C0	**VOUTM_stream_in_contorl1**	Input SMEM channels
0x5C4	**VOUTM_stream_out_control**	Output SMEM channel setup
0x5C8	**VOUTM_frame_enable**	ENABLE_FRAME
0x2C00 - 0x2FFF	**VOUTM_clut**	Color-lookup-table for the OSD

Table 7-15. Mixer Section 1 Registers Map.

Confidential Ambarella Property

7.7 VOUT: Mixer Section Registers—Detail

7.7.1 VOUTM_control

Bits	Name	Access	Reset	Description
31	**reset_mixer_section**	RW	0	Set to 1 to reset the mixer section
30:5	-	-	-	Reserved
5	**420_video_output**	RW	0	1: Output 420 video
4	**420_video_input**	RW	0	1: Input 420 video
3	**reverse_mode_enable**	RW	0	0: Normal mode 1: Video data is horizontally reversed
2:1	**csc_enable**	RW	0	0: Mixer CSC disabled 1: Mixer CSC enabled for video 2 - mixer CSC enabled for OSD
0	**mixer_color_resolution**	RW	0	0: Mix in YUV 4:2:2 1: Mix in YUV 4:4:4 or RGB

Table 7-16. VOUTM_control.

7.7.2 VOUTM_status

Bits	Name	Access	Reset	Description
31	**reset_mixer_section_complete**	RW	0	Hardware sets to 1 when the mixer section reset is complete
30:0	-	-	-	Reserved

Table 7-17. VOUTM_status.

7.7.3 VOUTM_active_size

The Mixer section supports one video region and one OSD region. The active region represents the size of the output and should contain both the video and OSD regions. Portions of the active region not covered by the video region will be filled in with the background color prior to mixing in the OSD.

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**active_size_h**	RW	0	Number of pixels per line - 1
15:14	-	-	-	Reserved
13:0	**active_size_v**	RW	0	Number of lines in the active region - 1

Table 7-18. VOUTM_active_size.

For Mekrang only
Confidential
1.5 (08/12/2025)

7.7.4 VOUTM_video_start

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**video_start_h**	RW	0	Starting column of video region
15:14	-	-	-	Reserved
13:0	**video_start_v**	RW	0	Starting line of video region

Table 7-19. VOUTM_video_start.

7.7.5 VOUTM_video_end

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**video_end_h**	RW	0	Ending column of video region
15:14	-	-	-	Reserved
13:0	**video_end_v**	RW	0	Ending line of video region

Table 7-20. VOUTM_video_end.

7.7.6 VOUTM_osd_start

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**osd_start_h**	RW	0	Starting column of OSD region
15:14	-	-	-	Reserved
13:0	**osd_start_v**	RW	0	Starting line of OSD region

Table 7-21. VOUTM_osd_start.

7.7.7 VOUTM_osd_end

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**osd_end_h**	RW	0	Ending column of OSD region
15:14	-	-	-	Reserved
13:0	**osd_end_v**	RW	0	Ending line of OSD region

Table 7-22. VOUTM_osd_end.

7.7.8 VOUTM_dummy_access

This register is added as a safety mechanism to detect common a cause failure in the mixer reset. SW can read/ write/reset this 32-bit register to test whether the register has been successfully reset, or whether a reset problem exists.

For Mekra Lang only Confidential 1.5 (Oct 2025)

7.7.9 VOUTM_osd_background

Bits	Name	Access	Reset	Description
31:24	-	-	-	Reserved
23:16	**background_y**	RW	0	Background Y value
15:8	**background_u**	RW	0	Background U value
7:0	**background_v**	RW	0	Background V value

Table 7-23. VOUTM_osd_background.

Note:

To enable users to see the highlights in an image, this register supports replacing any pixel with a Luma value higher than the Highlight Threshold value with a pre-defined color.

7.7.10 VOUTM_threshold

Bits	Name	Access	Reset	Description
31:24	**highlight_thresh-old**	RW	255	Highlight threshold
23:16	**highlight_y**	RW	0	Highlight Y value
15:8	**highlight_u**	RW	0	Highlight U value
7:0	**highlight_v**	RW	0	Highlight V value

Table 7-24. VOUTM_threshold.

7.7.11 VOUTM_osd_control

Whether loaded directly from memory or stored in the CLUT, the YUV components might have been multiplied by alpha in advance. If multiplication has been performed, the VOUTM_osd_control register osd_premultiplied field must be set to 1.

The OSD also supports a global blend feature. If the alpha value of any pixel is less than the global blend value it will be replaced by the global blend value. This enables users to fade out the OSD window without reprogramming the CLUT or the alpha values in memory.

However, the global blend feature cannot be used when the OSD data is premultiplied. In this case, the osd_global_blend field should be set to 8'hFF. The OSD data can be YUV or RGB regardless of the final output color space.

Note that the mixing operation can occur in the YUV or RGB color spaces. To facilitate this function, either the OSD or video data (but not both) can be converted to a different color space before mixing.

Bits	Name	Access	Reset	Description
31:16	**osd_transpar-ent_color**	RW	0	16-bit value corresponding to the transparent color
15	**enable_transpar-ent_color**	RW	0	If 1, replace any OSD pixel matching the OSD transparent color value with a transparent pixel (only for 16-bit mode)

For Wenta Lan Sony (08012025)
Confidential

Bits	Name	Access	Reset	Description
14	**osd_unpack_ swap_bytes**	RW	0	0: OSD Data is [15:0] 1: OSD Data is {[7:0], [15:8], [23:16],[31:24]} Compatible with OpenGL UNPACK_SWAP_BYTES option
13:9	**osd_mode**	RW	0	The OSD data format in 16-bit mode. 0: 5:6:5 (VYU) or (RGB) 1: 5:6:5 (UYV) or (BGR) 2: 4:4:4:4 (AYUV) 3: 4:4:4:4 (RGBA) 4: 4:4:4:4 (BGRA) 5: 4:4:4:4 (ABGR) 6: 4:4:4:4 (ARGB) 7: 1:5:5:5 (AYUV) 8: 1:5:5:5 (MSB ignored, YUV) 9: 5:5:5:1 (RGBA) 10: 5:5:5:1 (BGRA) 11: 1:5:5:5 (ABGR) 12: 1:5:5:5 (ARGB) The OSD data format in 32-bit mode. 27: 8:8:8:8 (AYUV) 28: 8:8:8:8 (RGBA) 29: 8:8:8:8 (BGRA) 30: 8:8:8:8 (ABGR) 31: 8:8:8:8 (ARGB) Other values reserved—must be set in conjunction with OSD In Enable field, ignored in OSD Mapped mode (8-bit)
8	**osd_premultiplied**	RW	0	0: OSD data not premultiplied by alpha 1: OSD data premultiplied by software
7:0	**osd_global_blend**	RW	255	OSD global blend value

Table 7-25. VOUTM_osd_control.

7.7.12 VOUTM Color Space Conversion Programming

The mixing operation can take place in the YUV or RGB color spaces. To facilitate this either the OSD or video data (but not both) can be converted to a different color space before mixing. The conversion equations include:

- Output 0 (Y/G) = clamp(((C0*A0 + C1*A1 + C2*A2 + rounding) >> (fractional bits)) + B0)
- Output 1 (Cb/B) = clamp(((C0*A3 + C1*A4 + C2*A5 + rounding) >> (fractional bits)) + B1)
- Output 2 (Cr/R) = clamp(((C0*A6 + C1*A7 + C2*A8 + rounding) >> (fractional bits)) + B2)

7.7.13 VOUTM_csc_param_0/1/2/3

Bits	Name	Access	Reset	Description
31:29	-	-	-	Reserved
28:16	**coefficient_a1/a3/ a5/a7**	W	NR	Signed 2.10 bits
15:13	-	-	-	Reserved

For Mentor Internal Use Only 1.5 (08012025)

Bits	Name	Access	Reset	Description
12:0	**coefficient_a0/a2/ a4/a6**	W	NR	Signed 2.10 bits

Table 7-26. VOUTM_csc_param_0/1/2/3.

7.7.14 VOUTM_csc_param_4

Bits	Name	Access	Reset	Description
31:27	-	-	-	Reserved
26:16	**constant_b0**	W	NR	Signed 10 bits
15:13	-	-	-	Reserved
12:0	**coefficient_a8**	W	NR	Signed 2.10 bits

Table 7-27. VOUTM_csc_param_4.

7.7.15 VOUTM_csc_param_5

Bits	Name	Access	Reset	Description
31:27	-	-	-	Reserved
26:16	**constant_b2**	W	NR	Signed 10 bits
15:11	-	-	-	Reserved
10:0	**constant_b1**	W	NR	Signed 10 bits

Table 7-28. VOUTM_csc_param_5.

7.7.16 VOUTM_csc_param_6/7/8

Bits	Name	Access	Reset	Description
31:24	-	-	-	Reserved
23:16	**output_0/1/2_ clamp_high**	W	NR	The maximum allowed output value for output 0/1/2 (Y, U, V) Anything greater will be clamped to this value
15:8	-	-	-	Reserved
7:0	**output_0/1/2_ clamp_low**	W	NR	The minimum allowed output value for output 0/1/2 (Y, U, V). Anything less will be clamped to this value

Table 7-29. VOUTM_csc_param_6/7/8.

7.7.17 VOUTM_Chroma_Vertical_Resampler Configuration

The chroma vertical resampler is automatically enabled when the chroma input mode is set to 4:2:0. It is not used for 4:2:2 mode.

Confidential For Mentor Lang only 10801025

7.7.17.1 m_vertical_upsampler_control

Bits	Name	Access	Reset	Description
31:1	**Reserved**	R	0	
0	**Vertical shift**	RW	0	0: Vertical coefficients are S0.7 1: Vertical coefficients are S1.6

Table 7-30. m_vertical_upsampler_control.

7.7.17.2 m_vertical_upsampler_output_height

Bits	Name	Access	Reset	Description
31:14	-	-	-	Reserved
13:0	**Vertical output height**	RW		Number of lines per frame—1

Table 7-31. m_vertical_upsampler_output_height.

7.7.17.3 m_vertical_upsampler_phase_inc

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:0	**Vertical phase inc**	RW		Vertical phase increment

Table 7-32. m_vertical_upsampler_phase_inc.

7.7.17.4 m_vertical_upsampler_phase_init

Bits	Name	Access	Reset	Description
31:28	-	-	-	Reserved
27:0	**Vertical phase init**	RW		Vertical initial phase

Table 7-33. m_vertical_upsampler_phase_init.

7.7.17.5 m_vertical_upsampler_dither_settings

Bits	Name	Access	Reset	Description
31:4	-	-	-	Reserved
3:2	**Vertical dither format**	RW	0	0: Dither term is 0.4 1: Dither term is s+0.4 2: Dither term is s+1.4 3: Dither term is s+2.4
1	-	-	-	Reserved

Confidential
For Melee Lang only
(2012025)

Bits	Name	Access	Reset	Description
0	**Vertical dither enable**	RW	0	1: Enable dithering 0: No dithering

Table 7-34. m_vertical_upsampler_dither_settings.

7.7.17.6 m_vertical_upsampler_dither_seed

Bits	Name	Access	Reset	Description
31	**Vertical dither seed write enable**	RW	0	Enable dither seed write
30:0	**Vertical dither seed**	RW	0	Random seed for dithering

Table 7-35. m_vertical_upsampler_dither_seed.

7.7.17.7 m_vertical_upsampler_coefficients 0/1/2/3/4...

Bits	Name	Access	Reset	Description
31:24	**Coefficient 3**	W		Signed 1.6 bits or signed 0.7 bits
23:16	**Coefficient 2**	W		Signed 1.6 bits or signed 0.7 bits
15:8	**Coefficient 1**	W		Signed 1.6 bits or signed 0.7 bits
7:0	**Coefficient 0**	W		Signed 1.6 bits or signed 0.7 bits

Table 7-36. m_vertical_upsampler_coefficients 0/1/2/3/4....

7.7.18 VOUTM Input / Output Configuration

The input to the Mixer section always comes from SMEM. There are three input streams for video luma, video chroma, and OSD data. The OSD data can be configured to load an 8-bit CLUT index (mapped mode), a 16-bit pixel value, or a 32-bit pixel value (direct mode).

There are two output streams, luma and chroma, as well as the direct output to the Display section. At least one output must be enabled; both can be used together.

7.7.18.1 m_stream_enables

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**Drive Display Enable**	RW	0	0: Disabled (do not drive display section) 1: Drive display section
3	**Mixer Out Enable**	RW	0	0: Disabled (no output to memory) 1: Output result to memory

Confidential For Meeting only (Oct 2025)

Bits	Name	Access	Reset	Description
2:1	**OSD In Enable**	RW	0	0: Disabled 1: Mapped in enabled 2: 16-bit Direct in enabled 3: 32-bit Direct in enabled
0	**Mixer In Enable**	RW	0	0: Disabled (no video) 1: Video enabled

Table 7-37. m_stream_enables.

7.7.18.2 Synchronization Counters

The Mixer section includes access to three synchronization counters: input, output, and timing. When enabled, the input counter performs a receive before any data is fetched from SMEM. The amount of data fetched is determined by the Input Video Sync Enable and Input Video Sync Line Count fields.

In frame synchronization mode, the entire frame is fetched without any further receives. In line synchronization mode, the number of lines indicated by the Input Video Sync Line Count field is fetched. In either case, after the data has been loaded, a decrement is performed if the Input Video Sync Count Decrement field is true. Then, if there is still more data (line sync mode), the process repeats, starting with the receive.

When the Mixer section processes 4:2:0 input data, the chroma becomes skewed relative to luma by passing through the polyphase vertical upsampler; this is due to the initial phase in chroma vertical upsampler. In this case, the field Input Video Sync Skew must be programmed correctly to avoid deadlock. The product of value set, N, and the 2**Input Video Sync Line Count must be greater than or equal to four lines of skew. Note that when the input video uses frame sync instead of line sync, this field can be left as 0 and will be ignored.

One output sync counter is tied to the SMEM output streams, enabling the user to perform line synchronization on the SMEM output streams. If enabled, a send is performed on this counter each time the number of lines specified by the Output Line Sync Line Count field are stored to SMEM. After a wake is received, the next store is initiated.

Finally, one counter performs a send when the current frame has finished processing. The frame sync counter sends only when the Mixer section has completed the current frame and is available to be reprogrammed for the next frame.

7.7.18.3 m_input_sync_counter_control

Bits	Name	Access	Reset	Description
31:17	**Reserved**	R	0	-
16:14	**Input Video Sync Skew**	RW	0	0: No skew allowed (4:2:2 input mode) N: N* 2 **(Input Video Sync Line Count -1) lines of sync allowed
13	**Input Video Sync Decrement**	RW	0	0: No decrements are performed 1: Perform decrement after loading N lines (line sync mode) or frame (frame sync mode)
12:5	**Input Video Sync Counter Number**	RW	0	Sync counter number for input video sync
4:2	**Input Video Sync Line Count**	RW	0	Number of lines to sync in line sync mode Is 2**(value of this register)

Bits	Name	Access	Reset	Description
1:0	**Input Video Sync Enable**	RW	0	0: Disabled 1: Line synchronization 2: Frame synchronization 3: Reserved

Table 7-38. m_input_sync_counter_control.

7.7.18.4 m_output_sync_counter_control

Bits	Name	Access	Reset	Description
31:22	-	-	-	Reserved
21:14	**Output Frame Sync Counter Number**	RW	0	Sync counter number for output sync
13	**Output Frame Sync Enable**	RW	0	0: Sync counter not used 1: Send on frame sync counter after input DMA completion
12:5	**Output Line Sync Counter Number**	RW	0	Sync counter number for output line sync
4:2	**Output Line Sync Line Count**	RW	0	Number of lines to sync is 2**(value of this register)
1	**Reserved**	R	0	
0	**Output Line Sync Enable**	RW	0	0: Sync counter not used 1: Perform line synchronization on output streams

Table 7-39. m_output_sync_counter_control.

7.7.18.5 SMEM Stream Control

Each Mixer section SMEM stream can be linked to any of SMEM's channels by setting the appropriate SMEM Register Number. The VOUT's internal interface to SMEM enables all the SMEM streams in VOUT access to SMEM on a priority round-robin basis.

The Input Internal Request Priority controls the priority of the requests made by this stream. In general, because the Display section is required to output data in real-time, no other streams in VOUT should be set to a priority higher than that assigned to the Display section input streams.

7.7.18.6 m_stream_in_control0

Bits	Name	Access	Reset	Description
31:37	-	-	-	Reserved
26:18	**OSD in SMEM register**	RW	0	SMEM register number for OSD in
17:9	**Video chroma in SMEM register**	RW	0	SMEM register number for chroma video in

For Mekro Lang only 15-10801203 Confidential

Bits	Name	Access	Reset	Description
8:0	**Video luma in SMEM register**	RW	0	SMEM register number for luma video in

Table 7-40. m_stream_in_control0.

7.7.18.7 m_stream_in_control1

Bits	Name	Access	Reset	Description
31:37	-	-	-	Reserved
5:4	**Osd_input_ length_lsbs**	RW	0	Must be programmed to (length-1) % 4
3:2	**Video_input_ length_lsbs**	RW	0	Must be programmed to (length-1) % 4
1:0	**Input_internal_ priority**	RW	0	Determines the priority of the input streams for making requests to SMEM. 0 is the lowest.

Table 7-41. m_stream_in_control1.

7.7.18.8 m_stream_out_control

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
19:18	**Output internal priority**	RW	0	Determines the priority of the input streams for making requests to SMEM. 0 is the lowest/
17:9	**Chroma out SMEM register**	RW	0	SMEM register number for chroma out
8:0	**Luma out SMEM register**	RW	0	SMEM register number for luma out

Table 7-42. m_stream_in_control0.

7.7.18.9 Frame Enable

When the Mixer section is fully configured and ready to output a frame of data, the Frame Enable bit should be set to 1.

When the frame has completed output and the Mixer section performs a send on the frame sync counter, the hardware automatically clears this bit to 0.

When the frame sync is received by the ORC thread controlling the Mixer section, ORC proceeds to update any registers it must before setting the Frame Enable to 1 for the next frame.

Confidential
For Mekronik use only
1.5 (Oct 2025)

7.7.18.10 VOUTM_clut

Bits	Name	Access	Reset	Description
31:24	**alpha_clut**	RW	NR	Value of alpha for color look up
23:16	**y_clut**	RW	NR	Value of Y/G for color look up
15:8	**cb_clut**	RW	NR	Value of Cb/B for color look up
7:0	**cr_clut**	RW	NR	Value of Cr/R for color look up

Table 7-43. VOUTM_clut.

7.8 VOUT: Display Section Registers—Map

The following section includes the register maps for Display A and Display B.

7.8.1 Display A Registers

Offset	Register	Description
0x300	**VOUTD_control**	Display section control
0x304	**VOUTD_status**	Status register
0x308	**VOUTD_frame_size**	Defines HV region for all outputs From display section
0x30C	**VOUTD_frame_height_field_1**	Defines frame height for bottom field (Ignored for progressive output formats)
0x310	**VOUTD_active_region_start_0**	Defines start of active region in top field Or progressive frame
0x314	**VOUTD_active_region_end_0**	Defines end of active region in top field Or progressive frame
0x318	**VOUTD_active_region_start_1**	Defines start of active region in bottom field (ignored for progressive output formats)
0x31C	**VOUTD_active_region_end_1**	Defines end of active region in bottom field (ignored for progressive output formats)
0x320	**VOUTD_background**	Color to display if enable deadline missed
0x324	**VOUTD_digital_output_mode**	Controls digital output options
0x328	**VOUTD_digital_hsync_control**	Controls HSync for digital output
0x32C	**VOUTD_digital_vsync_start_0**	Defines start of VSync in top field Or progressive frame
0x330	**VOUTD_digital_vsync_end_0**	Defines end of VSync in top field Or progressive frame
0x334	**VOUTD_digital_vsync_start_1**	Defines start of VSync in bottom field (Ignored for progressive output formats)
0x338	**VOUTD_digital_vsync_end_1**	Defines end of VSync in bottom field (Ignored for progressive output formats)
0x33C-0x340	**Reserved**	-
0x344	**VOUTD_MIPI_MAX_FRAME_COUNT**	For CSI-2 frame count output. 1, 2, 3, 4, , N-1, N
0x348-0x350	**Reserved**	-
0x354	**VOUTD_digital_csc_param_0**	Color-space-conversion setting for digital output
0x358	**VOUTD_digital_csc_param_1**	Color-space-conversion setting for digital output
0x35C	**VOUTD_digital_csc_param_2**	Color-space-conversion setting for digital output

For Mekra Confidential
Issue Date: 01/12/2018

Offset	Register	Description
0x360	VOUTD_digital_csc_param_3	Color-space-conversion setting for digital output
0x364	VOUTD_digital_csc_param_4	Color-space-conversion setting for digital output
0x368	VOUTD_digital_csc_param_5	Color-space-conversion setting for digital output
0x36C	VOUTD_digital_csc_param_6	Color-space-conversion setting for digital output
0x370	VOUTD_digital_csc_param_7	Color-space-conversion setting for digital output
0x374	VOUTD_digital_csc_param_8	Color-space-conversion setting for digital output
0x378	VOUTD_watchdog_timeout_value	Threshold value for vout display frame time
0x37C	VOUTD_luma_stream_checksum-VOUTD_dummy_access	Read only register for frozen image detection-SW core clock domain reset test register
0x380	VOUTD_luma_stream_checksum-VOUTD_dummy_access_clk_vo	Read only register for frozen image detection-SW CLK_VO domain reset test register
0x384	VOUTD_dummy_access_byte_clkVOUTD_luma_stream_checksum	SW MIPI BYTECLOCK domain reset test regis-terRead only register for frozen image detection
0x388 - 0x450	Reserved	-
0x454	VOUTD_digital_dither_settings	Dithering settings for digital output
0x458	VOUTD_digital_dither_seed	Dithering seed
0x45C	VOUTD_vout_vout_sync	Controls output of VOUT-VOUT SYNC signal
0x460	VOUTD_mipi_hbp_sync	Defines HBP and HSync blankings
0x464	VOUTD_mipi_hfp_vb	Defines HFP and vertical blankings
0x480	VOUTD_input_config	Defines input source and color resolution
0x484	VOUTD_input_sync_ctrl	Defines input SYNC parameters
0x488	VOUTD_output_sync_ctrl	Defines output SYNC parameters
0x48C	VOUTD_stream_ctrl	Defines input SMEM channel settings
0x490	VOUTD_force_background	Defines VOUT background debug mode
0x498	VOUTD_frame_enable	Signal to start an active frame
0x49C	VOUTD_luma_stream_checksum	Read only register for frozen image detection
0x4A0	VOUTD_chroma_stream_checksum	Read only register for frozen image detection

Table 7-44. Display A Registers Map.

7.8.2 DSI Command Registers and Gamma Table

Offset	Register	Description
0xB00	VOUTD_mipi_cmd_param	Defines MIPI command header and data type
0xB04	VOUTD_mipi_cmd_long_word_0	First word of long MIPI command
0xB08	VOUTD_mipi_cmd_long_word_1	Second word of long MIPI command
0xB0C	VOUTD_mipi_cmd_long_word_2	Third word of long MIPI command
0xB10	VOUTD_mipi_cmd_long_word_3	Fourth word of long MIPI command
0xB14	VOUTD_mipi_cmd_ctrl	Defines MIPI command type and mode
0x1000 - 0x13FC	VOUTD_gamma_green	Green or luma gamma table
0x1400 - 0x17FC	VOUTD_gamma_blue	Blue or CB gamma table
0x1800 - 0x1BFC	VOUTD_gamma_red	Red or CR gamma table

Table 7-45. DSI Command Registers and Gamma Table.

For Confidential Training only
Ambarella Oct 2025

7.8.3 Display B Registers

Offset	Register	Description
0x600	**VOUTD_control**	Display section control
0x604	**VOUTD_status**	Status register
0x608	**VOUTD_frame_size**	Defines HV region for all outputs From display section
0x60C	**VOUTD_frame_height_field_1**	Defines frame height for bottom field (Ignored for progressive output formats)
0x610	**VOUTD_active_region_start_0**	Defines start of active region in top field or progressive frame
0x614	**VOUTD_active_region_end_0**	Defines end of active region in top field or progressive frame
0x618	**VOUTD_active_region_start_1**	Defines start of active region in bottom field (ignored for progressive output formats)
0x61C	**VOUTD_active_region_end_1**	Defines end of active region in bottom field (ignored for progressive output formats)
0x620	**VOUTD_background**	Color to display if enable deadline missed
0x624	**VOUTD_digital_output_mode**	Controls digital output options
0x628	**VOUTD_digital_hsync_control**	Controls HSync for digital output
0x62C	**VOUTD_digital_vsync_start_0**	Defines start of VSync in top field or progressive frame
0x630	**VOUTD_digital_vsync_end_0**	Defines end of VSync in top field or progressive frame
0x634	**VOUTD_digital_vsync_start_1**	Defines start of VSync in bottom field (Ignored for progressive output formats)
0x638	**VOUTD_digital_vsync_end_1**	Defines end of VSync in bottom field (Ignored for progressive output formats)
0x63C-0x640	**Reserved**	-
0x644	**VOUTD_MIPI_MAX_FRAME_COUNT**	For CSI-2 frame count output. 1, 2, 3, 4, , N-1, n
0x648-0x650	**Reserved**	-
0x654	**VOUTD_digital_csc_param_0**	Color-space-conversion setting for digital output
0x658	**VOUTD_digital_csc_param_1**	Color-space-conversion setting for digital output
0x65C	**VOUTD_digital_csc_param_2**	Color-space-conversion setting for digital output
0x660	**VOUTD_digital_csc_param_3**	Color-space-conversion setting for digital output
0x664	**VOUTD_digital_csc_param_4**	Color-space-conversion setting for digital output
0x668	**VOUTD_digital_csc_param_5**	Color-space-conversion setting for digital output
0x66C	**VOUTD_digital_csc_param_6**	Color-space-conversion setting for digital output
0x670	**VOUTD_digital_csc_param_7**	Color-space-conversion setting for digital output
0x674	**VOUTD_digital_csc_param_8**	Color-space-conversion setting for digital output
0x678	**VOUTD_watchdog_timeout_value**	Threshold value for VOUT display frame time
0x67C	**VOUTD_dummy_access**	SW core clock domain reset test registerRead only register for frozen image detection
0x680	**VOUTD_dummy_access_clk_vo**	Read only register for frozen image detection-SW CLK_VO domain reset test register
0x684	**VOUTD_dummy_access_byte_clk**	SW MIPI BYTECLOCK domain reset test registerRead only register for frozen image detection
0x688 - 0x750	**Reserved**	-
0x754	**VOUTD_digital_dither_settings**	Dithering settings for digital output

Offset	Register	Description
0x758	**VOUTD_digital_dither_seed**	Dithering seed
0x75C	**VOUTD_vout_vout_sync**	Controls output of VOUT-VOUT SYNC signal
0x760	**VOUTD_mipi_hbp_sync**	Defines HBP and HSync blankings
0x764	**VOUTD_mipi_hfp_vb**	Defines HFP and vertical blankings
0x780	**VOUTD_input_config**	Defines input source and color resolution
0x784	**VOUTD_input_sync_ctrl**	Defines input SYNC parameters
0x788	**VOUTD_output_sync_ctrl**	Defines output SYNC parameters
0x78C	**VOUTD_stream_ctrl**	Defines input SMEM channel settings
0x790	**VOUTD_force_background**	Defines VOUT background debug mode
0x798	**VOUTD_frame_enable**	Signal to start an active frame
0x79C	**VOUTD_luma_input_checksum**	Read only register for frozen image detection
0x7A0	**VOUTD_chroma_input_checksum**	Read only register for frozen image detection

Table 7-46. Display B Registers Map.

7.8.4 DSI Command Registers and Gamma Table

Offset	Register	Description
0xE00	**VOUTD_mipi_cmd_param**	Defines MIPI command header and data type
0xE04	**VOUTD_mipi_cmd_long_word_0**	First word of long MIPI command
0xE08	**VOUTD_mipi_cmd_long_word_1**	Second word of long MIPI command
0xE0C	**VOUTD_mipi_cmd_long_word_2**	Third word of long MIPI command
0xE10	**VOUTD_mipi_cmd_long_word_3**	Fourth word of long MIPI command
0xE14	**VOUTD_mipi_cmd_ctrl**	Defines MIPI command type and mode
0x1C00 - 0x1FFC	**VOUTD_gamma_green**	Green or luma gamma table
0x2000 - 0x23FC	**VOUTD_gamma_blue**	Blue or CB gamma table
0x2400 - 0x27FC	**VOUTD_gamma_red**	Red or CR gamma table

Table 7-47. DSI Command Registers and Gamma Table.

7.9 VOUT: Display Section Registers—Detail

CV22FS includes two identical display sections, connected to two DSI PHY. The input data originates from either the mixer section or SMEM. After an ASYNC FIFO, the data is synced from the core clock domain to the clk_vo domain. Next, optional basic operations are performed on the data, such as 4:2:2 to 4:4:4 horizontal upsampling, color space conversion, and gamma table. The data is then reordered for the desired output format and sent to either the DSI or CSI-2 link layer logic. After the DSI and CSI-2 link layers, MIPI data passes through a shared MIPI PHY interface layer, and is then passed to the VOUT PHY.

The following figure displays the VOUT data flow.

For Ambarella Confidential only (Version 1.0.1 01/21/2025)

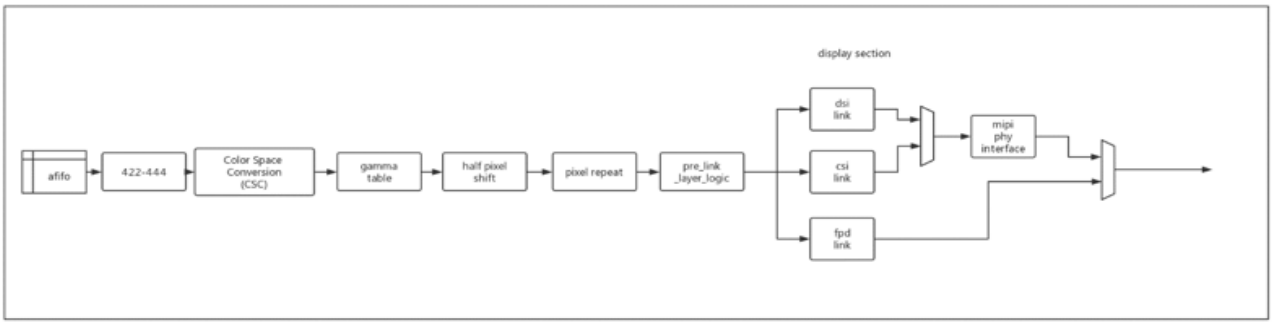


Figure 7-2. VOUT Data Flow.

7.9.1 VOUTD_control

Bits	Name	Access	Reset	Description
31	**software_reset**	RW	0	Software Reset Setting of 1 resets the Display section
30:28	-	-	-	Reserved
27	**digital_output_enable**	RW	0	Digital Output Enable 0: Disable digital output 1: Enable digital output (DSI, and CSI-2 require “1”)
26	**vin_vout_sync_enable**	RW	0	VIN-VOUT SYNC Enable 0: Disable VIN-VOUT Sync 1: Enable VIN-VOUT Sync
25	**vout_vout_sync_enable**	RW	0	VOUT-VOUT SYNC Enable 0: Disable VOUT-VOUT SYNC 1: Enable VOUT-VOUT SYNC
24	-	-	-	Reserved
23:22	**mipi_lane_num**	RW	0	Number of MIPI Lanes 0: 1 lane 1: 2 lanes 3: 4 lanes Note: 3 lanes is not supported
21	**mipi_eotp_en**	RW	0	MIPI DSI EoTP Enable 0: Disable MIPI DSI EoTP 1: Enable MIPI DSI EoTP
20	**mipi_sync_end_en**	RW	0	MIPI SYNC End Enable 0: Disable MIPI SYNC End 1: Enable MIPI SYNC End
19	**Select_csi**	RW	0	0: Not CSI-2 frame 1: CSI-2 frame
18:10	-	-	-	Reserved
9	**half_pixel_shift_even_lines**	RW	0	Half Pixel Shift Even Lines (Display A only) 0: Apply half-pixel shift to odd lines 1: Apply half-pixel shift to even lines
8	**half_pixel_shift_enable**	RW	0	Half Pixel Shift Enable (Display A only) 0: Half-pixel shift disabled 1: Half-pixel shift enabled

Bits	Name	Access	Reset	Description
7	**gamma_mapping_enable**	RW	0	Gamma Mapping Enable (Display A only) 0: Gamma mapping block disabled 1: Gamma mapping block enabled
6	**reverse_mode_enable**	RW	0	Reverse Mode Enable 0: Normal mode 1: Video data is horizontally reversed
5	**interlace_enable**	RW	0	Set to 0 No interlace format
4:0	**fixed_format_select**	RW	0	Fixed Format Select 0 Fixed Format Select is disabled 1: 480i60 2: 480p60 3: 576i50 4: 576p50 5: 720p60 7: 1080i60 8: 1080i50 9: 1080i48/1080psf24 10: 1080p60 11: 1080p50 12: 1080p24 Other values reserved

Table 7-48. VOUTD_control.

7.9.2 VOUTD_status

Bits	Name	Access	Reset	Description
31	**reset_complete**	R_WC	1	Reset Complete This bit is set to 1 when a reset has completed and the section is ready to be reprogrammed
30	-	-	-	Reserved
29	**digital_underflow**	R_WC	0	Digital Underflow 1 if an underflow is detected on the digital output
28:7	-	-	-	Reserved
6	**Watchdog_timeout**	R_WC	0	1 indicates one frame takes longer to finish than the programmed watchdog timer threshold; possible hang can occur
5	**mipi_fifo_overflow**	R_WC	0	1 indicates MIPI FIFO overflow has occurred
4	**gamma_configuration_ready**	R_WC	0	Gamma Configuration Ready Display 0 Only – When 1, software can write a block of eight addresses in the Gamma table space. Software must then wait for bit to be 1 again before the next write
3:0	-	-	-	Reserved

Table 7-49. VOUTD_status.

With the Status Register, writing a 1 clears bits with R_WC access. Bits with R access are updated continuously and cannot be cleared by software.

For Mekra Lang only
1.5 (08/12/2015)
Confidential

7.9.3 VOUTD_frame_size

The Display section can output a frame of any size up to 16,384 clock cycles wide by 16,384 lines high. In interlaced mode, the line lengths of both fields are the same, but the field heights can differ by an arbitrary amount, generally +/- 1 line.

The active region determines the area where valid pixel data is displayed. Pixel data outside of the active region is generally not guaranteed to have any particular value. Therefore, this region should cover the intended display's entire visible output region. Additionally, the active region is not allowed to occupy either the first or last lines of a frame vertically. Horizontally, there must be a minimum of eight cycles of blanking either at the beginning or the end of the line.

The horizontal size of the active region is always programmed in clock cycles, not pixels. Therefore, if outputting each pixel takes more than one clock cycle, the size of the active region must be multiplied by this factor. The active region should contain an even number of pixels horizontally. In CV22FS, all formats are 1 pixel per cycle.

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**frame_width**	RW	0	Frame Width Number of clock cycles per line (including all blanking times). Set to n-1 where n is the desired number of clock cycles (counter starts from 0)
15:14	-	-	-	Reserved
13:0	**frame_height_field_0**	RW	0	Frame Height Field 0 Number of lines per frame (including all blanking times). Set to n-1 where n is the desired number of lines (counter starts at 0)

Table 7-50. VOUTD_frame_size.

7.9.4 VOUTD_frame_height_field1

Bits	Name	Access	Reset	Description
31:14	-	-	-	Reserved
13:0	**frame_height_field_1**	RW	0	Frame Height Field 1 Number of lines per frame (including all blanking times). Set to n-1 where n is the desired number of lines (counter starts at 0). Ignored when output is progressive

Table 7-51. VOUTD_frame_height_field1.

7.9.5 VOUTD_active_region_start_end_0/1

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**active_region_start_row/column**	RW	0	Active Region Start Row / Column The active region will start on this row/column
15:14	-	-	-	Reserved

Bits	Name	Access	Reset	Description
13:0	**active_region_ end_row/column**	RW	0	Active Region End Row / Column The active region will end on this row/column

Table 7-52. VOUTD_active_region_start_end_0/1.

Note:

All field1 registers are not used.

7.9.6 VOUTD_background

The background color of the active region is controlled in the Mixer section. If Frame Enable is not programmed before the deadline, the Display section outputs the following color (listed in the table below) during the active region of the missed frame.

Bits	Name	Access	Reset	Description
31:24	-	-	-	Reserved
23:16	**Y**	W	NR	Value of Y component for background color
15:8	**Cb**	W	NR	Value of CB component for background color
7:0	**Cr**	W	NR	Value of CR component for background color

Table 7-53. VOUTD_background.

7.9.7 VOUTD_digital_output_mode

Only DSI and CSI-2 formats are allowed. 9-14 are MIPI formats—when **select_csi** is 1, the format is MIPI CSI-2; when 0, the format is MIPI DSI. To send out a stream through MIPI CSI-2 without affecting data, use mode 9, and put the data in display luma stream.

Note:

Mode 15 is not supported. To avoid an unintentional safety error resulting in chip failure under this mode, users should disable the VOUT safety error in AMBA_ECRU by setting error_mask_1[21] to 1. For further details, refer to Chapter 30 “AMBA ECRU”.

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:27	**digital_output_ mode**	RW	0	Digital Output Mode 0–8 Reserved 9: MIPI—RAW (only luma stream is sent out) 10: MIPI—RGB-8:8:8-24BIT (for DSI only) 11: MIPI—YUV-4:2:2-16BIT 12: MIPI—5:6:5-16BIT (for DSI only) 13: MIPI—6:6:6-18BIT UNPACKED (for DSI only) 14–15 Reserved

For Mekra Lang only (08.2023) Confidential

Bits	Name	Access	Reset	Description
26:24	**color_sequence_odd_lines**	RW	0	Color Sequence Odd Lines Sets the color output sequence for the odd lines of the frame. Different interpretations depending on DIGITAL_OUTPUT_MODE. In Digital Output Mode 8: 0 - R->G->R->G... 2 - G->R->G->R... 3 - G->B->G->B... 5 - B->G->B->R... Other values reserved
23:21	**color_sequence_even_lines**	RW	0	Color Sequence Even Lines Identical to COLOR_SEQUENCE_ODD_LINES except used on even output lines (0, 2, 4 etc)
20	-	-	-	Reserved
19*	**digital_hvld_polarity**	RW	1	Polarity of Digital Line Valid Signal (external pin VD1_HVLD) 0: VD1_HVLD asserted low 1: VD1_HVLD asserted high
18	**mipi_ecc_order**	RW	0	Set to 0
17	**mipi_line_timing_all**	RW		Always set to 1 in DSI mode 0: Disable line SYNC start and end in Vblank 1: Enable line SYNC start and end in Vblank ()
16	**mipi_send_blank_lines**	RW		MIPI VBlank Output , always set to 0 in DSI mode 0: Disable MIPI output packet in Vblank 1: Enable MIPI output packet in Vblank
15	**mipi_frame_count**	RW		MIPI Frame Count, always to 0 in DSI 0: Disable MIPI frame count 1: Enable MIPI frame count
14	**mipi_line_count**	RW		MIPI Line Count, always to 0 in DSI 0: Disable MIPI line count 1: Enable MIPI line count
13	**mipi_line_timing**	RW		MIPI Line SYNC Enable , set to 1 in DSI 0: Disable line SYNC packets 1: Enable line SYNC packets
12:2	-	-	-	Reserved
1*	**digital_vsync_polarity**	RW	0	Digital VSync Polarity 0: VSync asserted low 1: VSync asserted high
0*	**digital_hsync_polarity**	RW	0	Digital HSync Polarity 0: HSync asserted low 1: HSync asserted high

Table 7-54. VOUTD_background.

Note:

Used to extend flexibility but can be removed if not required for use case.

7.9.8 VOUTD Hsync_control

The horizontal and vertical output synchronization signals (HSync and VSync) can be programmed for any duration. In the interlaced output modes, the HSync is the same for both fields, but VSync is programmed separately

for each field.

The polarity of each signal is controlled in the Digital Output Mode register **VOUTD_digital_output_ mode**. Avoid setting the start location to begin after the end location. The VSync pulse can span as many lines as needed. The sync signals can have different timings for each output path.

The Display section includes one active region only—when running multiple output paths at the same time, it can be beneficial to locate this region in different places within the frame for the different output paths. This method can be achieved by changing the HSync / VSync timings for the different output paths.

HSync and VSync should be delayed by the difference in effective location of the active region.

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**hsync_start_column**	RW	0	HSync Start Column HSync will be asserted on this column
15:14	-	-	-	Reserved
13:0	**hsync_end_column**	RW	0	HSync End Column HSync will be deasserted on this column

Table 7-55. VOUTD hsync_control.

7.9.9 VOUTD Vsync_start_end_0/1

Bits	Name	Access	Reset	Description
31:30	-	-	-	Reserved
29:16	**vsync_start_row/column**	RW	0	VSync Start Row/Column VSync will be asserted on this row/column
15:14	-	-	-	Reserved
13:0	**vsync_end_row/column**	RW	0	VSync End Row/Column VSync will be deasserted on this row/column

Table 7-56. VOUTD vsync_start_end_0/1.

Note:

CV22FS does not currently include field1 registers.

7.9.10 VOUTD_digital_csc_param_0/1/2/3

The input data to the Display section is always in a YUV 4:2:2 format. However, the output can be in a different YUV color space or in an RGB color space. The Color Space Converter (CSC) converts the input to the desired output color format. Note that the decimal point can be at different bit positions in different modes; clamping values also depend on the mode.

Bits	Name	Access	Reset	Description
31:29	-	-	-	Reserved
28:16	**coefficient_a1/a3/a5/a7**	W	NR	Coefficient a1 / a3 / a5 / a7 Signed 2.10 bits for all output mode
15:13	-	-	-	Reserved

For Mekra Lang only
1.5 (2015-05-25)
Confidential

Bits	Name	Access	Reset	Description
12:0	**coefficient_a0/a2/ a4/a6**	W	NR	Coefficient a0 / a2 / a4 / a6 Signed 2.10 bits for all output mode

Table 7-57. VOUTD_digital_csc_param_0/1/2/3.

7.9.11 VOUTD_digital_csc_param_4

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:16	**constant_b0**	W	NR	Constant b0 Signed 10 bits for all output mode
15:13	-	-	-	Reserved
12:0	**coefficient_a8**	W	NR	Coefficient a8 Signed 2.10 bits for all output mode

Table 7-58. VOUTD_digital_csc_param_4.

7.9.12 VOUTD_digital_csc_param_5

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:16	**constant_b2**	W	NR	Constant b2 Signed 10 bits for all output mode
15	-	-	-	Reserved
14:0	**constant_b1**	W	NR	Constant b1 Signed 10 bits for all output mode

Table 7-59. VOUTD_digital_csc_param_5.

7.9.13 VOUTD_digital_csc_param_6/7/8

Bits	Name	Access	Reset	Description
31:28	-	-	-	Reserved
27:16	**output_0/1/2_ clamp_high**	W	NR	Output 0 / 1 / 2 Clamp High The maximum allowed output value for output 0 / 1 / 2. Anything greater will be clamped to this value. For digital output this value should be <= 0xFF.
15:12	-	-	-	Reserved
11:0	**output_0/1/2_ clamp_low**	W	NR	Output 0 / 1 / 2 Clamp Low The minimum allowed output value for output 0 / 1 / 2. Anything less will be clamped to this value.

Table 7-60. VOUTD_digital_csc_param_6/7/8.

For Mekrang only
1.5 (Oct 2025)
Confidential

7.9.14 VOUTD_digital_dither_settings

Dithering can be used on the digital output for formats with resolution less than 8-bits / component. Dithering is legal only when the output format is less than 8-bits / component (5:6:5); enabling dithering for other output formats is illegal.

Bits	Name	Access	Reset	Description
31:3	-	-	-	Reserved
2:1	**dither_format**	RW	0	Dither Format 0: Dither term is 0.4 1: Dither term is s+0.4 2: Dither term is s+1.4 3: Dither term is s+2.4
0	**dither_enable**	RW	0	Dither Enable 0: Dither disabled (round only) 1: Dither enabled

Table 7-61. VOUTD_digital_dither_settings.

7.9.15 VOUTD_digital_dither_seed

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:0	**dither_seed**	RW	0	Dither seed

Table 7-62. VOUTD_digital_dither_seed.

7.9.16 VOUTD_vout_vout_sync

The two Display sections can be synchronized using this field and the **VOUTD_control** bit, v**out_vout_ sync_enable**. To avoid a system error, set only one enable bit to 1 in a display.

If Display 0 is to begin first, Display 1 should set **vout_vout_sync_enable** to 1 since it receives the sync signal from Display 0.

The following register allows software to delay the start of Display 1 relative to Display 0 by a fixed number of lines from 0 V.

Bits	Name	Access	Reset	Description
31:17	-	-	-	Reserved
16	**field_select**	RW	0	Field Select Field on which to start Display A / B. Must be set to 0 if in progressive output mode
15:14	-	-	-	Reserved
13:0	**display_a/b_start_row**	RW	0	Display A / B Start Row Row on which to start running Display A / B if VOUT-VOUT SYNC is enabled

Table 7-63. VOUTD_vout_vout_sync.

Confidential For Mekel Lang only 1.5 (08012023)

7.9.17 VOUTD Watchdog Timeout Value

This is a 32-bit threshold value to help determine the undesired VOUT frame-rate drop or hang. When VOUT does not receive the next zero_v sync before the core clock counts to the threshold value from the previous zero_v sync, VOUT reports the timeout value.

7.9.18 VOUTD force_background

This configuration is used to test DSI PHY without using SMEM or ORC and enables the continuous and independent display of the background color bar. Important for bring-up, users can program VOUT only through the AHB debug port without needing to drive VOUT programming for each frame.

Bits	Name	Access	Reset	Description
31:12	-	-	-	Reserved
11:2	**background bar width_m1**	RW	127	Width of each background color bar when background mode is set 1
1	**background mode**	RW	0	0: Background color 1: Background color bar
0	**force background**	RW	0	0: Enter normal mode 1: Enter background mode directly

Table 7-64. VOUTD force_background.

7.9.19 VOUTD Luma / Chrom Stream Checksum

These two checksum values are read only registers generated from the Display section input luma stream and chroma stream. The checksum generation function is the same as the function used in IDSP checksum generation. If users move the video output to VOUT, the checksum number should be different for each frame. Therefore, the software can use this register to detect a frozen image to VOUT.

7.9.20 VOUTD dummy_access_core/clk_vo/byte_clk

These three 32-bit registers are reset by the core domain reset, clk_vo domain reset, and byte clk domain reset. Because software has read / write access to these three registers, they can be used to detect a “stuck at common cause” failure in all three clock domains.

7.9.21 VOUTD MIPI DSI Programming

The Display_A section includes a 4-lane MIPI DSI output path, supporting a high speed transmission mode of data commands, and low-power mode transmission of commands based on the MIPI DSI specification.

In the MIPI DSI specification, VOUT is required to enter a low-power state at least once per frame, and is recommended to enter low-power state at each line. However, because the minimum transition time from HS->LP->HS required by MIPI DSI PHY varies between different panels, VOUT might not be able to enter a low-power state for each line, and might not be able to enter low-power mode per frame.

When VOUT cannot enter low-power mode between meaningful packets (data packets, sync packets, command packets), VOUT sends blanking packets to occupy the bytes and keep the DSI lane in high-speed mode.

For the packet sequence, refer to MIPI_DSI_SPECIFICATION V01-02-00 or contact an Ambarella support team member.

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30	**Dsi_sync_use_blank**	RW	0	At SYNC position (between SYNC start and SYNC end, ignored when send_sync == 0) 1: Use blanking packet 0: Do not use blanking; use low power
29:16	**Dsi_sync_wc**	RW	0	Number of bytes in the payload of SYNC blanking packet
15	-	-	-	Reserved
14	**Dsi_hbp_use_blank**	RW	0	At back porch during active lines: 1: Use blanking packet 2: Do not use blanking, use low power
13:0	**Dsi_hbp_wc**	RW	0	Number of bytes in the payload of HBP blanking packet

Table 7-65. VOUTD MIPI DSI Programming.

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30	**Dsi_vblank_use_blank**	RW	0	After SYNC events at Vblank lines: 1: Use blanking packet 0: Do not use blanking, use low power
29:16	**Dsi_vblank_wc**	RW	0	Number of bytes in the payload of Vblank blanking packet
15	-	-	-	Reserved
14	**Dsi_hfp_use_blank**	RW	0	At front porch during active lines: 1: Use blanking packet 2: Do not use blanking, use low power
13:0	**Dsi_hfp_wc**	RW	0	Number of bytes in the payload of HFP blanking packet

Table 7-66. VOUTD MIPI DSI Programming.

The hardware automatically sends out video data packets, sync packets (HSync, VSync), EOTP packet (send_eotp == 1), and blanking packets (when configured). In addition, using the configuration interface, VOUT can send out commands with less than 16 bytes(17 for DCS) payload in either high-speed speed mode or low-power mode.

Before setting the frame enable for **display_a** section, software can configure VOUT to send out no limit number of commands in either high-speed mode or low-power mode (set **mipi_command_timing** to 0). After display begins, two options are available for timing to send commands: when **command_timing**==0, VOUT sends out the command at the next available packet window; when **command_timing**== 1, VOUT sends the command at **command_timing_n** lines after the VSync begins.

After sending the command, VOUT generates the **command_complete** (bit 14 in VOUT special signals to ORC) pulse indicating that it is ready to accept the next command.

7.9.22 Mipi_command_header

Bits	Name	Access	Reset	Description
31:24	**Dsi_dcs_long_type**	RW	0	When sending DCS long command, this is the first byte (command type) in the payload
23:22	-	-	-	Reserved
21:16	**Dsi_command_di**	RW	0	The data identifier of the sending command
15:8	**Dsi_command_header_0**	RW	0	For short DCS command: command type hex For short other command: parameter 0 For long DCS command: WC_0
7:0	**Dsi_command_header_1**	RW	0	For short DCS command: parameters For short other command: parameter 1 For long DCS command: WC_1

Table 7-67. Mipi_command_header.

7.9.23 Mipi_command_para_0/1/2/3

Bits	Name	Access	Reset	Description
31:24	**Dsi_cmd_byte_3/7/11/15**	RW		Fourth byte of MIPI long command
23:16	**Dsi_cmd_byte_2/6/10/14**	RW		Third byte of MIPI long command
15:8	**Dsi_cmd_byte_1/5/9/13**	RW		Second byte of MIPI long command
7:0	**Dsi_cmd_byte_0/4/8/12**	RW		First byte of MIPI long command

Table 7-68. Mipi_command_para_0/1/2/3.

Notes:

- For short commands, the output packet is similar to the following:

  {2'd0, mipi_command_di, mipi_command_header_0, mipi_command_header_1, ECC};

- For normal long commands, the output packet is similar to the following:

  {2'd0, mipi_command_di, mipi_command_header_0, mipi_command_header_1, ECC, command_long_byte_0, command_long_byte_1,..., CHECKSUM}

- For DCS long commands, the output packet is similar to the following:

  {2'd0, mipi_command_di, mipi_command_header_0, mipi_command_header_1, ECC, mipi_dcs_long_type, command_long_type_1, command_long_type_1, …, CHECKSUM}

  The ECC byte and two CHECKSUM bytes are generated automatically by hardware logic.

7.9.24 Mipi_command_control

Bits	Name	Access	Reset	Description
31:22	-	-	-	Reserved

Confidential
For Media Lang only
1.5 / 08/12/2021

Bits	Name	Access	Reset	Description
21	**Dsi_command_bta**	RW	0	0: Normal command 1: After the command, PHY will start read back sequence
20	**Dsi_command_mode**	RW	0	0: Send command with low-power mode 1: Send command with high-speed mode
19:18	**Dsi_command_type**	RW	0	0: Short packet command 1: Normal long packet command 2: DCS long packet command
17:16	**Dsi_command_timing**	RW		0: Send the command at next available time 1: Send during Vblank, n lines after VSync start (n[15:0])
15:0	**Dsi_command_timing_n**	RW		When COMMAND_TIMING == 1, this number represents at how many lines after VSync, the command will be sent

Table 7-69. Mipi_command_control.

Note:

bit21 dsi_command_bta bit is added for read-back function. To issue a read command to receiver for a response from the receiver, set this bit to 1 in the read command configuration. With this bit set, VOUT drives **dsi_phy** to read-back state after the command is issued.

Refer to Section 7.14 “VOUT: BTA Programming Flow” for information about BTA packet response under DCS Write and DCS Read.

7.9.25 VOUTD_mipi_max_frame_count

A programmable **max_frame_count** for CSI-2 is used when **csi_frame_count** is enabled, and the **frame_count** in SOF / EOF packet would count from 1,2,…,N-1, N, 1, 2, …, N-1, N, 1…….

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:0	**Mipi_max_frame_count**	RW	0	Max frame count for CSI-2 frame counter

Table 7-70. VOUTD_mipi_max_frame_count.

7.9.26 VOUTD_input_config

Bits	Name	Access	Reset	Description
31:2	-	-	-	Reserved
1	**color_res**	RW		Input Color Resolution 0: YUV 422 input 1: YUV 444 or RGB input
0	**stream_en**	RW		SMEM Input Enable 0: Disable SMEM input, input from Mixer 1: Enable SMEM input

Table 7-71. VOUTD_input_config.

For Confidential Mekra Lang only 1.5 (2021025)

7.9.27 VOUTD_input_sync_ctrl

Bits	Name	Access	Reset	Description
31:24	-	-	-	Reserved
13	**Input_decrement**	RW	0	Input SYNC Count Decrement 0: No decrement 1: Decrement counter after N lines received in line SYNC mode or EOF received in frame SYNC mode
12:5	**Input_sync_counter**	RW	0	Input SYNC Counter Number
4:2	**Input_sync line_countr**	RW	0	Log2 of Lines to Sync in Line SYNC Mode
1:0	**Input_sync_enable**	RW	0	Input SYNC Enable 0: Disable input SYNC 1: Enable input SYNC

Table 7-72. VOUTD_input_sync_ctrl.

7.9.28 VOUTD_output_sync_ctrl

Bits	Name	Access	Reset	Description
31:18	-	-	-	Reserved
17:10	**output_0v_sync_counter**	RW	0	0v SYNC Counter Number
9	**Output_0v_sync_enable**	RW	0	0v SYNC Enable 0: Disable 0v SYNC 1: Enable 0v SYNC
8:1	**Output_frame_sync_cntr**	RW	0	Output Frame SYNC Counter Number
0	**Output_frame_sync_enable**	RW	0	Frame SYNC Enable 0: Disable frame SYNC 1: Enable frame SYNC

Table 7-73. VOUTD_output_sync_ctrl.

7.9.29 VOUTD_stream_ctrl

Bits	Name	Access	Reset	Description
31:24	-	-	-	Reserved
23:22	**chroma_line_lsb**	RW	0	Input Chroma Line Length LSBs Must be programmed to (length-1) % 4
21:20	**luma_line_lsb**	RW	0	Input Luma Line Length LSBs Must be programmed to (length-1) % 4
19:18	**priority**	RW	0	Internal SMEM Priority of the Input Streams 0 is lowest
17:9	**chroma_smem_channel**	RW	0	SMEM channel for chroma input

Confidential
For Marketing only
(Log 08/15/2015)

Bits	Name	Access	Reset	Description
8:0	**luma_smem_ channel**	RW	0	SMEM channel for chroma input

Table 7-74. VOUTD_stream_ctrl.

7.9.30 VOUTD_gamma_green/blue/red

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7:0	**output**	W	NR	Gamma output value

Table 7-75. VOUTD_gamma_green/blue/red.

7.10 VOUT: OSD Rescale Registers—Map

Offset	Register	Description
0x800	**O_rescale_control**	OSD rescale control
0x804	**O_rescale_width**	Output width
0x808	**O_rescale_horizontal_phase_ inc**	Horizontal phase increment
0x80C	**O_rescale_horizontal_phase_ init_even**	Horizontal initial phase for even lines
0x810	**O_rescale_horizontal_phase_ init_odd**	Horizontal initial phase for odd lines
0x814	**O_rescale_height**	Output height
0x818	**O_rescale_vertical_phase_inc**	Vertical phase increment
0x81C	**O_rescale_vertical_phase_init**	Vertical initial phase
0x820-0x89C	**O_rescale_horizontal_coeff0-31**	Horizontal coefficients
0x8A0-0x91C	**O_rescale_vertical_coeff0-31**	Vertical coefficients
0x920	**O_rescale_horizontal_dither_ control**	Horizontal dither settings
0x924	**O_rescale_horizontal_dither_ seed**	Horizontal dither seed
0x928	**O_rescale_vertical_dither_con- trol**	Vertical dither settings
0x92C	**O_rescale_vertical_dither_seed**	Vertical dither seed

Table 7-76. OSD Rescale Registers: Map.

7.11 VOUT OSD Rescale Registers—Detail

The OSD Rescale filter can be configured to operate in either Mixer 0 or Mixer 1, but not both. It is a six-tap, 16-phase resampling filter. Resampling can be performed independently in the horizontal and vertical directions; this enables a single OSD bitmap to be reused for multiple output resolutions, and avoid regeneration. The OSD Rescaler and the Mixer section should be programmed at the same time.

The maximum horizontal upsampling ratio allowed is 8x. The resampling is separated into vertical and horizontal passes with the vertical performed first. The maximum line length that can be input into the OSD Rescale is 1920

pixels. After the vertical pass, the horizontal filter can be used to upsample this to a larger horizontal output size such as 3840 or 4096 pixels.

7.11.1 O_rescale_control

Bits	Name	Access	Reset	Description
31:6	-	-	-	Reserved
5:4	**input_select**	RW	0	Input select 0: Mixer A CLUT 1: Mixer A Direct 2: Mixer B CLUT 3: Mixer B Direct
3	**horizontal_shift**	RW	0	Horizontal shift 0: Horizontal coefficients are S0.7 1: Horizontal coefficients are S1.6
2	-	-	-	Reserved
1	**vertical_shift**	RW	0	Vertical shift 0: Coefficients are S0.7 1: Vertical coefficients are S1.6
0	**enable**	RW	0	Enable Enables both horizontal and vertical resampling

Table 7-77. O_rescale_control.

7.11.2 O_rescale_width

Bits	Name	Access	Reset	Description
31:14	-	-	-	Reserved
13:0	**horizontal_output_width**	RW	0	Number of pixels per line to output - 1

Table 7-78. O_rescale_width.

7.11.3 O_rescale_horizontal_phase_inc

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:0	**horizontal_phase_inc**	RW	0	Horizontal phase increment

Table 7-79. O_rescale_horizontal_phase_inc.

For Mekra Lang only (08/2025) Confidential 1.5

7.11.4 O_rescale_horizontal_phase_init_even/odd

Bits	Name	Access	Reset	Description
31:28	-	-	-	Reserved
27:0	**horizontal_phase_init**	RW	0	Horizontal initial phase

Table 7-80. O_rescale_horizontal_phase_init_even/odd.

7.11.5 O_rescale_height

Bits	Name	Access	Reset	Description
31:14	-	-	-	Reserved
13:0	**vertical_output_height**	RW	0	Number of lines per frame - 1

Table 7-81. O_rescale_height.

7.11.6 O_rescale_vertical_phase_inc

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:0	**vertical_phase_inc**	RW	0	Vertical phase increment

Table 7-82. O_rescale_vertical_phase_inc.

7.11.7 O_rescale_vertical_phase_init

Bits	Name	Access	Reset	Description
31:28	-	-	-	Reserved
27:0	**vertical_phase_init**	RW	0	Vertical initial phase

Table 7-83. O_rescale_vertical_phase_init.

7.11.8 O_horizontal_coeff0/2/4/6/8...

Bits	Name	Access	Reset	Description
31:24	**coefficient_3**	W	NR	Signed 1.6 bits or signed 0.7 bits
23:16	**coefficient_2**	W	NR	Signed 1.6 bits or signed 0.7 bits
15:8	**coefficient_1**	W	NR	Signed 1.6 bits or signed 0.7 bits
7:0	**coefficient_0**	W	NR	Signed 1.6 bits or signed 0.7 bits

Table 7-84. O_horizontal_coeff0/2/4/6/8...

Confidential For Meeting Lang only 1 Oct 2025)

7.11.9 O_horizontal_coeff1/3/5/7...

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:8	**coefficient_5**	W	NR	Signed 1.6 bits or signed 0.7 bits
7:0	**coefficient_4**	W	NR	Signed 1.6 bits or signed 0.7 bits

Table 7-85. O_horizontal_coeff1/3/5/7...

7.11.10 O_vertical_coeff0/2/4/6/8...

Bits	Name	Access	Reset	Description
31:24	**coefficient_3**	W	NR	Signed 1.6 bits or signed 0.7 bits
23:16	**coefficient_2**	W	NR	Signed 1.6 bits or signed 0.7 bits
15:8	**coefficient_1**	W	NR	Signed 1.6 bits or signed 0.7 bits
7:0	**coefficient_0**	W	NR	Signed 1.6 bits or signed 0.7 bits

Table 7-86. O_vertical_coeff0/2/4/6/8...

7.11.11 O_vertical_coeff1/3/5/7...

Bits	Name	Access	Reset	Description
31:16	-	-	-	Reserved
15:8	**coefficient_5**	W	NR	Signed 1.6 bits or signed 0.7 bits
7:0	**coefficient_4**	W	NR	Signed 1.6 bits or signed 0.7 bits

Table 7-87. O_vertical_coeff1/3/5/7...

7.11.12 O_horizontal_dither_control

Bits	Name	Access	Reset	Description
31:4	-	-	-	Reserved
3:2	**horizontal_dither_format**	RW	0	Horizontal Dither Forma 0: Dither term is 0.4 1: Dither term is s+0.4 2: Dither term is s+1.4 3: Dither term is s+2.4
1	**horizontal_dither_alpha_enable**	RW	0	Horizontal Dither Alpha Enable 1: Dither alpha 0: Round alpha (no dithering) Ignored if horizontal dither enable is 0
0	**horizontal_dither_enable**	RW	0	Horizontal Dither Enable 1: Enable dithering 0: No dithering

Table 7-88. O_horizontal_dither_control.

For Mekra Lang only
1.5 (Sept 2025)
Confidential

7.11.13 O_horizontal_dither_seed

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:0	**horizontal_dither_seed**	RW	0	Horizontal dither seed random seed for dithering

Table 7-89. O_horizontal_dither_seed.

7.11.14 O_vertical_dither_control

Bits	Name	Access	Reset	Description
31:4	-	-	-	Reserved
3:2	**vertical_dither_format**	RW	0	Vertical Dither Format 0: Dither term is 0.4 1: Dither term is s+0.4 2: Dither term is s+1.4 3: Dither term is s+2.4
1	**vertical_dither_alpha_enable**	RW	0	Vertical Dither Alpha Enable 1: Dither alpha 0: Round alpha (no dithering) Ignored if vertical dither enable is 0
0	**vertical_dither_enable**	RW	0	Vertical Dither Enable 1: Enable dithering 0: No dithering

Table 7-90. O_vertical_dither_control.

7.11.15 O_vertical_dither_seed

Bits	Name	Access	Reset	Description
31	-	-	-	Reserved
30:0	**vertical_dither_seed**	RW	0	Vertical dither seed random seed for dithering

Table 7-91. O_vertical_dither_seed.

7.12 VOUT: Safety-Related Registers in VOUT ECR—Map

Offset	Register	Description
0xA40	**VOUT_ECR_CTRL**	Final error valid and clear error
0xA44	**VOUT_ECR_ERR**	ECR error bits
0xA48	**VOUT_ECR_MASK**	Mask bits to disable a specific section error valid

Table 7-92. OSD Rescale Registers: Map.

For Mechanical Lang only (08012025)

7.12.1 Vout_Ecr_Ctrl

Bits	Name	Access	Reset	Description
31:2	-	-	-	Reserved
1	**Inj_err**	RW	0	Write 1 will assert VOUT safety error valid to CEHU
0	**Clr_status**	RW	0	Write 1 will clear VOUT safety error valid

Table 7-93. Vout_Ecr_Ctrl.

7.12.2 Vout_Ecr_Error

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6	**smemif_err**	RW	0	Error detected in SMEM interface module
5	**display_b_err**	RW	0	Error detected in DISPLAY_B module
4	**display_a_err**	RW	0	Error detected in DISPLAY_A module
3	**mixer_b_err**	RW	0	Error detected in MIXER_B module
2	**mixer_a_err**	RW	0	Error detected in MIXER_A module
1	**osd_rescale_err**	RW	0	Error detected in OSD_RESCALE module
0	**top_misc_err**	RW	0	Error detected in other top-level modules, such as the SYNC counter interface, reset controller, AHB_INTERFACE

Table 7-94. Vout_Ecr_Error.

7.12.3 Vout_Ecr_Mask

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6	**smemif_err_mask**	RW	1	Error mask in SMEM interface module 0: Do not report error 1: Report error when detected
5	**display_b_err_mask**	RW	1	Error mask in DISPLAY_B module 0: Do not report error 1: Report error when detected
4	**display_a_err_mask**	RW	1	Error mask in DISPLAY_A module 0: Do not report error 1: Report error when detected
3	**mixer_b_err_mask**	RW	1	Error mask in MIXER_B module 0: Do not report error 1: Report error when detected
2	**mixer_a_err_mask**	RW	1	Error mask in MIXER_A module 0: Do not report error 1: Report error when detected
1	**osd_rescale_err_mask**	RW	1	Error mask in OSD_RESCALE module 0: Do not report error 1: Report error when detected

For Ambarella Confidential Use Only

Bits	Name	Access	Reset	Description
0	**top_misc_err_ mask**	RW	1	Error mask in other top-level modules, such as the SYNC counter interface, reset controller, AHB_INTERFACE 0: Do not report error 1: Report error when detected

Table 7-95. Vout_Ecr_Mask.

7.13 VOUT: Safety Notes

The following lists safety notes related to VOUT, procedures for disabling error progagation from VOUT to CEHU, and a list of the non-fatal errors implemented in VOUT.

To force an error to the CEHU on the VOUT unit:

- Set VOUT_ecr_ctrl[1] (offset 0xA40) to high to inject fault.

To clear the error status:

- Set VOUT_ecr_ctrl[0] (offset 0xA40) to high.

To disable error propagation from VOUT to CEHU (two methods):

- Set VOUT_ecr_ctrl[0] (offset 0xA40) to high to clear VOUT safety error.
- Because VOUT error status is recorded in ECRU on AHB bus, software can set error_mask_1[21] or input_ mask_1[21] in ECRU to high to mask VOUT safety error.

  The errors on VOUT are recorded in VOUT_ECR. Software can read VOUT_ECR configuration register VOUT_ECR_ERR (offset 0xA44) to check error status. It can be accessed via the AHB debug port.

Notes:

There are four non-fatal errors implemented in VOUT:

- Digital output underflow (no valid data output in active region).
- MIPI FIFO overflow (another valid data input to async FIFO when FIFO is full).
- Watchdog timeout error (helps to determine undesired VOUT frame-rate drop or hang).
- Luma / chroma stream checksum (helps to detect frozen a image to VOUT).

Confidential
For Mistral Lang only
Tue Oct 28 12:02:25 2025

7.14 VOUT: BTA Programming Flow

This section describes the programing flow for VOUT, including the BTA packet response under DCS Write and DCS Read. For additional questions not addressed in this section, contact an Ambarella support team member.

7.14.1 BTA Packet Response Under DCS Write

BTA Packets Response (I)

- Bytes from display panels can be accessed in DSI RCT registers using the following
  - Set **DSI_CTRL_AUX0**[26:24] = 5
    - **DSI_OBSV_REG0**[31:0] = **rx_lp_out0**[31:0]
    - **DSI_OBSV_REG1**[31:0] = **rx_lp_out1**[31:0]
  - Set **DSI_CTRL_AUX0**[26:24] = 6
    - **DSI_OBSV_REG**[31:0] = **rx_lp_out2**[31:0]
- {**rx_lp_out2**[31:0], **rx_lp_out1**[31:0], **rx_lp_out0**[31:0]} are the received bytes from the panels, where **rx_lp_out0** is received first, then **rx_lp_out1**, and finally, **rx_lp_out2**

Panel Response Case I—Host sends DCS write command, followed by BTA

- Sequence DCSWN-S => BTA
- If no write error occurs, then the panel returns an Acknowledge Trigger (0x84)
- If a write error occurs, then the panel returns an LPDT Trigger (0x87), followed by AwER

The following table lists panel response case (I): The host sends DCS Write Command, followed by BTA.

	AUX0[26:24]	OBSV_REG1[31:0]				OBSV_REG0[31:0]			
No Write Error	5	xx	xx	xx	xx	xx	xx	xx	0x84
	6	N/A							xx
Any Write Error	5	xx	xx	xx	AwER				0x87
	6	N/A							xx

Table 7-96. Host Sends DCS Write Command, Followed by BTA

7.14.2 BTA Packet Response Under DCS Read

BTA Packets Response (II)

- Bytes from display panels can be accessed in DSI RCT registers using the following
  - Set **DSI_CTRL_AUX0**[26:24] = 5
    - **DSI_OBSV_REG0**[31:0] = **rx_lp_out0**[31:0]
    - **DSI_OBSV_REG1**[31:0] = **rx_lp_out1**[31:0]
  - Set **DSI_CTRL_AUX0**[26:24] = 6
    - **DSI_OBSV_REG**[31:0] = **rx_lp_out2**[31:0]

Confidential Land Only
For Mekra Lang (Oct 2015)

- {**rx_lp_out2**[31:0], **rx_lp_out1**[31:0], **rx_lp_out0**[31:0]} are the received bytes from the panels, where **rx_lp_out0** is received first, then **rx_lp_out1**, and finally, **rx_lp_out2**

Panel Response Case II—Host sends DCS read (short) command, followed by BTA

- Sequence SMRPS-S => DCSRN-S = BTA
- If no read error occurs, then the panel returns an LPDT Trigger (0x87), followed by a 1 byte response (DCSRR1-S)
- If a correctable error occurs, then the panel returns an LPDT Trigger (0x87), followed by a 1 byte response (DCSRR-1) and AWER
- If a non-correctable error occurs, then the panel sends back an LPDT Trigger (0x87), followed by AwER

The following table lists panel response case (II): The host sends DCS Read Command, followed by BTA.

	AUX0[26:24]	OBSV_REG1[31:0]			OBSV_REG0[31:0]	
No Error	5	xx	xx	xx	DCSRR1-S	0x87
	6	N/A				xx
Correctable Error	5	AwER			DCSRR-S	0x87
	6	N/A				AwER
Non-Correctable Error	5	xx	xx	xx	AwER	0x87
	6	N/A				xx

Table 7-97. Host Sends DCS Read (Short) Command, Followed by BTA.

7.15 VOUT: MIPI DSI-PHY Protocol Timing

The following lists the protocol for adjusting the MIPI DSI-PHY timing.

Parameter	Timing (UI)
$T_{CLK-POST}$	648 ~ $T_{HS-TRAIL}$
$T_{CLK-TRAIL}$	8 ~ 256
$T_{HS-EXIT}$	8 ~ 512
T_{LPX}	8 ~ 2048
$T_{CLK-PREPARE}$	8 ~ 512
$T_{CLK-ZERO}$	8 ~ 512
$T_{CLK-PRE}$	16
$T_{HS-PREPARE}$	8 ~ 512
$T_{HS-ZERO}$	16 ~ 1024
$T_{HS-TRAIL}$	8 ~ 256

Table 7-98. MIPI DSI-PHY Protocol Timing.

Confidential for Meitu Lang only (2025-01-08 15:01:01)

8. DELAYED VSYNC GENERATOR

8.1 Delayed VSync: Overview

Delayed VSync generates delayed versions of original VSync / HSync signals. The VSync signal can be delayed by numbers of HSync pulses and/or numbers of clock cycles, and the HSync signal can be delayed by a number of clock cycles, resulting in two delayed versions of VSync signal and one delayed version of HSync signal.

To enable adaptability to various sensors, VSync and HSync include the following features:

- Source selection
  - VSync input can originate from one of the three sources: master(vin0 or vin1) or external signals.
  - HSync input can originate from one of the two sources: mater (vin0 or vin1).
- Polarity
  - VSync / HSync input / output signals can be either active-low or active-high.
- Pulse width
  - Pulse widths of output signals can be either the same as input signals or based on a fixed, specified value.

8.2 Delayed VSync Parameters

For the registers of the delayed VSync parameters beginning on page 383 in the Scratchpad chapter.

8.3 Delayed VSync: Software Programming Flow

The single-bit signal, **update_config**, indicates a new configuration. The programming flow is as follows.

1. Set **update_config** to 1
2. Write new value of delay parameters
3. Set **update_config** to 0

In typical scenarios, none of the parameters change after the first configuration, except for **dly_v_pulse**, which is applied at the next VSync falling edge (i.e., the start of the next frame).

The signal **vsdelay_enable** can be used to enable / disable vsdelay. Internal state machines and counters preserve their previous values when **vsdelay_enable** is equal to 0.

8.4 Delayed VSync: Delay Parameters

The following types of delay parameters are used to fulfill arbitrary alignments of VSync and HSync:

- dly_v_pulse
  - Indicates number of HSync pulses in which to delay VSync for various sensors.
- dly_v_clk
  - Indicates number of clock cycles in which to delay VSync, enabling VSync egdes to be slightly delayed after HSync egdes.

Confidential For Mekra Lang only 1.5 (08012025)

- dly_h_clk
  - Indicates number of clock cycles in which to delay HSync. This enables HSync egdes to be slightly delayed after VSync egdes.

The above parameters enable VSync to be delayed to any point within the timeline of a frame, while simultaneously fulfilling arbitrary alignments of VSync and HSync. Note that in the case of perfect alignment, **dly_v_clk** is equal to **dly_h_clk**.

8.5 Delayed VSync: Waveform

The following figure displays an example of a waveform (with parameters labeled) in fixed value width mode.

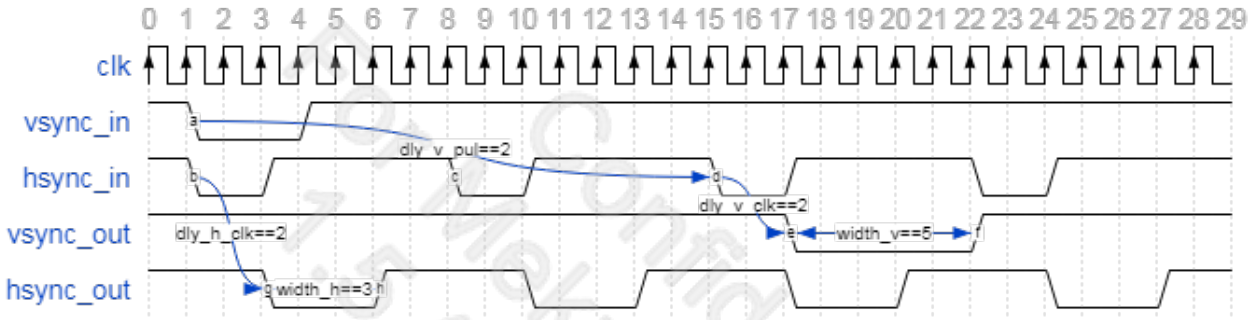


Figure 8-1. Example of a VSync Delay Waveform.

Notes:

- The counter of dly_v_pul counts the number of HSync falling edges. In the waveform example, the counter counts 0 at node b, 1 at node c, and 2 at node d.
- Synchronizers, edge detectors, and output flops contribute to some cycles of delay between input and output signals. In the figure above, the extra cycles are not included.

8.6 Delayed VSync: Assumptions

The following include assumptions for delayed VSync.

- **vsync_out** should be settled within the current frame and not delayed to the point beyond next **vsync_in** falling edge. Similarly, **hsync_out** should be settled within the current line.
- Synchronizers at the input side of VSync and HSync signals can include random delay effects as a result of metastability. All parameters for delay and pulse width are applied to VSync and HSync after the synchronizers.
- The counter of **dly_v_pul** counts the number of HSync falling edges, beginning with the first complete HSync pulse.

Confidential Ambarella Language only (08012025)

9. I2S INTERFACE

This section covers register programming for the I2S interface as follows:

- (Section 9.1) I2S: Overview
- (Section 9.2) I2S: Operation and Clocking
- (Section 9.3) I2S: Selecting the Data Format
- (Section 9.4) I2S: Setting the Transmitter Control Register
- (Section 9.5) I2S: Resetting the Transceiver
- (Section 9.6) I2S: Set Receiver Control Register
- (Section 9.7) I2S: Registers
- (Section 9.8) I2S: Software Programming Flow

9.1 I2S: Overview

The CV22FS I2S transceivers support full duplex transmit/receive digital audio. In addition, CV22FS I2S enables connections with external audio AD / DA devices and supports audio recording / playback without HDMI.

The CV22FS provides one I2S port that can be used for two-channel audio.

9.2 I2S: Operation and Clocking

- The CV22FS I2S interface provides various data formats, which can be configured using a mode register through the port AHB interface.
- The I2S interface supports two data transfer paths; i.e., one for transmit and the other for receive. Each data transfer path contains a FIFO, a FIFO threshold, FIFO flags, an interrupt signal, a DMA request and a DMA acknowledge signal. Interrupts to the Arm core are routed through the generic interrupt controller (GIC) on the AHB bus. DMA transfers are handled through the AHB DMA engine. The I2S can generate two interrupt signals to the Arm (i.e., one for transmit and the other for receive).
- The primary I2S clock GCLK_AU generally is sourced and configured by CV22FS system software using a set of APIs, and the register programming defined here is used for adjustment.
- The I2S bit clock **I2S_CLK** can be an input / output in I2S mode. In all other modes the transceiver acts as a master only, and **I2S_CLK** is an output.
- When the I2S is in master mode, **I2S_CLK** is an output, and the bit clock is generated by dividing the audio master clock signal **CLK_AU** with **I2S_clock** register **clk_div** field.
- Audio sampling rates of 8 kHz, 11.025 kHz, 12 kHz, 16 kHz, 22.05 kHz, 24 kHz, 32 kHz, 44.1 kHz, 48 kHz and 96 kHz are supported.

Confidential For Viekra Lane only (0801 only)

9.3 I2S: Selecting the Data Format

The I2S transceiver provides the following registers to determine the audio data format.

- Mode register **I2S_mode** which selects between five audio data formats.
- Word length register **I2S_wlen** which selects the audio data precision.
- Word position register **I2S_wpos** which selects the audio data location.
- Slot counter register **I2S_slot** which selects the data time slot for DSP mode.

The sections below cover:

- (Section 9.3.1) I2S Format: I2S Mode
- (Section 9.3.2) I2S Format: Left Justified Mode
- (Section 9.3.3) I2S Format: Right Justified Mode
- (Section 9.3.4) I2S Format: MSB Extend Mode
- (Section 9.3.5) I2S Format: DSP Mode

9.3.1 I2S Format: I2S Mode

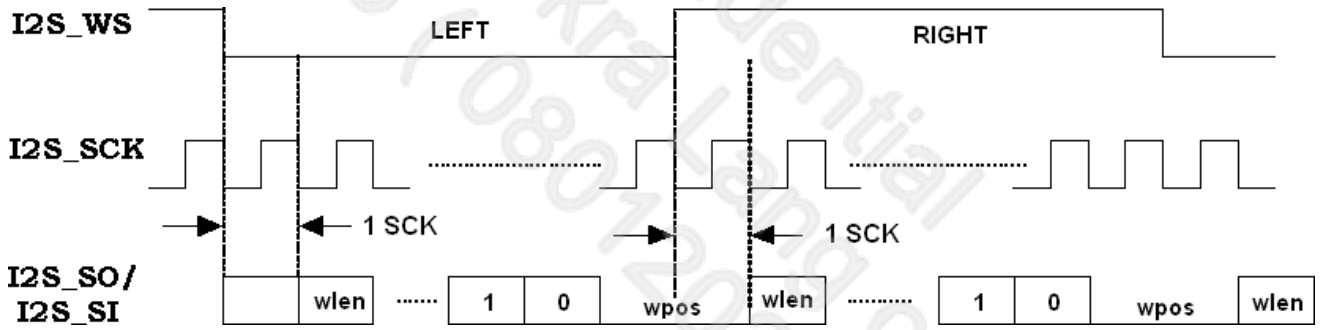


Figure 9-1. I2S Mode Timing Diagram.

I2S mode is triggered by setting the **I2S_mode** register **mode** bit to 0x4. In I2S mode, the MSBs of **I2S_SO** and **I2S_SI** are valid on the second **I2S_CLK** cycle period following a WS transition. When the data format is I2S mode, the Transmitter Control register (**I2S_tx_ctrl**) bit **tx_ws_inv** and the **rx_ws_inv** bit of the Receiver Control register (**I2S_rx_ctrl**) must be set to 0. If the corresponding **ss(I2S_clock)** is a master, the configuration table below applies.

Word Precision	Word Length	Word Position	WS Channel Width
16	0xF	0	16 bits
16	0xF	>1	32 bits
18	0x11	Ignored	32 bits
20	0x13	Ignored	32 bits
22	0x15	Ignored	32 bits
24	0x17	Ignored	32 bits
32	0x1F	Ignored	32 bits

Table 9-1. Configuration for I2S Mode.

Confidential Mekra Lang only 1.5 / 08012025

9.3.2 I2S Format: Left Justified Mode

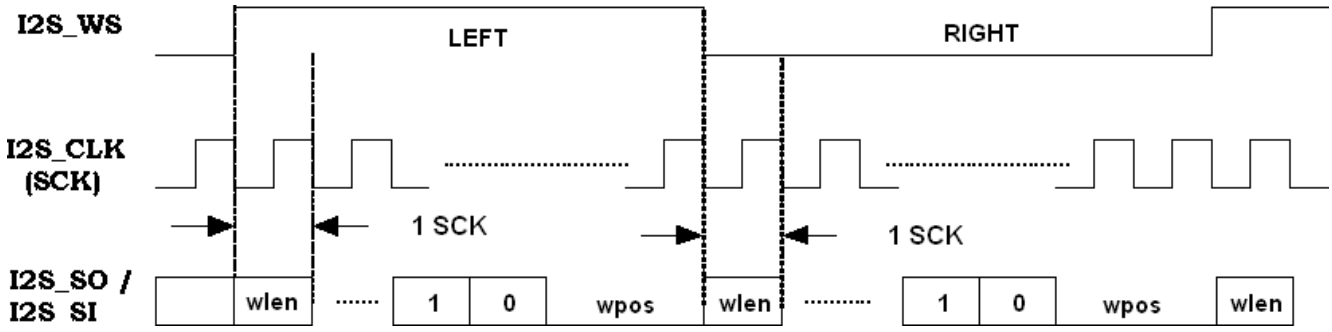


Figure 9-2. I2S Left Justified Mode Timing Diagram.

In left justify mode (**I2S_mode** bit **mode** set to 0x0), the MSB of **I2S_SI** and **I2S_SO** is valid on the first **I2S_CLK** cycle period following a **I2S_WS** transition. When the data format is set to left justify mode, the **tx_ws_inv** bit of the Transmitter Control register (**I2S_tx_ctrl**) and the **rx_ws_inv** bit of the Receiver Control register (**I2S_rx_ctrl)** must be set to 1. If **ss(I2S_clock)** is a master the configuration table below applies.

Word Precision	Word Length	Word Position	WS Channel Width
16	0xF	0	16 bits
16	0xF	>1	32 bits
18	0x11	Ignored	32 bits
20	0x13	Ignored	32 bits
22	0x15	Ignored	32 bits
24	0x17	Ignored	32 bits
32	0x1F	Ignored	32 bits

Table 9-2. Configuration for Left Justified Mode.

9.3.3 I2S Format: Right Justified Mode

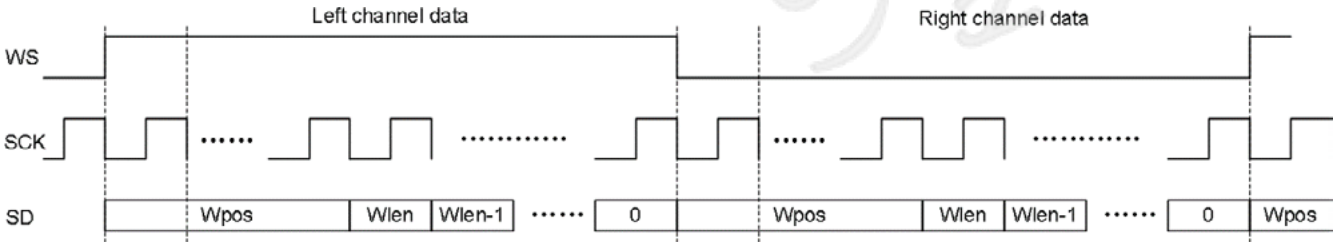


Figure 9-3. I2S Right Justified Mode Timing Diagram.

In right justify mode (**I2S_mode** bit **mode** set to 0x1), the LSB of **I2S_SI** and **I2S_SO** is valid on the **I2S_CLK** cycle period preceding a ws transition. When the data format is set to right justify mode, the **tx_ws_inv** bit of the Transmitter Control register (**I2S_tx_ctrl**) and the **rx_ws_inv** bit of the Receiver Control register (**I2S_rx_ctrl**) must be set to 1.

When I2S_wpos register is more than zero, the transmitter will assert the SD output LOW and the receiver will

For Confidential Mekra Training only (08012025)

ignore the SD input on the wpos period.
If the **ss(I2S_clock)** is a master, use the following configuration table.

Word Precision	Word Length	Word Position	WS Channel Width
16	0xF	0x0	16 bits
16	0xF	0xF	32 bits
18	0x11	0xD	32 bits
20	0x13	0xB	32 bits
22	0x15	0x9	32 bits
24	0x17	0x7	32 bits
32	0x1F	0x0	32 bits

Table 9-3. Configuration for Right Justified Mode.

9.3.4 I2S Format: MSB Extend Mode

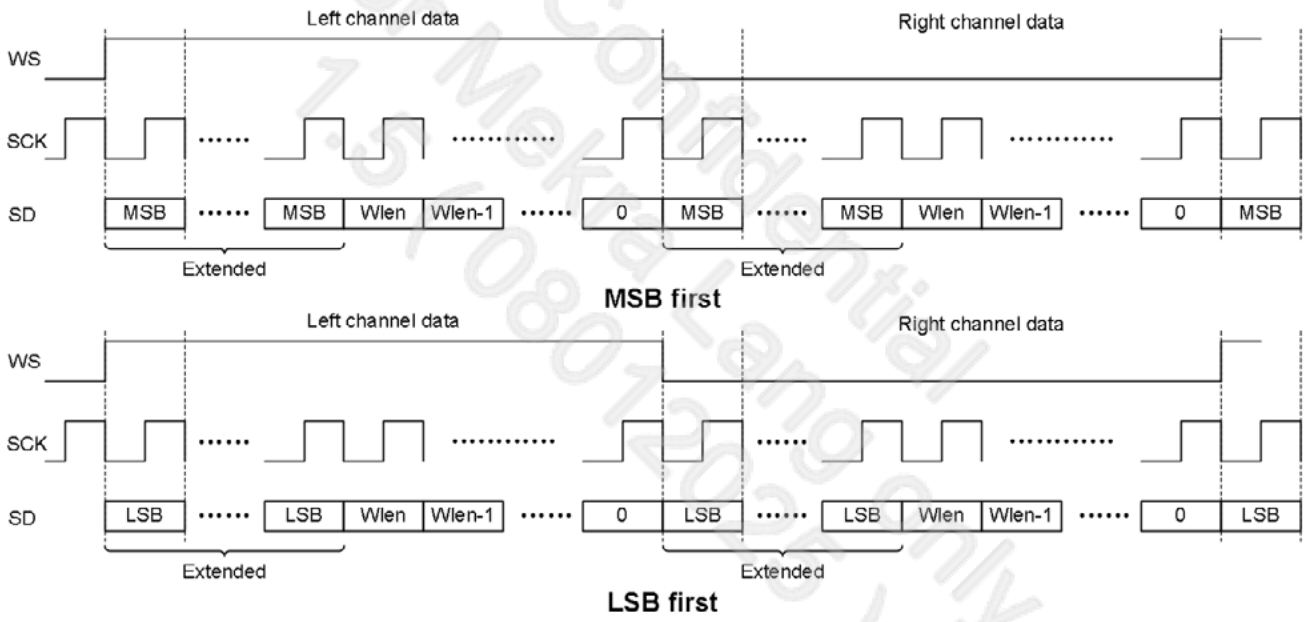


Figure 9-4. I2S MSB Mode Timing Diagram.

In MSB mode (**I2S_mode** bit **mode** set to 0x2), the MSB of **I2S_SO** and **I2S_SI** is extended from the first **I2S_CLK** cycle period following the current **I2S_WS** transition to let that the LSB of **I2S_SO** and **I2S_SI** is valid on the **I2S_CLK** cycle period preceding the next **I2S_WS** transition.

When the data format is set to MSB extend mode, **tx_ws_inv** (**I2S_tx_ctrl**) and **rx_ws_inv** (**I2S_rx_ctrl**) must be set to 1. When the I2S_wpos register is more than zero, the transmitter will keep the MSB of current channel data to output and the receiver will ignore the **I2S_SO** and **I2S_SI** input on the wpos period.

If the **ss(I2S_clock)** is a master, use the following configuration table.

Word precision	Word Length	Word Position	WS Channel Width
16	0xF	0	16 bits
16	0xF	0xF	32 bits
18	0x11	0xD	32 bits
20	0x13	0xB	32 bits
22	0x15	0x9	32 bits
24	0x17	0x7	32 bits
32	0x1F	0x0	32 bits

Table 9-4. Configuration for MSB Extend Mode.

9.3.5 I2S Format: DSP Mode

The following figure shows data transmission of the DSP mode.

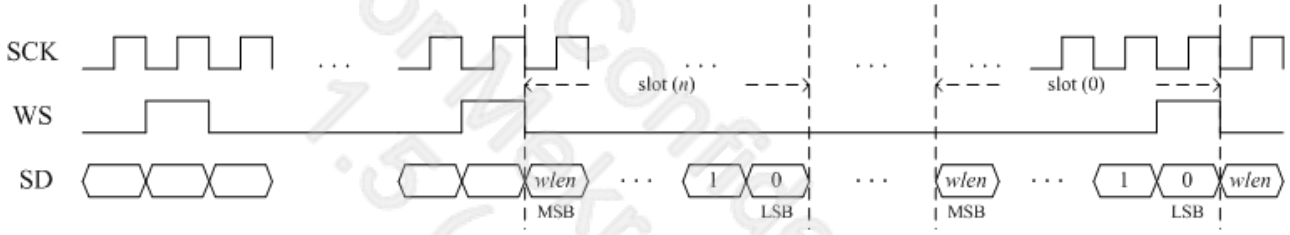


Figure 9-5. DSP Mode Data Transmission.

In DSP mode (**I2S_mode** bit **mode** set to 0x6), the MSB of transmitted data is valid on the second **SCK** cycle following a **WS** pulse. The (wlen+1) bits data word is termed "slot". Each **WS** strobe is followed by (n+1) slots when the slot count register "**I2S_SLOT**" is set to n. If "**I2S_SLOT**" is zero, each **WS** strobe is followed by a single slot, or (wlen+1) bits data. The "**rx_ws_inv**" is ignored in the DSP mode, while "**tx_ws_inv**" is used to determine which channel should be sent first.

If "**tx_ws_inv**" is 0 (see the following figure), the first slot following the first **WS** strobe is left channel after transmitter is enabled. If "**tx_ws_inv**" is 1 (see the following figure), the first slot following the first **WS** strobe is right channel after transmitter is enabled. After the first slot, the data will be transmitted from the left and the right channels alternatively.

For Ambarella Confidential Use Only 1.5 (08/2023)

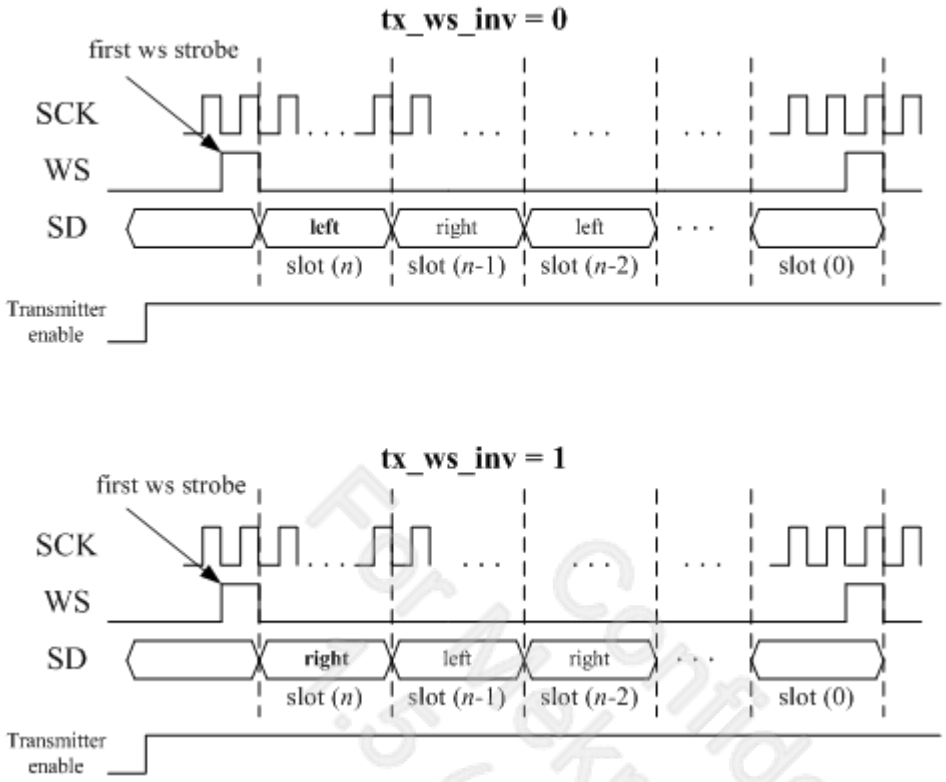


Figure 9-6. Tx_ws_inv Determines the First Transmitted Slot.

9.4 I2S: Setting the Transmitter Control Register

The sections below cover I2S transmitter settings:

- (Section 9.4.1) I2S Transmitter: Loop Back Test
- (Section 9.4.2) I2S Transmitter: I2S_SO Bit Order
- (Section 9.4.3) I2S Transmitter: ws Invert
- (Section 9.4.4) I2S Transmitter: Unison
- (Section 9.4.5) I2S Transmitter: Transmit Mute
- (Section 9.4.6) I2S Transmitter: Transmit Mono

9.4.1 I2S Transmitter: Loop Back Test

When the Transmitter Control register (**I2S_tx_ctrl**) Loop Back Test bit (**loopback**) is set to 1, the transmitter reads the received data and the **I2S_SO** delays one **I2S_WS** cycle before sending the received data.

The following figure shows the signal data of the loop back test. When the Loop Back Test bit of Transmitter Control register is set to 1, the receiver writes the data into the receive FIFO and the transmitter reads from the transmit FIFO normally, but the data read from the FIFO is dropped.

For Hokra Lang only (08/01/2025) Confidential

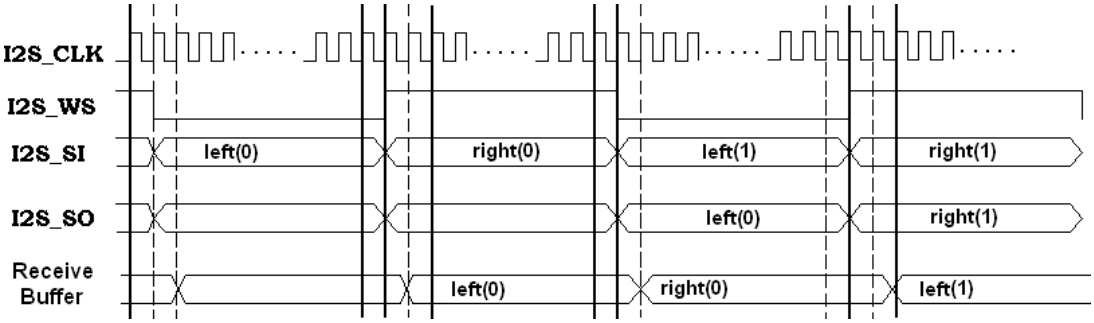


Figure 9-7. I2S Loop Back Test..

9.4.1.1 DSP Mode Loop Back Test

The DSP mode loop back test is enabled by writing 1 to the transmitter loop back test bit (loopback) of **i2s_tx_ctrl** register. The transmitter reads the received data from the receiver and sends it after one channel data.

In the following figure, the first data is left channel since **tx_ws_inv=0**. The first data “left(0)” is received and transmitted after receiving “right(0)”. The following transmission follows this rule. In transmitter loop back test mode, the receiver fills data into the FIFO normally; however, the transmitter reads data from the FIFO, but drops those data.

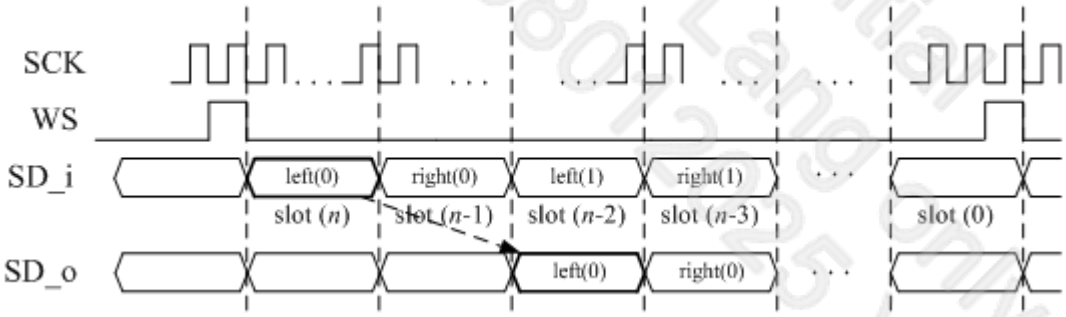


Figure 9-8. Transmitter Loop Back Test for DSP Mode.

9.4.2 I2S Transmitter: I2S_SO Bit Order

The Transmitter Control register **I2S_tx_ctrl** bit 6 **order** is used to control the send order of **I2S_SO**. If this bit is 0, the first bit of **I2S_SO** following the edge of **I2S_WS** is the MSB of FIFO data output bus. Otherwise, the first bit of **I2S_SO** is the LSB of FIFO data output bus. For example, if the word length is 15 and the bit order is 0, the order of the **I2S_SO** bit data is bit 15 to bit 0. Otherwise, the order of the **I2S_SO** bit data is bit 0 to bit 15.

9.4.3 I2S Transmitter: ws Invert

When the transmitter is enabled, the left channel data will be sent first. The Transmitter Control register **I2S_tx_ctrl** bit **tx_ws_inv** is used to control if the first left channel data is sent on **I2S_WS** = 0 or **I2S_WS** = 1.

- When **tx_ws_inv** is set to 0, the first left channel data will be sent on **I2S_WS** = 0.
- When **tx_ws_inv** is set to 1, the first left channel data will be sent on **I2S_WS** = 1.
- When the I2S Transceiver is in I2S mode, **tx_ws_inv** should be set to 0.
- When one of the left justified, right justified or MSB extend modes is selected, **tx_ws_inv** bit should be set to 1.

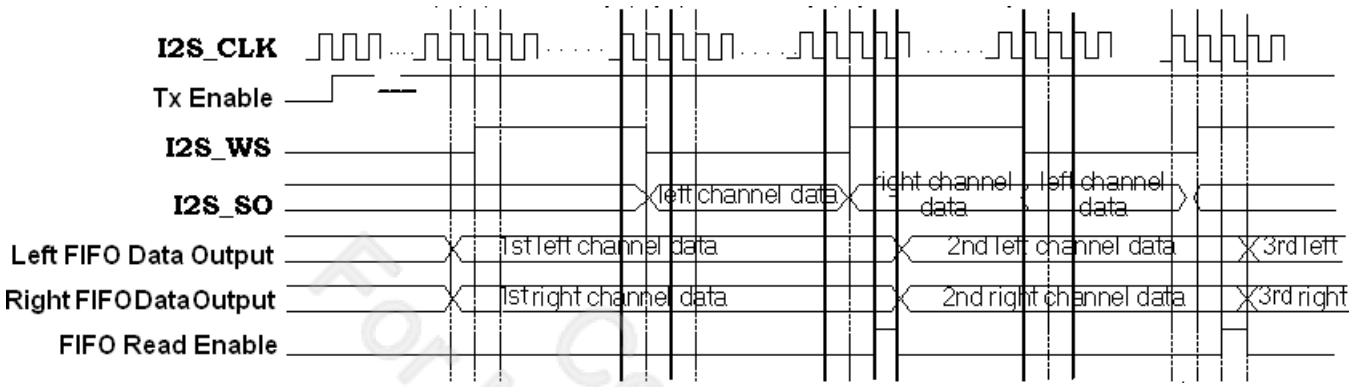


Figure 9-9. I2S tx_ws_inv = 0.

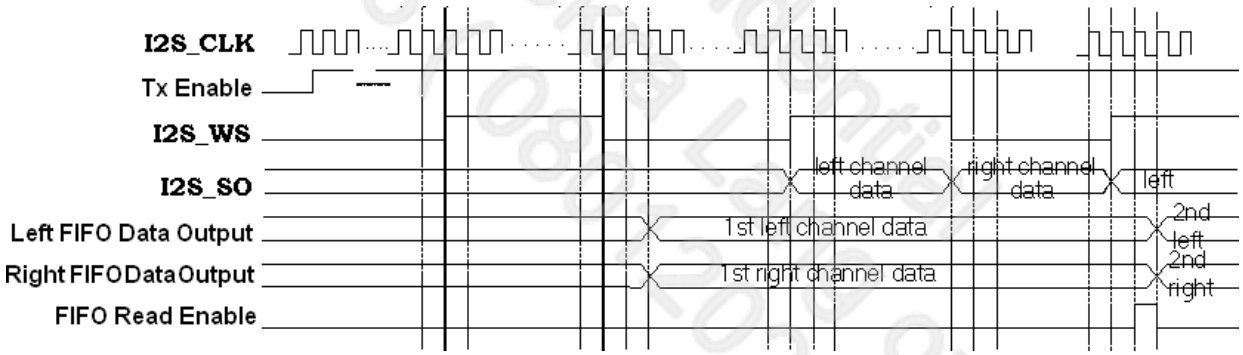


Figure 9-10. I2S tx_ws_inv = 1.

Confidential For Media Use Only 1.5 Rev 1 (Oct 2025)

9.4.4 I2S Transmitter: Unison

The Transmitter Control register (**I2S_tx_ctrl**) **unison** bit determines how the data is written into the FIFOs by the AHB interface. If this bit set to 0, the left channel FIFO will be written when AHB applies writes to offset 0x2C and the right channel FIFO will be written at offset 0x30. If this bit set to 1, writing to offset 0x2C will write the left and right channel FIFOs simultaneously. The right channel value is bits 31:16 and the left channel is bits 15:0. Writing to offset 0x30 also will write the right channel FIFO but not the left channel FIFO.

9.4.5 I2S Transmitter: Transmit Mute

When the Transmitter Control register (**I2S_tx_ctrl**) **mute** bit is written to 1, the mute sequence will be sent after any on-going transfer is finished. When the mute sequence is starting the transmitter read FIFO operation, FIFO related interrupt and DMA request signals all work normally.

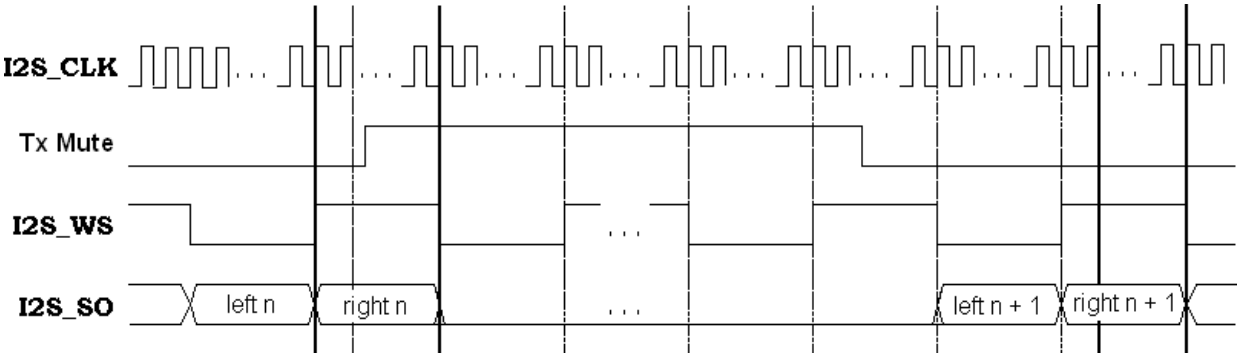


Figure 9-11. Mute Function Timing Diagram.

9.4.6 I2S Transmitter: Transmit Mono

The Transmitter Control register (**I2S_tx_ctrl**) **mono** bits control the transfer channel. If the **mono[1]** is 0, the transmitter is dual channel data transfer mode. Otherwise, only one channel data will be transmitted. If **mono[0]** is 1, the I2S transmitter is left monophonic (one channel) and the **I2S_SO** sends the left channel data at the right channel period. The right channel FIFO will be read normally and the right channel data will be dropped.

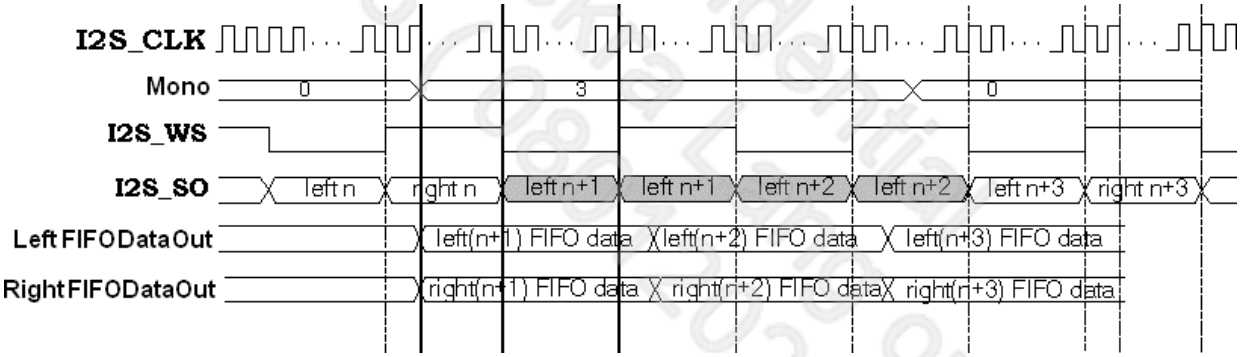


Figure 9-12. The Mono Left Channel Timing Diagram.

Otherwise, the I2S transmitter is right monophonic (one channel) transfer mode and the **I2S_SO** sends right channel data at the left channel period. The left channel FIFO will be read normally and the left channel data will be dropped.

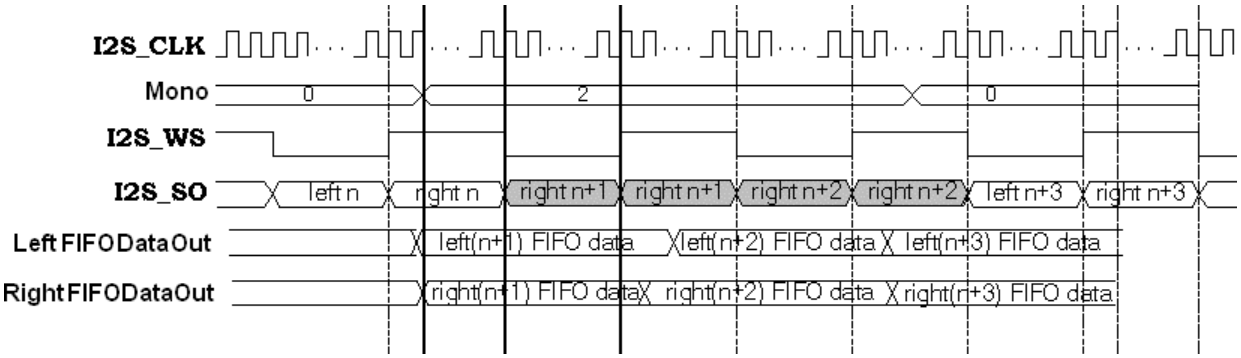


Figure 9-13. The Mono Right Channel Timing Diagram.

9.5 I2S: Resetting the Transceiver

There are two types of resets in the I2S transceiver: power on and FIFO. In power on reset, all the registers of I2S transceiver will be reset.

When the FIFO reset bit of Initial Control register is set to 1, only the transmitter and receiver FIFOs are reset and the other configurable registers will retain their original values.

When the transmitter FIFO reset bit is set, then only the transmitter FIFOs are reset and the other configurable registers including the receiver FIFO will retain their values.

Similarly, when the receiver FIFO reset bit is set, just the receiver FIFO will be reset and all other configurable registers retain their values.

9.6 I2S: Set Receiver Control Register

The sections below cover I2S receiver settings:

- (Section 9.6.1) I2S Receiver: Loop Back Test
- (Section 9.6.2) I2S Receiver: I2S_SI Bit Order
- (Section 9.6.3) I2S Receiver: WS Invert

9.6.1 I2S Receiver: Loop Back Test

The Receiver Control register (**I2S_rx_ctrl**) **loopback** bit controls whether the serial data source is from **I2S_SI** or from **I2S_SO**. If **loopback** is 1, the serial data source is **I2S_SO**. If the bit is 0, the serial data source is **I2S_SI**.

9.6.2 I2S Receiver: I2S_SI Bit Order

The Receiver Control register (**I2S_rx_ctrl**) bit **order** is used to control the sampling order of **I2S_SI**. If **order** is 0, the first sampled bit of **I2S_SO** following the edge of **I2S_WS** source will be written in the MSB of the FIFO data. Otherwise, the first sampled bit of **I2S_SI** will be written in the LSB of the FIFO data.

9.6.3 I2S Receiver: WS Invert

The Receiver Control register (**I2S_rx_ctrl**) bit **rx_ws_inv** indicates the word select **I2S_WS** signal polarity. In I2S mode (default value), driving the signal **I2S_WS** low or high corresponds to the left channel or right channel, respectively.

If the **rx_ws_inv** bit is set to 1, the **I2S_WS** signal has the opposite polarity. The following figure shows the left / right channel associations for the **rx_ws_inv** bit.

Confidential
For Fitipower Only
1.5 (2012/12/25)

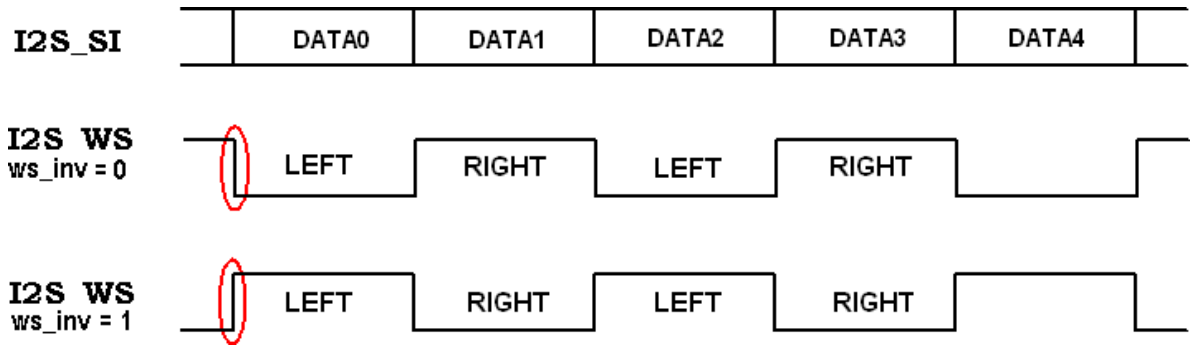


*Figure 9-14. Left / Right Channel for **rx_ws_inv**.*

9.7 I2S: Registers

This section provides a map of the I2S registers (Chapter 9.7.1) and details on register settings (Chapter 9.7.2). The remainder of the chapter provides notes on I2S programming specifics.

9.7.1 I2S Registers: Map

Offset	Register Name	Description
0x00	**I2S_mode**	Mode register
0x04	**I2S_rx_ctrl**	Receiver control
0x08	**I2S_tx_ctrl**	Transmitter control
0x0C	**I2S_wlen**	Word length
0x10	**I2S_wpos**	Ignored bits between two WS edges
0x14	**I2S_slot**	Number of slots for DSP mode
0x18	**I2S_tx_fifo_lth**	Transmitter FIFO threshold
0x1C	**I2S_rx_fifo_gth**	Receiver FIFO threshold
0x20	**I2S_clock**	Clock and word select control
0x24	**I2S_init**	Enable and reset register
0x28	**I2S_tx_fifo_flag**	Transmitter status register
0x2C	**I2S_tx_left_data**	Transmitter left channel data (for non-DMA transfers only)
0x30	**I2S_tx_right_data**	Transmitter right channel data (for non-DMA transfers only)
0x34	**I2S_rx_fifo_flag**	Receiver status
0x38	**I2S_rx_data**	Receiver FIFO threshold (for non-DMA transfers only)
0x3C	**I2S_tx_fifo_cntr**	Transmitter FIFO reside data count
0x40	**I2S_rx_fifo_cntr**	Receiver FIFO reside data count
0x44	**I2S_tx_interrupt_enable**	Transmitter interrupt enable
0x48	**I2S_rx_interrupt_enable**	Receiver interrupt enable
0x4C	**I2S_rx_echo**	Receiver echo enable
0x50	**I2S_32bitmux_mode**	DMA data L/R interleave (Only left address is valid; right address should not be used.)
0x54	**I2S_shift**	TX/RX shift
0x58	**I2S_channel_select**	Select channel
0x5C	**I2S_ws**	Word select enable
0x60 - 0x7C	-	Reserved

For Mekotronics Only
Confidential
1.5 (08/2014)

Offset	Register Name	Description
0x80	**I2S_rx_data_dma**	Required address for DMA transfers
0X84 – 0XBC	-	Reserved
0xC0	**I2S_tx_left_data_dma**	Required address for DMA transfers (This precludes the use of non L/R interleaved mode)

Table 9-5. I2S Digital Audio interface (DAI) Registers.

9.7.2 I2S Registers: Detail

9.7.2.1 I2S Mode Register

Bits	Name	Access	Reset	Description
31:3	-	-	-	Reserved
2:0	**mode**	RW	0x4	0x0 - Left justified mode 0x1 - Right justified mode 0x2 - MSB extend mode 0x4 - I2S mode 0x6 - DSP mode

Table 9-6. I2S Mode Register.

9.7.2.2 I2S_rx_ctrl Register

Bits	Name	Access	Reset	Description
31:4	-	-	-	Reserved
3	**loopback**	RW	0	Receiver loop back test 0 - The receiver will receive serial data from **I2S_SI** 1 - The receiver will receive serial data from **I2S_SI**
2	**order**	RW	0	Receiver bit order 0 - MSB is first 1 - LSB is first
1	-	-	-	Reserved
0	**rx_ws_inv**	RW	0	Receiver WS invert 0 - Receive first data on **I2S_WS** = 0 1 - Receive first data on **I2S_WS** = 1

Table 9-7. I2S Receiver Control Register.

9.7.2.3 I2S_tx_ctrl Register

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved

For Mokra Lang Only (08012015)

Bits	Name	Access	Reset	Description
7	**loopback**	RW	0	Transmitter loop back test 0 - The transmitter operates normally 1 - The transmitter will transmit the data that is received by the receiver
6	**order**	RW	0	Transmitter bit order 0 - MSB is first 1 - LSB is first
5	-	-	-	Reserved
4	**tx_ws_inv**	RW	0	Transmitter WS invert 0 - Left channel under **I2S_WS** = 0 1 - Left channel under **I2S_WS** = 1
3	**unison**	RW	0	Transmitter unison 0 tx_left_data[31:0] = left_data[31:0] 1 - tx_left_data[31:0] = {right_data[15:0], left_data[15:0]} Cannot set ping pong register (0x50) in this mode
2	**mute**	RW	0	Transmitter mute 0 - Normal 1 - Mute
1:0	**mono**	RW	0	Transmitter mono 0x0 - Stereo 0x1 - Reserved 0x2 - Mono right channel 0x3 - Mono left channel

Table 9-8. I2S Transmitter Control Register.

9.7.2.4 I2S_wlen Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4:0	**wlen**	RW	0	Word precision - 1. For example, 16 bit = 0xF and 24 bit = 0x17.

Table 9-9. I2S Word Length Register.

9.7.2.5 I2S_wpos Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4:0	**ignored_bits**	RW	0	The content of this register represents the number of ignored bits between WS rising and falling edge.

Table 9-10. I2S Word Position Register.

For Media Lang only (08012023) Confidential 1.5

9.7.2.6 I2S_slot Register

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7:0	**slot count**	RW	0	Data slot count after each WS strobe (only in DSP mode).

Table 9-11. I2S Slot Register.

9.7.2.7 I2S_tx_fifo_lth Register

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6:0	**ft**	RW	0	Transmitter FIFO Threshold Register When the FIFO contains fewer words than this threshold, the DMA request and interrupt signal will be asserted.

Table 9-12. I2S Transmitter FIFO Threshold Register.

9.7.2.8 I2S_rx_fifo_gth Register

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6:0	**ft**	RW	0x7F	Transmitter FIFO Threshold Register When the FIFO contains more words than this threshold, the DMA request and interrupt signal will be asserted.

Table 9-13. I2S Receiver FIFO Threshold Register.

9.7.2.9 I2S_clock Register

Bits	Name	Access	Reset	Description
31:13	-	-	-	Reserved
12	**data mode**	RW	0	Set this field to enable data mode. When data mode is enabled, I2S_CLK will not toggle if TX FIFO is empty while transmitting data or RX FIFO is full while receiving data.
11:10	**scale**	RW	0	clk_div scaling 0x0 – no scaling 0x1 – div_clk * 2 0x2 – div_clk * 4 0x3 – div_clk * 8
9	**woe**	RW	0	**I2S_WS** signal output enable 0 - Disable output to the **I2S_WS** pin 1 - Enable output to the **I2S_WS** pin **Note**: The I2S interface supports Slave mode for I2S mode only. Software must program this field to 1 for all other modes except I2S.

For Confidential Meeting only 1.5 (Oct 2025)

Bits	Name	Access	Reset	Description
8	**soe**	RW	0	**I2S_CLK** signal output enable 0 - Disable output to the **I2S_CLK** pin 1 - Enable output to the **I2S_CLK** pin **Note**: The I2S Interface supports Slave mode for I2S mode only. Software must program this field to 1 for all other modes except I2S.
7	**ss**	RW	1	**I2S_CLK** source selection 0 - **I2S_CLK** is input (slave mode) 1 - **I2S_CLK** is output (master mode) **Note**: The I2S interface supports Slave mode for I2S mode only. Software must program this field to 1 for all other modes except I2S.
6	**tsp**	RW	0	Transmitter **I2S_CLK** polarity 0 - **I2S_SO** data change at rising edge 1 - **I2S_SO** data change at falling edge
5	**rsp**	RW	0	Receiver **I2S_CLK** polarity 0 - Sample **I2S_SI** data at rising edge 1 - Sample **I2S_SI** data at falling edge
4:0	**clk_div**	RW	0	Clock divider: **I2S_CLK** frequency = **CLK_AU** frequency / [2 (**clk_div * (2^scale)**+1)]

Table 9-14. I2S Clock Control Register.

9.7.2.10 I2S_init Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**txfrst**	W	0	Transmitter FIFO Reset. Write 1 to reset Transmitter FIFO
3	**rxfrst**	W	0	Receiver FIFO Reset. Write 1 to reset Receiver FIFO
2	**te**	RW	0	Transmitter enable
1	**re**	RW	0	Receiver enable
0	**frst**	W	0	FIFO reset Write 1 to reset both Transmitter & Receiver FIFOs. Automatically cleared by hardware after FIFO reset. Returns 0 on read by software.

Table 9-15. I2S Initial Control Register.

9.7.2.11 I2S_tx_fifo_flag Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**ti**	R	1	Transmitter idle flag
3	**ftv**	R	0	FIFO threshold is valid
2	-	-	-	Reserved
1	**ff**	R	0	FIFO full flag

For Mekra Lang only
Confidential
1.5 (Oct 2013)

Bits	Name	Access	Reset	Description
0	**fe**	R	1	FIFO empty flag

Table 9-16. I2S Transmitter Status Register.

9.7.2.12 I2S_tx_left_data Register

When the **I2S_tx_ctrl** bit **unison** is 1, the data width of this address is 32 bits and the following register set detail applies.

Bits	Name	Access	Reset	Description
31:0	**data**	W	-	The data width of this address is 32 bits when the **I2S_tx_ctrl** bit **unison** is 1. The high 16-bits data is written into right channel data FIFO and the low 16-bits data is written into left channel FIFO data.

Table 9-17. I2S Transmitter Left Channel Data Register When the Transmit Unison Bit of I2S_tx_ctrl is 1.

When the **I2S_tx_ctrl** bit **unison** is 0, the following register set detail applies.

Bits	Name	Access	Reset	Description
31:0	**data**	W	-	When **unison** is 0 and the I2S is not in L/R interleaved mode (as described in Section 9.3 "I2S: Selecting the Data Format"), the data width of this address is 32 bits, which is only for left channel FIFO data. When in L/R interleaved mode, this register is the write port of left/right interleaved data.

Table 9-18. I2S Transmitter Left Channel Data Register When the Transmit Unison Bit of I2S_tx_ctrl is 0.

9.7.2.13 I2S_tx_right_data Register

Bits	Name	Access	Reset	Description
31:0	**data**	W	-	The data width of this address is 32 bits for the right channel FIFO data.

Table 9-19. I2S Transmitter Right Channel Data Register.

9.7.2.14 I2S_rx_fifo_flag Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**ri**	R	1	Receiver idle flag
3	**ftv**	R	0	FIFO threshold is valid
2	**fo**	R	0	FIFO overflow flag
1	**ff**	R	0	FIFO full flag

Confidential
For Meng Zhang only
(Oct 2025)

Bits	Name	Access	Reset	Description
0	**fe**	R	1	FIFO empty flag

Table 9-20. I2S Receiver FIFO Flag Register.

9.7.2.15 I2S_rx_data Register

Bits	Name	Access	Reset	Description
31:0	**data**	R	0	When AHB read data form this address, the FIFO will be popped one entry data. When FIFO is empty, the AHB read data is always zero.

Table 9-21. I2S Receiver Data Register.

9.7.2.16 I2S_tx_fifo_cntr Register

Bits	Name	Access	Reset	Description
31:7	-	-	-	Reserved
6:0	**c**	R	0	Transmitter FIFO data count

Table 9-22. I2S Transmitter FIFO Counter Register.

9.7.2.17 I2S_rx_fifo_cntr Register

Bits	Name	Access	Reset	Description
31:8	-	-	-	Reserved
7:0	**c**	R	0	Receiver FIFO data counter

Table 9-23. I2S Receiver FIFO Counter Register.

9.7.2.18 I2S_tx_interrupt_enable Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**ti**	RW	0	Transmitter idle interrupt enable
3	**ftv**	RW	1	FIFO valid threshold interrupt enable
2	-	-	-	Reserved
1	**ff**	RW	0	FIFO full interrupt enable
0	**fe**	RW	0	FIFO empty interrupt enable

Table 9-24. I2S Transmitter Interrupt Enable Register.

Confidential For Ambarella Lang only (08012025)

9.7.2.19 I2S_rx_interrupt_enable Register

Bits	Name	Access	Reset	Description
31:5	-	-	-	Reserved
4	**ri**	RW	0	Receiver idle interrupt enable
3	**ftv**	RW	1	FIFO valid threshold interrupt enable
2	**fo**	RW	0	FIFO overflow interrupt enable
1	**ff**	RW	0	FIFO full interrupt enable
0	**fe**	RW	0	FIFO empty interrupt enable

Table 9-25. I2S Receiver Interrupt Enable Register.

9.7.2.20 I2S_rx_echo Register

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**echo**	RW	0	0 - No echo 1 - Echo RX data to TX as well as forwarding to main memory

Table 9-26. I2S Receiver RX Echo Register.

9.7.2.21 I2S_32bitmux_mode Register

When the 32-bit multiplexed mode is enabled the Transmit Left register (**I2S_tx_left_data**) contains interleaved left and right channel data. The first data in the FIFO after reset is left channel, the second is right channel and so on. This field should be set to 1 in DMA mode except when using unison mode.

Bits	Name	Access	Reset	Description
31:1	-	-	-	Reserved
0	**multi_32_en**	RW	0	0 - Disable 32-bit multiplexed mode 1 - Enable 32-bit multiplexed mode Word length register **I2S_wlen** must be set to 32-bit operation

Table 9-27. I2S 32-Bit Multiplexed Mode Register.

9.7.2.22 I2S_shift Register

Bits	Name	Access	Reset	Description
31:4	-	-	-	Reserved
3	**rx_shift_16**	RW	0	0 - RX shift 8 bits 1 - RX shift 16 bits

Confidential
Ambarella Internal Only
For Pandora 1.5T 0801 V25

Bits	Name	Access	Reset	Description
2	**tx_shift_16**	RW	0	0 - TX shift 8 bits 1 - TX shift 16 bits
1	**rx_shift_en**	RW	0	0 - Disable RX shift 1 - Enable RX shift
0	**tx_shift_en**	RW	0	0 - Disable TX shift 1 - Enable TX shift

Table 9-28. I2S Shift Register.

9.7.2.23 I2S_channel_select Register

Bits	Name	Access	Reset	Description
31:2	-	-	-	Reserved
1:0	**channel_select**	RW	0	0x0 - 2 channels enabled 0x1 - 4 channels enabled 0x2 - Reserved 0x3 - Reserved

Table 9-29. I2S Channel Select Register.

9.7.2.24 I2S_ws Register

Bits	Name	Access	Reset	Description
31:2	-	-	-	Reserved
1	**ws_set**	RW	0	WS reset value. 0 – WS reset to 0 1 – WS reset to 1 WS will reset to the setting value when **WS_EN** = 0
0	**ws_en**	RW	0	WS enable. WS is workable when this bit is set.

Table 9-30. I2S WS Register.

9.7.2.25 I2S_rx_data_dma Register

This register must be programmed for RX data when using the AHB DMA engine for transfer.

Software must use the no-increment mode of the DMA channel control registers and set the src/dest blk increment to 64 bytes or lower on the peripheral side (peripheral side refers to I2S).

Bits	Name	Access	Reset	Description
31:0	**data**	R	0	When the AHB reads data from this address the FIFO will pop one entry. When FIFO is empty, the AHB read data is always zero.

Table 9-31. I2S Receive DMA Data Register.

For Mekra Lang only (08/2023)

9.7.2.26 I2S_tx_left_data_dma Register

This register must be programmed for TX data when using the AHB DMA engine for transfer. If the sample word length is 16 bits/sample, unison mode must be enabled. If the sample word length is 32 bits/sample, the 32-bit L/R multiplexed mode must be enabled (Section 9.3 "I2S: Selecting the Data Format"). This effectively precludes the use of non L/R (left/right) interleaved data when using the AHB DMA engine for transfer.

Software must also use no-increment mode of DMA channel control register and set the source / destination block increment to 64 bytes or lower on the peripheral side (peripheral side refers to I2S).

Bits	Name	Access	Reset	Description
31:0	**data**	W	-	This address is to transfer data using the AHB DMA Engine When sample word length is 16 bits/sample, unison mode applies.

Table 9-32. I2S Transmit DMA Left Data Register When Sample Word Length is 16 Bits/Sample.

Bits	Name	Access	Reset	Description
31:0	**data**	W	-	This address is to transfer data using the AHB DMA Engine When sample word length is 32 bits/sample, the 32-bit L/R multiplexed mode applies.

Table 9-33. I2S Transmit DMA Left Data Register When Sampled Word Length is 32 Bits/Sample.

For Nvidia Tegra Lagony
Confidential
1.5 (08012025)

9.8 I2S: Software Programming Flow

The following sections describe the software programming flow for the I2S transmitter and receiver.

9.8.1 I2S Transmitter

1. Set up the **I2S_mode** register to decide which mode is to use.
2. Determine using how many I2S channels by setting I2S_channel_select register. (CV22FS supports two-channel only.)
3. Set up the **I2S_clock** register to determine I2S serial clock frequency, set **I2S_CLK** and **I2S_WS** output enable, and decide serial clock source (master or slave mode).
4. Set up the **I2S_tx_ctrl** register to decide TX-related settings.
5. Set up the **I2S_slot** register for slot count. (DSP mode only.)
6. In DMA mode, set up the **I2S_32bitmux_mode** register.
7. Set the TX FIFO threshold by setting the **I2S_tx_fifo_lth** register.
8. Set up **I2S_wlen** to decide the word length.
9. In non-DMA mode, write left / right data into left / right FIFOs by setting the **I2S_tx_left_data** and **I2S_tx_right_data** registers. In DMA mode, set the **I2S_tx_left_data_dma** register.
10. Set up **I2S_INIT** to enable transmitter.
11. Set up **I2S_WS** to enable WS. (In DSP mode, delay one **I2S_CLK** clock cycle time from previous step to set this register.)
12. Start to transmit data.

9.8.2 I2S Receiver

1. Set up the **I2S_mode** register to decide which mode is to be used.
2. Determine using how many I2S channels by setting the **I2S_channel_select** register. (CV22FS supports two-channel only.)
3. Set up the **I2S_clock** register to determine I2S serial clock frequency and set **I2S_CLK** and **I2S_WS** output enable, and decide serial clock source (master or slave mode).
4. Set up the **I2S_rx_ctrl** register to decide RX-related settings.
5. Set up the **I2S_slot** register for slot count. (DSP mode only.)
6. In DMA mode, set up the **I2S_32bitmux_mode** register.
7. Set the Rx FIFO threshold by setting **I2S_rx_fifo_gth** register.
8. Set up **I2S_wlen** to decide the word length.
9. Set up **I2S_init** to enable receiver.
10. Set up **I2S_WS** to enable WS. (In DSP mode, delay one **I2S_CLK** clock cycle time from previous step to set this register.)
11. Start to receive data.
12. In non-DMA mode, poll **I2S_rx_fifo_cntr** to the expected value and then read **I2S_rx_data** to get RX data. In DMA mode, read RX data from the **I2S_rx_data_dma** register.

For Confidential Mekra Lang only (08012025)

10. GIGABIT ETHERNET CONTROLLER

This chapter covers the CV22FS Ethernet controller:

- (Section 10.1) Ethernet: Overview
- (Section 10.2) Ethernet: Register Maps
- (Section 10.3) Ethernet: Register Details
- (Section 10.4) Ethernet: Alternate (Enhanced) Descriptors
- (Section 10.5) Ethernet: Programming Guidelines

10.1 Ethernet: Overview

The Ethernet controller enables a host to transmit and receive data over Ethernet in compliance with the "*IEEE 802.3-2002 Standard for Ethernet Based LANs".* Features include the following:

- Two ENET ports with 10/100/1000-Mbps data transfer rates with IEEE 802.3-compliant RGMII and RMII interfaces to communicate with an external Fast Ethernet PHY.
- Supports both full-duplex and half-duplex operation
- Preamble and start-of-frame data (SFD) insertion in Transmit (TX), and deletion in Receive (RX) paths
- Options for automatic pad generation and CRC stripping on receive frames
- Received data FIFO and transmit data FIFO at 4 KB each
- Programmable InterFrameGap (40-96 bit-times in steps of 8)
- Supports a variety of flexible address filtering modes:
  - Three additional 48-bit perfect (DA) address filters with masks for each byte
  - Three 48-bit SA address comparison check with masks for each byte
  - 64-bit hash filter (optional) for multicast and uni-cast (DA) addresses
  - Option to pass all multicast addressed frames
  - Promiscuous mode support to pass all frames without any filtering for network monitoring
  - Passes all incoming packets (as per filter) with a status report
- Separate 32-bit status returned for transmission and reception packets
- Supports IEEE 802.1Q VLAN tag detection for reception frames
- MDIO master interface (optional) for PHY device configuration and management
- Module for detection of LAN wakeup frames and AMD Magic Packet frames
- Receive module for checksum off-load for received IPv4 and TCP packets encapsulated by the Ethernet frame

Confidential
Ambarella Internal Use Only

10.1.1 Ethernet Overview: Address Space

The Ethernet controller is mapped to AHB base addresses 0xE000.E000 (MAC) and 0xE000.F000 (DMA).
The second Ethernet controller is mapped to AHB base addresses 0xE003.2000 (MAC) and 0xE003.3000 (DMA).

10.1.2 Ethernet Overview: Interrupt Vector

Ethernet controller interrupts are mapped to the general interrupt controller (GIC). The MAC Subsystem SBD_ INTR_O interrupt maps to GIC line 64. MAC core interrupt PMT_INTR_O maps to GIC line 63.
The second Ethernet controller interrupts are also mapped to the general interrupt controller (GIC). The MAC Subsystem SBD_ INTR_O interrupt maps to GIC line 133. MAC core interrupt PMT_INTR_O maps to GIC line 132.

10.1.3 Ethernet Overview: Clocking

The Ethernet clock GCLK_GTX is generally configured by CV22FS system software.

10.1.4 Ethernet Overview: System Level Block Diagram

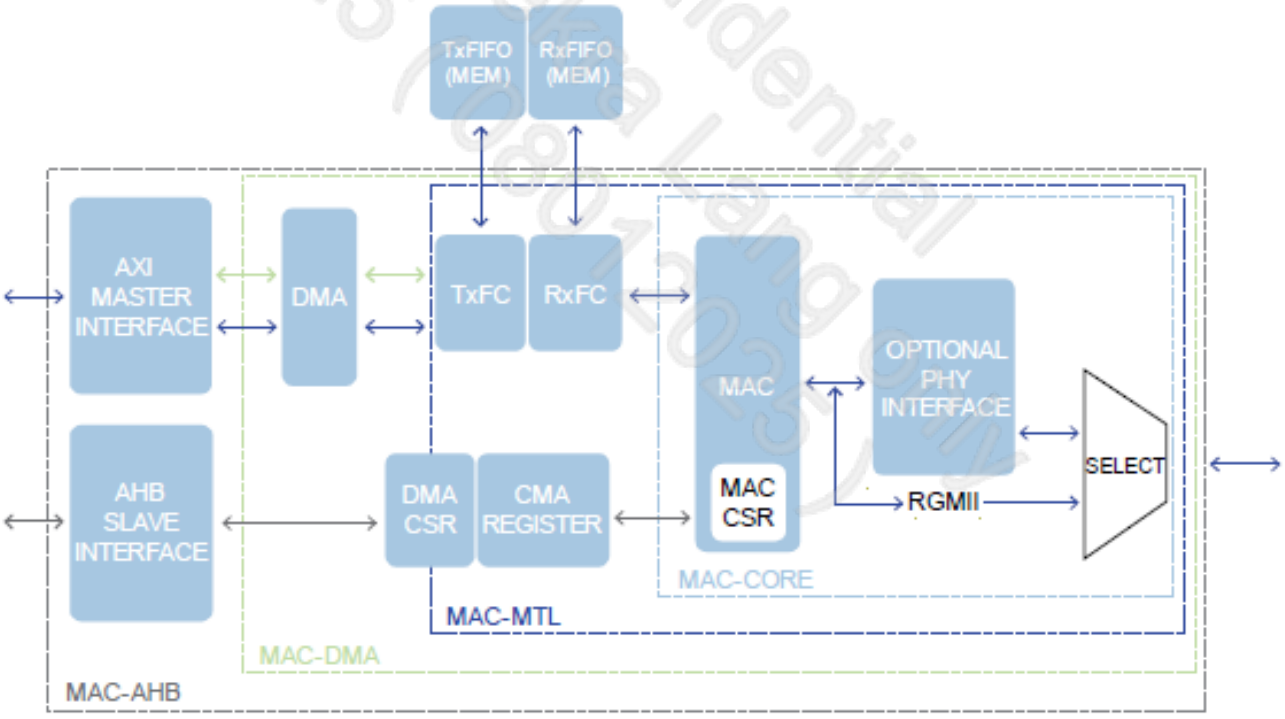


Figure 10-1. CV22FS Ethernet Controller System-Level Block Diagram.

10.2 Ethernet: Register Maps

The register maps in this section provide high-level summaries of each register or group of registers.

10.2.1 Ethernet: DMA Register Map

Offset	Register	Description
0x0000	**DMA_R0_bus_mode**	Register 0 - Bus Mode Controls the Host Interface Mode
0x0004	**DMA_R1_tx_poll**	Register 1 - Transmit Poll Demand Used by the host to instruct the DMA to poll the Transmit Descriptor List
0x0008	**DMA_R2_rx_poll**	Register 2 - Receive Poll Demand Used by the Host to instruct the DMA to poll the Receive Descriptor List
0x000C	**DMA_R3_rx_descriptor**	Register 3 - Receive Descriptor List Address Points DMA to the start of the Receive Descriptor List
0x0010	**DMA_R4_tx_descriptor**	Register 4 - Transmit Descriptor List Address Points DMA to the start of the Transmit Descriptor List
0x0014	**DMA_R5_status**	Register 5 - Status Read by software driver (application) during interrupt service routine or polling to determine the DMA status
0x0018	**DMA_R6_operation_mode**	Register 6 - Operation Mode Establishes RX and TX operating modes and command
0x001C	**DMA_R7_interrupt_enable**	Register 7 - Interrupt Enable Enables the interrupts reported by the Status Register
0x0020	**DMA_R8_fmbuff_count**	Register 8 - Missed Frame and Buffer Overflow Count Contains counters for discarded frames because host Receive Descriptor unavailable and Receive FIFO Overflow
0x0024 - 0x0044		Reserved
0x0048	**DMA_R18_host_tx_descript**	Register 18 - Current Host Transmit Descriptor Points to start of current Transmit Descriptor read by the DMA
0x004C	**DMA_R19_host_rx_descript**	Register 19 - Current Host Receive Descriptor Points to the start of current Receive Descriptor read by the DMA
0x0050	**DMA_R20_host_tx_buffer**	Register 20 - Current Host Transmit Buffer Address Points to the current Transmit Buffer address read by the DMA.
0x0054	**DMA_R21_host_rx_buffer**	Register 21 - Current Host Receive Buffer Address Points to the current Receive Buffer address read by the DMA

Table 10-1. Map of the Ethernet Controller DMA Registers.

For Mekra Confidential 1.5 (2801) 2015-10-21

10.2.2 Ethernet: MAC Register Map

Offset	Register	Description
0x0000	**MAC_R0_MAC_config**	Register 0 - MAC Configuration The operation mode register for the MAC
0x0004	**MAC_R1_MAC_fm_filter**	Register 1 - MAC Frame Filter Contains the frame filtering controls
0x0008	**MAC_R2_hash_table_high**	Register 2 - Hash Table High Contains the higher 32 bits of the Multicast Hash table
0x000C	**MAC_R3_hash_table_low**	Register 3 - Hash Table Low Contains the lower 32 bits of the Multicast Hash table
0x0010	**MAC_R4_GMII_address**	Register 4 - GMII Address Controls the management cycles to an external PHY
0x0014	**MAC_R5_GMII_data**	Register 5 - GMII Data register Contains data to write to or read from the PHY register
0x0018	**MAC_R6_flow_control**	Register 6 - Flow Control register Controls the generation of control frames
0x001C	**MAC_R7_VLAN_tag**	Register 7 - VLAN Tag register Identifies IEEE 802.1Q VLAN type frames
0x0020	**MAC_R8_version**	Register 8 - Version Identifies the version of the Core
0x0024 - 0x002B		Reserved
0x002C	**MAC_R11_PMT_Control_and_Status**	Register 11 - PMT Control and Status register
0x0030 - 0x0037		Reserved
0x0038	**MAC_R14_interrupt_status**	Interrupt Status register
0x003C	**MAC_R15_interrupt_mask**	Interrupt Mask register
0x0040	**MAC_R16_MAC_add0_high**	Register 16 - MAC Address0 High Contains the higher 16 bits of the 1st MAC address
0x0044	**MAC_R17_MAC_add0_low**	Register 17 - MAC Address0 Low Contains the lower 32 bits of the 1st MAC address
0x0048	**MAC_R18_MAC_add1_high**	Register 18 - MAC Address1 High Contains the higher 16 bits of the 2nd MAC address
0x004C	**MAC_R19_MAC_add1_low**	Register 19 - MAC Address1 Low Contains the lower 32 bits of the 2nd MAC address
0x0050	**MAC_R20_MAC_add2_high**	Register 20 - MAC Address2 High Contains the higher 16 bits of the 3rd MAC address
0x0054	**MAC_R21_MAC_add2_low**	Register 21 - MAC Address2 Low Contains the lower 32 bits of the 3rd MAC address
0x00D8	**MAC_R54_RGMII_Control_and_Status**	Register 54 - RGMII Control and Status register
0x0700	**MAC_R448_timestamp_control**	Register 448 - Timestamp Control Controls the timestamp generation and update logic
0x0704	**MAC_R449_sub_second_increment**	Register 449 - Sub-Second Increment Contains the 8-bit value by which the Sub-Second register is incremented
0x0708	**MAC_R450_system_time_seconds**	Register 450 - System Time – Seconds Contains the lower 32 bits of the seconds field of the system time
0x070C	**MAC_R451_system_time_nanoseconds**	Register 451 - System Time – Nanoseconds Contains 32 bits of the nano-seconds field of the system time

Offset	Register	Description
0x0710	**MAC_R452_system_time_seconds_update**	Register 452 - System Time – Seconds Update Contains the lower 32 bits of the seconds field to be written to, added to, or subtracted from the System Time value.
0x0714	**MAC_R453_system_time_nanoseconds_update**	Register 453 - System Time – Nanoseconds Update Contains 32 bits of the nano-seconds field to be written to, added to, or subtracted from the System Time value.
0x0718	**MAC_R454_timestamp_addend**	Register 454 - Timestamp Addend This register is used by the software to readjust the clock frequency linearly to match the master clock frequency
0x071C	**MAC_R455_target_time_seconds**	Register 455 - Target Time Seconds Contains the higher 32 bits of time to be compared with the system time for interrupt event generation or to start the PPS signal output generation
0x0720	**MAC_R456_target_time_nanoseconds**	Register 456 - Target Time Nanoseconds Contains the lower 32 bits of time to be compared with the system time for interrupt event generation or to start the PPS signal output generation
0x0728	**MAC_R458_timestamp_status**	Register 458 - Timestamp Status Contains the PTP status
0x072C	**MAC_R459_PPS_control**	Register 459 - PPS Control This register is used to control the interval of the PPS signal output.
0x073C–0x75C		Reserved
0x0760	**MAC_R472_PPS0_interval**	Register 472 - PPS0 Interval Contains the number of units of sub-second increment value between the rising edges of PPS0 signal output
0x0764	**MAC_R473_timestamp_status**	Register 473 - PPS0 Width Contains the number of units of sub-second increment value between the rising and corresponding falling edges of PPS0 signal output

Table 10-2. Map of the Ethernet Controller MAC Registers.

For Mekra Use Only (08012025)

10.3 Ethernet: Register Details

The register descriptions that follow specify how the application and the core can access the register fields of the control and status registers (CSRs).

Please note the following with regard to the Ethernet control registers:

- All CSR registers are implemented as 32-bit registers and can be accessed in one Read / Write cycle with a 32-bit AHB slave port. If the user initiates a non-32-bit register access, the byte-enable signals are used to qualify the corresponding register bytes. Only Write operations are qualified with byte-enables while read-operations are always 32-bit.
- Note that register bits[7:0] correspond to address[1:0] = 00 in Little Endian mode, while register bits[31:24] correspond to address[1:0] = 00 in Big Endian mode.
- The transfer of particular register contents (e.g., MAC DA address register) to the Transmit or Receive clock domains are initiated when a particular byte is written. This is indicated in the register descriptions, where applicable.
- When any register content is transferred to a different clock domain after a write operation, there should be no further writes to the same location until the first write is updated. Otherwise, the second write operation will not update to the destination clock domain. Thus, the delay between two writes to the same register location should be at least four cycles of the destination clock.

10.3.1 Ethernet Register Details: DMA

This section defines the bits for each DMA register. For AHB slave interfaces, byte, half-word or word accesses are possible.

10.3.1.1 DMA_R0_bus_mode Register

The Bus Mode register (**DMA_R0_bus_mode**) establishes the bus operating modes for the DMA.

Bits	Name	Access	Reset	Description
31:26		R	0	Reserved
25	**aal**	RW	0	Address-Aligned Beats When this bit is set high and the **fb** bit equals 1, the AHB interface generates all bursts aligned to the start address LS bits. If the **fb** bit equals 0, the first burst (accessing the data buffer's start address) is not aligned, but subsequent bursts are aligned to the address.
24	**8xpbl**	RW	0	8xPBL mode When set high, this bit multiplies the programmable burst length (PBL) value (bits [22:17] and bits [13:8]) eight times. Therefore, the DMA will transfer data in 8, 16, 32, 64, 128 and 256 beats depending on the PBL value.

For Sekra Lang only (09012025) Confidential

Bits	Name	Access	Reset	Description
23	**usp**	RW	0	Use Separate PBL When set high, it configures the RXDMA to use the value configured in bits [22:17] or **rpbl** as PBL while the PBL value in bits [13:8] or **pbl** is applicable to TXDMA operations only. When reset to low, the PBL value in **pbl** or bits [13:8] is applicable for both DMA engines.
22:17	**rpbl**	RW	0x01	RXDMA programmable burst length (RPBL) These bits indicate the maximum number of beats to be transferred in one RXDMA transaction. This will be the maximum value that is used in a single block Read / Write. The RXDMA will always attempt to burst as specified in RPBL each time it starts a Burst transfer on the host bus. RPBL can be programmed with permissible values of 1, 2, 4, 8, 16, and 32. Any other value will result in undefined behavior. These bits are valid and applicable only when USP is set high.
16	**fb**	RW	0	Fixed Burst This bit controls whether the AHB Master interface performs fixed burst transfers or not. When set, the AHB will use only SINGLE, INCR4, INCR8 or INCR16 during start of normal burst transfers. When reset, the AHB will use SINGLE and INCR burst transfer operations.
15:14	**pr**	RW	0	RX:TX priority ratio RXDMA requests are given priority over TXDMA requests in the following ratio. This is valid only when the **da** bit is reset. 0x0 - 1:1 0x1 - 2:1 0x2 - 3:1 0x3 - 4:1
13:8	**pbl**	RW	0x01	Programmable Burst Length These bits indicate the maximum number of beats to be transferred in one DMA transaction. This will be the maximum value that is used in a single block Read / Write. The DMA will always attempt to burst as specified in **pbl** each time it starts a Burst transfer on the host bus. The bit **pbl** can be programmed with permissible values of 1, 2, 4, 8, 16, and 32. Any other value will result in undefined behavior. When USP is set high, this **pbl** value is applicable for TXDMA transactions only. The **pbl** values have the following limitations. The maximum number of beats (**pbl**) possible is limited by the size of the TX FIFO and RX FIFO in the MTL layer and the data bus width on the DMA. The FIFO has a constraint that the maximum beat supported is half the depth of the FIFO. For different data bus widths and FIFO sizes, the valid **pbl** range (including x4 mode) is provided in the following table (Table 10-4). If **pbl** is common for both transmit and receive DMA, the minimum RX FIFO and TX FIFO depths must be considered. Do not program out-of-range **pbl** values, as this may result in system instability.

For Mercedes Confidential 1.5 (08/20/2020) Last Update

Bits	Name	Access	Reset	Description
7	**atds**	RW	0	Alternate Descriptor Size When set, the size of the alternate descriptor increases to 32 bytes (8 DWORDS). This is required when the IPC Full Checksum Offload Engine (Type 2) is enabled in the receiver. The enhanced descriptor is not required if IPC Full Checksum Offload Engine (Type 2) features are not enabled. In such case, the 16-byte descriptor can be used to save 4 bytes of memory. When reset, the descriptor size reverts back to 4 DWORDs (16 bytes).
6:2	**dsl**	RW	0	Descriptor Skip Length This bit specifies the number of Word / DWord / LWord (depending on 32/64/128-bit bus) to skip between two unchained descriptors. The address skipping begins from the end of the current descriptor to the start of the next descriptor. When **dsl** value equals zero, then the descriptor table is taken as contiguous by the DMA, in Ring mode.
1	**da**	RW	0	DMA Arbitration scheme 0 - Round-robin with RX:TX priority given in bits [15:14] 1 - RX has priority over TX
0	**swr**	R_WS_SC	0	Software Reset When this bit is set, the MAC DMA controller resets all MAC subsystem internal registers and logic. It is cleared automatically after the reset operation has completed in all core clock-domains.

Table 10-3. Ethernet Controller DMA Register 0 (Bus Mode) Which Controls the Host Interface Mode.

Data Bus Width	FIFO Depth	Valid PBL Range in Full-Duplex Mode	Valid PBL Range in Half-Duplex Mode
64	4KB	All	All

Table 10-4. Ethernet PBL Values.

10.3.1.2 DMA_R1_tx_poll Register

The Transmit Poll Demand register (**DMA_R1_tx_poll**) enables the Transmit DMA to verify whether or not the current descriptor is owned by DMA.

Bits	Name	Access	Reset	Description
31:0	**tpd**	RO_WT	0	Transmit Poll Demand When these bits are written with any value, the DMA reads the current descriptor pointed to by the Current Host Transmit Descriptor register or **DMA_R18_host_tx_descript**. If that descriptor is not available (owned by Host), transmission returns to the Suspend state and the DMA Status register (**DMA_R5_status**[2]) is asserted. If the descriptor is available, transmission resumes.

Table 10-5. Ethernet Controller DMA Register 1 is Used to Instruct the DMA to Poll the Transmit Descriptor List.

For Mental Image only (08/12/2025)

10.3.1.3 DMA_R2_rx_poll Register

The Receive Poll Demand register (**DMA_R2_rx_poll**) enables the receive DMA to check for new descriptors.

Bits	Name	Access	Reset	Description
31:0	**rpd**	RO_WT	0	Receive Poll Demand When these bits are written with any value, the DMA reads the current descriptor pointed to by **DMA_R19_host_rx_descript**. If that descriptor is not available (owned by Host), reception returns to the Suspended state and the DMA status register (**DMA_R5_status**[7]) is not asserted. If the descriptor is available, reception resumes.

Table 10-6. Ethernet Controller DMA Register 2 is Used to Instruct the DMA to Poll the Receive Descriptor List.

10.3.1.4 DMA_R3_rx_descriptor Register

The Receive Descriptor List Address register (**DMA_R3_rx_descriptor**) points to the start of the Receive Descriptor List. The descriptor lists reside in the host's physical memory space and must be Word / DWord / LWord-aligned (for 32 / 64 / 128-bit data bus). The DMA internally converts the lists to bus-width aligned addresses by making the corresponding LS bits low. Writing to **DMA_R3_rx_descriptor** is permitted only when reception is stopped. When stopped, **DMA_R3_rx_descriptor** must be written to before the receive Start command is given.

Bits	Name	Access	Reset	Description
31:0	**srl**	RW	0	Start of Receive List This field contains the base address of the First Descriptor in the Receive Descriptor list. The LSB bits [1/2/3:0] for 32/64/128-bit bus-width) will be ignored and taken as all-zero by the DMA internally. Hence these LSB bits are read only.

Table 10-7. Ethernet Controller DMA Register 3 Points the DMA to the Start of the Receive Descriptor List.

10.3.1.5 DMA_R4_tx_descriptor Register

The Transmit Descriptor List Address register (**DMA_R4_tx_descriptor**) points to the start of the Transmit Descriptor List. The descriptor lists reside in the host's physical memory space and must be Word / DWord / LWord-aligned (for 32 / 64 / 128-bit data bus). The DMA internally converts the lists to bus-width aligned addresses by making the corresponding LSB bits low. Writing to **DMA_R4_tx_descriptor** is permitted only when transmission has stopped. When stopped, **DMA_R4_tx_descriptor** can be written before the transmission Start command is given.

Bits	Name	Access	Reset	Description
31:0	**start_tx**	RW	0	Start of Transmit List This field contains the base address of the first descriptor in the Transmit Descriptor list. The LSB bits [1/2/3:0] for 32/64/128-bit bus-width) will be ignored and taken as all-zero by the DMA internally. Hence these LSB bits are read only.

Table 10-8. Ethernet Controller DMA Register 4 Points the DMA to the Start of the Transmit Descriptor List.

For Confidential Use Only (08/07/2025)

10.3.1.6 DMA_R5_status Register

The Status register (**DMA_R5_status**) contains all the status bits that the DMA reports to the host. This Register 5 (**DMA_R5_status**) is typically read by the software driver during an interrupt service routine or polling. Most of the fields in this register cause the host to be interrupted. **DMA_R5_status** bits are not cleared when read. Writing 1 to (unreserved) bits in this **DMA_R5_status**[16:0] clears them and writing 0 has no effect. Each field (bits[16:0]) can be masked by masking the appropriate bit in **DMA_R7_interrupt_enable**.

Bits	Name	Access	Reset	Description
31:30				Reserved
29	**tti**	R	0	Timestamp Trigger Interrupt This bit indicates an interrupt event in the Timestamp Generator block of the DWC_gmac. The software must read the corresponding registers in the DWC_gmac to get the exact cause of the interrupt and clear its source to reset this bit to 1'b0. When this bit is high, the interrupt signal from the DWC_gmac subsystem (sbd_intr_o) is high. This bit is applicable only when the IEEE 1588 Timestamp feature is enabled. Otherwise, this bit is reserved.
28	**gpi**	R	0	MAC PMT Interrupt This bit reflects the PMT_INTR_O signal output of the MAC core. The software must read the corresponding registers in the MAC core to retrieve the exact cause of interrupt and clear the source of interrupt to make this bit 0. The interrupt signal from the MAC subsystem (SBD_INTR_O) is high when this bit is high.
27	**gmi**	R	0	MAC MMC Interrupt This bit reflects an interrupt event in the MMC module of the MAC core. The software must read the corresponding registers in the MAC core to retrieve the exact cause of interrupt and clear the source of interrupt to make this bit 0. The interrupt signal from the MAC subsystem (SBD_INTR_O) is high when this bit is high.
26	**gli**	R	0	MAC Line interface Interrupt This bit reflects an interrupt event in the PCS interface block of the MAC Core. The software must read the corresponding registers in the MAC core to retrieve the exact cause of interrupt and clear the source of interrupt to make this bit 0. The interrupt signal from the MAC subsystem (SBD_INTR_O) is high when this bit is high.

For Meeting Confidential
1.5 (08/16/2019)

Bits	Name	Access	Reset	Description
25:23	**eb**	R	0	Error Bits These bits indicate the type of error that triggered a Bus Error (error response on the AHB interface). Valid only with Fatal Bus Error bit (**DMA_R5_status**[13]) set. This field does not generate an interrupt. Bit 23 0 - Error during data transfer by RXDMA 1 - Error during data transfer by TXDMA Bit 24 0 - Error during write transfer 1 - Error during read transfer Bit 25 0 - Error during data-buffer access 1 - Error during descriptor access
22:20	**ts**	R	0	Transmit Process State These bits indicate the Transmit DMA FSM state. This field does not generate an interrupt. 0x0 - Stopped; Reset or Stop Transmit Command issued. 0x1 - Running; Fetching Transmit Transfer Descriptor. 0x2 - Running; Waiting for status. 0x3 - Running; Reading Data from host memory buffer and queuing it to transmit buffer (TX FIFO). 0x4, 0x5 - Reserved. 0x6 - Suspended; Transmit Descriptor Unavailable or Transmit Buffer Underflow. 0x7 - Running; Closing Transmit Descriptor.
19:17	**rs**	R	0	Receive Process State These bits indicate the Receive DMA FSM state. This field does not generate an interrupt. 0x0 - Stopped: Reset or Stop Receive Command issued. 0x1 - Running: Fetching Receive Transfer Descriptor. 0x2 - Reserved. 0x3 - Running: Waiting for receive packet. 0x4 - Suspended: Receive Descriptor Unavailable. 0x5 - Running: Closing Receive Descriptor. 0x6 - Reserved. 0x7 - Running: Transferring the receive packet data from receive buffer to host memory.
16	**nis**	R_WC	0	Normal Interrupt Summary Normal Interrupt Summary bit value is the logical OR of the following when the corresponding interrupt bits are enabled in **DMA_R7_interrupt_enable**: **DMA_R5_status**[0] - Transmit Interrupt **DMA_R5_status**[2] - Transmit Buffer Unavailable **DMA_R5_status**[6] - Receive Interrupt **DMA_R5_status**[14] - Early Receive Interrupt Only unmasked bits affect the Normal Interrupt Summary bit. This is a sticky bit and must be cleared (by writing 1 to this bit) each time a corresponding bit that causes **nis** to be set is cleared.

For Mentor Confidential 1.5 (08/16/2013)

Bits	Name	Access	Reset	Description
15	**ais**	R_WC	0	Abnormal Interrupt Summary Bit value is the logical OR of the following when the corresponding interrupt bits are enabled in **DMA_R7_interrupt_enable**: **DMA_R5_status**[1] - Transmit Process Stopped **DMA_R5_status**[3] - Transmit Jabber Timeout **DMA_R5_status**[4] - Receive FIFO Overflow **DMA_R5_status**[5] - Transmit Underflow **DMA_R5_status**[7] - Receive Buffer Unavailable **DMA_R5_status**[8] - Receive Process Stopped **DMA_R5_status**[9] - Receive Watchdog Timeout **DMA_R5_status**[10] - Early Transmit Interrupt **DMA_R5_status**[13] - Fatal Bus Error Only unmasked bits affect the Abnormal Interrupt Summary bit. This bit and must be cleared each time a corresponding bit that causes **ais** to be set is cleared.
14	**eri**	R_SS_SC_WC	0	Early Receive Interrupt This bit indicates that the DMA has filled the first data buffer of the packet. Receive Interrupt **DMA_R5_status**[6] automatically clears this bit.
13	**fbi**	R_WC	0	Fatal Bus Error Interrupt This bit indicates that a Bus Error occurred (**DMA_R5_status**[25:23]). When this bit is set, the DMA disables all its bus accesses.
12:11				Reserved
10	**eti**	R_WC	0	Early Transmit Interrupt This bit indicates that the frame to be transmitted was fully transferred to the MTL Transmit FIFO
9	**rwt**	R_WC	0	Receive Watchdog Timeout This bit is asserted when a frame with a length greater than 2048 bytes is received.
8	**rps**	R_WC	0	Receive Process Stopped This bit is asserted when the Receive Process enters the Stopped state.
7	**ru**	R_WC	0	Receive Buffer Unavailable This bit indicates that the next descriptor in the Receive List is owned by the host and cannot be acquired by the DMA. The Receive Process is suspended. To resume processing Receive descriptors, the host should change the ownership of the descriptor and issue a Receive Poll Demand command. If no Receive Poll Demand is issued, Receive Process resumes when the next recognized incoming frame is received. **DMA_R5_status**[7] is set only when the previous Receive Descriptor was owned by the DMA.
6	**ri**	R_WC	0	Receive Interrupt This bit indicates the completion of frame reception. Specific frame status information has been posted in the descriptor. Reception remains in the Running state.

For Mentor Confidential 1.5 (08/08/2014) Only

Bits	Name	Access	Reset	Description
5	**unf**	R_WC	0	Transmit Underflow This bit indicates that the Transmit Buffer experienced an underflow during frame transmission. Transmission is suspended and an Transmit Descriptor Underflow Error is set using Transmit Descriptor 0 bit 1.
4	**ovf**	R_WC	0	Receive Overflow This bit indicates that the Receive Buffer experienced an overflow during frame reception. If the partial frame is transferred to the application, the Receive Descriptor overflow status is set using Receive Descriptor 0 bit 11.
3	**tjt**	R_WC	0	Transmit Jabber Timeout This bit indicates that the Transmit Jabber Timer expired, meaning that the transmitter had been excessively active. The transmission process is aborted and placed in the Stopped state. This causes the Transmit Jabber Timeout flag to assert (i.e., Transmit Descriptor 0 bit 14).
2	**tu**	R_WC	0	Transmit Buffer Unavailable This bit indicates that the Next Descriptor in the Transmit List is owned by the host and cannot be acquired by the DMA. Transmission is suspended. Bits[22:20] explain the Transmit Process state transitions. To resume processing transmit descriptors, the host should change the ownership of the bit of the descriptor and then issue a Transmit Poll Demand command.
1	**tps**	R_WC	0	Transmit Process Stopped This bit is set when the transmission is stopped
0	**ti**	R_WC	0	Transmit Interrupt This bit indicates that frame transmission is complete and Transmit Descriptor 1 bit 31 is set

Table 10-9. Ethernet Controller DMA Register 5 is Read to Determine the Status of the DMA.

For Meeting only 1.5 / 08/ 2025

10.3.1.7 DMA_R6_operation_mode Register

The Operation Mode register (**DMA_R6_operation_mode**) establishes the Transmit and Receive operating modes and commands. This register should be the last CSR to be written as part of DMA initialization.

Bits	Name	Access	Reset	Description
31:27				Reserved
26	**dt**	RW	0	Disable dropping of TCP/IP checksum error frames When this bit is set, the core does not drop frames that only have errors detected by the Receive Checksum Offload engine. Such frames do not include errors (including FCS errors) in the Ethernet frame received by the MAC, but have errors in the encapsulated payload only. When this bit is reset, all error frames are dropped if the **fef** bit is reset.

Bits	Name	Access	Reset	Description
25	**rsf**	RW	0	Receive Store and Forward When this bit is set, the MTL only reads a frame from the RX FIFO after the complete frame has been written to it, ignoring RTC bits. When this bit is reset, the RX FIFO operates in Cut-Through mode, subject to the threshold specified by the RTC bits.
24	**dff**	RW	0	Disable Flushing of Received Frames When this bit is set, the RXDMA does not flush any frames due to the unavailability of receive descriptors / buffers as it does normally when this bit is reset.
23:22				Reserved
21	**tsf**	RW	0	Transmit Store and Forward When this bit is set, transmission begins when a full frame resides in the MTL Transmit FIFO. When this bit is set, the **ttc** values specified in **DMA_R6_operation_mode**[16:14] are ignored. This bit should be changed only when transmission is stopped.
20	**ftf**	R_WS_SC	0	Flush Transmit FIFO When this bit is set, the transmit FIFO controller logic is reset to its default values and thus all data in the TX FIFO is lost/flushed. This bit is cleared internally when the flushing operation is completed. **DMA_R6_operation_mode** should not be written to until this bit is cleared.
19:17				Reserved
16:14	**ttc**	RW	0	Transmit Threshold Control These three bits control the threshold level of the MTL Transmit FIFO. Transmission begins when the frame size within the MTL Transmit FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are also transmitted. These bits are used only when the corresponding store-forward bit is reset. 0x0 - 64 0x1 - 128 0x2 - 192 0x3 - 256 0x4 - 40 0x5 - 32 0x6 - 24 0x7 - 16

For Mentor Internal Use only (1.5 / 08/29/2025)

Bits	Name	Access	Reset	Description
13	**st**	RW	0	Start/Stop Transmission Command When this bit is set, transmission is placed in the running state, and the DMA checks the Transmit List at the current position for a frame to be transmitted. Descriptor acquisition is attempted either from the current position in the list, which is the Transmit List Base Address set by **DMA_R4_tx_descriptor**, or from the position retained when transmission was stopped previously. If the current descriptor is not owned by the DMA, transmission enters the Suspended state and the Transmit Buffer Unavailable (**DMA_R5_status**[2]) bit is set. The Start Transmission command is effective only when transmission is stopped. If the command is issued before setting **DMA_R4_tx_descriptor**, this will result in unpredictable DMA behavior. When this bit is reset, the transmission process is placed in the Stopped state after completing the transmission of the current frame. The next descriptor position in the Transmit List is saved, and becomes the current position when transmission resumes. The stop transmission command is effective only when the transmission of the current frame is complete or when the transmission is in the Suspended state.
12:11	**rfd**	RW	0	Threshold for de-activating flow control (in both HD and FD) These bits control the threshold (fill-level of RX FIFO) at which flow-control is deasserted after activation. 0x0 - Full 1 KB 0x1 - Full 2 KB 0x2 - Full 3 KB 0x3 - Full 4 KB Note that the deassertion is effective only after the flow-control is asserted.
10:8				Reserved
7	**fef**	RW	0	Forward Error frames When reset, the RX FIFO will drop frames with error status (CRC error, collision error, RGMII_ER, giant frame, watchdog timeout, overflow). However, if the frame's start byte (write) pointer is already transferred to the Read Controller-side, then the frames will not be dropped. When **fef** is set, all frames, except runt-error frames, will be forwarded to the DMA.
6	**fuf**	RW	0	Forward Undersized good frames When set, the RX FIFO will forward Undersized frames (runt frames containing no errors and with lengths less than 64 bytes, including pad-bytes and CRC). When reset, the RX FIFO will drop all frames of size less than 64 bytes, unless they have already been transferred due to a lower value of Receive Threshold (e.g., RTC = 01).
5				Reserved

For Mercedes Confidential 1.5 (08/20/2015)

Bits	Name	Access	Reset	Description
4:3	**rtc**	RW	0	Receive Threshold control These two bits control the threshold level of the MTL Receive FIFO. Transfer (request) to DMA starts when the frame size within the MTL Receive FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are transferred automatically. 0x0 - 64 0x1 - 32 0x2 - 96 0x3 - 128
2	**osf**	RW	0	Operate on Second Frame When this bit is set, the DMA is instructed to process a second frame of Transmit data even before the status of the first frame is obtained.
1	**sr**	RW	0	Start/Stop Receive When this bit is set, the Receive process is placed in the Running state. The DMA attempts to acquire the descriptor from the Receive list and processes incoming frames. Descriptor acquisition is attempted from the current position in the list, which is the address set in **DMA_R3_rx_descriptor** or the position retained when the Receive process was previously stopped. If no descriptor is owned by the DMA, reception is suspended and the Receive Buffer Unavailable (**DMA_R5_status**[7]) bit is set. The Start Receive command is effective only when reception has stopped. If the command was issued before setting the register **DMA_R3_rx_descriptor**, unpredictable DMA behavior will result. When this bit is cleared, RXDMA operation is terminated following the transfer of the current frame. The next descriptor position in the Receive list is saved and becomes the current position after the Receive process resumes. The Stop Receive command is effective only when the Receive process is in either the Running (waiting for receive packet) or in the Suspended state.
0				Reserved

Table 10-10. Ethernet Controller DMA Register 6 for Receive and Transmit Operating Modes and Commands.

For Mercury Evaluation Only 1.5 (08/25/15)

10.3.1.8 DMA_R7_interrupt_enable Register

The Interrupt Enable register (**DMA_R7_interrupt_enable**) enables the interrupts reported by **DMA_R5_status**. Setting a bit to 0x1 enables a corresponding interrupt. After a hardware or software reset, all interrupts are disabled.

Bits	Name	Access	Reset	Description
31:17				Reserved

Bits	Name	Access	Reset	Description
16	**nie**	RW	0	Normal Interrupt Summary Enable When this bit is set, a normal interrupt is enabled. When this bit is reset, a normal interrupt is disabled. This bit enables the following bits: **DMA_R5_status**[0] - Transmit Interrupt **DMA_R5_status**[2] - Transmit Buffer Unavailable **DMA_R5_status**[6] - Receive Interrupt **DMA_R5_status**[14] - Early Receive Interrupt
15	**aie**	RW	0	Abnormal Interrupt Summary Enable When this bit is set, an abnormal interrupt is enabled. When this bit is reset, an abnormal interrupt is disabled. This bit enables the following bits. **DMA_R5_status**[1] - Transmit Process Stopped **DMA_R5_status**[3] - Transmit Jabber Timeout **DMA_R5_status**[4] - Receive Overflow **DMA_R5_status**[5] - Transmit Underflow **DMA_R5_status**[7] - Receive Buffer Unavailable **DMA_R5_status**[8] - Receive Process Stopped **DMA_R5_status**[9] - Receive Watchdog Timeout **DMA_R5_status**[10] - Early Transmit Interrupt **DMA_R5_status**[13] - Fatal Bus Error
14	**ere**	RW	0	Early Receive Interrupt Enable When this bit is set with the Normal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[16]) bit, Early Receive Interrupt is enabled. When this bit is reset, Early Receive Interrupt is disabled.
13	**fbe**	RW	0	Fatal Bus Error Enable When this bit is set with the Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]) bit, the Fatal Bus Error Interrupt is enabled. When this bit is reset, Fatal Bus Error Enable Interrupt is disabled.
12:11				Reserved
10	**ete**	RW	0	Early Transmit Interrupt Enable When this bit is set with an Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]), Early Transmit Interrupt is enabled. When this bit is reset, Early Transmit Interrupt is disabled.
9	**rwe**	RW	0	Receive Watchdog Timeout Enable When this bit is set with an Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]), the Receive Watchdog Timeout Interrupt is enabled. When this bit is reset, Receive Watchdog Timeout Interrupt is disabled.
8	**rse**	RW	0	Receive Stopped Enable When this bit is set with an Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]), Receive Stopped Interrupt is enabled. When this bit is reset, Receive Stopped Interrupt is disabled.
7	**rue**	RW	0	Receive Buffer Unavailable Enable When this bit is set with an Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]), Receive Buffer Unavailable Interrupt is enabled. When this bit is reset, the Receive Buffer Unavailable Interrupt is disabled.

Bits	Name	Access	Reset	Description
6	**rie**	RW	0	Receive Interrupt Enable When this bit is set with the Normal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[16]) bit, Receive Interrupt is enabled. When this bit is reset, Receive Interrupt is disabled.
5	**une**	RW	0	Underflow Interrupt Enable When this bit is set with the Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]) bit, Transmit Underflow Interrupt is enabled. When this bit is reset, Underflow Interrupt is disabled.
4	**ove**	RW	0	Overflow Interrupt Enable When this bit is set with the Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]) bit, Receive Overflow Interrupt is enabled. When this bit is reset, Overflow Interrupt is disabled.
3	**tje**	RW	0	Transmit Jabber Timeout Enable When this bit is set with the Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]) bit, Transmit Jabber Timeout Interrupt is enabled. When this bit is reset, Transmit Jabber Timeout Interrupt is disabled.
2	**tue**	RW	0	Transmit Buffer Unavailable Enable When this bit is set with the Normal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[16]) bit, Transmit Buffer Unavailable Interrupt is enabled. When this bit is reset, Transmit Buffer Unavailable Interrupt is disabled.
1	**tse**	RW	0	Transmit Stopped Enable When this bit is set with the Abnormal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[15]) bit, Transmission Stopped Interrupt is enabled. When this bit is reset, Transmission Stopped Interrupt is disabled.
0	**tie**	RW	0	Transmit Interrupt Enable When this bit is set with the Normal Interrupt Summary Enable (**DMA_R7_interrupt_enable**[16]) bit, Transmit Interrupt is enabled. When this bit is reset, Transmit Interrupt is disabled.

Table 10-11. Ethernet Controller DMA Register 7 Enables the Interrupts Reported by the Status Register.

For Mem Only (08/15/2019)

10.3.1.9 DMA_R8_fmbuff_count Register

The DMA maintains two counters to track the number of missed frames during reception. This register (**DMA_R8_fmbuff_count**) reports the current value of the counter. The counter is used for diagnostic purposes. Bits[15:0] indicate missed frames due to the unavailability of the host buffer. Bits[27:17] indicate missed frames due to buffer overflow conditions (MTL and MAC) and runt frames (good frames of size less than 64 bytes) dropped by the MTL.

Bits	Name	Access	Reset	Description
31:29				Reserved
28	**over_fifo**	R_WC	0	Overflow bit for FIFO Overflow Counter
27:17	**fms_count**	R_WC	0	Number of frames missed by the application This counter is incremented each time the MTL asserts the sideband signal MTL_RXOVERFLOW_O. The counter is cleared when this register is written with any value with MCI_BE_I[2] as 1.
16	**over_fms_count**	R_WC	0	Overflow bit for Missed Frame Counter
15:0	**fms_cntl**	R_WC	0	Number of frames missed by the controller Indicates the number of frames missed by the controller due to the unavailability of the Host Receive Buffer. This counter is incremented each time the DMA discards an incoming frame. The counter is cleared when this register is written with any value with MCI_BE_I[0] as 1.

Table 10-12. Ethernet Controller DMA Register 8 for the Missed Frame and Buffer Overflow Counter.

10.3.1.10 DMA_R18_host_tx_descript Register

The Current Host Transmit Descriptor register (**DMA_R18_host_tx_descript**) points to the start address of the current Transmit Descriptor read by the DMA.

Bits	Name	Access	Reset	Description
31:0	**clr_txptr**	R	0	Host Transmit Descriptor Address Pointer Cleared on reset. Pointer updated by DMA during operation.

Table 10-13. Ethernet Controller DMA Register 18 Points to the Start of the Host Transmit Descriptor.

10.3.1.11 DMA_R19_host_rx_descript Register

The Current Host Receive Descriptor register (**DMA_R19_host_rx_descript**) points to the start address of the current Receive Descriptor read by the DMA.

Bits	Name	Access	Reset	Description
31:0	**clr_rxptr**	R	0	Host Receive Descriptor Address Pointer Cleared on reset. Pointer updated by DMA during operation.

Table 10-14. Ethernet Controller DMA Register 19 Points to the Start of the Receive Descriptor.

For Media Lane only
Confidential
1.5 (08012025)

10.3.1.12 DMA_R20_host_tx_buffer Register

The Current Host Transmit Buffer Address register (**DMA_R20_host_tx_buffer**) points to the current Transmit Buffer Address being read by the DMA.

Bits	Name	Access	Reset	Description
31:0	**clr_txbuff**	R	0	Host Transmit Buffer Address Pointer Cleared on reset. Pointer updated by DMA during operation.

Table 10-15. Ethernet Controller DMA Register 20 Points to the Transmit Buffer Address.

10.3.1.13 DMA_R21_host_rx_buffer Register

The Current Host Receive Buffer Address register (**DMA_R21_host_rx_buffer**) points to the current Receive Buffer address being read by the DMA.

Bits	Name	Access	Reset	Description
31:0	**clr_rxbuff**	R	0	Host Receive Buffer Address Pointer Cleared on reset. Pointer updated by DMA during operation.

Table 10-16. Ethernet Controller DMA Register 21 Points to the Receive Buffer Address.

10.3.2 Ethernet Register Details: MAC

10.3.2.1 Register 0 (MAC Configuration Register)

MAC Register 0 for MAC Configuration (**MAC_R0_MAC_config**) establishes receive and transmit operating modes.

Bits	Name	Access	Reset	Description
31:25				Reserved
24	**tc**	RW	0	Transmit Configuration in RGMII, SGMII, or SMII When set, this bit enables the transmission of duplex mode, link speed, and link up or down information to the PHY in the RGMII, SMII, or SGMII port. When this bit is reset, no such information is driven to the PHY. This bit is reserved (and RO) if the RGMII, SMII, or SGMII PHY port is not selected during core configuration.
23	**wd**	RW	0	Watchdog Disable When this bit is set, the MAC disables the watchdog timer on the receiver. When this bit is reset, the MAC allows no more than 2048 (10240 if **je** is set high) bytes of the receiving frame and cuts off any bytes subsequently received.
22	**jd**	RW	0	Jabber Disable When this bit is set, the MAC disables the jabber timer on the transmitter. When this bit is reset, the MAC cuts-off the transmitter if the Application sends out more than 2048 (10240 if **je** is set high) bytes of data during transmission.

Bits	Name	Access	Reset	Description
21	**be**	RW	0	Frame Burst Enable When this bit is set, the MAC allows frame-bursting during transmission in RGMII Half-Duplex mode. This bit is reserved in 10 / 100 / 1000-Mbps-only or Full-Duplex-only configurations
20	**je**	RW	0	Jumbo Frame Enable When this bit is set, MAC allows Jumbo frames of size 9018 bytes (9022 bytes for VLAN tagged frames) without reporting a giant frame error in the receive frame status. Jabber Disable bit should be 1 to transmit jumbo frames.
19:17	**ifg**	RW	0	Inter Frame Gap These bits control the minimum **ifg** between frames during transmission. 0x0 - 96 bit times 0x1 - 88 bit times 0x2 - 80 bit times …. 0x7 - 40 bit times Note that, in Half-Duplex mode, the minimum **ifg** can be configured up to 64 bit-times (**ifg** = 100) only.
16	**dcrs**	RW	0	Disable Carrier Sense (CRS) During Transmission When set high, this bit causes the MAC transmitter to ignore the RGMII CRS signal during frame transmission in Half-Duplex mode. This request results in no errors generated due to Loss of Carrier or No Carrier during such transmission. When this bit is low, the MAC transmitter generates such errors due to Carrier Sense and will even abort these transmissions.
15	**ps**	RW	0	This bit selects the Ethernet line speed. 0: For 1000 Mbps operations 1: For 10 or 100 Mbps operations In 10 or 100 Mbps operations, this bit, along with FES bit, selects the exact line speed.
14	**fes**	RW	0	Speed This bit selects the speed of the MII, RMII, SMII, RGMII, SGMII, or RevMII interface. 0: 10 Mbps 1: 100 Mbps This bit is enabled only when the RMII, SMII, RGMII, SGMII, or RevMII interface is enabled during core configuration. This bit generates link speed encoding when TC (bit 24) is set in RMII, SMII, or SGMII mode. This bit is always driven for RevMII. If the MAC is configured with an RMII, SMII, SGMII, or RGMII PHY interface, this bit can optionally be driven as an output signal (MAC_SPEED_O[0]) to reflect the value of this bit in the MAC_SPEED_O signal. Moreover, this bit is reserved when RMII is enabled without enabling the Output Port for Speed Selection option during core configuration.

For Mentor Internal Use Only 1.5 (08/03/2015)

Bits	Name	Access	Reset	Description
13	**do**	RW	0	Disable Receive Own When this bit is set, the MAC disables the reception of frames when the RGMII_TXEN_O is asserted in Half-Duplex mode. When this bit is reset, the MAC receives all packets that are provided by the PHY while transmitting. This bit is not applicable (R with default value) if the MAC is operating in Full-Duplex mode.
12	**lm**	RW	0	Loop-back Mode When this bit is set, the MAC operates in loop-back mode at RGMII. The RGMII Receive clock input (CLK_RX_I) is required for the loopback to function properly, as the Transmit clock is not looped-back internally.
11	**dm**	RW	0	Duplex Mode When this bit is set, the MAC operates in a Full-Duplex mode, allowing it to transmit and receive simultaneously. This bit is R with default value of 1 in Full-Duplex-only configuration.
10	**ipc**	RW	0	Checksum Offload When this bit is set, the MAC calculates the 16-bit 1's complement of the 1's complement sum of the payload data (16-bit) and sends the result to the application at the end of frame. It also verifies whether the IPv4 Header checksum (assumed to be bytes 25-26 or 29-30 (VLAN-tagged) of Received Ethernet frame) is correct for the received frame and provides the status in the Receive Status Word. In addition, the MAC Core appends the 16-bit Checksum calculated for the payload of the IP datagram (bytes after the header) to the Ethernet frame transferred to the application. When this bit is reset, then this function is disabled.
9	**dr**	RW	0	Disable Retry When this bit is set, the MAC will attempt only 1 transmission. When a collision occurs on the RGMII, the MAC will ignore the current frame transmission and report a Frame Abort with excessive collision error in the transmit frame status. When this bit is reset, the MAC will attempt retries based on the settings of **bl**. This bit is applicable only in Half-Duplex mode and is reserved in Full-Duplex-only configuration.
8		R	0	Reserved
7	**acs**	RW	0	Automatic Pad/CRC Stripping When this bit is set, the MAC will strip the Pad/FCS field on incoming frames only if the length's field-value is less than or equal to 1500 bytes. All received frames with a length-field greater than or equal to 1501 bytes will be passed to the application without the Pad/FCS field being stripped. When this bit is reset, the MAC will pass all incoming frames to the Host unmodified.

Bits	Name	Access	Reset	Description
6:5	**bl**	RW	0	Back-off Limit The Back-off limit determines the random integer number (r) of slot time delays (512 bit times for 10/100/1000 Mbps) the MAC waits prior to rescheduling a transmission attempt during retries after a collision. This bit is applicable only in Half-Duplex mode and is reserved (R) in Full-Duplex-only configuration. 0x0 - k = min (n, 10) 0x1 - k = min (n, 8) 0x2 - k = min (n, 4) 0x3 - k = min (n, 1) where n = retransmission attempt. The random integer r takes the value in the range 0 =< r < 2^k
4	**dc**	RW	0	Deferral Check When this bit is set, the deferral check function is enabled in the MAC. The MAC will issue a Frame Abort status, along with the excessive deferral error bit set in the transmit frame status when the transmit state machine is deferred for more than 24,288 bit-times in 10/100/1000 Mbps mode. If Jumbo frame mode is enabled in 10/100/1000-Mbps mode, the threshold for deferral is 155,680 bit-times. Deferral begins when the transmitter is ready to transmit, but is prevented because of an active carrier-sense (CRS) signal on the RGMII. Defer time is not cumulative. If the transmitter defers for 10,000 bit-times, then transmits, collides, backs off, and must defer again after completion of back-off, the deferral timer resets to 0 and restarts. When this bit is reset, the deferral check function is disabled and the MAC defers until the CRS signal goes inactive. This bit is applicable only in Half-Duplex mode and is reserved (R) in Full-Duplex-only configuration.
3	**te**	RW	0	Transmitter Enable When this bit is set, the transmit state machine of the MAC is enabled for transmission on the RGMII. When this bit is reset, the MAC transmit state machine is disabled after the completion of the transmission of the current frame, and will not transmit any further frames.
2	**re**	RW	0	Receiver Enable When this bit is set, the receiver state machine of the MAC is enabled for receiving frames from the RGMII. When this bit is reset, the MAC receive state machine is disabled after the completion of reception of the current frame, and will not receive further frames.
1:0				Reserved

Table 10-17. Ethernet Controller MAC Register 0 (MAC Configuration) Sets the Operation Mode.

10.3.2.2 MAC_R1_MAC_fm_filter Register

MAC Register 1 is the MAC Frame Filter register (**MAC_R1_MAC_fm_filter**) which contains the filter controls for receiving frames. Some of the controls from this register go to the address check block of the MAC, which performs the first level of address filtering. The second level of filtering is performed on the incoming frame, based on other controls such as Pass Bad Frames and Pass Control Frames.

Bits	Name	Access	Reset	Description
31	**ra**	RW	0	Receive All When this bit is set, the MAC Receiver module passes to the application all frames received irrespective of whether they pass the address filter or not. The result of the SA/DA Filtering is updated (pass or fail) in the corresponding bits in the Receive Status Word. When this bit is reset, the Receiver module passes to the Application only those frames that pass the SA/DA address filter.
30:11				Reserved
10	**hpf**	RW	0	Hash or Perfect Filter When set, this bit configures the address filter to pass a frame if it matches either the perfect filtering or the hash filtering requirements as set by **hmc** or **huc** bits. When set low and if the **huc**/**hmc** bit is set, the frame is passed only if it matches the Hash filter.
9	**saf**	RW	0	Source Address Filter Enable When this bit is set, the MAC Core compares the SA field of the received frames with the values programmed in the enabled SA registers. If the comparison fails, the MAC drops the frame. When this bit is reset, the MAC Core forwards the received frame to the application along with the updated SAF bit of the RxStatus, depending on the SA address comparison
8	**saif**	RW	0	SA Inverse Filtering When this bit is set, the Address Check block operates in inverse filtering mode for the SA address comparison. The frames whose SA matches the SA registers will be marked as failing the SA Address filter. (See **MAC_R16_MAC_add0_high** and **MAC_R17_MAC_add0_low**). When this bit is reset, frames whose SA does not match the SA registers will be marked as failing the SA Address filter.
7:6	**pcf**	RW	0	Pass Control Frames These bits control the forwarding of all control frames (including unicast and multicast PAUSE frames). Note that the processing of PAUSE control frames depends only on **rfe** of **MAC_R6_flow_control**[2]. 0x0, 0x1 - MAC prevents all control frames from reaching the application 0x2 - MAC forwards all control frames to the application even if they fail the Address Filter 0x3 - MAC only forwards control frames that pass the Address Filter.
5	**dbf**	RW	0	Disable Broadcast Frames When this bit is set, the AFM module filters all incoming broadcast frames. When this bit is reset, the AFM module passes all received broadcast frames.
4	**pm**	RW	0	Pass All Multicast When set, this bit indicates that all received frames with a multicast destination address (first bit in the destination address field is 1) are passed. When reset, filtering of multicast frame depends on **hmc** bit.

Bits	Name	Access	Reset	Description
3	**daif**	RW	0	DA Inverse Filtering When this bit is set, the Address Check block operates in inverse filtering mode for DA address comparison of both unicast and multicast frames. When reset, normal filtering of frames is performed.
2	**hmc**	RW	0	Hash MultiCast When set, the MAC performs destination address filtering of received multicast frames according to the hash table. When reset, the MAC performs perfect destination address filtering for multicast frames (i.e, compares the DA field with the values programmed in DA registers.) See **MAC_R16_MAC_add0_high** and **MAC_R17_MAC_add0_low**.
1	**huc**	RW	0	Hash UniCast When set, the MAC performs destination address filtering of unicast frames according to the hash table. When reset, the MAC performs perfect destination address filtering for unicast frames; i.e, compares the DA field with the values programmed in DA registers. (See **MAC_R16_MAC_add0_high** and **MAC_R17_MAC_add0_low**).
0	**pr**	RW	0	Promiscuous Mode When this bit is set, the Address Filter module passes all incoming frames regardless of their destination or source address. The SA / DA Filter Fails status bits of the Receive Status Word will always be cleared when **pr** is set.

Table 10-18. Ethernet Controller MAC Register 1 (MAC Frame Filter) Contains Frame Filtering Controls.

For Mentor Language Only (08012020) Confidential 1.5

10.3.2.3 MAC_R2_hash_high Register

The 64-bit Hash Table is used for group address filtering. For hash filtering, the contents of the destination address in the incoming frame is passed through the CRC logic, and the upper 6 bits of the CRC register are used to index the contents of the Hash table. The most significant bit determines the register to be used (Hash Table High or Hash Table Low), and the other 5 bits determine the bit within the register.

A hash value of 0 results in the selection of bit 0 of the register, and a value of 0x3F results in the selection of bit 31. For example, if the DA of the incoming frame is received as 0x1F52419CB6AF (0x1F is the first byte received on RGMII interface), then the internally calculated 6-bit hash value is 0x2C and the **MAC_R2_hash_table_high** register bit[12] is checked for filtering. If the DA of the incoming frame is received as 0xA00A98000045, then the calculated 6-bit hash value is 0x07 and **MAC_R3_hash_table_low** register bit[7] is checked for filtering. If the corresponding bit value of the register is 1, the frame is accepted. Otherwise, it is rejected.

If the **pm** (Pass All Multicast) bit is set in **MAC_R1_MAC_fm_filter**, then all multicast frames are accepted regardless of the multicast hash values. If the Hash Table register is configured to be double-synchronized to the RGMII clock domain, the synchronization is triggered only when Bits[31:24] (in Little Endian mode) or Bits[7:0] (in Big Endian mode) of the Hash Table High/Low Registers are written to. Please note that consecutive writes to these registers should be performed only after at least four clock cycles in the destination clock domain when double-synchronization is enabled.

The table below provides details for the MAC Hash Table High Register (**MAC_R2_hash_table_high**). This register contains the higher 32 bits of the Hash table.

Bits	Name	Access	Reset	Description
31:0	**hth**	RW	0	Hash Table High This field contains the upper 32 bits of Hash table

Table 10-19. Ethernet Controller MAC Register 2 Contains the High 32 Bits of the Multicast Hash Table.

10.3.2.4 MAC_R3_hash_table_low Register

The Hash Table Low register (**MAC_R3_hash_table_low**) contains the lower 32 bits of the Hash table.

Bits	Name	Access	Reset	Description
31:0	**htl**	RW	0	Hash Table Low This field contains the lower 32 bits of Hash table.

Table 10-20. Ethernet Controller MAC Register 3 Contains the Low 32 bits of the Multicast Hash Table.

10.3.2.5 MAC_R4_GMII_address Register

The GMII Address register (**MAC_R4_GMII_address**) controls the management cycles to the external PHY through the management interface. If CSR clock is over 300MHz, use AHBSP_NONSEC_GMII_ADDR_REG register in non-secure AHB scratchpad instead.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:11	**pa**	RW	0	Physical Layer Address This field indicates which of the 32 possible PHY devices are being accessed
10:6	**gr**	RW	0	GMII Register These bits select the desired GMII register in the selected PHY device
5				Reserved
4:2	**cr**	RW	0	CSR Clock Range The CSR Clock Range selection determines the CLK_CSR_I frequency and is used to select the frequency of the MDC clock: Selection CLK_CSR_I MDC Clock 0x0 - 60 MHz to 100 MHz CLK_CSR_I/42 0x1 - 100 MHz to 150 MHz CLK_CSR_I/62 0x2 - 20 MHz to 35 MHz CLK_CSR_I/16 0x3 - 35 MHz to 60 MHz CLK_CSR_I/26 0x4 - 150 MHz to 250 MHz CLK_CSR_I/102 0x5 - 250 MHz to 300 MHz CLK_CSR_I/122 0x6, 0x7 - Reserved Note: CLK_CSR_I is equivalent to gclk_core if the peripheral clock mode is 1, or it is equivalent to gclk_core/2 if the peripheral clock mode is 0.

Confidential
For Meeting Only
Ambarella Confidential

Bits	Name	Access	Reset	Description
1	**gw**	RW	0	GMII Write When set, this bit indicates to the PHY that this will be a Write operation using the GMII Data register (**MAC_R5_GMII_data**). If this bit is not set, this will be a Read operation, resulting in the placement of data in the GMII Data register.
0	**gb**	R_WS_SC	0	GMII Busy This bit should read a logic 0 before writing to **MAC_R4_GMII_address** and the GMII Data register (**MAC_R5_GMII_data**). This bit must also be set to 0 during a Write to **MAC_R4_GMII_address**. During PHY register access, this bit will be set to 1 by the application to indicate that a Read or Write access is in progress. **MAC_R5_GMII_data** (GMII Data) should be kept valid until this bit is cleared by the MAC during a PHY Write operation. The **MAC_R5_GMII_data** bit is invalid until cleared by the MAC during a PHY Read operation. The **MAC_R4_GMII_address** should not be written to until this bit is cleared.

Table 10-21. Ethernet Controller MAC Register 4 (GMII Address) for Management Cycles to External PHY.

10.3.2.6 MAC_R5_GMII_data Register

The GMII Data register (**MAC_R5_GMII_data**) stores Write data to be written to the PHY register located at the address specified in **MAC_R4_GMII_address**. **MAC_R5_GMII_data** also stores Read data from the PHY register located at the address specified by **MAC_R4_GMII_address**. If CSR clock is over 300MHz, use AHBSP_NONSEC_GMII_DATA_REG register in non-secure AHB scratchpad instead.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**gd**	RW	0	GMII Data This contains the 16-bit data value read from the PHY after a Management Read operation or the 16-bit data value to be written to the PHY prior to a Management Write operation.

Table 10-22. Ethernet Controller MAC Register 5 (GMII Data) Stores Read or Write Data.

10.3.2.7 MAC_R6_flow_control Register

The Flow Control register (**MAC_R6_flow_control**) controls the generation and receipt of the Control (Pause Command) frames by the MAC's flow-control module. A Write made to a register with the Busy bit (**fcb**) set to 1 triggers the Flow Control block to generate a Pause Control frame. The fields of the control frame are selected as specified in the *802.3x* specification, and the Pause Time value from this register is used in the Pause Time field **pt** of the control frame. The Busy bit remains set until the control frame is transferred onto the cable. The Host must ensure that the Busy bit is cleared before writing to the register.

Confidential
For Mekuang only
1 Preliminary

Bits	Name	Access	Reset	Description
31:16	**pt**	RW	0	Pause Time This field holds the value to be used in the Pause Time field in the transmit control frame. If the Pause Time bit is configured to be double-synchronised to the GMII clock domain, then consecutive writes to this register should be performed only after at least 4 clock cycles in the destination clock domain.
15:8				Reserved
7	**dzpq**	RW	0	Disable Zero-Quanta Pause When set, this bit disables the automatic generation of Zero-Quanta Pause Control frames on the deassertion of the flow-control signal from the FIFO layer (MTL or external sideband flow control signal SBD_FLOWCTRL_I/MTI_FLOWCTRL_I). When this bit is reset, normal operation with automatic Zero-Quanta Pause Control frame generation is enabled.
6				Reserved
5:4	**plt**	RW	0	Pause Low threshold This field configures the threshold of the PAUSE timer at which the input flow control signal MTI_FLOWCTRL_I (or SBD_FLOWCTRL_I) is checked for automatic re-transmission of the PAUSE Frame. The threshold values should always be greater than the Pause Time **pt** configured in Bits[31:16]. For example, if **pt** = 0x100 (256 slot-times), and **plt** = 01, then a second PAUSE frame is automatically transmitted if the MTI_FLOWCTRL_I signal is asserted at 228 (256-28) slot-times after the first Pause-frame is transmitted. Selection Threshold: 0x0 - Pause Time at 4 Slot time 0x1 - Pause Time at 28 Slot time 0x2 - Pause Time at 144 Slot time 0x3 - Pause Time at 256 Slot time Slot time is defined as the time required to transmit 512 bits (64 bytes) on the RGMII interface.
3	**up**	RW	0	Unicast Pause Frame detect When this bit is set, the MAC will detect the Pause frames with the station's unicast address specified in MAC Address0 High Register (**MAC_R18_MAC_add1_high**) and MAC Address0 Low Register (**MAC_R17_MAC_add0_low**), in addition to detecting Pause frames with the unique multicast address. When this bit is reset, the MAC will detect only a Pause frame with the unique multicast address specified in the *802.3x* standard.
2	**rfe**	RW	0	Receive Flow Control Enable When this bit is set, the MAC will decode the received Pause frame and disable its transmitter for a specified (Pause Time **pt**) period. When this bit is reset, the decode function of the Pause frame is disabled.
1	**tfe**	RW	0	Transmit Flow Control Enable In Full-Duplex mode, when this bit is set, the MAC enables the flow-control operation to transmit Pause frames. When this bit is reset, the flow-control operation in the MAC is disabled, and the MAC will not transmit any Pause frames. In Half-Duplex mode, when this bit is set, the MAC enables the back-pressure operation. When this bit is reset, the backpressure feature is disabled.

For Mem... 1.5 (08...

Bits	Name	Access	Reset	Description
0	**fcb/bpa**	RW	0	Flow Control Busy/Back-Pressure Activate This bit initiates a Pause Control frame in Full-duplex mode and activates the back-pressure function in Half-duplex mode if the **tfe** bit is set. In full-duplex mode, this bit should be read as 0 before writing to the Flow Control register **MAC_R6_flow_control**. To initiate a Pause control frame, the Application must set this bit to 1. During a transfer of the Control Frame, this bit will continue to be set to signify that a frame transmission is in progress. After the completion of Pause control frame transmission, the MAC will reset this bit to 0. **MAC_R6_flow_control** should not be written to until this bit is cleared. In Half-duplex mode, when this bit is set (and **tfe** is set), then back-pressure is asserted by the MAC Core. With back-pressure enabled, when the MAC receives a new frame, the transmitter begins sending a JAM pattern resulting in a collision. This control register bit is logically OR'ed with the MTI_FLOWCTRL_I input signal for the back-pressure function. When the MAC is configured to full-duplex mode, the **bpa** is automatically disabled.

Table 10-23. Ethernet Controller MAC Register 6 (Flow Control) Controls Generation of Control Frames.

10.3.2.8 MAC_R7_VLAN_tag Register

The VLAN Tag register (**MAC_R7_VLAN_tag**) contains the IEEE 802.1Q VLAN Tag to identify the VLAN frames. The MAC compares the 13th and 14th bytes of the receiving frame (Length/Type) with 16'h8100, and the following two bytes are compared with the VLAN tag; if a match occurs, it sets the received VLAN bit in the receive frame status. The legal length of the frame is increased from 1518 bytes to 1522 bytes. If the VLAN Tag register is configured to be double-synchronized to the RGMII clock domain, then consecutive writes made to these registers should be performed only after at least four clock cycles in the destination clock domain.

Bits	Name	Access	Reset	Description
31:17				Reserved
16	**etv**	RW	0	Enable 12-Bit VLAN Tag Comparison When this bit is set, a 12-bit VLAN identifier, rather than the complete 16-bit VLAN tag, is used for comparison and filtering. Bits[11:0] of the VLAN tag are compared with the corresponding field in the received VLAN-tagged frame. When this bit is reset, all 16 bits of the received VLAN frame's fifteenth and sixteenth bytes are used for comparison.
15:0	**vl**	RW	0	VLAN Tag Identifier This contains the 802.1Q VLAN Tag to identify VLAN frames, and is used as a comparison with the 15th and 16th bytes of the received VLAN frames. If **vl** is all-zeros, then the MAC does not check the 15th and 16th bytes for VLAN tag comparison.

Table 10-24. Ethernet Controller MAC Register 7 (VLAN Tag) Identifies IEEE 802.1Q VLAN Frames .

Confidential
For Mekra Lang Only
1.5 (Oct 2023)

10.3.2.9 MAC_R8_version Register

The MAC Register 8 is the Version register (**MAC_R8_version**) which identifies the Ethernet Controller version.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**version**	R	0x4437	Version

Table 10-25. Ethernet Controller MAC Register 8 Identifies the Version of the Core.

10.3.2.10 MAC_R11_PMT_Control_and_Status Register

The MAC Register 11 is the Control and Status register (**MAC_R11_PMT_Control_and_Status**) which provides the PMT control and status.

Bits	Name	Access	Reset	Description
31:7				Reserved
6	**RWKPRCVD**	R	0	Remote Wakeup Frame Received When set, this bit indicates that the power management event is generated because of the reception of a remote wakeup frame. This bit is cleared by a Read into this register.
5	**MGKPRCVD**	R	0	Magic Packet received When set, this bit indicates that the power management event is generated because of the reception of a magic packet. This bit is cleared by a Read into this register.
4:3		R	00	Reserved
2	**RWKPKTEN**	RW	0	Remote Wakeup Frame Enable When set, enables generation of a power management event because of remote wakeup frame reception.
1	**MGKPKTEN**	RW	0	Magic Packet Enable When set, enables generation of a power management event because of magic packet reception.
0				Reserved

Table 10-26. Ethernet Controller MAC Register 11 Provides PMT Control and Status.

10.3.2.11 MAC_R14_interrupt_status Register

The MAC Interrupt Status register (**MAC_R14_interrupt_status**) identifies the events in the MAC-CORE that can generate interrupts.

Bits	Name	Access	Reset	Description
31:10				Reserved

Bits	Name	Access	Reset	Description
9	**tsis**	R / R_SS_RC	0	Timestamp Interrupt Status When the Advanced Timestamp feature is enabled, this bit is set when any of the following conditions is true: • The system time value equals or exceeds the value specified in the Target Time High and Low registers. • There is an overflow in the seconds register. • The Auxiliary snapshot trigger is asserted. This bit is cleared on reading Bit 0 of Register 458 (Timestamp Status Register). If default Timestamping is enabled, when set, this bit indicates that the system time value is equal to or exceeds the value specified in the Target Time registers. In this mode, this bit is cleared after the completion of the read of this bit. In all other modes, this bit is reserved.
4:8				Reserved
3	**mgc_wake**	R	0	This bit is set whenever a Magic packet or Wake-on-LAN frame is received in Power-Down mode. This bit is cleared when both bits[6:5] are cleared due to a read operation made to the PMT Control and Status Register as described in Section 3.8.1.1 of the *Ethernet MAC Universal Databook* from Synopses.
2:1				Reserved
0	**rgsmiiis**	R	0	RGMII interrupt status. This bit is set due to any change in the value of the link status of the RGMII interface (Bit 3 of MAC_R54_RGMII_Control_and_Status register). This bit is cleared when users perform a read operation on the MAC_R54_RGMII_Control_and_Status register. This bit is valid only when the Ethernet controller operates in the RGMII mode.

Table 10-27. Ethernet Controller MAC Register 14 Provides Interrupt Status.

For Meeting only 1.5 (08 / 2025)

10.3.2.12 MAC_R15_interrupt_mask Register

The bits of the MAC Interrupt Mask register (**MAC_R15_interrupt_mask**) enable the user to mask the interrupt signal due to the corresponding event in the Interrupt Status register **MAC_R14_interrupt_status**.

Bits	Name	Access	Reset	Description
31:10				Reserved
9	**tsim**	RW	0	Timestamp Interrupt Mask When set, this bit disables the assertion of the interrupt signal because of the setting of Timestamp Interrupt Status bit in Register 14 (Interrupt Status Register).
8:4				Reserved
3	**int_off**	RW	0	PMT Interrupt Mask When set, this bit disables the assertion of the interrupt signal because of the setting of PMT Interrupt Status bit in Register 14 (Interrupt Status Register).
2:1				Reserved
0	**rgsmiiim**	RW	0	Disables RGMII interrupt generation when set to 1

Table 10-28. Ethernet Controller MAC Register 15 is the Interrupt Mask Register.

10.3.2.13 MAC_R16_MAC_add0_high Register

The MAC Address0 High register (**MAC_R16_MAC_add0_high**) holds the upper 16 bits of the first 6-byte MAC address of the station. Note that the first DA byte that is received on the RGMII interface corresponds to the least significant byte (Bits [7:0]) of the MAC Address Low register (**MAC_R17_MAC_add0_low**). For example, if 0x112233445566 is received (0x11 is the first byte) on the RGMII as the destination address, then the MAC Address0 Register [47:0] is compared with 0x665544332211.

If the MAC address registers are configured to be double-synchronized to the RGMII clock domains, then the synchronization is triggered only when Bits[31:24] (in Little Endian mode) or Bits[7:0] (in Big Endian mode) of the MAC Address Low Register (**MAC_R17_MAC_add0_low**) are written to. Please note that consecutive writes made to the Address Low Register should be performed only after at least four clock cycles in the destination clock domain for proper synchronization updates.

Bits	Name	Access	Reset	Description
31	**mo**	R	1	Always 1
30:16				Reserved
15:0	**A[47:32]**	RW	0xFFFF	MAC Address0 [47:32] This field contains the upper 16 bits (47:32) of the first 6-byte MAC address. This is used by the MAC for filtering of received frames and for inserting the MAC address in the Transmit Flow Control (PAUSE) Frames.

Table 10-29. Ethernet Controller MAC Register 16 is the MAC Address0 High Register.

10.3.2.14 MAC_R17_MAC_add0_low Register

The MAC Address0 Low register (**MAC_R17_MAC_add0_low**) holds the lower 32 bits of the first 6-byte MAC address of the station.

Bits	Name	Access	Reset	Description
31:0	**A[31:0]**	RW	0xFFFF_FFFF	MAC Address0 [31:0] This field contains the lower 32 bits of the first 6-byte MAC address. This is used by the MAC for filtering of received frames and for inserting the MAC address in the Transmit Flow Control (PAUSE) Frames.

Table 10-30. Ethernet Controller MAC Register 17 is the MAC Address0 Low Register.

10.3.2.15 MAC_R18_MAC_add1_high Register

The MAC Address1 High register (**MAC_R18_MAC_add1_high**) holds the upper 16 bits of the second 6-byte MAC address of the station. If the MAC address registers are configured to be double-synchronized to the RGMII clock domains, then the synchronization is triggered only when Bits[31:24] (in Little Endian mode) or Bits[7:0] (in Big Endian mode) of the MAC Address Low Register (**MAC_R19_MAC_add1_low**) are written to. Please note that consecutive writes made to the Address Low Register should be performed only after at least 4 clock cycles in the destination clock domain for proper synchronization updates.

Bits	Name	Access	Reset	Description
31	**ae**	RW	0	Address Enable When this bit is set, the Address filter module uses the second MAC address for perfect filtering. When reset, the address filter module will ignore the address for filtering.
30	**sa**	RW	0	Source Address When this bit is set, the MAC Address1[47:0] is used to compare with the SA fields of the received frame. When this bit is reset, the MAC Address1[47:0] is used to compare with the DA fields of the received frame.
29:24	**mbc**	RW	0	Mask Byte Control These bits are mask control bits for comparison of each of the MAC Address bytes. When set high, the MAC core does not compare the corresponding byte of received DA/SA with the contents of MAC Address1 registers. Each bit controls the masking of bytes as follows: Bit 24 - **MAC_R19_MAC_add1_low**[7:0] Bit 25 - **MAC_R19_MAC_add1_low**[15:8] Bit 26 - **MAC_R19_MAC_add1_low**[23:16] Bit 27 - **MAC_R19_MAC_add1_low**[31:24] Bit 28 - **MAC_R18_MAC_add1_high**[7:0] Bit 29 - **MAC_R18_MAC_add1_high**[15:8]
23:16				Reserved
15:0	**a**	RW	0xFFFF	MAC Address1 [47:32] This field contains the upper 16 bits (47:32) of the second 6-byte MAC address

Table 10-31. Ethernet Controller MAC Register 18 is the MAC Address1 High Register.

Note:

The descriptions for Register 20 (**MAC_R20_MAC_add2_high**) are as shown in the table above for **MAC_R18_MAC_add1_high**.

10.3.2.16 MAC_R19_MAC_add1_low Register

The MAC Register19 or MAC Address1 Low register (**MAC_R19_MAC_add1_low**) holds the lower 32 bits of the second 6-byte MAC address of the station.

Bits	Name	Access	Reset	Description
31:0	**A**	RW	0xFFFF_FFFF	MAC Address1 [31:0] This field contains the lower 32 bits of the second 6-byte MAC address. The content of this field is undefined until loaded by the Application after the initialization process.

Table 10-32. Ethernet Controller MAC Register 19 is the MAC Address1 Low Register.

Note:

The descriptions for Register 21 (**MAC_R20_MAC_add2_low**) are as shown in Table 21-31 for **MAC_R19_MAC_add1_low**.

10.3.2.17 MAC_R54_RGMII_Control_and_Status Register

The MAC Register 54 is the RGMII Control and Status register (**MAC_R54_RGMII_Control_and_Status**) which provides the RGMII control and status.

Bits	Name	Access	Reset	Description
31:3				Reserved
2:1	**LNKSPEED**	R	00	Link Speed. This bit indicates the current speed of the link: 00: 2.5 MHz 01: 25 MHz 10: 125 MHz
0	**LNKMOD**	R	0	Link Mode. This bit indicates the current mode of operation of the link: 1'b0: Half-duplex mode 1'b1: Full-duplex mode

Table 10-33. Ethernet Controller MAC Register 11 Provides PMT Control and Status.

10.3.2.18 MAC_R448_Timestamp_Control Register

The MAC_R448_Timestamp_Control register controls the operation of the System Time generator and the processing of PTP packets for timestamping in the Receiver.

Bits	Name	Access	Reset	Description
31:19	-	R	0	Reserved

Confidential For Mekra Lang only 1.5 (08/25/2025)

Bits	Name	Access	Reset	Description
18	**tsenmacaddr**	RW	0	Enable MAC address for PTP Frame Filtering When set, the DA MAC address (that matches any MAC Address register) is used to filter the PTP frames when PTP is directly sent over Ethernet.
17:16	**snaptypsel**	RW	00	Select PTP packets for Taking Snapshots These bits along with Bits 15 and 14 decide the set of PTP packet types for which snapshot needs to be taken.
15	**tsmstrena**	RW	0	Enable Snapshot for Messages Relevant to Master When set, the snapshot is taken only for the messages relevant to the master node. Otherwise, the snapshot is taken for the messages relevant to the slave node.
14	**tsevntena**	RW	0	Enable Timestamp Snapshot for Event Messages When set, the timestamp snapshot is taken only for event messages (SYNC, Delay_Req, Pdelay_Req, or Pdelay_Resp). When reset, the snapshot is taken for all messages except Announce, Management, and Signaling.
13	**tsipv4ena**	RW	1	Enable Processing of PTP Frames Sent over IPv4-UDP When set, the MAC receiver processes the PTP packets encapsulated in UDP over IPv4 packets. When this bit is clear, the MAC ignores the PTP transported over UDP-IPv4 packets. This bit is set by default.
12	**tsipv6ena**	RW	0	Enable Processing of PTP Frames Sent over IPv6-UDP When set, the MAC receiver processes PTP packets encapsulated in UDP over IPv6 packets. When this bit is clear, the MAC ignores the PTP transported over UDP-IPv6 packets.
11	**tsipena**	RW	0	Enable Processing of PTP over Ethernet Frames When set, the MAC receiver processes the PTP packets encapsulated directly in the Ethernet frames. When this bit is clear, the MAC ignores the PTP over Ethernet packets.
10	**tsver2ena**	RW	0	Enable PTP packet Processing for Version 2 Format When set, the PTP packets are processed using the 1588 version 2 format. Otherwise, the PTP packets are processed using the version 1 format.
9	**tsctrlssr**	RW	0	Timestamp Digital or Binary Rollover Control When set, the Timestamp Low register rolls over after 0x3B9A_C9FF value (that is, 1 nanosecond accuracy) and increments the timestamp (High) seconds. When reset, the rollover value of sub-second register is 0x7FFF_FFFF. The sub-second increment has to be programmed correctly depending on the PTP reference clock frequency and the value of this bit.
8	**tsenall**	RW	0	Enable Timestamp for All Frames When set, the timestamp snapshot is enabled for all frames received by the MAC
7:6	-	R	00	Reserved
5	**tsaddreg**	R_WS_SC	0	Addend Reg Update When set, the content of the Timestamp Addend register is updated in the PTP block for fine correction. This is cleared when the update is completed. This register bit should be zero before setting it.

For Mengshen 1.5 (08) Preliminary

Bits	Name	Access	Reset	Description
4	**tstrig**	R_WS_SC	0	Timestamp Interrupt Trigger Enable When set, the timestamp interrupt is generated when the System Time becomes greater than the value written in the Target Time register. This bit is reset after the generation of the Timestamp Trigger Interrupt.
3	**tsupdt**	RW	0	Timestamp Update When set, the system time is updated (added or subtracted) with the value specified in Register 452 (System Time – Seconds Update Register) and Register 453 (System Time – Nanoseconds Update Register). This bit should be read zero before updating it. This bit is reset when the update is completed in hardware.
2	**tsinit**	R_WS_SC	0	Timestamp Initialize When set, the system time is initialized (overwritten) with the value specified in the Register 452 (System Time – Seconds Update Register) and Register 453 (System Time – Nanoseconds Update Register). This bit should be read zero before updating it. This bit is reset when the initialization is complete.
1	**tscfupdt**	RW	0	Timestamp Fine or Coarse Update When set, this bit indicates that the system times update should be performed using the fine update method. Reset indicates the system timestamp update should be performed using the Coarse method.
0	**tsena**	RW	0	Timestamp Enable When set, the timestamp is added for the transmit and receive frames. When disabled, timestamp is not added for the transmit and receive frames and the Timestamp Generator is also suspended. The user must initialize the Timestamp (system time) after enabling this mode. On the receive side, the MAC processes the 1588 frames *only* if this bit is set.

Table 10-34. Ethernet Controller MAC Register 448 is the Timestamp and Control Register.

Table 11-35 indicates the PTP messages for which a snapshot is taken depending on Bits [17:14] (SNAPTYPSEL), in Register 448 (Timestamp Control Register).

SNAPTYPSEL (Bits 17:16)	TSMSTRENA (Bit 15)	TSEVNTENA (Bit 14)	PTP Messages
00	**X**	0	SYNC, Follow_Up, Delay_Req, Delay_Resp
00	**0**	1	SYNC
00	**1**	1	Delay_Req
01	**X**	0	SYNC, Follow_Up, Delay_Req, Delay_Resp, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_Up
01	**0**	1	SYNC, Pdelay_Req, Pdelay_Resp
01	**1**	1	Delay_Req, Pdelay_Req, Pdelay_Resp
10	**X**	X	SYNC, Delay_Req

SNAPTYPSEL (Bits 17:16)	TSMSTRENA (Bit 15)	TSEVNTENA (Bit 14)	PTP Messages
11	X	X	Pdelay_Req, Pdelay_Resp

Table 10-35. Timestamp Snapshot Dependency on Register Bits.

10.3.2.19 MAC_R449_Subsecond_Increment Register

In the Coarse Update mode (TSCFUPDT bit in Register 448), the value in this register is added to the system time every clock cycle of clk_ptp_ref_i. In the Fine Update mode, the value in this register is added to the system time whenever the Accumulator gets an overflow.

Bits	Name	Access	Reset	Description
31:8	-	R	000000H	Reserved
7:0	ssinc	RW	00H	Sub-second Increment Value The value programmed in this field is accumulated every clock cycle (of clk_ptp_i) with the contents of the sub-second register. For example, when the PTP clock is 50 MHz (period is 20 ns), the user should program 20 (0x14) when the System Time-Nanoseconds register has an accuracy of 1 ns [Bit 9 (TSC-TRLSSR) is set in Register 448 (Timestamp Control Register)]. When TSCTRLSSR is clear, the Nanoseconds register has a resolution of ~0.465 ns. In this case, the user should program a value of 43 (0x2B) that is derived by 20 ns/0.465.

Table 10-36. Ethernet Controller MAC Register 449 is the Subsecond Increment Register.

10.3.2.20 MAC_R450_System_Time_Seconds Register

The MAC_R449_System_Time_Seconds register, along with System Time—Nanoseconds register, indicates the current value of the system time maintained by the MAC. Although it is updated on a continuous basis, there is some delay from the actual time because of clock domain transfer latencies (from clk_ptp_ref_i to CSR clock).

Bits	Name	Access	Reset	Description
31:0	tss	R	00000000H	Timestamp Second The value in this field indicates the current value in seconds of the System Time maintained by the MAC

Table 10-37. Ethernet Controller MAC Register 450 is the System Time - Seconds Register.

10.3.2.21 MAC_R451_System_Time_Nanoseconds Register

Bits	Name	Access	Reset	Description
31	-	R	0	Reserved

Bits	Name	Access	Reset	Description
30:0	**tsss**	R	00000000H	Timestamp Sub Seconds The value in this field has the sub second representation of time, with an accuracy of 0.46 ns. When Bit 9 (TSCTRLSSR) is set in Register 448 (Timestamp Control Register), each bit represents 1 ns and the maximum value is 0x3B9A_C9FF, after which it rolls-over to zero.

Table 10-38. Ethernet Controller MAC Register 451 is the System Time - Nanoseconds Register.

10.3.2.22 MAC_R452_System_Time_Seconds_Update Register

The MAC_R451_System_Time_Seconds_Update register, along with the System Time—Nanoseconds Update register, initializes or updates the system time maintained by the MAC. The user must write both of these registers before setting the TSINIT or TSUPDT bits in the Timestamp Control register.

Bits	Name	Access	Reset	Description
31:0	**tss**	RW	00000000H	Timestamp Second The value in this field indicates the time in seconds to be initialized or added to the system time

Table 10-39. Ethernet Controller MAC Register 452 is the System Time - Seconds Update Register.

10.3.2.23 MAC_R453_System_Time_Nanoeconds_Update Register

Bits	Name	Access	Reset	Description
31	**addsub**	RW	0	Add or Subtract Time When this bit is set, the time value is subtracted with the contents of the update register. When this bit is reset, the time value is added with the contents of the update register
30:0	**tsss**	RW	00000000H	Timestamp Sub Seconds The value in this field has the sub second representation of time, with an accuracy of 0.46 ns. When Bit 9 (TSCTRLSSR) is set in Register 448 (Timestamp Control Register), each bit represents 1 ns and the maximum value is 0x3B9A_C9FF, after which it rolls-over to zero.

Table 10-40. Ethernet Controller MAC Register 453 is the System Time - Nanoseconds Update Register.

10.3.2.24 MAC_R454_Timestamp_Addend Register

This register value is used only when the system time is configured for Fine Update mode (TSCFUPDT bit in Register 448). This register content is added to a 32-bit accumulator in every clock cycle (of clk_ptp_ref_i) and the system time is updated whenever the accumulator overflows.

Bits	Name	Access	Reset	Description
31:0	**tsar**	RW	00000000H	Timestamp Addend Register This field indicates the 32-bit time value to be added to the Accumulator register to achieve time synchronization

Table 10-41. Ethernet Controller MAC Register 454 is the Timestamp Addend Register.

10.3.2.25 MAC_R455_Target_Time_Seconds Register

The MAC_R455_Target_Time_Seconds Register register, along with Target Time Nanoseconds register, is used to schedule an interrupt event (Register 458[1] when Advanced Timestamping is enabled; otherwise, TS interrupt bit in Register14[9]) when the system time exceeds the value programmed in these registers.

Bits	Name	Access	Reset	Description
31:0	**tstr**	RW	00000000H	Target Time Seconds Register This register stores the time in seconds. When the timestamp value matches or exceeds both Target Timestamp registers, then based on Bits [6:5] of Register 459 (PPS Control Register), the MAC starts or stops the PPS signal output and generates an interrupt (if enabled).

Table 10-42. Ethernet Controller MAC Register 455 is the Target Time Seconds Register.

10.3.2.26 MAC_R456_Target_Time_Nanoseconds Register

Bits	Name	Access	Reset	Description
31	**trgtbusy**	R_WS_SC	0	Target Time Register Busy The MAC sets this bit when the PPSCMD field (Bit [3:0]) in Register 459 (PPS Control Register) is programmed to 010 or 011. Programming the PPSCMD field to 010 or 011, instructs the MAC to synchronize the Target Time Registers to the PTP clock domain. The MAC clears this bit after synchronizing the Target Time Registers to the PTP clock domain. The application must not update the Target Time Registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted. This bit is reserved when the Enable Flexible Pulse-Per-Second Output feature is not selected.

For Meera Lab only 1.5 (08/12/25)

Bits	Name	Access	Reset	Description
30:0	**ttslo**	RW	00000000H	Target Timestamp Low Register This register stores the time in (signed) nanoseconds. When the value of the timestamp matches the both Target Timestamp registers, then based on the TRGTMODSEL0 field (Bits [6:5]) in Register 459 (PPS Control Register), the MAC starts or stops the PPS signal output and generates an interrupt (if enabled). This value should not exceed 0x3B9A_C9FF when Bit 9 (TSC-TRLSSR) is set in Register 448 (Timestamp Control Register). The actual start or stop time of the PPS signal output may have an error margin up to one unit of sub-second increment value.

Table 10-43. Ethernet Controller MAC Register 456 is the Target Time Nanoseconds Register.

10.3.2.27 MAC_R458_Timestamp_Status Register

Bits	Name	Access	Reset	Description
31:4	-	R	00H	Reserved
3	**tstrgterr**			Timestamp Target Time Error This bit is set when the target time, being programmed in Register 455 and Register 456, is already elapsed. This bit is cleared when read by the application.
2	-			Reserved
1	**tstargt**			Timestamp Target Time Reached When set, this bit indicates that the value of system time is greater than or equal to the value specified in the Register 455 (Target Time Seconds Register) and Register 456 (Target Time Nanoseconds Register)
0	**tssovp**			Timestamp Seconds Overflow When set, this bit indicates that the seconds value of the timestamp (when supporting version 2 format) has overflowed beyond 32'hFFFF_FFFF

Table 10-44. Ethernet Controller MAC Register 458 is the Timestamp Status Register.

For Ambarella Confidential Only 1.5 (08/15/2019)

10.3.2.28 MAC_R459_PPS_Control Register

Bits	Name	Access	Reset	Description
31:7	-	R	0	Reserved
6:5	**trgtmodsel0**	RW	00	Target Time Register Mode for PPS0 Output This field indicates the Target Time registers (register 455 and 456) mode for PPS0 output signal: • 00: Indicates that the Target Time registers are programmed only for generating the interrupt event. • 01: Reserved • 10: Indicates that the Target Time registers are programmed for generating the interrupt event and starting or stopping the generation of the PPS0 output signal. • 11: Indicates that the Target Time registers are programmed only for starting or stopping the generation of the PPS0 output signal. No interrupt is asserted.
4	**ppsen0**	RW	0	Flexible PPS Output Mode Enable When set low, Bits [3:0] function as PPSCTRL (backward compatible). When set high, Bits[3:0] function as PPSCMD.

For Mekra Lang only
Confidential
1.5 (08012025)

Bits	Name	Access	Reset	Description
3:0	**ppsctrl0**	RW	0H	PPSCTRL0: PPS0 Output Frequency Control This field controls the frequency of the PPS0 output (ptp_pps_o[0]) signal. The default value of PPSCTRL is 0000, and the PPS output is 1 pulse (of width clk_ptp_i) every second. For other values of PPSCTRL, the PPS output becomes a generated clock of following frequencies: • 0001: The binary rollover is 2 Hz, and the digital rollover is 1 Hz. • 0010: The binary rollover is 4 Hz, and the digital rollover is 2 Hz. • 0011: The binary rollover is 8 Hz, and the digital rollover is 4 Hz. • 0100: The binary rollover is 16 Hz, and the digital rollover is 8 Hz. • ... • 1111: The binary rollover is 32.768 KHz, and the digital rollover is 16.384 KHz. **Note:** In the binary rollover mode, the PPS output (ptp_pps_o) has a duty cycle of 50 percent with these frequencies. In the digital rollover mode, the PPS output frequency is an average number. The actual clock is of different frequency that gets synchronized every second. For example: • When PPSCTRL = 0001, the PPS (1 Hz) has a low period of 537 ms and a high period of 463 ms • When PPSCTRL = 0010, the PPS (2 Hz) is a sequence of: ◦ One clock of 50 percent duty cycle and 537 ms period ◦ Second clock of 463 ms period (268 ms low and 195 ms high) • When PPSCTRL = 0011, the PPS (4 Hz) is a sequence of: ◦ Three clocks of 50 percent duty cycle and 268 ms period ◦ Fourth clock of 195 ms period (134 ms low and 61 ms high) This behavior is because of the non-linear toggling of bits in the digital rollover mode in Register 451 (System Time – Nanoseconds Register).

Confidential Only
For Melfas Only
1.5 (08/2023)

Bits	Name	Access	Reset	Description
	ppscmd0	R_WS_SC	0H	Flexible PPS0 Output (ptp_pps_o[0]) Control Programming these bits with a non-zero value instructs the MAC to initiate an event. When the command is transferred or synchronized to the PTP clock domain, these bits get cleared automatically. The Software should ensure that these bits are programmed only when they are “all-zero”. The following list describes the values of PPSCMD0: • 0000: No Command • 0001: START Single Pulse This command generates single pulse rising at the start point defined in Target Time Registers (register 455 and 456) and of a duration defined in the PPS0 Width Register. • 0010: START Pulse Train This command generates the train of pulses rising at the start point defined in the Target Time Registers and of a duration defined in the PPS0 Width Register and repeated at interval defined in the PPS Interval Register. By default, the PPS pulse train is free-running unless stopped by ‘STOP Pulse train at time’ or ‘STOP Pulse Train immediately’ commands. • 0011: Cancel START This command cancels the START Single Pulse and START Pulse Train commands if the system time has not crossed the programmed start time. • 0100: STOP Pulse train at time This command stops the train of pulses initiated by the START Pulse Train command (PPSCMD = 0010) after the time programmed in the Target Time registers elapses. • 0101: STOP Pulse Train immediately This command immediately stops the train of pulses initiated by the START Pulse Train command (PPSCMD = 0010). • 0110: Cancel STOP Pulse train This command cancels the STOP pulse train at time command if the programmed stop time has not elapsed. The PPS pulse train becomes free-running on the successful execution of this command. • 0111-1111: Reserved

Table 10-45. Ethernet Controller MAC Register 459 is the PPS Control Register.

For Mercury 1.5 (080) Preliminary Confidential

10.3.2.29 MAC_R472_PPS0_Interval Register

The MAC_R472_PPS0_Interval register contains the number of units of sub-second increment value between the rising edges of PPS0 signal output (ptp_pps_o[0]).

Bits	Name	Access	Reset	Description
31:0	**ppsint**	RW	00000000H	PPS0 Output Signal Interval These bits store the interval between the rising edges of PPS0 signal output in terms of units of sub-second increment value. The user must program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and the desired interval between rising edges of PPS0 signal output is 100 ns (that is, five units of sub-second increment value), then the user should program value 4 (5 – 1) in this register.

Table 10-46. Ethernet Controller MAC Register 472 is the PPS0 Interval Register.

10.3.2.30 MAC_R473_PPS0_Width Register

The **MAC_R473_PPS0_Width** register contains the number of units of sub-second increment value between the rising and corresponding falling edges of the PPS0 signal output (ptp_pps_o[0]).

Bits	Name	Access	Reset	Description
31:0	**ppswidth**	RW	00000000H	PPS0 Output Signal Width These bits store the width between the rising edge and corresponding falling edge of the PPS0 signal output in terms of units of sub-second increment value. The user must program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns), and the desired width between the rising and corresponding falling edges of PPS0 signal output is 80 ns (that is, four units of sub-second increment value), then the user should program value 3 (4 – 1) in this register. **Note**: The value programmed in this register must be lesser than the value programmed in Register 472 (PPS0 Interval Register).

Table 10-47. Ethernet Controller MAC Register 473 is the PPS0 Width Register.

10.4 Ethernet: Alternate (Enhanced) Descriptors

Receive Descriptors and Transmit Descriptors are detailed in Section 10.4.1 and Section 10.4.2, respectively. Please be advised that the register descriptions are intended for Little Endian mode.

Confidential
For Ambarella Internal Use Only

10.4.1 Ethernet Descriptors: Receive

The figure below illustrates the receive descriptor format in Little Endian mode with a 32-bit data bus. Descriptor addresses must be aligned to 32 bits.

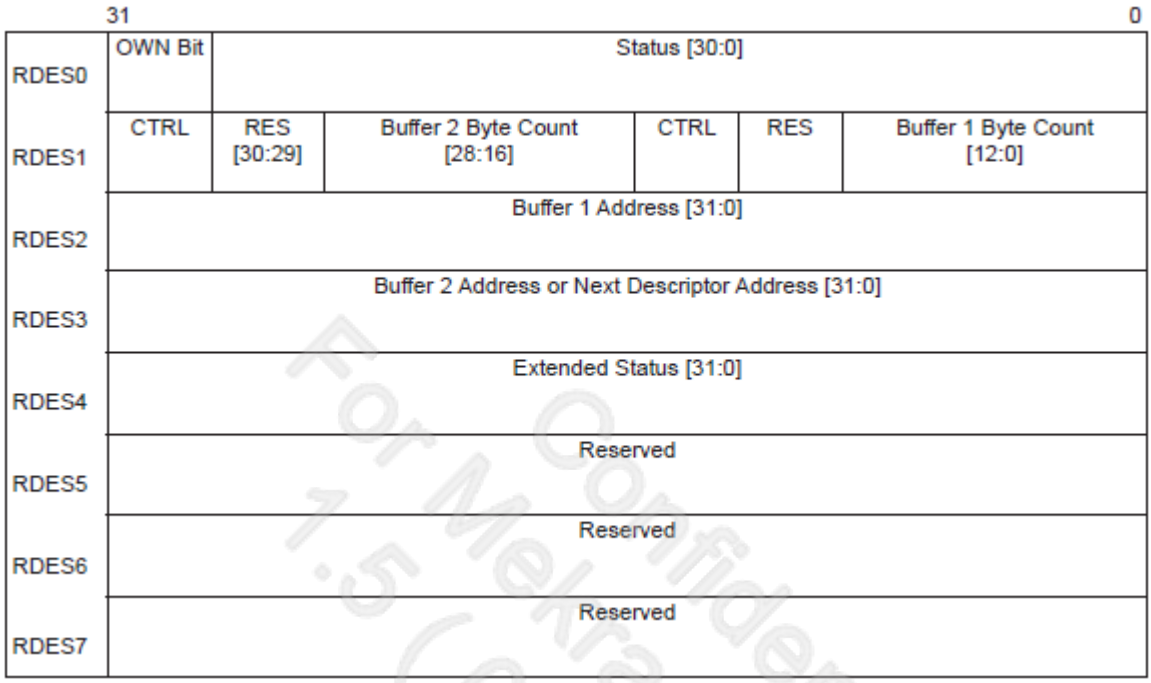


Figure 10-2. Ethernet Receive Descriptor Format in Little Endian Mode with a 32-Bit Data Bus.

10.4.1.1 Ethernet Receive Descriptor 0

Receive Descriptor 0 (RDES0) contains the received frame status, frame length, and descriptor ownership information.

Bits	Description
31	OWN: Own Bit When set, this bit indicates that the descriptor is owned by the DMA of the MAC Subsystem. When this bit is reset, it indicates that the descriptor is owned by the Host. The DMA clears this bit when it either completes the frame reception or when the buffers that are associated with this descriptor are full.
30	AFM: Destination Address Filter Fail When set, this bit indicates that a frame failed in the DA Filter in the MAC Core.
29:16	FL: Frame Length These bits indicate the byte length of the received frame that was transferred to Host memory (including CRC). This field is valid only when the Last Descriptor (RDES0[8]) is set and the Descriptor Error (RDES0[14]) is reset. The Frame Length also includes the 2 bytes appended to the Ethernet Frame when IP Checksum calculation is enabled and the received frame is not a MAC Control frame. This field is valid when the Last Descriptor (RDES0[8]) is set. When the Last Descriptor and Error Summary bits are not set, this field will indicate the current number of bytes transferred for that frame.

Confidential For Mekra Lang only 1.5 (08012015)

Bits	Description
15	ES: Error Summary Indicates the logical OR of the following bits: RDES0[1]: CRC Error RDES0[3]: Receive Error RDES0[4]: Watchdog Timeout RDES0[6]: Late Collision RDES0[7]: Giant Frame RDES4[4:3]: IP Header or Payload Error RDES0[11]: Overflow Error RDES0[14]: Descriptor Error This field is valid only when the Last Descriptor (RDES0[8]) is set.
14	DE: Descriptor Error When set, this bit indicates a frame truncation occurred on a frame that exceeded the current descriptor buffers, and that the DMA does not own the Next Descriptor. This field is valid only when the Last Descriptor (RDES0[8]) is set.
13	SAF: Source Address Filter Fail When set, this bit indicates that the SA field of the frame failed the SA Filter in the MAC Core.
12	LE: Length Error When set, this bit indicates that the actual length of the frame received does not match the information in the Length/ Type field. This bit is valid only when the Frame Type (RDES0[5]) bit is reset.
11	OE: Overflow Error When set, this bit indicates that the received frame was damaged due to buffer overflow in MTL.
10	VLAN : VLAN tag When set, this bit indicates that the frame pointed to by this descriptor is a VLAN frame tagged by the MAC Core.
9	FS: First Descriptor When set, this bit indicates that the descriptor contains the first buffer of the frame. If the size of the first buffer is 0, the second buffer contains the beginning of the frame. If the size of the second buffer is also 0, the following Descriptor contains the beginning of the frame.
8	LS: Last Descriptor When set, this bit indicates that the buffers pointed to by this descriptor are the last buffers of the frame
7	Timestamp Available, IP Checksum Error (Type1), or Giant Frame When Advanced Timestamp feature is present, when set, this bit indicates that a snapshot of the Timestamp is written in descriptor words 6 (RDES6) and 7 (RDES7). This is valid only when the Last Descriptor bit (RDES0[8]) is set. When IP Checksum Engine (Type 1) is selected, this bit (when set) indicates one of the following: - The 16-bit IPv4 header checksum calculated by the core did not match the received checksum bytes. - The header checksum checking is bypassed for non-IPv4 frames. Otherwise, this bit (when set) indicates the Giant Frame Status. Giant frames are larger than 1,518-byte (or 1,522-byte for VLAN or 2,000-byte when Bit 27 of the MAC Configuration register is set) normal frames and larger than 9,018-byte (9,022-byte for VLAN) frame when Jumbo Frame processing is enabled.
6	LC: Late Collision When set, this bit indicates that late collision has occurred while receiving the frame in Half-duplex mode.

Confidential Ambarella Internal Only

Bits	Description
5	FT: Frame Type When set, this bit indicates that the Receive Frame is an Ethernet-type frame (the LT field is greater than or equal to 1,536). When this bit is reset, it indicates that the received frame is an IEEE 802.3x standard frame. This bit is not valid for runt frames less than 14 bytes.
4	RWT: Receive Watchdog Timeout When set, this bit indicates that the Receive Watchdog Timer has expired while receiving the current frame and the current frame is truncated after the Watchdog Timeout.
3	RE: Receive Error When set, this bit indicates that the RGMII_PHY_RXER_I signal is asserted while RGMII_PHY_RXDV_I signal is asserted during frame reception. This error also includes carrier extension errors in RGMII and Half-duplex modes. Errors can occur due to lack of an extension, or an error (e.g., rxd != 0f) during extension.
2	DE: Dribble Bit Error When set, this bit indicates that the received frame has a non-integer multiple of bytes (odd nibbles). This bit is valid only in RMII Mode.
1	CE: CRC Error When set, this bit indicates that a Cyclic Redundancy Check (CRC) Error occurred on the received frame. This field is valid only when the Last Descriptor (RDES0[8]) is set.
0	Extended Status Available/RX MAC Address When either Advanced Timestamp or IP Checksum Offload (Type 2) is present, this bit (when set) indicates that the extended status is available in descriptor word 4 (RDES4). This is valid only when the Last Descriptor bit (RDES0[8]) is set. This bit is invalid when Bit 30 is set. When IP Checksum Offload (Type 2) is present, this bit is set even when the IP Checksum Offload engine bypasses the processing of a received frame. The bypassing may be because of non-IP frame or IP frame with a non-TCP/UDP/ICMP payload. When Advance Timestamp Feature or IPC Full Offload is not selected, this bit indicates RX MAC Address status. When set, this bit indicates that the RX MAC Address registers value (1 to 15) matched the frame's DA field. When reset, this bit indicates that the RX MAC Address Register 0 value matched the DA field.

Table 10-48. Ethernet Receive Descriptor 0 Register.

10.4.1.2 Ethernet Receive Descriptor 1

Receive Descriptor 1 (RDES1) contains the buffer sizes and other bits which control the descriptor chain/ring.

Bits	Description
31	Disable Interrupt on Completion When set, this bit prevents the setting of the receive interrupt (RI) (CSR5[6]) bit of the Status Register for the received frame that ends in the buffer pointed to by this descriptor. In turn, this disables the assertion of the interrupt to the Host due to an RI for that frame.
30:29	Reserved
28:16	RBS2: Receive Buffer 2 Size These bits indicate the second data buffer size, in bytes. The buffer size must be a multiple of 4, 8, or 16, depending on the bus widths (32, 64, or 128, respectively), even if the value of RDES3 (buffer2 address pointer) is not aligned to bus width. If the buffer size is not an appropriate multiple of 4, 8, or 16, the resulting behavior is undefined. This field is not valid if RDES1[14] is set.

Bits	Description
15	RER: Receive End of Ring When set, this bit indicates that the descriptor list reached its final descriptor. The DMA returns to the base address of the list, creating a Descriptor Ring.
14	RCH: Second Address Chained When set, this bit indicates that the second address in the descriptor is the Next Descriptor address rather than the second buffer address. When this bit is set, RBS2 (RDES1[28:16]) is a “don’t care” value. RDES1[15] takes precedence over RDES1[14].
13	Reserved
12:0	RBS1: Receive Buffer 1 Size Indicates the first data buffer size in bytes. The buffer size must be a multiple of 4, 8, or 16, depending upon the bus widths (32, 64, or 128), even if the value of RDES2 (buffer1 address pointer) is not aligned. When the buffer size is not a multiple of 4, 8, or 16, the resulting behavior is undefined. If this field is 0, the DMA ignores this buffer and uses Buffer 2 or the next descriptor, depending on the value of RCH (Bit 14).

Table 10-49. Ethernet Receive Descriptor 1 Register.

10.4.1.3 Ethernet Receive Descriptor 2

Receive Descriptor 2 (RDES2) contains the address pointer to the first data buffer in the descriptor.

Bits	Description
31:0	Buffer 1 Address Pointer These bits indicate the physical address of Buffer 1. There are no limitations on buffer address alignment with the exception of the following condition: The DMA uses the configured value for its address generation when the RDES2 value is used to store the start of frame. Note that the DMA performs a write operation with the RDES2[3/2/1:0] bits as “zero” during the transfer of the start of frame but the frame data is shifted as per the actual Buffer address pointer. The DMA ignores RDES2[3/2/1:0] (corresponding to bus-widths of 128/64/32) if the address pointer is to a buffer where the middle or last part of the frame is stored.

Table 10-50. Ethernet Receive Descriptor 2 Register.

Confidential Only For Meeka Use (Oct 2025)

10.4.1.4 Ethernet Receive Descriptor 3

Receive Descriptor 3 (RDES3) contains the address pointer either to the second data buffer in the descriptor or the next descriptor.

Bits	Description
31:0	Buffer 2 Address Pointer (Next Descriptor Address) These bits indicate the physical address of Buffer 2 when descriptor chaining is used. If the Second Address Chained (RDES1[24]) bit is set, then this address contains the pointer to the physical memory where the Next Descriptor is present. If RDES1[24] is set, then the buffer (Next Descriptor) address pointer must be bus-width aligned (RDES3[3/21:0] = 0 corresponding to bus-width of 128/64/32. Internally, the LSBs are ignored). However, when RDES1[24] is reset, there are no limitations on the RDES3 value with the exception of the following condition: The DMA uses the configured value for its buffer address generation when the RDES3 value is used to store the start of frame. The DMA ignores RDES3[3/2/1:0] (corresponding to bus-widths of 128/64/32) if the address pointer is to a buffer where the middle or last part of the frame is stored.

Table 10-51. Ethernet Receive Descriptor 3 Register.

10.4.1.5 Ethernet Receive Descriptor 4

Bits	Description
31:28	Reserved
27:26	Layer 3 and Layer 4 Filter Number Matched These bits indicate the number of the Layer 3 and Layer 4 Filter that matched the received frame. 00: Filter 0 01: Filter 1 10: Filter 2 11: Filter 3 This field is valid only when Bit 24 or Bit 25 is set high. When more than one filter matches, these bits provide only the lowest filter number.
25	Layer 4 Filter Match When set, this bit indicates that the received frame matches one of the enabled Layer 4 Port Number fields. This status is provided only when one of the following conditions is true: - Layer 3 fields are not enabled and all enabled Layer 4 fields match. - All enabled Layer 3 and Layer 4 filter fields match. When more than one filter matches, this bit provides the layer 4 filter status of the filter indicated by Bits [27:26].
24	Layer 3 Filter Match When set, this bit indicates that the received frame matches one of the enabled Layer 3 IP Address fields. This status is provided only when one of the following conditions is true: - All enabled Layer 3 fields match and all enabled Layer 4 fields are bypassed. - All enabled filter fields match. When more than one filter matches, this bit provides the layer 3 filter status of filter indicated by Bits [27:26].
23:8	Reserved

Ambarella Confidential
For Mekra Lang only
1.5 (Oct 2023)

Bits	Description
7	IPv6 Packet Received When set, this bit indicates that the received packet is an IPv6 packet. This bit is updated only when Bit 10 (IPC) of the MAC Configuration Register is set. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.
6	IPv4 Packet Received When set, this bit indicates that the received packet is an IPv4 packet. This bit is updated only when Bit 10 (IPC) of the MAC Configuration Register is set. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.
5	IP Checksum Bypassed When set, this bit indicates that the checksum offload engine has been bypassed. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.
4	IP Payload Error When set, this bit indicates that the 16-bit IP payload checksum (i.e., the TCP, UDP, or ICMP checksum) calculated by the core does not match the corresponding checksum field in the received segment. It is also set when the TCP, UDP, or ICMP segment length does not match the payload length value in the IP Header field. This bit is valid when either Bit 7 or Bit 6 is set. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.
3	IP Header Error When set, this bit indicates that either the 16-bit IPv4 header checksum calculated by the core does not match the received checksum bytes, or that the IP datagram version is not consistent with the Ethernet Type value. This bit is valid when either Bit 7 or Bit 6 is set. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.
2:0	IP Payload Type These bits indicate the type of payload encapsulated in the IP datagram processed by the Receive Checksum Offload Engine (COE). The COE also sets these bits to 2'b00 if the IP datagram's payload is not processed due to an IP header error or fragmented IP. 3'b000: Unknown or did not process IP payload 3'b001: UDP 3'b010: TCP 3'b011: ICMP 3'b1xx: Reserved This bit is valid when either Bit 7 or Bit 6 is set. This bit is available when the Enable Receive Full TCP/IP Checksum (Type 2) feature is selected.

Table 10-52. Ethernet Receive Descriptor 4 (RDES4) Register.

For Media Gang only (Confidential 2015)

10.4.2 Ethernet Descriptors: Transmit

The figure below illustrates the transmit descriptor format in Little Endian mode with a 32-bit data bus. Descriptor addresses must be aligned to 32 bits.

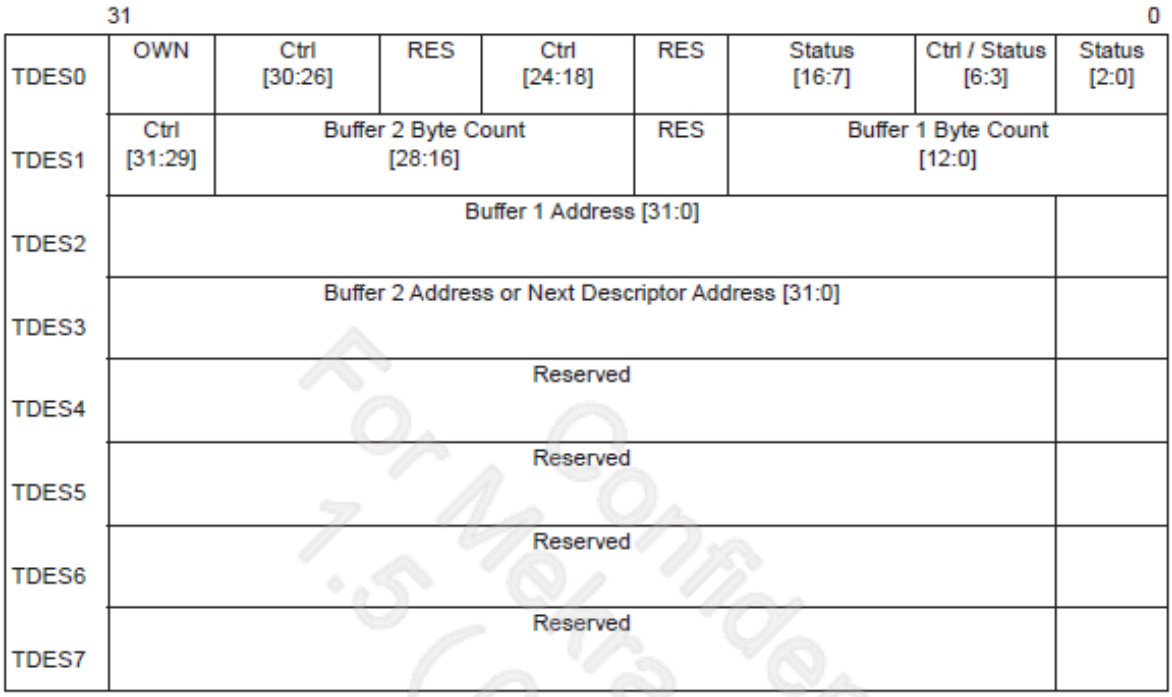


Figure 10-3. Ethernet Transmit Descriptor Format in Little Endian Mode with a 32-Bit Data Bus.

The figure below illustrates the transmit descriptor fetch (read) format. The DMA engine always reads or fetches four DWORDS of the descriptor from system memory to obtain the buffer and control information defined in this figure.

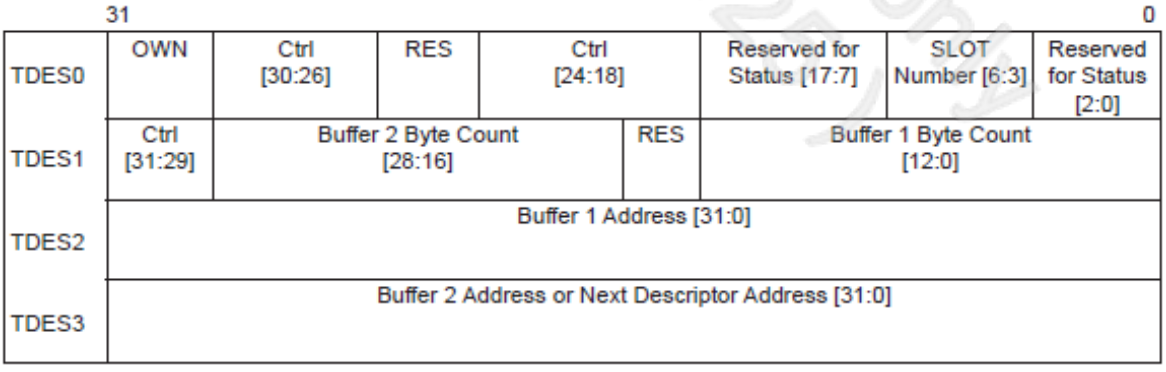


Figure 10-4. Ethernet Transmit Descriptor Fetch (Read) Format.

Confidential
For Mekra Lang only
1.5 (08012025)

10.4.2.1 Ethernet Transmit Descriptor 0

Transmit Descriptor 0 (TDES0) contains the transmitted frame status and the descriptor ownership information.

Bits	Description
31	OWN: Own Bit When set, this bit indicates that the descriptor is owned by the DMA. When this bit is reset, it indicates that the descriptor is owned by the Host. The DMA clears this bit when it either completes frame transmission or when the buffers allocated in the descriptor are empty. The ownership bit of the First Descriptor of the frame should be set after all subsequent descriptors belonging to the same frame have been set. This avoids a possible race condition between fetching a descriptor and the driver setting an ownership bit.
30	IC: Interrupt on Completion When set, this bit sets the Transmit Interrupt (Register 5[0]) after the present frame has been transmitted.
29	LS: Last Segment When set, this bit indicates that the buffer contains the last segment of the frame. When this bit is set, the TBS1 or TBS2 field in TDES1 should have a non-zero value.
28	FS: First Segment When set, this bit indicates that the buffer contains the first segment of a frame.
27	DC: Disable CRC When this bit is set, the MAC does not append a cyclic redundancy check (CRC) to the end of the transmitted frame. This is valid only when the first segment (TDES0[28]) is set.
26	DP: Disable Pad When set, the MAC does not automatically add padding to a frame shorter than 64 bytes. When this bit is reset, the DMA automatically adds padding and a CRC to a frame shorter than 64 bytes, and the CRC field is added regardless of the state of the DC (TDES0[27]) bit. This is valid only when the first segment (TDES0[28]) is set.
25	Reserved
24	CRCR: CRC Replacement Control When set, the MAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The host should ensure that the CRC bytes are present in the frame being transferred from the Transmit Buffer. This bit is valid when the Enable SA, VLAN, and CRC Insertion on TX option is selected during core configuration and the First Segment control bit (TDES0[28]) is set. In addition, CRC replacement is performed only when Bit 27 (DC) is set to 1.
23:22	CIC: Checksum Insertion Control These bits control the checksum calculation and insertion. The following list describes the bit encoding: 2'b00: Checksum Insertion Disabled. 2'b01: Only IP header checksum calculation and insertion are enabled. 2'b10: IP header checksum and payload checksum calculation and insertion are enabled, but the pseudo-header checksum is not calculated via hardware. 2'b11: IP Header checksum and payload checksum calculation and insertion are enabled, and pseudo-header checksum is calculated via hardware. This field is valid when the Enable Transmit Full TCP/IP Checksum (Type 2) option is selected during core configuration and the First Segment control bit (TDES0[28]) is set.
21	TER: Transmit End of Ring When set, this bit indicates that the descriptor list reached its final descriptor. The DMA returns to the base address of the list, creating a descriptor ring.
20	TCH: Second Address Chained When set, this bit indicates that the second address in the descriptor is the Next Descriptor address rather than the second buffer address. When TDES0[20] is set, TBS2 (TDES1[28:16]) is a “don’t care” value. TDES0[21] takes precedence over TDES0[20].

Bits	Description
19:18	VLIC: VLAN Insertion Control When set, these bits request the MAC to perform VLAN tagging or un-tagging prior to transmitting frames. If the frame is modified for VLAN tags, the MAC automatically recalculates and replaces the CRC bytes. The following list describes the values of these bits: 2'b00: Do not add a VLAN tag. 2'b01: Remove the VLAN tag from the frames before transmission. This option should be used only with VLAN frames. 2'b10: Insert a VLAN tag with the tag value programmed in the VLAN Tag Inclusion or Replacement Register. 2'b11: Replace the VLAN tag in frames with the Tag value programmed in the VLAN Tag Inclusion or Replacement Register. This option should be used only with VLAN frames. These bits are valid when the Enable SA, VLAN, and CRC Insertion on TX option is selected during core configuration and the First Segment control bit (TDES0[28]) is set.
17	Reserved
16	IHE: IP Header Error When set, this bit indicates that the MAC transmitter detected an error in the IP datagram header. The transmitter checks the header length in the IPv4 packet against the number of header bytes received from the application and indicates an error status if there is a mismatch. For IPv6 frames, a header error is reported if the main header length is not 40 bytes. Furthermore, the Ethernet Length/Type field value for an IPv4 or IPv6 frame must match the IP header version received with the packet. For IPv4 frames, an error status is also indicated if the Header Length field has a value less than 0x5. This bit is valid only when the TX Checksum Offload is enabled. Otherwise, it is reserved. If COE detects an IP header error, it still inserts an IPv4 header checksum if the Ethernet Type field indicates an IPv4 payload.
15	ES: Error Summary Indicates the logical OR of the following bits: TDES0[14]: Jabber Timeout TDES0[13]: Frame Flush TDES0[11]: Loss of Carrier TDES0[10]: No Carrier TDES0[9]: Late Collision TDES0[8]: Excessive Collision TDES0[2]: Excessive Deferral TDES0[1]: Underflow Error TDES0[16]: IP Header Error TDES0[12]: IP Payload Error
14	JT: Jabber Timeout When set, this bit indicates the MAC transmitter has experienced a jabber timeout. This bit is only set when Bit 22 (Jabber Disable) of Register 0 (MAC Configuration Register) is not set.
13	FF: Frame Flushed When set, this bit indicates that the DMA or MTL flushed the frame due to a software Flush command having been issued by the CPU.
12	IPE: IP Payload Error When set, this bit indicates that MAC transmitter detected an error in the TCP, UDP, or ICMP IP datagram payload. The transmitter checks the payload length received in the IPv4 or IPv6 header against the actual number of TCP, UDP, or ICMP packet bytes received from the application and issues an error status in the event of a mismatch.

For Confidential Use Only (20250801)

Bits	Description
11	LOC: Loss of Carrier When set, this bit indicates that a loss of carrier occurred during frame transmission (i.e., the phy_crs_i signal was inactive for one or more transmit clock periods during frame transmission). This is valid only for the frames transmitted without collision when the MAC operates in the half-duplex mode.
10	NC: No Carrier When set, this bit indicates that the Carrier Sense signal form the PHY was not asserted during transmission.
9	LC: Late Collision When set, this bit indicates that frame transmission has been aborted due to a collision occurring after the collision window (64 byte-times, including preamble, in RMII mode and 512 byte-times, including preamble and carrier extension, in RGMII mode). This bit is invalid if the Underflow Error bit is set.
8	EC: Excessive Collision When set, this bit indicates that the transmission was aborted after 16 successive collisions while attempting to transmit the current frame. If Bit 9 (Disable Retry) bit in the Register 0 (MAC Configuration Register) is set, this bit is set after the first collision, and the transmission of the frame is aborted.
7	VF: VLAN Frame When set, this bit indicates that the transmitted frame is a VLAN-type frame.
6:3	CC: Collision Count (Status field) These status bits indicate the number of collisions that occurred prior to frame transmission. This count is not valid when the Excessive Collisions bit (TDES0[8]) is set. The core updates this status field only in the half-duplex mode.
2	ED: Excessive Deferral When set, this bit indicates that the transmission was terminated due to excessive deferral of over 24,288 bit times (155,680 bits times in 1000-Mbps mode or if Jumbo Frames are enabled) if Bit 4 (Deferral Check) bit in Register 0 (MAC Configuration Register) is set high.
1	UF: Underflow Error When set, this bit indicates that the MAC aborted the frame because the data arrived late from Host memory. Underflow Error indicates that the DMA encountered an empty transmit buffer while transmitting the frame. The transmission process enters a Suspended state and sets both the Transmit Underflow (Register 5[5]) and Transmit Interrupt (Register 5[0]) registers.
0	DB: Deferred Bit When set, this bit indicates that the MAC defers prior to transmission due to the presence of carrier. This bit is valid only in half-duplex mode.

Table 10-53. Ethernet Transmit Descriptor 0 Register.

10.4.2.2 Ethernet Transmit Descriptor 1

Transmit Descriptor 1 (TDES1) contains buffer sizes and other bits to control the descriptor chain/ring and frame being transferred.

Bits	Description
31:29	SAIC: SA Insertion Control These bits request the MAC to add or replace the Source Address field in the Ethernet frame with the value given in the MAC Address 0 register. If the Source Address field is modified in a frame, the MAC automatically recalculates and replaces the CRC bytes. Bit 31 specifies the MAC Address Register (1 or 0) value used for Source Address insertion or replacement. The following list describes the values of Bits[30:29]: 2'b00: Do not include the source address. 2'b01: Include or insert the source address. For reliable transmission, the application must provide frames without source addresses. 2'b10: Replace the source address. For reliable transmission, the application must provide frames with source addresses. 2'b11: Reserved These bits are valid in the GMAC-DMA, GMAC-AXI, and GMAC-AHB configurations when Enable SA, VLAN, and CRC Insertion on TX is selected during core configuration and when the First Segment control bit (TDES0[28]) is set.
28:16	TBS2: Transmit Buffer 2 Size This field indicates the second data buffer size in bytes. This field is not valid if TDES0[20] is set.
15:13	Reserved
12:0	TBS1: Transmit Buffer 1 Size These bits indicate the first data buffer byte size, in bytes. If this field is 0, the DMA engine ignores this buffer and uses Buffer 2 or the next descriptor, depending on the value of TCH (TDES0[20]).

Table 10-54. Ethernet Transmit Descriptor 1 Register.

10.4.2.3 Ethernet Transmit Descriptor 2

Transmit Descriptor 2 (TDES2) contains the address pointer to the first buffer of the descriptor.

Bits	Description
31:0	Buffer 1 Address Pointer These bits indicate the physical address of Buffer 1. There is no limitation on buffer address alignment.

Table 10-55. Ethernet Transmit Descriptor 2 Register.

Confidential Ambarella Internal Use Only (Oct 2025)

10.4.2.4 Ethernet Transmit Descriptor 3

Transmit Descriptor 3 (TDES3) contains the address pointer either to the second buffer of the descriptor or the next descriptor.

Bits	Description
31:0	Buffer 2 Address Pointer (Next Descriptor Address) Indicates the physical address of Buffer 2 when descriptor chaining is used. If the Second Address Chained (TDES1[24]) bit is set, then this address contains the pointer to the physical memory where the Next Descriptor is present. The buffer address pointer must be aligned to the bus-width only when TDES1[24] is set. (Internally, LSBs are ignored.)

Table 10-56. Ethernet Transmit Descriptor 3 Register.

10.5 Ethernet: Programming Guidelines

10.5.1 Initializing DMA

Complete the following steps to initialize the DMA:

1. Provide a software reset. This resets all of the GMAC internal register and logic (**DMA_R0_bus_mode**—bit 0).
2. Wait for the completion of the reset process (poll bit 0 of the **DMA_R0_bus_mode**, which is only cleared after the reset operation is completed).
3. Program **DMA_R0_bus_mode** register by setting proper values.
4. Create a proper descriptor chain for transmit and receive. In addition, ensure that the receive descriptors are owned by DMA (bit 31 of descriptor should be set). When OSF mode is used, at least two descriptors are required.
5. Ensure that the software creates three or more different transmit or receive descriptors in the chain before reusing any of the descriptor.
6. Initialize receive and transmit descriptor list address with **DMA_R3_rx_descriptor** and **DMA_R4_tx_descriptor**, respectively.
7. Program **DMA_R6_operation_mode** with the proper value.
8. Enable the interrupts by programming **DMA_R7_interrupt_enable**.
9. Start the receive and transmit DMA by setting **sr** (bit 1) and **st** (bit 13) of the **DMA_R6_operation_mode** register.

10.5.2 Initializing MAC

The following MAC initialization operation can be performed after DMA initialization. If the MAC initialization is completed before the DMA is set up, then enable the MAC receiver (last step in the following sequence) only after the DMA is active. Otherwise, received frames fill the RX FIFO and overflow.

1. Program **MAC_R4_GMII_address** register for controlling the management cycles for external PHY.
2. Read the 16-bit data of **MAC_R5_GMII_data** register from the PHY for link up, speed of operation and mode of operation, by specifying the appropriate address in bit 15-11 of **MAC_R4_GMII_address** register.

3. Provide the MAC address by setting **MAC_R16_MAC_add0_high** and **MAC_R17_MAC_add0_low**. If more than one MAC address is enabled, program the MAC addresses appropriately.
4. If needed, program **MAC_R2_hash_high** and **MAC_R3_hash_low** registers for the hash filtering.
5. Program the following fields to set the appropriate filters for the incoming frames in **MAC_R1_ MAC_fm_filter** register.
    a. Receive all (**ra**)
    b. Promiscuous mode (**pr**)
    c. Hash or perfect filter (**hpf**)
    d. Unicast, multicast, broadcast, and control frames filter settings.
6. Program the following fields for proper flow control in **MAC_R6_flow_control** register.
    a. Pause time (**pt**) and other pause frame control bits.
    b. Receive and transmit flow control bits
    c. Flow control busy / back-pressure activate (**fcb** / **bpa**)
7. Program **MAC_R15_interrupt_mask** register bits as required.
8. Program the appropriate fields in **MAC_R0_MAC_config** register. For example, based on the auto-negotiation you can set the duplex mode or port select.
9. Set bit 3 (**te**) and bit 2 (**re**) in **MAC_R0_MAC_config** register.

Note:

Do not change the configuration (such as duplex mode, speed, port or loopback) when the MAC is actively transmitting or receiving.

For WAFA Lang only (Confidential 08012025)

11. USB DEVICE CONTROLLER

This chapter describes the CV22FS USB device controller as follows:

- (Section 11.1) USB Device: Overview
- (Section 11.2) USB Device: Clocking
- (Section 11.3) USB Device: Host / Device Select
- (Section 11.4) USB Device: USB Boot Mode
- (Section 11.5) USB Device: Device Controller Operations
- (Section 11.6) USB Device: Controller / CSR Registers
- (Section 11.7) USB Device: Initialization
- (Section 11.8) USB Device: DMA Operation

11.1 USB Device: Overview

The CV22FS chip provides an interface for a USB device controller (UDC). The features of the USB Device include:

- Compliance with USB2.0 and USB1.1.
- Support for high-speed and full-speed operations.
- Built-in DMA controller with Scatter-Gather support.
- A total of three configurations (i.e., configuration 0 plus configurations 1/2).
- Support for up to six interfaces (and seven alternate interfaces) for configurations 1/2.
- Support for five in/out endpoints as well as endpoint 0 (the mandatory default control endpoint EP0 per USB standards).
- 256 * 4 bytes of RX FIFO (RAM) and 528 * 4 bytes of TX FIFO (RAM).

The USB controller sits on the AHB bus, and can function both as a master (i.e., in DMA mode) and as a slave (i.e., in Slave mode).

In both DMA and Slave modes, the subsystem acts as a Slave for Control and Status Register (CSR) access. In Slave mode, the internal FIFOs are memory-mapped.

DMA mode should be used with applications that require software controllability and throughput. Slave mode should be used with applications that are software-independent and can be functionally implemented in hardware.

Both of these modes are explained in detail in the following sections.

Note:

The USB device and USB PHY shall not be activated for safety-related applications.

Confidential For Mekra Lang only 1.5 (08/12/2025)

11.2 USB Device: Clocking

CV22FS system software generally controls the USB PHY clock GCLK_USB_PHY.

11.3 USB Device: Host / Device Select

The CV22FS provides one USB high-speed interface that can be configured to work in master or slave mode. Use USBP0_SEL_REG to configure the USB interface for host / device operation.

11.4 USB Device: USB Boot Mode

The CV22FS chip provides a USB power on boot mode (see Section 2.2.3 “POC: Boot Mode Summary”).

11.5 USB Device: Device Controller Operations

The USB controller subsystem supports two modes of operation. The first is a DMA-based implementation in which the subsystem can master AHB for data transfers (Section 11.5.1). The second is a slave implementation in which the subsystem is slaved to the application and the application bus master(s) reads data from, or writes data to, the memory-mapped subsystem FIFOs (Section 11.5.2). In both DMA and Slave modes all data transfers are interrupt-driven.

11.5.1 Operations: DMA Mode

In DMA mode, before the start of an action, the application must initialize the buffer descriptor chains for all active endpoints, and must program the necessary CSRs in the UDC and subsystem during the USB reset. The setup and data descriptor pointers must be 16-byte aligned and the data buffer pointer in the descriptor information field must be 8-byte aligned.

11.5.2 Operations: Slave Mode

In Slave mode, the subsystem operates as a slave and the application bus master(s) reads data from or writes data to the memory-mapped subsystem FIFOs. All data transfers are interrupt-driven, except ISO-IN and interrupt-IN transfers, which are periodic. The USB host initiates USB traffic and the application responds to all USB host commands. In this mode, the USB controller subsystem can be used only in device-type applications. Before an operation, the application must completely program the necessary CSRs in the subsystem.

11.5.3 Operations: USB Control and Status Registers (CSRs)

The subsystem control and status registers provide a high degree of control, making the subsystem both configurable and scalable. The CSRs are divided into two basic categories: global CSRs, which are specific to the USB device, and endpoint CSRs, which are specific to a particular endpoint. Each endpoint has a set of these endpoint-specific CSRs. Interrupt bits in the interrupt registers are cleared on a write of 1 to that bit (R_WC).

11.6 USB Device: Controller / CSR Registers

11.6.1 USB Device Controller / CSR Registers: Map

The endpoint-specific CSRs occupy the first 512 bytes of the map. Each endpoint consumes 32 bytes of memory space in both the IN and OUT directions. The global CSRs, which serve the device, consume 32 bytes of memory space. The following table shows the address allocation for the CSRs.

Offset	Register	Description
IN Endpoint-Specific registers		
0x000	**USB_end0_ctrl_in**	Endpoint 0 control
0x004	**USB_end0_status_in**	Endpoint 0 status
0x008	**USB_end0_buffsize_in**	Endpoint 0 buffer size
0x00C	**USB_end0_packetsize_in**	Endpoint 0 maximum packet size
0x010		Reserved
0x014	**USB_end0_desptr_in**	Endpoint 0 data descriptor pointer
0x018		Reserved
0x01C	**USB_end0_write_confirm**	Endpoint 0 write confirmation (for slave mode)
0x020 - 0x03C	**USB_end1 registers**	Endpoint 1 registers
0x040 - 0x05C	**USB_end2 registers**	Endpoint 2 registers
0x060 - 0x07C	**USB_end3 registers**	Endpoint 3 registers
0x080 - 0x09C	**USB_end4 registers**	Endpoint 4 registers
0x0A0 - 0x0BC	**USB_end5 registers**	Endpoint 5 registers
0x0C0 - 0x1FC		Reserved
OUT Endpoint-Specific registers		
0x200	**USB_end0_ctrl_out**	Endpoint 0 control
0x204	**USB_end0_status_out**	Endpoint 0 status
0x208	**USB_end0_packet_fm_out**	Endpoint 0 packet frame number
0x20C	**USB_end0_buffpacket_out**	Endpoint 0 buffer size out/maximum packet size
0x210	**USB_end0_setup_buffptr**	Endpoint 0 SETUP buffer pointer
0x214	**USB_end0_desptr_out**	Endpoint 0 data descriptor pointer
0x218		Reserved
0x21C	**USB_end0_read_confirm**	Endpoint 0 read confirmation register for zero-length OUT data (for slave mode)
0x220 - 0x23C	**USB_end1 registers**	Endpoint 1 registers
0x240 - 0x25C	**USB_end2 registers**	Endpoint 2 registers
0x260 - 0x27C	**USB_end3 registers**	Endpoint 3 registers
0x280 - 0x29C	**USB_end4 registers**	Endpoint 4 registers
0x2A0 - 0x2BC	**USB_end5 registers**	Endpoint 5 registers
0x2C0 - 0x3FC		Reserved
Global registers		
0x400	**USB_dev_config**	Device configuration
0x404	**USB_dev_ctrl**	Device control
0x408	**USB_dev_status**	Device status
0x40C	**USB_dev_int**	Device interrupt
0x410	**USB_dev_int_mask**	Device interrupt mask
0x414	**USB_end_int**	Endpoint interrupt

Confidential
For Mekra Lang 080124 Internal Use Only

Offset	Register	Description
0x418	**USB_end_int_mask**	Endpoint interrupt mask
0x41C	**USB_test**	Test mode
0x420 - 0x4FC		Reserved
0x500 - 0x7FC	**USB_udc**	UDC registers

Table 11-1. USB CSR Memory Map.

Apart from the subsystem CSRs, the UDC contains CSRs that require 768 bytes of memory space. These CSRs are mapped to the 0x500 - 0x7FC address space. The SETUP command address pointer register (offset address 500H) is no longer writable and returns 0 when read, because the SETUP command address pointer is already hard coded to 0xFFFO in both the subsystem and the UDC core. The application must use offset addresses starting from 0x504, then 0x508, and so on to program the UDC core endpoint buffers. The subsystem maps these offset addresses in turn with the UDC CSR addresses, starting from 0x4, then 0x8, and so on.

Writing to the offset address (offset 0x01C + [endpoint number x 0x020]) confirms the IN data into the TX FIFO. This is applicable only in the Slave mode of operation. The data in the RX FIFO is mapped from address 0x800 up to an address of the (0x800 + 256 * 4 bytes of RX FIFO (RAM)), which is followed by the address space of the TX FIFO.

11.6.2 USB Device Controller / CSR Registers: Details

The global registers are detailed below.

11.6.2.1 USB_dev_config Register

Bits	Name	Access	Reset	Description
31:19				Reserved
18	**set_desc**	RW	0	Indicates the device supports set descriptor requests. Values for this field: 0 - The USB controller subsystem returns a STALL handshake to the USB host 1 - The SETUP packet for the set descriptor request passes to the application
17	**csr_prg**	RW	0	Indicates the device supports dynamic UDC register programming. The application can program the **USB_udc** registers dynamically whenever it has received an interrupt for either a Set Configuration or a Set Interface request. If this bit is enabled, the USB controller subsystem returns a NAK handshake during the status IN stage of both the Set Configuration and Set Interface requests until the application has written 1 to the **csr_done** bit 13 of the **USB_dev_ctrl** register.
16	**halt_status**	RW	0	Indicates whether the UDC-AHR subsystem responds with a STALL or an ACK handshake when the USB host has issued a Clear_Feature (ENDPOINT_HALT) request for Endpoint 0. Values for this field are as follows: 0 - ACK 1 - STALL

Bits	Name	Access	Reset	Description
15:13	**hs_timeout_calib**	RW	0	These three bits indicate the number of PHY clocks to the UDC20-AHB subsystem timeout counter. The application uses these bits to increase the timeout value (736 to 848 bit times in high-speed operation), which depends on the PHY's delay in generating a line state condition. The default timeout value is 736 bit times. These bits are reserved for the UDC-AHB subsystem.
12:10	**fs_timeout_calib**	RW	0	These three bits indicate the number of PHY clocks to the UDC20-AHB subsystem timeout counter. The application uses these bits to increase the timeout value (16 to 18 bit times in full-speed operation), which depends on the PHY's delay in generating line state condition. The default timeout value is 16 bit times. These bits are reserved for the UDC-AHB subsystem.
9	**phy_error_detect**	RW	0	If the application sets this bit, the device detects the PHY_RXVALID or PHY_RXACTIVE input signal to be continuously asserted for 2 ms, indicating PHY error. This bit is reserved for the UDC-AHB subsystem.
8	**status_1**	RW		This bit, together with **status** (Bit 7), provides the option for the UDC-AH9 Subsystem to respond to the USS host with a STALL or ACK handshake if the USB host has issued a non-zero-length data packet during the STATUS-OUT stage of a CONTROL transfer.
7	**status**	RW		This bit, together with **status_1** (Bit 8), provides the option for the USB controller subsystem to respond to the USB host with a STALL or ACK handshake if the USB host has issued a non-zero-length data packet during the STATUS-OUT stage of a CONTROL transfer.
6				Reserved
5	**p1**	RW	1	PHY interface Indicates if the UTMI PHY must support an 8-bit or 16-bit interface. Values for this field are: 0 - 16-bit 1 - 8-bit Note: This bit is only used for the USB controller subsystem.
4	**ss**	RW	0	Indicates that the device supports SyncFrame
3	**sp**	RW	0	Indicates that the device is self-powered
2	**rwkp**	RW	0	Indicates that the device is remote wake up capable
1:0	**spd**	RW	0	Device speed This is the expected speed the application programs to the subsystem. The actual speed the subsystem operates depends on the enumeration speed (**enum_spd**) in the **USB_dev_status** register. Possible values for this field: 0x0 - HS (PHY clock = 30 or 60 MHz) 0x1 - FS (PHY clock = 30 or 60 MHz) 0x2 - LS (PHY clock = 6 MHz) 0x3 - FS (PHY clock = 48 MHz) Note: This bit is only used for the UDC-AHR subsystem.

Table 11-2. USB Device Configuration Register (Global).

For Marvell Internal 1.5 (Oct 2025) Use Only

11.6.2.2 USB_dev_ctrl Register

This register is set at run time and controls the device after device configuration.

Bits	Name	Access	Reset	Description
31:24	**thlen**	RW	0	Threshold Length Indicates the number (**thlen** + 1) of 32-bit entries in the RX FIFO before the DMA can start data transfer.
23:16	**brlen**	RW	0	Burst Length Indicates the length, in 32-bit transfers, of a single burst on the AHB. The subsystem sends number of 32-bit transfers equal to (**brlen** + 1).
15				Reserved
14	**srx_flush**	RW	0	Receive FIFO Flush for Single Receive FIFO. The core must be configured with the UDC20AHB_SNAK_ENH_CC enhancement to use this bit, as this will enable the application to set the **nak** bit (**USB_end[n]_ctrl_in/out)** when the receive FIFO is not empty. When the application wants to flush the receive FIFO, it must first set the **devnak** bit in this **USB_dev_ctrl** register. If the receive DMA is in progress, then the core will finish the current descriptor, terminate the DMA, and flush the data in the receive FIFO and clear this bit. The application must clear this bit after the receive FIFO is empty, by checking the **rxfifo_empty** bit in the **USB_dev_status** register.
13	**csr_done**	W	0	The application uses this bit to notify the USB controller subsystem that the application has completed programming all required UDC registers, and the Subsystem can acknowledge the current Set Configuration or Set Interface command.
12	**devnak**	RW	0	When the application sets this bit, the Subsystem core returns a NAK handshake to all OUT endpoints. By writing 1 to this bit, the application does not need to write 1 to the **snak** (bit 7) of **USB_end[n]_ctrl_in/out**.
11	**scale**	RW	0	Scale Down This bit reduces the timer values inside the USB controller subsystem when running gate-level simulation only. When this bit is set to 1, timer values are scaled down to reduce simulation time. Reset this bit to 0 for normal operation.
10	**sd**	RW	0	Soft Disconnect The application software uses this bit to signal the UDC2O to soft-disconnect. When set to 1, this bit causes the device to enter the disconnected state.
9	**mode**	RW	0	Enables the application to dictate subsystem operation in either DMA mode (1) or Slave mode (0) operation.
8	**bren**	RW	0	Burst Enable When this bit is set, transfers on the AHB are split into bursts.
7	**the**	RW	0	Threshold Enable When this bit is set, a number of quadlets equivalent to the threshold value is transferred from the RX FIFO to the memory.
6	**bf**	RW	0	The DMA is in Buffer Fill mode and transfers data into contiguous locations pointed to by the buffer address.
5	**be**	RW	0	System Endianness A value of 1 indicates a big endian system.

For Marketing Only 1.5 (Confidential)

Bits	Name	Access	Reset	Description
4	**du**	RW	0	Descriptor Update When this bit is set, the DMA updates the descriptor at the end of each packet processed.
3	**tde**	RW	0	Transmit DMA is enabled
2	**rde**	RW	0	Receive DMA is enabled
1				Reserved
0	**res**	RW	0	Resuming Signaling on the USB To perform a remote wake-up resume, the application sets this bit to 1. The USB controller subsystem signals the USB host to resume the USB bus; however, the application must first set **rwkp** (bit 2) in t**SB_dev_config** (indicating that the Subsystem supports the Remote Wakeup feature)

Table 11-3. USB Device Control Registers (Global).

11.6.2.3 USB_dev_status Register

This register reflects status information needed to service some of the interrupts. This is a read-only register.

Bits	Name	Access	Reset	Description
31:18	**ts**	R	0	Frame number of the received SOF For high-speed operation: [31:21] - Millisecond frame number [20:18] - Microframe number For full-speed operation: [31:29] - Reserved [28:18] - Millisecond frame number
17	**rmtwkp_state**	R	0	The state of remote wakeup feature due to Set/Clear Feature (Remotewakeup) command from the host. A value of 1 indicates a Set Feature (Remotewakeup) has been received. A value of 0 indicates Clear Feature (Remotewakeup) has been received. Any change to this bit sets an interrupt for **rmtwkp_state_int** (bit 7 of **USB_dev_int**) if not masked.
16	**phy_error**	R	0	Either the PHY_RXVALID or PHY_RXACTIVE input signal is detected to be continuously asserted for 2 ms, indicating PHY error. The UDC2O-AHB Subsystem goes to the Suspend state as a result. When the application serves the early suspend interrupt **es** (bit 2 of **USB_dev_int**) it also must check this bit to determine if the early suspend interrupt was generated due to PHY error detection.
15	**rxfifo_empty**	R	1	RX FIFO emptiness: 1 - RX FIFO is empty 0 - RX FIFO is not empty

For Ambarella Confidential Use Only

Bits	Name	Access	Reset	Description
14:13	**enum_spd**	R	0	Enumerated Speed These bits hold the speed at which the subsystem comes up after the speed enumeration. If the expected speed (the **spd** bit in **USB_dev_config**) is high speed and the subsystem connects to a 1.1 host controller, then after speed enumeration. These bits indicate that the subsystem is operating in full speed mode (0x1 for **spd** = 0x0). If the **spd** is high speed and the subsystem connects to a 2.0 host controller, then after speed enumeration, these bits indicate that the subsystem is operating in high speed mode (0x0 for **spd** = 0x0). If the speed is low speed or full speed and the subsystem connects to either a 1 .1 or a 2.0 host controller, then after speed enumeration, these bits indicate that the subsystem is operating in low speed mode (0x2 for **spd** = 0x2) or full speed mode (0x1 for **spd** = 0x1, and 0x3 for **spd** = 0x3). Possible values for this field: 0x0 - High speed 0x1 - Full speed 0x2 - Low speed 0x3 - Full speed
12	**susp**	R	0	Suspend status This bit is set as long as a suspend condition is detected on the USB
11:8	**alt**	R	0	This 4-bit field represents the alternate setting, to which the above interface is switched
7:4	**intf**	R	0	This 4-bit field reflects the interface set by the SetInterface command
3:0	**cfg**	R	0	This 4-bit field reflects the configuration set by the SetConfiguration command

Table 11-4. USB Device Status Register (Global).

For Ambarella Confidential use only
1.5 (Oct 13 2025)

11.6.2.4 USB_dev_int Register

Device interrupts are set when there are system-level events. Interrupts are used by the application to make system-level decision. After checking the register, the application must clear the interrupt by writing 1 to the correct bit.

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**rmtwkp_state_int**	R_WC	0	A Set / Clear Feature (Remote Wakeup) is received by the core. This bit is set by the core when **rmtwkp_state (**bit 17 of **USB_dev_status**) changes: HIGH to LOW or LOW to HIGH.
6	**enum**	R_WC	0	Speed enumeration is complete
5	**sof**	R_WC	0	An SOF token is detected on the USB
4	**us**	R_WC	0	A suspend is detected on the USB Note: For the UDC2O-AHB subsystem, there is no Suspend interrupt to the application if the PHY clock is suspended via the SUSPENDM signal
3	**ur**	R_WC	0	A reset is detected on the USB
2	**es**	R_WC	0	An idle state has been detected on the USB bus for 3 milliseconds
1	**si**	R_WC	0	The device has received a Set_Interface command Note: If the application has not served this interrupt, the subsystem returns a NAK handshake to all transactions except the 8 SETUP packet bytes coming from the USB host.
0	**sc**	R_WC	0	The device has received a Set_Configuration command Note: If the application has not served this interrupt, the subsystem returns a NAK handshake to all transactions except the 8 SETUP packet bytes coming from the USB host.

Table 11-5. USB Device Interrupt Register (Global).

11.6.2.5 USB_dev_int_mask Register

The device interrupt mask can be set for system-level interrupts using this register. Programming 1 in the appropriate hit position in the Interrupt Mask register masks the designated interrupt. Once masked, an interrupt signal will not reach the application, nor will its interrupt bit be set.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**mask**	RW	1	Mask equivalent device interrupt bit

Table 11-6. USB Device Interrupt Mask Register (Global).

For Ambarella Internal Training only (.5 (08/ /2025))

11.6.2.6 USB_end_int Register

The endpoint interrupt or USB_end_int register is used to set endpoint-level interrupts. Since all 6 endpoints can be bidirectional, each endpoint has two interrupt bits (one for each direction). The application needs to clear the interrupt by writing 1 to the correct bit after checking the register.

Bits	Name	Access	Reset	Description
31:22				Reserved
21:16	**out_ep**	R_WC	0	One bit per OUT endpoint, set when event on that endpoint
15:6				Reserved
5:0	**in_ep**	R_WC	0	One bit per IN endpoint, set when event on that endpoint

Table 11-7. USB Endpoint Interrupt Register (Global).

11.6.2.7 USB_end_int_mask Register

This register is used to mask endpoint interrupts. A write of 1 to any bit in this register masks the corresponding endpoint for possible interrupts. Once masked, an interrupt signal does not reach the application nor will its interrupt bit be set.

Bits	Name	Access	Reset	Description
31:22				Reserved
21:16	**out_ep_mask**	RW	1	Masks interrupts to the OUT endpoint equivalent to this value
15:6				Reserved
5:0	**in_ep_mask**	RW	1	Masks interrupts to the IN endpoint equivalent to this value

Table 11-8. USB Endpoint Interrupt Mask Register (Global).

The USB device endpoint-specific CSR registers are detailed below.

11.6.2.8 USB_end[n]_ctrl_in/out Registers

This register is used to program each endpoint (number N) as required by the application. If the endpoint is bidirectional, there will be two such endpoint registers.

Bits	Name	Access	Reset	Description
31:12				Reserved

Confidential
For Beijing Yang only
(2025-08-01)

Bits	Name	Access	Reset	Description
11	**close_desc**	RW	0	Close descriptor channel for this endpoint This bit applies only to OUT endpoints. The application sets this bit to close the descriptor channel, and the UDC Subsystem clears this bit after the channel is closed. This bit provides the application with a mechanism to close the descriptors in cases where the USB host does not indicate an end-of-transfer (by issuing a short packet to the USB device). To close the descriptor channel for a particular endpoint, the application sets the **close desc** bit in the Endpoint Control register (**USB_end[n]_ctrl_in/out**). When the channel is closed, the UDC Subsystem clears this bit and generates an interrupt. This bit must be used only for bulk and interrupt OUT endpoints. In addition, before closing the descriptor, software must ensure that the current descriptor reachable by the DMA is active (buffer status is Host Ready). When closed, the descriptor is marked with the Last bit set. When the descriptor is closed, it is assigned one of the following descriptor statuses: Buffer Fill mode: Accumulated byte count is available in the RX bytes field. Packet-Per-Buffer With Descriptor Update mode: Current reachable descriptor is marked with the Last descriptor, and the RX Bytes field is set to 0. Packet-Per-Buffer Without Descriptor Update mode: Current reachable descriptor is marked with the Last descriptor, and the accumulated byte count is available in the RX bytes field.
10	**send_null**	RW	0	Send NULL packet This bit provides the application with a mechanism to instruct the UDC subsystem to send a NULL (zero-length) packet when no data is available in the particular endpoint's TX FIFO. If this bit is set, when no data is available in the endpoint's TX FIFO, the UDC Subsystem sends a NULL packet.
9	**rrdy**	RW	0	Receive Ready If this bit is set by the application, on receiving an OUT packet, the DMA sends the packet to system memory. This bit is deasserted at the end of packet if the Descriptor Update bit **du** is set in the Device Control register **USB_dev_ctrl**. This bit is deasserted at the end of payload if the Descriptor Update bit is deasserted. This bit can be set by the application at any time. The application cannot clear this bit if the DMA is busy transferring the data.
8	**cnak**	W	0	Clear NAK Used by the application to clear **nak** (bit 6, below). After the subsystem sets **nak**, the application must clear it with a write of 1 to **cnak**. (For example, after the application has decoded the SETUP packet and determined it is not an invalid command, the application must set the **cnak** bit of the control endpoint to 1 to clear the **nak** bit.) The application also must clear the **nak** bit whenever the subsystem sets it. (The subsystem sets it due to the application setting the Stall bit.) The application can write **cnak** without waiting for a RX FIFO empty condition. The **nak** bit is cleared immediately upon write to **cnak** bit. No polling is necessary.

For Mass Deployment 1.5 (Confidential)

Bits	Name	Access	Reset	Description
7	**snak**	W	0	Set NAK Used by the application to set **nak** (bit 6). The application must not set the **nak** bit for an IN endpoint until it has received an IN token interrupt indicating that the TX FIFO is empty.
6	**nak**	R	0	After a SETUP packet, which is decoded by the application, is received by the core, the core sets the **nak** bit for all control IN/OUT endpoints. **nak** is also set after a STALL response for the endpoint (the STALL bit **s** is bit 0 in this Endpoint Control register **USB_end[n]_ctrl_in/out**). 1 - The endpoint responds to the USB host with a NAK handshake. 0 - The endpoint responds normally. Note 1: A SETUP packet is sent to the application regardless of whether the **nak** bit is set. Note 2: If the NAK for ISOC OUT endpoint is set, then the data out from the host is accepted, provided there is space in the FIFO.
5:4	**et**	RW	0	Endpoint Type The possible values for this field are: 0x0 - Control endpoint 0x1 - Isochronous endpoint 0x2 - Bulk endpoint 0x3 - Interrupt endpoint
3	**p**	RW	0	Poll Demand Poll demand from the application. The application can set this bit after an IN token is received from the endpoint. The application can also set this bit before an IN token is received for the endpoint, if it has the IN transfer data in advance. Note: After sending a zero-length or short packet, the application must wait for the next **xferdone_txempty** interrupt (**USB_end[n]_status_in/out**), before setting up the **p** bit for the next transfer.
2	**sn**	RW	0	Configures the endpoint for Snoop mode. In this mode, the subsystem does not check the correctness of OUT packets before transferring them to application memory. Reserved for IN endpoints.
1	**f**	RW	0	Flush the TX FIFO Reserved for OUT endpoints. The application firmware sets this bit to 1 after it has detected a disconnect/connect on the USB cable, then waits for the IN token endpoint interrupt before resetting this bit to 0. This flushes the stale data out of the TX FIFO. This bit is cleared by the core when Transmit DMA Complete occurs on this endpoint (**tdc** bit in **USB_end[n]_status_in/out**)

For Mass Production
1.5 (CV22FS_Datasheet_1.5.pdf)

Bits	Name	Access	Reset	Description
0	**s**	RW	0	STALL Handshake On successful reception of a SETUP packet (decoded by the application), the subsystem clears both IN and OUT stall bits (**s**), and sets both the IN and OUT **nak** bits. The application must check that the RX FIFO is empty before setting the IN and OUT STALL bit **s**. For non-SETUP packets, the subsystem clears either IN or out **s** bits only if a STALL handshake is returned to the USB host, then sets the corresponding **nak** bit. The subsystem returns a STALL handshake for the subsequent transactions of the stalled endpoint until the USB host issues a CLEAR_FEATURE command to clear it. Once this bit is set, if the subsystem has already returned a STALL handshake to the USB host, the application firmware cannot clear the **s** bit to stop the subsystem from sending the STALL handshake on the endpoint. The host must instead send one of the following commands to clear the endpoint Halt status: Clear Feature (Halt) SetConfiguration SetInterface Note: The application can set the STALL bit without waiting for a RX FIFO empty condition. No polling is necessary.

Table 11-9. USB Endpoint Control Register (Endpoint Specific).

11.6.2.9 USB_end[n]_status_in/out Registers

Bits	Name	Access	Reset	Description
31:29				Reserved
28	**cdc_clear**	R_WC	0	This bit is valid only for OUT endpoints when **close_desc** is enabled in **USB_end[n]_ctrl_in/out**. This bit will be set by the core HW when **close_desc** is cleared by the HW generating an interrupt. After servicing the interrupt the application SW must clear this bit.
27	**xferdone_txempty**	R_WC	0	This bit indicates that the TX FIFO is empty after the DMA transfer has been completed. The application can use this bit to set the poll bit for the next transfer. The application must first clear this bit after servicing the interrupt.
26	**rss**	R_WC	0	Received Set Stall This bit indicates that the Set Feature (ENDPOINT_HALT) command is received for this endpoint. To stall this endpoint, the application can set the stall bit **s** in **USB_end[n]_ctrl_in/out**. After this, the application clears this **rss** bit to acknowledge the reception of Set Feature stall command. Once this **rss** bit is cleared, the core sends the zero-length packet for the Status IN phase of Set Feature command. Received Set Stall **rss** indication is applicable only for Bulk and Interrupt transactions.

Confidential
For Mekra Lang
1 (Oct 2020)

Bits	Name	Access	Reset	Description
25	**rcs**	R_WC	0	Received Clear Stall Indication This bit indicates that the Clear Feature (ENDPOINT_HALT) command is received for this endpoint. To continue to stall the endpoint, the application must set the stall bit **s** in **USB_end[n]_ctrl_in/out**. After this, the application clears this **rcs** bit to acknowledge the reception of clear stall command. Once this bit is cleared, the core sends the zero-length packet for the Status IN phase of clear stall command. Received Clear Stall indication is applicable only for Bulk and Interrupt transactions.
24				Reserved
23	**iso_in_done**	RW	0	Isochronous IN transaction for the current microframe is complete. This bit indicates that the isochronous IN transaction for this endpoint is complete. The application can use this information to program the isochronous IN data for the next microframe. This bit is used only in Slave-Only mode.
22:11	**rx_pkt_size**	RW	0	Receive Packet Size Indicates the number of bytes in the current receive packet the RX FIFO is receiving. Because the USB host always sends 8 bytes of SETUP data, these bits do not indicate the receipt of 8 bytes of SETUP data for a SETUP packet. Rather, these bits indicate the configuration status (Configuration number [22:19], Interface number [18:15], and Alternate Setting number [14:11]). This field is used in slave mode only. In DMA mode, the application must check the status from the endpoint data descriptor.
10	**tdc**	R_WC	0	Transmit DMA Completion Indicates the transmit DMA has completed transferring a descriptor chain's data to the TX FIFO. After servicing the interrupt, the application must clear this bit.
9	**he**	R_WC	0	Error response on the host bus (AHB) when doing a data transfer, descriptor fetch, or descriptor update for this particular endpoint. After servicing the interrupt, the application must clear this bit.
8				Reserved
7	**bna**	R_WC	0	Buffer Not Available The subsystem sets this bit when the descriptor status is either Host Busy or DMA Done to indicate that the descriptor was not ready at the time the DMA tried to access it. After servicing the interrupt, the application must clear this bit.
6	**in**	R_WC	0	An IN token has been received by this endpoint. After servicing the interrupt, the application must clear this bit. (Reserved for OUT endpoints.)

For Marvell Confidential 1.5 Use Only

Bits	Name	Access	Reset	Description
5:4	**out**	R_WC	0	An OUT packet has been received by this endpoint. The encoding of these two bits indicates the type of data received. The possible values for this field are: 0x0 - None 0x1 - Received data 0x2 - Received SETUP data (8 bytes) 0x3 - Reserved (The application must write the same values to clear these bits.) Note: In Slave mode the application must clear these bits only after clearing the FIFO (in other words, only when the RX FIFO is empty). Clearing these bits while FIFO is not empty causes these bits to remain set until the FIFO is cleared. If by that time the Set/Clear Feature command referring to this endpoint is received, then this Endpoint Status register **USB_end[n]_status_in/out** shows both **out** and **rcs/rss** bits, which the application might not be able to handle properly.
3:0				Reserved

Table 11-10. USB Endpoint Status Register (Endpoint Specific).

11.6.2.10 USB_end[n]_buffsize_in / USB_end[n]_packet_fm_out Registers

This dual-function register holds the endpoint buffer size when the endpoint is an IN endpoint. When the endpoint is an OUT endpoint, the register contains the frame number in which the packet is received, updated in bits 13:0. This frame number information is useful when handling isochronous traffic.

Bits	Name	Access	Reset	Description
31:18				Reserved
17:16 (In)	**iso_in_pid**	RW	0	Initial data PID to be sent for a high-bandwidth isochronous IN transaction. This field is used only in Slave-Only mode. 0x0 - DATA0 PID is sent 0x1 - DATA0 PID is sent 0x2 - DATA1 PID is sent 0x3 - DATA2 PID is sent
17:16 (Out)	**iso_out_pid**	R	0	Data PID received for a high-bandwidth isochronous OUT transaction. This field indicates that the data PID for the current packet is available in the Receive FIFO. This field is used only in Slave-Only mode. 0x0 - DATA0 PID is received 0x1 - DATA1 PID is received 0x2 - DATA2 PID is received 0x3 - MDATA PID is received
15:0 (In)	**buff_size**	RW	0	Buffer size required for this endpoint The application can program this field to make each endpoint's buffers adaptive, providing flexibility in buffer size when the interface or configuration is changed. This value is in 32-bit words, and indicates the number of 32-bit word entries in the Transmit FIFO. (IN only)

For Weka Beijing Only (1.5 Confidential)

Bits	Name	Access	Reset	Description
15:0	**frame_number**	R	0	Frame number in which the packet is received For high-speed operation: [15:14] - Reserved [13:3] - Millisecond frame number [2:0] - Microframe number For full-speed operation: [15:11] - Reserved [10:0] - Millisecond frame number

Table 11-11. USB Buffer Size/Frame Number Register (Endpoint Specific).

11.6.2.11 USB_end[n]_buffpacket_out Register

This register holds the endpoint buffer size when the endpoint is an OUT endpoint. This register also specifies the maximum packet size an endpoint should support. This maximum size is used to calculate whether the Receive FIFO has sufficient space to accept a packet. When changing the maximum packet size for a specific endpoint, the user must also program the UDC register space.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**max_pkt_size**	RW	0	Maximum packet size for the endpoint (value in bytes)

Table 11-12. USB Endpoint Buffer Size OUT/Maximum Packet Size Register (Endpoint Specific).

11.6.2.12 USB_end[n]_setup_buffptr Register

Endpoint SETUP buffer pointers are used for SETUP commands. This Endpoint Setup Buffer Pointer register **USB_end[n]_SETUP_buffptr** tracks control endpoint buffer registers; endpoint buffer registers for all other endpoint types are reserved. The **USB_end[n]_SETUP_buffptr** is used only in DMA mode. This is applicable only to control endpoints. For all other endpoints the register is reserved.

Bits	Name	Access	Reset	Description
31:0	**subptr**	RW	0	SETUP Buffer Pointer

Table 11-13. USB Endpoint SETUP Buffer Pointer Register (Endpoint Specific).

11.6.2.13 USB_end[n]_desptr_out Register

This register contains data descriptor pointers. Both IN and OUT endpoints have one data descriptor pointer each.

Bits	Name	Access	Reset	Description
31:0	**desptr**	RW	0	Descriptor Pointer

Table 11-14. USB Endpoint Data Descriptor Pointer Register (Endpoint Specific).

11.6.2.14 USB_end[n]_read/write_confirm Registers

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**confirm**	W	0	Read or Write confirm bit

Table 11-15. USB Endpoint [n] Read Confirm and Write Confirm registers.

11.6.2.15 USB_test Register

The Test Mode register (**USB_test**) sets the USB Controller Subsystem to Test mode. In Test mode, the application can use the AHB to read from a TX FIFO or write to an RX FIFO. Test mode is supported only in the Slave operational mode. In Test mode, only single DWORD transactions are supported: byte and word transactions on the AHB are not supported.

While in Test mode, all transactions from the USB host, except the SETUP packet, receive a NAK response. When the SETUP packet is received, it is accepted, but the data is not written into the Receive FIFO. After writing the TX FIFO, the data must be confirmed by writing to the offset address (offset 0x01C + [endpoint number x 0x020]), which is similar to the normal mode of operation. In non-Test modes, reading from a TX FIFO or writing to an RX FIFO results in an AHB error response. The application must never read an empty TX FIFO or write a full RX FIFO even though an AHB error response is not provided. Writing to the RX FIFO in Test mode and enabling the DMA to transfer data are not supported.

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**tstmode**	RW	0	Test Mode indicator

Table 11-16. USB Test Mode Register.

11.6.2.16 USB_udc2O Endpoint Register

Bits	Name	Access	Reset	Description
31:30				Reserved
29:19	**max_packet_size**	RW	0	Maximum packet size
18:15	**alt_setting_end**	RW	0	Alternate setting to which this endpoint belongs
14:11	**interface_no_ end**	RW	0	Interface number to which this endpoint belongs
10:7	**config_no_end**	RW	0	Configuration number to which this endpoint belongs
6:5	**end_type**	RW	0	Endpoint type The possible values for this field are: 0x0 - Control 0x1 - Isochronous 0x2 - Bulk 0x3 - Interrupt
4	**end_direction**	RW	0	Endpoint direction The possible values for this field are: 0 - Out 1 - In

Bits	Name	Access	Reset	Description
3:0	**logical_end_no**	RW	0	Logical Endpoint Number

Table 11-17. USB UDC2O Endpoint Register.

11.7 USB Device: Initialization

To initialize the UDC20-AHB Subsystem, application software must program the following registers:

- Device Configuration register (**USB_dev_config** in Section 11.6.2.1).
- Device Control register (**USB_dev_ctrl** in Section 11.6.2.2).
- Endpoint Control registers for Logical Endpoint 0, 1, 2, and so on (**USB_end[n]_ctrl_in/out** in Section 11.6.2.8).
- Endpoint Buffer Size registers for Logical Endpoint 0, 1, 2, and so on (**USB_end[n]_buffsize_in** in Section 11.6.2.10).
- Endpoint Max Packet Size registers for Logical Endpoints 0, 1, 2, and so on (**USB_end[n]_buffpacket_out** in Section 11.6.2.11).
- Endpoint 0 Setup Buffer Pointer register (**USB_end[n]_setup_buffptr** in Section 11.6.2.12).
- UDC20 registers for Physical Endpoint 1, 2, and so on (**USB_udc2O Endpoint** in Section 11.6.2.16).

11.8 USB Device: DMA Operation

11.8.1 Overview

This section explains the functionality of the UDC-AHB subsystem in DMA mode. A DMA-based implementation of the UDC-AHB subsystem is best suited to designs requiring a high degree of configurability and software control. A major advantage of DMA-based implementations is that they spare the main processor's computing power from involvement in data movement tasks. Use of a scatter-gather DMA helps applications make efficient and optimal use of system memory, a major design constraint on portable systems. The DMA mode of implementation demands that data be presented in predefined memory structure formats. (These memory structures are explained in detail in the following sections.)

In DMA mode, the UDC-AHB subsystem implements a true scatter-gather memory distribution, in which memory structures are scattered over the system memory. Each endpoint memory structure is implemented as a linked list, where each element of the list is a data buffer of a predefined size. In addition to data, each buffer also has a status quadlet and a pointer to the next buffer. The last element of such a linked list can point either to a null pointer, or, if the list is implemented as a ring buffer, to the first element of the list. All the endpoints implement these structures in the memory, and apart from these structures, all control endpoints implement an additional 16-byte buffer to store SETUP data. The SETUP data structure does not implement a linked-list structure.

The descriptor memory structures are displayed in the following figure.

For Media Lang Confidential 1.5 (08012008)

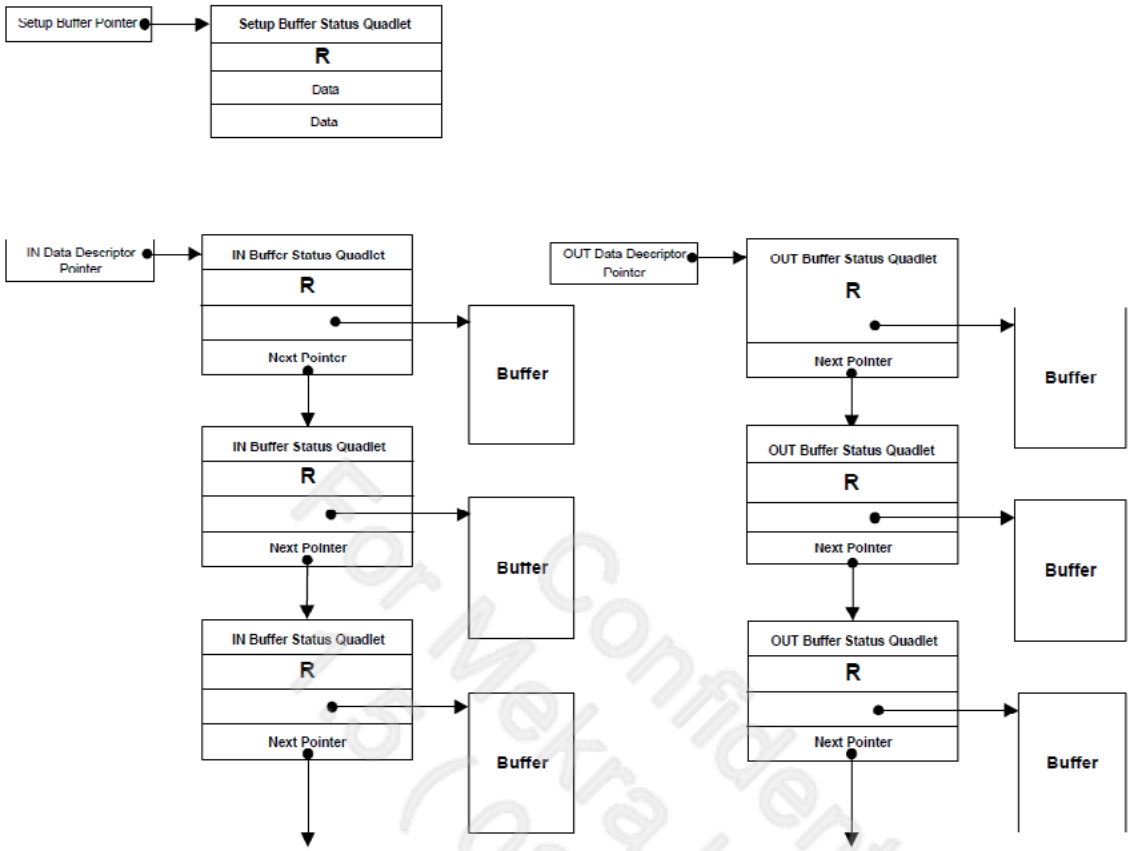


Figure 11-1. Descriptor Memory Structure.

11.8.2 SETUP Data Memory Structure

The following figure shows the memory structure for SETUP data. The accompanying table describes the SETUP data memory structure values.

Notes:

- The subsystem receives SETUP data only from the USB host
- The buffer size for SETUP data is always 16 bytes
- The SETUP buffer applies only to control endpoints

A “Host Busy” buffer status indicates that the application is accessing the location. DMA does not check the buffer status before transferring the 8-byte SETUP packet. The application must always give highest priority to, and have the buffer ready for, a SETUP packet.

Confidential For Mexra Lang only (08012023)

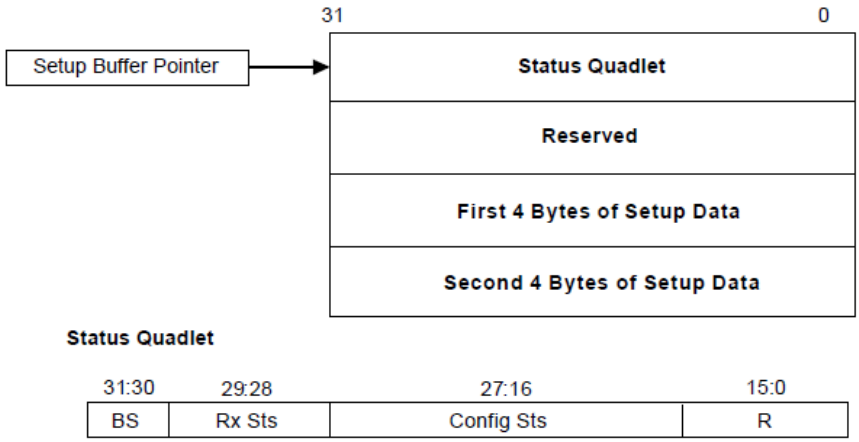


Figure 11-2. SETUP Data Memory Structure.

Bit	Bit ID	Description
BS [1:0]	**Buffer status**	Status of the data buffer. The options are as follows: 2'b00: Host ready 2'b01: DMA busy 2'b10: DMA done 2'b11: Host busy
Rx Sts [1:0]	**Receive status**	Status of received SETUP data. This reflects whether the SETUP data is received correctly or with errors. The possible combinations are as follows: 2'b00: Success 2'b01: DESERR 2'b10: Reserved 2'b11: BUFERR
Config Sts [27:16]	**Configuration status**	Status of the current configuration associated with the SETUP packet for a control endpoint (endpoint type “00”, which is different from the default endpoint 0). • Configuration number [27:24] • Interface number [23:20] • Alternate setting number [19:16]
R [15:0]	**Reserved**	Reserved for future use

Table 11-18. SETUP Data Memory Structure Values.

11.8.3 OUT Data Memory Structure

All endpoints that support OUT direction transactions (endpoints that receive data from the USB host), must implement a memory structure with the following characteristics:

- Each data buffer must have a descriptor associated with it that provides the status of the buffer. The buffer itself contains only raw data.
- Each buffer descriptor is 4 quadlets in length.

If the buffer status of the first descriptor is “Host Ready,” the DMA fetches and processes its data buffer; otherwise the DMA skips to the next descriptor until it reaches the end of the descriptor chain. The buffers the descriptor points to hold packet data for non-isochronous endpoints and frame data for isochronous endpoints. Buffer-fill mode does not support Isochronous OUT endpoints; instead, use packet-per-buffer mode.

For OUT data memory structure:

- **Host Ready** indicates that the descriptor is available for the DMA to process.
- **DMA Busy** indicates that the DMA is still processing the descriptor.
- **DMA Done** indicates that the buffer data transfer is complete.
- **Host Busy** indicates that the application is processing the descriptor.
- **A DESERR receive status** indicates that the buffer status is something other than host ready (2'b00) during descriptor fetch.

The OUT data memory structure is shown in the following figure, followed by a table of OUT data memory structure values.

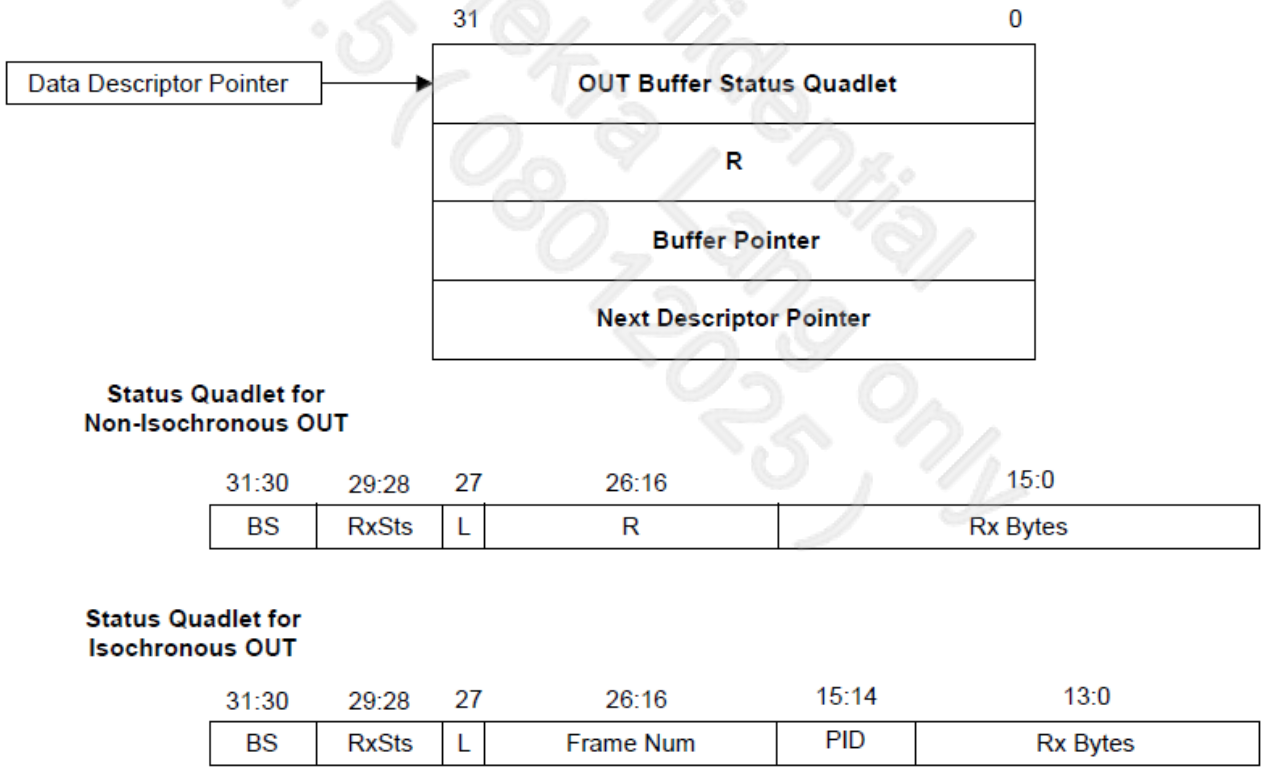


Table 11-19. OUT Data Memory Structure.

For Mekta Lang only
Confidential
1.5 (08012025)

Bit	Bit ID	Description
BS [31:30]	**Buffer status**	This 2-bit value describes data buffer status. Possible options are: • 2'b00: Host Ready • 2'b01: DMA Busy • 2'b10: DMA Done • 2'b11: Host Busy
Rx Sts [29:28]	**Receive status**	This 2-bit value describes the status of the received data. This reflects whether OUT data has been received correctly or with errors. The possible combinations are as follows: • 2'b00: Success • 2'b01: DESERR • 2'b10: Reserved • 2'b11: BUFERR
L [27]	**Last**	When set, this bit indicates that this descriptor is the last one of the chain.
Frame Number [26:16]	**Frame number**	The 11-bit frame number in which the current ISO-OUT packet is received.
PID [15:14] (ISO OUT)	**ISO Received data PID**	This field is for only high-speed isochronous transactions, and indicates the data PID for an isochronous receive packet. • 2'b00: The packet contained in this descriptor is received with a data PID of DATA0. • 2'b01: The packet contained in this descriptor is received with a data PID of DATA1. • 2'b10: The packet contained in this descriptor is received with a data PID of DATA2. • 2'b11: The packet contained in this descriptor is received with a data PID of MDATA. **Note**: These PID bits are only used for database Release 2.20a and later, and cannot be applied to older releases. For full-speed isochronous transactions, this is a “don't care” field.
Rx Bytes [15:0] (non-ISO OUT)	**Received number of bytes**	This 16-bit value can take values from 0 to 64K bytes*, depending on the packet size of data received from the USB host. ***Note**: Actual value is (64K – 1) bytes. Equivalent to 'hFFFF.
Rx Bytes [13:0] (ISO OUT)	**Received number of bytes**	This 14-bit value can take values from 0 to 16K bytes*, depending on the packet size of data received from the USB host. ***Note**: Actual value is (16K – 1) bytes. Equivalent to 'h3FFF.

Table 11-20. OUT Data Memory Structure Values.

11.8.4 IN Data Memory Structure

All endpoints that support IN direction transactions (transmitting data to the USB host) must implement the following memory structure. Each buffer must have a descriptor associated with it. The application fills the data buffer, updates its status in the descriptor, and sets the Poll Demand bit. The DMA fetches this descriptor and processes it, moving on in this fashion until it reaches the end of the descriptor chain.

The buffers the descriptor points to hold packet data for non-isochronous endpoints and frame data for isochronous endpoints.

The IN data memory structure is shown in the following figure.

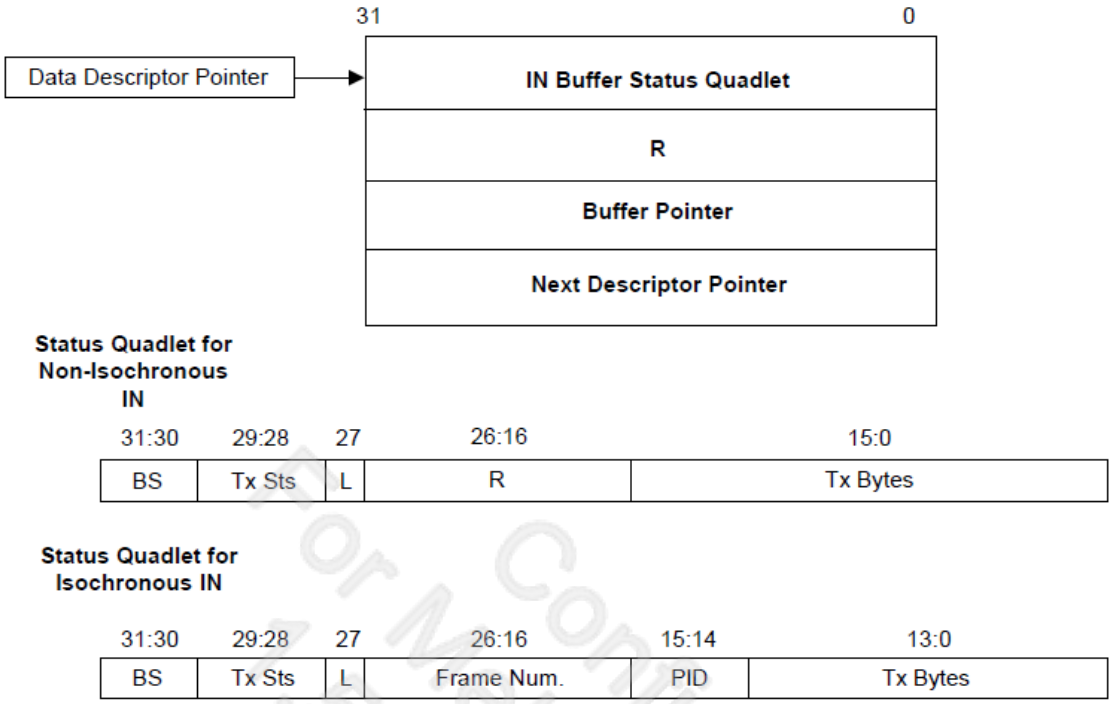


Figure 11-3. IN Data Memory Structure.

Bit	Bit ID	Description
BS [31:30]	**Buffer status**	This 2-bit value describes the status of the data buffer. The possible options are: • 2'b00: Host ready • 2'b01: DMA busy • 2'b10: DMA done • 2'b11: Host busy
Tx Sts [29:28]	**Transmit status**	The status of the transmitted data. This reflects if the IN data has been transmitted correctly or with errors. The possible combinations are as follows: • 2'b00: Success • 2'b01: DESERR • 2'b10: Reserved • 2'b11: BUFERR
L [27]	**Last**	When set, this bit indicates that this descriptor is the last one of the chain
Frame Number [26:16]	**Frame number**	Frame number in which the current packet must be transmitted. • [26:19]: Millisecond frame number • [18:16]: Microframe number

Confidential
For Nekra Lang only
4.5 (08 Dec 2021)

Bit	Bit ID	Description
PID [15:14] (ISO IN)	**Number of packets per frame**	This 2-bit value indicates the number of packets per µSOF (microframe) for isochronous IN transfers during high-speed operation. The application must program these bits in the descriptor (these bits must be the same for all descriptors of the same µSOF) such that the subsystem core returns an isochronous packet with an appropriate data PID per frame. These bits are reserved for full-speed operation. • 2'b00 and 2'b01: 1 packet per microframe • 2'b10: 2 packets per microframe • 2'b11: 3 packets per microframe **Note**: These PID bits are only used for database release 1.8b and later, and cannot be applied to older releases. For full-speed transactions, these bits are reserved.
Tx bytes [15:0] (non-ISO IN)	**Number of bytes to be transmitted**	This 16-bit value can take values from 0 to 64K bytes*, indicating the number of bytes of data to be transmitted to the USB host. ***Note**: Actual value is (64K – 1) bytes. Equivalent to 'hFFFF
Tx bytes [13:0] (ISO IN)	**Number of bytes to be transmitted**	This 14-bit value can take values from 0 to 16K bytes*, indicating the number of bytes of data to be transmitted to the USB host. ***Note**: Actual value is (16K – 1) bytes. Equivalent to 'h3FFF

Table 11-21. IN Data Memory Structure Values.

For Mekra Lang only (08.01.2025) Confidential 1.5

11.8.5 Building Blocks

The DMA block, which is inactive in Slave-Only mode, is built of three basic components:

- Transfer engine—moves the actual data
- DMA controller—controls the movement of the data
- AHB interface—manages the flow of data between the DMA and AHB for both data transfer and CSR accesses

These three blocks are explained in greater detail in the following sections.

11.8.5.1 Transfer Engine

The transfer engine is slaved to the DMA controller for the movement of data to and from system memory. When there is a memory access, the transfer engine interfaces with the FIFOs and the AHB Interface module, and indicates to the DMA whether or not the transfer was successful.

If the transfer was unsuccessful, the transfer engine will indicate how many bytes were successfully transferred to the destination, to enable the DMA to decide whether to retry the transaction. When transferring data from the FIFOs, the transfer engine depends on status signals from the FIFO's respective FIFO controllers. When transferring the data to system memory, the transfer engine registers the data, waits for the request from the DMA controller, and then decides on the direction of the transfer. It completes the transfer whether the transaction is completed successfully or if an error occurs during the data transfer.

11.8.5.2 DMA Controller

The DMA controller controls data movement to and from system memory. It parses the descriptor structures and instructs the other subsystem blocks to perform data transfers accordingly. The DMA controller has two different controllers, enabling it to control both transmit (IN) and receive (OUT) data simultaneously (although transmit and receive functions cannot be performed simultaneously).

The DMA controller architecture has a descriptor buffer to store descriptors fetched from memory.

11.8.5.3 AHB Interface

This block contains all the subsystem's AHB protocol logic. This block has two functional states: it functions as a slave when the application programs the subsystem's or the UDC's CSRs; and, as master when the DMA performs data transfers. When acting as AHB master, the subsystem accesses the application memory for descriptors and data buffers. When the subsystem is in a non-DMA (Slave-Only) mode, the AHB Interface also functions as a slave. In this mode, all the FIFOs are memory-mapped, and the application writes directly to the FIFOs.

11.8.5.3.1 AHB Master BIU

The AHB master BIU breaks the transfer at the 1K-byte page boundary and then immediately starts a new transfer with a new incremented address without inserting any wait state. This block interfaces with the transfer engine and translates the transfer engine's commands into appropriate AHB commands. This block also handles retry and split responses from the AHB target. The AHB master BIU conveys only error response information to the transfer engine and lets either the DMA or the transfer engine decide on the response.

This block's basic functionality block is summarized below.

- Supports 8-, 16-, and 32-bit read/write access on the AHB
- Supports both big and little endian FIFO data access (default is little endian)
- Supports only little endian descriptor access
- Supports HBURST values SINGLE, INCR, INCR 4/8/16 (WRAP 4/8/16 is not supported)
- Accepts split, retry, and error responses
- Cannot lock transactions

The AHB master BIU does not automatically align all transfers within a burst to an address boundary equal to the size of the transfer. The application must perform this alignment. Descriptor information in system memory and buffer pointers must be word-aligned.

The following table describes the HBURST generation by the AHB master based on different conditions.

Descriptor Access	Remaining Bytes	Burst Enable	Burst Length	HBURST
Yes	Don't care	No	Don't care	SINGLE
Yes	Don't care	Yes	8'h00, 8'h01, or 8'h02	INCR

Confidential For Model Language only (08/12/2025)

Descriptor Access	Remaining Bytes	Burst Enable	Burst Length	HBURST
Yes	Don't care	Yes	8'h03 or greater	INCR4
No	Don't care	No	Don't care	SINGLE/INCR
No	< Burst length	Yes	Don't care	SINGLE/INCR
No	> or = Burst length	Yes	8'h00	SINGLE
No	> or = Burst length	Yes	8'h03	INCR4
No	> or = Burst length	Yes	8'h07	INCR8
No	> or = Burst length	Yes	8'h0f	INCR16
No	> or = Burst length	Yes	Other than 00, 03, 07, and 0f	INCR

Table 11-22. HBURST Generation.

For Ambarella Confidential Use Only (08012025)

11.8.5.3.2 AHB Slave BIU

The AHB slave BIU block acts as a slave on the AHB and responds to all subsystem CSR accesses from the AHB master. The basic functionality of this block is summarized below.

- Supports 8-, 16-, and 32-bit read/write access on the AHB
- Supports both big and little endian FIFO data access (default is little endian)
- Supports only little endian CSR access
- Supports retry and error responses
- Does not support split transactions
- Accepts single transfers using either an unspecified-length incrementing burst (INCR) with a burst length of one, or HBURST=single with a burst length of one
- Accepts HBURST values SINGLE, INCR, INCR 4/8/16 (except WRAP 4/8/16)
- Generates an error response under the following conditions:
  - A wrap transfer is attempted
  - A read from the TXFIFO or write to the RXFIFO is attempted
  - A read from an empty RXFIFO is attempted
  - A read of excess data from the RXFIFO is attempted when the RXFIFO has run out of data
  - A write to a full TXFIFO is attempted
  - A transfer is attempted with a non-SINGLE or non-INCR (with one data beat) signal to the CSR.
  - A transfer is attempted with an illegal HSIZE setting (other than 8, 16, or 32 bits)
- Does not support back-to-back transfers for two packets. For example, if there are two packets in the RX-FIFO, after the first is read, the second cannot be read back-to-back. The UDC-AHB subsystem expects the application to wait for the proper interrupt before reading data from the RXFIFO. This also applies to reads followed back-to-back by writes.

11.8.6 Features

In all modes, an interrupt for an OUT transaction automatically disables the DMA Receive Engine, which must be re-enabled by the application. This provides the application sufficient time to service the interrupt.

11.8.6.1 Packet-Per-Buffer Mode

In this mode, the DMA transfers packet by packet to various addresses as indicated by the descriptor. This is a true scatter-gather mechanism. A descriptor update can either happen at the end of each packet transfer or only at the end of the descriptor chain. The application may be interrupted after processing each descriptor or at the end of a descriptor chain.

Writing 1'b1 to the DU bit of the Device Control register enables descriptor updating and application interrupt to the software at the end of each packet.

11.8.6.1.1 Descriptor Update Enable Operation

In this mode, the DMA updates the descriptor and interrupts the application after each packet is transferred, until it reaches the end of the descriptor chain.

IN Transaction

The following figures show the DMA processing an IN token with and without descriptor update, respectively.

For Mekra Lang only (08012025) Confidential 1.5

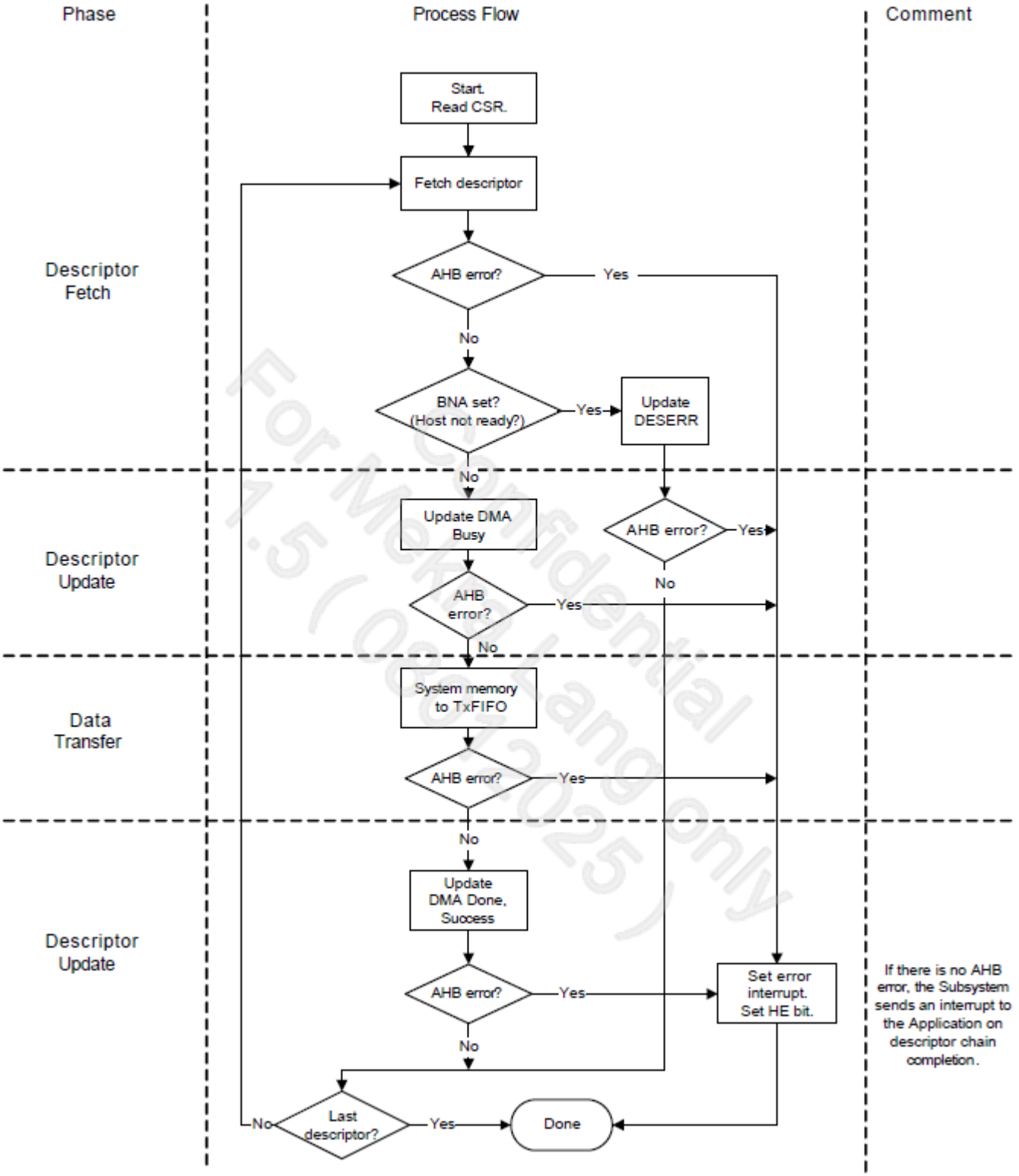


Table 11-23. Control / Bulk-IN Transaction in Packet-Per-Buffer With Descriptor Update Mode.

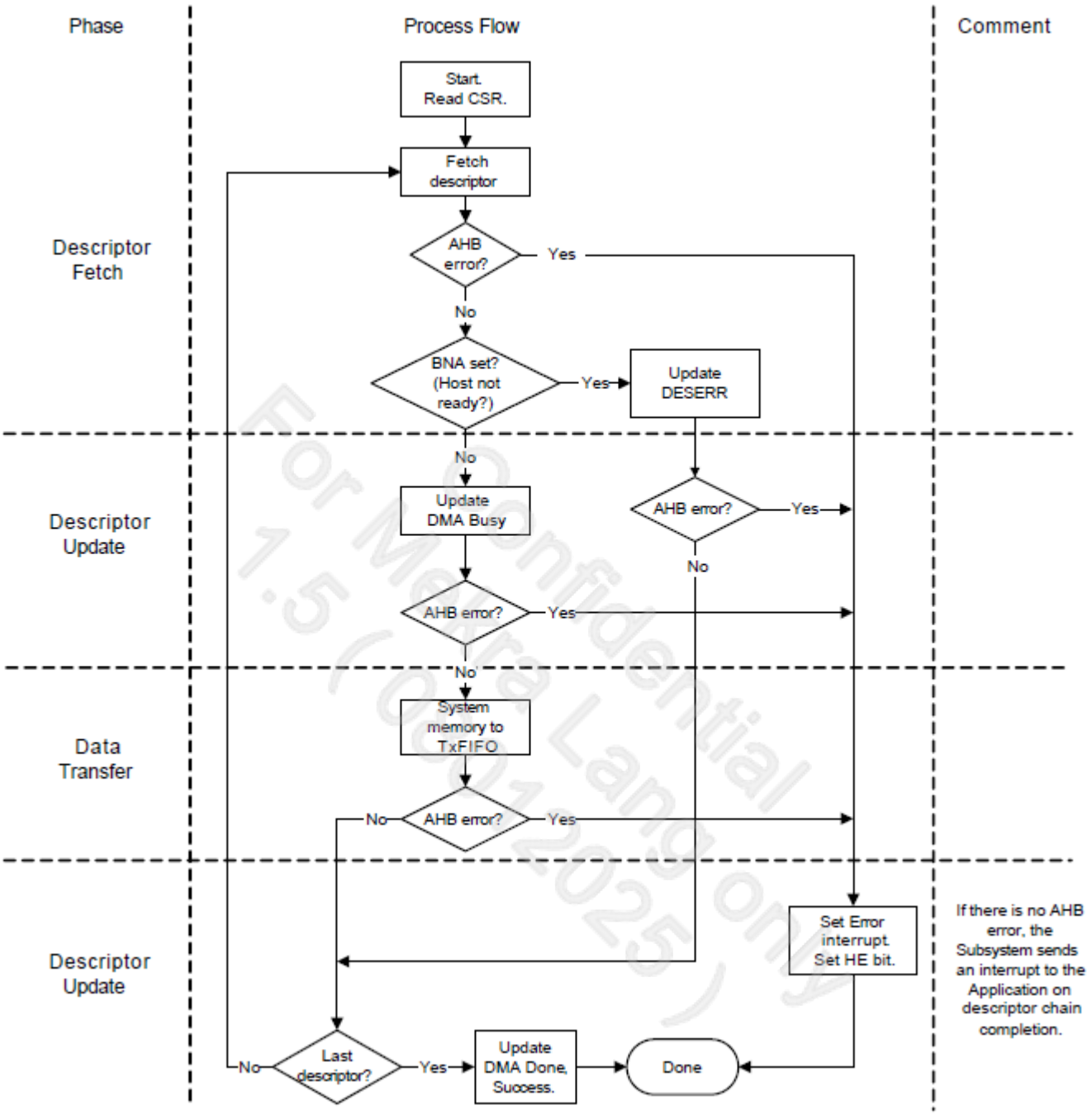


Table 11-24. Control/Bulk-IN Transaction in Packet-Per-Buffer Without Descriptor Update Mode.

OUT Transaction

The following figures show the DMA processing an OUT token with and without descriptor update, respectively.

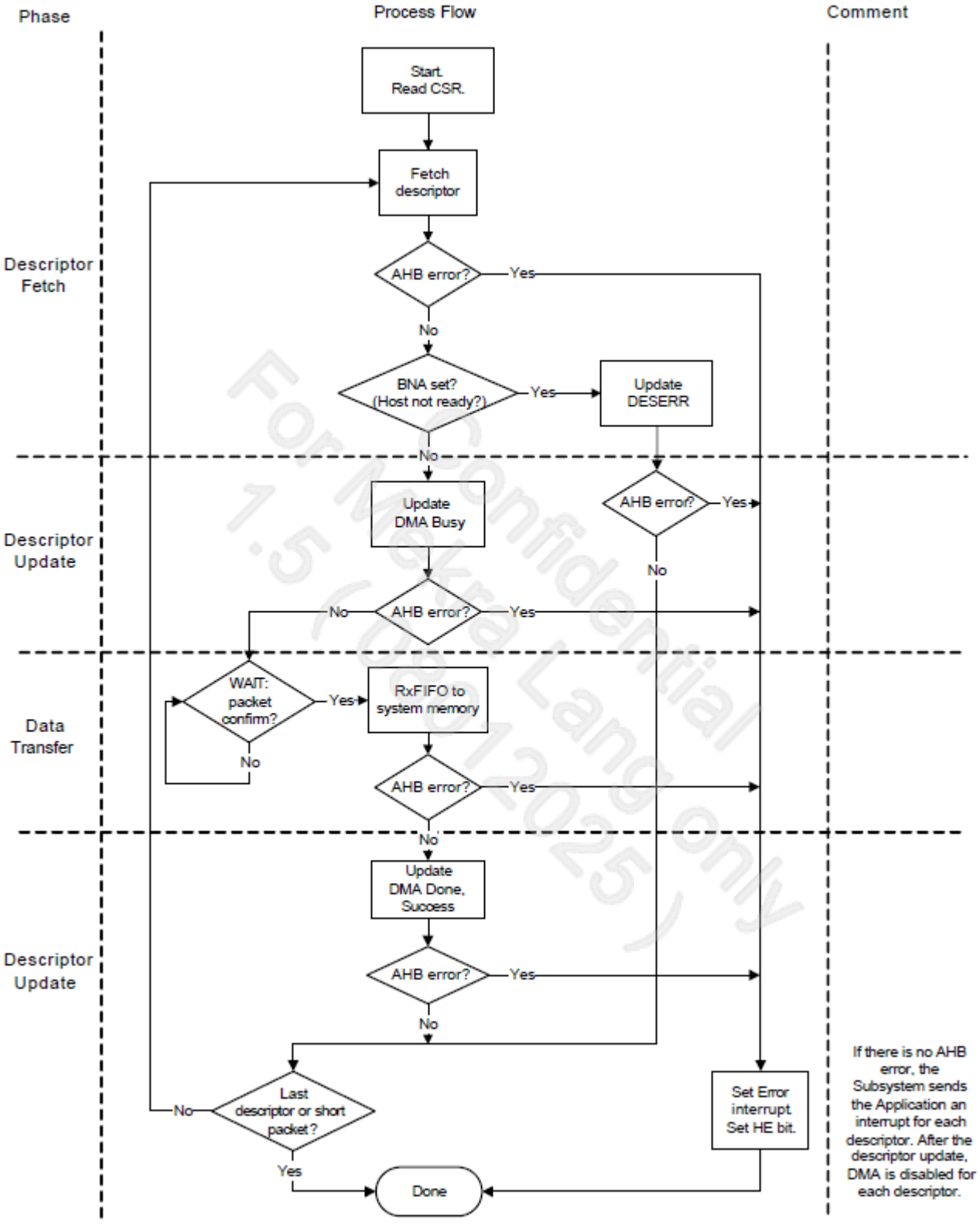


Figure 11-4. OUT Transaction in Packet-Per-Buffer Mode With Descriptor Update.

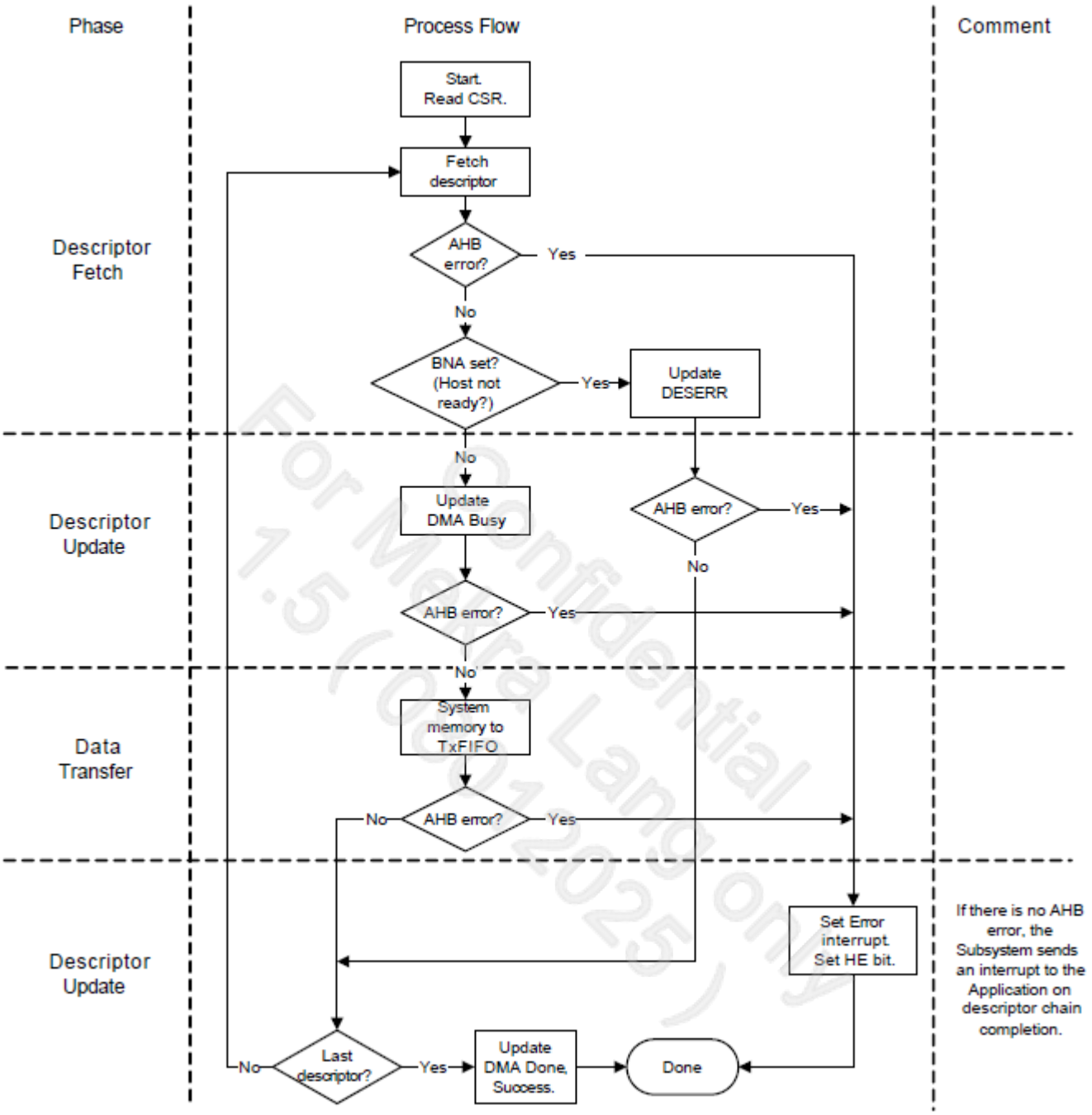


Figure 11-5. OUT Transaction in Packet-Per-Buffer Mode Without Descriptor Update.

11.8.6.2 Buffer Fill Mode (OUT Transfer)

In this mode of operation, the DMA transfers all packets to the large buffer whose address is indicated by the single out data memory structure descriptor. The DMA controller updates buffer status when a short packet is received, and simultaneously sends an interrupt to the application. Buffer fill mode is enabled by writing 1'b1 to the BF bit of the Device Control register.

This mode of DMA operation requires fewer memory accesses than Packet-Per-Buffer With Descriptor Update mode, increasing throughput. The following figure shows DMA operation in buffer fill mode for an OUT *transfer.*

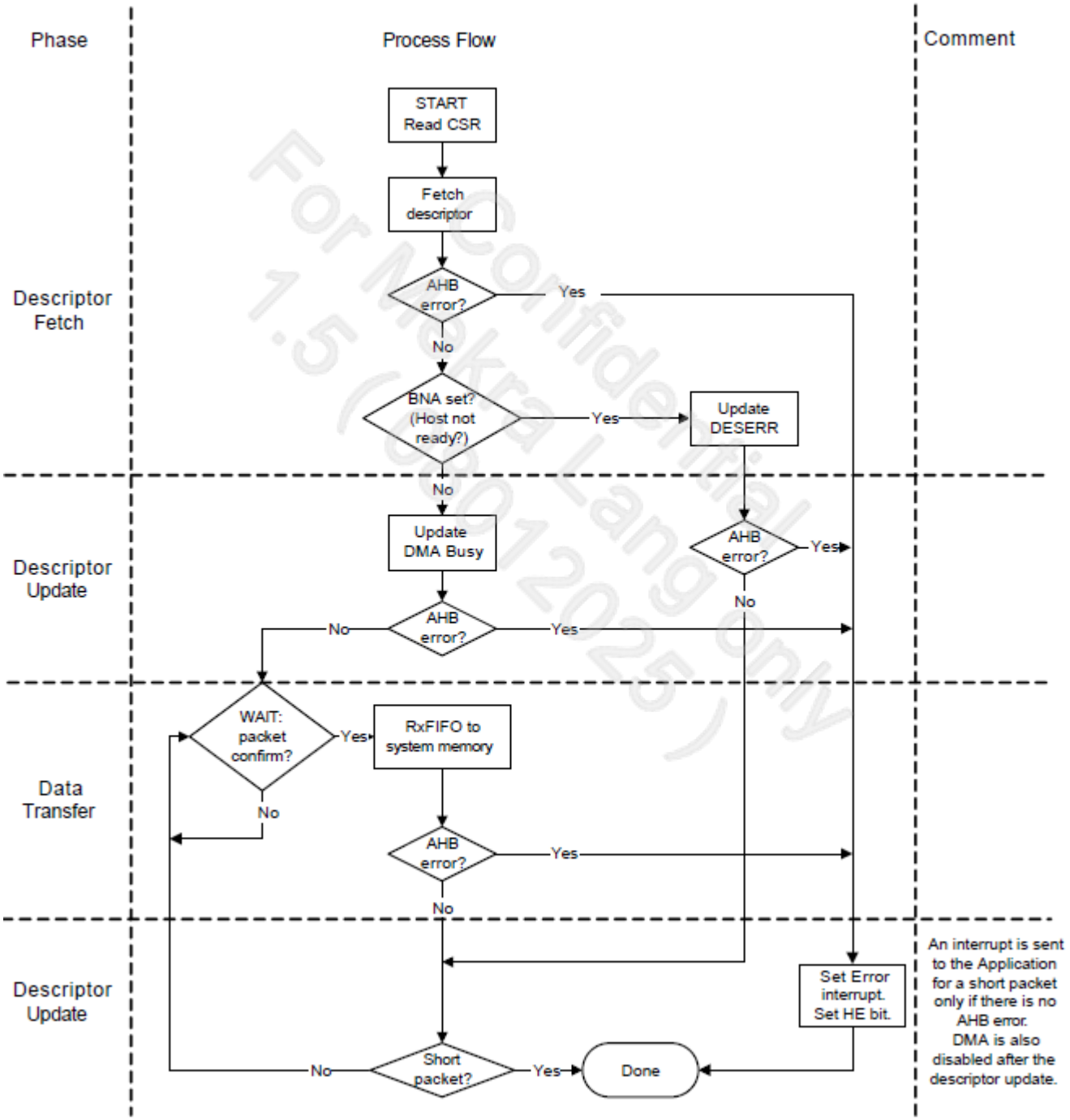


Table 11-25. DMA Processing OUT Packets in Buffer Fill Mode.

11.8.6.3 Buffer Fill Mode (IN Transfer)

This mode of operation can be entered by using packet-per-buffer mode with only one descriptor indicating the system memory starting address, the number of bytes to be transferred to the USB host (TX bytes can be greater than a packet), and the “L” bit set in the status quadlet. The DMA controller updates buffer status after all data has been transferred to the TXFIFO. The subsystem then sends the application an interrupt.

11.8.6.4 Threshold Enable

The threshold enable feature is used for transferring packets in the OUT direction and for DMA operation only. There is no threshold enable for the IN direction. In this context, “thresholding” means emptying the OUT RXFIFO as soon as it receives a certain number (the threshold value) of 32-bit words.

Thresholding is enabled by writing 1'b1 to the THE bit in the Device Control register. The 8-bit threshold value is programmed by writing the THLEN bits in the Device Control register. The threshold value is the number of 32-bit words (quadlets) that must be received by the RXFIFO before the DMA can start the transfer.

When thresholding is disabled, the DMA waits for the complete packet before starting the data transfer. When thresholding is enabled, the transfer of the packet to host memory starts before the validity of the packet is assessed. If, at the end of the transfer, the packet is found to be corrupt, the descriptor is not updated and the next clean packet overwrites the previous corrupt packet. This conceals the USB error from the application.

The following figure shows DMA operation with thresholding enabled.

For Sakra Lang only
1.5 (08012025)
Confidential

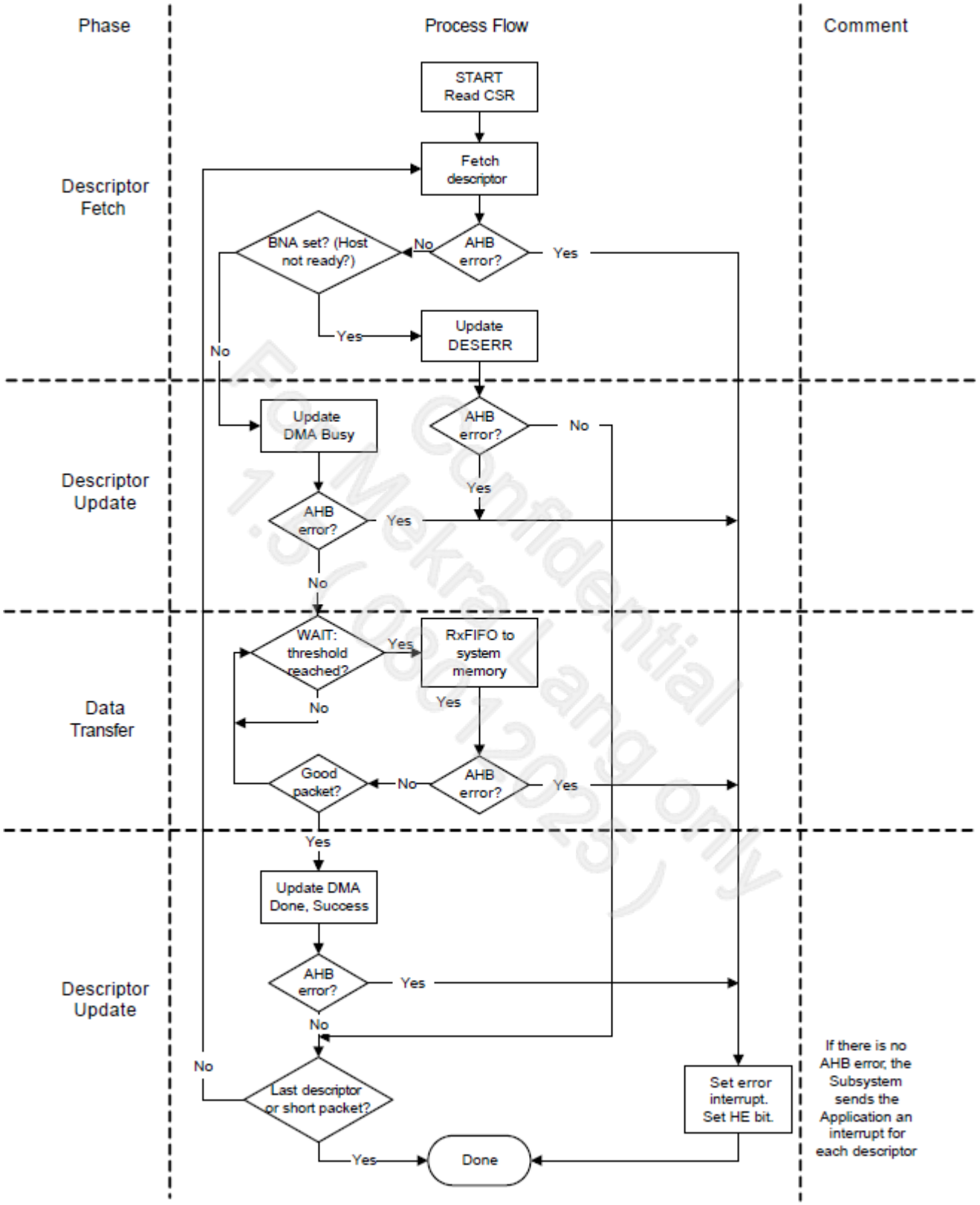


Figure 11-6. DMA Processing OUT Packets in Packet-Per-Buffer With Descriptor Update Mode With Threshold Enabled.

11.8.6.5 Burst Split Enable

When Burst Split is enabled, all AHB transfers (transfers from the DMA to system memory and from system memory to the TXFIFO) are split into bursts of a specified length. This mode can be enabled by writing 1'b1 to the BREN bit of the Device Control register.

Burst Length is programmed by writing the 8-bit value of the BRLEN to the Device Control register. This field indicates the number of 32-bit transfers that should occur in a single burst.

11.8.6.6 Byte Ordering (Endianness)

System byte ordering can be changed from little endian (default) to big endian and vice versa by writing 1'b1 to the BE bit in the Device Control register. This conversion is shown in the following figure. Only data accesses are endian-sensitive (in both Slave-Only and DMA modes). Descriptor and CSR accesses are always in little endian mode.

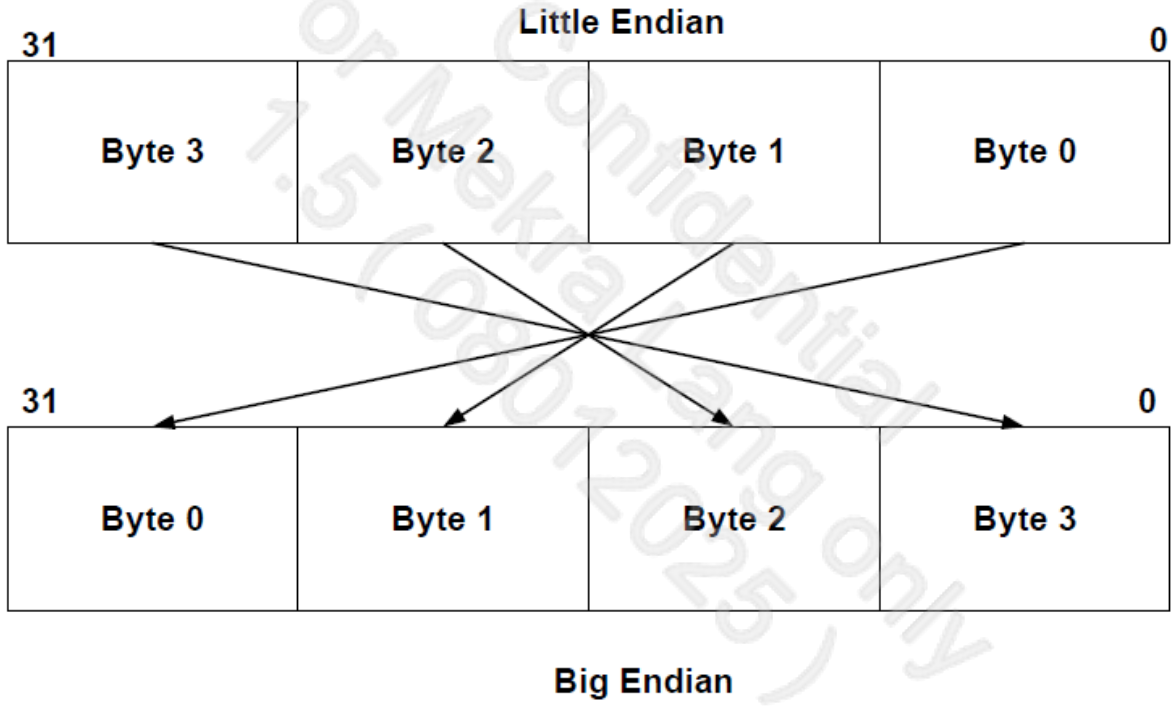


Table 11-26. Byte Ordering (Endianness) Conversion.

For Makrotang only
Confidential
1.5 (08012025)

11.8.6.7 Relationship Between Threshold Length and Burst Length

- Threshold length (THLEN) must be greater than (multiples of burst length) or equal to burst length (BRLEN).

  THLEN+1 = N*(BRLEN+1)

  Where N can be 1, 2, 3...
- Threshold length is used only to start the transfer. For example, if the threshold length is programmed as 7 and the total transfer size is 16 DWORDS, Receive DMA starts the transfer as soon as 8 DWORDS are received from the USB. The AHB master introduces busy states on the AHB bus in case there is a data underrun in the receive direction.
- BRLEN is used to make the AHB master transfer in chunks of BRLEN. If the burst length is 3, the AHB master transfers 4 DWORDS at a time.
- If the threshold length is 8'h07 and the burst length is 8'h03, DMA transfer starts as soon as 8 DWORDS are received from the USB. On the AHB side, data is transferred in chunks of 4 DWORDS.

11.8.7 High-Bandwidth Isochronous Transfers

This section describes OUT and IN packets for high-bandwidth isochronous transfers.

11.8.7.1 OUT Packet in DMA Mode

Each descriptor stores one maximum packet size of data. The data PID associated with the packet is available in the PID field, bits [15:14]. If the microframe contains three packets, data and the corresponding data PID are stored in three descriptors.

11.8.7.2 IN Packet in DMA Mode

The application creates data of one maximum packet size per descriptor. If the application must send three packets in the microframe, the application must create three descriptors. Data PID information must be provided in the PID field, bits [15:14].

Note:

At low AHB frequencies, with multiple IN endpoint descriptors active, the core transmit DMA engine may not get an opportunity to service some Isochronous IN endpoints on time. This causes the device core to respond with zero-length packets to host. This scenario is more likely with low AHB frequencies, and when the other noniso-chronous IN endpoints have large buffers allocated. This is because the TX DMA controller services endpoints in a round-robin fashion, and gets out of the endpoint only when the descriptor chain is empty, or the TXFIFO is full. In particular cases, the TXFIFO for non-isochronous endpoints never becomes full, due to due to 2x mps of the FIFO size, and slow AHB. The TX DMA retains that endpoint as long the host continues sending IN tokens for that endpoint.

Confidential For Mekra Lang Only 1.5 (20120305)

12. USB HOST CONTROLLER

The CV22FS USB host controller is covered here as follows:

- (Section 12.1) USB Host: Overview
- (Section 12.2) USB Host: Host / Device Select
- (Section 12.3) USB Host: Registers

12.1 USB Host: Overview

The USB host controller is an open host controller interface (OHCI) and enhanced host controller interface (EHCI) compliant USB 2.0/1.1 Host. The EHCI controller implements data and descriptor prefetching for higher performance. There is a 4-KB data buffer and a 272-B descriptor buffer.

The sections below document the OHCI, ECHI and implementation-specific registers of the USB host controller. For more information on OHCI, please reference *OpenHCI: Open Host Controller Interface Specification for USB*. For more information on EHCI, please reference *Enhanced Host Controller Interface Specification for Universal Serial Bus*. Both of these documents are available online.

Note that system software generally sets / configures the USB PHY clock sources using a set of APIs with RCT registers. The register programming defined in this chapter is used for adjustment and refinement.

12.2 USB Host: Host / Device Select

The CV22FS provides one USB high-speed interface that can be configured to work in master or slave mode. For information on this interface, please refer to the previous chapter.

12.3 USB Host: Registers

This section gives the registers of the USB Host Controller as follows:

- (Section 12.3.1) USB Host Registers: OpenHCI Map
- (Section 12.3.2) USB Host Registers: OpenHCI Details
- (Section 12.3.3) USB Host Registers: EnhancedHCI Map
- (Section 12.3.4) USB Host Registers: EnhancedHCI Details.

12.3.1 USB Host Registers: OpenHCI Map

Offset	Register	Description
0x00	**Hc_revision**	HcRevision Gives the HCI version implemented by this HC
0x04	**Hc_control**	HcControl Register Defines the operating modes for the Host Controller

Offset	Register	Description
0x08	**Hc_commandstatus**	HcCommandStatus Reflects the current status of the Host Controller and is used by the Host Controller to receive commands issued by the Host Controller Driver
0x0C	**Hc_interruptstatus**	HcInterruptStatus Provides status on various events that cause hardware interrupts
0x10	**Hc_interruptenable**	HcInterruptEnable Provides enable bits that control which events generate a hardware interrupt and correspond to an associated interrupt bit in **Hc_interruptstatus**
0x14	**Hc_interruptdisable**	HcInterruptDisable Provides disable bits that correspond to an associated interrupt bit in **Hc_interruptstatus.** Writing 1 to a bit in this register clears the corresponding bit in the **Hc_interruptenable**
0x18	**Hc_hcca**	HcHCCA Contains the physical address of the host controller communication area (HCCA)
0x1C	**Hc_periodcurrented**	HcPeriodCurrentED Contains the physical address of the current isochronous or Interrupt Endpoint Descriptor
0x20	**Hc_controlheaded**	HcControlHeadED Contains the physical address of the first Endpoint Descriptor of the Control list
0x24	**Hc_controlcurrented**	HcControlCurrentED Contains the physical address of the current Endpoint Descriptor of the Control list
0x28	**Hc_bulkheaded**	HcBulkHeadED Contains the physical address of the first Endpoint Descriptor of the Bulk list
0x2C	**Hc_bulkcurrented**	HcBulkCurrentED Contains the physical address of the current endpoint of the Bulk list
0x30	**Hc_donehead**	HcDoneHead Contains the physical address of the last completed Transfer Descriptor that was added to the Done queue
0x34	**Hc_fminterval**	HcFmInterval Allows to define frame time intervals to synchronize with an external clocking resource and to adjust an unknown local clock offset
0x38	**Hc_fmremaining**	HcFmRemaining This 14-bit down counter shows the bit time remaining in the current frame
0x3C	**Hc_fmnumber**	HcFmNumber This 16-bit counter provides a timing reference among events happening in the Host Controller and the Host Controller Driver

For Mekra Lang (080125) Only
Confidential
1.5

Offset	Register	Description
0x40	**Hc_periodicstart**	HcPeriodicStart Has a 14-bit programmable value which determines when is the earliest time HC should start processing the periodic list
0x44	**Hc_lsthreshold**	HcLSThreshold Contains an 11-bit value used by the Host Controller to determine whether to commit to the transfer of a maximum of 8-byte LS packet before EOF
0x48	**Hc_rhdescriptor_a**	HcRhDescriptorA The first of two registers that describe the Root Hub
0x4C	**Hc_rhdescriptor_b**	HcRhDescriptorB The second of two registers that describe the Root Hub
0x50	Hc_rhstatus	HcRhStatus Two-part register that represents the Hub Status field and the Hub Status Change field
0x54	**Hc_rhportstatus_1**	HcRhPortStatus 1 Controls and reports Port 1 events
0x58	**Hc_rhportstatus_2**	HcRhPortStatus 2 Controls and reports Port 2 events

Table 12-1. OpenHCI Registers.

12.3.2 USB Host Registers: OpenHCI Details

12.3.2.1 Hc_revision Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**rev**	R	0x10	Revision Contains the BCD representation of the version of the HCI specification that is implemented by this HC. For example, a value of 0x11 corresponds to version 1.1. All of the HC implementations that are compliant with this specification will have a value of 0x10

Table 12-2. OpenHCI Revision register.

For Mekra Lang only
1.5 (08012021)
Confidential

12.3.2.2 Hc_control Register

Bits	Name	Access	Reset	Description
31:11				Reserved
10	**rwe**	RW	0	RemoteWakeupEnable This bit is used by the HCD to enable or disable the remote wakeup feature upon the detection of upstream resume signaling. When this bit is set and the ResumeDetected bit (**rd**) in **Hc_interruptstatus** is set, a remote wakeup is signaled to the host system. Setting this bit has no impact on the generation of hardware interrupt.
9	**rwc**	RW	0	RemoteWakeupConnected This bit indicates whether the HC supports remote wakeup signaling. If remote wakeup is supported and used by the system it is the responsibility of system firmware to set this bit during POST. The HC clears the bit upon a hardware reset but does not alter it upon a software reset.
8	**ir**	RW	0	InterruptRouting This bit determines the routing of interrupts generated by events registered in **Hc_interruptstatus**. If clear, all interrupts are routed to the normal host bus interrupt mechanism. If set, interrupts are routed to the System Management Interrupt. The HCD clears this bit upon a hardware reset, but it does not alter this bit upon a software reset. The HCD uses this bit as a tag to indicate the ownership of the HC.
7:6	**hcfs**	RW	0	HostControllerFunctionalState for USB 0x0 - USBRESET 0x1 - USBRESUME 0x2 - USBOPERATIONAL 0x3 - USBSUSPEND A transition to USBOPERATIONAL from another state causes SOF generation to begin 1 ms later. The HCD may determine whether the HC has begun sending SOFs by reading the StartofFrame field (**sf**) of **Hc_interruptstatus**. This field may be changed by the HC only when in the USBSUSPEND state. The HC may move from the USBSUSPEND state to the USBRESUME state after detecting the resume signaling from a downstream port. The HC enters USBSUSPEND after a software reset, whereas it enters USBRESET after a hardware reset. The latter also resets the Root Hub and asserts subsequent reset signaling to downstream ports.
5	**ble**	RW	0	BulkListEnable This bit is set to enable the processing of the Bulk list in the next frame. If cleared by the HCD, processing of the Bulk list does not occur after the next SOF. The HC checks this bit when it determines to process the list. When disabled, the HCD may modify the list. If **Hc_bulkcurrented** is pointing to an ED to be removed, the HCD must advance the pointer by updating **Hc_bulkcurrented** before re-enabling processing of the list.

For Mass Production
1.5 Confidential Only

Bits	Name	Access	Reset	Description
4	**cle**	RW	0	ControlListEnable This bit is set to enable the processing of the Control list in the next frame. If cleared by the HCD, processing of the Control list does not occur after the next SOF. The HC must check this bit when it determines to process the list. When disabled, the HCD may modify the list. If **Hc_controlcurrented** is pointing to an ED to be removed, the HCD must advance the pointer by updating **Hc_controlcurrented** before re-enabling processing of the list.
3	**ie**	RW	0	IsochronousEnable This bit is used by the HCD to enable/disable processing of isochronous EDs. While processing the periodic list in a frame, the HC checks the status of this bit when it finds an Isochronous ED (F = 1). If set (enabled). The HC continues processing the EDs. If cleared (disabled), the HC halts processing of the periodic list (which now contains only isochronous EDs) and begins processing the Bulk/Control lists. Setting this bit is guaranteed to take effect in the next frame (not the current frame).
2	**ple**	RW	0	PeriodicListEnable This bit is set to enable the processing of the periodic list in the next frame. If cleared by the HCD, processing of the periodic list does not occur after the next SOF. The HC must check this bit before it starts processing the list.
1:0	**cbsr**	RW	0	ControlBulkServiceRatio This specifies the service ratio between Control and Bulk EDs. Before processing nonperiodic lists, the HC must compare the ratio specified with its internal count on how many nonempty Control EDs have been processed, in determining whether to continue serving another Control ED or switching to Bulk EDs. The internal count will be retained when crossing the frame boundary. In case of reset, the HCD is responsible for restoring this value.

Table 12-3. OpenHCI Control Register.

12.3.2.3 Hc_commandstatus Register

Bits	Name	Access	Reset	Description
31:18				Reserved
17:16	**soc**	RW	0	SchedulingOverrunCount These bits are incremented on each scheduling overrun error. It is initialized to 0x0 and wraps around at 0x3. This will be incremented when a scheduling overrun is detected even if SchedulingOverrun bit (**so**) in **Hc_interruptstatus** has already been set. This is used by the HCD to monitor persistent scheduling problems.
15:4				Reserved
3	**ocr**	RW	0	OwnershipChangeRequest This bit is set by an OS HCD to request a change of control of the HC. When set the HC will set the OwnershipChange field (**oc**) in **Hc_interruptstatus**. After the changeover, this bit is cleared and remains so until the next request from the OS HCD.

For Marketing only (c) 2025

Bits	Name	Access	Reset	Description
2	**blf**	RW	0	BulkListFilled This bit is used to indicate whether there are TDs on the Bulk list. It is set by the HCD when it adds a TD to an ED in the Bulk list. When the HC begins to process the head of the Bulk list, it checks BF. As long as BulkListFilled **blf** is 0, the HC will not start processing the Bulk list. If **blf** is 1, HC will start processing the Bulk list and will set BF to 0. If the HC finds a TD on the list, then the HC will set BulkListFilled to 1 causing the Bulk list processing to continue. If no TD is found on the Bulk list, and if the HCD does not set BulkListFilled, then **blf** will still be 0 when the HC completes processing the Bulk list and Bulk list processing will stop.
1	**clf**	RW	0	ControlListFilled This bit is used to indicate whether there are TDs on the Control list. It is set by the HCD when it adds a TD to an ED in the Control list. When the HC begins to process the head of the Control list, it checks **clf**. As long as ControlListFilled is 0, the HC will not start processing the Control list. If **cf** of **EHCI_configflag** is 1, HC will start processing the Control list and will set ControlListFilled to 0. If the HC finds a TD on the list, then the HC will set ControlListFilled to 1 causing the Control list processing to continue. If no TD is found on the Control list, and if the HCD does not set ControlListFilled, then ControlListFilled will still be 0 when the HC completes processing the Control list and Control list processing will stop.
0	**hcr**	RW	0	HostControllerReset This bit is set by the HCD to initiate a software reset of the HC. Regardless of the functional state of the HC, it moves to the USBSUSPEND state in which most of the operational registers are reset except those stated otherwise; e.g., the InterruptRouting field **ir** of **Hc_control**, and no Host bus accesses are allowed. This bit is cleared by the HC upon the completion of the reset operation. The reset operation must be completed within 10 ms. This bit, when set, should not cause a reset to the Root Hub and no subsequent reset signaling should be asserted to its downstream ports.

Table 12-4. OpenHCI Command and Status Register.

12.3.2.4 Hc_interruptstatus Register

Bits	Name	Access	Reset	Description
31				Reserved
30	**oc**	RW	0	OwnershipChange This bit is set by the HC when the HCD sets the OwnershipChangeRequest field (**ocr**) in **Hc_commandstatus**. This event, when unmasked, will always generate an System Management Interrupt (SMI) immediately. This bit is tied to 0 when the SMI is not implemented.
29:7				Reserved
6	**rhsc**	RW	0	RootHubStatusChange This bit is set when the content of **Hc_rhstatus** or the content of **Hc_rhportstatus_1/2** has changed.

Bits	Name	Access	Reset	Description
5	**fno**	RW	0	FrameNumberOverflow This bit is set when the MSB of **Hc_fmnumber** (bit 15) changes value, from 0 to 1 or from 1 to 0, and after HccaFrameNumber has been updated.
4	**ue**	RW	0	UnrecoverableError This bit is set when the HC detects a system error not related to USB. The HC should not proceed with processing or signaling before the system error has been corrected. The HCD clears this bit after the HC has been reset.
3	**rd**	RW	0	ResumeDetected This bit is set when the HC detects that a device on the USB is asserting resume signaling. It is the transition from no resume signaling to resume signaling causing this bit to be set. This bit is not set when the HCD sets the USBRESUME state.
2	**sf**	RW	0	StartofFrame This bit is set by the HC at each start of a frame and after the update of HccaFrameNumber. The HC also generates a SOF token at the same time.
1	**wdh**	RW	0	WritebackDoneHead This bit is set immediately after the HC has written **Hc_donehead** to HccaDoneHead. Further updates of HccaDoneHead will not occur until this bit has been cleared. The HCD should only clear this bit after it has saved the content of HccaDoneHead.
0	**so**	RW	0	SchedulingOverrun This bit is set when the USB schedule for the current frame overruns and after the update of HccaFrameNumber. A scheduling overrun will also cause the SchedulingOverrunCount bit (**soc**) of **Hc_commandstatus** to be incremented.

Table 12-5. OpenHCI Interrupt Status Register.

12.3.2.5 Hc_interruptenable Register

Writing 1 to a bit in this register sets the corresponding bit, whereas writing 0 to a bit in this register leaves the corresponding bit unchanged. On read, the current value of this register is returned.

Bits	Name	Access	Reset	Description
31	**mie**	RW	0	0 - Ignore 1 - Enables interrupt generation due to events specified in the other bits of this register (used by the HCD as a Master Interrupt Enable)
30	**oc**	RW	0	0 - Ignore 1 - Enable interrupt generation due to ownership change
29:7				Reserved
6	**rhsc**	RW	0	0 - Ignore 1 - Enable interrupt generation due to root hub status change
5	**fno**	RW	0	0 - Ignore 1 - Enable interrupt generation due to frame number overflow
4	**ue**	RW	0	0 - Ignore 1 - Enable interrupt generation due to unrecoverable error

For Internal Use Only 1.5 Confidential 01/2025

Bits	Name	Access	Reset	Description
3	**rd**	RW	0	0 - Ignore 1 - Enable interrupt generation due to resume detect
2	**sf**	RW	0	0 - Ignore 1 - Enable interrupt generation due to start of frame
1	**wdh**	RW	0	0 - Ignore 1 - Enable interrupt generation due to **Hc_donehead** writeback
0	**so**	RW	0	0 - Ignore 1 - Enable interrupt generation due to scheduling overrun

Table 12-6. OpenHCI Interrupt Enable Register.

12.3.2.6 Hc_interruptdisable Register

Each disable bit in the **Hc_interruptdisable** register corresponds to an associated interrupt bit in the **Hc_interruptstatus** register. The **Hc_interruptdisable** register is coupled with the **Hc_interruptenable** register. Thus, writing 1 to a bit in this register clears the corresponding bit in **Hc_interruptenable**, whereas writing 0 to a bit in this register leaves the corresponding bit in **Hc_interruptenable** unchanged. On read, the current value of the **Hc_interruptenable** register is returned.

Bits	Name	Access	Reset	Description
31	**mie**	RW	1	0 - Ignore 1 - Disables interrupt generation due to events specified in the other bits of this register
30	**oc**	RW	0	0 - Ignore 1 - Disable interrupt generation due to ownership change
29:7				Reserved
6	**rhsc**	RW	0	0 - Ignore 1 - Disable interrupt generation due to root hub status change
5	**fno**	RW	0	0 - Ignore 1 - Disable interrupt generation due to frame number overflow
4	**ue**	RW	0	0 - Ignore 1 - Disable interrupt generation due to unrecoverable error
3	**rd**	RW	0	0 - Ignore 1 - Disable interrupt generation due to resume detect
2	**sf**	RW	0	0 - Ignore 1 - Disable interrupt generation due to start of frame
1	**wdh**	RW	0	0 - Ignore 1 - Disable interrupt generation due to **Hc_donehead** Writeback
0	**so**	RW	0	0 - Ignore 1 - Disable interrupt generation due to scheduling overrun

Table 12-7. OpenHCI Interrupt Disable Register.

For Ambarella Confidential Use Only

12.3.2.7 Hc_hcca Register

Bits	Name	Access	Reset	Description
31:8	**hcca**	RW	0	Base address of the Host Controller Communication Area
7:0				Reserved

Table 12-8. OpenHCI Host Controller Communication Area Register.

12.3.2.8 Hc_periodcurrented Register

This register contains the physical address of the current Isochronous or Interrupt Endpoint Descriptor.

Bits	Name	Access	Reset	Description
31:4	**pced**	RW	0	PeriodCurrentED This is used by the HC to point to the head of one of the Periodic lists which will be processed in the current frame. The content of this register is updated by the HC after a periodic ED has been processed. The HCD may read the content in determining which ED is currently being processed at the time of reading.
3:0				Reserved

Table 12-9. OpenHCI Periodic Current ED Register.

12.3.2.9 Hc_controlheaded Register

This register contains the physical address of the current Isochronous or Interrupt Endpoint Descriptor

Bits	Name	Access	Reset	Description
31:4	**ched**	RW	0	ControlHeadED The HC traverses the Control list starting with the **Hc_controlheaded** pointer. The content is loaded from **Hc_hcca** during the initialization of the HC.
3:0				Reserved

Table 12-10. OpenHCI Control Head ED Register.

12.3.2.10 Hc_controlcurrented Register

This register contains the physical address of the current Endpoint Descriptor of the Control list.

For Mobarra Lang Only 1.5 (08/12/2015)

Bits	Name	Access	Reset	Description
31:4	**cced**	RW	0	ControlCurrentED This pointer is advanced to the next ED after serving the present one. The HC will continue processing the list from where it left off in the last frame. When it reaches the end of the Control list, HC checks the ControlListFilled bit (**clf**) in **Hc_commandstatus**. If set, it copies the content of **Hc_controlheaded** to **Hc_controlcurrented** and clears the bit. If not set, it does nothing. The HCD may modify this register only when the ControlListEnable or **cle** bit of **Hc_control** is cleared. When set, the HCD only reads the instantaneous value of this register. Initially, this is set to 0 to indicate the end of the Control list.
3:0				Reserved

Table 12-11. OpenHCI Control Current ED Register.

12.3.2.11 Hc_bulkheaded Register

This register contains the physical address of the first Endpoint Descriptor of the Bulk list.

Bits	Name	Access	Reset	Description
31:4	**bhed**	RW	0	BulkHeadED The HC traverses the Bulk list starting with the **Hc_bulkheaded** pointer. The content is loaded from **Hc_hcca** during the initialization of the HC.
3:0				Reserved

Table 12-12. OpenHCI Bulk Head ED Register.

12.3.2.12 Hc_bulkcurrented Register

This register is for the physical address of the current endpoint of the Bulk list.

Bits	Name	Access	Reset	Description
31:4	**bced**	RW	0	BulkCurrentED This is advanced to the next ED after the HC has served the present one. The HC continues processing the list from where it left off in the last frame. When it reaches the end of the Bulk list, the HC checks the ControlListFilled bit (**clf**) of **Hc_control**. If set, it copies the content of **Hc_bulkheaded** to **Hc_bulkcurrented** and clears the bit. If it is not set, it does nothing. The HCD is only allowed to modify this register when the BulkListEnable bit (**ble**) of **Hc_control** is cleared. When set, the HCD only reads the instantaneous value of this register. This is initially set to 0 to indicate the end of the Bulk list.
3:0				Reserved

Table 12-13. OpenHCI Bulk Current ED Register.

For Mass Lang only (2025)

12.3.2.13 Hc_donehead Register

This register contains the physical address of the last completed Transfer Descriptor that was added to the Done queue.

Bits	Name	Access	Reset	Description
31:4	**dh**	RW	0	DoneHead When a TD is completed, the HC writes the content of **Hc_donehead** to the NextTD field of the TD. The HC then overwrites the content of **Hc_donehead** with the address of this TD. This is set to 0 when the HC writes the content of this register to **Hc_hcca**. It also sets the WritebackDoneHead bit (**wdh**) of **Hc_interruptstatus**.
3:0				Reserved

Table 12-14. OpenHCI Done Head Register.

12.3.2.14 Hc_fminterval Register

This register allows to define frame time intervals to synchronize with an external clocking resource and to adjust unknown local clock offset.

Bits	Name	Access	Reset	Description
31	**fit**	RW	0	FrameIntervalToggle The HCD toggles this bit when it loads a new value to the FrameInterval bit (**fi**).
30:16	**fsmps**	RW	0	FSLargestDataPacket This field specifies a value which is loaded into the Largest Data Packet Counter at the beginning of each frame. The counter value represents the largest amount of data in bits which can be sent or received by the HC in a single transaction at any given time without causing scheduling overrun. The field value is calculated by the HCD.
15:14				Reserved
13:0	**fi**	RW	0x2EDF	FrameInterval This specifies the interval between two consecutive SOFs in bit times. The nominal value is set to be 11,999. The HCD should store the current value of this field before resetting the HC. By setting the HostControllerReset field (**hcr**) of **Hc_commandstatus** as this will cause the HC to reset this field to its nominal value. The HCD may choose to restore the stored value upon the completion of the Reset sequence.

Table 12-15. OpenHCI Frame Interval Register.

Confidential
For Mel Only
1.5 on 02/25/2015

12.3.2.15 Hc_fmremaining Register

This register uses a 14-bit down counter to show the bit time remaining in the current frame.

Bits	Name	Access	Reset	Description
31	**frt**	RW	0	FrameRemainingToggle This bit is loaded from the FrameIntervalToggle field (**fit**) of **Hc_fminterval** when FrameRemaining reaches 0. This bit is used by the HCD for the synchronization between FrameInterval (**fi**) bit in **Hc_fminterval**) and FrameRemaining (**fr**).
30:14				Reserved
13:0	**fr**	RW	0	FrameRemaining This counter is decremented at each bit time. When it reaches 0, it is reset by loading the FrameInterval (**fi**) value specified in **Hc_fminterval** at the next bit time boundary. When entering the USBOPERATIONAL state, the HC re-loads the content with the FrameInterval (**fi**) value and uses the updated value from the next SOF.

Table 12-16. OpenHCI Frame Remaining Register.

12.3.2.16 c_fmnumber Register

This register uses a 16-bit counter as a timing reference among events happening in the Host Controller and the Host Controller Driver.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**fn**	RW	0	FrameNumber This is incremented when **Hc_fmremaining** is re-loaded. It will be rolled over to 0x0 after 0xFFFF. When entering the USB OPERATIONAL state, this will be incremented automatically. The content will be written to **Hc_hcca** after the HC has incremented the FrameNumber **fn** at each frame boundary and sent a SOF but before the HC reads the first ED in that frame. After writing to **Hc_hcca**, HC will set the StartofFrame bit (**sf**) in **Hc_interruptstatus**.

Table 12-17. OpenHCI Frame Number Register.

Confidential For Mekra Lang 1 Feb 2019 Only

12.3.2.17 Hc_periodicstart Register

This register determines the earliest time the HC should start processing the periodic list.

Bits	Name	Access	Reset	Description
31:14				Reserved
13:0	**ps**	RW	0	PeriodicStart After a hardware reset, this field is cleared. This is then set by the HCD during the HC initialization. The value is calculated roughly as 10% off from **Hc_fminterval**. A typical value will be 0x3E67. When **Hc_fmremaining** reaches the value specified, processing of the periodic lists will have priority over Control/Bulk processing. The HC will therefore start processing the Interrupt list after completing the current Control or Bulk transaction that is in progress.

Table 12-18. OpenHCI Periodic Start Register.

12.3.2.18 Hc_lsthreshold Register

This register contains an 11-bit value used by the Host Controller to determine whether to commit to transfer of a maximum 8-byte LS packet before EOF.

Bits	Name	Access	Reset	Description
31:12				Reserved
11:0	**lst**	RW	0x0628	LSThreshold This field contains a value which is compared to the FrameRemaining field (**fr**) prior to initiating a Low Speed transaction. The transaction is started only if FrameRemaining (**fr**) ≥ this field. The value is calculated by the HCD with the consideration of transmission and setup overhead.

Table 12-19. OpenHCI Low Speed Threshold Register.

12.3.2.19 Hc_rhdescriptor_a Register

Bits	Name	Access	Reset	Description
31:24	**potpgt**	RW	2	PowerOnToPowerGoodTime This byte specifies the duration that the HCD waits before accessing a powered-on port of the Root Hub. The unit of time is 2 ms. The duration is calculated as **potpgt** * 2 ms.
23:13				Reserved
12	**nocp**	RW	0	NoOverCurrentProtection This bit describes how the over current status for the Root Hub ports are reported. When this bit is cleared, the OverCurrentProtectionMode field (**ocpm**) specifies global or per-port reporting. 0 - Over current status is reported collectively for all downstream ports 1 - No over current protection supported

Confidential Only For Meta Lang (2025)

Bits	Name	Access	Reset	Description
11	**ocpm**	RW	1	OverCurrentProtectionMode This bit describes how the overcurrent status for the Root Hub ports are reported. This field is valid only if the NoOverCurrentProtection field (**nocp**) is cleared. 0 - Over-current status is reported collectively for all downstream ports 1 - Over-current status is reported on a per-port basis
10	**dt**	R	0	DeviceType This bit specifies that the Root Hub is not a compound device.
9	**nps**	RW	0	NoPowerSwitching These bits are used to specify whether power switching is supported or ports are always powered. When this bit is cleared, the Power-SwitchingMode (**psm**) specifies global or per-port switching. 0 - Ports are power switched 1 - Ports are always powered on when the HC is powered on
8	**psm**	RW	1	PowerSwitchingMode This bit is used to specify how the power switching of the Root Hub ports is controlled. This field is only valid if the NoPowerSwitching field (**nps**) is cleared. 0 - All ports are powered at the same time 1 - Each port is powered individually. This mode allows port power to be controlled by either the global switch or per port switching. If the PortPowerControlMask (**ppcm**) is set, the port responds only to port power commands (Set/ClearPortPower). If the port mask is cleared, then the port is controlled only by the global power switch (Set/Clear-GlobalPower).
7:0	**ndp**	R	2	NumberDownstreamPorts These bits specify the number of downstream ports supported by the Root Hub.

Table 12-20. OpenHCI Root Hub Descriptor A Register.

12.3.2.20 Hc_rhdescriptor_b Register

Bits	Name	Access	Reset	Description
31:16	**ppcm**	RW	6	PortPowerControlMask Each bit indicates if a port is affected by a global power control command when PowerSwitchingMode (**psm**) is set. When set, the port's power state is only affected by per-port power control (Set/ClearPort-Power). When cleared, the port is controlled by the global power switch (Set/ClearGlobalPower). If the device is configured to global switching mode (**psm** = 0), this field is not valid. Bit [0] - Reserved Bit [1] - Ganged-power mask on Port 1 Bit [2:15] - Reserved

For Ambarella Internal Use Only (Confidential 1.5 (Confidential) 2025)

Bits	Name	Access	Reset	Description
15:0	**dr**	RW	0	DeviceRemovable Each bit is dedicated to a port of the Root Hub. When cleared, the attached device is removable. When set, the attached device is not removable. Bit [0] - Reserved Bit [1] - Device attached to Port 1 Bit [2:15] - Reserved

Table 12-21. OpenHCI Root Hub Descriptor B Register.

12.3.2.21 Hc_rhstatus Register

This is a two-part register that represents the Hub Status field and the Hub Status Change field

Bits	Name	Access	Reset	Description
31	**crwe**	W	0	ClearRemoteWakeupEnable 1 - Clears DeviceRemoteWakeupEnable (**drwe**) 0 - No effect
30:18				Reserved
17	**ocic**	RW	0	OverCurrentIndicatorChange This bit is set by hardware when a change has occurred to the OCI field of this register. The HCD clears this bit by writing 1. Writing 0 has no effect.
16	**lpsc**	RW	0	(Read) LocalPowerStatusChange The Root Hub does not support the local power status feature; thus, this bit is always read as 0. (Write) SetGlobalPower In global power mode (**psm** = 0). This bit is written to 1 to turn on power to all ports; i.e., clears the PortPowerStatus bit (**pps**). In per-port power mode, it sets **pps** only on ports whose PortPowerControl-Mask bit (**ppcm**) is not set. Writing 0 has no effect.
15	**drwe**	RW	0	(Read) DeviceRemoteWakeupEnable This bit enables the ConnectStatusChange (see the **csc bit** of **Hc_rh-portstatus_1/2**) as a resume event, causing a USBSUSPEND to US-BRESUME state transition and setting the ResumeDetected interrupt bit (**rd**). 0 - ConnectStatusChange is not a remote wakeup event 1 - ConnectStatusChange is a remote wakeup event (Write) SetRemoteWakeupEnable 0 - No effect 1 - Sets **drwe**
14:2				Reserved
1	**oci**	R	0	OverCurrentIndicator This bit reports overcurrent conditions when the global reporting is implemented. When set, an overcurrent condition exists. When cleared, all power operations are normal. If per-port overcurrent protection is implemented this bit is always 0

Bits	Name	Access	Reset	Description
0	**lps**	RW	0	(Read) LocalPowerStatus The Root Hub does not support the local power status feature; thus, this bit is always read as 0. (Write) ClearGlobalPower In global power mode (PowerSwitchingMode **psm** = 0), 1 to turn off power to all ports; i.e., clears the PortPowerStatus bit (**pps**). In per-port power mode, it clears **pps** only on ports whose PortPowerControlMask bit (**ppcm**) is not set. Writing 0 has no effect.

Table 12-22. OpenHCI Root Hub Status Register.

12.3.2.22 Hc_rhportstatus_1/2 Registers

The register set details for each of the two ports are as follows:

Bits	Name	Access	Reset	Description
31:21				Reserved
20	**prsc**	RW	0	PortResetStatusChange This bit is set at the end of the 10-ms port reset signal. The HCD writes 1 to clear this bit. Writing 0 has no effect. 0 - Port reset is not complete 1 - Port reset is complete
19	**ocic**	RW	0	PortOverCurrentIndicatorChange This bit is valid only if overcurrent conditions are reported on a per-port basis. This bit is set when Root Hub changes the PortOverCurrentIndicator bit (**poci**). The HCD writes 1 to clear this bit. Writing 0 has no effect. 0 - No change in **poci** 1 - The **poci** bit has changed
18	**pssc**	RW	0	PortSuspendStatusChange This bit is set when the full resume sequence has been completed. This sequence includes the 20-s resume pulse, LS EOP, and 3-ms resychronization delay. The HCD writes 1 to clear this bit. Writing 0 has no effect. This bit is also cleared when ResetStatusChange is set. 0 - Resume is not completed 1 - Resume completed
17	**pesc**	RW	0	PortEnableStatusChange This bit is set when hardware events cause the PortEnableStatus bit (**pes**) to be cleared. Changes from the HCD writes do not set this bit. The HCD writes 1 to clear this bit. Writing 0 has no effect. 0 - No change in **pes** 1 - Change in **pes**

For Confidential 1.3 Coding only

Bits	Name	Access	Reset	Description
16	**csc**	RW	0	ConnectStatusChange This bit is set when a connect or disconnect event occurs. The HCD writes 1 to clear this bit. Writing 0 has no effect. If CurrentConnectStatus bit (**ccs**) is cleared when a SetPortReset, SetPortEnable, or SetPortSuspend write occurs, this bit is set to force the driver to re-evaluate the connection status since these writes should not occur if the port is disconnected. 0 - No change in **ccs** 1 - Change in **ccs** Note: If the DeviceRemovable[NDP] bit (**dr**) is set, this bit is set only after a Root Hub reset to inform the system that the device is attached.
15:10		RW	0	Reserved
9	**lsda**	RW	0	(Read) LowSpeedDeviceAttached This bit indicates the speed of the device attached to this port. When set, a Low Speed device is attached to this port. When clear, a Full Speed device is attached to this port. This field is valid only when the CurrentConnectStatus bit (**ccs**) is set. 0 - Full speed device attached 1 - Low speed device attached (Write) ClearPortPower The HCD clears the PortPowerStatus bit (**pps**) by writing 1 to this bit. Writing 0 has no effect.
8	**pps**	RW		(Read) PortPowerStatus This bit reflects the port's power status, regardless of the type of power switching implemented. This bit is cleared if an overcurrent condition is detected. The HCD sets this bit by writing SetPortPower or SetGlobalPower. The HCD clears this bit by writing ClearPortPower or ClearGlobalPower. Which power control switches are enabled is determined by PowerSwitchingMode bit (**psm**) and PortPowerControlMask[NDP] bit (**ppcm**). In global switching mode (**psm** = 0), only Set/ClearGlobalPower controls this bit. In per-port power switching (**psm** = 1), if **ppcm** for the port is set, only Set/ClearPortPower commands are enabled. If the mask is not set, only Set/ClearGlobalPower commands are enabled. When port power is disabled, CurrentConnectStatus (**ccs**), PortEnableStatus (**pes**), PortSuspendStatus (**pss**), and PortResetStatus (**prs**) should be reset. 0 - Port power is off 1 - Port power is on (Write) SetPortPower The HCD writes 1 to set the PortPowerStatus bit (**pps**). Writing 0 has no effect.
7:5				Reserved

For Ambarella Confidential 1.5 (Ambarella Inc.)

Bits	Name	Access	Reset	Description
4	**prs**	RW	0	(Read) PortResetStatus When this bit is set by a write to SetPortReset, port reset signaling is asserted. When reset is completed, this bit is cleared when PortResetStatusChange (**prsc**) is set. This bit cannot be set if CurrentConnectStatus (**ccs**) is cleared. 0 - Port reset signal is not active 1 - Port reset signal is active (Write) SetPortReset The HCD sets the port reset signaling by writing 1 to this bit. Writing 0 has no effect. If CurrentConnectStatus (**ccs**) is cleared, this write does not set PortResetStatus (**prs**), but instead sets ConnectStatusChange (**csc**). This informs the driver that it attempted to reset a disconnected port.
3	**poci**	RW	0	(Read) PortOverCurrentIndicator This bit is only valid when the Root Hub is configured in such a way that over current conditions are reported on a per-port basis. If per-port over current reporting is not supported, this bit is set to 0. If cleared, all power operations are normal for this port. If set, an over current condition exists on this port. This bit always reflects the over current input signal 0 - No over current condition. 1 - Over current condition detected. (Write) ClearSuspendStatus The HCD writes 1 to initiate a resume. Writing 0 has no effect. A resume is initiated only if PortSuspendStatus (**pss**) is set.
2	**pss**	RW	0	(Read) PortSuspendStatus This bit indicates the port is suspended or in the resume sequence. It is set by a SetSuspendState write and cleared when PortSuspendStatusChange (**pssc**) is set at the end of the resume interval. This bit cannot be set if CurrentConnectStatus (**ccs**) is cleared. This bit is also cleared when PortResetStatusChange (**prsc**) is set at the end of the port reset or when the HC is placed in the USBRESUME state. If an upstream resume is in progress, it should propagate to the HC. 0 - Port is not suspended 1 - Port is suspended (Write) SetPortSuspend The HCD sets the PortSuspendStatus bit (**pss**) by writing 1 to this bit. Writing 0 has no effect. If CurrentConnectStatus (**ccs**) is cleared, this write does not set PortSuspendStatus (**pss**); instead it sets the ConnectStatusChange bit (**csc**). This informs the driver that it attempted to suspend a disconnected port.

For Marketing Only (1.5 Confidential)

Bits	Name	Access	Reset	Description
1	**pes**	RW	0	(Read) PortEnableStatus This bit indicates whether the port is enabled or disabled. The Root Hub may clear this bit when an overcurrent condition, disconnect event, switched-off power, or operational bus error such as babble is detected. This change also causes the PortEnableStatusChange bit (**pesc**) to be set. The HCD sets this bit by writing SetPortEnable and clears it by writing ClearPortEnable. This bit cannot be set when the CurrentConnectStatus bit (**ccs**) is cleared. This bit is also set, if not already, at the completion of a port reset when ResetStatusChange is set or port suspend when SuspendStatusChange is set. 0 - Port is disabled 1 - Port is enabled (Write) SetPortEnable The HCD sets PortEnableStatus by writing 1 to the bit **pes**. Writing 0 has no effect. If the CurrentConnectStatus bit (**ccs**) is cleared, this write does not set PortEnableStatus (**pes**), but instead sets ConnectStatusChange (**csc**). This informs the driver that it attempted to enable a disconnected port.
0	**ccs**	RW	0	(Read) CurrentConnectStatus This bit reflects the current state of the downstream port. 0 - No device connected 1 - Device connected (Write) ClearPortEnable The HCD writes 1 to this bit to clear the PortEnableStatus bit (**pes**). Writing 0 has no effect. The CurrentConnectStatus is not affected by a write. Note: This bit is always read 1 when the attached device is nonremovable (DeviceRemoveable[NDP]).

Table 12-23. OpenHCI Root Hub Port Status Register to Control and Report Port 1/2 Events.

For Internal Use Only (1.5 / 03-Dec-2025)

12.3.3 USB Host Registers: EnhancedHCI Map

Offset	Register	Description
0x00	**EHCI_caplength and EHCI_hciversion**	Capability and version
0x04	**EHCI_hcsparams**	Structural parameter
0x08	**EHCI_hccparams**	Capability parameter
0x0C		Reserved
0x10	**EHCI_usbcmd**	USB command
0x14	**EHCI_usbsts**	USB status
0x18	**EHCI_usbintr**	USB interrupt enable
0x1C	**EHCI_frindex**	USB frame index
0x20		Reserved
0x24	**EHCI_periodiclistbase**	Periodic frame list base address
0x28	**EHCI_asynclistaddr**	Asynchronous list address
0x2C – 0x4F		Reserved
0x50	**EHCI_configflag**	Configured flag
0x54	**EHCI_portsc_1**	Port 1 status / control
0x58	**EHCI_portsc_2**	Port 2 status / control

Table 12-24. EnhancedHCI Register Map.

12.3.4 USB Host Registers: EnhancedHCI Details

12.3.4.1 EHCI_caplength and EHCI_hciversion Register

Bits	Name	Access	Reset	Description
31:16	**hciversion**	R	0x0100	EHCI version supported
15:8				Reserved
7:0	**caplength**	R	0x10	Offset to start of operational registers

Table 12-25. EHCI Capability and Version Registers.

12.3.4.2 EHCI_hcsparams Register

Bits	Name	Access	Reset	Description
31:24				Reserved
23:20	**dpn**	R	0	Debug Port Number This register identifies which of the host controller ports is the debug port. The value is the port number (one-based) of the debug port. A nonzero value indicates a debug port is present.
19:17				Reserved

Confidential For Mekra Lang only 1.5 (08/01/25)

Bits	Name	Access	Reset	Description
16	**p_indicator**	R	0	Port Indicators This bit indicates whether the ports support port indicator control. When this bit is 1, the Port Status and Control registers (**EHCI_portsc_1/2**) include a read / writeable field for controlling the state of the port indicator.
15:12	**n_cc**	R	1	Number of Companion Controller This field indicates the number of companion controllers associated with this USB 2.0 host controller. A value of 0 indicates there are no companion host controllers. Port-ownership hand-off is not supported. Only high-speed devices are supported on the host controller root ports. A value larger than 0 indicates there are companion USB 1.1 host controller(s). Port-ownership hand-offs are supported. High, Full- and Low-speed devices are supported on the host controller root ports.
11:8	**n_pcc**	R	0x2	Number of Ports per Companion Controller This field indicates the number of ports supported per companion host controller. It is used to indicate the port routing configuration to system software.
7	**prr**	R	0	Port Routing Rules This field indicates the method used by this implementation for how all ports are mapped to companion controllers. The first N_PCC ports are routed to the lowest numbered function companion host controller, the next N_PCC ports are routed to the next lowest function companion controller, and so on.
6:5				Reserved
4	**ppc**	R	1	Port Power Control This field indicates whether the host controller implementation includes port power control. 1 - Ports have port power switches 0 - Ports do not have port power switches
3:0	**n_ports**	R	0x2	Number of Ports This field specifies the number of physical downstream ports implemented on this host controller. The value of this field determines how many port registers are addressable in the Operational Register Space.

Table 12-26. EHCI Structural Parameter Register.

12.3.4.3 EHCI_hccparams Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15:8	**eeco**	R	0xA0	EHCI Extended Capabilities Pointer (EECP) This optional field indicates the existence of a capabilities list. A value of 0 indicates no extended capabilities are implemented. A non-zero value in this register indicates the offset in PCI configuration space of the first EHCI extended capability. The pointer value must be 0x40 or greater if implemented to maintain the consistency of the PCI header defined for this class of device.

Bits	Name	Access	Reset	Description
7:4	**ist**	R	2	Isochronous Scheduling Threshold This field indicates, relative to the current position of the executing host controller, where the software can reliably update the isochronous schedule. When bit [7] is 0, the value of the least significant 3 bits indicates the number of micro-frames a host controller can hold a set of isochronous data structures (one or more) before flushing the state. When bit [7] is 1, then host software assumes the host controller may cache an isochronous data structure for an entire frame.
3				Reserved
2	**aspc**	R	1	Asynchronous Schedule Park Capability If this bit is set to 1, then the host controller supports the park feature for high-speed queue heads in the Asynchronous Schedule. The feature can be disabled or enabled and set to a specific level by using the Asynchronous Schedule Park Mode Enable and Asynchronous Schedule Park Mode Count fields in the **EHCI_usbcmd** register.
1	pflf	R	1	Programmable frame list flag
0				Reserved

Table 12-27. EHCI Capability Parameter Register.

12.3.4.4 EHCI_usbcmd Register

Bits	Name	Access	Reset	Description
31:24				Reserved
23:16	**itc**	RW	0x8	Interrupt Threshold Control This field is used by the system software to select the maximum rate at which the host controller will issue interrupts. The only valid values are defined below. If the software writes an invalid value to this register, the results are undefined. Value: Maximum Interrupt Interval 0x00 - Reserved 0x01 - 1 micro-frame 0x02 - 2 micro-frames 0x04 - 4 micro-frames 0x08 - 8 micro-frames (default, equates to 1 ms) 0x10 - 16 micro-frames (2 ms) 0x20 - 32 micro-frames (4 ms) 0x40 - 64 micro-frames (8 ms) Software modifications to this bit when **hchalted** equals 0 will produce undefined behavior.
15:12				Reserved
11	**aspme**	RW	1	Asynchronous Schedule Park Mode Enable The software uses this bit to enable or disable park mode. When this bit is 1, park mode is enabled. When this bit is 0, park mode is disabled.
10				Reserved

For Confidential Use Only
1.5 (Oct 2025)

Bits	Name	Access	Reset	Description
9:8	**aspmc**	RW	0x3	Asynchronous Schedule Park Mode Count Contains a count of the number of successive transactions the host controller may execute from a high-speed queue head on the Asynchronous schedule before continuing traversal of the Asynchronous schedule. Valid values are 0x1 to 0x3. Do not write 0 to these bits when **aspme** is 1 as this results in undefined behavior.
7	**lhcr**	RW	0	Light Host Controller Reset Allows the driver to reset the EHCI controller without affecting the state of the ports or the relationship to the companion host controllers. For example, the **EHCI_portsc** registers should not be reset to their default values and the **cf** bit of **EHCI_configflag** setting should not go to 0 (retaining port ownership relationships). If the host software reads this bit as 0 indicates the Light Host Controller Reset has completed and it is safe for the host software to re-initialize the host controller. When the host software reads this bit as 1 indicates the Light Host Controller Reset has not yet completed.
6	**iaad**	RW	0	Interrupt on Async Advance Doorbell This bit is used as a doorbell by the software to tell the host controller to issue an interrupt the next time it advances asynchronous schedule. The software must write 1 to this bit to ring the doorbell. When the host controller has evicted all appropriate cached schedule state, it sets the **EHCI_usbsts** Interrupt on Async Advance status bit (**iaa**). If the **EHCI_usbintr** Interrupt on Async Advance Enable bit (**iaae**) is 1 then the host controller will assert an interrupt at the next interrupt threshold. The host controller sets this bit to 0 after it has set the Interrupt on **iaa** to 1. The software should not write 1 to this bit when the asynchronous schedule is disabled. Doing so will yield undefined results.
5	**ase**	RW	0	Asynchronous Schedule Enable This bit controls whether the host controller skips processing the Asynchronous Schedule as follows: 0 - Do not process the Asynchronous Schedule 1 - Use **EHCI_periodiclistbase** to access the Asynchronous Schedule.
4	**pse**	RW	0	Periodic Schedule Enable This bit controls whether the host controller skips processing the Periodic Schedule as follows: 0 - Do not process the Periodic Schedule 1 - Use **EHCI_periodiclistbase** to access the Periodic Schedule.
3:2	**fls**	RW	0	Frame List Size This field specifies the size of the frame list. The size the frame list controls which bits in **EHCI_frindex** should be used for the Frame List Current index. Values mean: 0x0 - 1024 elements (4096 bytes) Default value 0x1 - 512 elements (2048 bytes) 0x2 - 256 elements (1024 bytes) – for resource-constrained environments 0x3 - Reserved

Bits	Name	Access	Reset	Description
1	**hcreset**	RW	0	Host Controller Reset The software uses this control bit to reset the host controller. The effects of this on Root Hub registers are similar to a chip hardware reset. When the software writes 1 to this bit, the host controller resets its internal pipelines, timers, counters, state machines, etc. to their initial value. Transactions currently in progress on USB are immediately terminated. A USB reset is not driven on downstream ports. All operational registers, including port registers and port state machines are set to their initial values. Port ownership reverts to the companion host controller. The software must reinitialize the host controller in order to return the host controller to an operational state. This bit is set to 0 by the host controller when the reset process is complete. The software cannot terminate the reset process early by writing 0 to this register. The software should not set this bit to 1 when the **EHCI_usbsts** HC Halted bit (**hchalted**) is 0. Attempting to reset an actively running host controller will result in undefined behavior.
0	**rs**	RW	0	Run / Stop 1 - Run 0 - Stop When set to 1, the host controller proceeds with execution of the schedule. The host controller continues execution as long as this bit is set to 1. When this bit is set to 0, the host controller completes the current and actively pipelined transactions on the USB and then halts. The host controller must halt within 16 micro-frames after the software clears the **run** bit. The **EHCI_usbsts** HC Halted bit (**hchalted**) indicates when the host controller has finished its pending pipelined transactions and has entered the stopped state. The software must not write 1 to this field unless the host controller is in the halted state (**hchalted** = 1). Doing so will yield undefined results.

Table 12-28. EHCI USB Command Register.

12.3.4.5 EHCI_usbsts Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15	**ass**	R	0	Asynchronous Schedule Status The bit reports the current real status of the Asynchronous Schedule. If this bit is 0 then the status of the Asynchronous Schedule is disabled. If this bit is 1 then the status of the Asynchronous Schedule is enabled. The host controller is not required to immediately disable or enable the Asynchronous Schedule when the software transitions the **EHCI_usbcmd** Asynchronous Schedule Enable bit (**ase**). When this bit and **ase** are the same value, the Asynchronous Schedule is either enabled (1) or disabled (0).

Bits	Name	Access	Reset	Description
14	**pss**	R	0	Periodic Schedule Status The bit reports the current real status of the Periodic Schedule. If this bit is 0 then the status of the Periodic Schedule is disabled. If this bit is 1 then the status of the Periodic Schedule is enabled. The host controller is not required to immediately disable or enable the Periodic Schedule when the software transitions the **EHCI_usbcmd** Periodic Schedule Enable bit (**pse**). When this bit and **pse** are the same value, the Periodic Schedule is either enabled (1) or disabled (0).
13	**rec**	R	0	Reclamation This is a read-only status bit, which is used to detect an empty asynchronous schedule.
12	**hchalted**	R	1	HCHalted This bit is 0 when the **EHCI_usbcmd** Run/Stop bit (**rs**) is 1. The host controller sets this bit to 1 after it has stopped executing as a result of the Run/Stop bit being set to 0, either by the software or by the host controller hardware (e.g., internal error).
11:6				Reserved
5	**iaa**	R_WC	0	Interrupt on Async Advance System software can force the host controller to issue an interrupt the next time the host controller advances the asynchronous schedule by writing 1 to the Interrupt on **EHCI_usbcmd** Async Advance Doorbell bit (**iaad**). This status bit indicates the assertion of that interrupt source.
4	**hse**	R_WC	0	Host System Error The host controller sets this bit to 1 when a serious error occurs during a host system access involving the host controller module. When this error occurs, the host controller clears the **EHCI_usbcmd** Run/Stop bit (**rs**) to prevent further execution of the scheduled TDs.
3	**flr**	R_WC	0	Frame List Rollover The host controller sets this bit to 1 when the Frame List Index rolls over from its maximum value to 0. The exact value at which the rollover occurs depends on the frame list size. For example, if the frame list size (as programmed in the Frame List Size field of the **EHCI_usbcmd** register) is 1024, **EHCI_frindex** rolls over when **EHCI_frindex**[13] toggles. Similarly, if the size is 512, the host controller sets this bit to 1 when **EHCI_frindex**[12] toggles.
2	**pcd**	R_WC	0	Port Change Detect The host controller sets this bit to 1 when a port for which the Port Owner bit is set to 0 has a change bit transition from 0 to 1 or a Force Port Resume **fpr** bit transition from 0 to 1 as a result of a J-K transition detected on a suspended port. This bit will also be set when the Connect Status Change bit (**csc**) is set to 1; i.e., after system software has relinquished ownership of a connected port by writing 1 to a port's Port Owner bit (**po**). This bit may be maintained in the Auxiliary power well. Alternatively, it is also acceptable that on a D3 to D0 transition of the EHCI HC device, this bit is loaded with the OR of all of the **EHCI_portsc** change bits (e.g., force port resume, over-current change, enable / disable change, and connect status change).

For Internal Use Only

Bits	Name	Access	Reset	Description
1	**usberrint**	R_WC	0	USB Error Interrupt The host controller sets this bit to 1 when completion of a USB transaction results in an error condition (e.g., error counter underflow). If the Transfer Descriptor on which the error interrupt occurred also had its Interrupt on Complete bit (IOC) set, then both this bit and **usbint** are set.
0	**usbint**	R_WC	0	USB Interrupt The host controller sets this bit to 1 on the completion of a USB transaction, which results in the retirement of a Transfer Descriptor that had its Interrupt on Complete bit (IOC) set. The host controller also sets this bit to 1 when a short packet is detected (actual number of bytes received was less than the expected number of bytes).

Table 12-29. EHCI USB Status Register.

12.3.4.6 EHCI_usbintr Register

Bits	Name	Access	Reset	Description
31:6				Reserved
5	**iaae**	RW	0	When this bit is 1, and the **EHCI_usbsts** Interrupt on Async Advance bit (**iaa**) is 1, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged when the software clears **iaa**.
4	**hsee**	RW	0	When this bit is 1, and the **EHCI_usbsts** Host System Error Status bit (**hse**) is 1, the host controller will issue an interrupt. The interrupt is acknowledged by the software clearing **hse**.
3	**flre**	RW	0	When this bit is 1, and the **EHCI_usbsts** Frame List Rollover bit (**flr**) is 1, the host controller will issue an interrupt. The interrupt is acknowledged by the software clearing **flr**.
2	**pcie**	RW	0	When this bit is 1, and the **EHCI_usbsts** Port Change Detect bit (**pcd**) is 1, the host controller will issue an interrupt. The interrupt is acknowledged by the software clearing **pcd**.
1	**usbeie**	RW	0	When this bit is 1, and the **EHCI_usbsts** USB interrupt bit (**usberrint**) is 1, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by the software clearing **usberrint**.
0	**usbie**	RW	0	When this bit is 1, and the **EHCI_usbsts** USB interrupt bit (**usberrint**) is 1, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by the software clearing **usbint**.

Table 12-30. EHCI USB Interrupt Enable Register.

For Ambarella Confidential 1.5 (CONFIDENTIAL)

12.3.4.7 EHCI_frindex Register

Bits	Name	Access	Reset	Description
31:14				Reserved
13:0	**frindex**	RW	0	Frame Index The value in this register increments at the end of each time frame (e.g.micro-frame). Bits [N:3] are used for the Frame List current index. This means that each location of the frame list is accessed 8 times (frames or micro-frames) before moving to the next index. The following illustrates values of N based on the value of the Frame List Size field **fls** in the **EHCI_usbcmd** register: **fls** = 0x0 - 1024 elements and N = 12 **fls** = 0x1 - 512 elements and N = 11 **fls** = 0x2 - 256 elements and N = 10 **fls** = 0x3 - Reserved

Table 12-31. EHCI USB Frame Index Register.

12.3.4.8 EHCI_periodiclistbase Register

Bits	Name	Access	Reset	Description
31:12	**ba**	RW	0	Base Address These bits correspond to memory address signals [31:12], respectively.
11:0				Reserved and must be 0x000

Table 12-32. EHCI Periodic Frame List Base Address Register.

12.3.4.9 EHCI_asynclistaddr Register

Bits	Name	Access	Reset	Description
31:5	**lpl**	RW	NR	Link Pointer Low These bits correspond to memory address signals [31:5], respectively. This field may only reference a Queue Head (QH).
4:0				Reserved

Table 12-33. EHCI Asynchronous List Address Register.

Confidential For Mediatek only (Oct 2025)

12.3.4.10 EHCI_configflag Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**cf**	RW	0	Configure Flag Host software sets this bit as the last action in its process of configuring the Host Controller. This bit controls the default port-routing control logic. Bit values and side-effects are listed below. 0 - Port routing control logic default-routes each port to an implementation dependent classic host controller. 1 - Port routing control logic default-routes all ports to this host controller.

Table 12-34. EHCI Configured Flag Register.

12.3.4.11 EHCI_portsc_1/2 Registers

Bits	Name	Access	Reset	Description
31:23				Reserved
22	**wkoc_e**	RW	0	Wake on Over-Current Enable Writing this bit to 1 enables the port to be sensitive to over-current conditions as wake-up events. This field is 0 if Port Power bit (**pp**) is 0.
21	**wkdscnnt_e**	RW	0	Wake on Disconnect Enable Writing this bit to 1 enables the port to be sensitive to device disconnects as wake-up events. This field is 0 if **pp** is 0.
20	**wkcnne_e**	RW	0	Wake on Connect Enable Writing this bit to 1 enables the port to be sensitive to device connects as wake-up events. This field is 0 if **pp** is 0.
19:16	**ptc**	RW	0	Port Test Control When this field is 0, the port is NOT operating in a test mode. Non-zero values indicate test mode operation and specific test modes. The encoding of the test mode bits are as follows: 0x0 - Test mode not enabled 0x1 - Test J_STATE 0x2 - Test K_STATE 0x3 - Test SE0_NAK 0x4 - Test Packet 0x5 - Test FORCE_ENABLE 0x6 through 0xF - Reserved
15:14	**pic**	RW	0	Port Indicator Control Writing to these bits has no effect if the **EHCI_hcsparams** register **p_indicator** bit is 0. If **p_indicator** is 1, then the bit encodings are as follows: 0x0 - Port indicators are off 0x1 - Amber 0x2 - Green 0x3 - Undefined

For Confidential 1.5 Customers Only

Bits	Name	Access	Reset	Description
13	**po**	RW	1	Port Owner This bit unconditionally goes to 0 when the **EHCI_configflag** configured bit (**cf**) transitions from 0 to 1. This bit unconditionally goes to 1 when **cf** is 0. The system software uses this field to release ownership of the port to a selected host controller (in the event that the attached device is not a high-speed device). The software writes 1 to this bit when the attached device is not a high-speed device. When this bit is 1 the companion host controller owns and controls the port.
12	**pp**	RW	0	Port Power This bit represents the current setting of the port power switch (0 = off, 1 = on). When power is not available on a port (**pp** = 0), the port is nonfunctional and will not report attaches, detaches, etc. When an over-current condition is detected on a powered port and the **EHCI_hcsparams** bit **ppc** is 1, the **pp** bit in each affected port may be transitioned by the host controller from 1 to 0 (removing power from the port).
11:10	**ls**	R	0	Line Status These bits reflect the current logical levels of the D+ (bit 11) and D- (bit 10) signal lines. These bits are used for detection of low-speed USB devices prior to the port reset and enable sequence. This field is valid only when the Port Enable bit (**ped**) is 0 and the current connect status bit is set to 1. 0x0 - USB state of SE0. This is not a low-speed device, EHCI reset should be performed. 0x1 - USB K_STATE. This is a low-speed device, and ownership of port should be released. 0x2 - USB J_STATE. This is not a low-speed device, and EHCI reset shouldl be performed. 0x3 - USB State is Undefined. This is not a low-speed device, and EHCI reset should be performed. This field is undefined if Port Power (**pp**) is 0.
9				Reserved

For Ambarella internal use only (1.5 (CV22FS_0825))

Bits	Name	Access	Reset	Description
8	**pr**	RW	0	Port Reset 1 - Port is in Reset 0 - Port is not in Reset When the software writes 1 to this bit (from 0), the bus reset sequence is started per the USB Specification Revision 2.0. The software writes 0 to this bit to terminate the bus reset sequence and must keep this bit at 1 long enough to ensure the reset sequence completes. When the software writes this bit to 1, it must also write 0 to the Port Enable bit (**ped**). When the software writes this bit to 0 there may be a delay before the bit status changes to 0. The bit status will not read 0 until after the reset completes. If the port is in high-speed mode after reset is complete, the host controller will automatically enable this port (e.g., set the Port Enable bit to 1). A host controller must terminate the reset and stabilize the state of the port within 2 ms of transitioning this bit from 1 to 0. For example, if the port detects that the attached device is high-speed during reset, then the host controller must have the port in the enabled state within 2 ms of writing this bit to 0. The **EHCI_usbsts** bit **hchalted** should be 0 before the software attempts to use this bit. The host controller may hold Port Reset **pr** asserted to 1 when **hchalted** is 1. This field is 0 if Port Power (**pp**) is 0.
7	**spd**	RW	0	Suspend 1 - Port in suspend state 0 - Port not in suspend state Port Enabled (**ped)** and Suspend (**spd**) bits of this register define the port states: Disable - **ped** = 0 and **spd** = X Enable - **ped** = 1 and **spd** = 0 Suspend - **ped** = 0 and **spd** = 1 In suspend state, downstream propagation of data is blocked on this port, except for port reset. The blocking occurs at the end of the current transaction, if a transaction was in progress when this bit was written to 1. In the suspend state, the port is sensitive to resume detection. Note that the bit status does not change until the port is suspended and there may be a delay in suspending a port if there is a transaction currently in progress on the USB. A write of 0 to this bit is ignored by the host controller. The host controller will unconditionally set this bit to 0 when: The software sets the Force Port Resume bit (**fpr**) to 0 (from 1). The software sets the Port Reset bit (**pr**) to 1 (from 0). If the host software sets this bit to 1 when the port is not enabled (**ped** = 0) the results are undefined. This field is 0 if Port Power (**pp**) is 0.

For Mass Confidential 1.5 Only Ambarella Inc.

Bits	Name	Access	Reset	Description
6	**fpr**	RW	0	Force Port Resume 1 - Resume detected/driven on port 0 - No resume (K_STATE) detected/driven on port Functionality defined for manipulating this bit depends on the value of the Suspend bit (**spd**). For example, if the port is not suspended (**spd** = **ped** = 1) and the software transitions this bit to 1, then the effects on the bus are undefined. The software sets this bit to 1 to drive resume signaling. The Host Controller sets this bit to 1 if a J-to-K transition is detected while the port is in the Suspend state. When this bit transitions to 1 because a J-to-K transition is detected, the Port Change Detect bit (**pcd**) also is set to 1. If the software sets this bit to 1, the host controller must not set **pcd**. Note that when the EHCI controller owns the port, the resume sequence follows that defined in the USB Specification Revision 2.0. The resume signaling (Full-speed 'K') is driven on the port while this bit remains 1. The software must appropriately time the Resume and set this bit to 0 when the time has elapsed. Writing 0 (from 1) causes the port to return to high-speed mode (forcing the bus below the port into a high-speed idle). This bit will remain 1 until the port has switched to the high-speed idle. The host controller must complete this transition within 2 ms after the software has set this bit to 0. This field is 0 if Port Power (**pp**) is 0.
5	**occ**	R_WC	0	Over-Current Change 1 - This bit gets set to 1 when there is a change to over current active. This bit is cleared by a write of 1.
4	**oca**	R	0	Over-Current Active 1 - This port currently has an over-current condition 0 - This port does not have an over-current condition This bit will automatically transition from 1 to 0 when the over current condition is removed.
3	**pedc**	R_WC	0	Port Enable / Disable Change 1 - Port enabled/disabled status has changed 0 - No change For the root hub, this bit gets set to 1 only when a port is disabled due to the appropriate conditions existing at the EOF2 point. This bit is cleared by writing it to 1. This field is 0 if Port Power (**pp**) is 0.
2	**ped**	RW	0	Port Enabled / Disabled 1 - Enable 0 - Disable Ports can only be enabled by the host controller as a part of the reset and enable. The software cannot enable a port by writing 1 to this field. The host controller will set this bit to 1 only when the reset sequence determines that the attached device is a high-speed device. Ports can be disabled by either a fault condition (disconnect event or other fault condition) or the host software. Note that the bit status does not change until the port state actually changes. There may be a delay in disabling or enabling a port due to other host controller and bus events. When the port is disabled (0) downstream propagation of data is blocked on this port, except for reset. This field is 0 if Port Power (**pp**) is 0.

For Ambarella Internal Use Only 1.5 (Confidential)

Bits	Name	Access	Reset	Description
1	**csc**	R_WC	0	Connect Status Change 1 - Change in Current Connect Status 0 - No change Indicates a change has occurred in the port's Current Connect Status (**ccs**). The host controller sets this bit for all changes to the port device connect status, even if the system software has not cleared an existing connect status change. For example, the insertion status changes twice before the system software has cleared the changed condition, hub hardware will be attempting to "set" an already-set bit (i.e., the bit remains set). The software sets this bit to 0 by writing 1 to it. This field is 0 if Port Power (**pp**) is 0.
0	**ccs**	R	0	Current Connect Status 1 - Device is present on port 0 - No device is present This value reflects the current state of the port, and may not correspond directly to the event that caused the Connect Status Change bit (**csc**) to be set. This field is 0 if Port Power (**pp**) is 0.

Table 12-35. EHCI Status/Control Registers for Ports 1/2.

For Mekra Lang only
1.5 (08012025)
Confidential

13. SPI NAND FLASH

The CV22FS processor provides both a flash I/O subsystem and a SPI NAND flash controller. For the purposes of this manual, these will be collectively referred to as the flash interface.

This chapter includes the following sections:

- (Section 13.1) Flash: Overview
- (Section 13.2) Flash: Flash I/O Subsystem
- (Section 13.3) Flash: SPI NAND Flash Controller
- (Section 13.4) Flash: FDMA Engine
- (Section 13.5) Flash: Safety Notes
-

Refer to Chapter 2 for SPI NAND flash boot mode details. For interface pins and descriptions, refer to the data-sheet for CV22FS.

13.1 Flash: Overview

13.1.1 Overview: Features

The flash I/O subsystem and the SPI NAND flash controller enable interaction with external SPI NAND flash devices. Features of the flash interface include:

- Flash I/O subsystem
  - Programming of BCH error correction for the SPI NAND flash controller
- SPI NAND flash controller
  - Up to 128-Gbit flash chip size
  - Page Size: 2048 + 64/128 bytes or 4096 + 128/256 bytes
  - Support for SPI NAND flash
  - Custom commands with data to/from FDMA engine
  - BCH error correction
  - SPI NAND flash boot option
- FDMA engine
  - Programming of DMA / FIFO transactions for SPI NAND flash
  - Enables flash DMA dual-space mode for the SPI NAND flash controller

Confidential For Mekra Lang only 1.5 (08012025)

13.1.2 Overview: Block Diagram

Figure 13-1 illustrates the functional blocks of the flash I/O (FIO) subsystem

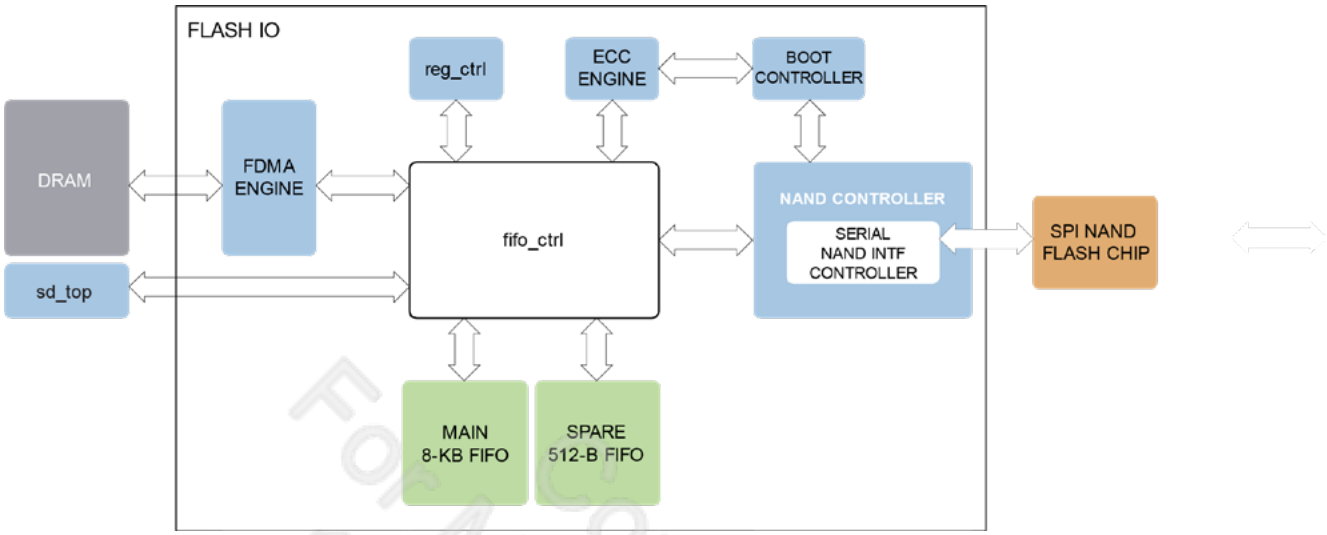


Figure 13-1. Functional Block Diagram of the Flash I/O Subsystem.

The flash I/O subsystem utilizes a CPU interface and an interface to the AXI FDMA engine. The subsystem uses the AXI FDMA engine for DMA transfers to and from main memory. The FDMA engine offers normal and dual-space mode operations for fast DMA transactions. There is a DMA FIFO controller for the Data FIFO that buffers data from / to the FDMA engine for data write / read requests to / from the SPI NAND flash controller. The VIC generates interrupts for the subsystem, including the SPI NAND flash controller CPU Command-done and DMA Command-done Interrupts.

13.2 Flash: Flash I/O Subsystem

This section discusses address space, programming and registers for the overall flash I/O subsystem as follows:

- (Section 13.2.1) Flash I/O: Address Space
- (Section 13.2.2) Flash I/O: Operation
- (Section 13.2.3) Flash I/O: BCH Error Correction and Dual-Space Mode
- (Section 13.2.4) Flash I/O: BCH Codec Protected Range
- (Section 13.2.5) Flash I/O: Enabling BCH and Dual-Space Mode
- (Section 13.2.6) Flash I/O: Subsystem Initialization
- (Section 13.2.7) Flash I/O: Register Detail

For SPI NAND flash controller registers and programming see Section 13.3.

13.2.1 Flash I/O: Address Space

The flash I/O subsystem and the DMA interface occupy a single 4-KB address block on the AHB.

13.2.1.1 Flash I/O: CPU Interface Address Space

The base address space starting at E000.2000 is designated for the flash I/O CPU interface. This range is further divided into sub-ranges for the control and status registers (CSRs) of the flash I/O subsystem. The table below shows the AHB bus transaction information when the CPU address space is accessed.

Access Targets	Legacy AHB Base Address	Revised AHB Base Address	AHB Offset
Flash I/O Global Registers	AHB_REG_FLASH_CPU_BASE	AHB_REG_FLASH_CPU_BASE	0x000 – 0x0FF
Flash SPI NAND Controller Registers	AHB_REG_FLASH_CPU_BASE	AHB_REG_FLASH_CPU_BASE	0x120 – 0x1FF
FDMA Registers	AHB_REG_DMA_FIOS_BASE	AHB_REG_FLASH_CPU_BASE	0x200 – 0x3FF

Table 13-1. Flash I/O Subsystem CPU Interface Address Space.

13.2.2 Flash I/O: Operation

Once the flash I/O subsystem is initialized and configured, operations may be initiated in the memory targets (flash chips). The two basic types of operations include the following:

1. Data transfer operations such as read/write of the memory targets or the DMA transfers of data between main memory and the flash I/O subsystem.
2. Non-data transfer operations such as the configuration of memory targets, the retrieval of read memory status information, and commands that do not require block data transfers, such as erase and reset.

13.2.3 Flash I/O: BCH Error Correction and Dual-Space Mode

The CV22FS chip supports BCH (6- or 8-bit) error correction. The FDMA engine must be in dual-space mode to enable the BCH codec controller and BCH encoding / decoding.

Enabling dual-space mode (Section 13.2.5) improves the efficiency of the BCH codec controller by interleaving the read/write sequence between memory and the data FIFO. Note that programming is required to prepare main-area and spare-area space in advance. Dual-space mode is also used for flash chips with only one partial programming cycle; i.e., the operational mode that can simultaneously access flash main and spare areas.

Operations performed in dual-space mode proceed according to the following general steps:

1. The BCH codec controller generates handshake signals with the BCH encoder/decoder and enables transfers between memory and the BCH encoder/decoder.
2. Initially, the FDMA engine transfers one 2-KB page of data with the interleave sequence of 4 x 512 B of main-area data and 32 B of spare-area data, or one 4-KB page of data with the interleave sequence of 8 x 512 B of main-area data and 64 B of spare-area data. The FIFO controller stores the main- and spare-area data in the data FIFO buffer and spare RAM, respectively.

Ambarella Confidential
For Meta Lang only
15 Oct 2025

3. When the first pair of data is ready, the BCH codec controller begins fetching data and feeding it uncoded to the BCH encoder. After BCH parity is reached, the BCH codec controller writes the coded data back to Spare RAM. Meanwhile, the FIFO controller begins fetching the main-area data (whole) from the FIFO buffer and sending it to the SPI NAND flash controller. The FIFO controller then begins reading spare-area data from the spare RAM when the parity data are ready.

13.2.4 Flash I/O: BCH Codec Protected Range

The following diagrams illustrate the protected ranges for the BCH codec:

- Section 13.2.4.1 “Flash BCH - 2K + 64 Bytes, x8 SPI NAND”
- Section 13.2.4.2 “Flash BCH - 2K + 128 Bytes, x8 SPI NAND”
- Section 13.2.4.3 “Flash BCH - 4K + 128 Bytes, x8 SPI NAND”
- Section 13.2.4.4 “Flash BCH - 4K + 256 Bytes, x8 SPI NAND”

13.2.4.1 Flash BCH - 2K + 64 Bytes, x8 SPI NAND

For this mode, the protected message includes all data in the main area and the first six bytes of each 16-byte subset in the spare area. To function correctly, **fdma_dsm_control** register bits **main_jp_size** must be 0x9 and **spare_jp_size** must be 0x4**.** The DMA transaction byte count at **fdma_mn_ctrl** (0x300) bits **xact_blen** must be a multiple of 2112 bytes.

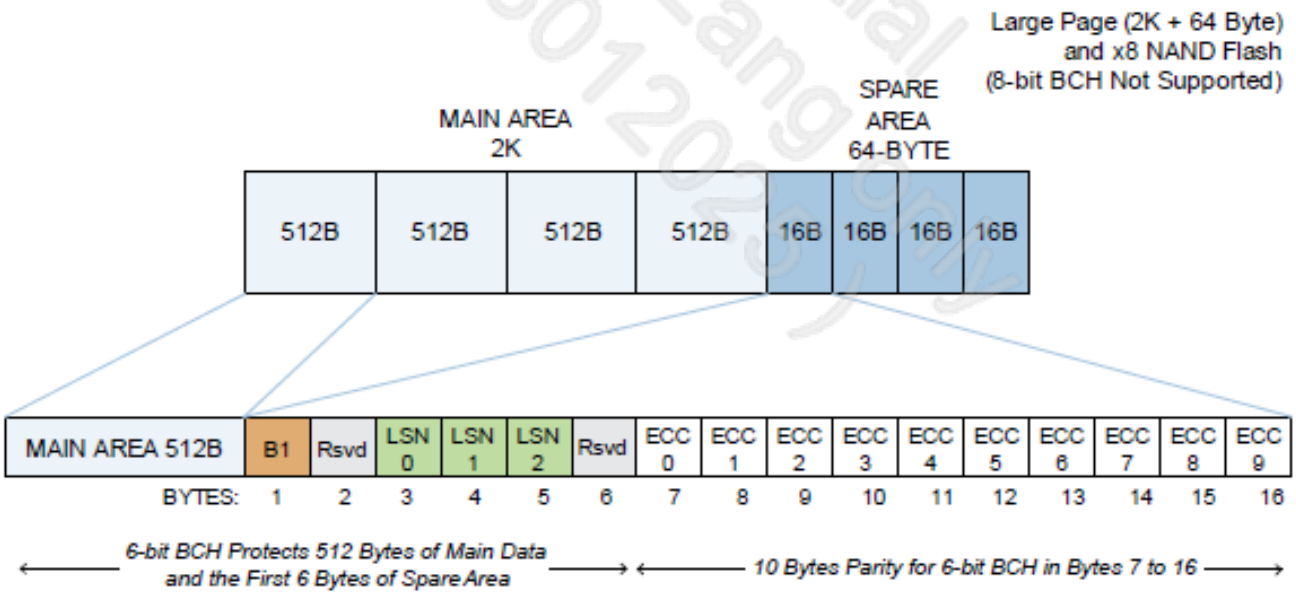


Figure 13-2. Flash BCH Codec: 2K + 64 Bytes and x8 SPI NAND.

13.2.4.2 Flash BCH - 2K + 128 Bytes, x8 SPI NAND

For 8-bit BCH, the protected message range includes all data in the main area and the first 19 bytes of each 32-byte subset in the spare area. To function correctly with 8-bit BCH, **fdma_dsm_control** register bits **main_jp_size** must be 0x9 and **spare_jp_size** must be 0x5**.** The DMA transaction byte count at **fdma_mn_ctrl** (0x300) bits **xact_blen** must be a multiple of 2176 bytes.

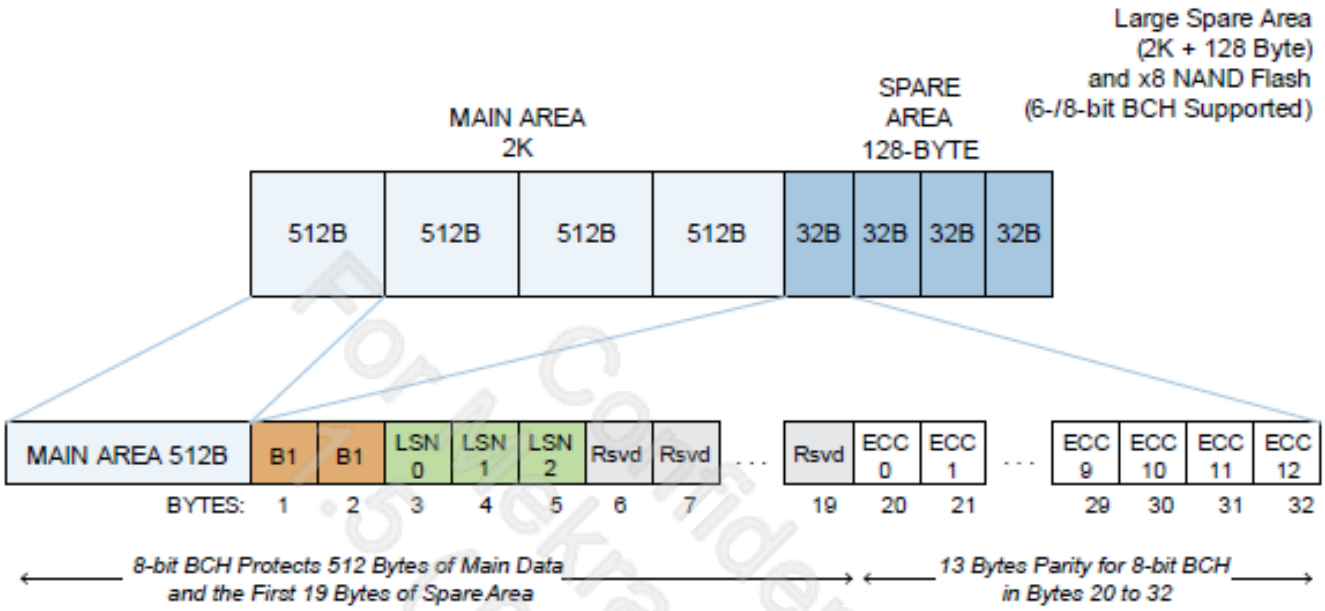


Figure 13-3. Flash BCH Codec: 2K + 128 Bytes, x8 SPI NAND.

13.2.4.3 Flash BCH - 4K + 128 Bytes, x8 SPI NAND

For this mode, the protected message includes all data in the main area and the first six bytes of each 16-byte subset in the spare area. To function correctly, **fdma_dsm_control** register bits **main_jp_size** must be 0x9 and **spare_jp_size** must be 0x4**.** The DMA transaction byte count at **fdma_mn_ctrl** (0x300) bits **xact_blen** must be a multiple of 4224 bytes.

Refer to the figure on the following page.

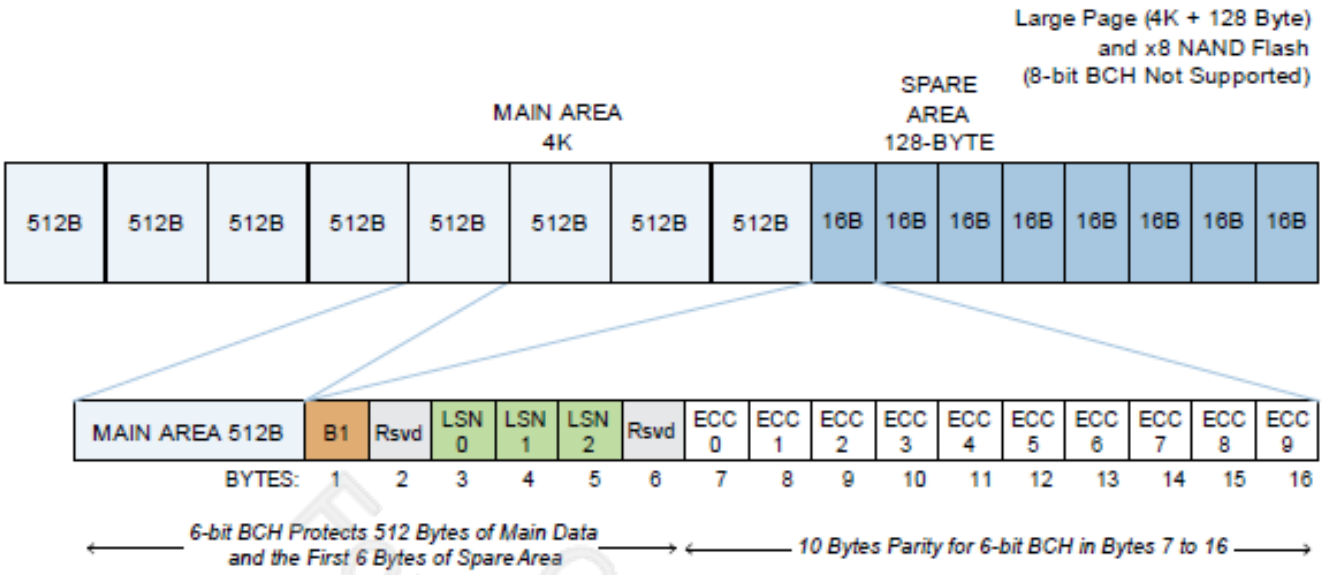


Figure 13-4. Flash BCH Codec: 4K + 128 Bytes, x8 SPI NAND.

For Mekatang Confidential
1.5 Oct 01 2025

13.2.4.4 Flash BCH - 4K + 256 Bytes, x8 SPI NAND

For 8-bit BCH, the protected message range includes all data in the main area and the first 19 bytes of each 32-byte subset in the spare area. To function correctly with 8-bit BCH, **fdma_dsm_control** register bits **main_jp_size** must be 0x9 and **spare_jp_size** must be 0x5. The DMA transaction byte count at **fdma_mn_ctrl** (0x300) bits **xact_blen** must be a multiple of 4352 bytes.

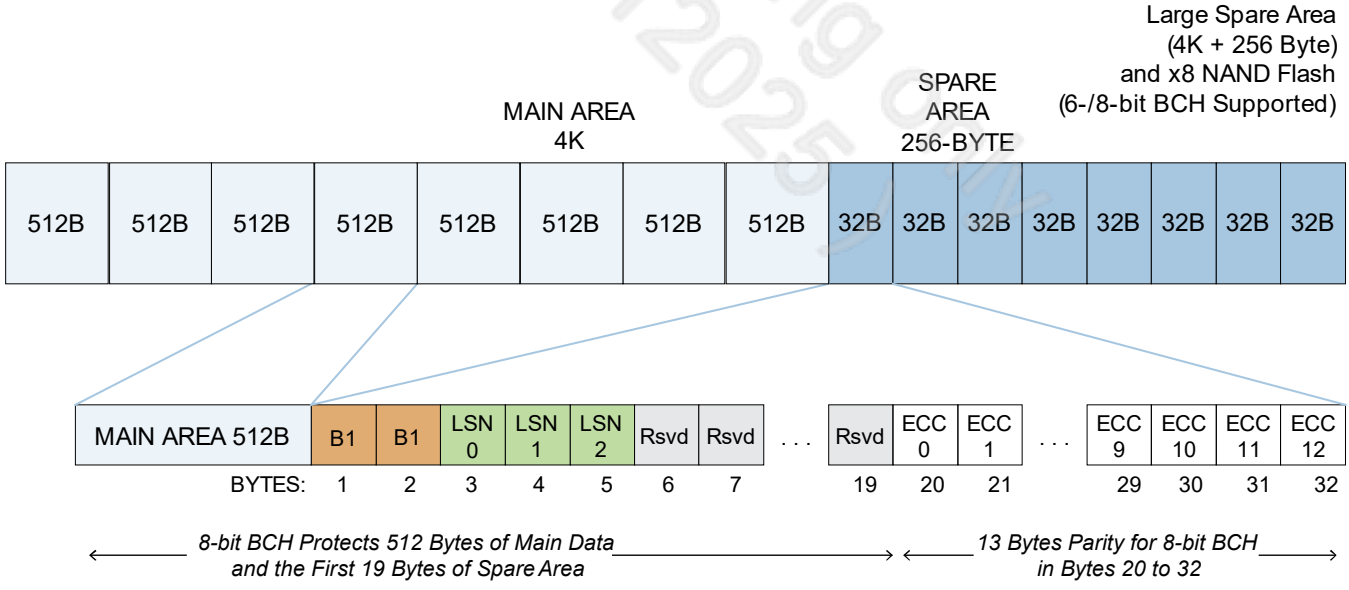


Figure 13-5. Flash BCH Codec: 4K + 256 Bytes, x8 SPI NAND.

13.2.5 Flash I/O: Enabling BCH and Dual-Space Mode

The BCH codec should be enabled prior to using **fdma_dsm_control** to enable the DMA engine. Refer to Section 13.2.6 below for the flash I/O initialization steps.

1. Switch to dual-space mode by following the steps below.
   A. Setup the main / spare jump size using the **fdma_dsm_control** register (0x3A0) bits **main_jp_size** (bits 7:4) and **spare_jp_size** (bits 3:0). The flash main-area byte count should be 512, and the spare-area byte count should be 16 or 32 depending on the BCH codec protected-range category.
2. Use the **Flash_IO_control** (0x000) register to setup the **ecc_bch_8t** and **ecc_bch_en** options as appropriate.

The DMA FIFO controller uses the ECC error-detected interrupt for read-disturb issues that occur in flash chips using 6/8-bit ECC. When the BCH decoder detects an error, the SPI NAND flash block number is recorded and the status is set in the **Flash_IO_ecc_rpt_status** and **Flash_IO_ecc_rpt_status2** registers before the interrupt is asserted. If there are errors detected in the same block on different pages, the interrupt will be asserted only once. Note that the **Flash_IO_ecc_rpt_status** register records only the first block number with a detected error; i.e., register content may be overwritten before software can read it.

If there is an overwritten event, the **ecc_rpt_corr_fail_ov** bit in the **Flash_IO_ecc_rpt_status** register will be set.

Note

To disable dual-space mode, **set main_jp_size** and **spare_jp_size** registers to 0.

13.2.6 Flash I/O: Subsystem Initialization

Before a flash chip can be accessed, the flash I/O subsystem registers must be initialized after reset. Initialization steps are as follows:

1. Exit random-read mode by writing to the flash I/O Control register (**Flash_IO_control**).
2. Read the Flash I/O DMA Status register to retrieve the completion status of the automatic Data FIFO fill (**Flash_IO_dma_status**).
3. Clear the Flash I/O DMA Status register (**Flash_IO_dma_status**).
4. Initialize the SPI NAND flash controller.

Note

To re-enter random-read mode, S/W must ensure FDMA engine and SPI NAND flash controller are both in the IDLE state.

Ambarella Confidential Only (Wakra Lang 1.5 (08012023))

13.2.7 Flash I/O: Register Detail

The flash I/O subsystem contains registers that hold configuration information, control information, timing parameters, status, IDs, and commands. These registers communicate with the major blocks of the flash I/O subsystem including the following:

- Flash I/O Subsystem Global Registers
- 8-KB Data FIFO
- Flash Controller

The registers listed in the following table (beginning on the next page) are memory-mapped into the AHB bus CPU slave address space of the flash I/O subsystem. They are accessed with single 4-byte AHB bus transactions. During an AHB bus transaction, AHB address[31:10] is the base address[31:10] of the flash I/O subsystem CPU slave address space, and AHB address[9:0] is the register index that indicates the register being accessed. The table lists the flash I/O subsystem registers, their register indexes, access permissions, and brief descriptions.

For SPI NAND flash controller registers see Section 13.3.2 “SPI NAND Flash: Register Detail”

Confidential
For Mekra Lang only
1.5 (08012025)

Offset	Register	Field	Bits	Access	Reset	Description
0x000	io_ctrl	Reserved	31:9	-	-	Reserved
		non_dma_mode	8	RW	0	Enables reading data from flash to FIFO without enabling FDMA. 1 - Enable 0 - Disable
		skip_blank_page_ecc	7	RW	POC	Skip BCH decoding failure when the page is blank. 1 - Enable 0 - Disable The reset value is POC[16] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
		ecc_bch_en	6	RW	POC	BCH encoding/decoding enable. 1 - Enable 0 - Disable The reset value is POC[15] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
		ecc_bch_8t	5	RO	POC	t parameter of BCH codec. This bit is bundled with nd_sp_2x register and thus is RO-type register. 1 - Set t = 8 0 - Set t = 6 The reset value is POC[14] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
		Reserved	4	-	-	Reserved
		rderr_stop	3	RW	0	Stop on error 1 - DMA will end early when uncorrectable ECC errors are detected for SPI NAND flash reads.
		Reserved	2	-	-	Reserved
		fifo_random	1	RW	1	Random read mode 1 - Flash I/O Subsystem is in random read mode, and a location in the 4-KB Data FIFO can be read by the host processor 0 - No 4-KB Data FIFO random reads can be done. When this bit is changed from 1 to 0, the 4 KB Data FIFO is emptied.
		Reserved	0	-	-	Reserved

For Mekra Lang GmbH & Co. KG
Confidential
1.5 10801202

Offset	Register	Field	Bits	Access	Reset	Description
0x004	raw_int_sts	Reserved	31:6	-	-	Reserved
		ecc_rpt_uncorr_raw_int	5	RW1C	0	Interrupt status indicating there is uncorrectable BCH error detected.
		ecc_rpt_raw_int	4	RW1C	0	Interrupt status indicating there are BCH error bits greater or equal to ecc_rpt_err_num_th corrected during the DMA operation.
		Reserved	3	-	-	Reserved
		axi_bus_err_raw_int	2	RW1C	0	Interrupt status indicating DMA bus returns error response on either R channel or B channel.
		snd_loop_timeout_raw_int	1	RW1C	0	SPI-NAND timeout while executing auto-status-update.
		fio_raw_int	0	RW1C	0	Flash operation done interrupt status. If DMA engine is in descriptor mode, this interrupt status is asserted when all descriptors are processed.
0x008	io_ctrl2	Reserved	31:2	-	-	Reserved
		spi_nd	1	RW	POC	1: SPI-NAND 0: (Parallel) NAND The reset value is POC[22] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
		nand_flash	0	RO	1	SPI NAND flash mode. NOR flash is not supported in this controller, and thus this bit is tied to 1.
0x00c	int_en	Reserved	31:6	-	-	Reserved
		ecc_rpt_uncorr_int_en	5	RW	1	Interrupt enable of ecc_rpt_uncorr_int.
		ecc_rpt_int_en	4	RW	1	Interrupt enable of ecc_rpt_int.
		Reserved	3	-	-	Reserved
		axi_bus_err_int_en	2	RW	1	Interrupt enable of axi_bus_err_int
		snd_loop_timeout_int_en	1	RW	1	Interrupt enable of snd_loop_timeout_int
		fio_int_en	0	RW	1	Interrupt enable of fio_int
0x010	int_sts	Reserved	31:6	-	-	Reserved
		ecc_rpt_uncorr_int	5	RO	0	Equivalent to (ecc_rpt_uncorr_raw_int & ecc_rpt_uncorr_int_en)
		ecc_rpt_int	4	RO	0	Equivalent to (ecc_rpt_raw_int & ecc_rpt_int_en)
		Reserved	3	-	-	Reserved
		axi_bus_err_int	2	RO	0	Equivalent to (axi_bus_err_raw_int & axi_bus_err_int_en)
		snd_loop_timeout_int	1	RO	0	Equivalent to (snd_loop_timeout_raw_int & snd_loop_timeout_int_en)
		fio_int	0	RO	0	Equivalent to (fio_raw_int & fio_int_en)

Confidential
For Meta Lab Only
15 Oct 2021

Offset	Register	Field	Bits	Access	Reset	Description
0xa0	ecc_rpt_cfg	Reserved	31:20	-	-	Reserved
		ecc_rpt_err_num_th	19:16	RW	POC	The min number of correctable bits in single BCH block not to trigger ecc_rpt_int., for example. ecc_rpt_int asserts when correctable error bit number is more than ecc_rpt_err_num_th. When POC[15] is 0, the default value is 6. Otherwise, the default value is 8. *Range:“{[0:8]}”
		Reserved	15:0	-	-	Reserved
0xa4	ecc_rpt_sts	ecc_rpt_err_det	31	R	0	Flag indicates there is data error detected during the last DMA operation. This field is self-cleared when a new DMA operation begins.
		ecc_rpt_corr_fail	30	R	0	Flag indicates that data error detected in flash block address #err_rpt_uncorr_blk_addr is not correctable. This field is self-cleared when a new DMA operation begins.
		ecc_rpt_corr_fail_ov	29	R	0	Flag indicates that uncorrectable error is detected in more than one SPI NAND block. This field is self-cleared when a new DMA operation begins.
		Reserved	28:24	-	-	Reserved
		ecc_rpt_max_err_num	23:20	R	0x0	The max error bits (in single BCH block) corrected by BCH decoder during one DMA operation. This field is self-cleared when a new DMA operation begins.
		Reserved	19:17	-	-	Reserved
		ecc_rpt_blk_addr	16:0	R	0x0	The first flash block address that contains corrected error data with corrected bit count = ecc_rpt_max_err_num. Valid only when ecc_rpt_max_err_num is not zero. The controller assumes each block contains 64 pages. For the SPI NAND flash with 128 pages per block, the software should regard ecc_rpt_blk_addr[16:1] as the block address instead.
0xa8	ecc_rpt_sts2	Reserved	31:17	-	-	Reserved
		ecc_rpt_uncorr_blk_addr	16:0	R	0x0	The first flash block address that contains uncorrectable error data. Valid only when ecc_rpt_err_det=1 and ecc_rpt_corr_fail=1. The controller assumes each block contains 64 pages. For the SPI NAND flash with 128 pages per block, the software should regard ecc_rpt_blk_addr[16:1] as the block address.

Table 13-2. Flash IO Registers.

For Mokra Lab 1 (C) 2012 2023 Confidential

13.3 Flash: SPI NAND Flash Controller

This section includes information regarding address space, programming, and registers for the SPI NAND flash controller as follows:

- (Section 13.3.1) SPI NAND Flash: Overview
- (Section 13.3.2) SPI NAND Flash: Register Detail
- (Section 13.3.3) SPI NAND Flash: Interrupts
- (Section 13.3.4) SPI NAND Flash: Initialization

- For flash I/O subsystem registers and programming, see Section 13.2.

All SPI NAND flash registers are mapped to the Arm processor's memory space via the AHB bus. The FDMA engine and flash controller share the same AHB base address, while the 8K FIFO uses a different base address.

Confidential For Mera Lang only (08012025)

13.3.1 SPI NAND Flash: Overview

The CV22FS SPI NAND flash controller is compatible with SPI NAND flash chips of various types and from a wide range of manufacturers. Features of the CV22FS flash controller include:

- Support for SPI NAND flash memory systems between 8 MB and 16 GB in size
- SPI-compatible serial bus interface
- 2048-byte or 4096-byte page size
- Bose-Chaudhuri-Hocquenghem (BCH) error correction (6-bit or 8-bit)
- Configurable spare-area usage
  - Option for 2x spare area size
  - 2-KB-page type with 64/128-byte spare area
  - 4-KB-page type with 128/256-byte spare area
- Custom command support
- Interrupt or Arm core polling for flash command completion
- Optional burst mode
- Software-programmable timing parameters to accommodate a wide range of flash speed grades and system clock frequencies.

The Arm core initiates flash chip operations by writing commands to the flash controller. When a flash chip operation completes, the flash controller optionally signals a maskable interrupt to the Arm core. The Arm core may also poll flash controller registers to determine when a flash chip operation has completed. Refer to Section 13.3.3 "SPI NAND Flash: Interrupts" for further information on flash interrupts. Refer to Section 13.3.4 "SPI NAND Flash: Initialization" for details on SPI NAND flash controller initialization.

The SPI NAND flash controller provides registers to configure the main and spare area used by the DMA FIFO controller. Included is an additional option to double the spare area available to FDMA. The 2x spare-area feature is enabled using the **NAND_extended_control** register **flash_ctl_sp_2x** bit. Refer to Section 13.2.3 “Flash I/O: BCH Error Correction and Dual-Space Mode” in this chapter for details on flash I/O dual-space mode and its use with BCH error correction.

The SPI NAND flash controller includes an optional burst mode setting. Burst mode can be enabled for the spare area by configuring **NAND_control** register spare-area address (**sa**) bits.

Note

To burst access the main area and spare area together at one go, use dual-space mode.

13.3.1.1 NAND Flash: Custom Command for SPI NAND

When **spi_nd**=1, the custom command is composed of five phases, which are command phase, dummy-addr phase, address phase, dummy-data phase, and data phase. Unlike the custom command for parallel NAND, the command phase is mandatory, while the other phases are optional.

The ordering of the five phases is fixed as follows:

command phase → dummy-addr phase → address phase → dummy-data phase → data phase

To issue a custom command, software must prepare the proper command word(s) to **cc_cmd1_val** register. For WE-type command, the software also need to prepare data to be written in either **cc_data** register(s) or memory depending on **cc_data_src** bit in the **snd_cc1** register. For program-type SPI NAND commands, users may need to set **auto_we** to issue write-enable command (0x06) first before the custom command is sent. For SPI NAND commands which need to check status after the custom command is issued, users may need to set **auto_stschk** to execute read-status command (0x0F) recursively until the desired pattern is obtained. The custom command is executed immediately after we write to **snd_cc1** register.

For certain operations that require two custom commands, such as READ or PROGRAM, the **snd_cc2** register can be enabled by setting **snd_cc2_en** as 1. When set, the SPI NAND controller will issue the **snd_cc2** command right after the **snd_cc1** command is done.

The following table shows the currently supported SPI-NAND commands.

Command	OP Code	Dummy Bytes	Address Bytes	Dummy Bytes	Data Bytes
Reset	FFh	0	0	0	0
Get feature	0Fh	0	1	0	1
Set feature	1Fh	0	1	0	1
Read ID (micron)	9Fh	0	0	1	2
Read ID (GigaDevice)	9Fh	0	0	0	3
Read page	13h	0	3	0	0
Read page cache random	30h	0	3	0	0
Read page cache last	3Fh	0	0	0	0
Read from cache x1	03h/0Bh	0	2	1	N
Read from cache x1 (GigaDevice)	03h/0Bh	1	2	1	N
Read from cache x2	3Bh	0	2	1	N
Read from cache x2 (gigadevice)	3Bh	1	2	1	N

Command	OP Code	Dummy Bytes	Address Bytes	Dummy Bytes	Data Bytes
Read from cache x4	6Bh	0	2	1	N
Read from cache x4 (GigaDevice)	6Bh	1	2	1	N
Read from cache dual IO	BBh	0	2	1	N
Read from cache quad IO	EBh	0	2	2	N
Read from cache quad IO (GigaDevice)	EBh	0	2	1	N
Write enable	06h	0	0	0	0
Write disable	04h	0	0	0	0
Block erase	D8h	0	3	0	0
Program execute	10h	0	3	0	0
Program load x1	02h	0	2	0	N
Program load x4	32h	0	2	0	N
Program load random x1	84h	0	2	0	N
Program load random x4	34h	0	2	0	N

Table 13-3. Supported SPI-NAND Commands.

Note that the SPI-NAND-related registers can be found in Section 13.3.2 (offset 0x1a0 to 0x1CC) on the following pages.

Confidential
For Mekra Lang only
1.5 (08012025)

13.3.2 SPI NAND Flash: Register Detail

The flash controller contains registers that hold SPI NAND flash system configuration information, SPI NAND flash control information, SPI NAND flash timing parameters, SPI NAND flash chip status, SPI NAND flash chip ID, SPI NAND flash controller commands, and the SPI NAND flash interrupt status.

Offset	Register	Field	Bits	Access	Reset	Description
0x120	nd_ctrl	Reserved	31:30	-	-	Reserved
		nd_a33_32	29:28	RW	0x0	Address[33:32]. These address bits are concatenated with the 32 bit address of each command to form the 34 bit address that is used in systems with more than 4 Gbytes of flash memory [1:0] are address[33:32] of the 34 bit address.
		nd_sp_a	27	RW	0x0	Spare area address. Writing this bit to “1” indicates that the address is a spare area address. The column portion of the address refers to a location within the spare area. When this bit is “0” the address is a main area address and the column address is a location in the main area. the software should set this bit only if it wants to access spare-area only.
		Reserved	26:25	-	-	Reserved
		nd_3row	24	RW	0x1	Page address in 3 cycles. Writing this bit to "1" indicates that three page address cycles are required. When this bit is "0", only two page address cycles are needed.
		nd_rid4	23	RW	0x0	ID register read in 4 cycles. Register nd_rid4 and nd_rid5 define how many RE# pulse will be issued in READ-ID command.
		Reserved	22:11	-	-	Reserved
		nd_was	10	RW	0x0	Write with automatic status update. When enabled, the status register will contain a OR’ed result of all SPI NAND flash transactions. It is used to detect errors that occur at a long transaction sequence with many write commands.
		nd_wp	9	RW	0x1	Write protect. If wp==0, write protecting is disabled and the controller is allowed write data into SPI NAND flash. If wp==1, write protecting is enabled and the controller cannot write any data into SPI NAND flash.
		Reserved	8:7	-	-	Reserved
		nd_dev_sz2_0	6:4	RW	0x0	Flash chip size. This field indicates the size of the flash chip. One more bit called “nd_dev_sz3” in nd_ext_ctrl register is defined to support larger flash chip size.
		Reserved	3:0	-	-	Reserved

Confidential For Ambarella Internal Use Only 15 Oct 2021 20:53:08

Offset	Register	Field	Bits	Access	Reset	Description
0x124	**nd_cmd**	**nd_a31_4**	31:4	RW	x	Address[31:4]. Address[31:4] for the active command when the active command is Read, Read Status, Read ID, or Program. This field is not used by the other commands. The information below shows the command specific address information in this field.
		nd_cmd	3:0	RW	0x0	Command. Indicates the flash chip operation in progress. The information below shows the SPI NAND operation to be executed for each command. This field is for backward-compatible to legacy nand controller. To execute commands not listed in the table, please issue custom commands: *Range:"{2,10,12,14,15}" Cmd — SPI NAND operation 0x2 — RESET 0xA — READ ID 0xC — READ STAT 0xE — READ 0xF — PROGRAM Others — Reserved
0x128	**nd_tim0**	**nd_tcls_cnum**	31:24	RW	0x20	tCLS cycle count. nd_tcls_cnum (command setup to write pulse rising edge time) is expressed as a number of nand_clk clock cycles minus 1. For example, setting this field as 0 means setting tCLS as 1 cycle time of nand_clk.
		nd_tals_cnum	23:16	RW	0x20	tALS cycle count. nd_tals_cnum (address setup to write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tALS as 1 cycle time of nand_clk.
		nd_tcs_cnum	15:8	RW	0x20	tCS cycle count. nd_tcs_cnum (chip enable setup to write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCS as 1 cycle time of nand_clk.
		nd_tds_cnum	7:0	RW	0x20	tDS cycle count. nd_tds_cnum (data setup to write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tDS as 1 cycle time of nand_clk.

Confidential For Mekra Lang GmbH 1.5 (08/12/2025)

Offset	Register	Field	Bits	Access	Reset	Description
0x12c	**nd_tim1**	**nd_tclh_cnum**	31:24	RW	0x20	tCLH cycle count. nd_tclh_cnum (command hold from write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCLH as 1 cycle time of nand_clk.
		nd_talh_cnum	23:16	RW	0x20	tALH cycle count. nd_talh_cnum (address hold from write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCS as 1 cycle time of nand_clk.
		nd_tch_cnum	15:8	RW	0x20	tCH cycle count. nd_tch_cnum (chip enable hold from write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCH as 1 cycle time of nand_clk.
		nd_tdh_cnum	7:0	RW	0x20	tDH cycle count. nd_tdh_cnum (data hold from write pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tDH as 1 cycle time of nand_clk.
0x130	**nd_tim2**	**nd_twp_cnum**	31:24	RW	0x20	tWP cycle count. nd_twp_cnum (write pulse width) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWP as 1 cycle time of nand_clk.
		nd_twh_cnum	23:16	RW	0x20	tWH cycle count. nd_twh_cnum (write high hold time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWH as 1 cycle time of nand_clk.
		nd_twb_cnum	15:8	RW	0x60	tWB cycle count. nd_twb_cnum (write pulse rising edge to busy time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWB as 1 cycle time of nand_clk.
		nd_trr_cnum	7:0	RW	0x20	tRR cycle count. nd_trr_cnum (ready setup to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRR as 1 cycle time of nand_clk.

For Mekra Lang Confidential 1.5 (080120250)

Offset	Register	Field	Bits	Access	Reset	Description
0x134	**nd_tim3**	**nd_trp_cnum**	31:24	RW	0x20	tRP cycle count. nd_trp_cnum (read pulse width) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRP as 1 cycle time of nand_clk.
		nd_treh_cnum	23:16	RW	0x20	tREH cycle count. nd_treh_cnum (read high hold time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tREH as 1 cycle time of nand_clk.
		nd_trb_cnum	15:8	RW	0x20	tRB cycle count. nd_trb_cnum (read pulse rising edge to busy time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRB as 1 cycle time of nand_clk.
		nd_tceh_cnum	7:0	RW	0x20	tCEH cycle count. nd_tceh_cnum (chip enable hold from read pulse rising edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCEH as 1 cycle time of nand_clk.
0x138	**nd_tim4**	**nd_trdelay_cnum**	31:24	RW	0x20	tRDELAY cycle count. nd_trdelay_cnum (read pulse falling edge to data valid time including all C to Q delays, transit time delays, buffer delays, etc) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRDELAY as 1 cycle time of nand_clk. nd_trdelay_cnum must be the value between trp and min((nd_trp_cnum+nd_treh_cnum+1), nd_trhz_cnum).
		nd_tclr_cnum	23:16	RW	0x20	tCLR cycle count. nd_tclr_cnum (command setup to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCLR as 1 cycle time of nand_clk.
		nd_twhr_cnum	15:8	RW	0x40	tWHR cycle count. nd_twhr_cnum (write pulse rising edge to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWHR as 1 cycle time of nand_clk.
		nd_tir_cnum	7:0	RW	0x20	tIR cycle count. nd_tir_cnum (flash chip output high Z to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tIR as 1 cycle time of nand_clk.

Confidential
For Mekra Lang
1 [illegible] 08012023

Offset	Register	Field	Bits	Access	Reset	Description
0x13c	nd_tim5	Reserved	31:24	R	-	Reserved
		nd_tww_cnum	23:16	RW	0x20	tWW cycle count. nd_tww_cnum (ready to write pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWW as 1 cycle time of nand_clk.
		nd_trhz_cnum	15:8	RW	0x60	tRHZ cycle count. nd_trhz_cnum (read pulse rising edge to flash chip output high Z time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRHZ as 1 cycle time of nand_clk.
		nd_tar_cnum	7:0	RW	0x20	tAR cycle count. nd_tar_cnum (address setup to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tAR as 1 cycle time of nand_clk.
0x140	nd_stat	Reserved	31:8	-	-	Reserved
		flash_status	7:0	RW	0	Status from the flash chip connected to the least significant flash chip on the flash controller data bus.
0x144	nd_id	flash_id	31:0	R	0x0	Flash ID obtained by READ_ID command. In general, Maker (flash chip manufacturer code) is at bit[31:24], and Device code (flash chip device code) is at bit[23:16].
0x148	nd_cp_addr	nd_cp_addr	31:0	RW	0x0	Address. Either used for COPY command to define as the destination address for SPI NAND flash,, or used as address for custom command (in the case of cc_addr_src=0 or 3).
0x15c	nd_ext_ctrl	Reserved	31:24	R	-	Reserved
		nd_4k_pg	25	RW	POC	1: SPI NAND flash page size is 4K-byte. 0: SPI NAND flash page size is 2K-byte. The reset value is inversion of POC[18] in SPI NAND boot mode, or 1 in non-SPI NAND boot mode.
		nd_dev_sz3	24	RW	0x0	nd_dev_sz[3]
		nd_rid5	23	RW	0x0	ID register read in 5 cycles. Writing this bit to “1” indicates that five read cycles are required to read the flash chip ID register. Two or four read cycles are needed to read the flash chip ID register when this bit is “0”.
		Reserved	22:1	R	-	Reserved
		nd_sp_2x	0	RW	POC	2 times spare area enable. 0: Spare area is 16 bytes per 512-byte main area. i.e. 64 bytes for 2 K page, or 128 bytes for 4 K page. 1: Spare area is 32 bytes per 512-byte main area. i.e. 128 bytes for 2 K page, or 256 bytes for 4 K page.

For Nextra Lab Confidential 2023-08-01 15:50

Offset	Register	Field	Bits	Access	Reset	Description
0x15c (Cont'd)						The reset value is POC[15] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
0x160	**nd_ext_id**	**Reserved**	31:8	R	-	Reserved
		flash_ext_id	7:0	R	0x0	byte-4 of ID register
0x164	**nd_tim6**	**Reserved**	31:24	R	-	Reserved
		nd_trhw_cnum	23:16	RW	0x60	tRHW cycle count. nd_trhw_cnum (from read pulse rising edge to the write pulse falling edge) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tRHW as 1 cycle time of nand_clk.
		nd_tadl_cnum	15:8	RW	0x60	tADL cycle count. nd_tadl_cnum (from write pulse rising edge of the last address cycle to the write pulse rising edge of the first data cycle) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tADL as 1 cycle time of nand_clk.
		nd_tcrl_cnum	7:0	RW	0x8	tCRL cycle count. nd_tcrl_cnum (chip enable falling to read pulse falling edge time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCRL as 1 cycle time of nand_clk. In general, nd_tcrl_cnum should be set as max(0, (nd_tcea_cnum – nd_trea_cnum)).
0x170	**nd_cc**	**cc_termi-nate_ce**	31	RW	0x0	0x0: CE pin will keep low (active) after the custom command done. 0x1: CE pin will be high (inactive) after the custom command done.
		Reserved	30:23	R	-	Reserved
		cc_data_src	22	RW	0x0	0: Source of the data in data phase is from/to nd_cc_dat* registers. 1: Dource of the data in data phase is from/to DMA engine.
		Reserved	21:20	R	-	Reserved
		cc_addr2_ni	19	RW	0x0	The definition of this bit is the same as cc_addr1_ni but is applied to the 2nd address phase of two-plane access.
		cc_addr1_ni	18	RW	0x0	0: Increment address for multi-page access. 1: Don't increment address for multi-page access.

For Mekra Lang Confidential 1.5 (08/12/2023)

Offset	Register	Field	Bits	Access	Reset	Description
0x170 (Cont'd)	**nd_cc**	**cc_addr_src**	17:16	RW	0x0	0x0: Address specified in {nd_addr_h[7:0], nd_cp_addr[31:0]} register is issued during address phase. 0x1: Row address specified in {6'h0, nd_a33_32, nd_a31_4, 4'h0} register is issued during address phase. 0x2: Address specified in {6'h0, nd_a33_32, nd_a31_4, 4'h0} register is interpreted as row and column address and is then issued during address phase. The bit number of column address depends on page-size setting. 0x3: This setting is used for two-plane-like access, such as two-plane ERASE, two-plane PROGRAM, or custom COPY command. Address specified in {6'h0, nd_a33_32, nd_a31_4, 4'h0} register is interpreted as row and column address and is then used as address of the first plane. Address specified in {nd_addr_h[7:0], nd_cp_addr[31:0]} is also interpreted as row and column address and is then used as address of the second plane.
		cc_cmd1	15:14	RW	0x0	0x0: There is no command phase-1 before address phase. 0x1: cc_cmd1_val0 will be present in command phase-1. 0x2: cc_cmd1_val0 and then cc_cmd1_val1 will be present in command phase-1. 0x3: Reserved. This field must be 1 if cc_addr_src=3. *Range:"{[0:2]}"
		cc_addr_cycle	13:11	RW	0x0	0x0: There is no address phase in the custom command. 0x1~0x5: number of address cycle during address phase. 0x6~0x7: Reserved. *Range:"{[0:5]}"
		cc_cmd2	10:9	RW	0x0	0x0: There is no command phase-2 in the custom command. 0x1: If cc_rw=0x1, cc_cmd2_val0 is present after data phase. Otherwise, cc_cmd2_val0 is present after address phase. 0x2: cc_cmd2_val0 is followed by cc_cmd2_val1. 0x3: Reserved. This field must be either 1 or 0 if cc_addr_src=3. *Range:"{[0:2]}"
		cc_rw	8:7	RW	0x0	0x0: There is no data phase in the custom command. 0x1: The custom command is WE type. 0x2: The custom command is RE type. 0x3: Reserved. *Range:"{[0:2]}"

Confidential For Mekra Lang 15 (0801202)

Offset	Register	Field	Bits	Access	Reset	Description
0x170 (Cont'd)	**nd_cc**	**cc_wait**	6:5	RW	0x0	When cc_rw=0x1, the wait phase is after data phase. Otherwise, the wait phase is after command phase-2. 0x0: There is no wait cycles in the custom command. 0x1: The flash controller will wait for R/B during wait phase. 0x2: The flash controller will wait for tWHR during wait phase. 0x3: Reserved. *Range:"{[0:2]}"
		cc_data_cycle	4:0	RW	0x0	Number of RE/WE pulse in non-DMA mode (cc_data_src=0). Valid only if cc_rw is not zero. 0x0: 1 pulse. 0x1: 2 pulses. … 0x1F: 32 pulses.
0x174	**nd_cc_word**	**cc_cmd2_val1**	31:24	RW	0x0	The 2nd command byte in command phase-2 of the custom command. It's also used as the command byte in command phase-2 of the second plane if cc_addr_src=3
		cc_cmd2_val0	23:16	RW	0x0	The 1st command byte in command phase-2 of the custom command.
		cc_cmd1_val1	15:8	RW	0x0	The 2nd command byte in command phase-1 of the custom command. It's also used as the command byte in command phase-1 of the second plane if cc_addr_src=3
		cc_cmd1_val0	7:0	RW	0x0	The 1st command byte in command phase-1 of the custom command.
0x17c	**nd_cp_addr_h**	**Reserved**	31:8	R	-	Reserved
		nd_cp_addr_h	7:0	RW	0x0	The 5th byte of SPI NAND CP address. Used for address phase of custom command in the case of cc_addr_src=0 or 3.
0x180	**nd_cc_dat0**	**cc_dat0**	31:0	RW	0x0	The 1st data of custom command. If the custom command is RE type, this field is used to store the read data. If the custom command is WE type, this field is used to prepare write data.
0x184	**nd_cc_dat1**	**cc_dat1**	31:0	RW	0x0	The 2nd data of custom command.
0x188	**nd_cc_dat2**	**cc_dat2**	31:0	RW	0x0	The 3rd data of custom command.
0x18c	**nd_cc_dat3**	**cc_dat3**	31:0	RW	0x0	The 4th data of custom command.
0x190	**nd_cc_dat4**	**cc_dat4**	31:0	RW	0x0	The 5th data of custom command.
0x194	**nd_cc_dat5**	**cc_dat5**	31:0	RW	0x0	The 6th data of custom command.
0x198	**nd_cc_dat6**	**cc_dat6**	31:0	RW	0x0	The 7th data of custom command.
0x19c	**nd_cc_dat7**	**cc_dat7**	31:0	RW	0x0	The 8th data of custom command.

Confidential
For Meiya Lan 080 1.5 2023

Offset	Register	Field	Bits	Access	Reset	Description
0x1a0	**snd_ctrl**	**sck_mode**	31	RW	POC	0: SCK remains at 0 for CPOL=0, CPHA=0 (Mode 0) 1: SCK remains at 1 for CPOL=1, CPHA=1 (Mode 3) The reset value is POC[19] in SPI NAND boot mode, or 0 in non-SPI NAND boot mode.
		flow_ctrl	30	RW	0x0	0: Stop clock to halt data transmission. 1: Driving low on SO3(Hold#) to halt data transmission. This is applied to SPI NAND-read operations only.
		ps_sel	29:28	RW	0x0	Plane select. To choose which bit of row address should be the plane select bit in column address phase. This feature takes effect only if cc_addr_src is set to 2 or 3. 0: No plane select bit in column address phase 1: Choose bit-5 of row address as plane select bit 2: Choose bit-6 of row address as plane select bit 3: Choose bit-7 of row address as plane select bit
		Reserved	27:18	R	-	Reserved
		max_cmd_loop	17:0	RW	0x3ffff	Valid only if auto_stschk=1. 0x0: Infinite loop count 0x1 ~ 0x3FFFF: 1~262143 times retry. (up to 20.97ms @ 100MHz SCK)
0x1a4	**snd_cc1**	**auto_we**	31	RW	0x0	0x0: Do nothing before snd_cc command is issue. 0x1: Before snd_cc command is issued, execute write-enable command (0x06) first.
		auto_stschk	30	RW	0x0	0x0: Do nothing after snd_cc command is issued. 0x1: After snd_cc command is issued. execute read status command (0x0F) recursively and until (read data & done_patn_mask) matches done_patn, or (read data & err_patn_mask & err_patn) is non-zero, or issued command count reaches max_loop_num.
		Reserved	29:23	R	-	Reserved
		cc_data_src	22	RW	0x0	0: Source of the data in data phase is from/to nd_cc_dat* registers. 1: Source of the data in data phase is from/to DMA engine.
		cc_dmy_addr_num	21:19	RW	0x0	Dummy byte length in dummy-addr phase. 0x0: There is no dummy-address phase. 0x1~0x7: 1~7 dummy bytes.
		cc_dmy_data_num	18:16	RW	0x0	Dummy byte length in dummy-data phase. 0x0: There is no dummy-data phase. 0x1~0x7: 1~7 dummy bytes.

For Meera Lai 1 Dec 08 2023 Confidential

Offset	Register	Field	Bits	Access	Reset	Description
0x1a4 (Cont'd)	**snd_cc1**	**lane_type**	15:14	RW	0x0	Data in the following phase(s) is distributed to lanes specified by lane_num. 0: Data phase only. 1: Dummy-data phase and data phase. 2: Dummy-addr phase, address phase, dummy-data phase and data phase. 3: All phases.
		cc_addr_cycle	13:11	RW	0x0	0x0: There is no address phase in the custom command. 0x1~0x5: Number of address cycle during address phase. 0x6~0x7: Reserved. *Range:"{[0:5]}"
		cc_addr_src	10:9	RW	0x0	The definition of this field is the same as nd_cc.cc_addr_src register.
		cc_rw	8:7	RW	0x0	0x0: There is no data phase in the custom command. 0x1: The custom command is WE type. 0x2: The custom command is RE type. 0x3: Reserved. *Range:"{[0:2]}"
		lane_num	6:5	RW	0x0	0: 1 data lane. 1: 2 data lanes. 2: 4 data lanes. 3: Reserved. *Range:"{[0:2]}"
		cc_data_cycle	4:0	RW	0x0	Number of RE/WE pulse in non-DMA mode (cc_data_src=0). Valid only if cc_rw is not zero. 0x0: 1 pulse. 0x1: 2 pulses. … 0x1F: 32 pulses.
0x1a8	**snd_cc2**	**Reserved**	31	R	-	Reserved
		auto_stschk	30	RW	0x0	0x0: Do nothing after snd_cc command is issued. 0x1: After snd_cc command is issued. execute read status command (0x0F) recursively and until (read data & done_patn_mask) matches done_patn, or (read data & err_patn_mask & err_patn) is non-zero, or issued command count reaches max_loop_num.
		Reserved	29:27	R	-	Reserved
		sndcc2_en	26	RW	0x0	Setting this field 1 indicates snd_cc2 is valid and should be issued once snd_cc1 command is done.
		Reserved	25:23	R	-	Reserved
		cc_data_src	22	RW	0x0	0: Source of the data in data phase is from/to nd_cc_dat* registers. 1: Source of the data in data phase is from/to DMA engine.

Confidential
For Mekra Lang Only
1.5 (08/01/2020)

Offset	Register	Field	Bits	Access	Reset	Description
0x1a8 (cont)	snd_cc2	cc_dmy_addr_num	21:19	RW	0x0	Dummy byte length in dummy-addr phase. 0x0: There is no dummy-address phase. 0x1~0x7: 1~7 dummy bytes.
		cc_dmy_data_num	18:16	RW	0x0	Dummy byte length in dummy-data phase. 0x0: There is no dummy-data phase. 0x1~0x7: 1~7 dummy bytes.
		lane_type	15:14	RW	0x0	Data in the following phase(s) is distributed to lanes specified by lane_num. 0: Data phase only. 1: Dummy-data phase and data phase. 2: Dummy-addr phase, address phase, dummy-data phase and data phase. 3: All phases.
		cc_addr_cycle	13:11	RW	0x0	0x0: There is no address phase in the custom command. 0x1~0x5: number of address cycle during address phase. 0x6~0x7: reserved. *Range:“{[0:5]}”
		cc_addr_src	10:9	RW	0x0	The definition of this field is the same as nd_cc.cc_addr_src register.
		cc_rw	8:7	RW	0x0	0x0: There is no data phase in the custom command. 0x1: The custom command is WE type. 0x2: The custom command is RE type. 0x3: Reserved. *Range:“{[0:2]}”
		lane_num	6:5	RW	0x0	0: 1 data lane. 1: 2 data lanes. 2: 4 data lanes. 3: reserved. *Range:“{[0:2]}”
		cc_data_cycle	4:0	RW	0x0	Number of data byte in non-DMA mode (cc_data_src=0). Valid only if cc_rw is not zero. 0x0: 1 data byte. 0x1: 2 data bytes. … 0x1F: 32 data bytes.
0x1ac	done_patn	Reserved	31:24	R	-	Reserved
		done_patn_en	23:16	RW	0x1	Done pattern enable. For the bit(s) setting to 1, the corresponding bit(s) in done_patn will be compared while doing pattern-matching check.
		Reserved	15:8	R	-	Reserved
		done_patn	7:0	RW	0x0	Done pattern. Default value is 0x0 since bit-0 is OIP bit of status register.

For Mekra Lang (08/12/2025)
Confidential

Offset	Register	Field	Bits	Access	Reset	Description
0x1b0	**err_patn**	**Reserved**	31:24	R	-	Reserved
		err_patn_en	23:16	RW	0x20	Error pattern enable. For the bit(s) setting to 1, the corresponding bit(s) in err_patn will be compared while doing pattern-matching check.
		Reserved	15:8	R	-	Reserved
		err_patn	7:0	RW	0x20	Error pattern. Default value is 0x20 since bit-5 is uncorrectable ECC bit. Bit-3 is Program-Fail bit, and bit-2 is Erase-Fail bit. Ambarella recommends setting this field as 0x2C when the boot procedure is done.
0x1C0	**snd_tim0**	**snd_tclh_cnum**	31:24	RW	0x14	tCLH cycle count. snd_tclh_cnum (clock HIGH time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCLH as 1 cycle time of nand_clk.
		snd_tcll_cnum	23:16	RW	0x14	tCLL cycle count. snd_tcll_cnum (clock LOW time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCLL as 1 cycle time of nand_clk.
		snd_tcs_cnum	15:8	RW	0x36	tCS cycle count. snd_tcs_cnum (Command deselect time) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCS as 1 cycle time of nand_clk.
		snd_tclqv_cnum	7:0	RW	0x14	tCLQV cycle count. snd_tclqv_cnum (Clock LOW to read-data valid) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCLQV as 1 cycle time of nand_clk. Also see notes in Section 13.3.4 “SPI NAND Flash: Initialization” for the recommended setting.
0x1C4	**snd_tim1**	**snd_tchsl_cnum**	31:24	RW	0x3	tCHSL cycle count. snd_tchsl_cnum (from rising edge of SCK to falling edge of CS#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCHSL as 1 cycle time of nand_clk.
		snd_tslch_cnum	23:16	RW	0x3	tSLCH cycle count. snd_tslch_cnum (from falling edge of CS# to rising edge of SCK) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tSLCH as 1 cycle time of nand_clk.

For Mekra Lang GmbH & Co. KG 15.08.2023 Confidential

Offset	Register	Field	Bits	Access	Reset	Description
0x1C4 (Cont'd)		**snd_tchsh_cnum**	15:8	RW	0x3	tCHSH cycle count. snd_tchsh_cnum (from rising edge of SCK to rising edge of CS#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCHSH as 1 cycle time of nand_clk.
		snd_tshch_cnum	7:0	RW	0x3	tSHCH cycle count. snd_tshch_cnum (from rising edge of CS# to rising edge of SCK) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tSHCH as 1 cycle time of nand_clk.
0x1C8	**snd_tim2**	**Reserved**	31:24	R	-	Reserved
		snd_thhqx_cnum	23:16	RW	0x8	tHHQX cycle count. snd_thhqx_cnum (from rising edge of HOLD# to read-data valid) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tHHQX as 1 cycle time of nand_clk. Section 13.3.4 "SPI NAND Flash: Initialization"
		snd_twps_cnum	15:8	RW	0xb	tWPS cycle count. snd_twps_cnum (from rising edge of WP# to falling edge of CS#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWPS as 1 cycle time of nand_clk.
		snd_twph_cnum	7:0	RW	0x36	tWPH cycle count. snd_twph_cnum (from rising edge of CS# to falling edge of WP#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tWPH as 1 cycle time of nand_clk.
0x1CC	**snd_tim3**	**snd_tchhl_cnum**	31:24	RW	0x0	tCHHL cycle count. snd_tchhl_cnum (from rising edge of SCK to falling edge of HOLD#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCHHL as 1 cycle time of nand_clk.
		snd_tchhh_cnum	23:16	RW	0x0	tCHHH cycle count. snd_tchhh_cnum (from rising edge of SCK to rising edge of HOLD#) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tCHHH as 1 cycle time of nand_clk.
		snd_thlch_cnum	15:8	RW	0x0	tHLCH cycle count. snd_thlch_cnum (from falling edge of HOLD# to rising edge of SCK) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tHLCH as 1 cycle time of nand_clk.

Confidential For Mekra Lang 15 (08012023)

Offset	Register	Field	Bits	Access	Reset	Description
0x1CC (Cont'd)		**snd_thhch_ cnum**	7:0	RW	0x0	tHHCH cycle count. snd_thhch_cnum (from rising edge of HOLD# to rising edge of SCK) is expressed as a number of clock cycles minus 1. For example, setting this field as 0 means setting tHHCH as 1 cycle time of nand_clk.

Table 13-4. SPI NAND Flash Registers.

sz[3:0]	Flash Chip Size (Not Including Spare Area)	Flash Support Type (Page Size)
0	64 Mbits	2K
1	128 Mbits	2K
2	256 Mbits	2K
3	512 Mbits	2K
4	1 Gbits	4K/2K
5	2 Gbits	4K/2K
6	4 Gbits	4K/2K
7	8 Gbits	4K/2K
8	16 Gbits	4K/2K
9	32 Gbits	4K/2K
10	64 Gbits	4K/2K
11	128 Gbits	4K/2K

Table 13-5. Flash Size Configuration.

Confidential For Meka Lang only 1.5 (08012025)

13.3.3 SPI NAND Flash: Interrupts

When any of the following five events occur, the flash controller sets to 1 and also sets the interrupt status bit in a SPI NAND Interrupt register to 1.

- Command-done event
- SPI-NAND (get_feature) timeout event
- AXI bus error event
- ECC report event
- ECC uncorrectable error event

The flash controller signals an interrupt if the corresponding interrupt enable bit is 1. An interrupt bit remains active until the software has cleared it by writing 1 to the interrupt bit.

Note that the SPI NAND flash controller maintains interrupt lines directly to the generic interrupt controller (GIC).

13.3.4 SPI NAND Flash: Initialization

The SPI NAND flash controller registers must be initialized before flash chips can be accessed. The controller registers must be initialized after reset.

The initialization process involves entering the appropriate values in the flash controller configuration registers. Initialization depends on the mode; however, an example initialization process for SPI NAND is provided as follows:

- Before accessing the flash chip, initialize the following:
  - SPI NAND flash control register (**NAND_control**)
  - SPI NAND flash timer registers (**NAND_timing_int[n]**)
  - SPI NAND flash interrupt register (**NAND_interrupt**)

These registers can be initialized using two methods:

1. Hard-wired reset values can be loaded into these registers
2. The software can write to these registers when reset is deasserted

Notes:

- Hard-wired reset values represent worst-case values which will allow the controller to communicate with the external flash memory in which the system boot-code resides.
- After boot-up, the system software must program the timing values particular to the specific memory devices in use to ensure optimal I/O performance.
- To determine configuration values for these registers, refer to the relevant flash chip datasheet provided by the chip manufacturer.
- The timing parameter, snd_tclqv_cnum, is used to determine the read-data latch timing. Its setting depends on the AC characteristic of the external SPI NAND device; Ambarella recommends setting as follows
  - snd_tclqv_cnum = $ceil((tCLVQ + 7.97)*$_{fnand_pll}$ / 1000) - 1, where tCLQV stands for maximum output valid time of the external NAND device, and $_{fnand_pll}$ is the frequency of NAND PLL.

  The chosen snd_tclqv_cnum value shall not violate the following constraints; if it violates, consider adjusting the snd_tcll_cnum and / or snd_tclh_cnum settings.
  - (1) snd_tclqv_cnum <= ($floor((tHO + 4.03)*$_{fnand_pll}$ / 1000) + snd_tcll_cnum + snd_tclh_cnum + 1), where tHO stands for the minimum output hold time of the external NAND device
  - (2) snd_tclqv_cnum <= ((snd_tcll_cnum + snd_tclh_cnum + 2)*2.5 + snd_tcll_cnum)

  Additionally, the maximum frequency of NAND PLL ($_{fnand_pll}$) is 533 MHz for CV22FS.

  If the IO3 pin of the SPI NAND device is used for the HOLD# function, the recommended setting of snd_tclqv_cnum register is applied to the snd_thhqx_cnum register, as well.

For Mekalanguy Confidential 1.5 2012-07-20

13.4 Flash: FDMA Engine

13.4.1 FDMA Engine: DMA in Non-Descriptor Mode

Software should setup **nd_cmd** as READ/PROGRAM to enable the SPI NAND controller. To run custom commands in DMA mode, the software must ensure that the **cc_data_src** register is set to 0x1 so that the SPI NAND controller can fetch/store data (in data phase) through the FDMA engine.

Because the SPI NAND device address is derived from the source/destination address (for READ/PROGRAM, respectively) of FDMA operation, the software should setup FDMA registers before enabling the SPI NAND controller (by programming registers at offset 0x124 or 0x170 or 0x1a4 after enabling FDMA).

If the data (in the data phase) will not be encoded/decoded by the ECC engine, **bch_en** bit should be cleared when DMA engine is operated in non-descriptor mode.

13.4.2 FDMA Engine: DMA in Descriptor Mode for SPI NAND

When **spi_nd** is set to 0x1 (SPI NAND), the descriptor format is as shown below. Unlike legacy SPI NAND flash, SPI-NAND requires multiple commands to complete a program / read operation. To achieve this, each descriptor contains two commands, "**snd_cc1**" and "**snd_cc2**"

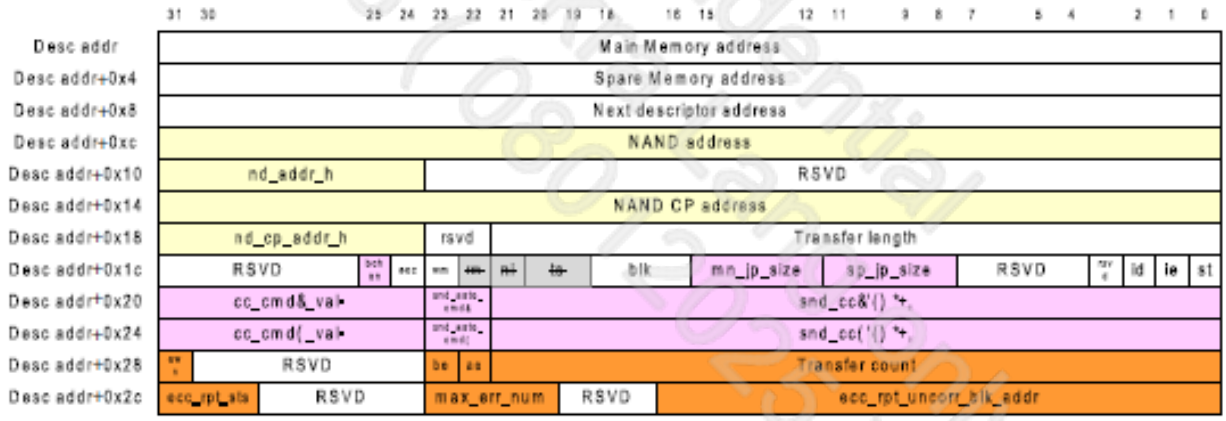


Figure 13-6. Descriptor Format for SPI NAND Chip.

- **snd_cc1**: The first SPI NAND custom command to be issued. The relationship of "**cc_rw**" and the data to be transferred is described below.
  - **nd_cc**: the content of this field will be written to **nd_cc** register, which is at register offset 0x170.
  - If there is no data phase in the custom command (i.e. **cc_rw**=0), there will be no data access through DMA engine. In such case, please ensure **transfer_count** is also set as 0.
  - If **cc_rw** is not 0, the number of data to be transferred will be determined by "Transfer count", "**mn_jp_size**", and "**sp_jp_size**" fields.
- **snd_cc2**: The second SPI NAND custom command to be issued. If the second command in the descriptor is not required, simply set this field to 0x0.
- **snd_auto_cmd1**: Two-bit register to be set in {**auto_we**, **auto_stschk**} register corresponding to "**snd_cc1**".

- **snd_auto_cmd2**: Two-bit register to be set in {**auto_we**, **auto_stschk**} register corresponding to "**snd_cc2**"; since **snd_cc2** doesn't support **auto_we**, please set it as 0 accordingly.

13.4.3 FDMA Engine: Register Detail

Offset	Register	Field	Bits	Access	Reset	Description
0x204	**fdma_sp_mem_addr**	**sp_mem_addr**	31:0	RW	0	Memory address of data of spare area. This field is valid only if spare_jp_sz is not zero.
0x300	**fdma_mn_ctrl**	**en**	31	RWSC	0	DMA channel enable. When set, DMA engine will start operations. Will self-clear when DMA operation is done.
		dm	30	RWSC	0	Descriptor mode Will self-clear when DMA operation is done.
		wm	29	RW	0	Write memory
		Reserved	28:27	-	-	Reserved
		blk	26:24	RW	0	The max burst length allowed for DRAM access. 0x2: 32 bytes 0x3: 64 bytes 0x4: 128 bytes 0x5: 256 bytes 0x6: 512 bytes 0x7: 1024 bytes Others: Reserved. *Range:"{[2:7]}"
		Reserved	23:22	-	-	Reserved
		xact_blen	21:0	RW	0	DMA transaction length (in byte). Number of bytes to be moved by the DMA operation. If dual-space mode is enabled, this field should be sum of data count in main area and spare area. *Range:"{[0:4194303]}"
0x304	**fdma_mn_mem_addr**	**mn_mem_addr**	31:0	RW	0	Main Memory address of the DMA request.
0x30c	**fdma_mn_sts**	**Reserved**	31	-	-	Reserved
		oe	30	R	0	Descriptor DMA operation error. When set, it means there is a bus error (be status) while processing the descriptor chain. Self-clear at the rising edge of fdma_mn_ctrl.
		da	29	R	0	Descriptor address error. When set, it means desc_addr register is not 8-byte aligned. Self-clear at the rising edge of fdma_mn_ctrl
		dd	28	R	0	Descriptor chain done. When set, it means all descriptors are processed. Self-clear at the rising edge of fdma_mn_ctrl.
		od	27	R	0	Descriptor operation done. When set, it means the first descriptor is done. Self-clear at the rising edge of fdma_mn_ctrl.

Confidential
For Mekra Lang GmbH
1.5 (Oct 2023)

Offset	Register	Field	Bits	Access	Reset	Description
0x30c (Cont'd)		**Reserved**	26	-	-	Reserved
		be	25	R	0	DMA bus error. When set, it means there is ECC decoding failure during DMA operation. Self-clear at the rising edge of fdma_mn_ctrl.
		Reserved	24	-	-	Reserved
		ae	23	R	0	DMA address error. When set, it means address of desc_addr register is not 8-byte aligned. Self-clear at the rising edge of fdma_mn_ctrl.
		dn	22	R	0	DMA operation done. Self-clear at the rising edge of fdma_mn_ctrl.
		xfer_count	21:0	R	0	DMA transferred byte count. Number of bytes moved by the DMA operation. When an error occurs, the DMA operation ends regardless of the number of bytes that have been transferred before the error. Self-clear at the rising edge of fdma_mn_ctrl.
0x380	**fdma_desc_addr**	**desc_addr**	31:0	RW	0	Byte address of the next descriptor that will be used to configure DMA Channel if it is in descriptor mode. Must be eight byte aligned.
0x3a0	**fdma_dsm_ctrl**	**Reserved**	31:8	-	-	Reserved
		main_jp_size	7:4	RW	POC	If dual space mode is enabled, FDMA engine will jump to transfer spare data when 2^main_jp_size bytes of main data is transferred. The legal values are 0, and any value larger than 3. Setting this field to 0 implies SPI NAND controller will read / write the data in a single command. *Range:"{0,[4:15]}" The default value depends on POC settings: poc_nand_ecc_bch / poc_nand_2k_pg / main_jp_size 1 / - / 0x9 0 / 0 / 0xC 0 / 1 / 0XB
		spare_jp_size	3:0	RW	POC	If dual space mode is enabled, FDMA engine will jump to transfer main data when 2^spare_jp_size bytes of spare data is transferred. The legal value are 0, 4, 5, 6, 7, and 8. Setting this field to 0 implies dual space mode is disabled.

For Ambarella Langley Only
Confidential
11 (08012025)

Offset	Register	Field	Bits	Access	Reset	Description
0x3a0 (Cont'd)						*Range:"{0, [4:8]}" The default value depends on POC settings: poc_nand_ecc_bch / poc_nand_2k_pg / poc_nand_sp_2x / spare_jp_size 1 / - / 0 / 0x4 1 / - / 1 / 0x5 0 / 0 / 0 / 0x7 0 / 0 / 1 / 0x8 0 / 1 / 0 / 0x6 0 / 1 / 1 / 0x7

Table 13-6. FDMA Register Detail.

13.5 Flash: Safety Notes

The SPI NAND controller relies on pure SW safety mechanism. For any fault detected by SW, SW should set the sw_arm_safety_error register in AXI Configuration so that bit-7 of safety errors in CEHU will be asserted.

For Confidential Mekra Lang only 1.5 (08012025)

14. SD CONTROLLERS

The CV22FS processor provides up to two SD controllers. This chapter includes the following sections:

- (Section 14.1) SD: SD PHY
- (Section 14.2) SD: SD Controllers
- (Section 14.3) SD: Safety Notes
-

Refer to Chapter 2 for SD eMMC boot mode details. For interface pins and descriptions, refer to the CV22FS chip datasheet.

14.1 SD: SD PHY

14.1.1 SD PHY: Register Map

The SD PHY blocks are controlled by the reset clock and test (RCT) debug unit.

SD PHY register descriptions are provided below.

Offset	Field	Bits	Access	Reset	Description
0x4C0	**Reserved**	31:28	-	-	Reserved
	sd_din_clk_pol	27	RW	0	Change clock polarity of clock latching incoming data / cmd 0: Do not inverse polarity 1: Inverse polarity
	sd_clkout_bypass	26	RW	1	Bypass **clk_sdcard_out** 1: Bypass **clk_sdcard_out** 0: Delay the phase of **clk_sdcard_out**
	sd_rst	25	RW	0	Active-high reset to DLL inside sd_phy
	Reserved	24:20	-	-	Reserved
	sd_dll_clk_pol	19	RW	0	Change clock polarity to DLL
	sd_data_cmd_by-pass	18	RW	1	Bypass SD command and data
	sd_dll_bypass	17	RW	1	Bypass DLL
	Reserved	16:0	-	-	Reserved
0x4C4	**sd_sel3**	31:24	-	-	Fine delay select for delay chain #3 of sd_phy
	sd_sel2	23:16	RW	0	Fine delay select for delay chain #2 of sd_phy
	Reserved	15:8	RW	0	
	sd_sel0	7:0	RW	0	Fine delay select for delay chain #0 of sd_phy

Confidential For Makrolab 2012-01-15 09:01:20

Offset	Field	Bits	Access	Reset	Description
0x4C8	**Reserved**	31:28	-	-	Reserved
	sdio0_din_clk_pol	27	RW	0	Change clock polarity of clock latching incoming data / cmd 0: Don't inverse polarity 1: Inverse polarity
	sdio0_clkout_bypass	26	RW	1	Bypass clk_sdcard_out 0: Bypass clk_sdcard_out 1: Delay the phase of clk_sdcard_out
	sdio0_rst	25	RW	0	Active-high reset to DLL inside sdio0_phy
	Reserved	24:20	-	-	
	sdio0_dll_clk_pol	19	RW	0	Change clk polarity to DLL
	sdio0_data_cmd_bypass	18	RW	1	Bypass sd cmd and data
	sdio0_dll_bypass	17	RW	1	Bypass DLL
	Reserved	16:0	-	-	Reserved
0x4CC	**sdio0_sel3**	31:24	RW	0	Fine delay select for delay chain #3 of sdio0_phy
	sdio0_sel2	23:16	RW	0	Fine delay select for delay chain #2 of sdio0_phy
	Reserved	15:8	RW	0	
	sdio0_sel0	7:0	RW	0	Fine delay select for delay chain #0 of sdio0_phy
0x4F0	**Reserved**	31	R	-	Reserved
	sdio0_debug_out	30	R	-	debug_out status of sdio0_phy
	sdio0_st_cur	29:24	R	-	st_cur status of sdio0_phy
	sdio0_sc_out	23:21	R	-	sc_out status of sdio0_phy
	sdio0_vshift	20:16	R	-	Vshift value of sdio0_phy
	Reserved	15	R	-	
	sd_debug_out	14	R	-	debug_out status of sd_phy
	sd_st_cur	13:8	R	-	st_cur status of sd_phy
	sd_sc_out	7:5	R	-	sc_out status of sd_phy
	sd_vshift	4:0	R	-	Vshift value of sd_phy
0x664	**Reserved**	31:21	-	-	
	sd_sbc	20:18	RW	0	Clock reference delay cancel
		17	RW	0	Enable boost mode
		16	RW	0	Enable automatic coarse adjustment
		15	RW	0	Lock, then Master stops moving
		14	RW	0	Force lock enable
		13	RW	0	Enable overflow lock
		12	RW	0	Select FSM
		11	RW	0	Bypass filter when asserted high.
		10:8	RW	0	Debug selection
		7:6	RW	0	Clock divider selection (2'b10 to divide by 8)
		5:4	-	-	Reserved
		3:1	RW	0	Adjust coarse delay
		0	RW	0	Enable DLL

For Ambarella Internal Use Only Confidential

Offset	Field	Bits	Access	Reset	Description
0x668	**Reserved**	31:21	-	-	Reserved
	sdio0_sbc	20:18	RW	0	Clock reference delay cancel
		17	RW	0	Enable boost mode
		16	RW	0	Enable automatic coarse adjustment
		15	RW	0	Lock, then Master stops moving
		14	RW	0	Force lock enable
		13	RW	0	Enable overflow lock
		12	RW	0	Select FSM
		11	RW	0	Bypass filter when asserted high
		10:8	RW	0	Debug selection
		7:6	RW	0	Clock divider selection (2'b10 to divide by 8)
		5:4	-	-	Reserved
		3:1	RW	0	Adjust coarse delay
		0	RW	0	Enable DLL

Table 14-1. SD PHY Registers.

For Mekra Lang only Confidential 1.5 (08012025)

14.1.2 SD PHY: Programming

14.1.2.1 SD PHY Programming: Bypass SD PHY Block

To bypass the entire SD PHY block, set the register fields as shown below.

- **sd_clkout_bypass** = 1
- **sd_dll_bypass** = 1
- **sd_data_cmd_bypass** = 1

14.1.2.2 SD PHY Programming: DLL

To use the DLL, following the steps below.

- Enable the SD clock by setting bit 2 of the Clock Control register (offset 0x2C) in the SD (host) controller (Section 14.2.3 “SD Controllers: Clocking”).
- Set **sbc[0]** = 1 to enable the DLL.
- Assert **sd_rst** high longer than 1 us for normal operation.
- When the DLL is locked, **Vshift[7:0]** will be a stable value.
- When **sel[5]** is 0, incrementing **sel[4:0]** by 1 will increase delay by one step.
- When **sel[5]** is 1, incrementing **sel[4:0]** by 1 will decrease delay by one step.

14.1.2.3 SD PHY Programming: Additional Notes

- Set **sd_data_cmd_bypass**=1 when the SD clock is off. This allows some PME signals to be bypassed.
- Set **sbc[15]** to 1 such that Vshift will be stable once the DLL is in a locked state.
- The SD PHY allows users to invert the clock polarity by setting **mx_clk_pol** = 1. Note that when this function is enabled with **sd_clkout_bypass**=0, the polarity of the outgoing clock will also be inverted. e.g., If the idle state of **clk_in** is 0, then the idle state of **clk_sdcard_out** will become 1.
- When the SD controller is running in DDR50 mode, please disable the SD PHY and use **sd_delay_mux** to fine-tune timing instead.
- Reference setting of sbc[20:6]
  - sbc[20:18] = 0
  - sbc[17] = 0
  - sbc[16] = 0
  - sbc[15] = 1
  - sbc[14] = 1
  - sbc[13] = 1
  - sbc[12] = 1
  - sbc[11] = 1
  - sbc[10:8] = 1
  - sbc[7:6] = 2

Confidential
For Mekra Lang only
1.5 (08012025)

14.2 SD: SD Controllers

This section provides information regarding the CV22FS SD controllers including:

- (Section 14.2.1) SD Controllers: Overview
- (Section 14.2.2) SD Controllers: Base Address Information
- (Section 14.2.3) SD Controllers: Clocking
- (Section 14.2.4) SD Controllers: SDMA and ADMA Operation
- (Section 14.2.5) SD Controllers: Register Map
- (Section 14.2.6) SD Controllers: Register Details
- (Section 14.2.7) SD Controllers: ADMA Block Diagram and Descriptor Programming

- Refer to the CV22FS chip datasheet for interface pin and power-rail details, including multiplexing information.
- Refer to Chapter 2 of this manual for eMMC power on configuration information.

14.2.1 SD Controllers: Overview

The CV22FS chip includes two SD controllers for SD / SDIO / SDHC / SDXC / MMC / eMMC operations. These controllers conform to the specifications outlined in "*SD Specifications Part 1 Physical Layer Specification Version 3.00* (*SD Spec*)," "*SDIO Specification Version 3.00*," and "*Embedded Multimedia Card (eMMC) Electrical Standard 4.41*." Additionally, CV22FS supports the feature of HS200 in eMMC Standard 4.5.1.

For information regarding specific SD controller features, please refer to the relevant CV22FS chip datasheet.

14.2.2 SD Controllers: Base Address Information

The SD controllers are located at the AHB base addresses shown below.

- SD0: E000.4000
- SDIO0: E000.5000

14.2.3 SD Controllers: Clocking

The CV22FS chip configures the SD0 controller clock (gclk_sd48) and the SDIO0 controller clock (gclk_sdxc) using a set of APIs. The register programming described below is used for adjustment and refinement. Contact an Ambarella representative for additional detail.

14.2.4 SD Controllers: SDMA and ADMA Operation

The CV22FS SD controllers maintain an independent bus mastering DMA engine and do not rely on the AXI DMA

Confidential For Mekrang only 1.5 (Oct 01 2025)

Engine for DMA transfers. The SD controllers support the single DMA (SDMA) transfers used with previous Ambarella chips and also include an advanced DMA (ADMA) algorithm for high transfer speeds.

14.2.4.1 SD Controllers: SDMA Operation

SDMA transfer operations involve the use of DMA Interrupts at every page boundary. These interrupts trigger the CPU to reprogram a new system address to the page. While this process is generally effective, it places a limitation on transfer speed.

The command flow for SDMA transfer operations is illustrated in the figure on the next page.

Confidential
For Mekra Lang only
1.5 (08012025)

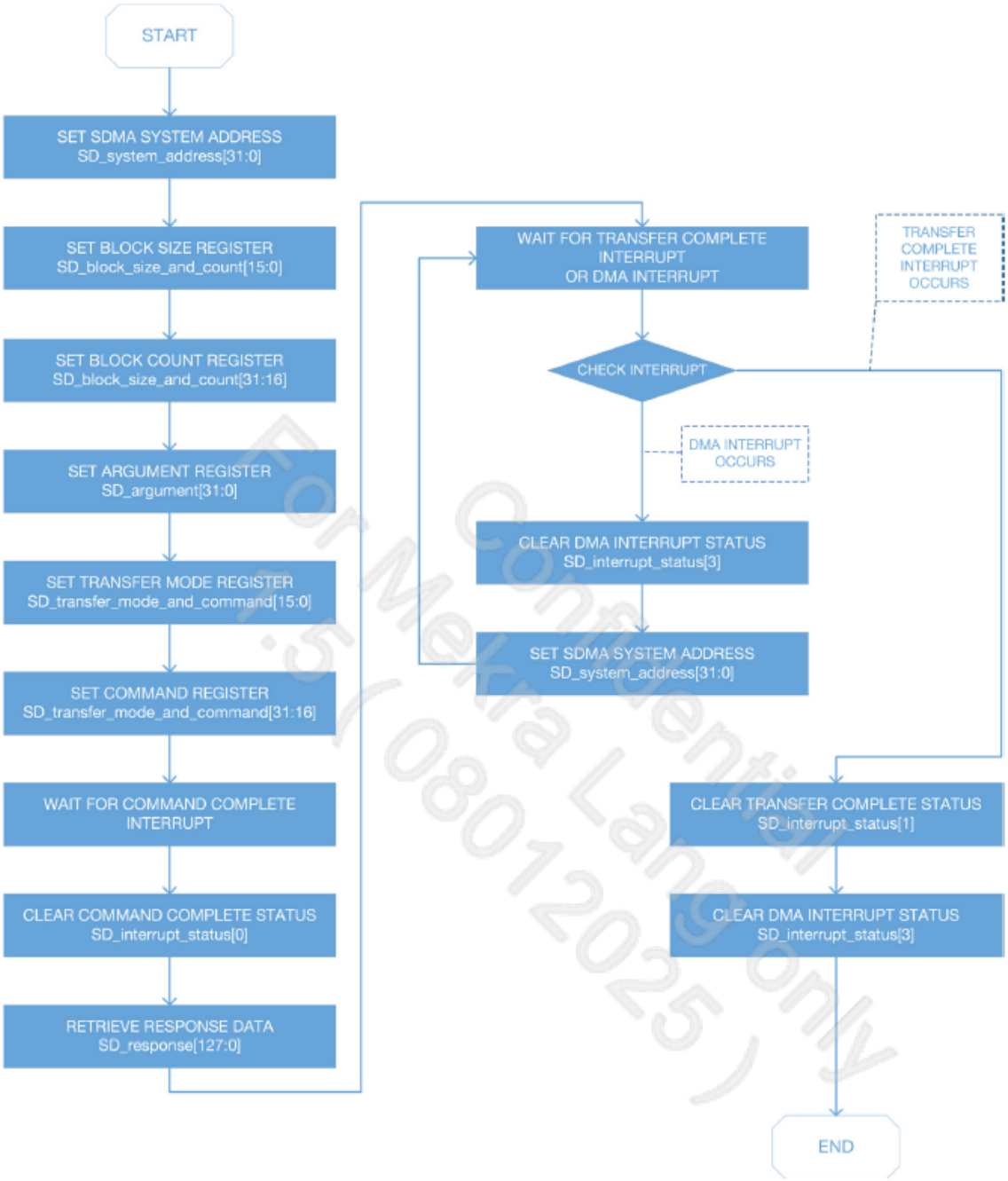


Figure 14-1. SD Controllers: SDMA Command Flow.

14.2.4.2 SD Controllers: ADMA Operation

The ADMA engine enables higher data-transfer speeds by using a scatter-gather DMA algorithm. Using this algorithm, the host driver programs a Descriptor Table with a list of pending data transfers between system memory and the SD card. This enables the ADMA engine to operate without interrupting the host driver. Refer to Section 14.2.7 for an ADMA block diagram and Descriptor Table programming detail.

The command flow for ADMA transfer operations is illustrated in the figure below.

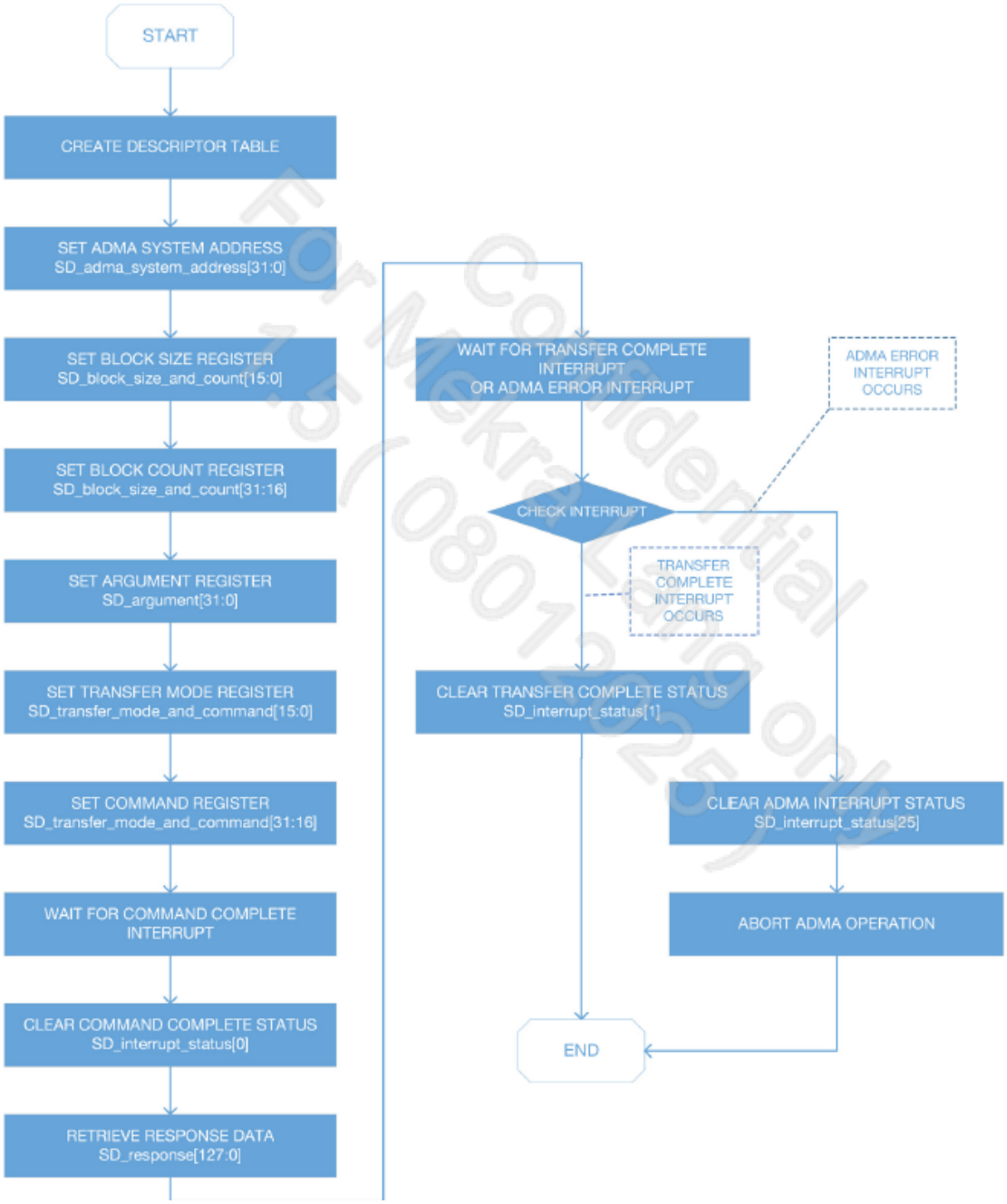


Figure 14-2. SD Controllers: ADMA Command Flow.

The following is a list of steps in the command flow for ADMA transfer operations.

1. Create the Descriptor table for ADMA in the system memory .
2. Set the Descriptor address for ADMA in the ADMA System Address register (**SD_adma_system_address[31:0]**).
3. Set the value corresponding to the executed data byte length of one block in the Block Size register (**SD_block_size_and_count[15:0]**). This step can be executed simultaneously with Step 4.
4. Set the value corresponding to the executed data block count in the Block Count register (**SD_block_size_and_count[31:16]**) in accordance with the Determination of Transfer Access as described in Table 14-8. This step can be executed simultaneously with Step 3.
5. Set the Argument register value (**SD_argument[31:0]**).
6. Set the Transfer Mode register value (**SD_transfer_mode_and_command[15:0]**). The host driver determines Multi / Single Block Select (**sin_multi**), Block Count Enable (**blk_cnt_en**), Data Transfer Direction (**data_trans_dir_select**), Auto CMD12 Enable (**auto_cmd12_enable**) and DMA Enable (**dma_en**) bits. Multi / Single Block Select and Block Count Enable are determined according to the Determination of Transfer Access as described in Table 14-8. This step can be executed simultaneously with Step 7.
7. Set the Command register value (**SD_transfer_mode_and_command[31:16]**). When writing the upper byte, the SD command is issued and DMA is initialized. This step can be executed simultaneously with Step 6.
8. Wait for the Command Complete interrupt.
9. Write 1 to Normal Interrupt Status register Command Complete bit (**SD_interrupt_status[0]**) to clear the interrupt.
10. Read the Response register (**SD_response[127:0]**) for necessary information about the issued command.
11. Wait for the Transfer Complete interrupt or the ADMA Error interrupt.
12. If the Transfer Complete interrupt is 1, proceed to Step 13. If the ADMA Error Interrupt is 1, proceed to Step 14.
13. Clear the Normal Interrupt Status register Transfer Complete Status (**SD_interrupt_status[1]**) by writing the bit to 1. The transfer operation is complete. Proceed to Step 16.
14. Clear the Normal Interrupt Status register ADMA Error Interrupt Status (**SD_interrupt_status[25]**) by writing the bit to 1.
15. Abort the ADMA operation. The SD card must be stopped by issuing an abort command. If necessary, the host driver checks the ADMA Error Status register to determine the reason for the ADMA error.
16. Transfer operation ends.

Ambarella Confidential Use Only (c) 2015

14.2.5 SD Controllers: Register Map

The following table contains a map of the SD controller registers. Note that several of the registers include fields with reset values that depend on SD / MMC boot modes.

Offset	Register	Description
0x000	**SD_system_address**	Single DMA (SDMA) System Address High/Low
0x004	**SD_block_size_and_count**	Block Size / Block Count (SD/MMC boot)
0x008	**SD_argument**	Argument
0x00C	**SD_transfer_mode_and_command**	Transfer Mode (SD/MMC boot) / Command (SD/MMC boot)
0x010 - 0x01E	**SD_response**	Response
0x020	**SD_buffer_data_port**	Buffer Data Port
0x024	**SD_present_state**	Present State
0x028	**SD_hpbw_control**	Host Control (SD/MMC boot) / Power Control / Block Gap / Wakeup
0x02C	**SD_clockcntl_timeoutcntl_swreset**	Clock Control (SD/MMC boot) / Timeout Control / Software Reset
0x030	**SD_interrupt_status**	Normal Interrupt Status / Error Interrupt Status
0x034	**SD_interrupt_status_enable**	Normal Interrupt Status Enable (SD/MMC boot) / Error Interrupt Status Enable
0x038	**SD_interrupt_signal_enable**	Normal Interrupt Signal Enable / Error Interrupt Signal Enable
0x03C	**SD_auto_cmd12_error_status**	Auto CMD12 Error Status register Host Control 2 register
0x040 - 0x044	**SD_capabilities**	Capabilities register provides host driver with Information Specific to host controller Implementation
0x048 - 0x04C	**SD_max_current_capabilities**	Maximum Current Capabilities
0x050		Reserved
0x054	**SD_adma_error_status**	Advanced DMA (ADMA)Error Status
0x058	**SD_adma_system_address**	Advanced DMA (ADMA) System Address
0x060 - 0x06E		Reserved for Preset Value
0x070	**SD_boot_control**	Boot Control (SD/MMC boot)
0x074	**SD_boot_status**	Boot Status (SD/MMC boot)
0x078	**SD_block_count_cmd23**	Block Count for CMD23
0x07C	**SD_voltage_switch**	Voltage Switch
0x080 - 0x0E0		Reserved
0x0F8	**SD_read_latency_control**	Read Latency Control
0x0FC	**SD_slot_interrupt_and_version**	Slot Interrupt / Version ID

Table 14-2. SD Controller AHB Registers.

For Ambarella Confidential Use Only

14.2.6 SD Controllers: Register Details

14.2.6.1 SD_system_address Register

This register contains the system memory address for a Single DMA (SDMA) transfer. When the host controller (HC) stops an SDMA transfer, this register points to the system address of the next contiguous data position. It can be accessed only if the execution of all transfers has ended. Read operations performed during a transfer will return an invalid value. The host driver (HD) initializes this register prior to initiating an SDMA transaction.

After the SDMA transfer has concluded, the system address of the next contiguous data position can be read from this register. The SDMA engine pauses at every boundary specified by the Host DMA Buffer Size in the Block Size register (**SD_block_size_and_count**) in order for the register to be updated. The host controller generates a DMA Interrupt to request that this update occur, and then the host driver sets the system address of the next data position. When the most significant upper byte of this register (003h) is written, the host controller restarts the SDMA transfer.

When restarting SDMA with the Resume command or by setting the Block Gap Control register (**SD_hpbw_control)** field **continue_request**, the host controller starts at the next contiguous address stored in the System Address register **SD_system_address**.

Bits	Name	Access	Reset	Description
31:0	**dma_system_address**	RW	0	System Address for SDMA transfers (Not used for ADMA transfers)

Table 14-3. SD Controller SDMA System Address Register.

For Confidential Meeka Lang only (10.08012025)

14.2.6.2 SD_block_size_and_count Register

Bits	Name	Access	Reset	Description
31:16	**block_count**	RW	0 #	Blocks Count for Current Transfer (Not used for CMD23) This register is enabled when the **SD_transfer_mode_and_command** field **block_count_enable is** set to 1 and is valid only for multiple block transfers. The host controller decrements the block count after each block transfer and stops when the count reaches zero. This register can be accessed only if no transaction is currently executing (i.e., after a transaction has stopped). Read operations performed during transfer will return an invalid value and write operations are ignored. When saving transfer context as a result of a Suspend command, the number of blocks yet to be transferred can be determined by reading this register. When restoring transfer context prior to issuing a Resume command, the host driver restores the previously saved block count. In SDXC mode, when executing CMD23 before CMB18/25, the effective block count is the programmed **SD_block_count_cmd23** register value (offset 0x78) and this register (offset 0x06) is ignored. 0x0000 - Stop Count 0x0001 - 1 block 0x0002 - 2 blocks --- --- 0xFFFF - 65535 blocks Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0x0000 (No data transfer) For MMC_BOOT = 1, the reset value is 0x0004 (4 blocks)
15	**block_size**	RW	0	Transfer Block Size 12th bit This bit is for 4-KB data block transfers (Continued Below)

For Medical Use Only
Confidential
1.5 (Obsolete)

Bits	Name	Access	Reset	Description
14:12	**dma_buffer_size**	RW	0	SDMA Buffer Size (Used for SDMA only. ADMA does not use this Register.) To perform lengthy SDMA transfers, the System Address register (**SD_system_address)** must be updated at every system boundary during transfer, as large contiguous memory space may not be available in the virtual memory system. The **host_dma_buffer_size** bits specify the size of the contiguous buffer in the system memory. The SDMA transfer process pauses at every boundary specified by these fields and the host controller generates the DMA Interrupt to request a **SD_system_address** update. For this function to be active: - The **SD_capabilities** field **dma_support** must be set to 1 - The **SD_transfer_mode_and_command** field **dma_enable** must be set to 1. 0x0 - 4 KB (Detects A11 Carry out) 0x1 - 8 KB (Detects A12 Carry out) 0x2 - 16 KB (Detects A13 Carry out) 0x3 - 32 KB (Detects A14 Carry out) 0x4 - 64 KB (Detects A15 Carry out) 0x5 -128 KB (Detects A16 Carry out) 0x6 - 256 KB (Detects A17 Carry out) 0x7 - 512 KB (Detects A18 Carry out)
11:0	**transfer_block_size**	RW	0 #	Transfer Block Size This register specifies the block size for block data transfers for SD commands CMD17, CMD18, CMD24, CMD25, and CMD53. The valid values range from 0 to 0x400 (1KB), which is less-than or equal to half of the SD FIFO size. This register can be accessed only if no transaction is currently executing (i.e., after a transaction has stopped). Read operations performed during transfer will return an invalid value and write operations are ignored. 0x000 - No Data Transfer 0x001 - 1 bytes 0x002 - 2 bytes 0x003 - 3 bytes 0x004 - 4 bytes --- --- 0x1FF - 511 bytes 0x200 - 512 bytes --- --- 0x800 - 2048 bytes

For Ambarella Confidential Only 1.5 (Do Not Distribute)

Bits	Name	Access	Reset	Description
11:0 (Cont'd)	**transfer_block_size**	RW	0 #	Note that during eMMC boot operation (MMC_BOOT = 1), **transfer_block_size** should be 512 bytes to match the MMC device block size. When the received data reach the block size, the received data are sent to the FIOS buffer. Reset values for this register depend on MMC_BOOT boot configuration inputs: For MMC_BOOT = 0, the reset value is 0x000 (No data transfer) For MMC_BOOT = 1, the reset value is 0x200 (512 B)

Table 14-4. SD Controller Block Size / Block Count Register.

14.2.6.3 SD_argument Register

Bits	Name	Access	Reset	Description
31:0	**command_argument**	RW	0	Command Argument The SD Command Argument is specified as bits 39:8 of Command-Format

Table 14-5. SD Controller Argument Register.

14.2.6.4 SD_transfer_mode_and_command Register

The host driver checks the **Command Inhibit (DAT)** bit and **Command Inhibit (CMD)** bit in the Present State register before writing to this register. Writing to the upper byte of this register triggers SD command generation. The host driver is responsible for writing this register because the SD controller does not protect for writing when **Command Inhibit (CMD)** is set.

Bits	Name	Access	Reset	Description
31:30				Reserved
29:24	**command_index**	RW	0	Command Index This bit is set to the command number (CMD0-63, ACMD0-63).
23:22	**command_type**	RW	0	Command Access This register involves three types of commands: Suspend, Resume and Abort. These bits must be set to 0x0 for all other commands. Suspend Command If the Suspend command succeeds, the host controller assumes the SD Bus has been released and it is therefore available to receive the next command that uses the DAT line. The host controller deasserts Read Wait for read transactions and stops checking for write transactions.

For Mekra Lang only
15 Oct 2025

Bits	Name	Access	Reset	Description
23:22 (Cont'd)	**command_type**	RW	0	The Interrupt cycle begins in 4-bit mode. If the Suspend command fails, the host controller maintains its current state, and the host driver restarts the transfer by setting the Block Gap Control register (**SD_hpbw_control)** field **continue_request**. Resume Command The host driver restarts the data transfer by restoring the registers in the range of 0x000-0x00D. The host controller checks for Busy before starting write transfers. Abort Command If this command is set when executing a read transfer, the host controller stops all reads to the buffer. If this command is set when executing a write transfer, the host controller stops driving the DAT line. After issuing the Abort command, the host driver must issue a software reset 0x0 - Normal 0x1 - Suspend 0x2 - Resume 0x3 - Abort
21	**data_present_select**	RW	0 #	Data Present Select This bit is set to 1 to indicate that data is present and will be transferred using the DAT line. It is set to 0 when the following three commands are issued: (1) Commands using only the CMD line (e.g., CMD52); (2) Commands with no data transfer instructions but which use a Busy signal on the DAT[0] line; (3) Resume commands. 0 - No Data Present 1 - Data Present Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value is 1
20	**command_index_check_enable**	RW	0	Command Index Check Enable If this bit is set to 1, the host controller checks the index field in the response to confirm whether it has the same value as the command index. If the values differ, it is reported as a Command Index Error. If this bit is set to 0, the Index field is not checked. 0 - Disable 1 - Enable
19	**command_crc_check_enable**	RW	0	Command CRC Check Enable If this bit is set to 1, the host controller checks the CRC field in the response. If an error is detected, it is reported as a Command CRC Error. If this bit is set to 0, the CRC field is not checked. 0 - Disable 1 - Enable
18				Reserved

For Mentor Internal Use Only 1.5 / 08 Last Edited

Bits	Name	Access	Reset	Description
17:16	**response_type_ select**	RW	0	Response Access Select 0x0 - No Response 0x1 - Response length 136 0x2 - Response length 48 0x3 - Response length 48 check Busy after response
15:7				Reserved
6	**cmd_comp_int_ en_ata**	RW	0 #	Command completion signal enable for CE-ATA Devices 0 - Device will not send a command completion signal 1 - Device will send a command completion signal Note that this bit is used for eMMC boot operation and the reset value is independent of the MMC_BOOT configuration for the CV22FS chip (see Section 2.3).
5	**sin_multi**	RW	0 #	Multi / Single Block Select This bit enables multiple block DAT line data transfers. 0 - Single Block 1 - Multiple Block Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value is 1
4	**data_trans_dir_ select**	RW	0 #	Data Transfer Direction Select This bit defines the direction of DAT line data transfers. 0 - Write (Host to Card) 1 - Read (Card to Host) Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value is 1
3				Reserved
2	**auto_cmd12_en- able**	RW	0 #	Auto CMD12 Enable Multiple block transfers from memory require CMD12 to stop the transaction. When this bit is set to 1, the host controller issues CMD12 automatically when the last block transfer is completed. The host driver does not set this bit to issue commands that do not require CMD12 to stop data transfer. 0 - Disable 1 - Enable Note that this bit is used for eMMC boot operation and the reset value is independent of the MMC_BOOT configuration for the CV22FS chip (see Section 2.3).

Confidential
For Mediatek Only
1.5 100 Page 2021

Bits	Name	Access	Reset	Description
1	**blk_cnt_en**	RW	0 #	Block Count Enable This bit is used to enable the Block Count register (**SD_block_size_and_count**), which is only relevant for multiple block transfers. When this bit is 0, the Block Count register is disabled, which is useful in executing an infinite transfer. This bit should be set to 1 for ADMA mode 0 - Disable 1 - Enable Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value is 1
0	**dma_en**	RW	0 #	DMA Enable DMA can be enabled only if the **SD_capabilities** field **dma_support** is set. If this bit is set to 1, a DMA operation will begin when the host driver writes to the upper byte of Command register **SD_transfer_mode_and_command** (0x00F). 0 - Disable 1 - Enable Note that this bit is used for eMMC boot operation and the reset value is independent of the MMC_BOOT configuration for the CV22FS chip (see Section 2.3).

Table 14-6. SD Controller Transfer Mode Register.

Response Access	Index Check Enable	CRC Check Enable	Name of Response Access
00	0	0	No Response
01	0	1	R2
10	0	0	R3, R4
10	1	1	R1, R6, R5
11	1	1	R1b, R5b

Table 14-7. Relation Between Parameters and the Name of Response Access.

Multi / Single Block Select sin_multi	Block Count Enable blk_cnt_en	Block Count blk_cnt	Function
0			Single Transfer
1	0		Infinite Transfer
1	1	Not Zero	Multiple Transfer
1	1	Zero	Stop Multiple Transfer

Table 14-8. SD Controller Determination of Transfer Access.

For Mediatek Lab only
Confidential
1.5 (Oct 2025)

14.2.6.5 SD_response Register

Bits	Name	Access	Reset	Description
127:0	**command_response**	R	0	Command Response The following table describes the mapping of command responses from the SD Bus to this register. In the table, R[] refers to a bit range within the response data as transmitted on the SD Bus. REP[] refers to a bit range within the Response register (**SD_response**).

Table 14-9. SD Controller Response Register.

Access of Response	Meaning of Response	Response Field	Response Register
R1, R1b (normal response)	Card Status	R[39:8]	REP[31:0]
R1b (Auto CMD12 response)	Card Status for Auto CMD12	R[39:8]	REP[127:96]
R2 (CID, CSD Register)	CID or CSD reg. incl.	R[127:8]	REP[119:0]
R3 (OCR Register)	OCR Register for memory	R[39:8]	REP[31:0]
R4 (OCR Register)	OCR Register for I/O etc.	R[39:8]	REP[31:0]
R5, R5b	SDIO Response	R[39:8]	REP[31:0]
R6 (Published RCA response)	New published RCA[31:16] etc.	R[39:8]	REP[31:0]

Table 14-10. Response Bit Definition for Each Response Access.

14.2.6.6 SD_buffer_data_port Register

Bits	Name	Access	Reset	Description
31:0	**buffer_data**	RW	NR	Buffer Data The Host controller Buffer can be accessed through this 32-bit Data Port register.

Table 14-11. SD Controller Buffer Data Port Register.

For Confidential Use Only (Hekra Lang 2025)

14.2.6.7 SD_present_state Register

This bit indicates whether one of the DAT lines on SD bus is in use.

Bits	Name	Access	Reset	Description
31:29				Reserved
28:25	**dat_line_signal_level**	R	0	DAT[7:4] Line Signal Level This register is used to check the DAT line level for error recovery, as well as for debugging. D25 - DAT[4] D26 - DAT[5] D27 - DAT[6] D28 - DAT[7]
24	**cmd_line_signal_level**	R	0	CMD Line Signal Level This register is used to check the CMD line level for error recovery, and for debugging.
23:20	**dat_line_signal_level**	R	0	DAT[3:0] Line Signal Level This register is used to check the DAT line level for error recovery, and for debugging. This is especially useful in detecting the Busy signal level from DAT[0]. D20 - DAT[0] D21 - DAT[1] D22 - DAT[2] D23 - DAT[3]
19	**write_protect_switch_pin_level**	R	0	Write Protect Switch Pin Level The Write Protect Switch is supported for memory and combination cards. This bit reflects the **SD[N]_WP** pin. 0 - Write protected (**SD[N]_WP** = 1) 1 - Write enabled (**SD[N]_WP** = 0)
18	**card_detect_pin_level**	R	0	Card Detect Pin Level This bit reflects the inverse value of the **SD[N]_CD** pin. 0 - No Card present (**SD[N]_CD** = 1) 1 - Card present (**SD[N]_CD** = 0)
17	**card_state_stable**	R	0	Card State Stable This bit is used for testing. If it is 0, the Card Detect Pin Level **card_detect_pin_level** is unstable. If this bit is set to 1, it means the Card Detect Pin Level is stable. The Software Reset register (**SD_clockcntl_timeoutcntl_swreset**) field **software_reset_for_all** will not affect this bit. 0 - Reset or Debouncing 1 - No Card or Incorrectly Inserted

For Modified Final 1.5 Only Confidential

Bits	Name	Access	Reset	Description
16	**card_inserted**	R	0	Card Inserted This bit indicates whether a card has been inserted. Changing from 0 to 1 generates a Card Insertion Interrupt in the Normal Interrupt Status register (**SD_interrupt_status**) and changing from 1 to 0 generates a Card Removal Interrupt in **SD_interrupt_status**. The Software Reset register (**SD_clockcntl_timeoutcntl_swreset**) field **software_reset_for_all** does not affect this bit. If a card is removed while powered and while its clock is oscillating, the host controller clears the Power Control register (**SD_hpbw_control**) bit **sd_bus_power** and the Clock control register (**SD_clockcntl_timeoutcntl_swreset**) bit **sd_ clock_enable**. In addition, the host driver will clear the host controller by **software_reset_for_all**. The card detect is active regardless of **sd_bus_power**. 0 - Reset or Debouncing or No Card 1 - Card Inserted
15:12				Reserved
11	**buffer_read_ enable**	R	0	Buffer Read Enable This register is used for non-DMA read transfers. This read-only flag indicates that valid data exists in the host-side buffer. If this bit is 1, readable data exists in the buffer. A change of this bit from 1 to 0 occurs when all the block data is read from the buffer. A change of this bit from 0 to 1 occurs when all the block data is ready in the buffer and generates the Buffer Read Ready Interrupt. 0 - Read Disable 1 - Read Enable
10	**buffer_write_ enable**	R	0	Buffer Write Enable This register is used for non-DMA write transfers. This read only flag indicates if space is available for write data. If this bit is 1, data can be written to the buffer. A change of this bit from 1 to 0 occurs when all the block data is written to the buffer. A change of this bit from 0 to 1 occurs when top of block data can be written to the buffer and generates the Buffer Write Ready Interrupt. 0 - Write Disable 1 - Write Enable

For Ambarella Confidential Use Only
1.5 (Oct 25, 2015)

Bits	Name	Access	Reset	Description
9	**read_transfer_active**	R	0	Read Transfer Active This register is used for detecting completion of a read transfer. This bit is set to 1 under either of the following two conditions: (1) After the end-bit of the read command; (2) When writing a 1 to the Block Gap Control register (**SD_hpbw_control)** field **continue_request** to restart a read transfer. This bit is cleared to 0 under any of the following three conditions: (1) When the last data block (as specified by block length) is transferred to the system; (2) In the case of ADMA, when the end-of-read operation is designated by Descriptor Table; (3) When all valid data blocks have been transferred to the system and no current block transfers are being sent as a result of the Block Gap Control register (**SD_hpbw_control**) field **stop_at_block_gap_request** being set to 1. A transfer complete interrupt is generated when this bit goes to 0. 0 - No valid data 1 - Transferring data
8	**write_transfer_active**	R	0	Write Transfer Active This register indicates a write transfer is active. If this bit is 0, it means no valid write data exists in the host controller. This bit is set under either of the following two conditions: (1) After the end-bit of the write command. (2) When writing a 1 to the Block Gap Control register (**SD_hpbw_control)** field **continue_request** to restart a write transfer. This bit is cleared under either of the following conditions: (1) After retrieving the CRC status of the last data block as specified by the transfer count (Single or Multiple). In the case of ADMA, the transfer count is designated by the Descriptor Table; (2) After retrieving the CRC status of a block where data transmission is about to be stopped by a Block Gap Control register (**SD_hpbw_control**) field **stop_at_block_gap_request**. During a write transaction, a Block Gap Event Interrupt is generated when this bit is changed to 0, as a result of the **stop_at_block_gap_request** being set. This register is useful for the host driver in determining when to issue commands during write Busy. 0 - No valid data 1 - Transferring data
7:3				Reserved

For Marvell Confidential 1.5 (Obsolete)

Bits	Name	Access	Reset	Description
2	**dat_line_active**	R	0	This bit indicates whether one of the DAT lines on the SD bus is in use. (a) In the case of read transactions: This bit indicates whether a read transfer is currently executing on the SD Bus. Changing this value from 1 to 0 generates a Block Gap Event interrupt in the Normal Interrupt Status register (**SD_interrupt_status**) as the result of the Stop At Block Gap Request being set. This bit will be set under either of the following conditions: (1) After the end-bit of the read command. (2) When writing a 1 to Continue Request in the Block Gap Control register (**SD_hpbw_control**) to restart a read transfer. This bit will be cleared under either of the following conditions: (1) When the end-bit of the last data block is sent from the SD Bus to the host controller. In the case of ADMA, the last block is designated by the last transfer in the Descriptor Table. (2) When a read transfer is stopped at the block gap due to a Stop At Block Gap Request. The host controller will stop the read operation at the start of the interrupt cycle of the next block gap by driving Read Wait or stopping the SD clock. (b) In the case of write transactions: This bit indicates that a write transfer is currently executing on the SD Bus. Changing this value from 1 to 0 generates a Transfer Complete interrupt in the Normal Interrupt Status register (**SD_interrupt_status**). This bit will be set under either of the following conditions: (1) After the end-bit of the write command. (2) When writing a 1 to Continue Request in the Block Gap Control register (**SD_hpbw_control**) to continue a write transfer. This bit will be cleared under either of the following conditions: (1) When the SD card releases Write Busy for the last data block. If the SD card does not drive Busy signals, the host controller will consider the card drive Not Busy. In the case of ADMA, the last block is designated by the last transfer in the Descriptor Table. (2) When the SD card releases Write Busy as a result of a Stop At Block Gap Request. (c) Command with Busy: This bit indicates whether a command that shows a Busy status (e.g., erase command) is executing on the SD Bus. This bit is set after the end-bit of the command and cleared when Busy is deasserted. (Continued on the following page)

For Marketing Use Only 1.5 (08/13/2015)

Bits	Name	Access	Reset	Description
2 (Cont'd)	**dat_line_active**	R	0	Changing this bit from 1 to 0 generates a Transfer Complete interrupt in the Normal Interrupt Status register (**SD_interrupt_status**). 0 - DAT line inactive 1 - DAT line active
1	**command_inhibit_dat**	R	0	Command Inhibit (DAT) This register bit is generated if either the DAT Line Active or the Read Transfer Active is set to 1. If this bit is 0, it indicates that the host controller can issue the next SD command. Changing this bit from 1 to 0 generates a Transfer Complete Interrupt in the Normal Interrupt Status register (**SD_interrupt_status**). Note: The SD host driver can save registers in the range of 0x000-0x00D for a suspend transaction after this bit has changed from 1 to 0. 0 - Can issue command which uses the DAT line 1 - Cannot issue command which uses the DAT line
0	**command_inhibit_cmd**	R	0	Command Inhibit (CMD) If this bit is 0, it indicates the CMD line is not in use and the host controller can issue a SD command using the CMD line. This bit is set immediately after the Command register **SD_transfer_mode_and_command** (0x00F) is written. This bit is cleared when the command response is received. Even if the Command Inhibit (DAT) is set to 1, Commands using only the CMD line can be issued if this bit is 0. Changing this bit from 1 to 0 generates a Command Complete Interrupt in the Normal Interrupt Status register (**SD_interrupt_status**). If the host controller cannot issue the command due to a command conflict error or a Command Not Issued By Auto CMD12 Error, this bit will remain 1 and the Command Complete will not be set. Status issuing Auto CMD12 is not read from this bit.

Table 14-12. SD Controller Present State Register.

Note that the host driver can issue CMD0, CMD12, CMD13 (for memory) and CMD52 (for SDIO) when the DAT lines are Busy during data transfer. These commands can be issued when Command Inhibit (CMD) is set to zero. Other commands can only be issued when Command Inhibit (DAT) is set to zero.

14.2.6.8 SD_hpbw_control Register

Bits	Name	Access	Reset	Description
31:27				Reserved
26	**wakeup_event_enable_on_sd_card_removal**	RW	0	Wakeup Event Enable On SD Card Removal This bit enables a wakeup event via Card Removal assertion in the Normal Interrupt Status register (**SD_interrupt_status**). FN_WUS (Wake up Support) in CIS does not affect this bit. 0 - Disable 1 - Enable

Bits	Name	Access	Reset	Description
25	**wakeup_event_enable_on_sd_card_insertion**	RW	0	Wakeup Event Enable On SD Card Insertion This bit enables a wakeup event via Card Insertion assertion in the Normal Interrupt Status register (**SD_interrupt_status**). FN_WUS (Wake up Support) in CIS does not affect this bit. 0 - Disable 1 - Enable
24	**wakeup_event_enable_on_card_interrupt**	RW	0	Wakeup Event Enable On Card Interrupt This bit enables a wakeup event via Card Interrupt assertion in the Normal Interrupt Status register (**SD_interrupt_status**). This bit can be set to 1 if FN_WUS (Wake Up Support) in CIS is set to 1. 0 - Disable 1 - Enable
23:20				Reserved
19	**interrupt_at_block_gap**	RW	0	Interrupt At Block Gap Setting this bit to 1 enables interrupt detection at the block gap for a multiple block transfer. If the SD card cannot signal an interrupt during a multiple block transfer, this bit must be set to 0. When the host driver detects an SD card insertion, it will set this bit according to the CCCR of the SDIO card.
18	**read_wait_control**	RW	0	Read Wait Control The read-wait function is optional for SDIO cards. If the card supports read-wait, set this bit to enable use of the read-wait protocol to stop reading data using the DAT[2] line. Otherwise, the host controller is required to stop the SD clock to hold read data, which restricts commands generation. When the host driver detects an SD card insertion, it will set this bit according to the CCCR of the SDIO card. If the card does not support read-wait, this bit must not be set to 1; otherwise, DAT line conflict may occur. If this bit is set to 0, Suspend / Resume is not supported. 0 - Disable Read Wait Control 1 - Enable Read Wait Control
17	**continue_request**	RW_SC	0	Continue Request This bit is used to restart a transaction which was stopped using the **stop_at_block_gap_request**. To cancel a Stop At Block Gap Request, set **stop_at_block_gap_request** to 0 and set this bit to restart the transfer. The host controller automatically clears this bit under either of the following conditions: (1) In the case of a read transaction, the DAT Line Active changes from 0 to 1 as a read transaction restarts; (2) In the case of a write transaction, the Write transfer active changes from 0 to 1 as the write transaction restarts. It is not necessary for the Host driver to set this bit to 0. If **stop_at_block_gap_request** is set to 1, a write to this bit is ignored. 0 - Ignored 1 - Restart

Bits	Name	Access	Reset	Description
16	**stop_at_block_gap_request**	RW	0	Stop At Block Gap Request This bit is used to stop executing a transaction at the next block gap for non-DMA transfers. Until the transfer complete bit is set to 1, indicating a transfer completion, the host driver must leave this bit set to 1. Clearing both the **stop_at_block_gap_request** and **continue_request** does not cause the transaction to restart. Read Wait is used to stop the read transaction at the block gap. The host controller honors the **stop_at_block_gap_request** for write transfers, but for read transfers it requires that the SD card support Read Wait. Therefore, the host driver does not set this bit during read transfers unless the SD card supports Read Wait and has set Read Wait Control to 1. In the case of write transfers in which the host driver writes data to the Buffer Data Port register (**SD_buffer_data_port**), the host driver will set this bit after all block data is written. If this bit is set to 1, the host driver will not write data to the Buffer Data Port register. This bit affects Read Transfer Active, Write Transfer Active, DAT line active and Command Inhibit (DAT) in the Present State register (**SD_present_state**). 0 - Transfer 1 - Stop
15:12				Reserved
11:9	**sd_bus_voltage_select**	RW	0	SD Bus Voltage Select By setting these bits, the host driver selects the voltage level for the SD card. Before setting this register, the host driver should check the Capabilities register (**SD_capabilities**) voltage support bits. If an unsupported voltage is selected, the host system will not supply the SD bus voltage 0x0 through 0x4 - Reserved 0x5 - 1.8 V (Typ.) 0x6 - 3.0 V (Typ.) 0x7 - 3.3 V (Flat-top.)
8	**sd_bus_power**	RW	0	SD Bus Power Before setting this bit, the SD host driver will set **sd_bus_voltage_select**. If the host controller detects a No Card State, this bit will be cleared. 0 - Power off 1 - Power on
7:5				Reserved
4	**dma_select**	RW	0	DMA Select One of the supported DMA modes can be selected using this bit. The host driver will verify support of DMA modes by referring the Capabilities register (**SD_capabilities**). Use of the selected DMA is determined by the DMA Enable fields of the Transfer Mode register (**SD_transfer_mode_and_command**). 0 - SDMA is selected 1 - 32-bit address ADMA is selected

For Marketing Use Only (08/15/2013)

Bits	Name	Access	Reset	Description
3	**sd8_bit_mode**	RW	0 #	Data Transfer Width - SD8 Bit Mode This bit selects the data width of the host controller. The host driver will set it to match the data width of the SD card. 0 - 8-bit mode is not selected 1 - 8-bit mode is selected Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value depends on the POC SD8_BOOT bit
2	**high_speed_enable**	RW	0 #	High Speed Enable If this bit is set to 0 (default), the host controller outputs CMD line and DAT lines at the falling edge of the SD clock (up to 25 MHz). If this bit is set to 1, the host controller outputs CMD line and DAT lines at the rising edge of the SD clock (up to 50 MHz) 0 - Normal Speed Mode 1 - High Speed Mode Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value depends on the POC HS_BOOT bit
1	**data_transfer_width_sd1_sd4**	RW	0 #	Data Transfer Width - SDIO0 bit mode / SD4 bit mode This bit selects the data width of the host controller. The host driver will select it to match the data width of the SD card. 0 - 1-bit mode 1 - 4-bit mode Note that SDIO0 mode is selected by setting this bit and **sd8_bit_mode** to 0. Note that this bit is used for eMMC boot operation and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value depends on the POC SD4_BOOT bit
0				Reserved

Table 14-13. SD Controller Host Control/Power Control/Block Gap/Wakeup Control Register.

For Medistar only 1.5 (Oct 2025)

14.2.6.9 SD_clockcntl_timeoutcntl_swreset Register

During host controller initialization, the host driver will set the **SD[N]_CLK** Frequency Select and the Data Timeout Counter Value according to the Capabilities register (**SD_capabilities**).

Bits	Name	Access	Reset	Description
31:27				Reserved
26	**software_reset_ for_dat_line**	RW_SC	0	Software Reset for DAT line Only a portion of the data circuit is reset. The following registers and bits are cleared by this bit: Buffer Data Port register (**SD_buffer_data_port**) Buffer is cleared and initialized Present State register (**SD_present_state**) Buffer Read Enable Buffer Write Enable Read Transfer Active Write Transfer Active DAT Line Active Command Inhibit (DAT) Block Gap Control register (**SD_hpbw_control**) Continue Request Stop At Block Gap Request Normal Interrupt Status register (**SD_interrupt_status**) Buffer Read Ready Buffer Write Ready Block Gap Event Transfer Complete 0 - Work 1 - Reset
25	**software_reset_ for_cmd_line**	RW_SC	0	Software Reset for CMD Line Only part of command circuit is reset. The following registers and bits are cleared by this bit: Present State register (**SD_present_state**) Command Inhibit (CMD) Normal Interrupt Status register (**SD_interrupt_status**) Command Complete 0 - Work 1 - Reset
24	**software_reset_ for_all**	RW_SC	0	Software Reset for All This reset affects the entire host controller with the exception of the card detection circuit. Register bits of type R, RW, RW_SC are cleared to 0. During its initialization, the host driver will set this bit to 1 to reset the host controller. The host controller will reset this bit to 0 when Capabilities registers (**SD_capabilities**) are valid and the host driver can read them. Additional use of Software Reset For All may not affect the value of **SD_capabilities**. If this bit is set to 1, the SD card will reset itself and must be re-initialized by the host driver. 0 - Work 1 - Reset
23:20				Reserved

For Mass Distribution
1.5 (Oct 2015)

Bits	Name	Access	Reset	Description
19:16	**data_timeout_counter_value**	RW	0	Data Timeout Counter Value This value determines the interval by which DAT line timeouts are detected. Refer to the Data Timeout Error in the Error Interrupt Status register (**SD_interrupt_status**) for information on factors that dictate timeout generation. Timeout clock frequency will be generated by dividing the base clock TMCLK by this value. When setting this register, prevent inadvertent timeout events by clearing the Data Timeout Error Status Enable (**SD_interrupt_status_enable**) 0x0 - TMCLK * 2^13 0x1 - TMCLK * 2^14 ---------- ---------- 0xE - TMCLK * 2^27 0xF - Reserved
15:8	**sdclk_frequency_select**	RW	0	**SD[N]_CLK** Frequency Select This register is used to select the frequency of the **SD[N]_CLK** pin. The frequency is not programmed directly; rather this register holds the divisor of the Base Clock Frequency for the SD clock in the Capabilities register (**SD_capabilities**). Only the following settings are permitted. 0x00 - base clock 0x01 - base clock divided by 2 0x02 - base clock divided by 4 0x04 - base clock divided by 8 0x08 - base clock divided by 16 0x10 - base clock divided by 32 0x20 - base clock divided by 64 0x40 - base clock divided by 128 0x80 - base clock divided by 256 Setting 0x00 specifies the highest frequency of the SD Clock. Multiple bits should not be set. The two default divider values can be calculated by the frequency that is defined by the Capabilities register (**SD_capabilities**) bit **base_clock_frequency_for_sd_clock**: (1) 25 MHz divider value, and (2) 400 KHz divider value The frequency of the **SD[N]_CLK** is set by the following formula: Clock Frequency = (Base clock) / divisor. Ambarella recommends choosing the smallest possible divisor which results in a clock frequency that is less-than or equal to the target frequency. Maximum Frequency = 50 MHz (base clock) Minimum Frequency = 195.3125 KHz (50 MHz / 256)
7:3				Reserved

For Mass Production 1.5 (08/2014) Ready

Bits	Name	Access	Reset	Description
2	**sd_clock_enable**	RW	1 #	SD Clock Enable The host controller stops **SD[N]_CLK** when writing this bit to 0. **SD[N]_CLK** Frequency Select can be changed when this bit is 0. The host controller maintains the same clock frequency until **SD[N]_CLK** is stopped (Stop at **SD[N]_CLK** = 0). If the host controller detects a No Card state, this bit will be cleared. 0 - Disable 1 - Enable Note that this bit is used for eMMC boot operation and the reset value is independent of the MMC_BOOT configuration for the CV22FS chip (see Section 2.3).
1	**internal_clock_stable**	R	0	Internal Clock Stable This bit is set to 1 when the SD clock is stable after writing the **internal_clock_enable** field to 1. The SD host driver will wait to set **sd_clock_enable** until this bit is set to 1. Note that this bit is useful when using a PLL that requires the setup time. 0 - Not Ready 1 - Ready
0	**internal_clock_enable**	RW	1 #	Internal Clock Enable This bit is set to 0 when the host driver is not using the host controller or when the host controller is awaiting a wakeup event. The host controller should stop its internal clock for a very low power state. Registers can still be read and written. The clock begins to oscillate when this bit is set to 1. When clock oscillation is stable, the host controller will set the **internal_clock_stable** field to 1. This bit will not affect card detection. 0 - Stop 1 - Oscillate Note that this bit is used for eMMC boot operation and the reset value is independent of the MMC_BOOT configuration for the CV22FS chip (see Section 2.3).

Table 14-14. SD Controller Clock Control / Timeout Control / Software Reset Register.

For Ambarella Internal Use Only (1.5 / 06/15/15)

14.2.6.10 SD_interrupt_status Register

The Interrupt Status register (**SD_interrupt_status**) combines Normal Interrupt Status and Error Interrupt Status functions. The Normal Interrupt Enable Status functions affect the read value of this register, while Normal Interrupt Signal register functions do not. An Interrupt is generated when the Normal Interrupt Signal Enable is enabled and at least one of the status bits is set to 1. For all bits except Card Interrupt and Error Interrupt, writing 1 to a bit clears it. The Card Interrupt is cleared when the Card Driver services the interrupt condition.

The Error Interrupt Status values defined in this register can be enabled via the Error Interrupt Status Enable register, but not by the Error Interrupt Signal Enable register. The Interrupt is generated when the Error Interrupt Signal Enable is enabled and at least one of the status bits is set to 1. Writing to 1 clears the bit and writing to 0 keeps the bit unchanged. More than one status can be cleared with one register write.

Bits	Name	Access	Reset	Description
31:26				Reserved
25	**adma_error**	RW	0	ADMA Error This bit is set when the host controller detects errors during an ADMA-based data transfer. The state of the ADMA at an error occurrence is saved in the ADMA Error Status register (**SD_adma_error_status**). In addition, the host controller generates this Interrupt when it detects invalid descriptor data (Valid=0) at the ST_FDS state. The ADMA Error State (**adma_error_state**) field in the ADMA Error Status register indicates whether an error occurs in ST_FDS state. The host driver may find that the Valid bit is not set at the error descriptor. 1: Error 0: No Error
24	**auto_cmd12_error**	RW	0	Auto CMD12 Error. This error occurs when one of the bits in Auto CMD12 Error Status register (**SD_auto_cmd12_error_status**) has changed from 0 to 1. In addition, this bit is set to 1 when Auto CMD12 is not executed due to the previous command error. 0 - No Error 1 - Error
23	**current_limit_error**	RW	0	Current Limit Error By setting the Power Control register (**SD_hpbw_control**) bit **sd_bus_power**, the host controller is requested to supply power for the SD Bus. If the host controller supports the Current Limit Function, it can be protected from an illegal card by terminating the power supply, in which case this bit indicates a failure status. Reading 1 indicates the host controller is not supplying power to the SD card. Reading 0 indicates that the host controller is supplying power and that no error has occurred. This bit will always be set to 0 if the host controller does not support this function. 0 - No Error 1 - Power Fail
22	**data_end_bit_error**	RW	0	Data End-Bit Error This error occurs when a 0 is detected at the end-bit position of read data which uses the DAT line or the end-bit position of the CRC status. 0 - No Error 1 - Error
21	**data_crc_error**	RW	0	Data CRC Error This error occurs when a CRC error is detected during a transfer of read data which uses the DAT line, or when the Write CRC Status returns a value other than 010. 0 - No Error 1 - Error
20	**data_timeout_error**	RW	0	Data Timeout Error This error occurs when one of following timeout conditions is detected: (1) Busy Timeout - R1b, R5b type; (2) Busy Timeout after Write CRC stat; (3) Write CRC status Timeout; (4) Read Data Timeout 0 - No Error 1 - Timeout

For Mass Production 1.5 (Confidential)

Bits	Name	Access	Reset	Description
19	**command_index_error**	RW	0	Command Index Error This error occurs if a Command Index error is indicated by the Command Response. 0 - No Error 1 - Error
18	**command_end_bit_error**	RW	0	Command End-Bit Error This error occurs when the end-bit of a Command Response is 0. 0 - No Error 1 - End-Bit Error Generated
17	**command_crc_error**	RW	0	Command CRC Error Command CRC Error is generated in two cases: (1) If a response is returned and the Command Timeout Error (**command_timeout_error**) is set to 0, this bit is set to 1 when detecting a CRT error in the command response. (2) The host controller detects a CMD-line conflict by monitoring the CMD line when a command is issued. If the host controller drives the CMD line to 1 level, but detects 0 level on the CMD line at the next **SD[N]_CLK** edge, then the host controller will abort the command (i.e., stop driving the CMD line) and set this bit to 1. The Command Timeout Error will also be set to 1 to distinguish CMD line conflict. 0 - No Error 1 - CRC Error Generated
16	**command_timeout_error**	RW	0	Command Timeout Error This error occurs if no response is returned within 64 **SD[N]_CLK** cycles from the end-bit of the command. If the host controller detects a CMD-line conflict, this bit will be set without waiting for 64 **SD[N]_CLK** cycles, as the command will be aborted. 0 - No Error 1 - Timeout
15	**error_interrupt**	R	0	Error Interrupt Setting any bit in the Error Interrupt Status register (**SD_interrupt_status**) sets this bit. Therefore, the host driver can test for an error by checking this bit first. 0 - No Error 1 - Error
14:9				Reserved

For Ambarella Confidential Only
1.5 (08/2020)

Bits	Name	Access	Reset	Description
8	**card_interrupt**	R	0	Card Interrupt Writing this bit to 1 does not clear this bit. It is cleared by resetting the SD card interrupt factor. In 1-bit mode, the host controller will detect the card interrupt without wakeup support. In 4-bit mode, the card-interrupt signal is sampled during the interrupt cycle. When this register has been set, Card Interrupt Status Enable in the Normal Interrupt Status register (**SD_interrupt_status**) will be set to 0 in order to clear the card-interrupt statuses latched in the host controller and stop driving the host system. After completion of the card-interrupt service (the reset factor in the SD card and the interrupt signal may not be asserted), set Card Interrupt Status Enable to 1 and begin sampling the interrupt signal again. 0 - No Card Interrupt 1 - Generate Card Interrupt
7	**card_removal**	RW	0	Card Removal This register is set if the Card Inserted bit in the Present State register (**SD_present_state**) changes from 1 to 0. When the host driver writes this bit to 1, the status of the Card Inserted bit should be confirmed. 0 - Card State Stable or Debouncing 1 - Card Removed
6	**card_insertion**	RW	0	Card Insertion This register is set if the Card Inserted bit in the Present State register (**SD_present_state**) changes from 0 to 1. When the host driver writes this bit to 1, the status of the Card Inserted bit should be confirmed.. 0 - Card State Stable or Debouncing 1 - Card Inserted
5	**buffer_read_ready**	RW	0	Buffer Read Ready This register is set if the Buffer Read Enable bit changes from 0 to 1. 0 - Not Ready to read Buffer 1 - Ready to read Buffer
4	**buffer_write_ ready**	RW	0	Buffer Write Ready This register is set if the Buffer Write Enable bit changes from 0 to 1. 0 - Not Ready to Write Buffer 1 - Ready to Write Buffer
3	**dma_interrupt**	RW	0	DMA Interrupt Status This register is set if the host controller detects the Host DMA Buffer Boundary in the Block Size register (**SD_block_size_and_count**). In the case of ADMA, the host controller generates this interrupt by setting the **Int** field in the Descriptor Table. 0 - No DMA Interrupt 1 - DMA Interrupt is Generated

For Motorola Only
1 5 Oct 2015

Bits	Name	Access	Reset	Description
2	**block_gap_event**	RW	0	Block Gap Event If the Block Gap Control register (**SD_hpbw_control**) field **stop_at_block_gap_request** is set, this bit is set. Read Transaction: This bit is set at the falling edge of the DAT Line Active Status (when the transaction is stopped). Read Wait access must be supported to use this function. Write Transaction: This bit is set at the falling edge of Write Transfer Active Status (after retrieving CRC status). 0 - No Block Gap Event 1 - Transaction stopped at Block Gap
1	**transfer_complete**	RW	0	Transfer Complete This bit is set when a read / write transaction is completed. Read Transaction: This bit is set at the falling edge of Read Transfer Active Status. There are two conditions under which the Interrupt is generated. The first is when a data transfer is completed as specified by data length; i.e., after the last data has been read to the host system. The second is when data has stopped at the block gap and data transfer is completed by setting the Block Gap Control register (**SD_hpbw_control**) field **stop_at_block_gap_request**; i.e., after valid data has been read to the host system. Write Transaction: This bit is set at the falling edge of the DAT Line Active Status. There are two conditions under which the Interrupt is generated. The first is when the last data has been written to the card (specified by data length) and the Busy signal is released. The second is when data transfers have been stopped at the block gap by setting the Block Gap Control register (**SD_hpbw_control**) field **stop_at_block_gap_request** and data transfers complete; i.e., after valid data is written to the SD card and the Busy signal is released. Command with Busy: This bit is set when Busy is de-asserted. Refer to DAT Line Active (**dat_line_active**) and Command Inhibit (DAT) (**command_inhibit_dat**) fields in the Present State register (**SD_present_state**). As shown in the table below, Transfer Complete has a higher priority than the **SD_interrupt_status** Data Timeout Error (**data_timeout_error**). If both bits are set to 1, execution of a command can be considered to be completed. 0 - No Data Transfer Complete 1 - Data Transfer Complete

For Marvell Internal Use Only
Confidential
1.5 (08/15/2013)

Bits	Name	Access	Reset	Description
0	**command_complete**	RW	0	Command Complete This bit is set when the end-bit of the command response is retrieved (Except Auto CMD12). Refer to Command Inhibit (CMD) (**command_inhibit_cmd**) in the Present State register (**SD_present_state**). Note: The **SD_interrupt_status** register Command Timeout Error (**command_timeout_error**) has a higher priority than **command_complete**. If both are set to 1, it can be assumed that the response was not received correctly. 0 - No Command Complete 1 - Command Complete

Table 14-15. SD Controller Normal Interrupt Status / Error Interrupt Status Register.

Transfer Complete transfer_complete	Data Timeout Error data_timeout_error	Description
0	0	Interrupted by Another Factor
0	1	Timeout occurred during transfer
1		Data Transfer Complete

Table 14-16. Normal Interrupt Status: Relation Between Transfer Complete and Data Timeout Error.

Command Complete command_complete	Command Timeout Error command_timeout_error	Description
0	0	Interrupted by Another Factor
	1	Response not received within 64 **SD[N]_CLK** cycles
1	0	Response Received

Table 14-17. Normal Interrupt Status: Relation Between Command Complete and Command Timeout Error.

Command CRC Error command_crc_error	Command Timeout Error command_timeout_error	Description
0	0	No Error
0	1	Response Timeout Error
1	0	Response CRC Error
1	1	CMD Line Conflict

Table 14-18. Error Interrupt Status: Relation Between Command CRC Error and Command Timeout Error.

For Mekra Use Only (2501-025) Confidential

14.2.6.11 SD_interrupt_status_enable Register

For the following, setting a bit to 1 enables the Interrupt Status.

Bits	Name	Access	Reset	Description
31:26				Reserved
25	**adma_error_sa-tus_enable**	RW	0	ADMA Error Status Enable 0 - Masked 1 - Enabled
24	**auto_cmd12_er-ror_status_enable**	RW	0	Auto CMD12 Error Status Enable 0 - Masked 1 - Enabled
23	**current_limit_er-ror_status_enable**	RW	0	Current Limit Error Status Enable 0 - Masked 1 - Enabled
22	**data_end_bit_er-ror_status_enable**	RW	0	Data End-Bit Error Status Enable 0 - Masked 1 - Enabled
21	**data_crc_error_ status_enable**	RW	0	Data CRC Error Status Enable 0 - Masked 1 - Enabled
20	**data_timeout_er-ror_status_enable**	RW	0	Data Timeout Error Status Enable 0 - Masked 1 - Enabled
19	**command_index_ error_status_en-able**	RW	0	Command Index Error Status Enable 0 - Masked 1 - Enabled
18	**command_end_ bit_error_ status_ enable**	RW	0	Command End-Bit Error Status Enable 0 - Masked 1 - Enabled
17	**command_crc_er-ror_status_enable**	RW	0	Command CRC Error Status Enable 0 - Masked 1 - Enabled
16	**command_time-out_error_sta-tus_enable**	RW	0	Command Timeout Error Status Enable 0 - Masked 1 - Enabled
15	**fixed_to_0**	R	0	Fixed to 0 The host controller will control error Interrupts using the Error Interrupt Status Enable register (**SD_interrupt_status_enable**).
14:9				Reserved
8	**card_interrupt_ status_enable**	RW	0	Card Interrupt Status Enable If this bit is set to 0, the host controller will clear the interrupt request to the System. The Card Interrupt detection is stopped when this bit is cleared and restarted when this bit is set to 1. The host driver should clear **card_interrupt_status_enable** before servicing the Card Interrupt and should set this bit again after all interrupt requests from the card are cleared to prevent inadvertent interrupts. 0 - Masked 1 - Enabled

For Mass Production (1.5 for Linux only 2015/03/13)

Bits	Name	Access	Reset	Description
7	**card_removal_status_enable**	RW	0	Card Removal Status Enable 0 - Masked 1 - Enabled
6	**card_insertion_status_enable**	RW	#	Card Insertion Status Enable 0 - Masked 1 - Enabled Note that this bit is used for eMMC boot operation, and the reset value depends on the MMC_BOOT configuration (see Section 2.3): For MMC_BOOT = 0, the reset value is 0 For MMC_BOOT = 1, the reset value is 1
5	**buffer_read_ready_status_enable**	RW	0	Buffer Read Ready Status Enable 0 - Masked 1 - Enabled
4	**buffer_write_ready_status_enable**	RW	0	Buffer Write Ready Status Enable 0 - Masked 1 - Enabled
3	**dma_interrupt_status_enable**	RW	0	DMA Interrupt Status Enable 0 - Masked 1 - Enabled
2	**block_gap_event_status_enable**	RW	0	Block Gap Event Status Enable 0 - Masked 1 - Enabled
1	**transfer_complete_status_enable**	RW	0	Transfer Complete Status Enable 0 - Masked 1 - Enabled
0	**command_complete_status_enable**	RW	0	Command Complete Status Enable 0 - Masked 1 - Enabled

Table 14-19. Normal Interrupt Status Enable / Error Interrupt Status Enable Register.

Notes:

- With Normal Interrupt Status Enable, the host controller may sample the card Interrupt signal during the interrupt period and may hold its value in the flip-flop. If Card Interrupt Status Enable is set to 0, the host controller clears all internal signals regarding Card Interrupt.
- For Error Interrupt Status Enable be aware that to Detect a CMD Line conflict, the host driver must set both the Command Timeout Error Status Enable and the Command CRC Error Status Enable bits to 1.

For Ambarella Confidential Use Only (2025)

14.2.6.12 SD_normal_interrupt_signal_enable Register

This register is used to select which status is indicated to the host system for a given interrupt. These status bits all share the same 1-bit interrupt line. Setting one of these bits to 1 enables interrupt generation.

Bits	Name	Access	Reset	Description
31:26				Reserved
25	**adma_error_signal_enable**	RW	0	ADMA Error Signal Enable 0 - Masked 1 - Enabled
24	**auto_cmd12_error_signal_enable**	RW	0	Auto CMD12 Error Signal Enable 0 - Masked 1 - Enabled
23	**current_limit_error_signal_enable**	RW	0	Current Limit Error Signal Enable 0 - Masked 1 - Enabled
22	**data_end_bit_error_signal_enable**	RW	0	Data End-Bit Error Signal Enable 0 - Masked 1 - Enabled
21	**data_crc_error_signal_enable**	RW	0	Data CRC Error Signal Enable 0 - Masked 1 - Enabled
20	**data_timeout_error_signal_enable**	RW	0	Data Timeout Error Signal Enable 0 - Masked 1 - Enabled
19	**command_index_error_signal_enable**	RW	0	Command Index Error Signal Enable 0 - Masked 1 - Enabled
18	**command_end_bit_error_signal_enable**	RW	0	Command End-Bit Error Signal Enable 0 - Masked 1 - Enabled
17	**command_crc_error_signal_enable**	RW	0	Command CRC Error Signal Enable 0 - Masked 1 - Enabled
16	**command_timeout_error_signal_enable**	RW	0	Command Timeout Error Signal Enable 0 - Masked 1 - Enabled
15	**fixed_to_0**	R	0	Fixed to 0 The host driver will control error Interrupts using the Error Interrupt Signal Enable register (**SD_interrupt_signal_enable**).
14:9				Reserved
8	**card_interrupt_signal_enable**	RW	0	Card Interrupt Signal Enable 0 - Masked 1 - Enabled
7	**card_removal_signal_enable**	RW	0	Card Removal Signal Enable 0 - Masked 1 - Enabled
6	**card_insertion_signal_enable**	RW	0	Card Insertion Signal Enable 0 - Masked 1 - Enabled

For Confidential Use Only (2025)

Bits	Name	Access	Reset	Description
5	**buffer_read_ready_signal_enable**	RW	0	Buffer Read Ready Signal Enable 0 - Masked 1 - Enabled
4	**buffer_write_ready_signal_enable**	RW	0	Buffer Write Ready Signal Enable 0 - Masked 1 - Enabled
3	**dma_interrupt_signal_enable**	RW	0	DMA Interrupt Signal Enable 0 - Masked 1 - Enabled
2	**block_gap_event_signal_enable**	RW	0	Block Gap Event Signal Enable 0 - Masked 1 - Enabled
1	**transfer_complete_signal_enable**	RW	0	Transfer Complete Signal Enable 0 - Masked 1 - Enabled
0	**command_complete_signal_enable**	RW	0	Command Complete Signal Enable 0 - Masked 1 - Enabled

Table 14-20. SD Controller Normal Interrupt Signal Enable / Error Interrupt Signal Enable Register.

For Mekra Lang
1.5 (2017-08-17)
Confidential

14.2.6.13 SD_auto_cmd12_error_status Register

When Auto CMD12 Error Status is set, the host driver will check this register to identify the error type. This register is valid only when the Auto CMD12 Error is set.

Bits	Name	Access	Reset	Description
31				Reserved for Preset Value Enabled
30	**asynchronous_interrupt_enable**	RW	1	This bit can be set to 1 if a card supports asynchronous interrupts and Asynchronous Interrupt Support is set to 1 in the Capabilities register. Asynchronous interrupt is effective when DAT[1] interrupt is used in 4-bit SD mode (and the Interrupt Pin Select is set to 0 in the Shared Bus Control register). If this bit is set to 1, the host driver is able to stop the SDCLK during an asynchronous interrupt period to save power. During this period, the host controller continues to deliver the Card Interrupt to the host when it is asserted by the Card. 1 - Enable 0 - Disable
29	**cmd23_disable**	RW	0	1: Disable SDXC CMD23 (for correct ACMD23 sequences; [CMD55+CMD23] when operating bus-muxed cards) 0: Enable SDXC CMD23
28-24				Reserved
23				Reserved for Sampling Clock Select
22				Reserved for Execute Tuning
21:20				Reserved for Driver Strength Select

Bits	Name	Access	Reset	Description
19	**1.8v_signaling_enable**	RW	0	This bit controls the voltage regulator for the I/O cell. Note that 3.3 V is supplied to the card regardless of signaling voltage. Setting this bit from 0 to 1 initiates a change in signal voltage from 3.3 V to 1.8 V. The 1.8-V regulator output will be stable within 5 ms. The host controller clears this bit if switching to 1.8-V signaling fails. Clearing this bit from 1 to 0 initiates a change in signal voltage from 1.8 V to 3.3 V. The 3.3-V regulator output will be stable within 5 ms. The host driver can set this bit to 1 when the host controller supports 1.8-V signaling (One of the support bits is set to 1: SDR50, SDR104, or DDR50 in the Capabilities register) and the card or device supports UHS-1 (S18A-1. Refer to Bus Signal Voltage Switch Sequence in the *Physical Layer Specification Version 3.0x*). 1 - 1.8-V Signaling 0 - 3.3-V Signaling
18	**ddr_mode**	RW	0	1 - ddr_mode 0 - sdr_mode
17:16				Reserved for UHS Mode Select
15:8				Reserved
7	**command_not_issued_by_auto_cms12_error**	R	0	Command Not Issued By Auto CMD12 Error Setting this bit to 1 means CMD_wo_DAT is not executed due to an Auto CMD12 error (D04 - D01) in this register. 0 - No Error 1- Not Issued
6:5				Reserved
4	**auto_cmd12_index_error**	R	0	Auto CMD12 Index Error Occurs if the Command Index occurs in response to a command. 0 - No Error 1 - Error
3	**auto_cmd12_end_bit_error**	R	0	Auto CMD12 End Bit Error Occurs when detecting that the end bit of command response is 0. 0 - No Error 1 - End Bit Error Generated
2	**auto_cmd12_crc_error**	R	0	Auto CMD12 CRC Error Occurs when detecting a CRC error in the command response. 0 - No Error 1 - CRC Error Generated
1	**auto_cmd12_timeout_error**	R	0	Auto CMD12 Timeout Error Occurs if no response is returned within 64 **SD[N]_CLK** cycles from the end bit of the command. If this bit is set to 1, the other error status bits (D04 - D02) are meaningless. 0 - No Error 1 - Timeout

For Ambarella Confidential Use Only

Bits	Name	Access	Reset	Description
0	**auto_cmd12_not_executed**	R	0	Auto CMD12 not Executed If memory multiple block data transfer does not start due to command error, this bit is not set because it is not necessary to issue Auto CMD12. Setting this bit to 1 means the host controller cannot issue Auto CMD12 to stop memory multiple block transfer due to some error. If this bit is set to 1, other error status bits (D04 - D01) are meaningless. 0 - Executed 1 - Not Executed

Table 14-21. SD Controller Auto CMD12 Error Status and Host Control 2 Register.

Auto CMD12 CRC Error	Auto CMD12 Timeout Error	Description
0	0	No Error
0	1	Response Timeout Error
1	0	Response CRC Error
1	1	CMD Line Conflict

Table 14-22. Relationship Between Auto CMD12 CRC Error and Auto CMD12 Timeout Error.

14.2.6.14 SD_capabilities Register

This register provides the host driver with information specific to the host controller implementation. The host controller may implement these values as fixed or loaded from Flash memory during power on initialization.

Bits	Name	Access	Reset	Description
63:32				Reserved
31:30	**slot_type**	RW	0	This field indicates usage of a slot by a specific host system. (A host controller register set is defined per-slot.). Embedded Slot for One Device (01) indicates that only one non-removable device is connected to a SD bus slot. 00 - Removable Card Slot 01 - Embedded Slot for One Device 11 - Reserved Note: The register type is HWI in [5], but is RW here.
29	**asynchronous_interrupt_support**	HWI	1	1 - Asynchronous Interrupt Supported 0 - Asynchronous Interrupt Not Supported
28				Reserved
27	**interrupt_mode**	HWI	1	Interrupt mode 0 - Not Supported 1 - Supported

For Mekra Lang only Confidential 1.5

Bits	Name	Access	Reset	Description
26	**voltage_ support_1.8v**	HWI	1	Voltage Support 1.8 V 0 - 1.8 V Not Supported 1 - 1.8 V Supported Note that the host driver sets the Power Control register (**SD_ hpbw_control**) field **sd_bus_voltage_select** according to support bits[26:24]. If multiple voltages are supported, select the lowest usable voltage by comparing the OCR value from the card.
25	**voltage_ support_3.0v**	HWI	0	Voltage Support 3.0 V 0 - 3.0 V Not Supported 1 - 3.0 V Supported Note that the host driver sets the Power Control register (**SD_ hpbw_control**) field **sd_bus_voltage_select** according to support bits[26:24]. If multiple voltages are supported, select the lowest usable voltage by comparing the OCR value from the card.
24	**voltage_ support_3.3v**	HWI	1	Voltage Support 3.3 V 0 - 3.3 V Not Supported 1 - 3.3 V Supported Note that the host driver sets the Power Control register (**SD_ hpbw_control**) field **sd_bus_voltage_select** according to support bits[26:24]. If multiple voltages are supported, select the lowest usable voltage by comparing the OCR value from the card.
23	**suspend_re- sume_support**	HWI	1	Suspend / Resume Support This bit indicates whether the host controller supports Suspend / Resume functionality. If this bit is 0, the Suspend and Resume mechanisms are not supported and the host driver will not issue either Suspend or Resume commands. 0 - Not Supported 1 - Supported
22	**dma_support**	HWI	1	DMA Support This bit indicates whether the host controller is capable of using DMA to transfer data between system memory and the host controller directly. 0 - DMA Not Supported 1 - DMA Supported
21	**high_speed_ support**	HWI	1	High Speed Support This bit indicates whether the host controller and the host sys- tem support High-Speed mode. 0 - High Speed Not Supported 1 - High Speed Supported
20		R	0	Reserved
19	**adma_support**	HWI	1	ADMA Support This bit indicates whether the host controller is ADMA-capable. 0 - ADMA Not Supported 1 - ADMA Supported
18		R	0	Reserved

For Meeting Only 1.5 (08/23/2013)

Bits	Name	Access	Reset	Description
17:16	**max_block_ length**	HWI	1	Max Block Length This value specifies the maximum block size that the host driver can read and write to the buffer in the host controller. The buffer will transfer this block size without wait cycles. Three sizes can be defined as indicated below. 0x0 - 512 bytes 0x1 - 1024 bytes 0x2 - 2048 bytes 0x3 - Reserved
15:0				Reserved

Table 14-23. SD Controller Capabilities Register.

14.2.6.15 SD_max_current_capabilities Register

Bits	Name	Access	Reset	Description
63:24				Reserved
23:16	**maximum_current_for_1.8v**	R	0	Maximum current for 1.8 V
15:8	**maximum_current_for_3.0v**	R	0	Maximum current for 3.0 V
7:0	**maximum_current_for_3.3v**	R	0x01	Maximum current for 3.3 V

Table 14-24. SD Controller Maximum Current Capabilities Register.

The maximum current values are defined as follows:

Register Value	Current Value
0	Retrieve information via another method
1	4 mA
2	8 mA
3	12 mA
255	1020 mA

Table 14-25. SD Controller Definitions for Maximum Current Value.

Confidential For Media Lab only (Rosetta 35)

14.2.6.16 SD_adma_error_status Register

When an ADMA Error Interrupt occurs, the ADMA Error State field (**adma_error_state**) in this register holds the ADMA state, and the ADMA System Address register (**SD_adma_system_address**) holds the address around the error descriptor.

Bits	Name	Access	Reset	Description
31:6				Reserved
5	**adma_byte_error**	R	0	ADMA Byte Error Total data length cannot be divided by the block length. 0 – No error 1 – Error This bit is reserved in the SDA specification.
4	**adma_block_error**	R	0	ADMA Block Error The total data length specified by the Descriptor table differs from that specified by Block Count and Block Length. This bit is reserved in the SDA specification.
3	**invalid_adma_descriptor**	R	0	Invalid ADMA Descriptor This bit is set after fetching an ADMA descriptor to Valid = 0. 0 – No error 1 – Error This bit is reserved in the SDA specification.
2	**adma_length_mismatch_error**	R	0	ADMA Length Mismatch Error This error occurs under two conditions: (1) The total data length specified by the Descriptor table differs from that specified by the Block Count and Block Length (bit[4]=1); (2) Total data length cannot be divided by the block length (bit[5]=1) bit[2] = bit[4] | bit[5]. 0 – No error 1 – Error
1:0	**adma_error_state**	R	0	ADMA Error State This field indicates the state of ADMA when the error occurred. This field never takes on value 0x2, since the ADMA never stops in this state. 0x0 - Error state is ST_STOP (Stop DMA) and SYS_SDR register points to the next error descriptor 0x1 - Error state is ST_FDS (Fetch Descriptor) and SYS_SDR points to the error descriptor 0x2 - Not used 0x3 - Error state is ST_TFR (Transfer Data) and SYS_SDR points to the next error descriptor

Table 14-26. SD Controller ADMA Error Status Register.

14.2.6.17 SD_adma_system_address Register

This register contains the physical descriptor address used for ADMA data transfer.

Bits	Name	Access	Reset	Description
31:0	**adma_system_address**	RW	0	This register holds the byte address of the executing command from the Descriptor table. At the start of an ADMA transaction, the host driver should set the start address of the Descriptor table. The ADMA increments the register address, which points to the next line, when fetching every Descriptor line. When the ADMA Error Interrupt is generated, this register holds the valid Descriptor address depending on the ADMA state. The host driver should program the Descriptor Table on a 32-bit boundary and set a 32-bit boundary address to this register. The ADMA ignores the lower two bits of this register and assumes it to be 0x0.

Table 14-27. SD Controller ADMA System Address Register.

14.2.6.18 SD_boot_control Register

Bits	Name	Access	Reset	Description
31:17				Reserved
16	**rst_fio_boot_en**	RW	0	FIO Boot Enable on SD Software Reset This bit is used for eMMC boot operation (Section 2.3). If this bit is 1 during an SD software reset, an RST_FIO_BOOT is asserted and an IO_RESET_MISC of the original FIOS also will be asserted to enter boot mode.
15:14				Reserved
13:0	**boot_size**	RW	0x1000	Boot Code Length in Bytes This bit is used for eMMC boot operation (Section 2.3). When the size of received boot codes reaches this value, eMMC boot logic will terminate the boot operation. As FIOS FIFO is 64 bits in width, the size of this field should be a multiple of 8 bytes.

Table 14-28. SD Controller Boot Control Register.

14.2.6.19 SD_boot_status Register

Bits	Name	Access	Reset	Description
31:25				Reserved
24	**boot_end_alt**	R	0	Boot End Alt This bit is used for eMMC boot operation (Section 2.3). When the fetched boot code reaches the value of the **SD_boot_control** field **boot_size**, this bit will be asserted. Both hardware and software resets clear this bit.
23:17				Reserved

Bits	Name	Access	Reset	Description
16	**boot_end**	R_SC	0	Boot End This bit is used for eMMC boot operation (Section 2.3). When the fetched boot code reaches **boot_size**, this bit will be asserted. Software reset does not clear this bit.
15:1				Reserved
0	**rdy_to_boot**	R	0	Ready to Boot This bit is used for eMMC boot operation (Section 2.3). This bit is deasserted after a reset operation and is asserted 74 cycles after a power on or a reset operation (including the SD command CMD0 of value 0xF0F0F0F0).

Table 14-29. SD Controller Boot Status Register.

14.2.6.20 SD_block_count_cmd23 Register

Bits	Name	Access	Reset	Description
31:0	**block_cnt**	R	0	Block Count for CMD23 This register is updated by the Argument register (**SD_argument**) when the command index is written as 23. This register is independent from the Block Count register **SD_block_size_and_count** (offset 0x000). If no CMD23 is executed before CMD18/25, the effective block count is the programmed value at **SD_block_size_and_count** (offset 0x006), and this register (offset 0x078) is ignored.

Table 14-30. SD Controller Block Count (CMD23) Register.

14.2.6.21 SD_voltage_switch Register

This register is used to detect activity on the bus in preparation for switching the voltage.

Bits	Name	Access	Reset	Description
31:17				Reserved
16	**cmd_stat**	RW	0	CMD Status Asserted when the CMD signal driven by the card is high for 1 cycle (this bit has a value of 1). Cleared when written to 0 by the software. This field can be used in operating the voltage switch. Contact an Ambarella representative for details.
15:4				Reserved
3:0	**dat_stat**	RW	0	DAT Status Asserted when DAT[3:0] signals driven by the card are high for 1 cycle (this bit has a value of 1). Cleared when written to 0 by the software. This field can be used in operating the voltage switch. Contact an Ambarella representative for details.

Table 14-31. SD Controller Voltage Switch Register.

Confidential
For Wang Yang only
1.5 (Oct 01 2023)

14.2.6.22 SD_read_latency_control Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15	**disable_hs_cmd_conf**	RW	0	0x0: Enable command conflict check in high-speed mode 0x1: Disable command conflict check in high-speed mode
14:12	**d_cmd_out_hs_sel**	RW	0	Selects which tap should be used to check for command conflict in high-speed mode.
11	**disable_ds_cmd_conf**	RW	0	0x0: Enable command conflict check in default-speed mode 0x1: Disable command conflict check in default-speed mode
10:8	**d_cmd_out_ds_sel**	RW	0	Selects which tap should be used to check for command conflict in default-speed mode.
7:4	**trn_to_crc_status**	RW	0	Determines the cycle count between write data and CRC status. The SD controller will begin to check the start bit of CRC status (**trn_to_crc_status** + 1) cycles after the end bit of the write data. 0x0 - 1 cycle 0x1 - 2 cycles -------- 0xF - 16 cycles
3:0	**ncr_reg**	RW	0	Determines the cycle count between command and response. The SD controller will begin to check the start bit of the received response (**ncr_reg** + 1) cycles after the end bit of the transmitted command. 0x0 - Reserved 0x1 - Reserved 0x2 - 3 cycles -------- 0xF - 16 cycles **Note:** NCR_REG must be greater than or equal to 2 (NCR>=2) to avoid spurious CMD

Table 14-32. Read Latency Control Register.

14.2.6.23 SD_slot_interrupt_and_version Register

Bits	Name	Access	Reset	Description
31:24				Reserved
23:16	**specification_version_number**	R	2	Specification Version Number This register indicates the host controller Specification Version. The Upper and Lower 4 bits specify the version. The CV22FS default value is 0x2. 00 - SD host specification version 1.0 01 - SD host specification version 2.0 02 - SD host specification version 3.0 Others - Reserved
15:8				Reserved

For Mass Confidential 1.5 (08 Jan 2025) Beijing only

Bits	Name	Access	Reset	Description
7:0	**interrupt_signal_for_each_slot**	R	0	Interrupt Signal for Each Slot These status bits indicate the logical OR of the Interrupt signal and Wakeup signal for each slot. A maximum of eight slots can be defined. If one interrupt signal is associated with multiple slots, the host driver can identify which interrupt is generated by reading these status bits. A power on reset or a "software reset for all," will clear these bits and deassert the interrupt signal (zero). Bit 00 - Slot 1 Bit 01 - Slot 2 Bit 02 - Slot 3 ----- ----- Bit 07 - Slot 8

Table 14-33. SD Controller Slot Interrupt Status and Controller Version Register.

14.2.7 SD Controllers: ADMA Block Diagram and Descriptor Programming

14.2.7.1 ADMA: Descriptor Overview

The Descriptor Table is created by the host driver in system memory. Each descriptor line (one executable unit) consists of an address, length and attribute field:

- The attribute field specifies the operation of the descriptor line and is used to control the descriptor.
- There are three action symbols associated with the Attribute field: (1) **Nop** (No operation) skips the current descriptor line and fetches the next one; (2) **Tran** transfers data designated by an address and length field; and (3) **Link** connects two separated descriptors. For further information on these symbols, see Table 14-36 below. For Attribute[3:0] information, refer to Table 14-37.
- The address field for the link points to the next Descriptor Table.
- The address field is set on the 32-bit boundary (the lower 2 bits always are set to 0.
- One descriptor line consumes 64-bit (8-byte) memory space.
- For data length information see Table 14-35 below.

Address			Length			Reserved			Attribute					
63		32	31		16	15		06	05	04	03	02	01	00
32-bit Address			16-bit Length			10'b0			Act2	Act1	Word	Int	End	Valid

Table 14-34. SD Controllers: ADMA Descriptor Table.

Length Field	Data Length	
	Word = 0	Word = 1
0x0000	65536 bytes	65536 words = 262144 bytes
0x0001	1 byte	1 word = 4 bytes

Length Field	Data Length	
	Word = 0	Word = 1
0x0002	2 bytes	2 words = 8 bytes
0xFFFF	65535 bytes	65535 words = 262140 bytes

Table 14-35. SD Controllers: ADMA Descriptor Data Length Fields.

Attribute		Symbol	Comment	Operation
05	04			
Act2	Act1			
0	0	Nop	No operation	Do not execute current line and go to next line
0	1	Rsv	Reserved	Do not execute current line and go to next line
1	0	Tran	Transfer Data	Transfer data of one descriptor line
1	1	Link	Link Descriptor	Link to another descriptor

Table 14-36. SD Controllers: ADMA Descriptor Attribute[5:4] fields Act2 and Act1.

Attribute		Description
Bit	Name	
03	Word	0 - Indicates bytes are 16 bits in length, and the maximum data length that can be transferred in one descriptor is 65536 bytes = 64 KB 1 - Indicates words are 16 bits in length, the maximum data length that can be transferred in one descriptor is 65536 words = 64 K words = 256 KB.
02	Int	0 - Not triggered 1 - Generates DMA Interrupt when the operation of the descriptor line is completed
01	End	0 - Not defined 1 - Indicates end of the descriptor. The Transfer Complete Interrupt is generated when the operation of the descriptor line is completed.
00	Valid	0 - Generate ADMA Error Interrupt and stop ADMA to prevent runaways 1 - Indicates this line of descriptor is effective

Table 14-37. SD Controllers: ADMA Descriptor Attribute[3:0] fields Word, Int, End and Valid.

Confidential For Melfas Only (2015-10-29)

The following shows a block diagram of the CV22FS ADMA engine, including descriptor table detail.

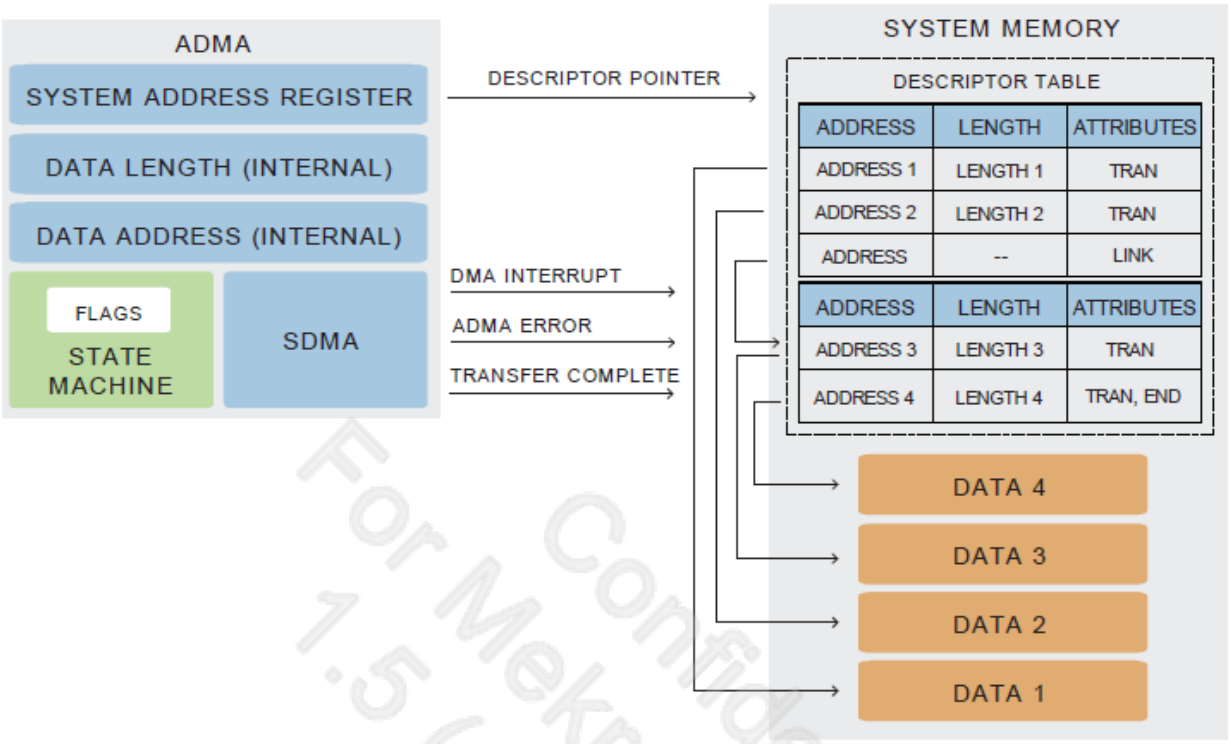


Figure 14-3. SD Controllers: Block Diagram Showing ADMA Transfer and Descriptor Table.

The ADMA engine includes the SDMA engine, a state machine and register circuits. The ADMA engine uses the Advanced DMA System Address register (offset 0x58) for the descriptor pointer rather than the SDMA System Address register (offset 0x00)

Writing to the Command register triggers an ADMA transfer. The ADMA engine fetches one descriptor line and executes it. This procedure is repeated until the end-of-descriptor is located (End=1 in the attribute field).

14.2.7.2 ADMA: Descriptor Programming

The figure below illustrates a typical ADMA descriptor program. The data area is sliced into various lengths, and each slice exists somewhere in system memory. The Descriptor Table consists of the address, length and attributes described by the host driver. The data is transferred slice-by-slice as programmed in the Descriptor Table.

Confidential
For Mekra Lang only
1.5 (08/12/25)

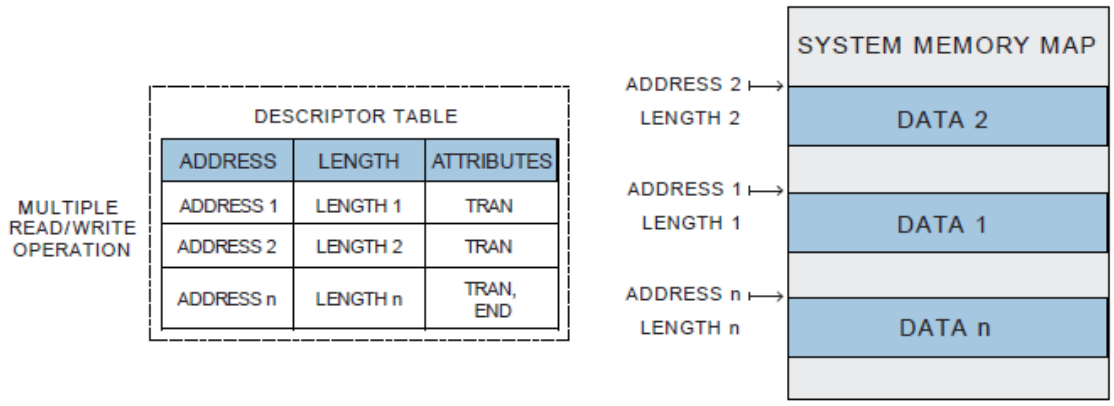


Figure 14-4. SD Controllers: ADMA Descriptor Table and Data Transfer Programming.

Notes:

- The minimum data length (unit) in each descriptor line is 4 bytes when the attribute Word=0.
- The minimum address (unit) is 4 bytes.
- The maximum data length per descriptor line is 64 KB or 256 KB depending on the Word attribute.
- Total Length = Length 1 + Length 2 + Length 3 + ... + Length n = a multiple of Block Size. Note that the ADMA transfer may fail to terminate if the total length of a descriptor is not a multiple of block size.

When operating the ADMA engine, set the Transfer Mode register (**SD_transfer_mode_and_command**) Block Count Enable bit (**block_count_enable**) to 1. The Block Count field (**block_count**) of the same register is a 16/32-bit register for non-CMD23/CMD23, and it limits transferred blocks to a maximum of 65535 / 0xFFFFFFFF.

The ADMA limits block transfer to 65535 / 0xFFFFFFFF for non-CMD23/CMD23 operation. Block Count registers (**SD_block_size_and_count/SD_block_count_cmd23**) specify the entire block count of an ADMA operation, which can include multiple descriptor lines. In this case of multiple descriptor lines, the total length of the Descriptor Table is related to block size and block count.

14.3 SD: Safety Notes

SD controller relies on pure SW safety mechanism. For any fault detected by SW, SW should set sw_arm_safety_error register in AXI Configuration so that bit-7 of safety errors in CEHU will be asserted.

15. SCRATCHPAD

The CV22FS non-secure and secure scratchpad modules provide the fundamental infrastructure to enable communication protocols, clock monitors control, and security and safety status. The secure scratchpad includes 8 KB RAM for general software purposes which the system uses primarily for DDR training.

Additionally, the scratchpad modules include a safety feature which supports lock access. Refer to section 15.1 for details.

This chapter includes the following sections:

- (Section 15.1) Scratchpad: Lock Access
- (Section 15.2) Scratchpad: Non-Secure Registers
- (Section 15.3) Scratchpad: Secure Registers

15.1 Scratchpad: Lock Access

The scratchpad modules support lock access for write transactions to internal control registers from AHB master.

The corresponding control registers of Lock Access are as follows:

- Non-secure region scratchpad: **AHBSP_NON- SEC_LOCK_ CTRL_NONSEC_ REG** (offset 0x0)
- Secure region scratchpad: **AHBSP_SEC_ LOCK_CTRL_ SEC_REG** (offset 0x20)

Using the non-secure region as an example, the following lists the flow for executing lock access:

1. At the reset, all accesses are allowed without any locking.
2. After initialization is complete, the software enables LOCKING by performing the following:
   - Write **AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG** register sp_nonsec_lock_wr_lock bit to 0.
   - Write **AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG** register sp_nonsec_lock bit to 1.
3. Next, all register write transactions perform the following:
   - Write **AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG** register sp_nonsec_lock_wr_lock bit to 0.
   - Write **AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG** register sp_nonsec_lock bit to 0.
   - Write to target register.
4. When sp_nonsec_lock bit is written, the sp_nonsec_lock_wr_lock bit will return to 1, or remain 0.
5. When the target register is written, sp_nonsec_lock returns to 1 for the LOCK state.

   If time finishes but the target register is not written, sp_nonsec_lock bit returns to 1. The timeout cycle is configured by sp_nonsec_timeout_upper_bound bits of the **AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG** register.
6. With the exception of the sp_nonsec_lock_wr_lock bit, all write transactions to the registers in LOCK state are not allowed and will issue an interrupt.

Confidential For Media Lab Use Only 2015

15.2 Scratchpad: Non-Secure Registers

This register space is addressable from the Arm Cortex-A53 and Cortex-R52 cores. The base address for this register space is E002.2000 for the non-secure region.

Offset	Register	Bits	Field	Access	Reset	Description
0x0	**AHBSP_NONSEC_LOCK_CTRL_NONSEC_REG**	31:12	**Reserved**	-	X	
		11:2	**sp_nonsec_timeout_upper_bound**	RW	0x3ff	Upper bound of the timeout mechanism. SW must write sp_nonsec_lock after sp_nonsec_lock_wr_lock being unlocked within sp_nonsec_timeout_upper_bound cycles.
		1	**sp_nonsec_lock_wr_lock**	RW	0x1	Write lock for sp_nonsec_lock. Must set this to 0 each time before writing to sp_nonsec_lock. 0: Unlocked state, write access to sp_nonsec_lock is allowed. 1: Locked state, write access to sp_nonsec_lock is forbidden.
		0	**sp_nonsec_lock**	RW	0x0	Lock accesses to ahb_scratchpad_nonsec 0: Unlocked state, write accesses to registers in ahb_scratchpad_nonsec are allowed. 1: Locked state, write accesses to registers in ahb_scratchpad_nonsec except “AHBSP_NONSEC_LOCK_CTRL_REG” & “AHBSP_NONSEC_APBDBG_LOCK_CTRL_REG” are forbidden.
0x4	**AHBSP_NONSEC_APBDBG_LOCK_CTRL_REG**	31:12	**Reserved**	-	X	
		11:2	**apbdbg_timeout_upper_bound**	RW	0x3ff	Upper bound of the timeout mechanism. SW must write apbdbg_lock after apbdbg_lock_wr_lock being unlocked within apbdbg_timeout_upper_bound cycles.
		1	**apbdbg_lock_wr_lock**	RW	0x1	Write lock for apbdbg_lock. Must set this to 0 every time before writing to apbdbg_lock. 0: Unlocked state, write access to apbdbg_lock is allowed. 1: Locked state, write access to apbdbg_lock is forbidden.
		0	**apbdbg_lock**	RW	0x0	Lock accesses to debug port clients 0: Unlocked state, write accesses to debug port clients are allowed. 1: Locked state, write accesses to debug port clients are forbidden.
0x8	**AHBSP_NONSEC_OTHER_PE_REG**	31:7	**Reserved**	-	X	
		6:0	**ahb_iopad_other_pull_en**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For other IO PAD)
0xc	**AHBSP_NONSEC_OTHER_PS_REG**	31:7	**Reserved**	-	X	
		6:0	**ahb_iopad_other_pull_en**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For other IO PAD)

Confidential
For Mekra Lang
1.5 (08.01.2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x14	**AHBSP_NON-SEC_GPIO_EN0_REG**	31:0	**ahb_iopad_gpio_pull_en0**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For IO PAD controlled in iomux)
0x18	**AHBSP_NON-SEC_GPIO_EN1_REG**	31:0	**ahb_iopad_gpio_pull_en1**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For IO PAD controlled in iomux)
0x1c	**AHBSP_NON-SEC_GPIO_EN2_REG**	31:0	**ahb_iopad_gpio_pull_en2**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For IO PAD controlled in iomux)
0x20	**AHBSP_NON-SEC_GPIO_EN3_REG**	6:0	**ahb_iopad_gpio_pull_en3**	RW	0x0	Active High Weak PU/PD Resistor Enable 0: PU/PD Disable 1: PU/PD Enable (For IO PAD controlled in iomux)
0x30	**AHBSP_NON-SEC_GPIO_SEL0_REG**	31:0	**ahb_iopad_gpio_pull_sel0**	RW	0x0	Weak PU/PD Resistor Selection 0: PD Selection 1: PU Selection (For IO PAD controlled in iomux)
0x34	**AHBSP_NON-SEC_GPIO_SEL1_REG**	31:0	**ahb_iopad_gpio_pull_sel1**	RW	0x0	Weak PU/PD Resistor Selection 0: PD Selection 1: PU Selection (For IO PAD controlled in iomux)
0x38	**AHBSP_NON-SEC_GPIO_SEL2_REG**	31:0	**ahb_iopad_gpio_pull_sel2**	RW	0x0	Weak PU/PD Resistor Selection 0: PD Selection 1: PU Selection (For IO PAD controlled in iomux)
0x3c	**AHBSP_NON-SEC_GPIO_SEL3_REG**	6:0	**ahb_iopad_gpio_pull_sel3**	RW	0x0	Weak PU/PD Resistor Selection 0: PD Selection 1: PU Selection (For IO PAD controlled in iomux)

For Ambarella Confidential Only (15 Oct 2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x50	**AHBSP_NON-SEC_DMA_CHANNEL0_SEL0_REG**	31:29	**Reserved**	-	X	
		28:24	**dma_channel0_3_sel**	RW	0x0	dma channel 0 req/ack selection 5'd1 ssi0_tx 5'd2 ssi0_rx 5'd3 ssi1_tx 5'd4 ssi1_rx 5'd5 nor_spi_tx 5'd6 nor_spi_rx 5'd7 ssis0_tx 5'd8 ssis0_rx 5'd9 uart0_tx 5'd10 uart0_rx 5'd11 uart1_tx 5'd12 uart1_rx 5'd13 uart2_tx 5'd14 uart2_rx 5'd15 uart3_tx 5'd16 uart3_rx 5'd17 ssi2_tx 5'd18 ssi2_rx 5'd19 ssi3_tx 5'd20 ssi3_rx 5'd21 i2s_tx 5'd22 i2s_rx 5'd23 ssi4_tx 5'd24 ssi4_rx 5'd25 ssi5_tx 5'd26 ssi5_rx
		23:21	**Reserved**	-	X	
		20:16	**dma_channel0_2_sel**	RW	0x0	Same as above (dma_channel0_3_sel)
		15:13	**Reserved**	-	X	
		12:8	**dma_channel0_1_sel**	RW	0x0	Same as above (dma_channel0_3_sel)
		7:5	**Reserved**	-	X	
		4:0	**dma_channel0_0_sel**	RW	0x0	Same as above (dma_channel0_3_sel)
0x54	**AHBSP_NON-SEC_DMA_CHANNEL0_SEL1_REG**	31:29	**Reserved**	-	X	
		28:24	**dma_channel0_7_sel**	RW	0x0	Same as 0x50
		23:21	**Reserved**	-	X	
		20:16	**dma_channel0_6_sel**	RW	0x0	Same as 0x50
		15:13	**Reserved**	-	X	
		12:8	**dma_channel0_5_sel**	RW	0x0	Same as 0x50
		7:5	**Reserved**	-	X	
		4:0	**dma_channel0_4_sel**	RW	0x0	Same as 0x50

Confidential
For Mekra Lang only
1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x58	**AHBSP_NON-SEC_DMA_ CHANNEL1_ SEL0_REG**	31:29	**Reserved**	-	X	
		28:24	**dma_chan-nel1_3_sel**	RW	0x0	dma channel 1 req/ack selection 5'd1 ssi0_tx 5'd2 ssi0_rx 5'd3 ssi1_tx 5'd4 ssi1_rx 5'd5 nor_spi_tx 5'd6 nor_spi_rx 5'd7 ssis0_tx 5'd8 ssis0_rx 5'd9 uart0_tx 5'd10 uart0_rx 5'd11 uart1_tx 5'd12 uart1_rx 5'd13 uart2_tx 5'd14 uart2_rx 5'd15 uart3_tx 5'd16 uart3_rx 5'd17 ssi2_tx 5'd18 ssi2_rx 5'd19 ssi3_tx 5'd20 ssi3_rx 5'd21 i2s_tx 5'd22 i2s_rx 5'd23 ssi4_tx 5'd24 ssi4_rx 5'd25 ssi5_tx 5'd26 ssi5_rx
		23:21	**Reserved**	-	X	
		20:16	**dma_chan-nel1_2_sel**	RW	0x0	Same as above (dma_channel1_3_sel)
		15:13	**Reserved**	-	X	
		12:8	**dma_chan-nel1_1_sel**	RW	0x0	Same as above (dma_channel1_3_sel)
		7:5	**Reserved**	-	X	
		4:0	**dma_chan-nel1_0_sel**	RW	0x0	Same as above (dma_channel1_3_sel)
0x5C	**AHBSP_NON-SEC_DMA_ CHANNEL1_ SEL1_REG**	31:29	**Reserved**	-	X	
		28:24	**dma_chan-nel1_7_sel**	RW	0x0	Same as 0x58
		23:21	**Reserved**	-	X	
		20:16	**dma_chan-nel1_6_sel**	RW	0x0	Same as 0x58
		15:13	**Reserved**	-	X	
		12:8	**dma_chan-nel1_5_sel**	RW	0x0	Same as 0x58
		7:5	**Reserved**	-	X	
		4:0	**dma_chan-nel1_4_sel**	RW	0x0	Same as 0x58

Confidential
For Mekra Lang only
1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x60	AHBSP_NON-SEC_CTRL_REG	31	enet_gtx_clk_pol	RW	0x0	ENET output clock phase selection 0: Non-invert 1: Invert
		30	rct_vic_sd_cd	R	0x0	SD card card detection post debouncing signal
		29	rct_vic_sdio0_cd	R	0x0	SDIO card card detection post debouncing signal
		28	enet1_gtx_clk_pol	RW	0x0	ENET1 output clock phase selection. 0: Non-invert 1: Invert
		27	enet1_rst_ctrl_bypass	RW	0x0	Set 1 to bypass enet reset control module
		26	enet_rst_ctrl_bypass	RW	0x0	Set 1 to bypass enet reset control module
		25	i2s_sw_rst_l	RW	0x1	I2S software reset
		24	Reserved	-	X	Reserved
		23	enet_2nd_ref_clk_sel	RW	0x0	ENET PHY 2nd reference clock frequency selection. 1: 50 MHz 0: 25 MHz
		22	sbd1_flow_ctrl_i	RW	0x0	Sideband Flow Control When set high, instructs the GMAC to transmit PAUSE Control frames in Full-duplex mode. In half-duplex mode, the GMAC enables the back-pressure function until this signal is made low again.
		21	sbd_flow_ctrl_i	RW	0x0	Sideband Flow Control When set high, instructs the GMAC to transmit PAUSE Control frames in full-duplex mode. In half-duplex mode, the GMAC enables the back-pressure function until this signal is made low again.
		20	Reserved	-	X	Reserved
		19	Reserved	-	X	Reserved
		18	reset_chan0	RW	0x0	Reset fios_top channel 0
		17	Reserved	-	X	Reserved
		16	flash_clk_en	RW	0x1	Flash controller clock enable
		15	Reserved	-	X	Reserved
		14	enet1_tx_clk_enable	RW	0x1	clock gating signal of enet1 tx_clk
		13	ovrcur_pol_inv	RW	0x1	When set to 1, the polarity of app_prt_ovr_cur_i of usb_host will be inverted
		12	enet1_reset	RW	0x0	Active high reset signal of ethernet1 MAC IP
		11	enet1_rx_clk_pol_sel	RW	0x0	Select the polarity of enet1_clk_rx 0: enet1_clk_rx 1: Inverse of enet1_clk_rx

Confidential
For Meeta Langhe
15/08/2025

Offset	Register	Bits	Field	Access	Reset	Description
0x60 (Con'td)	AHBSP_NONSEC_CTRL_REG	10	uart_apb_sw_rst_l	RW	0x1	APB UART software reset
		9	uart3_sw_rst_l	RW	0x1	UART3 software reset
		8	uart2_sw_rst_l	RW	0x1	UART2 software reset
		7	uart1_sw_rst_l	RW	0x1	UART1 software reset
		6	uart0_sw_rst_l	RW	0x1	UART0 software reset
		5	nor_spi_sw_rst_l	RW	0x1	SPI NOR software reset
		4:3	Reserved	-	X	Reserved
		2	enet_tx_clk_enable	RW	0x1	Clock gating signal of enet tx_clk
		1	enet_reset	RW	0x0	Active high reset signal of Ethernet MAC IP
		0	enet_rx_clk_pol_sel	RW	0x0	Select the polarity of enet_clk_rx 0: enet_clk_rx 1: Inverse of enet_clk_rx
0x64	AHBSP_NONSEC_SET_INTERRUPT_REG	31:14	Reserved	-	X	Reserved
		13:0	amba_axi_software_irq	RW	0x0	AMBA to AXI software IRQ
0x68	AHBSP_NONSEC_CLEAR_INTERRUPT_REG	31:14	Reserved	-	X	Reserved
		13:0	amba_axi_software_irq	RW	0x0	AMBA to AXI software IRQ
0x6c	AHBSP_NONSEC_DATA0_REG	31:0	ahb_scratchdata0	RW	0	General-Purpose Scratchpad Register
0x70	AHBSP_NONSEC_DATA1_REG	31:0	ahb_scratchdata1	RW	0	General-Purpose Scratchpad Register
0x74	AHBSP_NONSEC_DATA2_REG	31:0	ahb_scratchdata2	RW	0	General-Purpose Scratchpad Register
0x78	AHBSP_NONSEC_DATA3_REG	31:0	ahb_scratchdata3	RW	0	General-Purpose Scratchpad Register
0x7c	AHBSP_NONSEC_SOFT_RESET_REG	31:2	Reserved	-	X	Reserved
		1	reset_n_clk_div_rgmii1	RW	0x1	ENET1 RGMII clock divider soft reset
		0	reset_n_clk_div_rgmii	RW	0x1	ENET RGMII clock divider soft reset
0x80	AHBSP_NONSEC_PTP_TIMESTAMP_L	31:0	ptp_timestamp_o_l	R	0x0	ENET PTP timestamp signal lower word
0x84	AHBSP_NONSEC_PTP_TIMESTAMP_H	31:0	ptp_timestamp_o_h	R	0x0	ENET PTP timestamp signal higher word
0x88	AHBSP_NONSEC_PTP1_TIMESTAMP_L	31:0	ptp1_timestamp_o_l	R	0x0	ENET1 PTP timestamp signal lower word
0x8c	AHBSP_NONSEC_PTP1_TIMESTAMP_H	31:0	ptp1_timestamp_o_h	R	0x0	ENET1 PTP timestamp signal higher word
0x90	AHBSP_NONSEC_USB_HOST_CTRL_REG	31	Reserved	-	X	

Confidential
For Early Longoing
Use Only 2015

Offset	Register	Bits	Field	Access	Reset	Description
0x90 (Cont'd)	**AHBSP_NON-SEC_USB_HOST_CTRL_REG**	30:25	**ss_fladj_val_host_i**	RW	0x20	Frame Length Adjustment Register This feature adjusts any offset from the clock source that drives the µSOF counter. The µSOF cycle time (number of µSOF counter clock periods to generate a µSOF micro-frame length) is equal to 59,488 plus this value. The default value is decimal 32 (0x20), which gives an SOF cycle time of 60,000 (each microframe has 60,000 bit times). Frame Length (decimal) FLADJ Value (decimal) 59488 0 (0x00) 59504 1 (0x01) 59520 2 (0x02) ... ... 59984 31 (0x1F) 60000 32 (0x20) 60496 63 (0x3F)
		24:23	**ss_fladj_val_5_i**	RW	0x3	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fladj_val_host_i for the adjustment value.
		22:21	**ss_fladj_val_4_i**	RW	0x0	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fladj_val_host_i for the adjustment value.
		20:19	**ss_fladj_val_3_i**	RW	0x0	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fladj_val_host_i for the adjustment value.
		18:17	**ss_fladj_val_2_i**	RW	0x0	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fladj_val_host_i for the adjustment value.
		16:15	**ss_fladj_val_1_i**	RW	0x0	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fladj_val_host_i for the adjustment value.

Confidential For Mekra Lang (2801025) 1.5

Offset	Register	Bits	Field	Access	Reset	Description
0x90 (Cont'd)	**AHBSP_NON-SEC_USB_HOST_CTRL_REG**	14:13	**ss_fladj_val_0_i**	RW	0x0	Frame Length Adjustment Register per Port This feature adjusts the frame length of the microframe per port. This value must be Same as that of ss_fladj_val_host_i, or the EHCI yields undefined results. See ss_fl-adj_val_host_i for the adjustment value.
		12:11	**ss_power_state_i**	RW	0x0	Power Management Power management for the current state output from PCI.
		10	**ss_power_state_valid_i**	RW	0x0	Power State Valid Active high input qualifier signal for ss_pow-er_state_i
		9	**ss_nxt_power_state_valid_i**	RW	0x0	Next Power Management State Valid Due to the difference between the host AHB and the PCI clocks, the ss_next_pow-er_state_i may not be in the correct state when input to the host controller. Therefore, ss_nxt_power_state_valid_i is used (as a synchronization signal) to validate the ss_next_power_state_i signal. When this signal is asserted, the ss_next_power_state_i input is valid.
		8	**ss_word_if_i**	RW	0x0	Word Interface Selects the data width of the UTMI/UTMI+ PHY interface. 1'b1: 16 -bit interface 1'b0: 8-bit interface
		7	**ss_simulation_mode_i**	RW	0x0	Simulation Mode When set to 1'b1, this bit sets the PHY in a non-driving mode so the EHCI can detect device connection. This signal is used only for simulation.
		6	**ss_autoppd_on_overcur_en_i**	RW	0x0	Auto Port Power Disable on Overcurren This strap signal enables automatic port power disable in the host controller. When this signal is active, if an over-current condi-tion is detected on a powered port and PPC is 1, the PP bit in each affected port is auto-matically transitioned by the host controller from a 1 to 0, removing power from the port. Note: If this strap signal is not high, then the software needs to disable port power when an over current condition occurs.
		5	**ss_ena_incr16_i**	RW	0x1	AHB Burst Type INCR16 Enable Enables the AHB master interface to utilize burst INCR16 when appropriate. 1'b1: Use INCR16 when appropriate 1'b0: Do not use INCR16; use other enabled INCRX bursts or unspecified length burst INCR

Confidential
For Mekra Lang
1.5 (08/12/2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x90 (Cont'd)	**AHBSP_NON-SEC_USB_HOST_CTRL_REG**	4	**ss_ena_incr8_i**	RW	0x1	AHB Burst Type INCR8 Enable Enables the AHB master interface to utilize burst INCR8 when appropriate. 1'b1: Use INCR8 when appropriate 1'b0: Do not use INCR8; use other enabled INCRX bursts or unspecified length burst INCR
		3	**ss_ena_incr4_i**	RW	0x1	AHB Burst Type INCR4 Enable Enables the AHB master interface to utilize burst INCR8 when appropriate. 1'b1: Use INCR4 when appropriate 1'b0: Do not use INCR4; use other enabled INCRX bursts or unspecified length burst INCR
		2	**ss_ena_incrx_align_i**	RW	0x1	Burst Alignment Enable Forces AHB master to start INCR4/8/16 busts only on burst boundaries. AHB requires that double word width burst be addressed-aligned only to the double-word boundary. 1'b1: Start INCRX burst only on burst x-aligned addresses 1'b0: Normal AHB operation; start bursts on any double word boundary
		1	**ss_utmi_backward_enb_i**	RW	0x0	UTMI Backward Enable This signal controls the Resume termination sequence as follows: 1. When this signal is tied high, ss_resume_utmi_pls_dis_i is ignored. Before the xver_sel signal is switched to high-speed, the line state condition is validated for the 2 µs duration of SE0. 2. When this signal is tied low and ss_resume_utmi_pls_dis_i is high, Enhancement #9000043392 (from Release 2.3) is disabled. 3. When this signal is tied low and ss_resume_utmi_pls_dis_i is low, Enhancement #9000043392 (from Release 2.3) is enabled. The switching of term_sel is done first followed by xver_sel to create EOP as part of resume sequence. Since the term_sel which is xver_sel for UTMI+ PHY is switched during the last byte of transmit data (for sending resume, data is transmitted with bit stuff and NRZ disable) is coming on USB as garbage data and the device does not see a clean EOP for the resume sequence.

Confidential
For Mekra Lang GmbH
1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x90 (Cont'd)	**AHBSP_NON-SEC_USB_HOST_CTRL_REG**	0	**ss_resume_utmi_pls_dis_i**	RW	0x0	Resume Disable. This signal is valid only if ss_utmi_backward_enb_i is tied low. ss_resume_utmi_pls_dis_i controls the Resume termination sequence. If ss_resume_utmi_pls_dis_i is tied low, then the EHCI simultaneously switches term_sel[1:0] to 2'b00 and xver_sel to 1'b0. This transition occurs if either of the following two conditions is satisfied: SE0 is detected on line_state or 255 PHY clocks elapse after txvalid is deasserted. If ss_resume_utmi_pls_dis_i is tied high, at the end of resume, txvalid is deasserted and term_sel[1:0] is switched simultaneously to 2'b00. At this point, xver_sel is still high. After line_state is SE0 or if 255 PHY clocks elapse after txvalid is deasserted, xver_sel switches to 1'b0.
0x94	**AHBSP_NON-SEC_USB_EHCI_SIDEBAND_REG**	31:28	**Reserved**	-	X	Reserved
		27	**usbp_rct_chgdet0**	R	0x0	USB device connection status.
		26	**rct_vic_usb_connect**	R	0x0	USB device connection status.
		25:20	**ehci_usbsts_o_reg**	R	X	USB Status EHCI USBSTS register[5:0] bits. This signal indicates pending interrupts and various host controller statuses. These 6 bits reflect the value in the USBSTS[5:0] register.
		19	**ehci_pme_status_o_reg**	R	X	PME Status This signal displays the PME status. This value is updated in the PCI PMCSR's pme_status bit.
		18	**ehci_power_state_ack_o_reg**	R	X	Power State ACK This signal indicates the power state change acknowledgement from the EHCI to the PCI for PCI power management.
		17:7	**ehci_xfer_cnt_o_reg**	R	X	Transfer Count Transfer byte count from the EHCI master of the current AHB transaction. This is a constant signal and its value is valid when the EHCI starts its AHB address phase. The host controller aborts an AHB transfer before completing the transactions specified in ehci_xfer_cnt when a data read transaction is in progress and both these conditions are true: • There is insufficient time to complete the USB transfer • The Root Hub detects an underrun condition In CONFIG1 mode, this signal must be used only if Break Memory Transfer (bit 0 of INSNREG03) is set.

For Elektra Langley Confidential 08012025

Offset	Register	Bits	Field	Access	Reset	Description
0x94 (Cont'd)	**AHBSP_NON-SEC_USB_EHCI_SIDEBAND_REG**	6	**ehci_bufacc_o_reg**	R	X	EHCI Buffer Access Level signal that is asserted whenever the host controller does a data read/write transfer. If this signal is deasserted while the AHB is active, the host controller is performing a descriptor read/write. This signal can be used to support Big Endian operation.
		5:2	**ehci_lpsmc_state_o_reg**	R	X	LPSMC State This signal indicates the state of the LPSMC module. It is used only for debugging.
		1:0	**ss_next_power_state_i**	RW	0x0	Next Power Management State Power management for the next state output from PCI
0x98	**AHBSP_NON-SEC_USB_OHCI_SIDEBAND_REG**	31:23	**Reserved**	-	X	Reserved
		22:21	**ohci_0_speed_o_reg**	R	X	OHCI USB Speed In the current database, this signal is not required for UTMI connection. It has been replaced with utmi_term_select.
		20:19	**ohci_0_suspend_o_reg**	R	X	OHCI Suspend In the current database, this signal is not required for UTMI connection. It has been replaced with utmi_suspend.
		18:17	**ohci_0_ccs_o_reg**	R	X	Current Connect Status of Each Port When set, this bit indicates that the port state machine is in a connected state. If the bit is cleared, the port state machine is in either a disconnected or powered-off state. Note that this is a status signal that the application can ignore under normal host controller operation. It is also used in external clock start/stop logic.
		16	**ohci_0_bufacc_o_reg**	R	X	Host Controller Buffer Access Indicator When active, this signal indicates that the host controller is currently accessing the data buffer indicated by the TD. This is simply a status signal to let the application know whether the host controller is reading from or writing to the data buffer indicated by TD, or reading from or writing to the ED, TD descriptor, etc. If this is set (1) during the cycle on the HCI master bus, it indicates buffer fetch and if reset (0), all other transfers (such as ED, TD fetch, and Status Writeback). This is a status signal; the application need not take any action when it is asserted.

Confidential for Keka Langat 1.5 / 08012025

Offset	Register	Bits	Field	Access	Reset	Description
0x98 (Cont'd)	**AHBSP_NON-SEC_USB_OHCI_SIDEBAND_REG**	15	**ohci_0_rmtwkp_o_reg**	R	X	Remote Wakeup Status
		14	**ohci_0_irq_o_n_reg**	R	X	HCI Bus General Interrupt This is one of two interrupts the host controller uses to notify the Host Controller Driver when an interrupt condition occurs. If the application bus is PCI, this must be tied to a standard PCI interrupt pin. The host controller uses this pin when the HcControl.IR bit is set to zero.
		13	**ohci_0_smi_o_n_reg**	R	X	HCI Bus System Management Interrupt This is one of the two interrupts the host controller uses to notify the Host Controller Driver when an interrupt condition occurs. The host controller uses this pin when the HcControl.IR bit is set to one.This signal is used only when Legacy Support is provided. Refer to OpenHCI 1.0a for legacy specifications.
		12	**ohci_0_sof_o_n_reg**	R	X	Host Controller's New Frame The host controller asserts this signal for one clock cycle, whenever its internal Frame Counter (HcFmRemaining) reaches 0 while it is in the Operational state. At this time, HcFmRemaining is reloaded with HcFmInterval. (Note that the reset value of HcFmInterval is 0.) On the next clock cycle, the first bit of SOF (first bit of the Sync Field for the SOF token) is sent to the USB. This signal's assertion alerts the application to the new frame and to the SOF token being sent onto the USB. The application need not take action. The host controller generates this signal only when it is in the Operational state and sending SOF tokens.
		11	**ohci_0_lgcy_irq1_o_reg**	R	X	OHCI Legacy IRQ1 This bit is set to 1'b1 when an emulation interrupt condition exists and OutputFull, IRQEn, and AuxOutputFull are set to 1'b1.
		10	**ohci_0_lgcy_irq12_o_reg**	R	X	OHCI Legacy IRQ12 This bit is set to 1'b1 when an emulation interrupt condition exists, OutputFull and IRQEn are 1'b1, and AuxOutputFull is 1'b0.
		9	**ohci_0_lgcy_emu_en_o_reg**	R	X	This signal indicates that Legacy emulation support is enabled for the OHCI controller and the application can read or write I/O ports 60h/64h when I/O access for the OHCI controller is enabled.

Confidential For Mekra Lang GmbH & Co. KG (08012025) 1.5

Offset	Register	Bits	Field	Access	Reset	Description
0x98 (Cont'd)	**AHBSP_NON-SEC_USB_OHCI_SIDEBAND_REG**	8	**ohci_0_globalsuspend_o_reg**	R	X	Host Controller is in Global Suspend State This signal is asserted 5 ms after the host controller enters the USB Suspend state, and is asserted as long as the host controller remains in this state. The host controller enters the USB Suspend state when the Host Controller Driver forces it by writing to the HcControl.HCFS bits. The host controller exits this state when either the Host Controller Driver moves it to USB Reset (Global USB Reset) or USB Resume (Global Resume) or when a remote wake-up event is seen at one of the downstream USB ports.
		7	**ohci_0_drwe_o_reg**	R	X	Device Remote Wake-Up Enable This signal reflects the HcRhStatus.DRWE bit. When active, it causes the host controller to treat a connect or disconnect event as a remote wake-up, which in turn causes the host controller to enter the Global Resume state from the Global Suspend state. If this bit is cleared, a connect or disconnect event is not treated as a remote wake-up event.
		6	**ohci_0_rwe_o_reg**	R	X	Remote Wake Up Enable This bit reflects the HcControl.RWE bit. This is brought out as a status signal and can be ignored by the application for normal host controller operation.
		5	**ohci_susp_lgcy_i**	RW	0x0	OHCI Clock Control Signal This is a static strap signal. • When tied HIGH and the USB port is owned by OHCI, the signal utmi_suspend_o_n reflects the status of the USB port: (suspended or not suspended). • When tied LOW and the USB port is owned by OHCI, then - utmi_suspend_o_n asserts (0) if all the OHCI ports are suspended, or if the OHCI is in global suspend state (HCFS=USBSUSOPEND). - utmi_suspend_o_n deasserts (1) if any of the OHCI ports are not suspended and OHCI is not in global suspend.
		4	**ohci_0_app_io_hit_i**	RW	0x0	Application I/O Hit This signal indicates a PCI I/O cycle strobe. (This signal is relevant only when using a PCI controller.)
		3	**ohci_0_app_irq1_i**	RW	0x0	External Interrupt 1 This external keyboard controller interrupt 1 causes an emulation interrupt.

Confidential
For Mekta Langer
1.5 - 08/12/2025

Offset	Register	Bits	Field	Access	Reset	Description
0x98 (Cont'd)	**AHBSP_NONSEC_USB_OHCI_SIDEBAND_REG**	2	**ohci_0_app_irq12_i**	RW	0x0	External Interrupt 12 This external keyboard controller interrupt 12 causes an emulation interrupt.
		1	**ohci_0_cntsel_i_n**	RW	0x0	Count Select Selects the counter value for simulation or real time for 1 ms. 1'b0: Count full 1 ms 1'b1: Simulation time
		0	**app_start_clk_i**	RW	0x0	OHCI Clock Control Signal This is an asynchronous primary input to the host core. When the OHCI clocks are suspended, the system has to assert this signal to start the clocks (12 and 48 MHz). This should be deasserted after the clocks are started and before the host is suspended again. (Host is suspended means HCFS = SUSPEND or all the OHCI ports are suspended).
0xa0	**AHBSP_NONSEC_GMII_DATA_REG**	31:16	**gmii1_data_reg**	RW	0x0	Write data for ahb_enet1_mdcmdio *Please note that this register (AHBSP_NONSEC_GMII_DATA_REG) take effect only if ENET_MDC and ENET_MDIO pins are switched to the fifth alternative function (ahb_mdc and ahb_mdio).
		15:0	**gmii_data_reg**	RW	0x0	Write data for ahb_enet_mdcmdio
0xa4	**AHBSP_NONSEC_GMII_ADDR_REG**	31:27	**gmii1_pa**	RW	0x0	Physical Layer Address This field indicates which of the 32 possible PHY devices are being accessed. *Please note that this register (AHBSP_NONSEC_GMII_ADDR_REG) take effect only if ENET_MDC and ENET_MDIO pins are switched to the fifth alternative function (ahb_mdc and ahb_mdio).
		26:22	**gmii1_gr**	RW	0x0	GMII Register These bits select the desired GMII register in the selected PHY device.
		21:18	**gmii1_cr**	RW	0x0	Clock Divider MDC clock rate will be generated with the frequency of 12/(cr+1) MHz. In general, it should be set to 4.
		17	**gmii1_gw**	RW	0x0	GMII Write When set, this bit indicates to the PHY that this will be a Write operation using the GMII Data register (AHBSP_NONSEC_GMII_ADDR_REG). If this bit is not set, this will be a Read operation, resulting in the placement of data in the GMII Data register.

Confidential For Mekro Langat 1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0xa4 (Cont'd)	**AHBSP_NONSEC_GMII_ADDR_REG**	16	**gmii1_gb**	RW	0x0	GMII Busy This bit should read a logic 0 before writing to AHBSP_NONSEC_GMII_ADDR_REG and the GMII Data register (AHBSP_NONSEC_GMII_DATA_REG). This bit must also be set to 1 during a Write to AHBSP_NONSEC_GMII_ADDR_REG to kick-off a PHY register Read or Write access. AHBSP_NONSEC_GMII_DATA_REG (GMII Data) should be kept valid until this bit is cleared by itself during a PHY Write operation. The AHBSP_NONSEC_GMII_DATA_REG bit is invalid until cleared by itself during a PHY Read operation. The AHBSP_NONSEC_GMII_ADDR_REG should not be written to until this bit is cleared.
		15:11	**gmii_pa**	RW	0x0	Physical Layer Address This field indicates which of the 32 possible PHY devices are being accessed.
		10:6	**gmii_gr**	RW	0x0	GMII Register These bits select the desired GMII register in the selected PHY device.
		5:2	**gmii_cr**	RW	0x0	Clock Divider MDC clock rate will be generated with the frequency of 12/(cr+1) MHz. In general, it should be set to 4.
		1	**gmii_gw**	RW	0x0	GMII Write When set, this bit indicates to the PHY that this will be a Write operation using the GMII Data register (AHBSP_NONSEC_GMII_ADDR_REG). If this bit is not set, this will be a Read operation, resulting in the placement of data in the GMII Data register.
		0	**gmii_gb**	RW	0x0	GMII Busy This bit should read a logic 0 before writing to AHBSP_NONSEC_GMII_ADDR_REG and the GMII Data register (AHBSP_NONSEC_GMII_DATA_REG). This bit must also be set to 1 during a Write to AHBSP_NONSEC_GMII_ADDR_REG to kick-off a PHY register Read or Write access. AHBSP_NONSEC_GMII_DATA_REG (GMII Data) should be kept valid until this bit is cleared by itself during a PHY Write operation. The AHBSP_NONSEC_GMII_DATA_REG bit is invalid until cleared by itself during a PHY Read operation. The AHBSP_NONSEC_GMII_ADDR_REG should not be written to until this bit is cleared.

Confidential
For Mekta Langkah
1:5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0xb0	**AHBSP_NON-SEC_SEC_ERR0_REG**	31	**sec_err_scratch-pad_sec**	RW	0x0	Security Access Error (non-secure CPU accesses secure IP) 1: Error 0: Normal Write “1” to clear the register field
		30	**sec_err_scratch-pad_nsec**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		29	**sec_err_dma1**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		28	**sec_err_dma0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		27	**sec_err_ehci**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		26	**sec_err_ohci**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		25	**sec_err_dai**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		24	**sec_err_fio8k_hi**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		23	**sec_err_fio8k_lo**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		22	**sec_err_uart3**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		21	**sec_err_uart2**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		20	**sec_err_uart1**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		19	**sec_err_uart0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		18	**sec_err_cehu1**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		17	**sec_err_cehu0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		16	**sec_err_ssi3**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		15	**sec_err_ssi2**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		14	**sec_err_ssi1**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		13	**sec_err_ssi0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		12	**sec_err_ssis0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		11	**sec_err_enet1**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		10	**sec_err_amba_ecru**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		9	**sec_err_gdma**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		8	**sec_err_enet**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		7	**sec_err_vout**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		6	**sec_err_usb**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		5	**sec_err_sdio0**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		4	**sec_err_sd**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		3	**sec_err_timer2**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		2	**sec_err_flash_cpu**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		1	**sec_err_nor_spi**	RW	0x0	Same as above (sec_err_scratchpad_sec)
		0	**sec_err_cpuid**	RW	0x0	Same as above (sec_err_scratchpad_sec)

Offset	Register	Bits	Field	Access	Reset	Description
0xb4	**AHBSP_NON-SEC_SEC_ERR1_REG**	31	**sec_err_ssi5**	RW	0x0	Same as 0xb0
		30	**sec_err_ssi4**	RW	0x0	Same as 0xb0
		29	**sec_err_wdt_r52**	RW	0x0	Same as 0xb0
		28	**sec_err_wdt_core3**	RW	0x0	Same as 0xb0
		27	**sec_err_wdt_core2**	RW	0x0	Same as 0xb0
		26	**sec_err_wdt_core1**	RW	0x0	Same as 0xb0
		25	**sec_err_wdt_core0**	RW	0x0	Same as 0xb0
		24	**sec_err_can5**	RW	0x0	Same as 0xb0
		23	**sec_err_can4**	RW	0x0	Same as 0xb0
		22	**sec_err_can3**	RW	0x0	Same as 0xb0
		21	**sec_err_can2**	RW	0x0	Same as 0xb0
		20	**sec_err_gpio3**	RW	0x0	Same as 0xb0
		19	**sec_err_gpio2**	RW	0x0	Same as 0xb0
		18	**sec_err_gpio1**	RW	0x0	Same as 0xb0
		17	**sec_err_gpio0**	RW	0x0	Same as 0xb0
		16	**sec_err_wdt**	RW	0x0	Same as 0xb0
		15	**sec_err_iomux**	RW	0x0	Same as 0xb0
		14	**sec_err_idc5**	RW	0x0	Same as 0xb0
		13	**sec_err_idc4**	RW	0x0	Same as 0xb0
		12	**sec_err_idc3**	RW	0x0	Same as 0xb0
		11	**sec_err_idc2**	RW	0x0	Same as 0xb0
		10	**sec_err_idc1**	RW	0x0	Same as 0xb0
		9	**sec_err_idc0**	RW	0x0	Same as 0xb0
		8	**sec_err_idcs**	RW	0x0	Same as 0xb0
		7	**sec_err_timer1**	RW	0x0	Same as 0xb0
		6	**sec_err_timer0**	RW	0x0	Same as 0xb0
		5	**sec_err_pwm2**	RW	0x0	Same as 0xb0
		4	**sec_err_pwm1**	RW	0x0	Same as 0xb0
		3	**sec_err_pwm0**	RW	0x0	Same as 0xb0
		2	**sec_err_uart**	RW	0x0	Same as 0xb0
		1	**sec_err_can1**	RW	0x0	Same as 0xb0
		0	**sec_err_can0**	RW	0x0	Same as 0xb0

Offset	Register	Bits	Field	Access	Reset	Description
0xb8	**AHBSP_NON-SEC_SEC_ERR2_REG**	31:28	**Reserved**	-	X	
		27	**sec_err_dbg_dbse**	RW	0x0	Same as 0xb0
		26	**sec_err_dbg_fex**	RW	0x0	Same as 0xb0
		25	**sec_err_dbg_ist**	RW	0x0	Same as 0xb0
		24	**Reserved**	-	X	
		23	**Reserved**	-	X	
		22	**Reserved**	-	X	
		21	**sec_err_dbg_fmem**	RW	0x0	Same as 0xb0
		20	**Reserved**	-	X	
		19	**sec_err_dbg_vmem0_pmem**	RW	0x0	Same as 0xb0
		18	**sec_err_dbg_vp0**	RW	0x0	Same as 0xb0
		17	**Reserved**	-	X	
		16	**sec_err_dbg_mipi**	RW	0x0	Same as 0xb0
		15	**sec_err_dbg_mdxf**	RW	0x0	Same as 0xb0
		14	**Reserved**	-	X	
		13	**Reserved**	-	X	
		12	**sec_err_dbg_idsp**	RW	0x0	Same as 0xb0
		11	**Reserved**	-	X	
		10	**sec_err_dbg_vmem0**	RW	0x0	Same as 0xb0
		9	**sec_err_dbg_rtc**	RW	0x0	Same as 0xb0
		8	**sec_err_dbg_dorc**	RW	0x0	Same as 0xb0
		7	**Reserved**	-	X	
		6	**sec_err_dbg_I2c**	RW	0x0	Same as 0xb0
		5	**sec_err_dbg_dram**	RW	0x0	Same as 0xb0
		4	**sec_err_dbg_rct**	RW	0x0	Same as 0xb0
		3	**sec_err_dbg_smem**	RW	0x0	Same as 0xb0
		2	**sec_err_dbg_corc**	RW	0x0	Same as 0xb0
		1	**sec_err_dbg_eorc**	RW	0x0	Same as 0xb0
		0	**sec_err_dbg_vorc**	RW	0x0	Same as 0xb0
0xbc	**AHBSP_NON-SEC_RESERVED_REG**	31:0	**ahb_nonsec_re-served_reg**	RW	0x0	Reserved register for ECO

Confidential Ambarella Inc. 2015

Offset	Register	Bits	Field	Access	Reset	Description
0xc0	**AHBSP_NON-SEC_SAF_ERR0_REG**	31	**saf_err_scratch-pad_sec**	RW	0x0	CPU Access Error (CA53 access module which is set by register in axi_config and is only allowed to be accessed by R52 CPU) 1: error, 0: normal Write “1” to clear the register field
		30	**saf_err_scratch-pad_nsec**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		29	**saf_err_dma1**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		28	**saf_err_dma0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		27	**saf_err_ehci**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		26	**saf_err_ohci**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		25	**saf_err_dai**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		24	**saf_err_fio8k_hi**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		23	**saf_err_fio8k_lo**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		22	**saf_err_uart3**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		21	**saf_err_uart2**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		20	**saf_err_uart1**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		19	**saf_err_uart0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		18	**saf_err_cehu1**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		17	**saf_err_cehu0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		16	**saf_err_ssi3**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		15	**saf_err_ssi2**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		14	**saf_err_ssi1**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		13	**saf_err_ssi0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		12	**saf_err_ssis0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		11	**saf_err_enet1**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		10	**saf_err_amba_ecru**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		9	**saf_err_gdma**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		8	**saf_err_enet**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		7	**saf_err_vout**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		6	**saf_err_usb**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		5	**saf_err_sdio0**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		4	**saf_err_sd**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		3	**saf_err_timer2**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		2	**saf_err_flash_cpu**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		1	**saf_err_nor_spi**	RW	0x0	Same as above (saf_err_scratchpad_sec)
		0	**saf_err_cpuid**	RW	0x0	Same as above (saf_err_scratchpad_sec)

Offset	Register	Bits	Field	Access	Reset	Description
0xc4	**AHBSP_NON-SEC_SAF_ERR1_REG**	31	**saf_err_ssi5**	RW	0x0	Same as 0xc0
		30	**saf_err_ssi4**	RW	0x0	Same as 0xc0
		29	**saf_err_wdt_r52**	RW	0x0	Same as 0xc0
		28	**saf_err_wdt_core3**	RW	0x0	Same as 0xc0
		27	**saf_err_wdt_core2**	RW	0x0	Same as 0xc0
		26	**saf_err_wdt_core1**	RW	0x0	Same as 0xc0
		25	**saf_err_wdt_core0**	RW	0x0	Same as 0xc0
		24	**saf_err_can5**	RW	0x0	Same as 0xc0
		23	**saf_err_can4**	RW	0x0	Same as 0xc0
		22	**saf_err_can3**	RW	0x0	Same as 0xc0
		21	**saf_err_can2**	RW	0x0	Same as 0xc0
		20	**saf_err_gpio3**	RW	0x0	Same as 0xc0
		19	**saf_err_gpio2**	RW	0x0	Same as 0xc0
		18	**saf_err_gpio1**	RW	0x0	Same as 0xc0
		17	**saf_err_gpio0**	RW	0x0	Same as 0xc0
		16	**saf_err_wdt**	RW	0x0	Same as 0xc0
		15	**saf_err_iomux**	RW	0x0	Same as 0xc0
		14	**saf_err_idc5**	RW	0x0	Same as 0xc0
		13	**saf_err_idc4**	RW	0x0	Same as 0xc0
		12	**saf_err_idc3**	RW	0x0	Same as 0xc0
		11	**saf_err_idc2**	RW	0x0	Same as 0xc0
		10	**saf_err_idc1**	RW	0x0	Same as 0xc0
		9	**saf_err_idc0**	RW	0x0	Same as 0xc0
		8	**saf_err_idcs**	RW	0x0	Same as 0xc0
		7	**saf_err_timer1**	RW	0x0	Same as 0xc0
		6	**saf_err_timer0**	RW	0x0	Same as 0xc0
		5	**saf_err_pwm2**	RW	0x0	Same as 0xc0
		4	**saf_err_pwm1**	RW	0x0	Same as 0xc0
		3	**saf_err_pwm0**	RW	0x0	Same as 0xc0
		2	**saf_err_uart**	RW	0x0	Same as 0xc0
		1	**saf_err_can1**	RW	0x0	Same as 0xc0
		0	**saf_err_can0**	RW	0x0	Same as 0xc0

Offset	Register	Bits	Field	Access	Reset	Description
0xc8	**AHBSP_NON-SEC_SAF_ERR2_REG**	31:28	**Reserved**	-	X	
		27	**saf_err_dbg_dbse**	RW	0x0	Same as 0xc0
		26	**saf_err_dbg_fex**	RW	0x0	Same as 0xc0
		25	**saf_err_dbg_ist**	RW	0x0	Same as 0xc0
		24	**Reserved**	-	X	
		23	**Reserved**	-	X	
		22	**Reserved**	-	X	
		21	**saf_err_dbg_fmem**	RW	0x0	Same as 0xc0
		20	**Reserved**	-	X	
		19	**saf_err_dbg_vmem0_pmem**	RW	0x0	Same as 0xc0
		18	**saf_err_dbg_vp0**	RW	0x0	Same as 0xc0
		17	**Reserved**	-	X	
		16	**saf_err_dbg_mipi**	RW	0x0	Same as 0xc0
		15	**saf_err_dbg_mdxf**	RW	0x0	Same as 0xc0
		14	**Reserved**	-	X	
		13	**Reserved**	-	X	
		12	**saf_err_dbg_idsp**	RW	0x0	Same as 0xc0
		11	**Reserved**	-	X	
		10	**saf_err_dbg_vmem0**	RW	0x0	Same as 0xc0
		9	**saf_err_dbg_rtc**	RW	0x0	Same as 0xc0
		8	**saf_err_dbg_dorc**	RW	0x0	Same as 0xc0
		7	**Reserved**	-	X	
		6	**saf_err_dbg_I2c**	RW	0x0	Same as 0xc0
		5	**saf_err_dbg_dram**	RW	0x0	Same as 0xc0
		4	**saf_err_dbg_rct**	RW	0x0	Same as 0xc0
		3	**saf_err_dbg_smem**	RW	0x0	Same as 0xc0
		2	**saf_err_dbg_corc**	RW	0x0	Same as 0xc0
		1	**saf_err_dbg_eorc**	RW	0x0	Same as 0xc0
		0	**saf_err_dbg_vorc**	RW	0x0	Same as 0xc0
0xd0	**AHBSP_NON-SEC_DUMMY_REG**	31:0	**ahb_nonsec_dummy_reg**	RW	0xbeef-babe	For safety purposes To detect if AHB reset source remains at inactive state with high effectiveness
0xd4	**AHBSP_NON-SEC_AXI0_DUM-MY_REG**	31:0	**ahb_nonsec_axi0_dummy_reg**	RW	0xbeef-babe	For safety purposes To detect if AXI0 reset source remains at inactive state with high effectiveness.

Confidential
For Ambarella Internal Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0xd8	AHBSP_NON-SEC_INSERT_ER-ROR_REG	31:9	Reserved	-	X	
		8	vsdelay_valid_ clk_insert_error	RW	0x0	Set this field to 1 to insert error to vsdelay_ valid_clk_sm_error.
		7	vsdelay_valid_ clk_config_in-sert_error	RW	0x0	Set this field to 1 to insert error to vsdelay_ valid_clk_config_sm_error.
		6	nic400_ahb_in-sert_error	RW	0x0	Set this field to 1 to insert error to nic400_ ahb_parity.
		5	apb_dbg_insert_ error	RW	0x0	Set this field to 1 to insert error to apb_dbg_ lockstep_top.
		4	apbbrg_insert_er-ror	RW	0x0	Set this field to 1 to insert error to apbbrg_ lockstep_top.
		3	vrom_ahb_in-sert_error	RW	0x0	Set this field to 1 to insert error to vrom_ahb_ lockstep_top.
		2	ahb_scratchpad_ nsec_insert_error	RW	0x0	Set this field to 1 to insert error to ahb_ scratchpad_nsec.
		1	ahb_bus_insert_ error	RW	0x0	Set this field to 1 to insert error to ahb_bus_ lockstep_top.
		0	dma_ahb_insert_ error	RW	0x0	Set this field to 1 to insert error to dma_ahb_ lockstep_top.
0xe0	AHBSP_NON-SEC_CKM_ CTRL0_REG	31	ckm_enable_ gclk_ssi_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_ bound_gclk_ssi_ dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_ bound_gclk_ssi_ dft	RW	0x0	Lower bound of clock monitor compare value
0xe4	AHBSP_NON-SEC_CKM_ CTRL1_REG	31	ckm_enable_ gclk_ssi2_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_ bound_gclk_ssi2_ dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_ bound_gclk_ssi2_ dft	RW	0x0	Lower bound of clock monitor compare value
0xe8	AHBSP_NON-SEC_CKM_ CTRL2_REG	31	ckm_enable_ gclk_ssi3_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_ bound_gclk_ssi3_ dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_ bound_gclk_ssi3_ dft	RW	0x0	Lower bound of clock monitor compare value

For Confidential Customer Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0xec	AHBSP_NON-SEC_CKM_CTRL3_REG	31	ckm_enable_gclk_uart0_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_uart0_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_uart0_dft	RW	0x0	Lower bound of clock monitor compare value
0xf0	AHBSP_NON-SEC_CKM_CTRL4_REG	31	ckm_enable_gclk_uart1_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_uart1_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_uart1_dft	RW	0x0	Lower bound of clock monitor compare value
0xf4	AHBSP_NON-SEC_CKM_CTRL5_REG	31	ckm_enable_gclk_uart2_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_uart2_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_uart2_dft	RW	0x0	Lower bound of clock monitor compare value
0xf8	AHBSP_NON-SEC_CKM_CTRL6_REG	31	ckm_enable_gclk_uart3_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_uart3_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_uart3_dft	RW	0x0	Lower bound of clock monitor compare value
0xfc	AHBSP_NON-SEC_CKM_CTRL7_REG	31	ckm_enable_gclk_uart_apb_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_uart_apb_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_uart_apb_dft	RW	0x0	Lower bound of clock monitor compare value

Confidential For Ambarella Internal Only (c) 2025

Offset	Register	Bits	Field	Access	Reset	Description
0x100	**AHBSP_NON-SEC_CKM_CTRL8_REG**	31	**ckm_enable_gclk_core_ahb**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	X	
		27:16	**ckm_upper_bound_gclk_core_ahb**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	X	
		11:0	**ckm_lower_bound_gclk_core_ahb**	RW	0x0	Lower bound of clock monitor compare value
0x104	**AHBSP_NON-SEC_CKM_CTRL9_REG**	31	**ckm_enable_gclk_core_dft**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	X	
		27:16	**ckm_upper_bound_gclk_core_dft**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	X	
		11:0	**ckm_lower_bound_gclk_core_dft**	RW	0x0	Lower bound of clock monitor compare value
0x108	**AHBSP_NON-SEC_CKM_CTRL10_REG**	31	**ckm_enable_gclk_ahb_dft**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	X	
		27:16	**ckm_upper_bound_gclk_ahb_dft**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	X	
		11:0	**ckm_lower_bound_gclk_ahb_dft**	RW	0x0	Lower bound of clock monitor compare value
0x10c	**AHBSP_NON-SEC_CKM_CTRL11_REG**	31	**ckm_enable_gclk_apb_dft**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	X	
		27:16	**ckm_upper_bound_gclk_apb_dft**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	X	
		11:0	**ckm_lower_bound_gclk_apb_dft**	RW	0x0	Lower bound of clock monitor compare value
0x110	**AHBSP_NON-SEC_CKM_CTRL12_REG**	31	**ckm_enable_gclk_dbg_dft**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	X	
		27:16	**ckm_upper_bound_gclk_dbg_dft**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	X	
		11:0	**ckm_lower_bound_gclk_dbg_dft**	RW	0x0	Lower bound of clock monitor compare value

Confidential
Ambarella Internal Only
Feb 20, 2025

Offset	Register	Bits	Field	Access	Reset	Description
0x114	AHBSP_NON-SEC_CKM_CTRL13_REG	31	ckm_enable_gclk_sdio0_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_sdio0_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_sdio0_dft	RW	0x0	Lower bound of clock monitor compare value
0x118	AHBSP_NON-SEC_CKM_CTRL14_REG	31	ckm_enable_gclk_sd48_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_sd48_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_sd48_dft	RW	0x0	Lower bound of clock monitor compare value
0x11c	AHBSP_NON-SEC_CKM_CTRL15_REG	31	ckm_enable_gclk_nand	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_nand	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_nand	RW	0x0	Lower bound of clock monitor compare value
0x120	AHBSP_NON-SEC_CKM_CTRL16_REG	31	ckm_enable_n0_gclk_sys_cnt_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_n0_gclk_sys_cnt_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_n0_gclk_sys_cnt_dft	RW	0x0	Lower bound of clock monitor compare value
0x124	AHBSP_NON-SEC_CKM_CTRL17_REG	31	ckm_enable_n1_gclk_sys_cnt_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_n1_gclk_sys_cnt_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_n1_gclk_sys_cnt_dft	RW	0x0	Lower bound of clock monitor compare value

Confidential
Ambarella Only
Feb 2025

Offset	Register	Bits	Field	Access	Reset	Description
0x128	AHBSP_NON-SEC_CKM_CTRL18_REG	31	ckm_enable_gclk_pwm_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_pwm_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_pwm_dft	RW	0x0	Lower bound of clock monitor compare value
0x12c	AHBSP_NON-SEC_CKM_CTRL19_REG	31	ckm_enable_gclk_can_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_can_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_can_dft	RW	0x0	Lower bound of clock monitor compare value
0x130	AHBSP_NON-SEC_CKM_CTRL20_REG	31	ckm_enable_gclk_au_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_au_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_au_dft	RW	0x0	Lower bound of clock monitor compare value
0x134	AHBSP_NON-SEC_CKM_CTRL21_REG	31	ckm_enable_gclk_gtx_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_gtx_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_gtx_dft	RW	0x0	Lower bound of clock monitor compare value
0x138	AHBSP_NON-SEC_CKM_CTRL22_REG	31	ckm_enable_gclk_axi1_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_axi1_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_axi1_dft	RW	0x0	Lower bound of clock monitor compare value

Confidential For Ambarella Employee Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x13c	AHBSP_NON-SEC_CKM_CTRL23_REG	31	ckm_enable_gclk_vo_a_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_vo_a_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_vo_a_dft	RW	0x0	Lower bound of clock monitor compare value
0x140	AHBSP_NON-SEC_CKM_CTRL24_REG	31	ckm_enable_gclk_vo_b_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_gclk_vo_b_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_gclk_vo_b_dft	RW	0x0	Lower bound of clock monitor compare value
0x144	AHBSP_NON-SEC_CKM_CTRL25_REG	31	ckm_enable_combophy_clka_out_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_combo-phy_clka_out_dft	RW	0x0	Upper bound of clock monitor compare value
		15	Reserved	-	X	
		14:0	ckm_lower_bound_combo-phy_clka_out_dft	RW	0x0	Lower bound of clock monitor compare value
0x148	AHBSP_NON-SEC_CKM_CTRL26_REG	31	ckm_enable_combophy_clkb_out_dft	RW	0x0	Enable signal of clock monitor
		30:16	ckm_upper_bound_combo-phy_clkb_out_dft	RW	0x0	Upper bound of clock monitor compare value. *{;;;FULLRANGE only needs to config one time}
		15	Reserved	-	x	
		14:0	ckm_lower_bound_combo-phy_clkb_out_dft	RW	0x0	Lower bound of clock monitor compare value. *{;;;FULLRANGE only needs to config one time}
0x14c	AHBSP_NON-SEC_CKM_CTRL27_REG	31	ckm_enable_gclk_vision_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_vi-sion_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_vi-sion_dft	RW	0x0	Lower bound of clock monitor compare value

Confidential
For Ambarella Internal Only
2025

Offset	Register	Bits	Field	Access	Reset	Description
0x150	AHBSP_NON-SEC_CKM_CTRL28_REG	31	ckm_enable_gclk_dram_div2_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_dram_div2_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_dram_div2_dft	RW	0x0	Lower bound of clock monitor compare value
0x154	AHBSP_NON-SEC_CKM_CTRL29_REG	31	ckm_enable_gclk_fex_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_gclk_fex_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_gclk_fex_dft	RW	0x0	Lower bound of clock monitor compare value
0x158	AHBSP_NON-SEC_CKM_MEA_CYCLE0_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_ssi_dft	R	X	Current clock monitor counting value for debugging purposes
0x15c	AHBSP_NON-SEC_CKM_MEA_CYCLE1_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_ssi2_dft	R	X	Current clock monitor counting value for debugging purposes
0x160	AHBSP_NON-SEC_CKM_MEA_CYCLE2_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_ssi3_dft	R	X	Current clock monitor counting value for debugging purposes
0x164	AHBSP_NON-SEC_CKM_MEA_CYCLE3_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_uart0_dft	R	X	Current clock monitor counting value for debugging purposes
0x168	AHBSP_NON-SEC_CKM_MEA_CYCLE4_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_uart1_dft	R	X	Current clock monitor counting value for debugging purposes
0x16c	AHBSP_NON-SEC_CKM_MEA_CYCLE5_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_uart2_dft	R	X	Current clock monitor counting value for debugging purposes
0x170	AHBSP_NON-SEC_CKM_MEA_CYCLE6_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_uart3_dft	R	X	Current clock monitor counting value for debugging purposes
0x174	AHBSP_NON-SEC_CKM_MEA_CYCLE7_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_uart_apb_dft	R	X	Current clock monitor counting value for debugging purposes
0x178	AHBSP_NON-SEC_CKM_MEA_CYCLE8_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_core_ahb	R	X	Current clock monitor counting value for debugging purposes

Ambarella Confidential Limited Distribution Only

Offset	Register	Bits	Field	Access	Reset	Description
0x17c	AHBSP_NON-SEC_CKM_MEA_CYCLE9_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_core_dft	R	X	Current clock monitor counting value for debugging purposes
0x180	AHBSP_NON-SEC_CKM_MEA_CYCLE10_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_ahb_dft	R	X	Current clock monitor counting value for debugging purposes
0x184	AHBSP_NON-SEC_CKM_MEA_CYCLE11_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_apb_dft	R	X	Current clock monitor counting value for debugging purposes
0x188	AHBSP_NON-SEC_CKM_MEA_CYCLE12_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_dbg_dft	R	X	Current clock monitor counting value for debugging purposes
0x18c	AHBSP_NON-SEC_CKM_MEA_CYCLE13_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_sdio0_dft	R	X	Current clock monitor counting value for debugging purposes
0x190	AHBSP_NON-SEC_CKM_MEA_CYCLE14_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_sd48_dft	R	X	Current clock monitor counting value for debugging purposes
0x194	AHBSP_NON-SEC_CKM_MEA_CYCLE15_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_nand	R	X	Current clock monitor counting value for debugging purposes
0x198	AHBSP_NON-SEC_CKM_MEA_CYCLE16_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_n0_gclk_sys_cnt_dft	R	X	Current clock monitor counting value for debugging purposes
0x19c	AHBSP_NON-SEC_CKM_MEA_CYCLE17_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_n1_gclk_sys_cnt_dft	R	X	Current clock monitor counting value for debugging purposes
0x1a0	AHBSP_NON-SEC_CKM_MEA_CYCLE18_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_pwm_dft	R	X	Current clock monitor counting value for debugging purposes
0x1a4	AHBSP_NON-SEC_CKM_MEA_CYCLE19_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_can_dft	R	X	Current clock monitor counting value for debugging purposes
0x1a8	AHBSP_NON-SEC_CKM_MEA_CYCLE20_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_au_dft	R	X	Current clock monitor counting value for debugging purposes
0x1ac	AHBSP_NON-SEC_CKM_MEA_CYCLE21_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_gtx_dft	R	X	Current clock monitor counting value for debugging purposes
0x1b0	AHBSP_NON-SEC_CKM_MEA_CYCLE22_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_axi1_dft	R	X	Current clock monitor counting value for debugging purposes
0x1b4	AHBSP_NON-SEC_CKM_MEA_CYCLE23_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_vo_a_dft	R	X	Current clock monitor counting value for debugging purposes

Confidential Limited Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x1b8	AHBSP_NON-SEC_CKM_MEA_CYCLE24_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_gclk_vo_b_dft	R	X	Current clock monitor counting value for debugging purposes
0x1bc	AHBSP_NON-SEC_CKM_MEA_CYCLE25_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_combophy_clka_out_dft	R	X	Current clock monitor counting value for debugging purposes
0x1c0	AHBSP_NON-SEC_CKM_MEA_CYCLE26_REG	31:15	Reserved	-	X	
		14:0	ckm_mea_cycle_combophy_clkb_out_dft	R	X	Current clock monitor counting value for debugging purposes
0x1c4	AHBSP_NON-SEC_CKM_MEA_CYCLE27_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_vision_dft	R	X	Current clock monitor counting value for debugging purposes
0x1c8	AHBSP_NON-SEC_CKM_MEA_CYCLE28_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_dram_div2_dft	R	X	Current clock monitor counting value for debugging purposes
0x1cc	AHBSP_NON-SEC_CKM_MEA_CYCLE29_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_gclk_fex_dft	R	X	Current clock monitor counting value for debugging purposes

For Hikra Lang only (08012025) Confidential

Offset	Register	Bits	Field	Access	Reset	Description
0x1d0	**VSDELAY_CONFIG**	31:13	**Reserved**	-	X	
		12	**hsync_vsync_src**	RW	0x0	Source of HSync and VSync signals if from master 0 = From vin0 1 = From vin1
		11	**sel_width_h**	RW	0x0	Selection bit for HSync output width 0 = Input as reference 1 = width_h
		10	**sel_width_v_1**	RW	0x0	Selection bit for VSync output 1 width 0 = Input as reference 1 = width_v_1
		9	**sel_width_v_0**	RW	0x0	Selection bit for VSync output 0 width 0 = Input as reference 1 = width_v_0
		8	**polarity_output_h**	RW	0x0	Polarity of HSync output 0 = Active low 1 = Active high
		7	**polarity_output_v_1**	RW	0x0	Polarity of VSync output 1 0 = Active low 1 = Active high
		6	**polarity_output_v_0**	RW	0x0	Polarity of VSync output 0 0 = Active low 1 = Active high
		5	**polarity_input_h**	RW	0x0	Polarity of HSync input 0 = Active low 1 = Active high
		4	**polarity_input_v**	RW	0x0	Polarity of VSync input 0 = Active low 1 = Active high
		3	**sel_source_h**	RW	0x0	Unused. HSync is always connected to the master signal (VIN)
		2	**sel_source_v**	RW	0x0	Selection bit for VSync source 0 = Master (VIN) 1 = External
		1	**vsdelay_enable**	RW	0x1	Enable bit for internal states 0 = Disable (stall) 1 = Enable
		0	**update_config**	RW	0x0	Set to 1 *before* changing parameters Set to 0 *after* changing parameters
0x1d4	**VSDELAY_DLY_V_PULSE_0**	31:0	**dly_v_pulse_0**	RW	0x0	The value of how many pulses of HSync should VSync be delayed of for vsync_out_0
0x1d8	**VSDELAY_DLY_V_PULSE_1**	31:0	**dly_v_pulse_1**	RW	0x0	The value of how many pulses of HSync should VSync be delayed of for vsync_out_1
0x1dc	**VSDELAY_DLY_V_CLK**	31:16	**dly_v_clk_1**	RW	0x0	The value of how many clock cycles should VSync be delayed of for vsync_out_1
		15:0	**dly_v_clk_0**	RW	0x0	The value of how many clock cycles should VSync be delayed of for vsync_out_0
0x1e0	**VSDELAY_DLY_H_CLK**	31:16	**Reserved**	-	X	
		15:0	**dly_h_clk**	RW	0x0	The value of how many clock cycles should HSync be delayed of for hsync_out

For Sekra Langley Confidential 1.5 08/12/2025

Offset	Register	Bits	Field	Access	Reset	Description
0x1e4	VSDELAY_SEL_WIDTH_V	31:16	width_v_1	RW	0x0	Assigned width of VSync output 1
		15:0	width_v_0	RW	0x0	Assigned width of VSync output 0
0x1e8	VSDELAY_SEL_WIDTH_H	31:16	Reserved	-	X	
		15:0	width_h	RW	0x0	Assigned width of HSync output
0x1ec	SAFETY_EN	31:12	Reserved	-	X	
		17:12	safety_i2c_en	RW	0x0	Disable output function enable bit for safety. When set 1'b0, each output channel of I2C/ CANC/SPI will be disabled when CEHU safety error pin is asserted.
		11:6	safety_canc_en	RW	0x0	
		5:0	safety_spi_en	RW	0x0	
0x1f0	AHBSP_NON-SEC_CKM_CTRL30_REG	31	ckm_enable_amba1_gclk_db-g2dbg	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_amba1_gclk_dbg2dbg	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_amba1_gclk_dbg2dbg	RW	0x0	Lower bound of clock monitor compare value
0x1f4	AHBSP_NON-SEC_CKM_CTRL31_REG	31	ckm_enable_amba1_gclk_core_ahb	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_amba1_gclk_core_ahb	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_amba1_gclk_core_ahb	RW	0x0	Lower bound of clock monitor compare value
0x1f8	AHBSP_NON-SEC_CKM_CTRL32_REG	31	ckm_enable_amba1_gclk_core_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_amba1_gclk_core_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_amba1_gclk_core_dft	RW	0x0	Lower bound of clock monitor compare value

Confidential Ambarella Internal Only 2025

Offset	Register	Bits	Field	Access	Reset	Description
0x1fc	AHBSP_NON-SEC_CKM_CTRL33_REG	31	ckm_enable_amba1_gclk_ahb_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_amba1_gclk_ahb_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_amba1_gclk_ahb_dft	RW	0x0	Lower bound of clock monitor compare value
0x200	AHBSP_NON-SEC_CKM_CTRL34_REG	31	ckm_enable_amba1_gclk_axi1_dft	RW	0x0	Enable signal of clock monitor
		30:28	Reserved	-	X	
		27:16	ckm_upper_bound_amba1_gclk_axi1_dft	RW	0x0	Upper bound of clock monitor compare value
		15:12	Reserved	-	X	
		11:0	ckm_lower_bound_amba1_gclk_axi1_dft	RW	0x0	Lower bound of clock monitor compare value
0x204	AHBSP_NON-SEC_CKM_MEA_CYCLE30_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_amba1_gclk_db-g2dbg	R	X	Current clock monitor counting value for debugging purposes
0x208	AHBSP_NON-SEC_CKM_MEA_CYCLE31_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cy-cle_amba1_gclk_core_ahb	R	X	Current clock monitor counting value for debugging purposes
0x20c	AHBSP_NON-SEC_CKM_MEA_CYCLE32_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cy-cle_amba1_gclk_core_dft	R	X	Current clock monitor counting value for debugging purposes
0x210	AHBSP_NON-SEC_CKM_MEA_CYCLE33_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cycle_amba1_gclk_ahb_dft	R	X	Current clock monitor counting value for debugging purposes
0x214	AHBSP_NON-SEC_CKM_MEA_CYCLE34_REG	31:12	Reserved	-	X	
		11:0	ckm_mea_cy-cle_amba1_gclk_axi1_dft	R	X	Current clock monitor counting value for debugging purposes

Table 15-1. Non-Secure Scratchpad Registers.

For Confidential Use Only
Ambarella Inc. 2025

15.3 Scratchpad: Secure Registers

This register space is addressable from the Arm Cortex-A53 and Cortex-R52 cores. The base address for this register space is E002.F000 for secure region.

Note:

SCRATCHPAD_RAM (E0030000-E0031FFF) should not be used for safety-related applications. Instead, set the **amba_ecru** error masks, including 0xE0027004 bit[14] and 0xE0027034 bit[14], to 1.

Offset	Register	Bits	Field	Access	Reset	Description
0x0	**AHBSP_SEC_RNG_CNT_REG**	31:6	**Reserved**	-	X	
		5:4	**rng_sample_rate**	RW	0x0	0 – 32K cycles 1 – 64K cycles 2 – 128K cycles 3 – 256K cycles
		3:2	**Reserved**	-	X	
		1	**rng_busy**	W	0x0	1 – Begin 160-bit sampling of RNG
		0	**rng_cnt**	RW	0x0	0 – This bit is reset to 0 when rng_start is set 1 – Sampling has completed and generated interrupt to GIC
0x4	**AHBSP_SEC_RNG_DATA0_REG**	31:0	**rng_data0**	RW	0x0	Random number sampled from RNG block
0x8	**AHBSP_SEC_RNG_DATA1_REG**	31:0	**rng_data1**	RW	0x0	Random number sampled from RNG block
0xc	**AHBSP_SEC_RNG_DATA2_REG**	31:0	**rng_data2**	RW	0x0	Random number sampled from RNG block
0x10	**AHBSP_SEC_RNG_DATA3_REG**	31:0	**rng_data3**	RW	0x0	Random number sampled from RNG block
0x14	**AHBSP_SEC_VROM_PDE_REG**	31:2	**Reserved**	-	X	
		1	**ahb_vrom_prn**	R	0x0	To monitor VROM PRN
		0	**ahb_vrom_pde**	RW	0x0	VROM enable signal
0x18	**AHBSP_SEC_JTAG_EN_REG**	31:1	**Reserved**	-	X	
		0	**jtag_en**	RW	0x1	0 – Disable JTAG 1 – Enable JTAG
0x1c	**AHBSP_SEC_INSERT_ERROR_REG**	31:1	**Reserved**	-	X	
		0	**ahb_scratchpad_sec_insert_error**	RW	0x0	Set this field to 1 to insert error to ahb_scratchpad_sec

Confidential For Meta Land only 1.5 (080/2023)

Offset	Register	Bits	Field	Access	Reset	Description
0x20	AHBSP_SEC_LOCK_CTRL_SEC_REG	31:12	Reserved	-	X	
		11:2	sp_sec_time-out_upper_bound	RW	0x3ff	Upper bound of the timeout mechanism. SW must write sp_sec_lock after sp_sec_lock_wr_lock being unlocked within sp_sec_time-out_upper_bound cycles.
		1	sp_sec_lock_wr_lock	RW	0x1	Write lock for sp_sec_lock. Need to set this to 0 every time before writing to sp_sec_lock. 0: Unlocked state, write access to sp_sec_lock is allowed. 1: Locked state, write access to sp_sec_lock is forbidden.
		0	sp_sec_lock	RW	0x0	Lock accesses to ahb_scratchpad_sec 0: Unlocked state, write accesses to registers in ahb_scratchpad_sec are allowed. 1: Locked state, write accesses to registers in ahb_scratchpad_sec except “AHBSP_SEC_LOCK_CTRL_REG” are forbidden.
0x30	AHBSP_SEC_RTC_CURT_REG	31:0	ahb_rtc_curt	RW	0x0	Set RTC current time
0x34	AHBSP_SEC_CURT_REG	31:0	ahb_reg_curt	R	0x0	RTC current time register
0x40	AHBSP_SEC_RTC_RST_REG	31:1	Reserved	-	X	
		0	ahb_rtc_pcrst	RW	0x0	RTC pcrst input
0x54	AHBSP_SEC_RTC_REG_STA_REG	31:8	Reserved	-	X	
		7:0	rtc_ahb_reg_sta_reg	R	0x0	RTC sta register, bit[2] self-refresh, bit[3] alarm wkup enable
0x58	AHBSP_SEC_RTC_STA_REG	31:8	Reserved	-	X	
		7:0	ahb_rtc_sta	RW	0x0	Set RTC sta, bit[2] self-refresh, bit[3] alarm wkup enable
0x90	AHBSP_SEC_RTC_STROBE_REG	31:1	Reserved	-	X	
		0	ahb_rtc_strobe	RW	0x0	RTC Strobe to transfer scrambling keys from RTC_DBG to RTC
0x94	AHBSP_SEC_BOOT_STS_REG	31:1	Reserved	-	X	
		0	ahb_sec_boot_sts	R	0x0	Monitor secure_boot status

Confidential For Meta Platforms Internal Only

Offset	Register	Bits	Field	Access	Reset	Description
0xa0	AHBSP_SEC_OTP_CTRL1_REG	31:23	Reserved	-	X	
		27:24	ahb_sec_otp_prog_wrt_pulse_cnt	RW	0x2	Maximum number of times of programming during one write operation
		23	Reserved	-	X	
		22	ahb_sec_otp_read_fsm_en	RW	0x0	This is an enable signal for read FSM. 0: Reset read FSM 1: Enable read FSM
		21	ahb_sec_otp_readen	RW	0x0	This is an enable signal for OTP read operation
		20	ahb_sec_otp_dbg_read_mode	RW	0x0	This bit selects reading through Register or auto FSM
		19	ahb_sec_otp_prog_disable_rd	RW	0x0	Disable verifying after programming. If it's set to 0, the controller programs twice during the write operation and then verifies the data. If it's set to 1, the controller programs until the program count reaches ahb_sec_otp_prog_wrt_pulse_cnt and will not verify the data.
		18	ahb_sec_otp_fsm_write_mode	RW	0x0	This bit selects read/write access to OTP. 0: Read mode 1: Write mode
		17	ahb_sec_otp_prog_en	RW	0x0	This is an enable signal for OTP write operation
		16	ahb_sec_otp_prog_fsm_en	RW	0x0	This is an enable signal for prog FSM. 0: Reset prog (write) FSM 1: Enable prog (write) FSM
		15:0	ahb_sec_otp_prog_addr	RW	0x0	This field configures the OTP address to be written (programmed)/read
0xa4	AHBSP_SEC_OTP_OBSV_REG	31:5	Reserved	-	X	
		4	ahb_sec_otp_write_ctrl_prog_done	R	0x0	It indicates the programming operation of current 1-bit data is done when this bit is 1
		3	ahb_sec_otp_write_ctrl_prog_fail	R	0x0	It indicates the programming operation of current 1-bit data failed when this bit is 1
		2	ahb_sec_otp_write_ctrl_prog_rdy	R	0x0	It indicates it's ready to receive next input data (1 bit) when this bit is 1
		1	ahb_sec_otp_read_ctrl_read_rdy	R	0x0	It indicates it's ready to receive next address to read next data when this bit is 1
		0	ahb_sec_otp_read_ctrl_read_done	R	0x0	It indicates the read operation of OTP data of current address is done when this bit is 1
0xa8	AHBSP_SEC_READ_CTRL_READ_DOUT0	31:0	ahb_sec_otp_read_ctrl_read_dout0	R	0x0	Observation register for OTP Read data out
0xac	AHBSP_SEC_RESERVED_REG	31:0	ahb_sec_reserved_reg	RW	0x0	Reserved register for ECO

Confidential
For Maka Lan only
1.3.2021

Offset	Register	Bits	Field	Access	Reset	Description
0xb0	**AHBSP_SEC_RNG_DATA4_REG**	31:0	**rng_data4**	RW	0x0	Random number sampled from RNG block
0xb4	**AHBSP_SEC_OTP_CTRL2_REG**	31:16	**ahb_sec_otp_prog_vpph_cnt**	RW	0xF0	External VPP Hold Time after CPUMPEN
		15:0	**ahb_sec_otp_prog_vpps_cnt**	RW	0xF0	External VPP Setup Time before CPUMPEN
0xb8	**AHBSP_SEC_READ_CTRL_READ_DOUT1**	31:8	**Reserved**	-	X	
		7:0	**ahb_sec_otp_read_ctrl_read_dout1**	R	0x0	Observation register for OTP Read data out

Table 15-2. Secure Scratchpad Registers.

Confidential
For Mekra Lang only
1.5 (08012025)

16. SECURITY

This chapter includes the following sections:

- (Section 16.1) Security: Overview
- (Section 16.2) Security: VROM and OTP
- (Section 16.3) Security: AXI Config Access
- (Section 16.4) Security: Scratchpad Access
- (Section 16.5) Security: DRAM Access
- (Section 16.6) Security: JTAG
- (Section 16.7) Security: OTP
- (Section 16.8) Security: Boot Process

16.1 Security: Overview

The diagram on the following page shows a detailed view of the CV22FS system bus architecture.

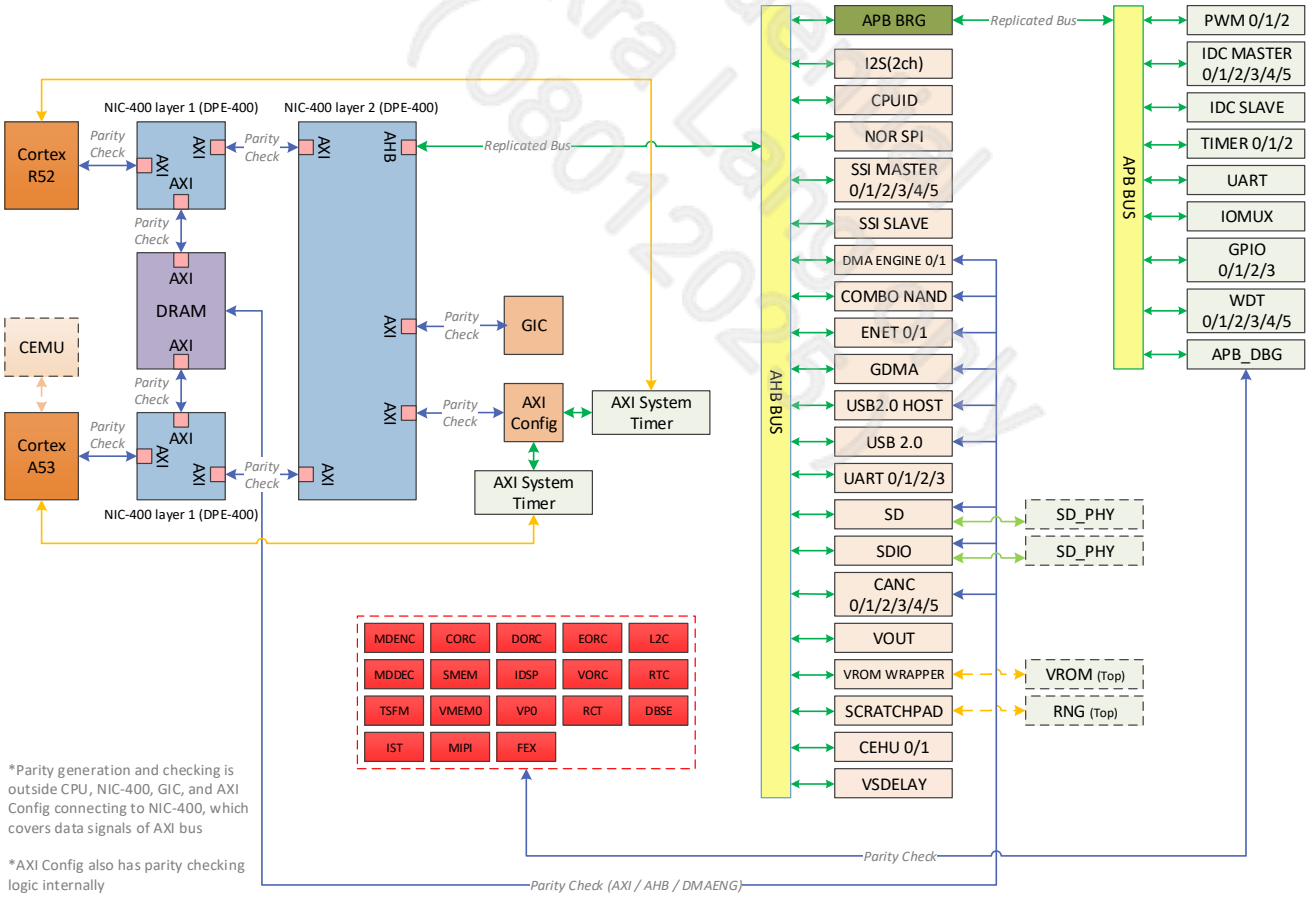


Figure 16-1. Security Overview: CV22FS System Bus Architecture.

PROT signals in AXI buses are used to distinguish between secure and non-secure access, and they are sent to AHB with each transaction as to enable access check in AHB buses. There is no security information in APB buses.

Software can configure the security level of AHB bus via GPV. The security level of each module (IP) can also be set in AXI Config. For more details, please refer to the following section.

16.1.1 Programmable Security Level

There are programmable security levels: bus-level and module-level. For bus-level security, the AHB interface in NIC-400 layer 2 can be configured as “secure” or “non-secure” with GPV. For module-level security, each module is configured as secure or non-secure in the AXI Config registers, which are in the secure region.

If the AHB bus is configured as secure, the non-secure CPU cannot get access to any module under this bus. Therefore, if the AHB bus (and the APB bus under it) contains both secure and non-secure modules, Ambarella recommends configuring the AHB bus as non-secure to enable all transactions to go to it; the secure access checker in the AHB bus will check whether or not the transaction is allowed.

If the non-secure CPU attempts to access a secure module, it is blocked, and the corresponding “secure access error” bit is asserted in **scratchpad_nsec** register. The blocked access also triggers an exception in the CPU.

Note:

Secure access error bits are asserted by module-level secure access check logics in the AHB bus. Note that when a non-secure CPU attempts to access a module under a secure AHB bus, which is configured with GPV, access is blocked and exception is triggered, but secure access error bit is not asserted. Also note that all the accesses from CR52 are considered as secure.

16.1.2 Security Overview: NIC-400

There are three NIC-400 instances in CV22FS: two 1st layer NIC-400 (NIC-400-A), and one 2nd layer NIC-400 (NIC-400-B).

For NIC-400-A, there is one slave interface for CA53/CR52 and two master interfaces (2 AXI) for DRAM and NIC-400-B.

For NIC-400-B, there are two slave interfaces for NIC-400-A and three master interfaces (1 AHB lite and 2 AXI) for AHB, GIC and AXI Config

NIC-400-A-A53 Interfaces		
Ports	**Protocol**	**Security**
Slave Interfaces		
CA53	AXI	Per access
Master Interfaces		
DRAM	AXI	Non-secure
NIC-400-B	AXI	Boot secure → non-secure

Table 16-1. Security Overview: NIC-400-A-A53 Interfaces.

Confidential
For Ambarella Lang only
1.5 - 08012025

NIC-400-A-R52 Interfaces		
Ports	**Protocol**	**Security**
Slave Interfaces		
CR52	AXI	Per access
Master Interfaces		
DRAM	AXI	Non-secure
NIC-400-B	AXI	Boot secure → non-secure

Table 16-2. Security Overview: NIC-400-A-R52 Interfaces.

NIC-400-B Interfaces		
Ports	**Protocol**	**Security**
Slave Interfaces		
NIC-400-A-A53	AXI	Per Access
NIC-400-A-R52	AXI	Per access
Master Interfaces		
GIC	AXI	Boot secure → Non-secure
AXI Config	AXI	Boot secure → secure
AHB	AHB	Boot secure → Non-secure

Table 16-3. Security Overview: NIC-400-B Interfaces.

For slave interfaces, both ports are configured as per access to allow both secure and non-secure access from masters.

For master interfaces, if one port is configured as non-secure, then both secure and non-secure accesses are allowed and the PROT signal (information) will be delivered. If programmable security level is used for each module, Ambarella recommends configuring AHB buses as non-secure to enable both secure and non-secure accesses to be passed through NIC-400.

Most master interface ports are configured as “boot secure” so that they can be set as either secure or non- secure via the GPV port of NIC-400 in the boot strapper. GPV of NIC-400-A can only be programmed via secure access. This prevents non-secure processes from obtaining access to the secure bus; GPV of NIC-400-B can only be programmed by CR52.

16.2 Security: VROM and OTP

VROM and OTP are used to store security-related codes and data correspondingly.

VROM can contain the following components:

- Disclosed secure hash algorithm SHA-256.
- AES or symmetric encryption algorithm for verifying the contents of the DRAM after a warm boot
- Boot codes with initial settings for NIC, GIC, AXI Config, flash controller, DRAM, and so on.

OTP contains the following components:

- Public keys to decrypt digital signature from an authentication authority
- A table of unique encryption certificates to authenticate other devices such as HDMI and other forms of digital

rights management (DRM), used by higher-level software

- Built-in redundancy-analyzer (BIRA) information
- Boot configurations
- Manufacturer information (lot number and other details)

Notes:

- Because Secure ROM is under the secure bus, it is only accessible via a secure process.
- OTP can be accessed through secure scratchpad. Its security level is the same as secure scratchpad.

16.3 Security: AXI Config Access

Only CR52 is allowed to access AXI Config; CA53 is *not* allowed to access AXI Config. If CA53 attempts to access AXI Config, it will receive a bus error.

16.4 Security: Scratchpad Access

There are two scratchpads in the AHB bus: secure scratchpad and non-secure scratchpad. Because the secure scratchpad consists of security-related registers, Ambarella recommends setting the security level of this scratchpad to “secure”. Conversely, Ambarella recommends setting the security level of the non-secure scratchpad to “non-secure,” as it does not consist of security-related registers.

16.5 Security: DRAM Access

In a secure bootstrapper, scrambling register, VPNs, and secure DRAM regions are configured to prevent non-secure clients from gaining access.

Clients	DRAM Clients
axi0 (CR52)	Security information is attached with AXI transaction. Can access secure DRAM via secure transaction.
axi1 (CA53)	Security information is attached with AXI transaction. Can access secure DRAM via secure transaction.
dma0	Not allowed to access secure DRAM
dma1	Not allowed to access secure DRAM
enet	Not allowed to access secure DRAM
enet1	Not allowed to access secure DRAM
fdma (Combo NAND)	Not allowed to access secure DRAM
canc0	Can be allowed to access secure DRAM
canc1	Can be allowed to access secure DRAM
canc2	Can be allowed to access secure DRAM
canc3	Can be allowed to access secure DRAM
canc4	Can be allowed to access secure DRAM
canc5	Can be allowed to access secure DRAM
gdma	Not allowed to access secure DRAM
sdxc0 (SD)	Not allowed to access secure DRAM
sdxc1 (SDXC)	Not allowed to access secure DRAM

Confidential
For Media Lang only
Fri Oct 20 2023

Clients	DRAM Clients
usb2 (Device)	Not allowed to access secure DRAM
usb2 (Host)	Not allowed to access secure DRAM
eorc	Not allowed to access secure DRAM
corc	Not allowed to access secure DRAM
vorc	Not allowed to access secure DRAM
ol2c	Not allowed to access secure DRAM
smem	Can be allowed to access secure DRAM
vmem0	Can be allowed to access secure DRAM
fex	Not allowed to access secure DRAM
dbse	Not allowed to access secure DRAM

Table 16-4. DRAM Access: DRAM Clients.

16.6 Security: JTAG

TAG is controlled by two signals—**jtag_dis_efuse** and **jtag_en_reg**.

jtag_dis_efuse is from OTP, and **jtag_en_reg** is a register in scratchpad under secure bus.

Control Signal			
jtag_dis_efuse	**jtag_en_reg**	**JTAG**	**Comments**
0	X	Enabled	For chip bring-up
1	0	Disabled	
1	1 (Default; normal boot)	Enabled	JTAG is enabled during normal boot
1	0 (Default; secure boot)	Disabled	JTAG is disabled during secure boot
1	1	Enabled	JTAG is re-enabled by secure process

Table 16-5. Security: JTAG.

If **jtag_dis_efuse** does not burn out (**jtag_dis_efuse** = 0), then the JTAG function is enabled no matter the value of **jtag_en_reg**.

If **jtag_dis_efuse** is burned out (**jtag_dis_efuse** = 1), then the JTAG function is controlled by **jtag_en_reg**.

- For normal boot, JTAG is enabled by default and can be disabled to prevent unintentional JTAG access or code tracing by users.
- For secure boot, JTAG is disabled by default; only secure CPU can program **jtag_en_reg** to re-enable JTAG access.

Note:

Although secure CPU enables JTAG by programming **jtag_en_reg**, if the system is warm reset (AHB is also reset), then the JTAG function is disabled and can be re-enabled by secure CPU.

Confidential For Internal Use Only 2025

16.7 Security: OTP

16.7.1 Security: OTP Data Format

Bit Address (DEC)	Bit Address (HEX)	# Bits	Function
0-63	0x0-0x3F	40	{sys_config[0:31], 8'b0}
64-127	0x40-0x7F	40	{sys_config[32:63], 8'b0}
128-191	0x80-0xBF	40	{wafer_id[0:31], 8'b0}
192-255	0xC0-0xFF	40	{wafer_id[32:47], 24'b0}
256-319	0x100-0x13F	40	{amba_id[0:7], 24'b0, CRC8} (CRC8 encoded with {24'b0, amba_id, 0x100})
320-383	0x140-0x17F	40	Reserved
384-447	0x180-0x1BF	40	{write_lock[0:31], 8'b0}
448-511	0x1C0-0x1FF	40	{write_lock_confirm[0:31], 8'b0}
512-575	0x200-0x23F	40	{out_9bit_ts0_l[0:8], out_9bit_ ts0_m[0:8], out_9bit_ts0_h[0:8], 13'b0}
576-639	0x240-0x27F	40	{out_9bit_ts1_l[0:8], out_9bit_ ts1_m[0:8], out_9bit_ts01h[0:8], 13'b0
640-703	0x280-0x2BF	40	{out_9bit_ts2_l[0:8], out_9bit_ ts2_m[0:8], out_9bit_ts2_h[0:8], 13'b0}
704-765	0x2C0-0x2FF	40	{out_9bit_ts3_l[0:8], out_9bit_ ts3_m[0:8], out_9bit_ts3_h[0:8], 13'b0}
768-831	0x300-0x33F	40	{out_9bit_ts4_l[0:8], out_9bit_ ts4_m[0:8], out_9bit_ts4_h[0:8], 13'b0}
832-895	0x340-0x37F	40	{out_9bit_ts5_l[0:8], out_9bit_ ts5_m[0:8], out_9bit_ts5_h[0:8], 13'b0
896-959	0x380-0x3BF	40	{out_9bit_ts6_l[0:8], out_9bit_ ts6_m[0:8], out_9bit_ts6_h[0:8], 13'b0}
960-1023	0x3C0-0x3FF	40	{out_9bit_ts7_l[0:8], out_9bit_ ts7_m[0:8], out_9bit_ts7_h[0:8], 13'b0}
1024-1087	0x400-0x43F	40	{out_9bit_ts8_l[0:8], out_9bit_ ts8_m[0:8], out_9bit_ts8_h[0:8], 13'b0}
1088-1151	0x440-0x47F	40	{out_9bit_ts9_l[0:8], out_9bit_ ts9_m[0:8], out_9bit_ts9_h[0:8], 13'b0}
1152-1215	0x480-0x4BF	40	{test_equipment_info[0:17], 22'b0}
1216-2559	0x4C0-0x9FF	40	Reserved
2560-3007	0xA00-0xBBF	280	CA53_bisr
3008-3903	0xBC0-0xF3F	560	CR52_bisr
3904-31743	0xF40-0x7BFF	17400	fullchip_bisr
31744-31999	0x7C00-0x7CFF	160	amba unique id (by Ambarella)
32000-32063	0x7D00-0x7D3F	40	Zone B0 - Data_invalid (by customer)
32064-32127	0x7D40-0x7D7F	40	Zone B0 - Data_invalid_confirm (by customer)
32128-32255	0x7D80-0x7DFF	80	Zone B0 - Reserved
32768-33279	0x8000-0x81FF	320	Zone D0 (by customer)
33280-33791	0x8200-0x83FF	320	Zone D1 - Monotonic counter (by customer)
33792-34303	0x8400-0x85FF	320	Zone D2 (by customer)

Confidential Ambarella Internal Use Only Meta Lang 08/12/2025

Bit Address (DEC)	Bit Address (HEX)	# Bits	Function
34304-34815	0x8600-0x87FF	320	Zone D3 (by customer)
34816-35327	0x8800-0x89FF	320	Zone D4 (by customer)
35328-35839	0x8A00-0x8BFF	320	Zone D5 (by customer)
35840-36531	0x8C00-0x8DFF	320	Zone D6 (by customer)
36532-36863	0x8E00-0x8FFF	320	Zone D7 (by customer)
36864-37375	0x9000-0x91FF	320	Zone D8 (by customer)
37376-37887	0x9200-0x93FF	320	Zone D9 (by customer)
37888-38399	0x9400-0x95FF	320	Zone D10 (by customer)
38400-38911	0x9600-0x97FF	320	Zone D11 (by customer)
38912-38423	0x9800-0x99FF	320	Zone D12 (by customer)
39424-39935	0x9A00-0x9BFF	320	Zone D13 (by customer)
39936-40447	0x9C00-0x9DFF	320	Zone D14 (by customer)
40448-40959	0x9E00-0x9FFF	320	Zone D15 (by customer)
40960-49151	0xA000-0xBFFF	5120	Zone C1 (by customer)
49152-57343	0xC000-0xDFFF	5120	Zone C2 (by customer)
57344-65535	0xE000-0xFFFF	5120	Zone C3 (by customer)

Table 16-6. Security: OTP Data Format.

Notes:

In each data entry there are 64 bits but only lower 40 bits are usable.

0x0 ~ 0x7CFF, except for 0x0~0x7F and 0x180~0x1FF, is zone A, which is programmed by Ambarella and locked before shipping to customers. 0x180-0x1BF (write_lock[0:31]), 0x1C0-0x1FF (write_lock_confirm[0:31]), and zone B0 are always programmable. Write lockout is controlled with write_lock[0:31] as shown in the following section.

For the AMBA unique ID, zone B* (except B0), zone C*, and zone D* (except D1), each 32-bit data follows 8-bit CRC8, which means the MSB is CRC8 for the data integrity check. Additionally, zones C* and D* (except D1) include CRC8 HW verification. CRC8 is encoded with 32-bit data and 16-bit addresses ({data, address}).

16.7.2 Security: OTP Write-Lockout

During power-on, OTP data at 0x180-0x1BF are read out automatically and stored in RCT register write_ lock[0:31], in which 25 bits are used to indicate write-lockout status of 25 zones in OTP as shown in the table below. If the write-lockout bit is blown (programmed), programming any bits in its corresponding zone fails.

After programming write_lock[0:31], software shall program write_lock_confirm[0:31] in reversed bit order, i.e. write_lock_confirm[31:0] = write_lock[0:31].

Note that write_lock [0:31] is only read out once at power-on; programming any OTP address in 0x180-0x1BF to block OTP write takes effect in the next power cycle.

Bit Address	OTP Address	Description
0	0x180	Write lockout of zone A at 0x0-0x7CFFF, except 0x0-0x7F and 0x180-0x1BF (0: write enable; 1: write prohibit)
1	0x181	Reserved

Bit Address	OTP Address	Description
2	0x182	Write lockout of zone B1 at 0x7E00-0x7EFF (0: write enable; 1: write prohibit)
3	0x183	Write lockout of zone B2 at 0x7F00-0x7FFF (0: write enable; 1: write prohibit)
4	0x184	Reserved
5	0x185	Write lockout of zone C1 at 0xA000-0xBFFF (0: write enable; 1: write prohibit)
6	0x186	Write lockout of zone C2 at 0xC000-0xDFFF (0: write enable; 1: write prohibit)
7	0x187	Write lockout of zone C3 at 0xE000-0xFFFF (0: write enable; 1: write prohibit)
8	0x188	Write lockout of zone D0 at 0x8000-0x81FF (0: write enable; 1: write prohibit)
9	0x189	Write lockout of zone D1 at 0x8200-0x83FF (0: write enable; 1: write prohibit)
10	0x18A	Write lockout of zone D2 at 0x8400-0x85FF (0: write enable; 1: write prohibit)
11	0x18B	Write lockout of zone D3 at 0x8600-0x87FF (0: write enable; 1: write prohibit)
12	0x18C	Write lockout of zone D4 at 0x8800-0x89FF (0: write enable; 1: write prohibit)
13	0x18D	Write lockout of zone D5 at 0x8A00-0x8BFF (0: write enable; 1: write prohibit)
14	0x18E	Write lockout of zone D6 at 0x8C00-0x8DFF (0: write enable; 1: write prohibit)
15	0x18F	Write lockout of zone D7 at 0x8E00-0x8FFF (0: write enable; 1: write prohibit)
16	0x190	Write lockout of zone D8 at 0x9000-0x91FF (0: write enable; 1: write prohibit)
17	0x191	Write lockout of zone D9 at 0x9200-0x93FF (0: write enable; 1: write prohibit)
18	0x192	Write lockout of zone D10 at 0x9400-0x95FF (0: write enable; 1: write prohibit)
19	0x193	Write lockout of zone D11 at 0x9600-0x97FF (0: write enable; 1: write prohibit)
20	0x194	Write lockout of zone D12 at 0x9800-0x99FF (0: write enable; 1: write prohibit)
21	0x195	Write lockout of zone D13 at 0x9A00-0x9BFF (0: write enable; 1: write prohibit)
22	0x196	Write lockout of zone D14 at 0x9C00-0x9DFF (0: write enable; 1: write prohibit)
23	0x197	Write lockout of zone D15 at 0x9E00-0x9FFF (0: write enable; 1: write prohibit)
24	0x198	Write lockout of 0x0-0x7F
25~31	0x199~0x19F	Reserved

Table 16-7. Security: OTP Write-Lock Format.

For Mentor Confidential Only
1.5 (08/16/2016)

16.7.3 Security: Public Key Management

Please contact the Ambarella support team for details about the management scheme of public keys in OTP for FW authentication.

16.8 Security: Boot Process

CV22FS has two boot modes – secure boot and normal boot, which can be selected by POC pins.

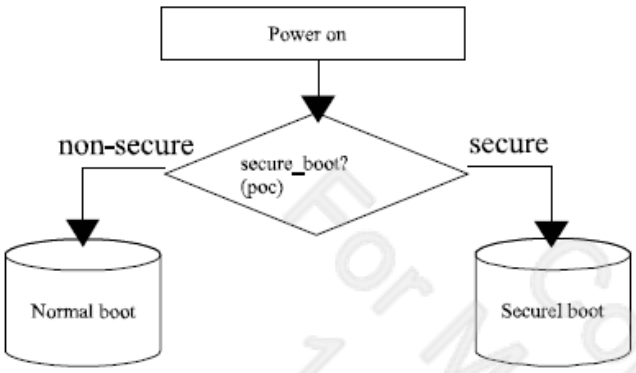


Figure 16-2. Security: Boot Mode Select.

After power-on, the system starts the bootstrapper in VROM. The bootstrapper configures the system, enables error interrupt from VROM and OTP, sets critical peripherals, executes authentication procedures for bootloader if secure_boot is selected, checks boot mode, and transfers control to the bootloader.

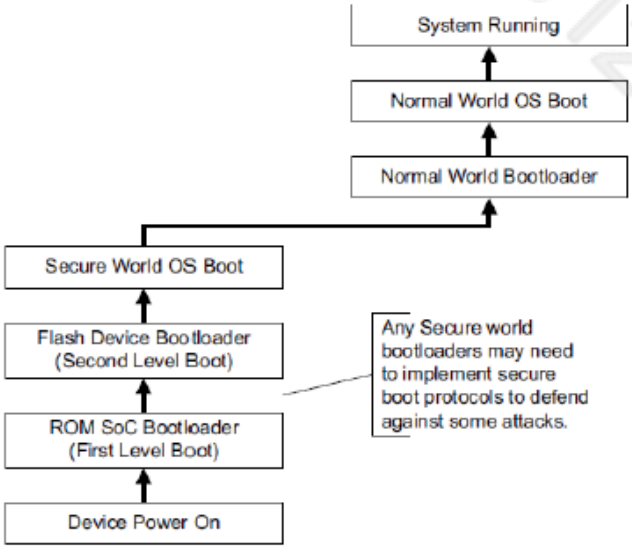


Figure 16-3. Security: Boot Flow.

Note that the boot mode (normal or secure) is decided only at power-on (cold boot) and cannot be programmed/ changed after system booting. This is to prevent a user from reading OTP in normal boot mode by switching boot modes.

Confidential For Mekrotang only 1.5 (20012025)

16.8.1 Boot: Normal Boot

The following lists the normal boot process.

1. CR52 starts at 0x7FFF_0000, which is mapped to 0xFFFF_0000 in VROM.
2. Jumps to bootloader after VROM boot completes.
3. The DDRC / DRAM initializes.
4. The registers are configured to copy the bootloader from external storage to DRAM.
5. The bootstrapper transfers control to the bootloader in DRAM.

16.8.2 Boot: Secure Boot

For details about the secure boot process, please contact the Ambarella support team.

Confidential
For Mekra Lang only
1.5 (08012025)

17. AXI CONFIGURATION

The AXI configuration module provides fundamental infrastructure to enable communication protocols, clock monitors control, and security and safety control. Additionally, the AXI configuration module includes a safety feature which supports lock access.

The AXI configuration block shall only be accessed by Cortex R52 CPU; access from Cortex A53 CPU is not allowed. The block will detect the source of the transaction, and if the transaction is not from Cortex R52 CPU, then it will issue response ERROR on the AXI bus.

This chapter includes the following sections:

- (Section 17.1) AXI: Lock Access
- (Section 17.2) AXI: Registers

17.1 AXI: Lock Access

The AXI configuration module supports lock access for write transactions to internal control registers from the AXI master. The corresponding control register of Lock Access is **axi_cfg_ctrl** located at offset 0x0.

The following lists the flow for executing lock access:

1. At the reset, all accesses are allowed without any locking.
2. After initialization is complete, the software enables LOCKING by performing the following:
   - Write **axi_cfg_ctrl** register axi_cfg_lock_wr_lock bit to 0.
   - Write **axi_cfg_ctrl** register axi_cfg _lock bit to 1.
3. Next, all register write transactions perform the following:
   - Write **axi_cfg_ctrl** register axi_cfg _lock_wr_lock bit to 0.
   - Write **axi_cfg_ctrl** register axi_cfg _lock bit to 0.
   - Write to target register.
4. When axi_cfg _lock bit is written, the axi_cfg _lock_wr_lock bit returns to 1, or remains 0.
5. When target register is written, the axi_cfg _lock returns to 1 for LOCK state.

   If the time is finished but target register is not written, axi_cfg _lock bit returns to 1. The timeout cycle is configured by timeout_upper_bound bits of **axi_cfg_ctrl** register.
6. With the exception of axi_cfg _lock_wr_lock bit, all write transactions to the registers in LOCK state are not allowed and will issue an interrupt.

Confidential
For Meka Lang only
Jan 2025

17.2 AXI: Registers

Offset	Register	Bits	Field	Access	Reset	Description
0x0	axi_cfg_ctrl	31:12	Reserved	-	-	Reserved
		11:2	timeout_upper_bound	RW	0x3ff	Upper bound of the timeout mechanism. SW must write axi_cfg_lock after axi_cfg_lock_wr_lock being unlocked within timeout_upper_bound cycles
		1	axi_cfg_lock_wr_lock	RW	0x1	Write lock for axi_cfg_lock. Must set this to 0 each time before writing to axi_cfg_lock. 0: Unlocked state, write access to axi_cfg_lock is allowed. 1: Locked state, write access to axi_cfg_lock is forbidden.
		0	axi_cfg_lock	RW	0x0	Lock accesses to axi_config 0: Unlocked state, write accesses to registers in axi_config are allowed 1: Locked state, write accesses to registers in axi_config except “axi_cfg_ctrl” are forbidden
0x4	cortex_misc_ctrl	31:5	Reserved	-	-	Reserved
		4	cfg_gic_cfgs_disable	RW	0x0	GIC secure disable, lockdown 1: Disable 0: Not disable
		3:0	n0_cfg_arm_cp15s_disable n1_cfg_arm_cp15s_disable	RW	0x0	Disable cortex0 P8QC CP15 Security Extensions registers 1: Disable 0: Not disable
0x8	axi_fault_inject	31:7	Reserved	-	-	Reserved
		6	nic400_b_gic_insert_error	RW	0x0	Set this field to 1 to insert error to nic400_b_parity (gic_wdata_par_failure)
		5	axi_config_insert_error	RW	0x0	Set this field to 1 to insert error to axi_config.
		4	nic400_b_axi_cfg_insert_error	RW	0x0	Set this field to 1 to insert error to nic400_b_parity (axi_cfg_wdata_par_failure)
		3	nic400_a_a53_insert_error	RW	0x0	Set this field to 1 to insert error to nic400_a_parity_a53
		2	system_timer_a53_insert_error	RW	0x0	Set this field to 1 to insert error to system_timer_a53
		1	nic400_a_r52_insert_error	RW	0x0	Set this field to 1 to insert error to nic400_a_parity_r52
		0	system_timer_r52_insert_error	RW	0x0	Set this field to 1 to insert error to nic400_a_parity_r52

Confidential
For Meta Lang
A53 (20125)

Offset	Register	Bits	Field	Access	Reset	Description
0x14	**axi_sys_timer_ctrl**	31:8	**Reserved**	-	-	Reserved
		7:5	**n1_update_rate_sel**	RW	0x0	System timer divide rate for Cortex n0 0: / 1 1: / 2 2: / 8 3: / 16 4: / 64 5: / 128 6: / 512 7: / 1024
		4	**Reserved**	-	-	Reserved
		3:1	**n0_update_rate_sel**	RW	0x0	System timer divide rate for Cortex n0 0: / 1 1: / 2 2: / 8 3: / 16 4: / 64 5: / 128 6: / 512 7: / 1024
		0	**Reserved**	-	-	Reserved
0x18	**nic400_gpv_ctrl**	31:2	**Reserved**	-	-	Reserved
		1	**n1_nic400_mainclk_r_en**	RW	0x1	NIC400 mainclk_r clock gating enable for axi_n1 and NIC400_R52 0: Disable 1: Enable
		0	**n0_nic400_mainclk_r_en**	RW	0x1	NIC400 mainclk_r clock gating enable for axi_n0 0: Disable 1: Enable
0x1C	**cortex_cryp-to_ctrl**	31:4	**Reserved**	-	-	Reserved
		3:0	**n1_crypto_dis**	RW	0x0	Cortex n1 Cryptography Extensions dis-able, each bit for one core 1: Disable 0: Not disable
0x20	**cortex_arch_ctrl**	31:4	**Reserved**	-	-	Reserved
		3:0	**n1_arch64_32**	RW	0xf	Cortex n1 CPU architecture 0: AArch32 1: AArch64

For Meeka Lang Confidential
Apr 08 2025

Offset	Register	Bits	Field	Access	Reset	Description
0x24	cortex_misc_sts	31:26	Reserved	-	-	Reserved
		25	n0_cpuhalt	RW	0x0	Core waits out of reset before taking reset exception and fetching instructions
		24	n0_cfginitreq	RW	0x0	Program-visible registers initialized to fixed value out of reset
		23:20	n1_SMPEN_reg	RO	0x0	Indicate if processor are in SMP or SAMP mode 0: In AMP mode, disable data coherency with other core 1: In SMP mode, enable data coherency with other core
		19:13	Reserved	-	-	Reserved
		12	n0_cortex_standby_wfe_reg	RO	0x0	Indicate if a processor in WFE (wait for event) state
		11	n0_cortex_standby_wfi_reg	RO	0x1	Indicate if a processor in WFI (wait for interrupt) state
		10	n1_cortex_standby_wfil2_reg	RO	0x1	Indicate if a processor in WFIL2 (L2 memory idle and doesn't accept transaction) state
		9:6	n1_cortex_standby_wfe_reg	RO	0x0	Indicate if a processor in WFE (wait for event) state
		5:2	n1_cortex_standby_wfi_reg	RO	0xf	Indicate if a processor in WFI (wait for interrupt) state
		1:0	Reserved	-	-	Reserved
0x28	cortex_reset	31:21	Reserved	-		Reserved
		20	n1_cortex_l2reset	RW	0x0	Reset to L2RESET of P8QC
		19:16	n1_cortex_reset	RW	0xf	Reset to n1 P8QC Core0~3, after power on reset
		15:1	Reserved	-	-	Reserved
		0	n0_cortex_reset	RW	0x0	Reset to n0 P8QC Core0, after power on reset
0x2C	flash_boot_done	31:1	Reserved	-	-	Reserved
		0	io_boot_done	RO	0x0	Flash boot done
0x30	cfgperiphbase	31:0	n0_cfgperiphbase	RW	0xf4000000	Base physical address of memory-mapped peripherals of Cortex n0 CPU
0x4C	jtagtop_sts	31:2	Reserved	-	-	Reserved
		1	n0_status_status_jtagtop	RO	0x0	Indicates TAP in TLR, RTI, Sel-DR or Sel-IR states
		0	n1_status_status_jtagtop	RO	0x0	Indicates TAP in TLR, RTI, Sel-DR or Sel-IR states

For Ambarella Confidential Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x50	**axi_dbg_ctrl**	31:13	**Reserved**	-	-	Reserved
		12	**n0_dbgen**	RW	0x1	Cortex n0 invasive debug enable
		11:8	**n1_dbgen**	RW	0xf	Cortex n1 invasive debug enable
		7:5	**Reserved**	-	-	Reserved
		4	**n0_spiden**	RW	0x1	Cortex n0 invasive secure privileged invasive debug enable
		3:0	**n1_spiden**	RW	0xf	Cortex n1 invasive secure privileged invasive debug enable
0x54	**n0_rvbaraddr0**	31:0	**n0_rvbaraddr0**	RW	0x7fff0000	Cortex n0 Reset Vector Base Address for executing in 64-bit state. This value is sampled only during reset of the processor
0x64	**n1_rvbaraddr0**	31:0	**n1_rvbaraddr0**	RW	0x0	Cortex n1 Reset Vector Base Address for executing in 64-bit state This value is sampled only during reset of the processor
0x68	**n1_rvbaraddr1**	31:0	**n1_rvbaraddr1**	RW	0x0	Cortex n1 Reset Vector Base Address for executing in 64-bit state This value is sampled only during reset of the processor
0x6C	**n1_rvbaraddr2**	31:0	**n1_rvbaraddr2**	RW	0x0	Cortex n1 Reset Vector Base Address for executing in 64-bit state This value is sampled only during reset of the processor
0x70	**n1_rvbaraddr3**	31:0	**n1_rvbaraddr3**	RW	0x0	Cortex n1 Reset Vector Base Address for executing in 64-bit state This value is sampled only during reset of the processor
0x78	**n1_vinithi**	3:0	**n1_vinithi**	RW	0x1	Location of the exception vectors at reset.
0x80	**bypass_boot_change**	31:0	**bypass_boot_change**	RW	0x0	For boot ROM to poll this register when booting with nand and boot_bypass=1
0x84	**bypass_boot_target**	31:0	**bypass_boot_target**	RW	0x0	Used as branch target

Confidential Language Only
For Mekra Lang (2025-08-01)

Offset	Register	Bits	Field	Access	Reset	Description
0x90	sec0	31	sec_scratch-pad_sec	RW	0x1	Security level of each module 1: Secure 0: Non-Secure The reset value of the security level register field sec_xxx depends on the system boot mode. If the system is in "secure boot" mode, the reset value is 1. Otherwise, it is 0.
		30	sec_scratch-pad_nsec	RW	0x1	Same as above. (sec_scratchpad_sec)
		29	sec_dma1	RW	0x1	Same as above. (sec_scratchpad_sec)
		28	sec_dma0	RW	0x1	Same as above. (sec_scratchpad_sec)
		27	sec_ehci	RW	0x1	Same as above. (sec_scratchpad_sec)
		26	sec_ohci	RW	0x1	Same as above. (sec_scratchpad_sec)
		25	sec_dai	RW	0x1	For I2S module Same as above. (sec_scratchpad_sec)
		24	sec_fio8k_hi	RW	0x1	Same as above. (sec_scratchpad_sec)
		23	sec_fio8k_lo	RW	0x1	Same as above. (sec_scratchpad_sec)
		22	sec_uart3	RW	0x1	Same as above. (sec_scratchpad_sec)
		21	sec_uart2	RW	0x1	Same as above. (sec_scratchpad_sec)
		20	sec_uart1	RW	0x1	Same as above. (sec_scratchpad_sec)
		19	sec_uart0	RW	0x1	Same as above. (sec_scratchpad_sec)
		18	sec_cehu1	RW	0x1	Same as above. (sec_scratchpad_sec)
		17	sec_cehu0	RW	0x1	Same as above. (sec_scratchpad_sec)
		16	sec_ssi3	RW	0x1	Same as above. (sec_scratchpad_sec)
		15	sec_ssi2	RW	0x1	Same as above. (sec_scratchpad_sec)
		14	sec_ssi1	RW	0x1	Same as above. (sec_scratchpad_sec)
		13	sec_ssi0	RW	0x1	Same as above. (sec_scratchpad_sec)
		12	sec_ssis0	RW	0x1	Same as above. (sec_scratchpad_sec)
		11	sec_enet1	RW	0x1	Same as above. (sec_scratchpad_sec)
		10	sec_amba_ecru	RW	0x1	Same as above. (sec_scratchpad_sec)
		9	sec_gdma	RW	0x1	Same as above. (sec_scratchpad_sec)
		8	sec_enet	RW	0x1	Same as above. (sec_scratchpad_sec)
		7	sec_vout	RW	0x1	Same as above. (sec_scratchpad_sec)
		6	sec_usb	RW	0x1	Same as above. (sec_scratchpad_sec)
		5	sec_sdio0	RW	0x1	Same as above. (sec_scratchpad_sec)
		4	sec_sd	RW	0x1	Same as above. (sec_scratchpad_sec)
		3	sec_timer2	RW	0x1	Same as above. (sec_scratchpad_sec)
		2	sec_flash_cpu	RW	0x1	Same as above. (sec_scratchpad_sec)
		1	sec_nor_spi	RW	0x1	Same as above. (sec_scratchpad_sec)
		0	sec_cpuid	RW	0x1	Same as above. (sec_scratchpad_sec)

Offset	Register	Bits	Field	Access	Reset	Description
0x94	sec1	31	sec_ssi5	RW	0x1	Same as 0x90
		30	sec_ssi4	RW	0x1	Same as 0x90
		29	sec_wdt_r52	RW	0x1	Same as 0x90
		28	sec_wdt_core3	RW	0x1	Same as 0x90
		27	sec_wdt_core2	RW	0x1	Same as 0x90
		26	sec_wdt_core1	RW	0x1	Same as 0x90
		25	sec_wdt_core0	RW	0x1	Same as 0x90
		24	sec_can5	RW	0x1	Same as 0x90
		23	sec_can4	RW	0x1	Same as 0x90
		22	sec_can3	RW	0x1	Same as 0x90
		21	sec_can2	RW	0x1	Same as 0x90
		20	sec_gpio3	RW	0x1	Same as 0x90
		19	sec_gpio2	RW	0x1	Same as 0x90
		18	sec_gpio1	RW	0x1	Same as 0x90
		17	sec_gpio0	RW	0x1	Same as 0x90
		16	sec_wdt	RW	0x1	Same as 0x90
		15	sec_iomux	RW	0x1	Same as 0x90
		14	sec_idc5	RW	0x1	Same as 0x90
		13	sec_idc4	RW	0x1	Same as 0x90
		12	sec_idc3	RW	0x1	Same as 0x90
		11	sec_idc2	RW	0x1	Same as 0x90
		10	sec_idc1	RW	0x1	Same as 0x90
		9	sec_idc0	RW	0x1	Same as 0x90
		8	sec_idcs	RW	0x1	Same as 0x90
		7	sec_timer1	RW	0x1	Same as 0x90
		6	sec_timer0	RW	0x1	Same as 0x90
		5	sec_pwm2	RW	0x1	Same as 0x90
		4	sec_pwm1	RW	0x1	Same as 0x90
		3	sec_pwm0	RW	0x1	Same as 0x90
		2	sec_uart	RW	0x1	Same as 0x90
		1	sec_can1	RW	0x1	Same as 0x90
		0	sec_can0	RW	0x1	Same as 0x90

Ambarella Confidential Under Non-Disclosure Agreement

Offset	Register	Bits	Field	Access	Reset	Description
0x98	sec2	31:28	Reserved	-	-	Same as 0x90
		27	sec_dbg_dbse	RW	0x1	Same as 0x90
		26	sec_dbg_fex	RW	0x1	Same as 0x90
		25	sec_dbg_ist	RW	0x1	Same as 0x90
		24	Reserved	-	-	
		23	Reserved	-	-	
		22	Reserved	-	-	
		21	sec_dbg_fmem	RW	0x1	Same as 0x90
		20	Reserved	-	-	
		19	sec_dbg_ vmem0_pmem	RW	0x1	Same as 0x90
		18	sec_dbg_vp0	RW	0x1	Same as 0x90
		17	Reserved	-	-	Same as 0x90
		16	sec_dbg_mipi	RW	0x1	Same as 0x90
		15	sec_dbg_mdxf	RW	0x1	Same as 0x90
		14	Reserved	-	-	
		13	Reserved	-	-	
		12	sec_dbg_idsp	RW	0x1	Same as 0x90
		11	Reserved	-	-	
		10	sec_dbg_ vmem0	RW	0x1	Same as 0x90
		9	sec_dbg_rtc	RW	0x1	Same as 0x90
		8	sec_dbg_dorc	RW	0x1	Same as 0x90
		7	Reserved	-	-	
		6	sec_dbg_I2c	RW	0x1	Same as 0x90
		5	sec_dbg_dram	RW	0x1	Same as 0x90
		4	sec_dbg_rct	RW	0x1	Same as 0x90
		3	sec_dbg_smem	RW	0x1	Same as 0x90
		2	sec_dbg_corc	RW	0x1	Same as 0x90
		1	sec_dbg_eorc	RW	0x1	Same as 0x90
		0	sec_dbg_vorc	RW	0x1	Same as 0x90

Confidential
For Ambarella Internal Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0xa0	**saf0**	31	**saf_scratch-pad_sec**	RW	0x1	Safety for access permission of each module 1: Only R52 CPU can access module 0: All CPUs can access module
		30	**saf_scratch-pad_nsec**	RW	0x1	Same as above. (saf_scratchpad_sec)
		29	**saf_dma1**	RW	0x1	Same as above. (saf_scratchpad_sec)
		28	**saf_dma0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		27	**saf_ehci**	RW	0x1	Same as above. (saf_scratchpad_sec)
		26	**saf_ohci**	RW	0x1	Same as above. (saf_scratchpad_sec)
		25	**saf_dai**	RW	0x1	For I2S module. Same as above. (saf_scratchpad_sec)
		24	**saf_fio8k_hi**	RW	0x1	Same as above. (saf_scratchpad_sec)
		23	**saf_fio8k_lo**	RW	0x1	Same as above. (saf_scratchpad_sec)
		22	**saf_uart3**	RW	0x1	Same as above. (saf_scratchpad_sec)
		21	**saf_uart2**	RW	0x1	Same as above. (saf_scratchpad_sec)
		20	**saf_uart1**	RW	0x1	Same as above. (saf_scratchpad_sec)
		19	**saf_uart0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		18	**saf_cehu1**	RW	0x1	Same as above. (saf_scratchpad_sec) Same as above. (saf_scratchpad_sec)
		17	**saf_cehu0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		16	**saf_ssi3**	RW	0x1	Same as above. (saf_scratchpad_sec)
		15	**saf_ssi2**	RW	0x1	Same as above. (saf_scratchpad_sec)
		14	**saf_ssi1**	RW	0x1	Same as above. (saf_scratchpad_sec)
		13	**saf_ssi0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		12	**saf_ssis0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		11	**saf_enet1**	RW	0x1	Same as above. (saf_scratchpad_sec)
		10	**saf_amba_ecru**	RW	0x1	Same as above. (saf_scratchpad_sec)
		9	**saf_gdma**	RW	0x1	Same as above. (saf_scratchpad_sec)
		8	**saf_enet**	RW	0x1	Same as above. (saf_scratchpad_sec)
		7	**saf_vout**	RW	0x1	Same as above. (saf_scratchpad_sec)
		6	**saf_usb**	RW	0x1	Same as above. (saf_scratchpad_sec)
		5	**saf_sdio0**	RW	0x1	Same as above. (saf_scratchpad_sec)
		4	**saf_sd**	RW	0x1	Same as above. (saf_scratchpad_sec)
		3	**saf_timer2**	RW	0x1	Same as above. (saf_scratchpad_sec)
		2	**saf_flash_cpu**	RW	0x1	Same as above. (saf_scratchpad_sec)
		1	**saf_nor_spi**	RW	0x1	Same as above. (saf_scratchpad_sec)
		0	**saf_cpuid**	RW	0x1	Same as above. (saf_scratchpad_sec)

Offset	Register	Bits	Field	Access	Reset	Description
0xa4	saf1	31	saf_ssi5	RW	0x1	Same as 0xa0
		30	saf_ssi4	RW	0x1	Same as 0xa0
		29	saf_wdt_r52	RW	0x1	Same as 0xa0
		28	saf_wdt_core3	RW	0x1	Same as 0xa0
		27	saf_wdt_core2	RW	0x1	Same as 0xa0
		26	saf_wdt_core1	RW	0x1	Same as 0xa0
		25	saf_wdt_core0	RW	0x1	Same as 0xa0
		24	saf_can5	RW	0x1	Same as 0xa0
		23	saf_can4	RW	0x1	Same as 0xa0
		22	saf_can3	RW	0x1	Same as 0xa0
		21	saf_can2	RW	0x1	Same as 0xa0
		20	saf_gpio3	RW	0x1	Same as 0xa0
		19	saf_gpio2	RW	0x1	Same as 0xa0
		18	saf_gpio1	RW	0x1	Same as 0xa0
		17	saf_gpio0	RW	0x1	Same as 0xa0
		16	saf_wdt	RW	0x1	Same as 0xa0
		15	saf_iomux	RW	0x1	Same as 0xa0
		14	saf_idc5	RW	0x1	Same as 0xa0
		13	saf_idc4	RW	0x1	Same as 0xa0
		12	saf_idc3	RW	0x1	Same as 0xa0
		11	saf_idc2	RW	0x1	Same as 0xa0
		10	saf_idc1	RW	0x1	Same as 0xa0
		9	saf_idc0	RW	0x1	Same as 0xa0
		8	saf_idcs	RW	0x1	Same as 0xa0
		7	saf_timer1	RW	0x1	Same as 0xa0
		6	saf_timer0	RW	0x1	Same as 0xa0
		5	saf_pwm2	RW	0x1	Same as 0xa0
		4	saf_pwm1	RW	0x1	Same as 0xa0
		3	saf_pwm0	RW	0x1	Same as 0xa0
		2	saf_uart	RW	0x1	Same as 0xa0
		1	saf_can1	RW	0x1	Same as 0xa0
		0	saf_can0	RW	0x1	Same as 0xa0

Ambarella Confidential and Proprietary

Offset	Register	Bits	Field	Access	Reset	Description
0xa8	saf2	31:28	Reserved	-	-	Same as 0xa0
		27	saf_dbg_dbse	RW	0x1	Same as 0xa0
		26	saf_dbg_fex	RW	0x1	Same as 0xa0
		25	saf_dbg_ist	RW	0x1	Same as 0xa0
		24	Reserved	-	-	
		23	Reserved	-	-	
		22	Reserved	-	-	
		21	saf_dbg_fmem	RW	0x1	Same as 0xa0
		20	Reserved	-	-	
		19	saf_dbg_ vmem0_pmem	RW	0x1	Same as 0xa0
		18	saf_dbg_vp0	RW	0x1	Same as 0xa0
		17	Reserved	-	-	
		16	saf_dbg_mipi	RW	0x1	Same as 0xa0
		15	saf_dbg_mdxf	RW	0x1	Same as 0xa0
		14	Reserved	-	-	
		13	Reserved	-	-	
		12	saf_dbg_idsp	RW	0x1	Same as 0xa0
		11	Reserved	-	-	
		10	saf_dbg_ vmem0	RW	0x1	Same as 0xa0
		9	saf_dbg_rtc	RW	0x1	Same as 0xa0
		8	saf_dbg_dorc	RW	0x1	Same as 0xa0
		7	Reserved	-	-	
		6	saf_dbg_I2c	RW	0x1	Same as 0xa0
		5	saf_dbg_dram	RW	0x1	Same as 0xa0
		4	saf_dbg_rct	RW	0x1	Same as 0xa0
		3	saf_dbg_smem	RW	0x1	Same as 0xa0
		2	saf_dbg_corc	RW	0x1	Same as 0xa0
		1	saf_dbg_eorc	RW	0x1	Same as 0xa0
		0	saf_dbg_vorc	RW	0x1	Same as 0xa0
0xbc	axi_dummy_ reg	31:0	axi_dummy_reg	RW	0xbeefbabe	For safety purpose. To detect with high effectiveness if any reset source is in the inactive state
0xc0	axi_cur_time	31:0	axi_cur_time	RO	0x0	Current_time from CODE
0xc4	axi_Reserved_ reg	31:0	axi_Reserved_ reg	RW	0x0	Reserved register for ECO
0xc8	addr_start_0	31:0	addr_start_0	RW	0x0	DRAM starting address
0xcc	addr_end_0	31:0	addr_end_0	RW	0xdffdffff	DRAM ending address
0xd0	addr_start_1	31:0	addr_start_1	RW	0xdffe0000	DRAM Config starting address
0xd4	addr_end_1	31:0	addr_end_1	RW	0xdffe7fff	DRAM Config ending address
0xd8	addr_start_2	31:0	addr_start_2	RW	0xf2000000	AXI Config starting address
0xdc	addr_end_2	31:0	addr_end_2	RW		AXI Config ending address
0xe0	addr_start_3	31:0	addr_start_3	RW	0xe0000000	Other starting address (Default value is AHB starting address)
0xe4	addr_end_3	31:0	addr_end_3	RW	0xe3ffffff	Other ending address (Default value is AHB ending address)

Ambarella Confidential
For Internal Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0xe8	addr_start_4	31:0	addr_start_4	RW	0xe4000000	Other starting address (Default value is APB starting address)
0xec	addr_end_4	31:0	addr_end_4	RW	0xe7ffffff	Other ending address (Default value is APB ending address)
0xf0	addr_start_5	31:0	addr_start_5	RW	0xed000000	Other starting address (Default value is APB_DBG starting address)
0xf4	addr_end_5	31:0	addr_end_5	RW	0xefffffff	Other ending address (Default value is APB_DBG ending address)
0xf8	addr_start_6	31:0	addr_start_6	RW	0xf3000000	Other starting address (Default value is GIC starting address)
0xfc	addr_end_6	31:0	addr_end_6	RW	0xf3ffffff	Other ending address (Default value is GIC ending address)
0x100	addr_start_7	31:0	addr_start_7	RW	0xf1000000	Other starting address (Default value is GPV starting address)
0x104	addr_end_7	31:0	addr_end_7	RW	0xf10fffff	Other ending address (Default value is GPV ending address)
0x108	addr_start_8	31:0	addr_start_8	RW	0x0	Other starting address
0x10c	addr_end_8	31:0	addr_end_8	RW	0x0	Other ending address
0x110	addr_start_9	31:0	addr_start_9	RW	0x0	Other starting address
0x114	addr_end_9	31:0	addr_end_9	RW	0x0	Other ending address
0x118	range_ck_en	31:10	Reserved	-	-	
		9	range_ck_en_9	RW	0x0	Address range checking enable
		8	range_ck_en_8	RW	0x0	Address range checking enable
		7	range_ck_en_7	RW	0x1	Address range checking enable
		6	range_ck_en_6	RW	0x1	Address range checking enable
		5	range_ck_en_5	RW	0x1	Address range checking enable
		4	range_ck_en_4	RW	0x1	Address range checking enable
		3	range_ck_en_3	RW		Address range checking enable
		2	range_ck_en_2	RW	0x1	AXI Config address range checking enable
		1	range_ck_en_1	RW	0x1	DRAM Config address range checking enable
		0	range_ck_en_0	RW	0x1	DRAM address range checking enable
0x11C	error_output_mask	31:3	Reserved	-	-	
		2	arc_mask	RW	0x0	1: Mask arc invalid from output
		1	nexterrirq_mask	RW	0x0	1: Mask nEXTERRIRQ from output
		0	ninterrirq_mask	RW	0x0	1: Mask nINTERRIRQ from output
0x120	invalid_addr	31:0	invalid_addr	RO	0x0	The invalid address which causes address range checking error

For Confidential Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x12C	**ckm_ctrl_0**	31	**ckm_enable_gclk_axi1**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	-	
		27:16	**ckm_upper_bound_gclk_axi1**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	-	
		11:0	**ckm_lower_bound_gclk_axi1**	RW	0x0	Lower bound of clock monitor compare value
0x130	**ckm_ctrl_1**	31	**ckm_enable_n0_pclkdbg_clk**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	-	
		27:16	**ckm_upper_bound_n0_pclkdbg_clk**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	-	
		11:0	**ckm_lower_bound_n0_pclkdbg_clk**	RW	0x0	Lower bound of clock monitor compare value
0x134	**ckm_ctrl_2**	31	**ckm_enable_n1_pclkdbg_clk**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	-	
		27:16	**ckm_upper_bound_n1_pclkdbg_clk**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	-	
		11:0	**ckm_lower_bound_n1_pclkdbg_clk**	RW	0x0	Lower bound of clock monitor compare value
0x138	**ckm_ctrl_3**	31	**ckm_enable_gclk_cortex0**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	-	
		27:16	**ckm_upper_bound_gclk_cortex0**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	-	
		11:0	**ckm_lower_bound_gclk_cortex0**	RW	0x0	Lower bound of clock monitor compare value
0x13C	**ckm_ctrl_4**	31	**ckm_enable_gclk_cortex1**	RW	0x0	Enable signal of clock monitor
		30:28	**Reserved**	-	-	
		27:16	**ckm_upper_bound_gclk_cortex1**	RW	0x0	Upper bound of clock monitor compare value
		15:12	**Reserved**	-	-	
		11:0	**ckm_lower_bound_gclk_cortex1**	RW	0x0	Lower bound of clock monitor compare value

Ambarella Confidential
For Internal Language only
(Dec 2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x140	**ckm_mea_cycle_0**	31:12	**Reserved**	-	-	
		11:0	**ckm_mea_cycle_gclk_axi1**	RO	0x0	Current clock monitor counting value for debugging purposes
0x144	**ckm_mea_cycle_1**	31:12	**Reserved**	-	-	
		11:0	**ckm_mea_cycle_n0_pclk_dbg_clk**	RO	0x0	Current clock monitor counting value for debugging purposes
0x148	**ckm_mea_cycle_2**	31:12	**Reserved**	-	-	
		11:0	**ckm_mea_cycle_n1_pclk_dbg_clk**	RO	0x0	Current clock monitor counting value for debugging purposes
0x14C	**ckm_mea_cycle_3**	31:12	**Reserved**	-	-	
		11:0	**ckm_mea_cycle_gclk_cortex0**	RO	0x0	Current clock monitor counting value for debugging purposes
0x150	**ckm_mea_cycle_4**	31:12	**Reserved**	-	-	
		11:0	**ckm_mea_cycle_gclk_cortex1**	RO	0x0	Current clock monitor counting value for debugging purposes
0x154	**dclscompin**	31:18	**Reserved**	-	-	
		17:0	**dclscompin**	RW	0x15555	R52 DCLS comparators control. ({DCLSCOMPIN87:84, DCLSCOMPIN13:0}) Core0: DCLSCOMPIN13:0 Global: DCLSCOMPIN87:84
0x158	**sw_arm_safety_error**	31:1	**Reserved**	-	-	
		0	**sw_arm_safety_error**	RW	0x0	Software write this bit to 1 to issue a safety error to CEHU.

Table 17-1. AXI Registers.

Confidential
For Ambarella Lang only
(08 2025)

18. GENERIC INTERRUPT CONTROLLER (GIC)

This chapter provides register programming information for the CV22FS generic interrupt controller (GIC). The chapter is organized as follows:

- (Section 18.1) GIC: Overview
- (Section 18.2) GIC: Interfaces
- (Section 18.3) GIC: Interrupt Vectors
- (Section 18.4) GIC: GIC-400 Registers
- (Section 18.5) GIC: Distributor Register Summary
- (Section 18.6) GIC: CPU Interface Register Summary
- (Section 18.7) GIC: GIC Virtual Interface Control Register Summary
- (Section 18.8) GIC: GIC Virtual CPU Interface Register Summary

18.1 GIC: Overview

The diagram below provides an overview of the generic interrupt controller (GIC) in a multiprocessor system. It shows the interrupts that are sent to the GIC-400 from various sources and the key phases of interrupt-related signaling in the SoC.

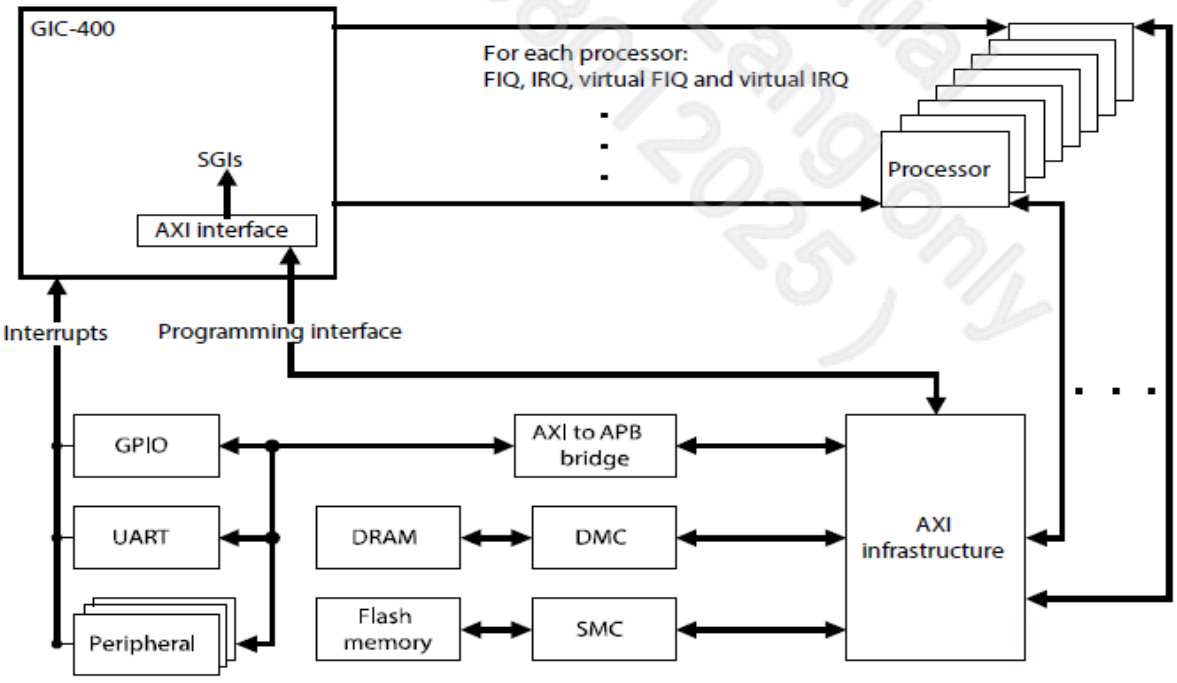


Figure 18-1. Overview Diagram of the CV22FS Generic Interrupt Controller (GIC).

The GIC-400 detects PPIs and SPIs from interrupt input signals. There is one signal for each processor for every PPI interrupt ID. There is only one input signal for each SPI interrupt ID, irrespective of the number of processors

in the SoC. SGIs do not have input signals and are generated in the GIC-400 using the AXI programming interface.

Notes:

- The GIC-400 does not synchronize any inputs. Therefore, all input signals, including the SPI and PPI inputs, must be synchronous to CLK.
- The GIC-400 notifies each processor of the presence of an interrupt or virtual interrupt by using interrupt output signals. Additionally, there are interrupt output signals to provide wakeup functionality to a system power controller.
- Virtual interrupts are created and managed by special software on each processor that runs on virtual machines. Such hypervisors are not part of the GIC-400 architecture, but are necessary for virtualizing interrupts using the interrupt controller.
- For further details about initialization and programming sequences, refer to *CoreLink™ GIC-400 Generic Interrupt Controller Technical Reference Manual (r0p1)*.

18.2 GIC: Interfaces

The GIC-400 provides an AMBA AXI4 slave interface. With this interface, users can program the Distributor, including generating SGIs, the CPU interfaces, and the virtual CPU interfaces.

The GIC-400 supports a set of interrupt signals that it samples to generate PPIs and SPIs. Additionally, it generates signals to the wakeup controller and to the processors to indicate that there are valid pending interrupts in the physical and virtual CPU interfaces.

18.3 GIC: Interrupt Vectors

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
223	-	*This bit is reserved for interrupt assignment from GIC to R52*			
222	dma1_int_dup	amba.amba0.ahb.dma_lockstep_top1	Level	Interrupt duplication of dma1	-
221	dma0_int_dup	amba.amba0.ahb.dma_lockstep_top0	Level	Interrupt duplication of dma0	-
220	i2c5_irq_dup	amba.amba0.apb.idc5	Level	Interrupt duplication of i2c5	-
219	i2c4_irq_dup	amba.amba0.apb.idc4	Level	Interrupt duplication of i2c4	-
218	i2c3_irq_dup	amba.amba0.apb.idc3	Level	Interrupt duplication of i2c3	-
217	i2c2_irq_dup	amba.amba0.apb.idc2	Level	Interrupt duplication of i2c2	-
216	i2c1_irq_dup	amba.amba0.apb.idc1	Level	Interrupt duplication of i2c1	-

For Meta Lang only Confidential (10/08/2025)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
215	i2c_irq_dup	amba.amba0.apb.idc0	Level	Interrupt duplication of i2c0	-
214	idcs_irq_dup	amba.amba0.apb.idcs	Level	Interrupt duplication of i2c slave	-
213	wdt_core3_intr_dup	amba.amba0.apb.wdt_core3	Edge	Interrupt duplication of wdt_core3	Interval is long enough
212	wdt_core2_intr_dup	amba.amba0.apb.wdt_core2	Edge	Interrupt duplication of wdt_core2	Interval is long enough
211	wdt_core1_intr_dup	amba.amba0.apb.wdt_core1	Edge	Interrupt duplication of wdt_core1	Interval is long enough
210	wdt_core0_intr_dup	amba.amba0.apb.wdt_core0	Edge	Interrupt duplication of wdt_core0	Interval is long enough
209	wdt_r52_intr_dup	amba.amba0.apb.wdt_r52	Edge	Interrupt duplication of wdt_r52	Interval is long enough
208	wdt_intr_dup	amba.amba0.apb.wdt	Edge	Interrupt duplication of wdt	Interval is long enough
207	ssi5_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi5	-
206	ssi4_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi4	-
205	ssi3_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi3	-
204	ssi2_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi2	-
203	ssi1_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi1	-
202	ssi0_ahb_intr_dup	amba.amba0.ahb.ssi5	Level	Interrupt duplication of ssi0	-
201	ssis0_ahb_intr_dup	amba.amba0.ahb.ssis0	Level	Interrupt duplication of ssi slave	-
200	canc5_irq_sm_dup	amba.amba0.ahb.canc5	Level	Interrupt duplication of canc5	-
199	canc4_irq_sm_dup	amba.amba0.ahb.canc4	Level	Interrupt duplication of canc4	-
198	canc3_irq_sm_dup	amba.amba0.ahb.canc3	Level	Interrupt duplication of canc3	-
197	canc2_irq_sm_dup	amba.amba0.ahb.canc2	Level	Interrupt duplication of canc2	-
196	canc1_irq_sm_dup	amba.amba0.ahb.canc1	Level	Interrupt duplication of canc1	-
195	canc0_irq_sm_dup	amba.amba0.ahb.canc0	Level	Interrupt duplication of canc0	-
194	gpiointr3_dup	amba.amba0.apb.gpio3	Level	Interrupt duplication of gpio3	-
193	gpiointr2_dup	amba.amba0.apb.gpio2	Level	Interrupt duplication of gpio2	-

Confidential For Weika Lab only (2023-03-13)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
192	gpiointr1_dup	amba.amba0.apb.gpio1	Level	Interrupt duplication of gpio1	-
191	gpiointr0_dup	amba.amba0.apb.gpio0	Level	Interrupt duplication of gpio0	-
190	vorc_thread4_irq_dup	amb_core.storc.vorc	Edge	Interrupt duplication	Program by software
189	vorc_thread3_irq_dup	amb_core.storc.vorc	Edge	Interrupt duplication	Program by software
188	vorc_thread2_irq_dup	amb_core.storc.vorc	Edge	Interrupt duplication	Program by software
187	vorc_thread1_irq_dup	amb_core.storc.vorc	Edge	Interrupt duplication	Program by software
186	vorc_thread0_irq_dup	amb_core.storc.vorc	Edge	Interrupt duplication	Program by software
185	I2c_arm_err_or_event_intr_dup	amb_core.storc.vorc	Level	Interrupt duplication	-
184	code_vout_b_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
183	code_vout_a_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
182	code_vin8_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
181	--	--			
180	--	--			
179	--	--			
178	code_vin4_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
177	code_vin3_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
176	code_vin2_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
175	code_vin1_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
174	code_vin0_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
173	code_vdsp3_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
172	code_vdsp2_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
171	code_vdsp1_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
170	code_vdsp0_irq_dup	vdsp1	Edge	Interrupt duplication	Program by software
169	--	--			

Confidential For Wekra Land only (08012025)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
168	idsp_arm_safety_intr	idsp1	Edge	Interrupt from IDSP to Arm to indicate watchdog timeout error in IDSP controller or VIN status error (this includes ECC / CRC error, wrong frame start/ frame end packets, short line/frame, SFIFO / AFIFO overflow and watchdog timeout)	-
167	dtte_tracking_fail_ intr_16b	ddrhoste	Level	DTTE training fail interrupt for 16-bit DDRHOST	-
166	dtte_tracking_fail_ intr	ddrhost0	Level	DTTE training fail interrupt for 32-bit DDRHOST	-
165	vorc_thread4_irq	amb_core.storc.vorc	Edge	Software initiated interrupt from the Vision ORC	Program by software
164	vorc_thread3_irq	amb_core.storc.vorc	Edge	Software initiated interrupt from the Vision ORC	Program by software
163	vorc_thread2_irq	amb_core.storc.vorc	Edge	Software initiated interrupt from the Vision ORC	Program by software
162	vorc_thread1_irq	amb_core.storc.vorc	Edge	Software initiated interrupt from the Vision ORC	Program by software
161	vorc_thread0_irq	amb_core.storc.vorc	Edge	Software initiated interrupt from the Vision ORC	Program by software
160	l2c_arm_err_or_ event_intr	amb_core.storc.vorc	Level	ORC_L2-Cache event monitor/counter interrupt	-
159	code_vout_b_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
158	code_vout_a_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
157	code_vin8_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
156	--	--			

Confidential Large Corp
For Mekra
1.5 (08/01/2025)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
155	--	--			
154	--	--			
153	code_vin4_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
152	code_vin3_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
151	code_vin2_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
150	code_vin1_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
149	code_vin0_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
148	code_vdsp3_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
147	code_vdsp2_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
146	code_vdsp1_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
145	code_vdsp0_irq	vdsp1	Edge	Software initiated interrupt from the Coding ORC	Program by software
144	vin8_arm_delayed_vsync	idsp	Edge	Sensor frame end interrupt from vin8 to Arm	No back-to-back event
143	vin8_arm_last_pix	idsp	Edge	Delayed sensor frame end interrupt from vin8 to Arm	No back-to-back event
142	vin8_arm_sof	idsp	Edge	First output pixel interrupt from vin8 to Arm	No back-to-back event
141	vin8_arm_vsync	idsp	Edge	Last output pixel interrupt from vin8 to Arm	No back-to-back event
140	wdt_core3_intr	amba.amba0.apb.wdt_core3	Edge	WDT interrupt for CEMU	Interval is long enough
139	wdt_core2_intr	amba.amba0.apb.wdt_core2	Edge	WDT interrupt for CEMU	Interval is long enough
138	wdt_core1_intr	amba.amba0.apb.wdt_core1	Edge	WDT interrupt for CEMU	Interval is long enough

Confidential For Mekra Lang only 1.5 (08012025)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
137	wdt_core0_intr	amba.amba0.apb.wdt_core0	Edge	WDT interrupt for CEMU	Interval is long enough
136	wdt_r52_intr	amba.amba0.apb.wdt_r52	Edge	WDT interrupt for CR52	Interval is long enough
135	ssi5_ahb_intr	amba.amba0.ahb.ssi5	Level	SSI5 interrupt	-
134	ssi4_ahb_intr	amba.amba0.ahb.ssi4	Level	SSI4 interrupt	-
133	sbd1_intr_o1	amba.amba0.ahb.enet1	Level	Ethernet interrupt event of enet1 instance to the application	-
132	pmt1_intr_o1	amba.amba0.ahb.enet1	Level	Power management interrupt event of enet1 instance.	-
131	sp_nonsec_lock_access_irq	amba.amba0.ahb.ahb_scratchpad_nsec	Edge	Lock access interrupt of non-secure scratchpad	5 AHB clock cycles
130	sp_sec_lock_access_irq	amba.amba0.ahb.ahb_scratchpad_sec	Edge	Lock access interrupt of secure scratchpad	5 AHB clock cycles
129	vin4_arm_delayed_vsync	idsp	Edge	delayed sensor frame end interrupt from vin4 to Arm	No back-to-back event
128	vin4_arm_last_pix	idsp	Edge	Last output pixel interrupt from vin4 to Arm	No back-to-back event
127	axi_cfg_lock_access_irq	amba.amba0.axi.axi_config	Edge	Lock access interrupt of axi_config	5 AXI1 clock cycles
126	vin4_arm_sof	idsp	Edge	First output pixel interrupt from vin4 to Arm	No back-to-back event
125	vin4_arm_vsync	idsp	Edge	Sensor frame end interrupt from vin4 to Arm	No back-to-back event
124	vin3_arm_delayed_vsync	idsp	Edge	Delayed sensor frame end interrupt from vin3 to Arm	No back-to-back event
123	vin3_arm_last_pix	idsp	Edge	Last output pixel interrupt from vin3 to Arm	No back-to-back event
122	apb_dbg_lock_access_irq	amba.amba0.apb.apb_dbg.apb_dbg_core	Edge	Lock access interrupt of apb_dbg	5 AHB clock cycles
121	vin3_arm_sof	idsp	Edge	First output pixel interrupt from vin3 to Arm	No back-to-back event

Confidential Lang 0801 2025 For Ambarella

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
120	vin3_arm_vsync	idsp	Edge	Sensor frame end interrupt from vin3 to Arm	No back-to-back event
119	vin2_arm_delayed_vsync	idsp	Edge	Delayed sensor frame end interrupt from vin2 to Arm	No back-to-back event
118	vin2_arm_last_pix	idsp	Edge	Last output pixel interrupt from vin2 to Arm	No back-to-back event
117	vin2_arm_sof	idsp	Edge	First output pixel interrupt from vin2 to Arm	No back-to-back event
116	vin2_arm_vsync	idsp	Edge	Sensor frame end interrupt from vin2 to Arm	No back-to-back event
115	vin1_arm_delayed_vsync	idsp	Edge	Delayed sensor frame end interrupt from vin1 to Arm	No back-to-back event
114	vin1_arm_last_pix	idsp	Edge	Last output pixel interrupt from vin1 to Arm	No back-to-back event
113	vin1_arm_master_vsync	idsp	Edge	Master vsync interrupt from vin1 to Arm	No back-to-back event
112	vin1_arm_sof	idsp	Edge	First output pixel interrupt from vin1 to Arm	No back-to-back event
111	vin1_arm_vsync	idsp	Edge	Sensor frame end interrupt from vin1 to Arm	No back-to-back event
110	vin0_arm_delayed_vsync	idsp	Edge	Delayed sensor frame end interrupt from vin0 to Arm	No back-to-back event
109	vin0_arm_last_pix	idsp	Edge	Last output pixel interrupt from vin0 to Arm	No back-to-back event
108	vin0_arm_master_vsync	idsp	Edge	Master vsync interrupt from vin0 to Arm	No back-to-back event
107	vin0_arm_sof	idsp	Edge	First output pixel interrupt from vin0 to Arm	No back-to-back event
106	vin0_arm_vsync	idsp	Edge	Sensor frame end interrupt from vin0 to Arm	No back-to-back event
105	vp0_exception_irq	amb_core.vp0	Level	VP exception interrupt	-

For Mekra Lang GmbH (1.5 / 08012025)
Confidential

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
104	dram_vic_error_intr	amba.amba1	Level	DRAM interrupt	-
103	amba_axi_software_irq[13]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
102	amba_axi_software_irq[12]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
101	amba_axi_software_irq[11]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
100	amba_axi_software_irq[10]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
99	amba_axi_software_irq[9]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
98	amba_axi_software_irq[8]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
97	amba_axi_software_irq[7]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
96	amba_axi_software_irq[6]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
95	amba_axi_software_irq[5]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
94	amba_axi_software_irq[4]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
93	amba_axi_software_irq[3]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
92	amba_axi_software_irq[2]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
91	amba_axi_software_irq[1]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
90	amba_axi_software_irq[0]	amba.amba0.ahb.ahb_scratchpad_nsec	Level	AMBA to AXI software interrupt	-
89	rng_intr	amba.amba0.ahb.ahb_scratchpad_nsec	Level	RNG interrupt	-
88	ahb_vout_a_arm_interrupt	amba.amba0.ahb.vout	Edge	VOUT_A to AHB interrupt	No back-to-back event
87	ahb_vout_b_arm_interrupt	amba.amba0.ahb.vout	Edge	VOUT_B to AHB interrupt	No back-to-back event
86	dma1_int	amba.amba0.ahb.dma_lockstep_top1	Level	DMA1 interrupt	-
85	dma0_int	amba.amba0.ahb.dma_lockstep_top0	Level	DMA0 interrupt	-
84	i2s_rx_irq	amba.amba0.ahb.i2s	Level	I2S receiver interrupt	-
83	i2s_tx_irq	amba.amba0.ahb.i2s	Level	I2S transmitter interrupt	-
82	ssis0_ahb_intr	amba.amba0.ahb.ssis0	Level	SSI slave interrupt	-
81	ssi3_ahb_intr	amba.amba0.ahb.ssi3	Level	SSI3 interrupt	-
80	ssi2_ahb_intr	amba.amba0.ahb.ssi2	Level	SSI2 interrupt	-

Ambarella Confidential Property Limited Copy

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
79	ssi1_ahb_intr	amba.amba0.ahb.ssi1	Level	SSI1 interrupt	-
78	ssi0_ahb_intr	amba.amba0.ahb.ssi0	Level	SSI0 interrupt	-
77	nor_spi_interrupt	amba.amba0.ahb.nor_spi	Level	NOR_SPI interrupt	-
76	intr_sd	amba.amba0.ahb.sd	Level	SD operation done interrupt	-
75	intr_sdio_0	amba.amba0.ahb.sdio0	Level	SDIO operation done interrupt	-
74	gdma_vic_comple-tion_interrupt_ex-tended	amba.amba0.ahb.gdma	Edge	GDMA interrupt	7 AHB clock cycles
73	fio_ecc_rpt_int	amba.amba0.ahb.combo_nand	Level	NAND ECC report interrupt	-
72	fio_int	amba.amba0.ahb.combo_nand	Level	NAND operation done interrupt	-
71	usb_irq_fo	amba.amba0.ahb.usb	Level	USB device interrupt	-
70	~ohci_0_irq_o_n	amba.amba0.ahb.usb_host	Level	USB host interrupt of OHCI mode	-
69	ehci_interrupt_o	amba.amba0.ahb.usb_host	Level	USB host interrupt of EHCI mode	-
68	uart3_ahb_intr	amba.amba0.ahb.uart3	Level	UART3 to AHB interrupt	-
67	uart2_ahb_intr	amba.amba0.ahb.uart2	Level	UART2 to AHB interrupt	-
66	uart1_ahb_intr	amba.amba0.ahb.uart1	Level	UART1 to AHB interrupt	-
65	uart0_ahb_intr	amba.amba0.ahb.uart0	Level	UART0 to AHB interrupt	-
64	sbd_intr_o1	amba.amba0.ahb.enet	Level	Ethernet interrupt event of enet0 instance to the application	-
63	pmt_intr_o1	amba.amba0.ahb.enet	Level	Power management interrupt event of enet0 instance.	-
62	rct_vic_sd_cd_change	amba.amba0.ahb	Edge	SD card detect/removal interrupt	No back-to-back event
61	rct_vic_sdio0_cd_change	amba.amba0.ahb	Edge	SD card detect/removal interrupt	No back-to-back event
60	usbp_rct_chgdet0_sync	amba.amba0.ahb	Edge	Interrupt of USB charger detection	No back-to-back event
59	rct_vic_usb_con-nect_change	amba.amba0.ahb	Edge	Interrupt of change of USB connect status	No back-to-back event
58	rct_vic_usb_connect	amba.amba0.ahb	Level	Interrupt of USB connect status	-

For Ambarella Confidential Use Only (2025)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
57	rct_ahb_usbp0_id-dig0_change	amba.amba0.ahb	Edge	Interrupt of change of USB iddig pin	No back-to-back event
56	gpiointr3	amba.amba0.apb.gpio3	Level	GPIO3 to APB interrupt	-
55	gpiointr2	amba.amba0.apb.gpio2	Level	GPIO2 to APB interrupt	-
54	gpiointr1	amba.amba0.apb.gpio1	Level	GPIO1 to APB interrupt	-
53	gpiointr0	amba.amba0.apb.gpio0	Level	GPIO0 to APB interrupt	-
52	wdt_intr	amba.amba0.apb.wdt	Edge	WDT interrupt	Interval is long enough
51	tm30_intr	amba.amba0.apb.timer2	Edge	Interrupt 10 of timer2	5 APB clock cycles
50	tm29_intr	amba.amba0.apb.timer2	Edge	Interrupt 9 of timer2	5 APB clock cycles
49	tm28_intr	amba.amba0.apb.timer2	Edge	Interrupt 8 of timer2	5 APB clock cycles
48	tm27_intr	amba.amba0.apb.timer2	Edge	Interrupt 7 of timer2	5 APB clock cycles
47	tm26_intr	amba.amba0.apb.timer2	Edge	Interrupt 6 of timer2	5 APB clock cycles
46	tm25_intr	amba.amba0.apb.timer2	Edge	Interrupt 5 of timer2	5 APB clock cycles
45	tm24_intr	amba.amba0.apb.timer2	Edge	Interrupt 4 of timer2	5 APB clock cycles
44	tm23_intr	amba.amba0.apb.timer2	Edge	Interrupt 3 of timer2	5 APB clock cycles
43	tm22_intr	amba.amba0.apb.timer2	Edge	Interrupt 2 of timer2	5 APB clock cycles
42	tm21_intr	amba.amba0.apb.timer2	Edge	Interrupt 1 of timer2	5 APB clock cycles
41	tm20_intr	amba.amba0.apb.timer1	Edge	Interrupt 10 of timer1	5 APB clock cycles
40	tm19_intr	amba.amba0.apb.timer1	Edge	Interrupt 9 of timer1	5 APB clock cycles
39	tm18_intr	amba.amba0.apb.timer1	Edge	Interrupt 8 of timer1	5 APB clock cycles
38	tm17_intr	amba.amba0.apb.timer1	Edge	Interrupt 7 of timer1	5 APB clock cycles
37	tm16_intr	amba.amba0.apb.timer1	Edge	Interrupt 6 of timer1	5 APB clock cycles
36	tm15_intr	amba.amba0.apb.timer1	Edge	Interrupt 5 of timer1	5 APB clock cycles
35	tm14_intr	amba.amba0.apb.timer1	Edge	Interrupt 4 of timer1	5 APB clock cycles

Confidential
For Weekly Release Only
1.3 (08-2023)

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
34	tm13_intr	amba.amba0.apb.timer1	Edge	Interrupt 3 of timer1	5 APB clock cycles
33	tm12_intr	amba.amba0.apb.timer1	Edge	Interrupt 2 of timer1	5 APB clock cycles
32	tm11_intr	amba.amba0.apb.timer1	Edge	Interrupt 1 of timer1	5 APB clock cycles
31	tm10_intr	amba.amba0.apb.timer0	Edge	Interrupt 10 of timer0	5 APB clock cycles
30	tm9_intr	amba.amba0.apb.timer0	Edge	Interrupt 9 of timer0	5 APB clock cycles
29	tm8_intr	amba.amba0.apb.timer0	Edge	Interrupt 8 of timer0	5 APB clock cycles
28	tm7_intr	amba.amba0.apb.timer0	Edge	Interrupt 7 of timer0	5 APB clock cycles
27	tm6_intr	amba.amba0.apb.timer0	Edge	Interrupt 6 of timer0	5 APB clock cycles
26	tm5_intr	amba.amba0.apb.timer0	Edge	Interrupt 5 of timer0	5 APB clock cycles
25	tm4_intr	amba.amba0.apb.timer0	Edge	Interrupt 4 of timer0	5 APB clock cycles
24	tm3_intr	amba.amba0.apb.timer0	Edge	Interrupt 3 of timer0	5 APB clock cycles
23	tm2_intr	amba.amba0.apb.timer0	Edge	Interrupt 2 of timer0	5 APB clock cycles
22	tm1_intr	amba.amba0.apb.timer0	Edge	Interrupt 1 of timer0	5 APB clock cycles
21	uart_0_intr	amba.amba0.apb.uart0	Level	UART to APB interrupt	-
20	idcs_irq	amba.amba0.apb.idcs	Level	I2C slave interrupt	-
19	i2c5_irq	amba.amba0.apb.idc5	Level	I2C5 interrupt	-
18	i2c4_irq	amba.amba0.apb.idc4	Level	I2C4 interrupt	-
17	i2c3_irq	amba.amba0.apb.idc3	Level	I2C3 interrupt	-
16	i2c2_irq	amba.amba0.apb.idc2	Level	I2C2 interrupt	-
15	i2c1_irq	amba.amba0.apb.idc1	Level	I2C1 interrupt	-
14	i2c_irq	amba.amba0.apb.idc	Level	I2C0 interrupt	-
13	~n1_nINTERRIRQ	amba.amba0.axi.axi_n1.axi_arm.u0	Level	CA53 interrupt for AXI or CHI transactions with a write response error condition	-
12	canc5_irq	amba.amba0.ahb.canc5	Level	CANC5 interrupt	-
11	canc4_irq	amba.amba0.ahb.canc4	Level	CANC4 interrupt	-
10	canc3_irq	amba.amba0.ahb.canc3	Level	CANC3 interrupt	-
9	canc2_irq	amba.amba0.ahb.canc2	Level	CANC2 interrupt	-
8	canc1_irq	amba.amba0.ahb.canc1	Level	CANC1 interrupt	-
7	canc0_irq	amba.amba0.ahb.canc0	Level	CANC0 interrupt	-

Confidential For Ambarella Use Only 1.3 08/20/2021

Bit / IRQ No.	Interrupt Name	Interrupt Source	Sensitivity	Description	Min. Interval for Edge-Triggered Interrupt
6	n1_PMUIRQ[3]	amba.amba0.axi.axi_n1.axi_arm.u0	Level	PMU interrupt for CA53 Core3	-
5	n1_PMUIRQ[2]	amba.amba0.axi.axi_n1.axi_arm.u0	Level	PMU interrupt for CA53 Core2	-
4	n1_PMUIRQ[1]	amba.amba0.axi.axi_n1.axi_arm.u0	Level	PMU interrupt for CA53 Core1	-
3	n1_PMUIRQ[0]	amba.amba0.axi.axi_n1.axi_arm.u0	Level	PMU interrupt for CA53 Core0	-
2	~n1_nEXTERRIRQ	amba.amba0.axi.axi_ n1.axi_arm.u0	Level	CA53 interrupt for L2 RAM double-bit ECC error	-
1	cehu1_intr	amba.amba0.ahb.cehu1	Level	CEHU1 interrupt	-
0	cehu0_intr	amba.amba0.ahb.cehu0	Level	CEHU0 interrupt	-

Table 18-1. GIC Interrupt Vectors

GIC-400 is also used for generating inter-communication interrupts between Cortex-A53 and Cortex-R52. To achieve this, Cortex-R52 can access GIC-400. It behaves as core4, while Cortex-CA53 behaves as core0~core3. With the SGI function, Cortex-R52 can interrupt any core of the Cortex-A53 cluster.

Furthermore, IRQ[4] from GIC-400 is connected to the 223th SPI input of Cortex-R52's internal GIC-400. This also enables Cortex-A53 cores to interrupt Cortex-R52.

18.4 GIC: GIC-400 Registers

18.4.1 GIC-400 Registers: Map

All of the GIC-400 registers have short names. In these names, the first three characters are GIC, and the fourth character indicates the functional block of the GIC-400:

- GICD: Distributor
- GICC: CPU interfaces
- GICH: Virtual interface control blocks
- GICV: Virtual CPU interfaces.

The GIC-400 provides the following register aliases for the virtual interface control block:

- An alias that provides access to the virtual CPU interface of the accessing processor using a single base address for all processors. This base address is at offset 0x4000.
- Aliases that permit any virtual CPU interface to be accessed explicitly from any other processor, using a different base address for each processor. The starting base address is at offset 0x5000, with address bits [11:9] as the CPU ID decode.

For Confidential Use Only (Cat 2012015)

Address Range	GIC-400 Functional Block
0x0000-0x0FFF	Reserved
0x1000-0x1FFF	Distributor
0x2000-0x3FFF	CPU Interfaces
0x4000-0x4FFF	Virtual interface control block for the processor that is performing the access
0x5000-0x5FFF	Virtual interface control block for the processor selected by address bits [11:9] 0x5000-0x51FF Alias for Processor 0 0x5200-0x53FF Alias for Processor 1 ... 0x5E00-0x5FFF.....Alias for Processor 7
0x6000-0x7FFF	Virtual CPU interfaces

18.5 GIC: Distributor Register Summary

Offset	Register	Description
0x000	**GICD_CTLR**	Distributor Control register
0x004	**GICD_TYPER**	Interrupt Controller Type register
0x008	**GICD_IIDR**	Distributor Implementer Identification Register (Section 18.5.1.1 “Distributor Implementer Identification Register, GICD_IIDR Register”)
0x080-0x0BC	**GICD_IGROUPRn**	Interrupt Group Registers
0x100 0x104-0x13C	**GICD_ISENABLERn**	Interrupt Set-Enable registers
0x180 0x184-0x1BC	**GICD_ICENABLERn**	Interrupt Clear-Enable registers
0x200-0x23C	**GICD_ISPENDRn**	Interrupt Set-Pending registers
0x280-0x2BC	**GICD_ICPENDRn**	Interrupt Clear-Pending registers
0x300-0x3BC	**GICD_ICACTIVERn**	Interrupt Clear-Active registers
0x400-0x5FC	**GICD_IPRIORITYRn**	Interrupt Priority registers
0x800-0x81C 0x820-0x9FC	**GICD_ITARGERSRn**	Interrupt Processor Targets registers
0xC00 0xC04 0xC08-0xC7C	**GICD_ICFGRn**	Interrupt Configuration registers ()
0xD00	**GICD_PPISR**	Private Peripheral Interrupt Status register (Section 18.5.1.2 “Private Peripheral Interrupt Status Register, GICD_PPISR”)
0xD04-0xD3C	**GICD_SPISRn**	Shared Peripheral Interrupt Status registers (Section 18.5.1.3 “Shared Peripheral Interrupt Status Register, GICD_SPISRn”)
0xF00	**GICD_SGIR**	Software Generated Interrupt register
0xF10-0xF1C	**GICD_CPENDSGIRn**	SGI Clear-Pending registers
0xF20-0xF2C	**GICD_SPENDSGIRn**	SGI Set-Pending registers
0xFD0	**GICD_PIDR4**	Peripheral ID 4 register
0xFD4	**GICD_PIDR5**	Peripheral ID 5 register
0xFD8	**GICD_PIDR6**	Peripheral ID 6 register
0xFDC	**GICD_PIDR7**	Peripheral ID 7 register
0xFE0	**GICD_PIDR0**	Peripheral ID 0 register
0xFE4	**GICD_PIDR1**	Peripheral ID 1 register

For Mekra Lang (0801) Confidential Only 2015-12-15

Offset	Register	Description
0xFE8	**GICD_PIDR2**	Peripheral ID 2 register
0xFEC	**GICD_PIDR3**	Peripheral ID 3 register
0xFF0	**GICD_CIDR0**	Component ID 0 register
0xFF4	**GICD_CIDR1**	Component ID 1 register
0xFF8	**GICD_CIDR2**	Component ID 2 register
0xFFC	**GICD_CIDR3**	Component ID 3 register

Table 18-2. Registers of the General Interrupt Controller (GIC).

1. n corresponds to the number of a CPU interface.
2. You cannot modify the EnableGrp0 bit if CFGSDISABLE is set.
3. The reset value depends on the configuration of the GIC-400. The configuration-dependent values are two fields of the Interrupt Controller.
4. This register is only accessible from a Secure access.
5. Writes to bits corresponding to the SGIs are ignored.
6. The reset value for the register that contains the SGI and PPI interrupts is 0x0000FFFF because SGIs are always enabled. However, SGIs are Group 0 on reset, so the reset value for non-secure reads is 0x00000000.
7. The registers that contain the SGI and PPI interrupts are read-only and the value is the CPU number of the current access. It is encoded in an 8-bit one-hot field, for each implemented interrupt, and zero for interrupts that are not implemented.
8. In uniprocessor systems, these registers are RAZ/WI.

18.5.1 Distributor Registers: Detail

18.5.1.1 Distributor Implementer Identification Register, GICD_IIDR Register

Bits	Name	Access	Reset	Description
31:24	**ProductID**			Indicates the product ID: 0x02 GIC-400
23:20				Reserved
19:16	**Variant**			Indicates the major revision or variant of the product: 0x0 Variant number
15:12	**Revision**			Indicates the minor revision of the product: 0x1 Revision number
11:0	**Implementer**			Indicates the implementer: 0x43B Arm

Table 18-3. Distributor Implementer Identification Register.

For Ambarella Confidential Use Only (08012023)

18.5.1.2 Private Peripheral Interrupt Status Register, GICD_PPISR

Bits	Name	Access	Reset	Description
31:16				Reserved
15:9	**PPI status**			Asserted when the PPI inputs to the Distributor are active. ID 31 **nLEGACYIRQ** signal ID 30 Non-secure physical timer event ID 29 Secure physical timer event ID 28 **nLEGACYFIQ** signal ID 27 Virtual timer event ID 26 Hypervisor timer event ID 25 Virtual maintenance interrupt
8:0				Reserved

Table 18-4. Private Peripheral Interrupt Status Register.

18.5.1.3 Shared Peripheral Interrupt Status Register, GICD_SPISRn

Bits	Name	Access	Reset	Description
31:0	**IRQS[N+31:N]**			Returns the status of the IRQS inputs on the Distributor. For each bit: 0 IRQS is LOW 1 IRQS is HIGH • The **IRQS** that a bit refers to depends on its bit position and the base address offset of the *Shared Peripheral Interrupt Status Registers*, GICD_SPISRn. • These bits return the actual status of the **IRQS** signals. The first *Interrupt Set-Pending Register*, GICD_ISPENDR0 and *Interrupt Clear-Pending Register*, GICD_ICPENDR0, can also provide the **IRQS** status but because you can write to these registers, they might not contain the actual status of the **IRQS** signals.

Table 18-5. Shared Peripheral Interrupt Status Register.

18.6 GIC: CPU Interface Register Summary

Offset	Register	Description
0x0000	**GICC_CTLR**	CPU Interface Control register
0x0004	**GICC_PMR**	Interrupt Priority Mask register
0x0008	**GICC_BPR**	Binary Point register The minimum value of the Binary Point register depends on which security-banked copy is considered 0x2 Secure copy 0x3 Non-secure copy
0x000C	**GICC_IAR**	Interrupt Acknowledge register
0x0010	**GICC_EOIR**	End of Interrupt register
0x0014	**GICC_RPR**	Running Priority register
0x0018	**GICC_HPPIR**	Highest Priority Pending Interrupt register

Confidential
For Web Use Only
1.5 (08/15/25)

Offset	Register	Description
0x001C	**GICC_ABPR**	Aliased Binary Point register The minimum value of the Aliased Binary Point register is 0x3
0x0020	**GICC_AIAR**	Aliased Interrupt Acknowledge register
0x0024	**GICC_AEOIR**	Aliased End of Interrupt register
0x0028	**GICC_AHPPIR**	Aliased Highest Priority Pending Interrupt register
0x00D0	**GICC_APR0**	Active Priority register
0x00E0	**GICC_NSAPR0**	Non-Secure Active Priority register
0x00FC	**GICC_IIDR**	CPU Interface Identification register (Section 18.6.1.1 "CPU Interface Identification Register, GICC_IIDR")
0x1000	**GICC_DIR**	Deactivate Interrupt register

18.6.1 CPU Interface Registers: Detail

18.6.1.1 CPU Interface Identification Register, GICC_IIDR

Bits	Name	Access	Reset	Description
31:20	**ProductID**			Identifies the product ID: 0x020 GIC-400
19:16	**Architecture version**			Identifies the architecture version of the GIC-400: 0x2 Version 2.0
15:12	**Revision**			Identifies the revision number for the CPU interface. For the GIC-400: 0x1 Revision r0p1
11:0	**Implementer**			Contains the JEP106 code of the company that implemented the CPU interface 0x43B Arm

Table 18-6. CPU Interface Identification Register.

18.7 GIC: GIC Virtual Interface Control Register Summary

Offset	Register	Description
0x000	**GICH_HCR**	Hypervisor Control register
0x004	**GICH_VTR**	VGIC Type register
0x008	**GICH_VMCR**	Virtual Machine Control register
0x010	**GICH_MISR**	Maintenance Interrupt Status register
0x020	**GICH_EISR0**	End of Interrupt Status register
0x030	**GICH_ELSR0**	Empty List Register Status register
0x0F0	**GICH_APR0**	Active Priority register
0x100	**GICH_LR0**	List Register 0
0x104	**GICH_LR1**	List Register 1
0x108	**GICH_LR2**	List Register 2
0x10C	**GICH_LR3**	List Register 3

For Internal Use Only (Confidential 2025)

18.7.1 GIC Virtual Interface Control Registers: Detail

18.7.1.1 VGIC Type Register, GICH_VTR

Bits	Name	Access	Reset	Description
31:29	**PRIbits**			Indicates the number of priority bits implemented, minus one: 0x4 5 bits of priority and 32 priority levels
28:26	**PREbits**			Indicates the number of preemption bits implemented, minus one: 0x4 5 bits of preemption and 32 preemption levels
25:6				Reserved
5:0	**ListRegs**			Indicates the number of implemented list registers, minus one: 0x3 4 list registers

Table 18-7. VGIC Type Register.

18.8 GIC: GIC Virtual CPU Interface Register Summary

Offset	Register	Description
0x0000	**GIVC_CTLR**	Virtual Machine Control register
0x0004	**GIVC_PMR**	VM Priority Mask register
0x0008	**GICV_BPR**	VM Binary Point register
0x000C	**GICV_IAR**	VM Interrupt Acknowledge register
0x0010	**GICV_EOIR**	VM End of Interrupt register
0x0014	**GICV_RPR**	VM Running Priority register
0x0018	**GICV_HPPIR**	VM Highest Priority Pending Interrupt register
0x001C	**GICV_ABPR**	VM Aliased Binary Point register
0x0020	**GICV_AIAR**	VM Aliased Interrupt Acknowledge register
0x0024	**GICV_AEOIR**	VM Aliased End of Interrupt register
0x0028	**GICV_AHPPIR**	VM Aliased Highest Priority Pending Interrupt register
0x00D0	**GICV_APR0**	VM Active Priority register
0x00FC	**GICV_IIDR**	VM CPU Interface Identification register (Section 18.6.1.1 “CPU Interface Identification Register, GICC_IIDR”)
0x1000	**GICV_DIR**	VM Deactivate Interrupt Register

Note:

This register returns the highest priority pending interrupt even when the appropriate GICV_CTLR.EnableGrp bit is set to 0.

18.8.1 GIC Virtual CPU Interface Registers: Detail

18.8.1.1 VM CPU Interface Identification Register, GICV_IIDR

For bit assignments for the GICV_IIDR which are identical to the corresponding register in the Physical CPU interface, please refer to Section 18.6.1.1 “CPU Interface Identification Register, GICC_IIDR”.

For Mekra Lang (080) Confidential 1.5

19. GPIO

Register programming for the CV22FS general purpose input / output (GPIO) interface is covered in this chapter as follows:

- (Section 19.1) GPIO: Overview
- (Section 19.2) GPIO: Registers
- (Section 19.3) GPIO: Interrupt Control
- (Section 19.4) GPIO: Function Selection
- (Section 19.5) GPIO: RCT Drive Strength Decoding in IOPAD

19.1 GPIO: Overview

The CV22FS chip includes up to 103 configurable multi-purpose GPIO pins, many of which can be configured as I/O pins for various hardware modules. Each set of GPIO registers occupies its own range of addresses on the APB bus.

All GPIO pins are interrupt-capable.

19.2 GPIO: Registers

19.2.1 GPIO Registers: Map

APB Base Address	Offset	Register	Description
E401.0000	0x00	**GPIO0_DATA**	Data register
	0x04	**GPIO0_DIR**	Data direction
	0x08	**GPIO0_IS**	Edge/level interrupt sensitivity control
	0x0C	**GPIO0_IBE**	Edge interrupt sensitivity control
	0x10	**GPIO0_IEV**	Interrupt event
	0x14	**GPIO0_IE**	Interrupt mask
	0x1C	**GPIO0_RIS**	Raw interrupt status
	0x20	**GPIO0_MIS**	Masked interrupt status
	0x24	**GPIO0_IC**	Interrupt clear
	0x28	**GPIO0_MASK**	GPIO mask
	0x2C	**GPIO0_ENABLE**	GPIO enable
	0x30	**GPIO0_FAULT_INJECT**	GPIO fault inject
E401.1000	0x00	**GPIO1_DATA**	Data register
	0x04	**GPIO1_DIR**	Data direction
	0x08	**GPIO1_IS**	Edge/level interrupt sensitivity control
	0x0C	**GPIO1_IBE**	Edge interrupt sensitivity control
	0x10	**GPIO1_IEV**	Interrupt event
	0x14	**GPIO1_IE**	Interrupt mask
	0x1C	**GPIO1_RIS**	Raw interrupt status

For Wekra Lab Only Confidential 1.5 (08012025)

APB Base Address	Offset	Register	Description
	0x20	**GPIO1_MIS**	Masked interrupt status
	0x24	**GPIO1_IC**	Interrupt clear
	0x28	**GPIO1_MASK**	GPIO mask
	0x2C	**GPIO1_ENABLE**	GPIO enable
	0x30	**GPIO1_FAULT_INJECT**	GPIO fault inject
E401.2000	0x00	**GPIO2_DATA**	Data register
	0x04	**GPIO2_DIR**	Data direction
	0x08	**GPIO2_IS**	Edge/level interrupt sensitivity control
	0x0C	**GPIO2_IBE**	Edge interrupt sensitivity control
	0x10	**GPIO2_IEV**	Interrupt event
	0x14	**GPIO2_IE**	Interrupt mask
	0x1C	**GPIO2_RIS**	Raw interrupt status
	0x20	**GPIO2_MIS**	Masked interrupt status
	0x24	**GPIO2_IC**	Interrupt clear
	0x28	**GPIO2_MASK**	GPIO mask
	0x2C	**GPIO2_ENABLE**	GPIO enable
	0x30	**GPIO2_FAULT_INJECT**	GPIO fault inject
E401.3000	0x00	**GPIO3_DATA**	Data register
	0x04	**GPIO3_DIR**	Data direction
	0x08	**GPIO3_IS**	Edge/level interrupt sensitivity control
	0x0C	**GPIO3_IBE**	Edge interrupt sensitivity control
	0x10	**GPIO3_IEV**	Interrupt event
	0x14	**GPIO3_IE**	Interrupt mask
	0x1C	**GPIO3_RIS**	Raw interrupt status
	0x20	**GPIO3_MIS**	Masked interrupt status
	0x24	**GPIO3_IC**	Interrupt clear
	0x28	**GPIO3_MASK**	GPIO mask
	0x2C	**GPIO3_ENABLE**	GPIO enable
	0x30	**GPIO2_FAULT_INJECT-GPIO3_FAULT_INJECT**	GPIO fault inject

Table 19-1. Address Offsets for Each Set of GPIO Registers.

Within each GPIO register, each bit corresponds to one GPIO pin. For example, bit 0 of the **GPIO0_data** register (Section 19.2.2.1) is the data bit for GPIO pin 0 (**SC_A0**); bit 25 of the **GPIO0_data** register is the data bit for GPIO pin 25 (**IR_IN**).

The **GPIO[n]_afsel** register is used to select whether the corresponding GPIO is configured to function as a GPIO pin (primary function) or to take on a secondary function.

For Ambarella Internal Use Only (Ambarella 2025)

19.2.2 GPIO Registers: Details

19.2.2.1 GPIO[n]_data Register

Bits	Name	Access	Reset	Description
31:0	**bits**	RW	0	Data register of the GPIO controller used to drive and read the PAD

Table 19-2. GPIO Data Register.

19.2.2.2 GPIO[n]_dir Register

Bits	Name	Access	Reset	Description
31:0	**io**	RW	0	Selects between input/output control of the GPIO lines 0 - Input 1 - Output

Table 19-3. GPIO Data Direction Register.

19.2.2.3 GPIO[n]_is Register

Bits	Name	Access	Reset	Description
31:0	**is**	RW	0	Interrupt sensitivity control 0 - Edge is detected 1 - Level is detected

Table 19-4. GPIO Edge/Level Interrupt Sensitivity Control Register.

19.2.2.4 GPIO[n]_ibe Register

Bits	Name	Access	Reset	Description
31:0	**ibe**	RW	0	Interrupt on both edges 0 - **GPIO_iev** register controls interrupt generation event 1 - Both edges on corresponding pin trigger interrupt

Table 19-5. GPIO Edge Interrupt Sensitivity Control Register.

Confidential For Mekra training only 1.5 (Oct 2025)

19.2.2.5 GPIO[n]_iev Register

Bits	Name	Access	Reset	Description
31:0	**iev**	RW	0	Interrupt event control 0 - Falling edges, or low levels trigger interrupts 1 - Rising edges, or high levels trigger interrupts

Table 19-6. GPIO Interrupt Event Register.

19.2.2.6 GPIO[n]_ie Register

Bits	Name	Access	Reset	Description
31:0	**ie**	RW	0	Interrupt enable control 0 - Corresponding pin interrupt is masked 1 - Corresponding pin is not masked

Table 19-7. GPIO Interrupt Mask Register.

19.2.2.7 GPIO[n]_ris Register

Bits	Name	Access	Reset	Description
31:0	**ris**	R	0	Raw interrupt status Reflect the status of interrupt trigger condition-detection on pins. 0 - Requirement not met 1 - Requirement met

Table 19-8. GPIO Raw Interrupt Status Register.

19.2.2.8 GPIO[n]_mis Register

Bits	Name	Access	Reset	Description
31:0	**mis**	R	0	Masked interrupt status 0 - GPIO line interrupt not active 1 - GPIO line asserting interrupt

Table 19-9. GPIO Masked Interrupt Status Register.

19.2.2.9 GPIO[n]_ic Register

Bits	Name	Access	Reset	Description
31:0	**clr**	W		Interrupt clear 0 - Has no effect 1 - Clears edge-detection logic

Table 19-10. GPIO Interrupt Clear Register.

For Mekrotang only
Confidential
15 Apr 2025

19.2.2.10 GPIO[n]_mask Register

The application of a mask makes it easier for the software to modify (or observe) only the bits in the **GPIO[n]_data** register of concern.

For example, consider the case that GPIO pin 0 (**SC_A0**) is connected to an LED (e.g., LED_0) and GPIO pin 1 (**SC_A1**) is connected to a separate LED (e.g., LED_1). Assume LED_1 is currently on, (i.e., **GPIO0_data[1]** is set to 1). If the goal is for LED_0 to blink without affecting LED_1, then the software can be set to **GPIO[n]_mask** = 0x1 followed by a sequence of writes to the **GPIO[n]_data** register with values 0,1,0,1,0,1, and so on.

In this case, only bit 0 is affected, and all other GPIO bits will not be affected by subsequent writes. Due to the mask, LED_1 will remain on even when the software writes 0x00000000 to the **GPIO[n]_data** register.

Bits	Name	Access	Reset	Description
31:0	**mask**	RW	0	Mask of data read/write. 0 - Corresponding bit of **GPIO[n]_data** is masked 1 - Corresponding bit of **GPIO[n]_data** is accessible

Table 19-11. GPIO Mask Register.

19.2.2.11 GPIO[n]_enable Register

GPIO[n]_enable registers are used to enable all GPIO pins. After reset, the bits in this register revert to zero and all GPIO pins are, in turn, disabled. When the software enables the GPIO function by writing 1's to the **GPIO[n]_enable** register, it also must write 0's to the corresponding bits in **GPIO[n]_afsel** registers to enable software control of the GPIO pins.

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**enb**	W	0	GPIO enable 0 - All GPIO is disabled 1 - All GPIO is enabled

Table 19-12. GPIO Enable Register.

19.2.2.12 GPIO[n]_fault_inject Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**fault_inject**	RW	0	Fault inject Writes “1” to the register to assert error output for SW error test

Table 19-13. GPIO Fault Inject Register.

Confidential
For Mekun Lang only
(15 Oct 2025)

19.3 GPIO: Interrupt Control

Each GPIO pin can be configured to generate an interrupt on a rising edge, a falling edge, both edges, a logic level high or a logic level low. The interrupt sensitivity can be configured using the **GPIO[n]_is**, **GPIO[n]_ibe**, **GPIO[n]_iev** and **GPIO[n]_ie** registers.

When an interrupt condition is detected on a GPIO line, the corresponding bit in the **GPIO[n]_ris** register will be set to 1. The **GPIO[n]_ris** register contains the "raw" interrupt status for each GPIO pin.

The **GPIO[n]_mis** register is the result of the bit-wise AND of the **GPIO[n]_ris** register and the **GPIO[n]_ie** register. When the **GPIO[n]_mis** register is non-zero, the interrupt signal to the GIC will be asserted. In servicing the GIC-GPIO**[n]** interrupt, the software should read the **GPIO[n]_mis** register to determine which GPIO pin(s) caused the interrupt.

19.4 GPIO: Function Selection

This section details the CV22FS GPIO pins, including function selection information. Refer to the CV22FS data-sheet for GPIO pin descriptions.

19.4.1 GPIO Function Selection: (IOMUX) Registers

There are 15 configurable word-sized (32-bit) registers within the **IOMUX** registry that can be sub-divided into 7 register sets, where each set consists of 3 registers.

Note that these registers will not take effect unless the **ctrl_set** register is set to 1. All registers, including **ctrl_set** registers, can be accessed through the APB bus at base address E400E000.

The following table contains a list of IOMUX registers, including address offset.

Offset	Register	Description
0x00	**IOMUX_REG0_0**	Register Set 0
0x04	**IOMUX_REG0_1**	
0x08	**IOMUX_REG0_2**	
0x0C	**IOMUX_REG1_0**	Register Set 1
0x10	**IOMUX_REG1_1**	
0x14	**IOMUX_REG1_2**	
0x18	**IOMUX_REG2_0**	Register Set 2
0x1C	**IOMUX_REG2_1**	
0x20	**IOMUX_REG2_2**	
0x24	**IOMUX_REG3_0**	Register Set 3
0x28	**IOMUX_REG3_1**	
0x2C	**IOMUX_REG3_2**	
0x30	**IOMUX_REG4_0**	Register Set 4
0x34	**IOMUX_REG4_1**	
0x38	**IOMUX_REG4_2**	
0xF0	**IOMUX_CTRL_SET**	Register Control
0xEC	**IOMUX_FAULT_INJ**	Fault Injection

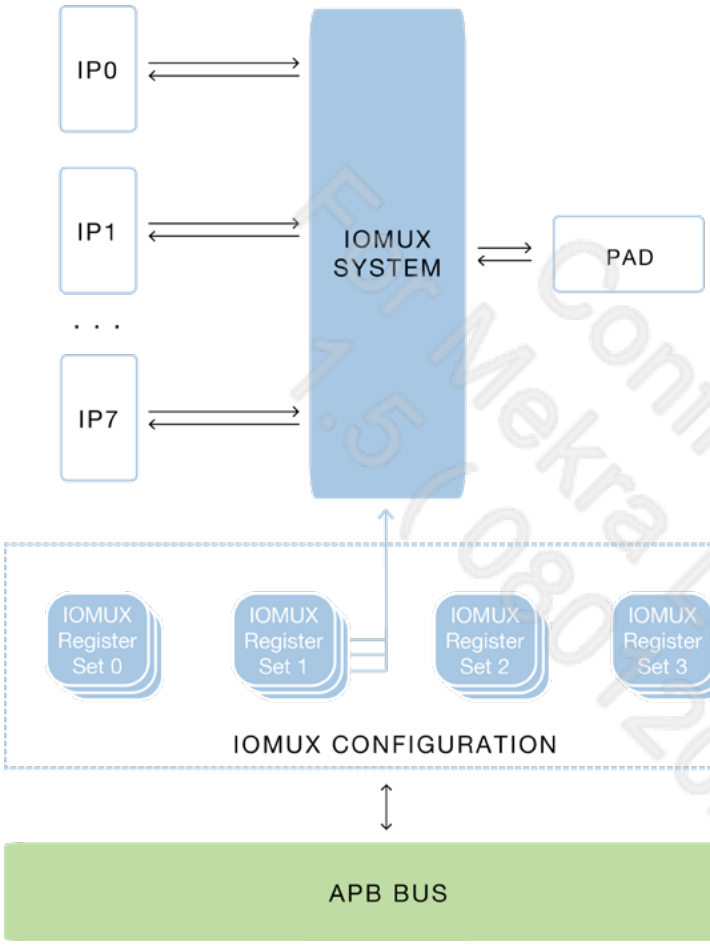
Table 19-14. GPIO Function Selection (IOMUX) Registers.

For Meitra Lang only (Confidential) 2025

19.4.2 GPIO Function Selection: Programming Procedures

GPIO functions are selected using the registers referenced above in Section 19.4.1. The 3-bit information, combined from the same bit positions of registers within each set, jointly determines an IO pad signal's source/destination mapping. Refer to the following figure.

Note that during system boot, the pin function of the active boot device is chosen internally, without configuring IOMUX registers. Refer to the *Ambarella CV22FS System Hardware* document for the description of GPIO function selection during system boot.



For Mekra Lang only (08/2025) Confidential 1.5

Figure 19-1. GPIO Function Selection: IOMUX Block Diagram.

Notes:

- IP0, IP1, etc., represent the CV22FS modules / subsystems connected to the IOMUX System, which differ depending on the pad under examination.
- Four register sets are shown in the diagram; refer to Table 19-14 for chip-specific register set detail.

Table 19-15 provides a list of register bit locations to be used during GPIO function selection. The first entry has been excerpted below to serve as a guide for interpreting the table.

GPIO	Pin Name	Multiplexed Function					
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5
0	**SC_A0** (1)	1: GPIO (IO) (2)	1: 1, sc_a0 (IO) (4)				
		2: {0x8[0], 0x4[0], 0x0[0]} = 4'h000 (3)	2: {0x8[0], 0x4[0], 0x0[0]} = 4'h001				

Referring to the excerpted entry above:

(1) Pin Name
(2) Alternative Function 0 Name
(3) Selection Bit Value for the Corresponding Register Set

In this example, bit 0 in **iomux_reg0_2** (0x8), **iomux_reg0_1** (0x4), and **iomux_reg0_0** (0x0) must be configured as {0, 0, 0 (4'h000)} in order to enable alternative function 0.

(4) Alternative Function 1 Name

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
0	**TIMER0**	1, GPIO (IO) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b000	1, tm11_clk (I) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b001	1, wdt_ext_rst_l (O) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b010	1, pwm3 (O) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b011	1, vpp_en (O) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b100	1, sdio0_hs_sel (O) 2, {0x8[0], 0x4[0], 0x0[0]} = 3'b101	gpio_en0[0], gpio_sel0[0]	gpio_ds1_reg_0[0], gpio_ds0_reg_0[0]]
1	**IDCSCLK**	1, GPIO (IO) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b000	1, idcsclk (I+) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b001	1, tm12_clk (I) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b010	1, ssi0_en6 (O) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b011	1, vin_strig0 (O+) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b100	1, ehci_app_prt_ovcurr0 (I) 2, {0x8[1], 0x4[1], 0x0[1]} = 3'b101	gpio_en0[1], gpio_sel0[1]	gpio_ds1_reg_0[1], gpio_ds0_reg_0[1]
2	**IDCSDATA**	1, GPIO (IO) 2, {0x8[2], 0x4[2], 0x0[2]} = 4'h000	1, idcsdata (IO) 2, {0x8[2], 0x4[2], 0x0[2]} = 3'b001	1, tm13_clk (I) 2, {0x8[2], 0x4[2], 0x0[2]} = 3'b010	1, ssi0_en7 (O) 2, {0x8[2], 0x4[2], 0x0[2]} = 3'b011	1, vin_strig1 (O+) 2, {0x8[2], 0x4[2], 0x0[2]} = 3'b100	1, ehci_prt_pwr_0 (O) 2, {0x8[2], 0x4[2], 0x0[2]} = 3'b101	gpio_en0[2], misc::gpio_sel0[2]	gpio_ds1_reg_0[2], rct::gpio_ds0_reg_0[2]
3	**SSI0CLK**	1, GPIO (IO) 2, {0x8[3], 0x4[3], 0x0[3]} = 3'b000	1, ssi0_sclk (O) 2, {0x8[3], 0x4[3], 0x0[3]} = 3'b001	1, idc0clk (IO) 2, {0x8[3], 0x4[3], 0x0[3] = 3'b010	1, pwm4 (O) 2, {0x8[3], 0x4[3], 0x0[3] = 3'b011			gpio_en0[3], gpio_sel0[3]	gpio_ds1_reg_0[3], gpio_ds0_reg_0[3]
4	**SSI0MOSI**	1, GPIO (IO) 2, {0x8[4], 0x4[4], 0x0[4]} = 3'b000	1, ssi0_txd (O+) 2, {0x8[4], 0x4[4], 0x0[4]} = 3'b001	1, idc0data (IO) 2, {0x8[4], 0x4[4], 0x0[4] = 3'b010	1, pwm5 (O) 2, {0x8[4], 0x4[4], 0x0[4] = 3'b011			gpio_en0[4], gpio_sel0[4]	gpio_ds1_reg_0[4], gpio_ds0_reg_0[4]

Confidential Material Logging Only Ambarella 2021

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
5	**SSI0MISO**	1, GPIO (IO) 2, {0x8[5], 0x4[5], 0x0[5]} = 3'b000	1, ssi0_rxd (I) 2, {0x8[5], 0x4[5], 0x0[5]} = 3'b001	1, idc1clk (IO) 2, {0x8[5], 0x4[5], 0x0[5]} = 3'b010	1, pwm6 (O) 2, {0x8[5], 0x4[5], 0x0[5]} = 3'b011			gpio_en0[5], gpio_sel0[5]	gpio_ds1_ reg_0[5], gpio_ds0_ reg_0[5]
6	**SSI0EN0**	1, GPIO (IO) 2, {0x8[6], 0x4[6], 0x0[6]} = 3'b000	1, ssi0_en0 (O) 2, {0x8[6], 0x4[6], 0x0[6]} = 3'b001	1, idc1data (IO) 2, {0x8[6], 0x4[6], 0x0[6]} = 3'b010	1, pwm7 (O) 2, {0x8[6], 0x4[6], 0x0[6]} = 3'b011			gpio_en0[6], gpio_sel0[6]	gpio_ds1_ reg_0[6], gpio_ds0_ reg_ 0[6]
7	**SSI0EN1**	1, GPIO (IO) 2, {0x8[7], 0x4[7], 0x0[7]} = 3'b000	1, ssi0_en1 (O) 2, {0x8[7], 0x4[7], 0x0[7]} = 3'b001	1, vpp_en (O) 2, {0x8[7], 0x4[7], 0x0[7]} = 3'b010	1, wdt_ext_ rst_l (O) 2, {0x8[7], 0x4[7], 0x0[7]} = 3'b011	1, sdio0_hs_ sel (O) 2, {0x8[7], 0x4[7], 0x0[7]} = 3'b100		gpio_en0[7], gpio_sel0[7]	gpio_ds1_ reg_0[7], gpio_ds0_ reg_0[7]
8	**SSI2CLK**	1, GPIO (IO) 2, {0x8[8], 0x4[8], 0x0[8]} = 3'b000	1, ssi2_sclk (O) 2, {0x8[8], 0x4[8], 0x0[8]} = 3'b001	1, idc2clk (IO) 2, {0x8[8], 0x4[8], 0x0[8]} = 3'b010	1, ssi0_en2 (O) 2, {0x8[8], 0x4[8], 0x0[8]} = 3'b011	1, vsdelay_ vsync_out_0 (O) 2, {0x8[8], 0x4[8], 0x0[8]} = 3'b100		gpio_en0[8], gpio_sel0[8]	gpio_ds1_ reg_0[8], gpio_ds0_ reg_0[8]
9	**SSI2MOSI**	1, GPIO (IO) 2, {0x8[9], 0x4[9], 0x0[9]} = 3'b000	1, ssi2_txd (O+) 2, {0x8[9], 0x4[9], 0x0[9]} = 3'b001	1, idc2data (IO) 2, {0x8[9], 0x4[9], 0x0[9]} = 3'b010	1, ssi0_en3 (O) 2, {0x8[9], 0x4[9], 0x0[9]} = 3'b011	1, vsdelay_ vsync_out_1 (O) 2, {0x8[9], 0x4[9], 0x0[9]} = 3'b100		gpio_en0[9], gpio_sel0[9]	gpio_ds1_ reg_0[9], gpio_ds0_ reg_0[9]
10	**SSI2MISO**	1, GPIO (IO) 2, {0x8[10], 0x4[10], 0x0[10]} = 3'b000	1, ssi2_rxd (I) 2, {0x8[10], 0x4[10], 0x0[10]} = 3'b001	1, idc3clk (IO) 2, {0x8[10], 0x4[10], 0x0[10]} = 3'b010	1, ssi0_en4 (O) 2, {0x8[10], 0x4[10], 0x0[10]} = 3'b011	1, vsdelay_ hsync_out_0 (O) 2, {0x8[10], 0x4[10], 0x0[10]} = 3'b100		gpio_en0[10], gpio_sel0[10]	gpio_ds1_ reg_0[10], gpio_ds0_ reg_0[10]
11	**SSI2EN0**	1, GPIO (IO) 2, {0x8[11], 0x4[11], 0x0[11]} = 3'b000	1, ssi2_en0 (O) 2, {0x8[11], 0x4[11], 0x0[11]} = 3'b001	1, idc3data (IO) 2, {0x8[11], 0x4[11], 0x0[11]} = 3'b010	1, ssi0_en5 (O) 2, {0x8[11], 0x4[11], 0x0[11]} = 3'b011	1, vsdelay_ hsync_out_1 (O) 2, {0x8[11], 0x4[11], 0x0[11]} = 3'b100		gpio_en0[11], gpio_sel0[11]	gpio_ds1_ reg_0[11], gpio_ds0_ reg_0[11]
12	**SSI3CLK**	1, GPIO (IO) 2, {0x8[12], 0x4[12], 0x0[12]} = 3'b000	1, ssi3_sclk (O) 2, {0x8[12], 0x4[12], 0x0[12]} = 3'b001	1, idc4clk (IO) 2, {0x8[12], 0x4[12], 0x0[12]} = 3'b010	1, ssis_sclk (I)2, {0x8[12], 0x4[12], 0x0[12]} = 3'b011	1, uart3_ahb_ rx (I) 2, {0x8[12], 0x4[12], 0x0[12]} = 3'b100		gpio_en0[12], gpio_sel0[12]	gpio_ds1_ reg_0[12], gpio_ds0_ reg_0[12]
13	**SSI3MOSI**	1, GPIO (IO) 2, {0x8[13], 0x4[13], 0x0[13]} = 3'b000	1, ssi3_txd (O+) 2, {0x8[13], 0x4[13], 0x0[13]} = 3'b001	1, idc4data (IO) 2, {0x8[13], 0x4[13], 0x0[13]} = 3'b010	1, ssis_txd (O+) 2, {0x8[13], 0x4[13], 0x0[13]} = 3'b011	1, uart3_ahb_ tx (O) 2, {0x8[13], 0x4[13], 0x0[13]} = 3'b100		gpio_en0[13], gpio_sel0[13]	gpio_ds1_ reg_0[13], gpio_ds0_ reg_0[13]
14	**SSI3MISO**	1, GPIO (IO) 2, {0x8[14], 0x4[14], 0x0[14]} = 3'b000	1, ssi3_rxd (I) 2, {0x8[14], 0x4[14], 0x0[14]} = 3'b001	1, idc5clk (IO) 2, {0x8[14], 0x4[14], 0x0[14]} = 3'b010	1, ssis_rxd (I) 2, {0x8[14], 0x4[14], 0x0[14]} = 3'b011	1, uart3_ahb_ cts_n (I) 2, {0x8[14], 0x4[14], 0x0[14]} = 3'b100		gpio_en0[14], gpio_sel0[14]	gpio_ds1_ reg_0[14], gpio_ds0_ reg_0[14]

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
15	**SSI3EN0**	1, GPIO (IO) 2, {0x8[15], 0x4[15], 0x0[15]} = 3'b000	1, ssi3_en0 (O) 2, {0x8[15], 0x4[15], 0x0[15]} = 3'b001	1, idc5data (IO) 2, {0x8[15], 0x4[15], 0x0[15]} = 3'b010	1, ssis_en (I) 2, {0x8[15], 0x4[15], 0x0[15]} = 3'b011	1, uart3_ahb_ rts_n (O) 2, {0x8[15], 0x4[15], 0x0[15]} = 3'b100		gpio_en0[15], gpio_sel0[15]	gpio_ds1_ reg_0[15], gpio_ds0_ reg_0[15],
16	**UART_APB_ RX**	1, GPIO (IO) 2, {0x8[16], 0x4[16], 0x0[16]} = 3'b000	1, uart_apb_ rx (I) 2, {0x8[16], 0x4[16], 0x0[16]} = 3'b001	1, ehci_app_ prt_ovcurr0 (I) 2, {0x8[16], 0x4[16], 0x0[16]} = 3'b010				gpio_en0[16], gpio_sel0[16]	gpio_ds1_ reg_0[16] gpio_ds0_ reg_0[16]
17	**UART_APB_ TX**	1, GPIO (IO) 2, {0x8[17], 0x4[17], 0x0[17]} = 3'b000	1, uart_apb_ tx (O) 2, {0x8[17], 0x4[17], 0x0[17]} = 3'b001	1, ehci_prt_ pwr_0 (O) 2, {0x8[17], 0x4[17], 0x0[17]} = 3'b010				gpio_en0[17], gpio_sel0[17]	gpio_ds1_ reg_0[17] gpio_ds0_ reg_0[17]
18	**PWM0**	1, GPIO (IO) 2, {0x8[18], 0x4[18], 0x0[18]} = 3'b000	1, pwm0 (O) 2, {0x8[18], 0x4[18], 0x0[18]} = 3'b001	1, vin_strig0 (O+) 2, {0x8[18], 0x4[18], 0x0[18]} = 3'b010				gpio_en0[18], gpio_sel0[18]	gpio_ds1_ reg_0[18], gpio_ds0_ reg_0[18]
19	**PWM1**	1, GPIO (IO) 2, {0x8[19], 0x4[19], 0x0[19]} = 3'b000	1, pwm1 (O) 2, {0x8[19], 0x4[19], 0x0[19]} = 3'b001	1, vin_strig1 (O+) 2, {0x8[19], 0x4[19], 0x0[19]} = 3'b010				gpio_en0[19], gpio_sel0[19]	gpio_ds1_ reg_0[19], gpio_ds0_ reg_0[19]
20	**PWM2**	1, GPIO (IO) 2, {0x8[20], 0x4[20], 0x0[20]} = 3'b000	1, pwm2 (O) 2, {0x8[20], 0x4[20], 0x0[20]} = 3'b001	1, wdt_ext_ rst_l (O) 2, {0x8[20], 0x4[20], 0x0[20]} = 3'b010				gpio_en0[20], gpio_sel0[20]	gpio_ds1_ reg_0[20], gpio_ds0_ reg_0[20]
21	**I2S_CLK**	1, GPIO (IO) 2, {0x8[21], 0x4[21], 0x0[21]} = 3'b000	1, i2s_clk (IO) 2, {0x8[21], 0x4[21], 0x0[21]} = 3'b001	1, ssi1_sclk (O) 2, {0x8[21], 0x4[21], 0x0[21]} = 3'b010				gpio_en0[21], gpio_sel0[21]	gpio_ds1_ reg_0[21], gpio_ds0_ reg_0[21]
22	**I2S_SI_0**	1, GPIO (IO) 2, {0x8[22], 0x4[22], 0x0[22]} = 3'b000	1, i2s_si_0 (I) 2, {0x8[22], 0x4[22], 0x0[22]} = 3'b001	1, ssi1_txd (O+) 2, {0x8[22], 0x4[22], 0x0[22]} = 3'b010				gpio_en0[22], gpio_sel0[22]	gpio_ds1_ reg_0[22], gpio_ds0_ reg_0[22]
23	**I2S_SO_0**	1, GPIO (IO) 2, {0x8[23], 0x4[23], 0x0[23]} = 3'b000	1, i2s_so_0 (O) 2, {0x8[23], 0x4[23], 0x0[23]} = 3'b001	1, ssi1_en0 (O) 2, {0x8[23], 0x4[23], 0x0[23]} = 3'b010				gpio_en0[23], gpio_sel0[23]	gpio_ds1_ reg_0[23], gpio_ds0_ reg_0[23]
24	**GPIO24**	1, GPIO (IO) 2, {0x8[24], 0x4[24], 0x0[24]} = 3'b000	1, i2s_si_1(I) 2, {0x8[24], 0x4[24], 0x0[24]} = 3'b001	1, ssi1_rxd (I) 2, {0x8[24], 0x4[24], 0x0[24]} = 3'b010	1, ehci_app_ prt_ovcurr0 (I) 2, {0x8[24], 0x4[24], 0x0[24]} = 3'b011			gpio_en0[24], gpio_sel0[24]	gpio_ds1_ reg_0[24], gpio_ds0_ reg_0[24]

Ambarella Confidential Lang only (c) 2025

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
25	**GPIO25**	1, GPIO (IO) 2, {0x8[25], 0x4[25], 0x0[25]} = 3'b000	1, i2s_ so_1(O) 2, {0x8[25], 0x4[25], 0x0[25]} = 3'b001		1, ehci_prt_ pwr_0 (O) 2, {0x8[25], 0x4[25], 0x0[25]} = 3'b011			gpio_en0[25], gpio_sel0[25]	gpio_ds1_ reg_0[25], gpio_ds0_ reg_0[25]
26	**I2S_WS**	1, GPIO (IO) 2, {0x8[26], 0x4[26], 0x0[26]} = 3'b000	1, i2s_ws (IO) 2, {0x8[26], 0x4[26], 0x0[26]} = 3'b001					gpio_en0[26], gpio_sel0[26]	gpio_ds1_ reg_0[26], gpio_ds0_ reg_0[26]
27	**ENET_TXEN**	1, GPIO (IO) 2, {0x8[27], 0x4[27], 0x0[27]} = 3'b000	1, enet_txen (O) 2, {0x8[27], 0x4[27], 0x0[27]} = 3'b001	1, enet_txen (O) 2, {0x8[27], 0x4[27], 0x0[27]} = 3'b010	1, pwm8 (O) 2, {0x8[27], 0x4[27], 0x0[27]} = 3'b011			gpio_en0[27], gpio_sel0[27]	gpio_ds1_ reg_0[27], gpio_ds0_ reg_0[27]
28	**ENET_TXD_0**	1, GPIO (IO) 2, {0x8[28], 0x4[28], 0x0[28]} = 3'b000	1, enet_ txd_0 (O) 2, {0x8[28], 0x4[28], 0x0[28]} = 3'b001	1, enet_ txd_0 (O) 2, {0x8[28], 0x4[28], 0x0[28]} = 3'b010	1, pwm9 (O) 2, {0x8[28], 0x4[28], 0x0[28]} = 3'b011			gpio_en0[28], gpio_sel0[28]	gpio_ds1_ reg_0[28], gpio_ds0_ reg_0[28]
29	**ENET_TXD_1**	1, GPIO (IO) 2, {0x8[29], 0x4[29], 0x0[29]} = 3'b000	1, enet_ txd_1 (O) 2, {0x8[29], 0x4[29], 0x0[29]} = 3'b001	1, enet_ txd_1 (O) 2, {0x8[29], 0x4[29], 0x0[29]} = 3'b010	1, pwm10 (O) 2, {0x8[29], 0x4[29], 0x0[29]} = 3'b011			gpio_en0[29], gpio_sel0[29]	gpio_ds1_ reg_0[29], gpio_ds0_ reg_0[29]
30	**ENET_TXD_2**	1, GPIO (IO) 2, {0x8[30], 0x4[30], 0x0[30]} = 3'b000		1, enet_ txd_2 (O) 2, {0x8[30], 0x4[30], 0x0[30]} = 3'b010	1, pwm11 (O) 2, {0x8[30], 0x4[30], 0x0[30]} = 3'b011			gpio_en0[30], gpio_sel0[30]	gpio_ds1_ reg_0[30], gpio_ds0_ reg_0[30]
31	**ENET_TXD_3**	1, GPIO (IO) 2, {0x8[31], 0x4[31], 0x0[31]} = 3'b000		1, enet_txd_3 (O) 2, {0x8[31], 0x4[31], 0x0[31]} = 3'b010				gpio_en0[31], gpio_sel0[31]	gpio_ds1_ reg_0[31], gpio_ds0_ reg_0[31]
32	**ENET_RXD_0**	1, GPIO (IO) 2, {0x14[0], 0x10[0], 0xc[0]} = 3'b000	1, enet_ rxd_0 (I) 2, {0x14[0], 0x10[0], 0xc[0]} = 3'b001	1, enet_ rxd_0 (I) 2, {0x14[0], 0x10[0], 0xc[0]} = 3'b010				gpio_en1[0], gpio_sel1[0]	gpio_ds1_ reg_1[0], gpio_ds0_ reg_1[0]
33	**ENET_RXD_1**	1, GPIO (IO) 2, {0x14[1], 0x10[1], 0xc[1]} = 3'b000	1, enet_ rxd_1 (I) 2, {0x14[1], 0x10[1], 0xc[1]} = 3'b001	1, enet_ rxd_1 (I) 2, {0x14[1], 0x10[1], 0xc[1]} = 3'b010				gpio_en1[1], gpio_sel1[1]	gpio_ds1_ reg_1[1], gpio_ds0_ reg_1[1]
34	**ENET_RXD_2**	1, GPIO (IO) 2, {0x14[2], 0x10[2], 0xc[2]} = 3'b000		1, enet_rxd_2 (I) 2, {0x14[2], 0x10[2], 0xc[2]} = 3'b010				gpio_en1[2], gpio_sel1[2]	gpio_ds1_ reg_1[2], gpio_ds0_ reg_1[2]

For Confidential Planning only 2025

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
35	**ENET_RXD_3**	1, GPIO (IO) 2, {0x14[3], 0x10[3], 0xc[3]} = 3'b000		1, enet_ rxd_3 (I) 2, {0x14[3], 0x10[3], 0xc[3]} = 3'b010	1, pwm3 (O) 2, {0x14[3], 0x10[3], 0xc[3]} = 3'b011			gpio_en1[3], gpio_sel1[3]	gpio_ds1_ reg_1[3], gpio_ds0_ reg_1[3]
36	**ENET_RXDV**	1, GPIO (IO) 2, {0x14[4], 0x10[4], 0xc[4]} = 3'b000	1, enet_rxdv (I) 2, {0x14[4], 0x10[4], 0xc[4]} = 3'b001	1, enet_rxdv (I) 2, {0x14[4], 0x10[4], 0xc[4]} = 3'b010	1, pwm4 (O) 2, {0x14[4], 0x10[4], 0xc[4]} = 3'b011			gpio_en1[4], gpio_sel1[4]	gpio_ds1_ reg_1[4], gpio_ds0_ reg_1[4]
37	**ENET_MDC**	1, GPIO (IO) 2, {0x14[5], 0x10[5], 0xc[5]} = 3'b000	1, enet_mdc (O) 2, {0x14[5], 0x10[5], 0xc[5]} = 3'b001	1, enet_mdc (O) 2, {0x14[5], 0x10[5], 0xc[5]} = 3'b010	1, pwm5 (O) 2, {0x14[5], 0x10[5], 0xc[5]} = 3'b011	1, ahb_mdc (O) 2, {0x14[5], 0x10[5], 0xc[5]} = 3'b100		gpio_en1[5], gpio_sel1[5]	gpio_ds1_ reg_1[5], gpio_ds0_ reg_1[5]
38	**ENET_MDIO**	1, GPIO (IO) 2, {0x14[6], 0x10[6], 0xc[6]} = 3'b000	1, enet_mdio (IO) 2, {0x14[6], 0x10[6], 0xc[6]} = 3'b001	1, enet_ mdio (IO) 2, {0x14[6], 0x10[6], 0xc[6]} = 3'b010	1, pwm6 (O) 2, {0x14[6], 0x10[6], 0xc[6]} = 3'b011	1, ahb_mdio (IO) 2, {0x14[6], 0x10[6], 0xc[6]} = 3'b100		gpio_en1[6], gpio_sel1[6]	gpio_ds1_ reg_1[6], gpio_ds0_ reg_1[6]
39	**ENET_PTP_ PPS**	1, GPIO (IO) 2, {0x14[7], 0x10[7], 0xc[7]} = 3'b000	1, enet_ptp_ pps_o (O) 2, {0x14[7], 0x10[7], 0xc[7]} = 3'b001	1, enet_ptp_ pps_o (O) 2, {0x14[7], 0x10[7], 0xc[7]} = 3'b010	1, pwm7 (O) 2, {0x14[7], 0x10[7], 0xc[7]} = 3'b011			gpio_en1[7], gpio_sel1[7]	gpio_ds1_ reg_1[7], gpio_ds0_ reg_1[7]
40	**ENET_CLK_ RX**	1, GPIO (IO) 2, {0x14[8], 0x10[8], 0xc[8]} = 3'b000	1, enet_ref_ clk (IO) 2, {0x14[8], 0x10[8], 0xc[8]} = 3'b001	1, enet_clk_ rx (I) 2, {0x14[8], 0x10[8], 0xc[8]} = 3'b010	1, pwm8 (O) 2, {0x14[8], 0x10[8], 0xc[8]} = 3'b011			gpio_en1[8], gpio_sel1[8]	gpio_ds1_ reg_1[8], gpio_ds0_ reg_1[8]
41	**ENET_GTX_ CLK**	1, GPIO (IO) 2, {0x14[9], 0x10[9], 0xc[9]} = 3'b000	1, enet_2nd_ ref_clk (O) 2, {0x14[9], 0x10[9], 0xc[9]} = 3'b001	1, enet_gtx_ clk (O) 2, {0x14[9], 0x10[9], 0xc[9]} = 3'b010	1, pwm9 (O) 2, {0x14[9], 0x10[9], 0xc[9]} = 3'b011			gpio_en1[9], gpio_sel1[9]	gpio_ds1_ reg_1[9], gpio_ds0_ reg_1[9]
42	**ENET_EXT_ OSC_CLK**	1, GPIO (IO) 2, {0x14[10], 0x10[10], 0xc[10]} = 3'b000	1, enet_ext_ osc_clk (I) 2, {0x14[10], 0x10[10], 0xc[10]} = 3'b001	1, enet_ext_ osc_clk (I) 2, {0x14[10], 0x10[10], 0xc[10]} = 3'b010				gpio_en1[10], gpio_sel1[10]	gpio_ds1_ reg_1[10], gpio_ds0_ reg_1[10]
43	**ENET_ 2ND_REF**	1, GPIO (IO) 2, {0x14[11], 0x10[11], 0xc[11]} = 3'b000	1, enet_2nd_ ref_clk (O) 2, {0x14[11], 0x10[11], 0xc[11]} = 3'b001	1, enet_2nd_ ref_clk (O) 2, {0x14[11], 0x10[11], 0xc[11]} = 3'b010	1, wdt_ext_ rst_l (O) 2, {0x14[11], 0x10[11], 0xc[11]} = 3'b011	11, ssi0_en2 (O) 2, {0x14[11], 0x10[11], 0xc[11]} = 3'b100		gpio_en1[11], gpio_sel1[11]	gpio_ds1_ reg_1[11], gpio_ds0_ reg_1[11]
44	**ENET1_ TXEN**	1, GPIO (IO) 2, {0x14[12], 0x10[12], 0xc[12]} = 3'b000	1, enet1_ txen (O) 2, {0x14[12], 0x10[12], 0xc[12]} = 3'b001	1, enet1_ txen (O) 2, {0x14[12], 0x10[12], 0xc[12]} = 3'b010			1, can2_ rx (I) 2, {0x14[12], 0x10[12], 0xc[12]} = 3'b101	gpio_en1[12], gpio_sel1[12]	gpio_ds1_ reg_1[12], gpio_ds0_ reg_1[12]

Ambarella Confidential Document only

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
45	**ENET1_ TXD_0**	1, GPIO (IO) 2, {0x14[13], 0x10[13], 0xc[13]} = 3'b000	1, enet1_ txd_0 (O) 2, {0x14[13], 0x10[13], 0xc[13]} = 3'b001	1, enet1_ txd_0 (O) 2, {0x14[13], 0x10[13], 0xc[13]} = 3'b010	1, ssi5_txd (O+) 2, {0x14[13], 0x10[13], 0xc[13]} = 3'b011		1, pwm11 (O) 2, {0x14[13], 0x10[13], 0xc[13]} = 3'b101	gpio_en1[13], gpio_sel1[13]	gpio_ds1_ reg_1[13], gpio_ds0_ reg_1[13]
46	**ENET1_ TXD_1**	1, GPIO (IO) 2, {0x14[14], 0x10[14], 0xc[14]} = 3'b000	1, enet1_ txd_1 (O) 2, {0x14[14], 0x10[14], 0xc[14]} = 3'b001	1, enet1_ txd_1 (O) 2, {0x14[14], 0x10[14], 0xc[14]} = 3'b010	1, ssi5_en0 (O) 2, {0x14[14], 0x10[14], 0xc[14]} = 3'b011		1, vpp_en (O) 2, {0x14[14], 0x10[14], 0xc[14]} = 3'b101	gpio_en1[14], gpio_sel1[14]	gpio_ds1_ reg_1[14], gpio_ds0_ reg_1[14]
47	**ENET1_ TXD_2**	1, GPIO (IO) 2, {0x14[15], 0x10[15], 0xc[15]} = 3'b000		1, enet1_ txd_2 (O) 2, {0x14[15], 0x10[15], 0xc[15]} = 3'b010	1, vsdelay_ hsync_out_0 (O) 2, {0x14[15], 0x10[15], 0xc[15]} = 3'b011			gpio_en1[15], gpio_sel1[15]	gpio_ds1_ reg_1[15], gpio_ds0_ reg_1[15],
48	**ENET1_ TXD_3**	1, GPIO (IO) 2, {0x14[16], 0x10[16], 0xc[16]} = 3'b000		1, enet1_ txd_3 (O) 2, {0x14[16], 0x10[16], 0xc[16]} = 3'b010	1, vsdelay_ hsync_out_1 (O) 2, {0x14[16], 0x10[16], 0xc[16]} = 3'b011		1, can2_ tx (O) 2, {0x14[16], 0x10[16], 0xc[16]} = 3'b101	gpio_en1[16], gpio_sel1[16]	gpio_ds1_ reg_1[16] gpio_ds0_ reg_1[16]
49	**ENET1_ RXD_0**	1, GPIO (IO) 2, {0x14[17], 0x10[17], 0xc[17]} = 3'b000	1, enet1_ rxd_0 (I) 2, {0x14[17], 0x10[17], 0xc[17]} = 3'b001	1, enet1_ rxd_0 (I) 2, {0x14[17], 0x10[17], 0xc[17]} = 3'b010	1, ssi5_sclk (O) 2, {00x14[17], 0x10[17], 0xc[17]} = 3'b011	1, uart2_ahb_ rx (I) 2, {0x14[17], 0x10[17], 0xc[17]} = 3'b100	1, ssis_ sclk (I) 2, {0x14[17], 0x10[17], 0xc[17]} = 3'b101	gpio_en1[17], gpio_sel1[17]	gpio_ds1_ reg_1[17] gpio_ds0_ reg_1[17]
50	**ENET1_ RXD_1**	1, GPIO (IO) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b000	1, enet1_ rxd_1 (I) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b001	1, enet1_ rxd_1 (I) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b010	1, ssi5_rxd (I) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b011	1, uart2_ahb_ tx (O) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b100	1, ssis_ txd (O+) 2, {0x14[18], 0x10[18], 0xc[18]} = 3'b101	gpio_en1[18], gpio_sel1[18]	gpio_ds1_ reg_1[18], gpio_ds0_ reg_1[18]
51	**ENET1_ RXD_2**	1, GPIO (IO) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b000	1, idc5clk (IO) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b001	1, enet1_ rxd_2 (I) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b010	1, vsdelay_ vsync_out_0 (O) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b011	1, uart2_ahb_ cts_n (I) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b100	1, ssis_ rxd (I) 2, {0x14[19], 0x10[19], 0xc[19]} = 3'b101	gpio_en1[19], gpio_sel1[19]	gpio_ds1_ reg_1[19], gpio_ds0_ reg_1[19]
52	**ENET1_ RXD_3**	1, GPIO (IO) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b000	1, idc5data (IO) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b001	1, enet1_ rxd_3 (I) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b010	1, vsdelay_ vsync_out_1 (O) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b011	1, uart2_ahb_ rts_n (O) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b100	1, ssis_en (I) 2, {0x14[20], 0x10[20], 0xc[20]} = 3'b101	gpio_en1[20], gpio_sel1[20]	gpio_ds1_ reg_1[20], gpio_ds0_ reg_1[20]

Ambarella Confidential or Under NDA

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
53	**ENET1_ RXDV**	1, GPIO (IO) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b000	1, enet1_ rxdv (I) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b001	1, enet1_ rxdv (I) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b010	1, ext_vsync_ src_input (I) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b011	1, ssi4_sclk (O) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b100	1, can3_ rx (I) 2, {0x14[21], 0x10[21], 0xc[21]} = 3'b101	gpio_en1[21], gpio_sel1[21]	gpio_ds1_ reg_1[21], gpio_ds0_ reg_1[21]
54	**ENET1_MDC**	1, GPIO (IO) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b000	1, enet1_ mdc (O) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b001	1, enet1_ mdc (O) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b010	1, ahb_mdc1 (O) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b011	1, ssi4_txd (O+) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b100	1, can3_ tx (O) 2, {0x14[22], 0x10[22], 0xc[22]} = 3'b101	gpio_en1[22], gpio_sel1[22]	gpio_ds1_ reg_1[22], gpio_ds0_ reg_1[22]
55	**ENET1_ MDIO**	1, GPIO (IO) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b000	1, enet1_ mdio (IO) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b001	1, enet1_ mdio (IO) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b010	1, ahb_mdio1 (IO) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b011	1, ssi4_rxd (I) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b100	1, can4_ rx (I) 2, {0x14[23], 0x10[23], 0xc[23]} = 3'b101	gpio_en1[23], gpio_sel1[23]	gpio_ds1_ reg_1[23], gpio_ds0_ reg_1[23]
56	**ENET1_PTP_ PPS**	1, GPIO (IO) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b000	1, enet1_ ptp_pps_o (O) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b001	1, enet1_ ptp_pps_o (O) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b010	1, ext_vsync_ src_input (I) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b011	1, ssi4_en0 (O) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b100	1, can4_ tx (O) 2, {0x14[24], 0x10[24], 0xc[24]} = 3'b101	gpio_en1[24], gpio_sel1[24]	gpio_ds1_ reg_1[24], gpio_ds0_ reg_1[24]
57	**ENET1_CLK_ RX**	1, GPIO (IO) 2, {0x14[25], 0x10[25], 0xc[25]} = 3'b000	1, enet1_ ref_clk (IO) 2, {0x14[25], 0x10[25], 0xc[25]} = 3'b001	1, enet1_ clk_rx (I) 2, {0x14[25], 0x10[25], 0xc[25]} = 3'b010		1, ehci_app_ prt_ovcurr0 (I) 2, {0x14[25], 0x10[25], 0xc[25]} = 3'b100	1, can5_ rx (I) 2, {0x14[25], 0x10[25], 0xc[25]} = 3'b101	gpio_en1[25], gpio_sel1[25]	gpio_ds1_ reg_1[25], gpio_ds0_ reg_1[25]
58	**ENET1_GTX_ CLK**	1, GPIO (IO) 2, {0x14[26], 0x10[26], 0xc[26]} = 3'b000	1, enet_2nd_ ref_clk (O) 2, {0x14[26], 0x10[26], 0xc[26]} = 3'b001	1, enet1_ gtx_clk (O) 2, {0x14[26], 0x10[26], 0xc[26]} = 3'b010		1, ehci_prt_ pwr_0 (O) 2, {0x14[26], 0x10[26], 0xc[26]} = 3'b100	1, can5_ tx (O) 2, {0x14[26], 0x10[26], 0xc[26]} = 3'b101	gpio_en1[26], gpio_sel1[26]	gpio_ds1_ reg_1[26], gpio_ds0_ reg_1[26]
59	**SMIO_0**	1, GPIO (IO) 2, {0x14[27], 0x10[27], 0xc[27]} = 3'b000		1, snand_ sck (O) 2, {0x14[27], 0x10[27], 0xc[27]} = 3'b010	1, norspi_clk (O) 2, {0x14[27], 0x10[27], 0xc[27]} = 3'b011			gpio_en1[27], gpio_sel1[27]	gpio_ds1_ reg_1[27], gpio_ds0_ reg_1[27]
60	**SMIO_1**	1, GPIO (IO) 2, {0x14[28], 0x10[28], 0xc[28]} = 3'b000		1, snand_ cs_n (O) 2, {0x14[28], 0x10[28], 0xc[28]} = 3'b010	1, norspi_en (O) 2, {0x14[28], 0x10[28], 0xc[28]} = 3'b011			gpio_en1[28], gpio_sel1[28]	gpio_ds1_ reg_1[28], gpio_ds0_ reg_1[28]
61	**SMIO_2**	1, GPIO (IO) 2, {0x14[29], 0x10[29], 0xc[29]} = 3'b000		1, sd_clk (O) 2, {0x14[29], 0x10[29], 0xc[29]} = 3'b010				gpio_en1[29], gpio_sel1[29]	gpio_ds1_ reg_1[29], gpio_ds0_ reg_1[29]

Ambarella Confidential Last Updated 2025

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
62	**SMIO_3**	1, GPIO (IO) 2, {0x14[30], 0x10[30], 0xc[30]} = 3'b000		1, sd_cmd (IO) 2, {0x14[30], 0x10[30], 0xc[30]} = 3'b010				gpio_en1[30], gpio_sel1[30]	gpio_ds1_ reg_1[30], gpio_ds0_ reg_1[30]
63	**SMIO_4**	1, GPIO (IO) 2, {0x14[31], 0x10[31], 0xc[31]} = 3'b000		1, sd_cd (I) 2, {0x14[31], 0x10[31], 0xc[31]} = 3'b010				gpio_en1[31], gpio_sel1[31]	gpio_ds1_ reg_1[31], gpio_ds0_ reg_1[31]
64	**SMIO_5**	1, GPIO (IO) 2, {0x20[0], 0x1c[0], 0x18[0]} = 3'b000		1, sd_wp (I) 2, {0x20[0], 0x1c[0], 0x18[0]} = 3'b010				gpio_en2[0], gpio_sel2[0]	gpio_ds1_ reg_2[0], gpio_ds0_ reg_2[0]
65	**SMIO_6**	1, GPIO (IO) 2, {0x20[1], 0x1c[1], 0x18[1]} = 3'b000		1, snand_ si[0] (IO) 2, {0x20[1], 0x1c[1], 0x18[1]} = 3'b010	1, norspi_dq[0] (IO) 2, {0x20[1], 0x1c[1], 0x18[1]} = 3'b011			gpio_en2[1], gpio_sel2[1]	gpio_ds1_ reg_2[1], gpio_ds0_ reg_2[1]
66	**SMIO_7**	1, GPIO (IO) 2, {0x20[2], 0x1c[2], 0x18[2]} = 3'b000		1, snand_ si[1] (IO) 2, {0x20[2], 0x1c[2], 0x18[2]} = 3'b010	1, norspi_dq[1] (IO) 2, {0x20[2], 0x1c[2], 0x18[2]} = 3'b011			gpio_en2[2], gpio_sel2[2]	gpio_ds1_ reg_2[2], gpio_ds0_ reg_2[2]
67	**SMIO_8**	1, GPIO (IO) 2, {0x20[3], 0x1c[3], 0x18[3]} = 3'b000		1, snand_ si[2] (IO) 2, {0x20[3], 0x1c[3], 0x18[3]} = 3'b010	1, norspi_dq[2] (IO) 2, {0x20[3], 0x1c[3], 0x18[3]} = 3'b011			gpio_en2[3], gpio_sel2[3]	gpio_ds1_ reg_2[3], gpio_ds0_ reg_2[3]
68	**SMIO_9**	1, GPIO (IO) 2, {0x20[4], 0x1c[4], 0x18[4]} = 3'b000		1, snand_ si[3] (IO) 2, {0x20[4], 0x1c[4], 0x18[4]} = 3'b010	1, norspi_dq[3] (IO) 2, {0x20[4], 0x1c[4], 0x18[4]} = 3'b011			gpio_en2[4], gpio_sel2[4]	gpio_ds1_ reg_2[4], gpio_ds0_ reg_2[4]
69	**SMIO_18**	1, GPIO (IO) 2, {0x20[5], 0x1c[5], 0x18[5]} = 3'b000		1, sd_d[0] (IO) 2, {0x20[5], 0x1c[5], 0x18[5]} = 3'b010				gpio_en2[5], gpio_sel2[5]	gpio_ds1_ reg_2[5], gpio_ds0_ reg_2[5]
70	**SMIO_19**	1, GPIO (IO) 2, {0x20[6], 0x1c[6], 0x18[6]} = 3'b000		1, sd_d[1] (IO) 2, {0x20[6], 0x1c[6], 0x18[6]} = 3'b010				gpio_en2[6], gpio_sel2[6]	gpio_ds1_ reg_2[6], gpio_ds0_ reg_2[6]
71	**SMIO_20**	1, GPIO (IO) 2, {0x20[7], 0x1c[7], 0x18[7]} = 3'b000		1, sd_d[2] (IO) 2, {0x20[7], 0x1c[7], 0x18[7]} = 3'b010				gpio_en2[7], gpio_sel2[7]	gpio_ds1_ reg_2[7], gpio_ds0_ reg_2[7]

For Ambarella Confidential Use (Cooling Only)

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
72	**SMIO_21**	1, GPIO (IO) 2, {0x20[8], 0x1c[8], 0x18[8]} = 3'b000		1, sd_d[3] (IO) 2, {0x20[8], 0x1c[8], 0x18[8]} = 3'b010				gpio_en2[8], gpio_sel2[8]	gpio_ds1_ reg_2[8], gpio_ds0_ reg_2[8]
73	**SMIO_22**	1, GPIO (IO) 2, {0x20[9], 0x1c[9], 0x18[9]} = 3'b000		1, sd_d[4] (IO) 2, {0x20[9], 0x1c[9], 0x18[9]} = 3'b010	1, norspi_dq[4] (IO) 2, {0x20[9], 0x1c[9], 0x18[9]} = 3'b011			gpio_en2[9], gpio_sel2[9]	gpio_ds1_ reg_2[9], gpio_ds0_ reg_2[9]
74	**SMIO_23**	1, GPIO (IO) 2, {0x20[10], 0x1c[10], 0x18[10]} = 3'b000		1, sd_d[5] (IO) 2, {0x20[10], 0x1c[10], 0x18[10]} = 3'b010	1, norspi_dq[5] (IO) 2, {0x20[10], 0x1c[10], 0x18[10]} = 3'b011			gpio_en2[10], gpio_sel2[10]	gpio_ds1_ reg_2[10], gpio_ds0_ reg_2[10]
75	**SMIO_24**	1, GPIO (IO) 2, {0x20[11], 0x1c[11], 0x18[11]} = 3'b000		1, sd_d[6] (IO) 2, {0x20[11], 0x1c[11], 0x18[11]} = 3'b010	1, norspi_dq[6] (IO) 2, {0x20[11], 0x1c[11], 0x18[11]} = 3'b011			gpio_en2[11], gpio_sel2[11]	gpio_ds1_ reg_2[11], gpio_ds0_ reg_2[11]
76	**SMIO_25**	1, GPIO (IO) 2, {0x20[12], 0x1c[12], 0x18[12]} = 3'b000		1, sd_d[7] (IO) 2, {0x20[12], 0x1c[12], 0x18[12]} = 3'b010	1, norspi_dq[7] (IO) 2, {0x20[12], 0x1c[12], 0x18[12]} = 3'b011			gpio_en2[12], gpio_sel2[12]	gpio_ds1_ reg_2[12], gpio_ds0_ reg_2[12]
77	**SMIO_26**	1, GPIO (IO) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b000	1, uart2_ ahb_rx (I) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b001	1, sdio0_clk (O) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b010	1, ssi1_sclk (O) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b011	1, snand_sck (O) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b100	1, norspi_ clk (O) 2, {0x20[13], 0x1c[13], 0x18[13]} = 3'b101	gpio_en2[13], gpio_sel2[13]	gpio_ds1_ reg_2[13], gpio_ds0_ reg_2[13]
78	**SMIO_27**	1, GPIO (IO) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b000	1, uart2_ ahb_tx (O) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b001	1, sdio0_cmd (IO) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b010	1, ssi1_txd (O+) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b011	1, snand_cs_n (O) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b100	1, norspi_ en (O) 2, {0x20[14], 0x1c[14], 0x18[14]} = 3'b101	gpio_en2[14], gpio_sel2[14]	gpio_ds1_ reg_2[14], gpio_ds0_ reg_2[14]
79	**SMIO_28**	1, GPIO (IO) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b000	1, uart2_ ahb_cts_n (I) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b001	1, sdio0_d[0] (IO) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b010	1, ssi1_rxd (I) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b011	1, snand_si[0] (IO) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b100	1, norspi_ dq[0] (IO) 2, {0x20[15], 0x1c[15], 0x18[15]} = 3'b101	gpio_en2[15], gpio_sel2[15]	gpio_ds1_ reg_2[15], gpio_ds0_ reg_2[15],
80	**SMIO_29**	1, GPIO (IO) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b000	1, uart2_ ahb_rts_n (O) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b001	1, sdio0_d[1] (IO) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b010	1, ssi1_en0 (O) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b011	1, snand_si[1] (IO) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b100	1, norspi_ dq[1] (IO) 2, {0x20[16], 0x1c[16], 0x18[16]} = 3'b101	gpio_en2[16], gpio_sel2[16]	gpio_ds1_ reg_2[16], gpio_ds0_ reg_2[16]

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
81	**SMIO_30**	1, GPIO (IO) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b000	1, uart3_ ahb_cts_n (I) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b001	1, sdio0_d[2] (IO) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b010	1, ssi1_en1 (O) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b011	1, snand_si[2] (IO) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b100	1, norspi_ dq[2] (IO) 2, {0x20[17], 0x1c[17], 0x18[17]} = 3'b101	gpio_en2[17], gpio_sel2[17]	gpio_ds1_ reg_2[17], gpio_ds0_ reg_2[17]
82	**SMIO_31**	1, GPIO (IO) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b000	1, uart3_ ahb_rts_n (O) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b001	1, sdio0_d[3] (IO) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b010	1, ssi1_en2 (O) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b011	1, snand_si[3] (IO) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b100	1, norspi_ dq[3] (IO) 2, {0x20[18], 0x1c[18], 0x18[18]} = 3'b101	gpio_en2[18], gpio_sel2[18]	gpio_ds1_ reg_2[18], gpio_ds0_ reg_2[18]
83	**SMIO_32**	1, GPIO (IO) 2, {0x20[19], 0x1c[19], 0x18[19]} = 3'b000	1, uart3_ ahb_rx (I) 2, {0x20[19], 0x1c[19], 0x18[19]} = 3'b001	1, sdio0_cd (I) 2, {0x20[19], 0x1c[19], 0x18[19]} = 3'b010	1, ssi1_en3 (O) 2, {0x20[19], 0x1c[19], 0x18[19]} = 3'b011			gpio_en2[19], gpio_sel2[19]	gpio_ds1_ reg_2[19], gpio_ds0_ reg_2[19]
84	**SMIO_33**	1, GPIO (IO) 2, {0x20[20], 0x1c[20], 0x18[20]} = 3'b000	1, uart3_ ahb_tx (O) 2, {0x20[20], 0x1c[20], 0x18[20]} = 3'b001	1, sdio0_wp (I) 2, {0x20[20], 0x1c[20], 0x18[20]} = 3'b010				gpio_en2[20], gpio_sel2[20]	gpio_ds1_ reg_2[20], gpio_ds0_ reg_2[20]
85	**SMIO_34**	1, GPIO (IO) 2, {0x20[21], 0x1c[21], 0x18[21]} = 3'b000			1, norspi_dqs (I) 2, {0x20[21], 0x1c[21], 0x18[21]} = 3'b011			gpio_en2[21], gpio_sel2[21]	gpio_ds1_ reg_2[21], gpio_ds0_ reg_2[21]
86	**SMIO_35**	1, GPIO (IO) 2, {0x20[22], 0x1c[22], 0x18[22]} = 3'b000		1, sd_reset (O) 2, {0x20[22], 0x1c[22], 0x18[22]} = 3'b010				gpio_en2[22], gpio_sel2[22]	gpio_ds1_ reg_2[22], gpio_ds0_ reg_2[22]
87	**SENSOR_ RST**	1, GPIO (IO) 2, {0x20[23], 0x1c[23], 0x18[23]} = 3'b000		1, wdt_ext_ rst_l (O) 2, {0x20[23], 0x1c[23], 0x18[23]} = 3'b010				gpio_en2[23], gpio_sel2[23]	gpio_ds1_ reg_2[23], gpio_ds0_ reg_2[23]
88	**SHSYNC**	1, GPIO (IO) 2, {0x20[24], 0x1c[24], 0x18[24]} = 3'b000	1, idsp_ vin0_iopad_ master_ hsync (O) 2, {0x20[24], 0x1c[24], 0x18[24]} = 3'b001	1, idsp_vin1_ iopad_mas- ter_hsync (O) 2, {0x20[24], 0x1c[24], 0x18[24]} = 3'b010				gpio_en2[24], gpio_sel2[24]	gpio_ds1_ reg_2[24], gpio_ds0_ reg_2[24]
89	**SVSYNC**	1, GPIO (IO) 2, {0x20[25], 0x1c[25], 0x18[25]} = 3'b000	1, idsp_ vin0_iopad_ master_ vsync (O) 2, {0x20[25], 0x1c[25], 0x18[25]} = 3'b001	1, idsp_vin1_ iopad_mas- ter_vsync (O) 2, {0x20[25], 0x1c[25], 0x18[25]} = 3'b010				gpio_en2[25], gpio_sel2[25]	gpio_ds1_ reg_2[25], gpio_ds0_ reg_2[25]

For Confidential Planning only (08012025)

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
90	**UART0_AHB_RX**	1, GPIO (IO) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b000	1, uart0_ahb_rx (I) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b001	1, ssi1_sclk (O) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b010	1, ehci_app_prt_ovcurr0 (I) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b011	1, tm11_clk (I) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b100	1, pwm10 (O) 2, {0x20[26], 0x1c[26], 0x18[26]} = 3'b101	gpio_en2[26], gpio_sel2[26]	gpio_ds1_reg_2[26], gpio_ds0_reg_2[26]
91	**UART0_AHB_TX**	1, GPIO (IO) 2, {0x20[27], 0x1c[27], 0x18[27]} = 3'b000	1, uart0_ahb_tx (O) 2, {0x20[27], 0x1c[27], 0x18[27]} = 3'b001	1, ssi1_txd (O+) 2, {0x20[27], 0x1c[27], 0x18[27]} = 3'b010			1, wdt_ext_rst_l (O) 2, {0x20[27], 0x1c[27], 0x18[27]} = 3'b101	gpio_en2[27], gpio_sel2[27]	gpio_ds1_reg_2[27], gpio_ds0_reg_2[27]
92	**UART0_AHB_CTS_N**	1, GPIO (IO) 2, {0x20[28], 0x1c[28], 0x18[28]} = 3'b000	1, uart0_ahb_cts_n (I) 2, {0x20[28], 0x1c[28], 0x18[28]} = 3'b001	1, ssi1_rxd (I) 2, {0x20[28], 0x1c[28], 0x18[28]} = 3'b010	1, ehci_prt_pwr_0 (O) 2, {0x20[28], 0x1c[28], 0x18[28]} = 3'b011		1, tm12_clk (I) 2, {0x20[28], 0x1c[28], 0x18[28]} = 3'b101	gpio_en2[28], gpio_sel2[28]	gpio_ds1_reg_2[28], gpio_ds0_reg_0[28]
93	**UART0_AHB_RTS_N**	1, GPIO (IO) 2, {0x20[29], 0x1c[29], 0x18[29]} = 3'b000	1, uart0_ahb_rts_n (O) 2, {0x20[29], 0x1c[29], 0x18[29]} = 3'b001	1, ssi1_en0 (O) 2, {0x20[29], 0x1c[29], 0x18[29]} = 3'b010			1, tm13_clk (I) 2, {0x20[29], 0x1c[29], 0x18[29]} = 3'b101	gpio_en2[29], gpio_sel2[29]	gpio_ds1_reg_2[29], gpio_ds0_reg_2[29]
94	**UART1_AHB_RX**	1, GPIO (IO) 2, {0x20[30], 0x1c[30], 0x18[30]} = 3'b000	1, uart1_ahb_rx (I) 2, {0x20[30], 0x1c[30], 0x18[30]} = 3'b001	1, tm12_clk (I) 2, {0x20[30], 0x1c[30], 0x18[30]} = 3'b010				gpio_en2[30], gpio_sel2[30]	gpio_ds1_reg_2[30], gpio_ds0_reg_2[30]
95	**UART1_AHB_TX**	1, GPIO (IO) 2, {0x20[31], 0x1c[31], 0x18[31]} = 3'b000	1, uart1_ahb_tx (O) 2, {0x20[31], 0x1c[31], 0x18[31]} = 3'b001					gpio_en2[31], gpio_sel2[31]	gpio_ds1_reg_2[31], gpio_ds0_reg_2[31]
96	**UART1_AHB_CTS_N**	1, GPIO (IO) 2, {0x2c[0], 0x28[0], 0x24[0]} = 3'b000	1, uart1_ahb_cts_n (I) 2, {0x2c[0], 0x28[0], 0x24[0]} = 3'b001	1, idsp_vin1_iopad_master_vsync (O) 2, {0x2c[0], 0x28[0], 0x24[0]} = 3'b010				gpio_en3[0], gpio_sel3[0]	gpio_ds1_reg_3[0], gpio_ds0_reg_3[0]
97	**UART1_AHB_RTS_N**	1, GPIO (IO) 2, {0x2c[1], 0x28[1], 0x24[1]} = 3'b000	1, uart1_ahb_rts_n (O) 2, {0x2c[1], 0x28[1], 0x24[1]} = 3'b001	1, idsp_vin1_iopad_master_hsync (O) 2, {0x2c[1], 0x28[1], 0x24[1]} = 3'b010				gpio_en3[1], gpio_sel3[1]	gpio_ds1_reg_3[1], gpio_ds0_reg_3[1]
98	**CAN0_RX**	1, GPIO (IO) 2, {0x2c[2], 0x28[2], 0x24[2]} = 3'b000	1, can0_rx (I) 2, {0x2c[2], 0x28[2], 0x24[2]} = 3'b001					gpio_en3[2], gpio_sel3[2]	gpio_ds1_reg_3[2], gpio_ds0_reg_3[2]

For Internal Use Only Confidential Ambarella Inc. 2025

GPIO	Pin Name	Multiplexed Function						GPIO Pull/Select Register	Drive Strength Register
		Alt0	Alt1	Alt2	Alt3	Alt4	Alt5		
99	**CAN0_TX**	1, GPIO (IO) 2, {0x2c[3], 0x28[3], 0x24[3]} = 3'b000	1, can0_tx (O) 2, {0x2c[3], 0x28[3], 0x24[3]} = 3'b001					gpio_en3[3], gpio_sel3[3]	gpio_ds1_ reg_3[3], gpio_ds0_ reg_3[3]
100	**CAN1_RX**	1, GPIO (IO) 2, {0x2c[4], 0x28[4], 0x24[4]} = 3'b000	1, can1_rx (I) 2, {0x2c[4], 0x28[4], 0x24[4]} = 3'b001					gpio_en3[4], gpio_sel3[4]	gpio_ds1_ reg_3[4], gpio_ds0_ reg_3[4]
101	**CAN1_TX**	1, GPIO (IO) 2, {0x2c[5], 0x28[5], 0x24[5]} = 3'b000	1, can1_tx (O) 2, {0x2c[5], 0x28[5], 0x24[5]} = 3'b001					gpio_en3[5], gpio_sel3[5]	gpio_ds1_ reg_3[5], gpio_ds0_ reg_3[5]
102	**SD_HS_SEL**	1, GPIO (IO) 2, {0x2c[6], 0x28[6], 0x24[6]} = 3'b000	1, sd_hs_sel (O) 2, {0x2c[6], 0x28[6], 0x24[6]} = 3'b001					gpio_en3[6], gpio_sel3[6]	gpio_ds1_ reg_3[6], gpio_ds0_ reg_3[6]
--	**CLK_AU***		1, clk_au (O) 2, {0x38[3], 0x34[3], 0x30[3]} = 3'b001					other_en[1], other_sel[1]	other_ds1_ reg[1], other_ds0_ reg[1]
--	**USB0_DE-TECT_VBUS***							other_en[2], other_sel[2]	other_ds1_ reg[2], other_ds0_ reg[2]
--	**CK_SI***							other_en[3], other_sel[3]	other_ds1_ reg[3], other_ds0_ reg[3]
--	**CLK_SI_1***							other_en[4], other_sel[4]	other_ds1_ reg[4], other_ds0_ reg[4]
--	**CHU_ SAFETY_ ERR_L_0***							other_en[5], other_sel[5]	other_ds1_ reg[5], other_ds0_ reg[5]
--	**CHU_ SAFETY_ ERR_L_1***							other_en[6], other_sel[6]	other_ds1_ reg[6], other_ds0_ reg[6]

Table 19-15. GPIO Function Selection: Assignment Table.

* Pads not muxed with GPIO, but use the register to program drive strength, pull en / pull sel, and mux alt.

Confidential Lang only Ambarella 08012025

19.4.2.1 Addresses for Drive Strength and GPIO Pull / Select Registers

Address	Register	Bits	Reset	Description
0xED080314	**GPIO_DS0_REG_0**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[31:0] (Refer to Section 19.5 for decoding)
0xED080318	**GPIO_DS1_REG_0**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[31:0] (Refer to Section 19.5 for decoding)
0xED08031C	**GPIO_DS0_REG_1**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[63:32] (Refer to Section 19.5 for decoding)
0xED080320	**GPIO_DS1_REG_1**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[63:32] (Refer to Section 19.5 for decoding)
0xED080324	**GPIO_DS0_REG_2**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[95:64] (Refer to Section 19.5 for decoding)
0xED080328	**GPIO_DS1_REG_2**	32	0xFFFFFFFF	Control bits to decode DS* input of GPIO[95:64] (Refer to Section 19.5 for decoding)
0xED08032C	**GPIO_DS0_REG_3**	32	0xFFFFFFFF	Control bits to deocde DS* input of GPIO[127:96] (Refer to Section 19.5 for decoding)
0xED080330	**GPIO_DS1_REG_3**	32	0xFFFFFFFF	Control bits to decode for DS* input of GPIO[127:96] (Refer to Section 19.5 for decoding)
0xED080450	**OTHER_DS0_REG**	7	0x7F	Control bit to decode DS* input of non GPIO pins (CLK_SI, CLK_AU, DETECT_VBUS, CLK_SI1) (Refer to Section 19.5 for decoding)
0xED080454	**OTHER_DS1_REG**	7	0x7F	Control bit to decode DS* input of non GPIO pins (CLK_SI, CLK_AU, DETECT_VBUS, CLK_SI1) (Refer to Section 19.5 for decoding)

Table 19-16. Addresses for Drive Strength Registers.

GPIO Pull / Select Register	Address
GPIO_EN0	0xE0022014
GPIO_SEL0	0xE0022030
GPIO_EN1	0xE0022018
GPIO_SEL1	0xE0022034
GPIO_EN2	0xE002201C
GPIO_SEL2	0xE0022038
GPIO_EN3	0xE0022020
GPIO_SEL3	0xE002203C
OTHER_EN	0xE0022008
OTHER_SEL	0xE002200C

Table 19-17. Addresses for GPIO Pull / Select Registers.

The bit locations in each register set must be programmed as a whole; i.e., treat the 3 bits in the same location on each of the three registers as a 3-bit storage. Refer to the following example.

Assume that the goal is to utilize function 0x1 on pin **0**. In order to accomplish this, the first bit location of register set 0 must be programmed to 0x1 as follows:

1. Load **iomux_reg0_0**.

For Mekra Lang only Confidential (Jan 2025)

OR **iomux_reg0_0** with 0x1.

2. Store value back to **iomux_reg0_0**.
3. Load **iomux_reg0_1**.

   AND **iomux_reg0_1** with 0xFFFE.
4. Store value back to **iomux_reg0_1**.
5. Load **iomux_reg0_2**.

   AND **iomux_reg0_2** with 0xFFFE.
6. Store value back to **iomux_reg0_2**.
7. Store 1 to **iomux_ctrl_set**.
8. Store 0 to **iomux_ctrl_set**.

In summary, bit 0 of **IOMUX** register set 0 can be jointly seen as {0, 0, 1}. Note that the configuration will not take effect without the completion of the final two steps.

19.5 GPIO: RCT Drive Strength Decoding in IOPAD

DS_Sel0	DS_Sel1	DS0/DS1/DS2 Setting	Output Driver Strength (Driver impedance)
0	0	010	X3 (50Ω)
0	1	011	X4 (40Ω)
1	0	100	X5 (33Ω)
1	1	101	X6 (25Ω)

Table 19-18. RCT Drive Strength Decoding in IOPAD.

For Confidential Mokra Lang only (08012025)

20. SSI / SPI

This chapter contains register programming information for the CV22FS serial synchronous / serial peripheral interfaces (SSI / SPI). The chapter is organized as follows:

- (Section 20.1) SSI / SPI: Overview
- (Section 20.2) SSI / SPI: Programming Notes
- (Section 20.3) SSI / SPI: Registers
- (Section 20.4) SSI / SPI: SSI Safety Notes
- (Section 20.5) SSI / SPI: SPI NOR Programming and Register Information
- (Section 20.6) SSI / SPI: SPI NOR Safety Notes

20.1 SSI / SPI: Overview

The CV22FS chip provides six SSI / SPI masters, providing up to sixteen device enables in total, and one dedicated slave. The SSI / SPI interface provides interrupt capability, various transfer modes, and serial master operation. The clock speed and device-enable polarity can be changed with register programming. The CV22FS provides Motorola interface capability.

In addition, the CV22FS chip includes a 4-bit SPI NOR Flash controller, which features asynchronous FIFO, synchronous FIFO, a clock divider, and a two-input AHB input arbiter.

20.1.1 Overview: Master / Slave Relationship

The following figure shows the relationship between a serial master (left) and two slave devices (right) located on the serial bus.

For Ambarella Lang only Confidential (08012025)

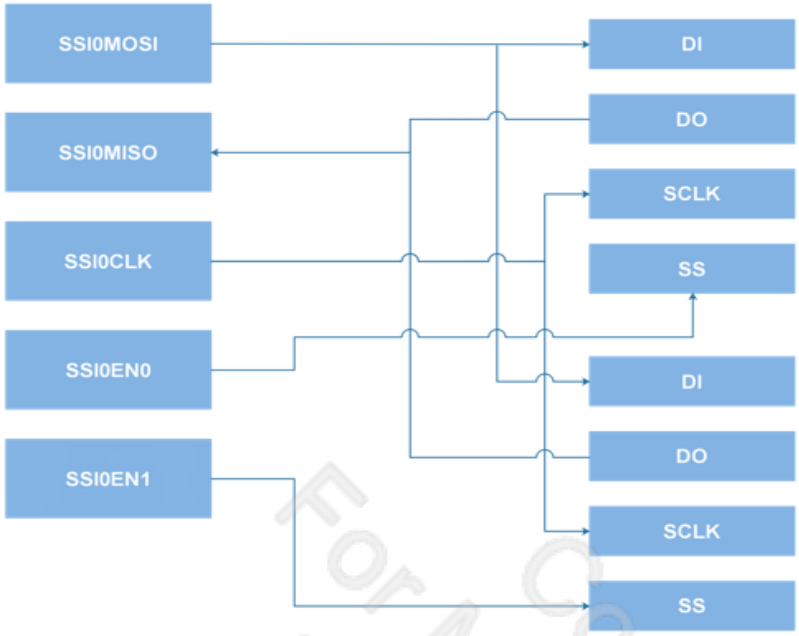


Figure 20-1. SPI Interface with the SSI / SPI as the Serial Master and Two Slave Devices on the Serial Bus.

Note that the pins used to configure the SSI / SPI interfaces are muxed with general purpose input / output (GPIO) functions.

20.1.2 Overview: Interfaces and Device Enables

The CV22FS SSI / SPI master and slave interfaces are located on the AHB bus and maintain their own register spaces. The following table provides general information on the SSI / SPI interfaces.

Port	Base Address	DMA	FIFO	Baud Rate (MHz)	Enables	Device Enable Pins	SSI/SPI Function
SSI0	E001.1000	Yes	32 entries (16 bits per entry)	20	8	**SSI0EN0**	ssi0_en0 Device Enable
						SSI0EN1	ssi0_en1 Device Enable
						SSI2CLK, ENET_2ND_REF	ssi0_en2 Device Enable
						SSI2MOSI	ssi0_en3 Device Enable
						SSI2MISO	ssi0_en4 Device Enable
						SSI2EN0	ssi0_en5 Device Enable
						IDCSCLK	ssi0_en6 Device Enable
						IDCSDATA	ssi0_en7 Device Enable
SSI1	E001.2000	Yes	32 entries (16 bits per entry)	20	4	**I2S_SO_0, SMIO_29, UART0_AHB_RTS_N**	ssi1_en0 Device Enable
						SMIO_30	ssi1_en1 Device Enable
						SMIO_31	ssi1_en2 Device Enable
						SMIO_32	ssi1_en3 Device Enable

Confidential For Mekta Lang Only 1.5 (08/12/2023)

Port	Base Address	DMA	FIFO	Baud Rate (MHz)	Enables	Device Enable Pins	SSI/SPI Function
SSI2	E001.3000	Yes	32 entries (16 bits per entry)	20	1	**SSI2EN0**	ssi2_en0 Device Enable
SSI3	E001.4000	Yes	32 entries (16 bits per entry)	20	1	**SSI3EN0**	ssi3_en0 Device Enable
SSI4	E003.4000	Yes	32 entries (16 bits per entry)	20	1	**ENET1_PTP_PPS**	ssi4_en0 Device Enable
SSI5	E003.5000	Yes	32 entries (16 bits per entry)	20	1	**ENET1_TXD_1**	ssi5_en0 Device Enable
SSIS0 (Slave)	E001.0000	Yes	32 entries (16 bits per entry)	10	Not Applicable		

Table 20-1. SSI / SPI Interface Overview Information.

Pins related to the SPI NOR controller are listed along with their associated functions in the table below.

SPI NOR Pins	Function
SMIO_0	SPI NOR Clock
SMIO_13	SPI NOR Data Lane 0
SMIO_14	SPI NOR Data Lane 1
SMIO_15	SPI NOR Data Lane 2
SMIO_16	SPI NOR Data Lane 3
SMIO_1	SPI NOR Data Lane 4
SMIO_6	SPI NOR Data Lane 5

For Meta Lang only Confidential (08012025)

SPI NOR Pins	Function
SMIO_7	SPI NOR Data Lane 6
SMIO_8	SPI NOR Data Lane 7
SMIO_9	norspi_en[0] Device Enable
SMIO_10	norspi_en[1] Device Enable
SMIO_11	norspi_en[2] Device Enable
SMIO_12	norspi_en[3] Device Enable
SMIO_22	norspi_en[4] Device Enable
SMIO_23	norspi_en[5] Device Enable
SMIO_24	norspi_en[6] Device Enable
SMIO_25	norspi_en[7] Device Enable

Table 20-2. SPI NOR Pins and Functions.

Confidential For Mekra Lang only (08012025)

20.2 SSI / SPI: Programming Notes

This section provides programming details for the CV22FS SSI / SPI interface as follows:

- (Section 20.2.1) SSI / SPI Clocking and Timing Diagrams
- (Section 20.2.2) SSI / SPI Transmit and Receive FIFO Buffers
- (Section 20.2.3) SSI / SPI Interrupts
- (Section 20.2.4) SSI / SPI Transfer Modes
- (Section 20.2.5) SSI / SPI Operational Modes
- (Section 20.2.6) SSI / SPI Super Mode

20.2.1 SSI / SPI Clocking and Timing Diagrams

The SSI / SPI serial bit clock is determined using the clock source (GCLK_SSI for the SSI master, GCLK_SSI2 for the SSI slave, or GCLK_SSI3 for SPI NOR) along with the clock divider register at register **SSSP_baudr** (offset 0x14). The maximum frequency of the SSI / SPI bit-rate clock is restricted to allow the shift-control logic to capture data on one clock edge and propagate data on the opposite edge.

The frequency of an SSI / SPI bit clock can be derived from the following equation:

Freq = Freq(CLK_IN) / **sckdv**

where **sckdv** is the **SSSP_baudr** bit field that can hold any even value in the range 0 to 65,534. If **sckdv** is 0, then **SSI[N]_CLK** is disabled. See Section 20.3.2.5 “SSSP_baudr Register” for details.

The clock polarity (**SSSP_ctrlr0** field **scpol**) configuration parameter determines whether the inactive state of the serial clock is high or low. To transmit data, both SSI / SPI peripherals must maintain the identical serial clock phase (**SSSP_ctrlr0** field **scph**) and clock polarity (**SSSP_ctrlr0** field **scpol**) values. The data frame can be 4 to 16 bits in length.

When the configuration parameter **scph** = 0, indicating that the serial clock toggles in the middle of the first data bit, the data transmission begins on the falling edge of the slave select signal. The first data bit is captured by the master and slave peripherals on the first edge of the serial clock; therefore, valid data must be present on the TXD and RXD lines prior to the first serial clock edge.

Figure 20-2 shows a timing diagram for a single-frame SSI / SPI data transfer with **scph** = 0. The serial clock is shown for configuration parameters **scpol** = 0, indicating a low inactive state, and **scpol** = 1, indicating a high inactive state.

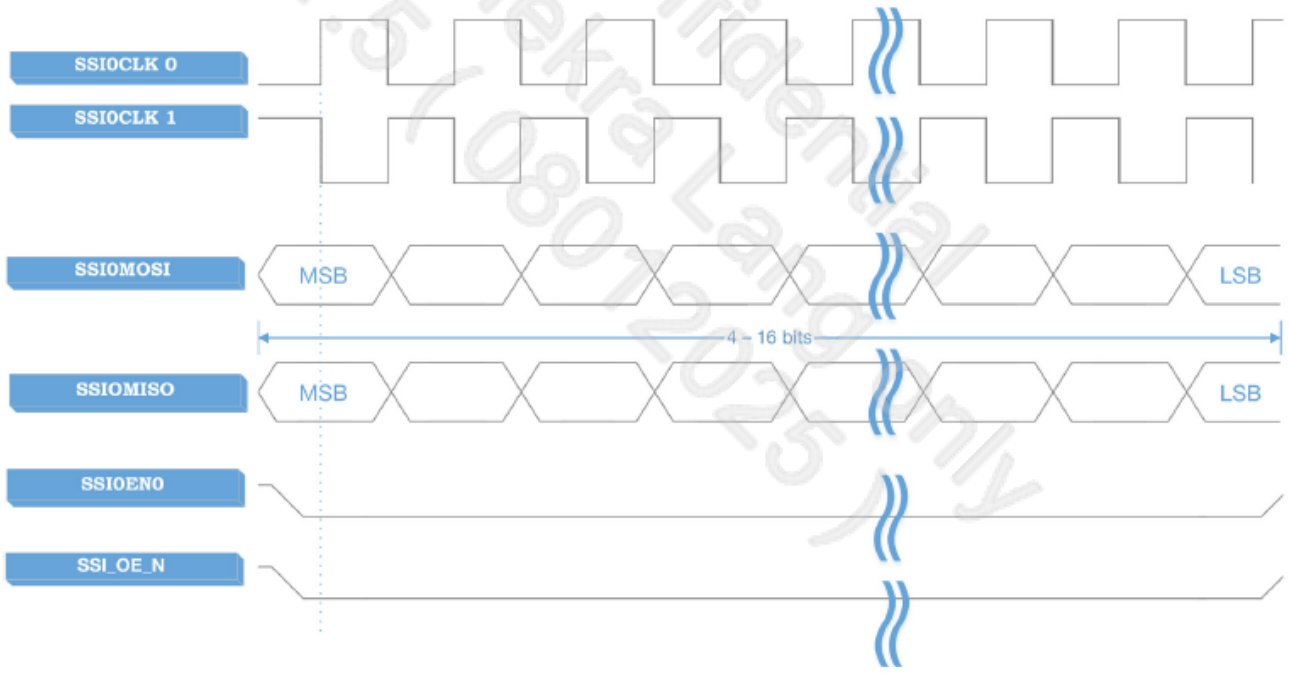


*Figure 20-2. SSI / SPI Single-Frame Transfer: Serial Clock Toggles in the Middle of First Data Bit (**scph** = 0).*

For continuous data transfers, the slave select signal is required to toggle before the start of the next data frame. This is illustrated in the following figure.

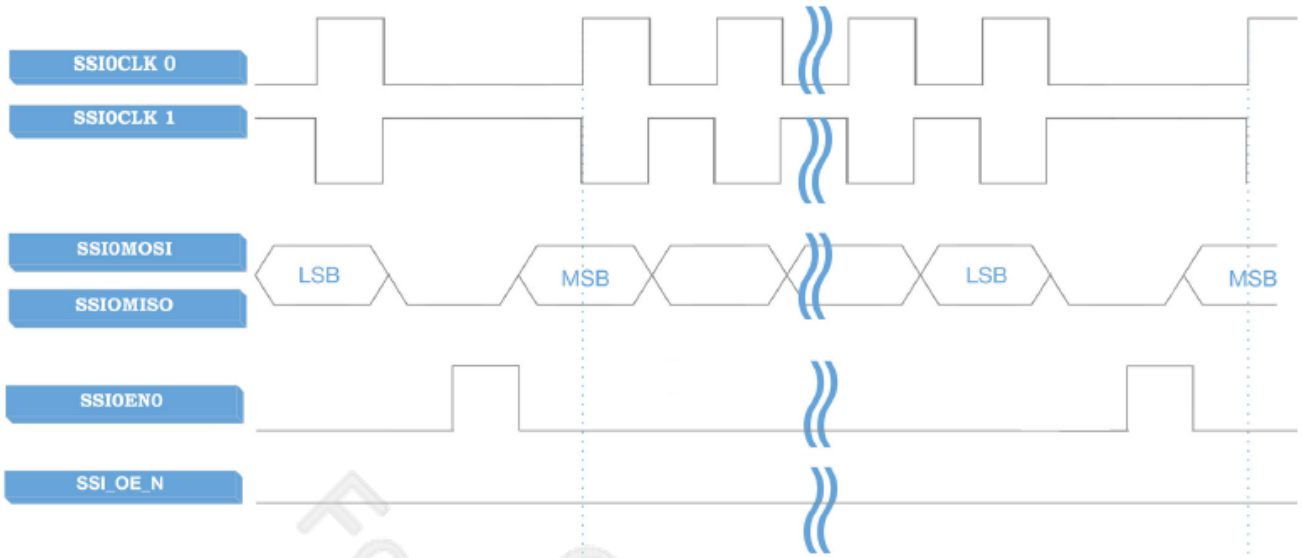


*Figure 20-3. SSI / SPI Continuous Transfer: Serial Clock Toggles in the Middle of First Data Bit (**scph** = 0).*

When the configuration parameter **scph** = 1, indicating that the serial clock toggles at the start of the first data bit, both master and slave peripherals begin transmitting data on the first serial clock edge after the slave select line is activated. The first data bit is captured on the second (trailing) serial clock edge. Data is propagated by the master and slave peripherals on the leading edge of the serial clock. The following figure shows a single-frame SSI / SPI data transfer with **scph** = 1.

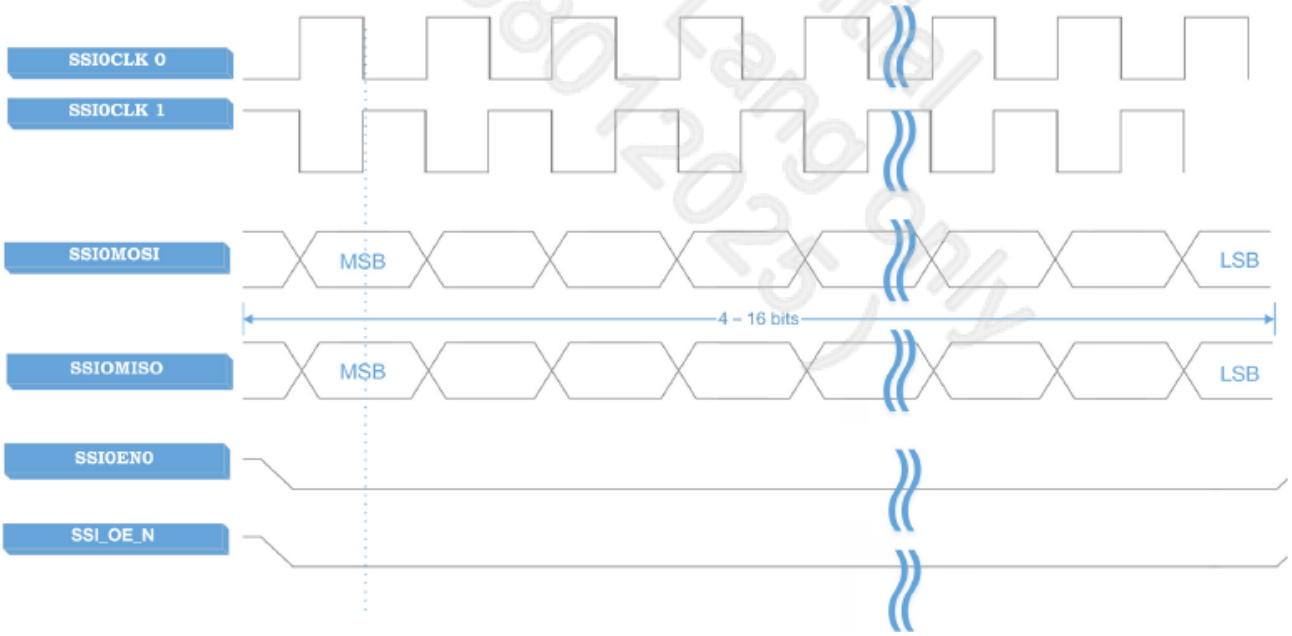


*Figure 20-4. SSI / SPI Single-Frame Transfer: Serial Clock Toggles at the Start of First Data Bit (**scph** = 1).*

Continuous data frames are transferred in the same way as single frames, with the most significant bit (MSB) of the next frame following directly after the least significant bit (LSB) of the current frame. The slave select signal is held active for the duration of the transfer.

The following figure shows the timing diagram for continuous SSI / SPI transfers when **scph** = 1.

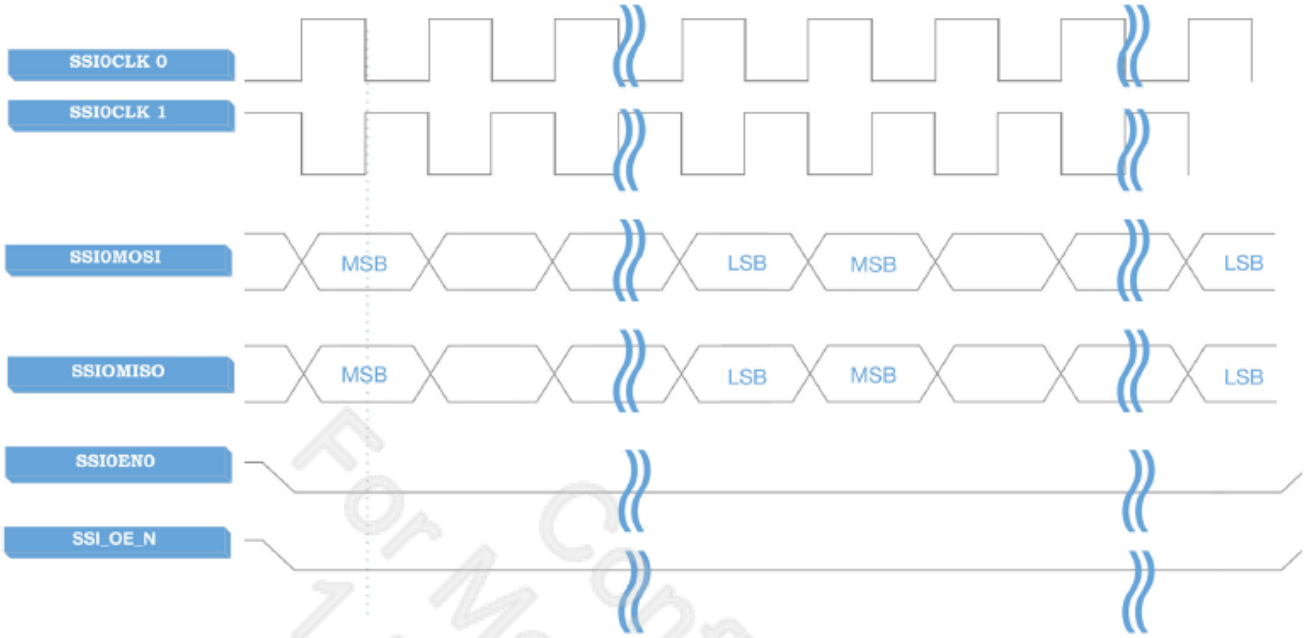


*Figure 20-5. SSI / SPI Continuous Transfer: Serial Clock Toggles at the Start of First Data Bit (**scph** = 1).*

The signals illustrated in the timing diagrams above include:

- **SSI0CLK**—serial clock from the SSI / SPI master:
  - **SSI0CLK 0** and **SSI0CLK 1** represent **scpol** = 0 and **scpol** = 1, respectively, with **scpol** = 0 indicating a low, inactive state, and **scpol** = 1 indicating a high, inactive state.
- **SSI0EN0**—slave select signal from the SSI / SPI master
- **SSI_OE_N**—output enable for the SSI / SPI master (internal hardware signal)
- **SSI0MOSI**—transmit data line for the SSI / SPI master
- **SSI0MISO**—receive data line for the SSI / SPI master

20.2.2 SSI / SPI Transmit and Receive FIFO Buffers

The FIFO buffers used by the SSI / SPI controllers are port register files with a depth of 32 entries (SSI0 ~ SSI5) and 32 entries (Dedicated Slave).

The width of the transmit and receive FIFO buffers is fixed at 16 bits. Data frames less than 16 bits must be right-justified when written into the transmit FIFO buffer. The shift-control logic automatically right-justifies receive data in the receive FIFO buffer. Each data entry in the FIFO buffers contains a single data frame. It is not possible to store multiple data frames in a single FIFO location.

Confidential For Mekra Lang only 1.5 (08/2023)

Note that there are three possible transfer modes on the SSI / SPI interface for performing serial transactions (see Section 20.2.4 “SSI / SPI Transfer Modes”):

1. Transmit and Receive transfer mode
2. Transmit Only transfer mode
3. Receive Only transfer mode

In Transmit and Receive transfer mode (**SSSP_ctrlr0** field **tmod**[9:8] = 0x0), data transmitted from the SSI / SPI to the external serial device is written into the transmit FIFO. Data received from the external serial device into the SSI / SPI is pushed into the receive FIFO.

The following figure shows the FIFO levels prior to a serial transfer and the FIFO levels on completion of the transfer. In this example, two data words are transmitted from the SSI / SPI to the external serial device in a continuous transfer. The external serial device is responding with two data words.

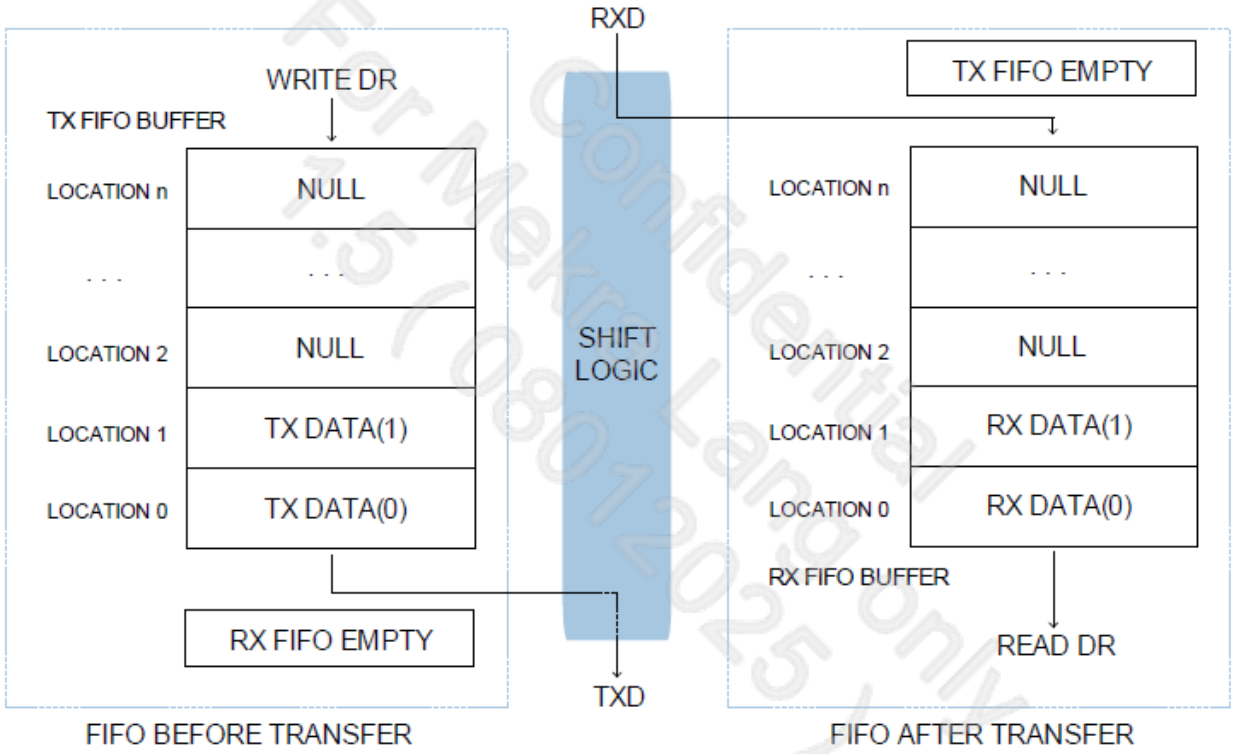


Figure 20-6. FIFO Status (Before and After Transfer) for Transmit and Receive Mode.

In Transmit Only transfer mode (**SSSP_ctrlr0** field **tmod**[9:8] = 0x1), data transmitted from the SSI / SPI to the external serial device is written into the transmit FIFO. As the data received from the external serial device is considered invalid, it is not stored in the SSI / SPI receive FIFO.

The following figure shows the FIFO levels prior to the beginning of a serial transfer and the FIFO levels on completion of the transfer. In this example, two data words are transmitted from the SSI / SPI to the external serial device in a continuous transfer.

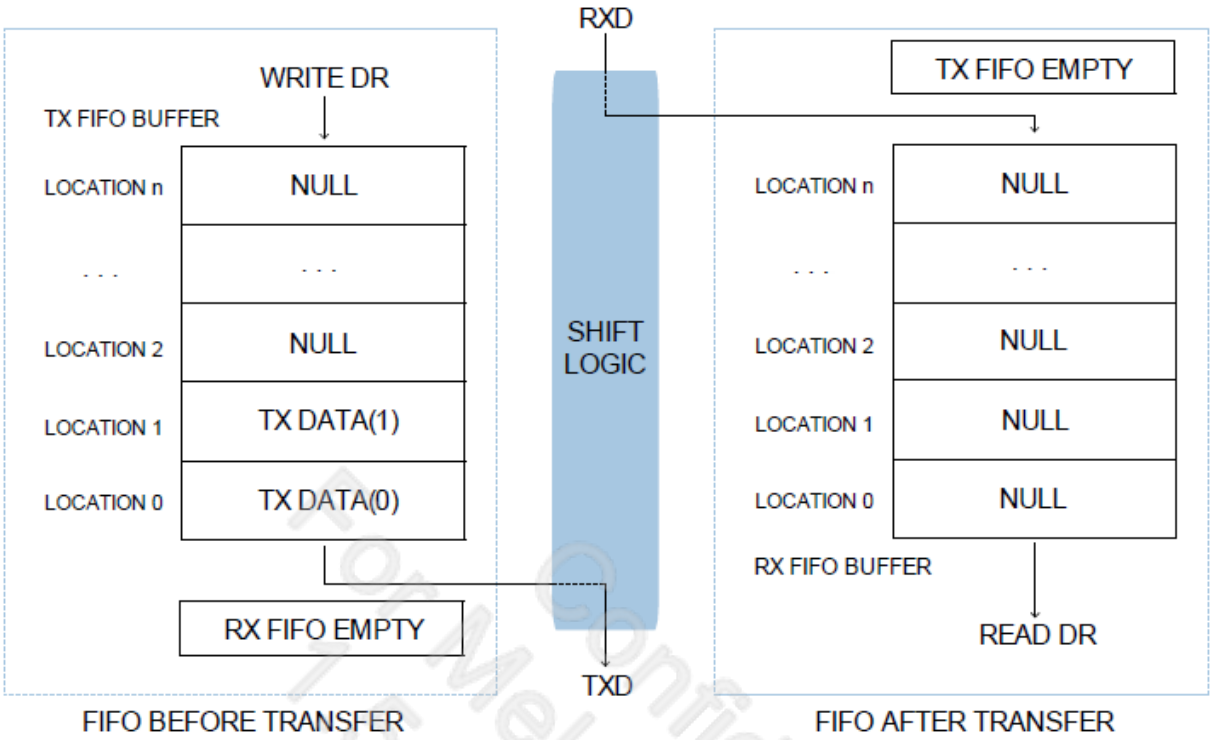


Figure 20-7. FIFO Status (Before and After Transfer) for Transmit Only Mode.

In Receive Only transfer mode (**SSSP_ctrlr0** field **tmod**[9:8] = 0x2), data transmitted from the SSI / SPI to the external serial device is invalid, so a single dummy word is written into the transmit FIFO to begin the serial transfer. This dummy word is re-transmitted by the SSI / SPI for the duration of the transfer. Data received from the external serial device into the SSI / SPI is pushed into the receive FIFO.

The following figure shows the FIFO levels prior to starting a serial transfer and the FIFO levels on completion of the transfer. In this example, two data words are received by the SSI / SPI from the external serial device in a continuous transfer.

Confidential For Wekalinx only (2015-05-25)

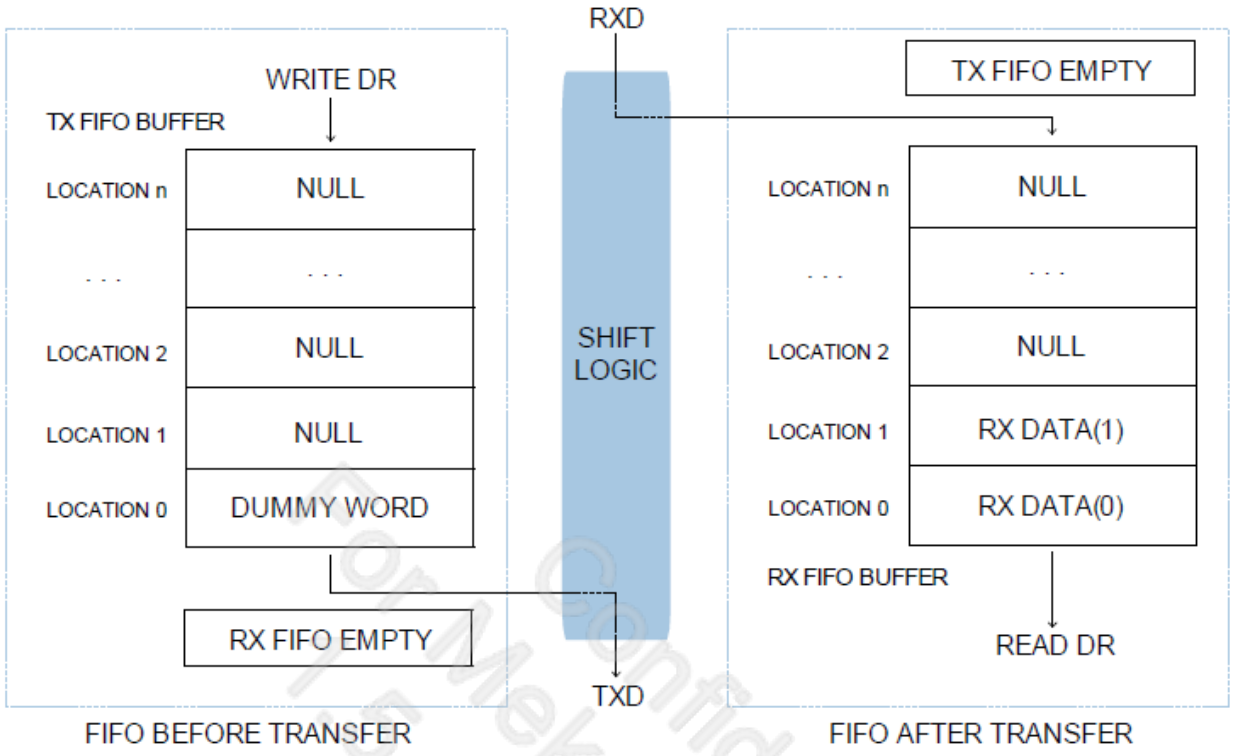


Figure 20-8. FIFO Status (Before and After Transfer) for Receive Only Mode.

Section 20.3 details the SSI / SPI registers. Here, the transmit FIFO is loaded by AHB write commands to the SSI / SPI data register (**SSSP_dr**). Data is popped (removed) from the transmit FIFO by shift-control logic into the transmit shift register. The transmit FIFO generates a FIFO empty interrupt request (SSI_TXE_INTR) when the number of entries in the FIFO is less-than or equal to the FIFO threshold value.

The threshold value, set through programmable Transmit FIFO Threshold Level register (**SSSP_txftlr**), determines the number of FIFO entries at which an interrupt is generated. The threshold value allows for sending an early indication to the processor that the transmit FIFO is nearly empty. A transmit FIFO overflow interrupt (SSI_TXO_INTR) is generated if there is an attempt to write data into an already full transmit FIFO.

Data is transferred from the receive FIFO by AHB read commands to the SSI / SPI data register (**SSSP_dr**). Transmit data traveling from the system bus to the SSI transmit FIFO must be word aligned. The receive FIFO is loaded from the receive shift register by shift-control logic. The receive FIFO generates a FIFO-full interrupt request (SSI_RXF_INTR) when the number of entries in the FIFO is greater-than or equal to the FIFO threshold value plus 1.

The threshold value, set through the programmable Receive FIFO Threshold Level register (**SSSP_rxftlr**), determines the number of FIFO entries at which an interrupt is generated. A receive FIFO overrun interrupt (SSI_RXO_INTR) is generated when the receive shift-logic attempts to load data into a completely full receive FIFO. However, this newly received data is lost. A receive FIFO underflow interrupt (SSI_RXU_INTR) is generated if there is an attempt to read from an empty receive FIFO; this alerts the processor that the read data is invalid.

20.2.3 SSI / SPI Interrupts

A number of events can trigger an SSI / SPI interrupt, and these individual interrupts are combined into one interrupt request to the Generic Interrupt Controller (GIC). Each individual interrupt request can be masked. The combined interrupt request is the OR'ed result of all other SPI interrupts after masking. The SSI / SPI interrupts are described as follows:

- Transfer Done Interrupt (master only):
  - In TX mode, if **spi_hold** is set , the interrupt asserts when number of transferred data reaches **SSSP_tx_frame_count** plus 1. Otherwise, the interrupt asserts when TX FIFO is empty.
  - In RX Only mode, the interrupt asserts when number of received data reaches **SSSP_ctrlr1** (ndf) plus 1.
- Frame Counter Reach Interrupt (master only): If **fc_en** is set, this interrupt asserts when the number of transmitted or received data frame counter reaches the **SSSP_target_counter** register.
- Transmit FIFO Empty Interrupt (SSI_TXE_INTR): Set when the transmit FIFO is equal to or below its threshold value and requires service to prevent an under-run. The threshold value, set through a software-programmable register, determines the number of transmit FIFO entries at which an interrupt is generated. This interrupt is cleared by hardware when data is written into the transmit FIFO buffer, bringing it over the threshold level.
- Transmit FIFO Overflow Interrupt (SSI_TXO_INTR): Set when an AHB access attempts to write into the transmit FIFO after it has been completely filled. When set, data written from the AHB is discarded. This interrupt remains set until the software reads the Transmit FIFO Overflow Interrupt Clear register (**SSSP_txoicr**).
- Receive FIFO Full Interrupt (SSI_RXF_INTR): Set when the receive FIFO is equal-to or above its threshold value plus 1 and requires service to prevent an overflow. The threshold value, set through a software-programmable register, determines the number of receive FIFO entries at which an interrupt is generated. This interrupt is cleared by hardware when data is read from the receive FIFO buffer, bringing it below the threshold level.
- Receive FIFO Overflow Interrupt (SSI_RXO_INTR): Set when the receive logic attempts to place data into the receive FIFO after it has been completely filled. When set, newly received data is discarded. This interrupt remains set until the software reads the Receive FIFO Overflow Interrupt Clear register (**SSSP_rxoicr**).
- Receive FIFO Underflow Interrupt (SSI_RXU_INTR): Set when an AHB access attempts to read from the receive FIFO when it is empty. When set, zeros are read back from the receive FIFO. This interrupt remains set until the software reads the Receive FIFO Underflow Interrupt Clear register (**SSSP_rxuicr**).
- Combined Interrupt Request (SSI_INTR): This interrupt is the OR'ed result of all the above interrupt requests after masking. To mask this interrupt signal, the software must mask all other SPI interrupt requests.

20.2.4 SSI / SPI Transfer Modes

The SSI / SPI operates in the following three modes when transferring data on the serial bus:

- (Section 20.2.4.1) SSI / SPI Transmit and Receive Transfer Mode
- (Section 20.2.4.2) SSI / SPI Transmit Only Transfer Mode
- (Section 20.2.4.3) SSI / SPI Receive Only Transfer Mode

The transfer mode is set by writing to the Control register 0 (**SSSP_ctrlr0**) transfer mode field (**tmod**), as described in Section 20.3.2.1.

20.2.4.1 SSI / SPI Transmit and Receive Transfer Mode

When the **SSSP_ctrlr0** transfer mode field (**tmod**) = 0x0 (Section 20.3.2.1), the SSI / SPI interface transfers data in Transmit and Receive mode. In this mode, both transmit and receive logic are valid, and the data transfer occurs as normal according to the selected frame format. Transmit data is popped (removed) from the transmit FIFO and sent through the TXD line to the target device, which replies with data on the RXD line. The receive data from the target device is moved from the receive shift register into the receive FIFO at the end of each data frame.

20.2.4.2 SSI / SPI Transmit Only Transfer Mode

When the **SSSP_ctrlr0** transfer mode field (**tmod**) = 0x1 (Section 20.3.2.1), the SSI / SPI interface transfers data in Transmit Only mode. In this mode, all receive data is invalid and should not be stored in the receive FIFO. Otherwise, the data transfer occurs as normal, according to the selected frame format (serial protocol). Transmit data is popped from the transmit FIFO and sent through the TXD line to the target device, which replies with data on the RXD line. At the end of the data frame, the receive shift register does not load its newly received data into the receive FIFO. Instead, the data in the receive shift register is overwritten by the next transfer. Masking interrupts originating from the receive logic is required when this mode is used.

20.2.4.3 SSI / SPI Receive Only Transfer Mode

When the **SSSP_ctrlr0** transfer mode field (**tmod**) = 0x2 (Section 20.3.2.1), the SSI / SPI interface transfers data in Receive Only mode. In this mode, all transmit data is invalid. The receive data from the target device is moved from the receive shift register into the receive FIFO at the end of each data frame. Masking interrupts originating from the transmit logic is required when this mode is used.

20.2.5 SSI / SPI Operational Modes

An SSI / SPI bus is capable of maintaining both serial masters and serial slaves, each of which can be leveraged differently under various operational modes. This section details the operational modes of the SSI / SPI interface.

- (Section 20.2.5.1) Operational Modes: SSI / SPI Serial-Master
- (Section 20.2.5.2) Operational Modes: Master SPI Serial Transfers
- (Section 20.2.5.3) Operational Modes: Slave SPI Serial Transfers

20.2.5.1 Operational Modes: SSI / SPI Serial-Master

This mode enables serial communication with serial-slave peripheral devices. As a serial-master device, the SSI / SPI module initiates and controls all serial transfers under this mode. Refer to Figure 20-1 for an illustration of this relationship.

The serial bit-rate clock, generated and controlled by the SSI / SPI interface, is driven out on the **SSIOCLK** pin. When the SSI / SPI interface is disabled (**SSSP_ssienr** bit **ssi_en** = 0), no serial transfers can occur and **SSIOCLK** is held in an inactive state.

Data transfers are initiated by the serial-master device, assuming there is at least one valid data entry in the transmit FIFO and a serial-slave device is selected. When actively transferring data, the Status register (**SSSP_sr**) busy flag (**busy**) is set. Software must wait until the busy flag is cleared before attempting a new serial transfer.

Note that the **busy** status (**SSSP_sr**) is not set when data is first written into the transmit FIFO. This bit is set only when the target slave has been selected and the transfer is underway. After writing data into the transmit FIFO, the shift-control logic does not initiate the serial transfer until a positive edge of the **SSIOCLK** signal is present. The delay in waiting for this positive edge depends on the baud rate of the serial transfer. Prior to polling the **busy** status, the software first should poll the Status register (**SSSP_sr**) status flag **txe** (wait for logic 1).

20.2.5.2 Operational Modes: Master SPI Serial Transfers

When the **SSSP_ctrlr0** register field transfer mode is set to Transmit and Receive or Transmit Only (**tmod** = 0x0 or **tmod** = 0x1, respectively), transfers are terminated by the shift-control logic when the transmit FIFO is empty (Section 20.3.2.1). For continuous data transfers, it is necessary to ensure that the transmit FIFO buffer does not become empty before all data has been transmitted. The transmit FIFO threshold level (**SSSP_txftlr**) can be used to prompt an early interrupt (SSI_TXE_INTR) to the processor indicating that the transmit FIFO buffer is nearly empty.

When the transfer mode is Receive Only (**tmod** = 0x2), a serial transfer is initiated by writing one “dummy” data word into the transmit FIFO when a serial slave is selected. If the serial transfer is continuous, this same data word is retransmitted until the serial transfer is completed. The transmit FIFO is popped only once at the beginning and may remain empty for the duration of the serial transfer. The end of the serial transfer is managed by using the Control register 1 (**SSSP_ctrlr1),** Number of Frames (**ndf**) field. The receive-logic will terminate the serial transfer when the number of frames received is equal to the **ndf** value + 1.

For example, to receive 14 data frames from a serial-slave peripheral, the value 13 would be entered into the **ndf** field. This transfer mode increases the bandwidth of the AHB bus as the transmit FIFO never needs to be serviced during the transfer process. However, the receive FIFO buffer must be read each time the receive FIFO generates a FIFO-full interrupt request to prevent an overflow. The receive FIFO threshold level (**SSSP_rxftlr**) reg-

ister can be used to provide early indication that the receive FIFO is nearly full.

A typical programming flow for completing a serial transfer from the SSI / SPI serial master is outlined as follows:

1. If the SSI / SPI interface is enabled, disable it by writing 0 to the SSI Enable register (**SSSP_ssienr**) bit **ssi_en**.
2. Disable safety error output by writing '1' to the fault injection register (FAULT_INJECT) bit-1.
3. Set up the SSI / SPI control registers for the transfer. These registers can be set in any order.
   - Write Control register 0 (**SSSP_ctrlr0**). For SSI / SPI transfers, the serial clock polarity and serial clock phase parameters must be identical to those of the target slave device.
   - If the transfer mode is Receive Only, write Control register 1 (**SSSP_ctrlr1**) with the number of frames in the transfer minus 1, as described above.
   - Write the Baud Rate Select register (**SSSP_baudr**) to set the baud rate for the transfer.
   - Write the Transmit and Receive FIFO Threshold Level registers (**SSSP_txftlr** and **SSSP_rxftlr**, respectively) to set FIFO threshold levels.
   - Write the **SSSP_imr** register to set up interrupt masks. In DMA mode, bit-0 (**txeim**) and bit-4 (**rxfim**) of this register must be set to 0.
   - The Slave Enable register (**SSSP_ser**) can be written to enable the target slave for selection. If a slave is enabled here, the transfer begins as soon as one valid data entry is present in the transmit FIFO. If no slaves are enabled prior to writing to the Data register (**SSSP_dr**), the transfer does not begin until a slave is enabled.

   Note:

   While transferring data, FIFO underflow can occur if the TX data cannot be quickly filled into FIFO. To prevent this occurrence:

   Set 0x2 in address 0x4C (**tdmae**) to the arrow filling TX data into FIFO before **ssi_en** is set.

   The workaround is applicable in both DMA or non-DMA modes.
4. Enable safety error output by writing '0' to the fault injection register (FAULT_INJECT) bit-1.
5. Enable the SSI / SPI interface by writing 1 to **ssi_en**.
6. Write the data to be transmitted into the transmit FIFO (write **SSSP_dr**). If no slaves have been enabled in the **SSSP_ser** register, they must be enabled at this stage to initiate the transfer.
7. In the Status Register (**SSSP_sr**) poll the **busy** status to wait for completion of the transfer. The **busy** status cannot be polled immediately. If a transmit FIFO-empty interrupt request is made, write the transmit FIFO (write **SSSP_dr**) register. If a receive FIFO-full interrupt request is made, read the receive FIFO (read **SSSP_dr**) register.
8. The transfer is terminated by the shift-control logic when the transmit FIFO is empty. If the transfer mode is Receive Only (**SSSP_ctrlr0** bit **tmod** = 0x2), the transfer is terminated by the shift-control logic when the specified number of frames have been received. When the transfer is completed, the **SSSP_sr busy** status is reset to 0.
9. If the transfer mode is not Transmit Only, read the receive FIFO until it is empty.
10. Disable the SSI / SPI interface by writing 0 to **ssi_en**.

Ambarella Confidential (08/02/2021)

20.2.5.3 Operational Modes: Slave SPI Serial Transfers

When an SSI / SPI slave transmits data, it is necessary to ensure that data exists in the transmit FIFO before a transfer is initiated by a serial-master device. If a master initiates a transfer to the SSI / SPI slave when no data exists in the transmit FIFO, an error flag (TXE) is set in **SSSP_sr** and one byte of 0x00 or 0x80 is transmitted (if the data frame size is 8 bits). More precisely, the MSB of the data is the same as the LSB of the previous data, and all other bits are 0. Then, 0x0 is transmitted successively.

Because TXE toggling is not allowed, do not let TX FIFO empty if a transaction is not finished. The transmit FIFO threshold level register (**SSSP_txftlr**) can be used to prompt an early interrupt (SSI_TXE_INTR) to the processor, indicating that the transmit FIFO buffer is nearly empty.

When a DMA controller is used, an early request (DMA_TX_REQ) may be sent to the DMA controller to indicate that the transmit FIFO is nearly empty. The FIFO can then be refilled with data to continue the serial transfer. Note that when the **SSSP_ctrlr0** transfer mode is set to Receive Only (**tmod** = 0x2), the transmit FIFO need not contain valid data as the transmit shift register resets each time the slave device is selected.

The receive FIFO buffer should be read each time the receive FIFO generates a FIFO-full interrupt request to prevent an overflow. The receive FIFO threshold level register (**SSSP_rxftlr**) can be used to provide an early indication that the receive FIFO is nearly full. When a DMA controller is used, an early request (DMA_RX_REQ) may be sent to the DMA controller to indicate that the receive FIFO is nearly full.

A typical programming flow for completing a continuous serial transfer from a serial slave is described as follows:

1. If the SSI / SPI interface is enabled, disable it by writing 0 to the SSI Enable register (**SSSP_ssienr**) bit **ssi_en**.
2. Disable safety error output by writing '1' to the fault injection register (FAULT_INJECT) bit-1.
3. Set up the control registers for the transfer. These registers can be set in any order.
   - Write **SSSP_ctrlr0** (set **scph** and **scpol** identical to the master device).
   - Write **SSSP_txftlr** and **SSSP_rxftlr** to set FIFO threshold levels.
   - Write the **SSSP_imr** register to set up the interrupt masks. In DMA mode, bit-0 (**txeim**) and bit-4 (**rxfim**) of this register must be set to 0.

   Note:

   While transferring data, FIFO underflow can occur if the TX data cannot be quickly filled into FIFO. To prevent this occurrence:

   Set 0x2 in address 0x4C (**tdmae**) to the arrow filling TX data into FIFO before **ssi_en** is set.

   The workaround is applicable in both DMA or non-DMA modes.
4. Enable safety error output by writing '0' to the fault injection register (FAULT_INJECT) bit-1.
5. Enable the SSI / SPI interface by writing 1 to **ssi_en**.
6. If the transfer mode is Transmit and Receive (**tmod** = 0x0) or Transmit Only (**tmod** = 0x1), write the data to be transmitted into the transmit FIFO (write **SSSP_dr**). If the transfer mode is Receive Only (**tmod** = 0x2), there is no need to write data into the transmit FIFO, and the current value in the transmit shift register is retransmitted.
7. The SSI / SPI slave is now ready for the serial transfer. The transfer is initiated when the slave is selected by a serial-master device.
8. When the transfer is underway, poll the **busy** status (**SSSP_sr**) register to return the transfer status. If a transmit FIFO-empty interrupt request is made, write the transmit FIFO (write **SSSP_dr**) regis-

ter. If a receive FIFO-full interrupt request is made, read the receive FIFO (read **SSSP_dr**) register.

9. The transfer is terminated when the serial master removes the select input to the SSI / SPI slave. When the transfer is completed, the **busy** status resets to 0.
10. If the transfer mode is not Transmit Only, read the receive FIFO until it is empty.
11. Disable the SSI / SPI interface by writing 0 to **ssi_en**.

20.2.6 SSI / SPI Super Mode

This mode can be only used in SPI operation. Write Control Register 0 (CTRL0) bit-29 to use this mode.

In super mode, the transfer mode register (TMOD) should be set as 2'b00 or 2'b01 (with transmit scheme) and serial clock phase (SCPH) must set to 1'b1. Each transfer data in transmit FIFO buffer consists of a length, which describes the number of data frames of the transfer, and the data frames (see the following figure).

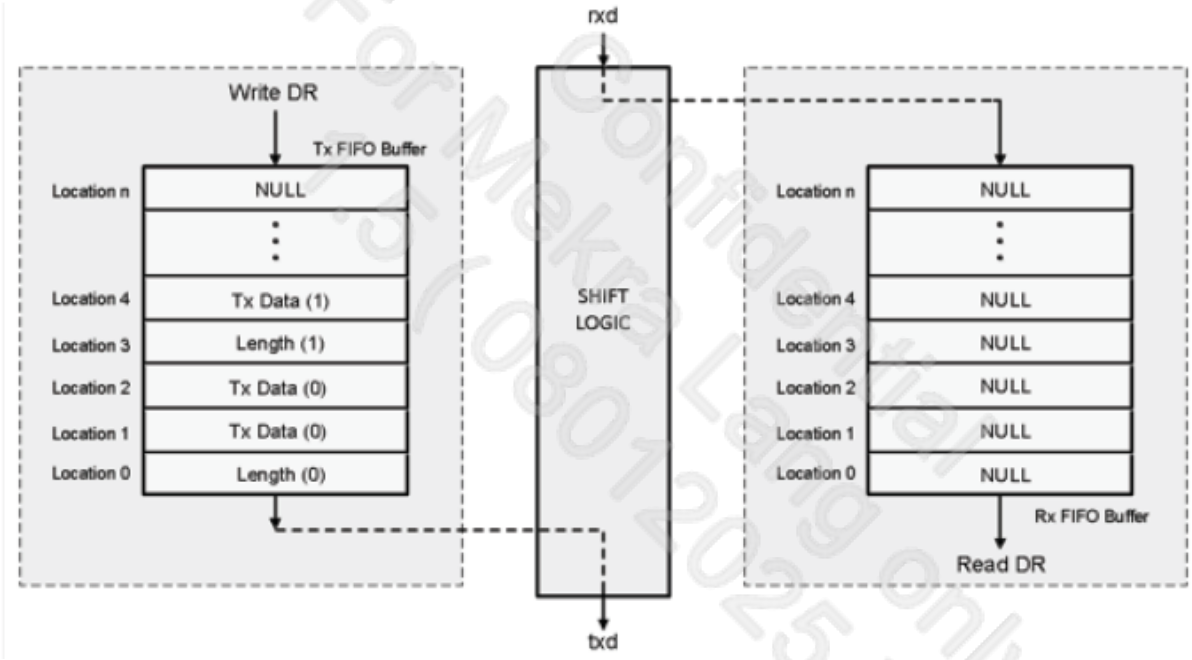


Figure 20-9. FIFO Status for Super Mode.

The following describes the figure above.

- If spi_hold = 1'b1:
  - SSI will keep transferring data until the transmit data frame number reaches the length.
- If spi_hold = 1'b0:
  - SSI waits until the data frames pushed into transmit FIFO buffer reaches the length, and then begins to transfer data. In this case, if the length is larger than FIFO's depth, the transfer begins when FIFO is full. Furthermore, the remaining data frames should be pushed into the FIFO buffer to reach the data frame length before FIFO is empty. Otherwise, a fatal error will occur.

Notes:

- If using byte wise access mode, the length must be less than or equal to 8 bits—typically,16 bits.

For Mekra Lang only (2501080 / 2025)
Confidential

- If **SSSP_ctrlr0** bit-16 (**transfer_doneie**) is set, the transfer done interrupt in super mode toggles when the master receives length=0xFF in byte wise access mode or length=0xFFFF in non-byte wise access mode from transmit FIFO after finishing a transaction.
- The software can read the transmit FIFO level (TXTFL) at anytime to determine if there are quotas requiring a push of data into the FIFO buffer.

20.3 SSI / SPI: Registers

The registers below are used for all SSI / SPI instances. Refer to Section 20.1.2 for bus and base address information.

20.3.1 SSI / SPI Registers: Map

Offset	Register	Description
0x000	**SSSP_ctrlr0**	Control register 0
0x004	**SSSP_ctrlr1**	Control register 1
0x008	**SSSP_ssienr**	SSI enable
0x00C		Reserved
0x010	**SSSP_ser**	Slave enable
0x014	**SSSP_baudr**	Baud rate select
0x018	**SSSP_txftlr**	Transmit FIFO threshold level
0x01C	**SSSP_rxftlr**	Receive FIFO threshold level
0x020	**SSSP_txflr**	Transmit FIFO level
0x024	**SSSP_rxflr**	Receive FIFO level
0x028	**SSSP_sr**	Status register
0x02C	**SSSP_imr**	Interrupt mask
0x030	**SSSP_isr**	Interrupt status
0x034	**SSSP_risr**	Raw Interrupt status
0x038	**SSSP_txoicr**	Transmit FIFO overflow interrupt clear
0x03C	**SSSP_rxoicr**	Receive FIFO overflow interrupt clear
0x040	**SSSP_rxuicr**	Receive FIFO underflow interrupt clear
0x044		Reserved
0x048	**SSSP_icr**	Interrupt clear
0x04C	**SSSP_dmacr**	DMA control register
0x050	**fault_inject**	Fault injection
0x054		Reserved
0x058	**SSSP_idr**	Identification
0x05C	**SSSP_version**	Version ID
0x060 - 0x25C	**SSSP_dr**	Data
0x260	**SSSP_ssienpolr**	SSI slave select signal polarity
0x264	**SSSP_sclk_out_dly**	**SSIOCLK** delay counter
0x268 - 0x278		Reserved
0x27C	**target_frame_count**	Target frame count for interrupt
0x280	**frame_count**	Current frame count
0x284	**fcricr**	Frame counter reaches interrupt
0x288	**tx_frame_count**	Total TX frame count

For Mekra Confidential 1.5 (080118) Only

Offset	Register	Description
0x28C	**transfer_doneicr**	Transfer done interrupt clear

Table 20-3. SSI / SPI Register Map.

20.3.2 SSI / SPI Registers: Detail

20.3.2.1 SSSP_ctrlr0 Register

This register controls the serial data transfer. It is not possible to write to this register when the SSI / SPI interface is enabled. The SSI / SPI interface can be enabled and disabled by writing to the register **SSSP_ssienr**.

Bits	Name	Access	Reset	Description
31:30				Reserved
29	**super_mode**	RW	0	SSI continuously transmits data if this register is set. It is only used in SPI mode (frf=2'b00)
28	**irq_mode**	RW	0	IRQ clear mechanism. 0 Read clear (Legacy scheme). 1 Write clear (Introduced at CV22FS) Write to the corresponding fields of the **SSSP_risr** register to clear interrupts.
27	**spi_hold**	RW	0	When this bit is set, SSI / SPI mode state machine will hold if TX FIFO frame does not reach the target frame count.
26	**byte_wise_access_mode**	RW	0	If DFS <= 8 bits, enable this register to double FIFO's length. It cannot enable while DFS > 8 bits.
25:22	**extra_rxd_margin**	RW	0	Round chip delay compensation mode. Bit-22 to Bit-24: delay data capture edge from 0 to 7 internal SPI clock cycles. Bit-25: delay data push time. Set this bit to 1 when the data capture edge delay setting is equal to sckdv/2. The maximum setting for data capture edge delay is sckdv/2.
21	**fc_en**	RW	0	Frame counter enable. 0 Disable frame counter if fcr_int is not in use to save power 1 Enable frame counter
20	**pack_mode**	RW	0	Used for CPU or DMA to burst read data in byte or half-word access in order to save the memory space. (No matter what the address offset is.)
19	**rx_lfb_first**	RW	0	RX LSB First mode. Used to indicate that the LSB bit of packets in RX FIFO was received first. 0 MSB first 1 LSB first
18	**tx_lfb_first**	RW	0	TX LSB First mode. Used to indicate that the LSB bit of packets in TX FIFO will be sent first. 0 MSB first 1 LSB first

Bits	Name	Access	Reset	Description
17	**residue_flush_mode**	RW	0	Residue flush mode. Used to assert RX DMA request when the transfer completes and the frame count in RX FIFO is less than the RX FIFO threshold. This function prevents residue frames in RX FIFO and CPU intervention. 0 Disable 1 Enable
16	**transfer_doneie**	RW	0	Transfer done interrupt enable. Used in super mode to enable transfer done interrupt. • **Byte wise access mode:** If this bit is set to 1, length=0xFF will be treated as interrupt request. The data sequence in TX FIFO should be length → data → length=0xFF. Otherwise, length = 0xFF represents that there are 256 data in next transaction. • **Non-byte wise access mode:** If this bit is set to 1, length=0xFFFF will be treated as interrupt request. The data sequence in TX FIFO should be length → data →length=0xFFFF 0 Disable 1 Enable
15:12				Reserved
11	**srl**	RW	0	Shift Register Loop Used for testing purposes only. When active internally, this bit connects the transmit shift register output to the receive shift register input. 0 Normal Mode Operation 1 Test Mode Operation
10	**slv_oe**	RW	0	Slave Output Enable Relevant only when the AHB SSI is configured as a serial-slave device. When configured as a serial master, this bit field has no functionality. This bit enables or disables the setting of the SSI_OE_N output from the AHB SSI serial slave. When **slv_oe** = 1, the SSI_OE_N output can never be active. When the SSI_OE_N output controls the tri-state buffer on the TXD output from the slave, a high impedance state is always present on the slave TXD output when **slv_oe** = 1. This is useful when the master transmits in broadcast mode (master transmits data to all slave devices). Only one slave may respond with data on the master RXD line. This bit is enabled after reset and must be disabled by software (when broadcast mode is used), if the device is not to respond with data. 0 Slave TXD is enabled 1 Slave TXD is disabled

For Mekra Lang Only
Confidential
1.5 (Oct 2018)

Bits	Name	Access	Reset	Description
9:8	**tmod**	RW	0	Transfer Mode Used to select the mode of transfer for serial communication. This field does not affect the transfer's duplicity. Only indicates whether the receive/transmit data is valid. In Transmit Only mode, data received from the external device is not valid and is not stored into the receive FIFO memory. It is overwritten on the next transfer. In Receive Only mode, transmitted data is not valid. After the first write to the transmit FIFO, the same word is retransmitted for the duration of the transfer. In Transmit and Receive mode, both transmit and receive data are valid. The transfer continues until the transmit FIFO is empty. Data received from the external device is stored into the receive FIFO memory, where it can be accessed by the host processor. 0x0 Transmit & Receive 0x1 Transmit Only 0x2 Receive Only 0x3 Reserved
7	**scpol**	RW	0	Serial Clock Polarity Valid when the frame format (**frf**) is set to Motorola SPI. Used to select the polarity of the inactive serial clock. The serial clock is held inactive when the SSI / SPI master is not actively transferring data on the serial bus. 0 Inactive state of serial clock is low 1 Inactive state of serial clock is high
6	**scph**	RW	0	Serial Clock Phase Valid when the frame format (**frf**) is set to Motorola SPI. The serial clock phase selects the relationship of the serial clock with the slave select signal. When **scph** = 0, data is captured on the first edge of the serial clock. When **scph** = 1, the serial clock starts toggling one cycle after the slave select line is activated, and data is captured on the second edge of the serial clock. 0 Serial clock toggles in middle of first data bit 1 Serial clock toggles at start of first data bit
5:4	**frf**	RW	0	Frame Format Used to select which serial protocol transfers the data. 0x0 Motorola SPI Others Reserved
3:0	**dfs**	RW	7	Data Frame Size (DFS) Used to select the data frame length. When the data frame size is programmed to be less than 16 bits, the receive data is automatically right-justified by the receive logic, with the upper bits of the receive FIFO zero-padded. Right-justify transmit data before writing into the transmit FIFO. The transmit logic ignores the upper unused bits when transmitting the data. Refer to Table 20-5 for DFS Decode.

Table 20-4. SPI Control0 Register.

For Marketing Use Only
1.5 (08/23/2013)

DFS Value	Description
0x0	Reserved - Undefined Operation
0x1	Reserved - Undefined Operation
0x2	Reserved - Undefined Operation
0x3	4-bit serial data transfer
0x4	5-bit serial data transfer
0x5	6-bit serial data transfer
0x6	7-bit serial data transfer
0x7	8-bit serial data transfer
0x8	9-bit serial data transfer
0x9	10-bit serial data transfer
0xA	11-bit serial data transfer
0xB	12-bit serial data transfer
0xC	13-bit serial data transfer
0xD	14-bit serial data transfer
0xE	15-bit serial data transfer
0xF	16-bit serial data transfer

Table 20-5. DFS Decode.

Ambarella Confidential
1.5 (08/12/2025)
Mekra Lang only

20.3.2.2 SSSP_ctrlr1 Register

This register exists only when the SSI / SPI is configured as a master device. When the SSI / SPI is configured as a serial slave, writing to this location has no effect; reading from this location returns zero.

Control register 1 controls the end of serial transfers when in Receive Only mode. It is not possible to write to this register when the SSI / SPI interface is enabled. The SSI / SPI interface is enabled and disabled by writing to the **SSSP_ssienr** register.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**ndf**	RW	0	Number of Data Frames When **SSSP_ctrlr0** bit **tmod** = 10, this register field sets the number of data frames to be received continuously by the SSI / SPI. The SSI / SPI continues to receive serial data until the number of data frames received is equal to this register value plus 1. This enables receipt of up to 64 KB of data in a continuous transfer.

Table 20-6. SPI Control1 Register.

20.3.2.3 SSSP_ssienr Register

This register enables and disables the SSI / SPI interface.

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**ssi_en**	RW	0	SSI Enable Used to enable and disable all SPI operations. When disabled, all serial transfers are halted immediately. It is not possible to program some of the SSI / SPI control registers when enabled. FIFO buffers are cleared when enable goes from 1 to 0, but not when enable transitions from 0 to 1. In the software flow sequence, **ssi_en** must be programmed last.

Table 20-7. SSI / SPI Enable Register.

20.3.2.4 SSSP_ser Register

This register enables the individual slave select output lines from the SSI / SPI master. It is not possible to write to this register when SPI is **busy** (**SSSP_sr**) and when **ssi_en** = 1 (**SSSP_ssienr**).

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**ser**	RW	0	Slave Select Enable Flag Each bit in this register corresponds to a slave select line from the SSI / SPI master. When a bit in this register is set (logic 1), the corresponding slave select line from the master is activated when a serial transfer begins. **Note**: Setting/clearing bits in this register has no effect on the corresponding slave select outputs until a transfer is initiated. Before beginning a transfer, enable the bit in this register that corresponds to the slave device with which the master wants to communicate. When not operating in broadcast mode, only one bit in this field should be set. 0 Not Selected 1 Selected

Table 20-8. SPI Slave Enable Register.

20.3.2.5 SSSP_baudr Register

This register is valid for the master device only. The register derives the frequency of the serial clock that regulates the data transfer. The 16-bit field in this register defines the **SSIOCLK** divider value. It is not possible to write to this register when the SSI / SPI interface is enabled. The SSI / SPI interface is enabled and disabled by writing to the **SSSP_ssienr** register.

Bits	Name	Access	Reset	Description
31:16				Reserved

Bits	Name	Access	Reset	Description
15:0	**sckdv**	RW	0	SSI Clock Divider (Section 20.2.1) The LSB for this field is always set to 0 and is unaffected by a write operation. This ensures an even value is held in this register. If the value is zero, the serial output clock (**SSI0CLK**) is disabled. The frequency of **SSI0CLK** is derived from CLK_IN with the following equation: Freq(**SSI0CLK**) = Freq(CLK_IN) / **sckdv** where **sckdv** is any even value between 0 and 65534. Where CLK_IN represents GCLK_SSI or GCLK_SSI2 from the RCT module and is controlled by the SSI clock generator in the RCT module. For Freq(CLK_IN) = 3.6864 MHz and **sckdv** =2 the calculation is: Freq(**SSI0CLK**) = 3.6864/2 = 1.8432 MHz

Table 20-9. Baud Rate Select Register.

20.3.2.6 SSSP_txftlr Register

This register controls the threshold value for the transmit FIFO memory. It is not possible to write to this register when the SSI / SPI interface is enabled. The SSI / SPI interface is enabled and disabled by writing to the **SSSP_ssienr** register. TFT values differ between the Master and Slave controllers as described in the tables below.

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	tft	RW	0	Transmit FIFO Threshold Used to control the level of entries (or below) at which the transmit FIFO controller triggers an interrupt. The FIFO depth is 32 or 64 entries depending on the frame size. Attempts to set this value greater than or equal to the depth of the FIFO, do not write to this field and it retains its current value. When the number of transmit FIFO entries is less than or equal to this value, the transmit FIFO empty interrupt is triggered.

Table 20-10. SPI Transmit FIFO Threshold Level Register.

TFT value	Description
0x00	SSI_TXE_INTR is asserted when 0 data entries are present in transmit FIFO
0x01	SSI_TXE_INTR is asserted when 1 data entries are present in transmit FIFO
0x02	SSI_TXE_INTR is asserted when 2 data entries are present in transmit FIFO
...	
0x3F	SSI_TXE_INTR is asserted when 64 data entries are present in transmit FIFO

*Table 20-11. **SSSP_txftlr** Register TFT Decode for the SSI / SPI Master Controller.*

TFT value	Description
0x00	SSI_TXE_INTR is asserted when 0 data entries are present in transmit FIFO
0x01	SSI_TXE_INTR is asserted when 1 data entries are present in transmit FIFO
0x02	SSI_TXE_INTR is asserted when 2 data entries are present in transmit FIFO
...	

For Internal Use Only Confidential (2015)

TFT value	Description
0x3F	SSI_TXE_INTR is asserted when 64 data entries are present in transmit FIFO

*Table 20-12. **SSSP_txftlr** Register TFT Decode for the SSI / SPI Dedicated Slave Controller.*

See Section 20.2.2 “SSI / SPI Transmit and Receive FIFO Buffers” for information on SSI_TXE_INTR assertion.

20.3.2.7 SSSP_rxftlr Register

This register controls the threshold value for the receive FIFO memory. It is not possible to write to this register when the SSI / SPI interface is enabled. The SSI / SPI interface is enabled and disabled by writing to the **SSSP_ssienr** register. RFT values differ between the Master and Slave controllers as described in the tables below.

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	**rft**	RW	0	Receive FIFO Threshold Used to control the level of entries (or above) at which the receive FIFO controller triggers an interrupt. The FIFO depth is configurable in the range to 32 or 64 depending on the frame size. This register is sized to the number of address bits needed to access the FIFO. Attempts to set this value greater than the depth of the FIFO do not write to this field and it retains its current value. When the number of receive FIFO entries is greater than or equal to this value + 1, the receive FIFO full interrupt is triggered.

Table 20-13. Receive FIFO Threshold Level Register.

RFT value	Description
0x00	SSI_RXF_INTR is asserted when 1 data entries are present in receive FIFO
0x01	SSI_RXF_INTR is asserted when 2 data entries are present in receive FIFO
0x02	SSI_RXF_INTR is asserted when 3 data entries are present in receive FIFO
...	
0x3F	SSI_RXF_INTR is asserted when 64 data entries are present in receive FIFO

*Table 20-14. **SSSP_rxftlr** Register RFT Decode for the SSI / SPI Master Controller.*

RFT value	Description
0x00	SSI_RXF_INTR is asserted when 1 data entries are present in receive FIFO
0x01	SSI_RXF_INTR is asserted when 2 data entries are present in receive FIFO
0x02	SSI_RXF_INTR is asserted when 3 data entries are present in receive FIFO
...	
0x3F	SSI_RXF_INTR is asserted when 64 data entries are present in receive FIFO

*Table 20-15. **SSSP_rxftlr** Register RFT Decode for the SSI / SPI Dedicated Slave Controller.*

See Section 20.2.2 “SSI / SPI Transmit and Receive FIFO Buffers” for more information on SSI_RXF_INTR assertion.

20.3.2.8 SSSP_txflr Register

This register contains the number of valid data entries in the transmit FIFO memory.

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	**txtfl**	R	0	Transmit FIFO Level

Table 20-16. Transmit FIFO Level Register.

20.3.2.9 SSSP_rxflr Register

This register contains the number of valid data entries in the transmit FIFO memory. This register can be read at any time.

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	**rxtfl**	R	0	Receive FIFO Level

Table 20-17. Receive FIFO Level Register.

20.3.2.10 SSSP_sr Register

This is a read-only register used to indicate the current transfer status, FIFO status, and transmission/reception errors that may have occurred. The status register may be read at any time. None of the bits in this register request an interrupt.

Bits	Name	Access	Reset	Description
31:6				Reserved
5	**txe**	R	0	Transmission Error Set if the transmit FIFO is empty when a transfer is started. This bit can only be set when the SSI / SPI is configured as a slave device. Data from the previous transmission is resent on the TXD line. This bit is cleared when read. 0 No error 1 Transmission error
4	**rff**	R	0	Receive FIFO Full When the receive FIFO is completely full, this bit is set. When the receive FIFO contains one or more empty location, this bit is cleared. 0 Receive FIFO is not full 1 Receive FIFO is full

For Wekra Lang only
1.5 (08/10/2015)
Confidential

Bits	Name	Access	Reset	Description
3	**rfne**	R	0	Receive FIFO Not Empty. Set when the receive FIFO contains one or more entries and is cleared when the receive FIFO is empty. This bit can be polled by software to completely empty the receive FIFO. 0 Receive FIFO is empty 1 Receive FIFO is not empty
2	**tfe**	R	1	Transmit FIFO Empty When the transmit FIFO is completely empty, this bit is set. When the transmit FIFO contains one or more valid entries, this bit is cleared. This bit field does not request an interrupt. 0 Transmit FIFO is not empty 1 Transmit FIFO is empty
1	**tfnf**	R	1	Transmit FIFO Not Full Set when the transmit FIFO contains one or more empty locations, and is cleared when the FIFO is full. 0 Transmit FIFO is full 1 Transmit FIFO is not full
0	**busy**	R	0	SSI Busy Flag When set, indicates that a serial transfer is in progress; when cleared indicates that the SSI / SPI is idle or disabled. 0 SPI is idle or disabled 1 SPI is actively transferring data

Table 20-18. SSI / SPI Status Register (SR).

For Mokra Lang only 1.5 / 08012025 Confidential

20.3.2.11 SSSP_imr Register

This read/write register masks or enables all interrupts generated by the SSI / SPI interface. In DMA mode, bit-0 (**txeim**) and bit-4 (**rxfim**) must be set to 0.

Bits	Name	Access	Reset	Description
31:10				Reserved
9	**transfer_doneim**	RW	1	Only valid in master configuration. SSI transfer done interrupt mask. 0 Masked 1 Not-masked
8	**fcrim**	RW	1	Only valid in the master configuration. Frame counter reaches target value interrupt mask. 0 Masked 1 Not-masked
7:5				Reserved
4	**rxfim**	RW	1	Receive FIFO Full Interrupt Mask 0 SSI_RXF_INTR interrupt is masked 1 SSI_RXF_INTR interrupt is not masked
3	**rxoim**	RW	1	Receive FIFO Overflow Interrupt Mask 0 SSI_RXO_INTR interrupt is masked 1 SSI_RXO_INTR interrupt is not masked

Bits	Name	Access	Reset	Description
2	**rxuim**	RW	1	Receive FIFO Underflow Interrupt Mask 0 SSI_RXU_INTR interrupt is masked 1 SSI_RXU_INTR interrupt is not masked
1	**txoim**	RW	1	Transmit FIFO Overflow Interrupt Mask 0 SSI_TXO_INTR interrupt is masked 1 SSI_TXO_INTR interrupt is not masked
0	**txeim**	RW	1	Transmit FIFO Empty Interrupt Mask 0 SSI_TXE_INTR interrupt is masked 1 SSI_TXE_INTR interrupt is not masked

Table 20-19. SPI Interrupt Mask Register.

20.3.2.12 SSSP_isr Register

This register reports the status of the SSI / SPI interrupts after they have been masked.

Bits	Name	Access	Reset	Description
31:10				Reserved
9	**transfer_doneis**	R	0	Only valid in master configuration. SSI transfer done interrupt status. 0 Not active 1 Active
8	**fcris**	R	0	Only valid in the master configuration. Frame counter reaches target value interrupt status. 0 Not active 1 Active
7:5				Reserved
4	**rxfis**	R	0	Receive FIFO Full Interrupt Status 0 SSI_RXF_INTR interrupt is not active after masking 1 SSI_RXF_INTR interrupt is active after masking
3	**rxois**	R	0	Receive FIFO Overflow Interrupt Status 0 SSI_RXO_INTR interrupt is not active after masking 1 SSI_RXO_INTR interrupt is active after masking
2	**rxuis**	R	0	Receive FIFO Underflow Interrupt Status 0 SSI_RXU_INTR interrupt is not active after masking 1 SSI_RXU_INTR interrupt is active after masking
1	**txois**	R	0	Transmit FIFO Overflow Interrupt Status 0 SSI_TXO_INTR interrupt is not active after masking 1 SSI_TXO_INTR interrupt is active after masking
0	**txeis**	R	0	Transmit FIFO Empty Interrupt Status 0 SSI_TXE_INTR interrupt is not active after masking 1 SSI_TXE_INTR interrupt is active after masking

Table 20-20. SPI Interrupt Status Register.

For Internal Use Only Confidential 1.5

20.3.2.13 SSSP_risr Register

This register reports the status of the SSI / SPI interrupts prior to masking.

Bits	Name	Access	Reset	Description
31:10				Reserved
9	**transfer_done-ir**	RW	0	Only valid for master configuration. SSI transfer done raw interrupt status. 0: Not active prior to masking 1 Active prior to masking
8	**fcrir**	RW	0	Only valid in the master configuration. Frame counter reaches target value raw interrupt status. 0 Not active prior to masking 1 Active prior to masking
7:5				Reserved
4	**rxfir**	RW	0	Receive FIFO Full Raw Interrupt Status 0 SSI_RXF_INTR interrupt is not active prior to masking 1 SSI_RXF_INTR interrupt is active prior to masking
3	**rxoir**	RW	0	Receive FIFO Overflow Raw Interrupt Status 0 SSI_RXO_INTR interrupt is not active prior to masking 1 SSI_RXO_INTR interrupt is active prior to masking
2	**rxuir**	RW	0	Receive FIFO Underflow Raw Interrupt Status 0 SSI_RXU_INTR interrupt is not active prior to masking 1 SSI_RXU_INTR interrupt is active prior to masking
1	**txoir**	RW	0	Transmit FIFO Overflow Raw Interrupt Status 0 SSI_TXO_INTR interrupt is not active prior to masking 1 SSI_TXO_INTR interrupt is active prior to masking
0	**txeir**	RW	0	Transmit FIFO Empty Raw Interrupt Status 0 SSI_TXE_INTR interrupt is not active prior to masking 1 SSI_TXE_INTR interrupt is active prior to masking

Table 20-21. Raw Interrupt Status Register.

Notes:

- When irq_mode = 0, SSSP_risr status is read clear.
- When irq_mode = 1, SSSP_risr status is write one to clear.

20.3.2.14 SSSP_txoicr Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**txoicr**	R	0	Clear Transmit FIFO Overflow Interrupt This register reflects the status of the interrupt. A read from this register clears the SSI_TXO_INTR interrupt; writing has no effect.

Table 20-22. Transmit FIFO Overflow Interrupt Clear Register.

For Marketing only (08/15/2025)

20.3.2.15 SSSP_rxoicr Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**rxoicr**	R	0	Clear Receive FIFO Overflow Interrupt This register reflects the status of the interrupt. A read from this register clears the SSI_RXO_INTR interrupt; writing has no effect.

Table 20-23. Receive FIFO Overflow Interrupt Clear Register.

20.3.2.16 SSSP_rxuicr Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**rxuicr**	R	0	Clear Receive FIFO Underflow Interrupt This register reflects the status of the interrupt. A read from this register clears the SSI_RXU_INTR interrupt; writing has no effect.

Table 20-24. Receive FIFO Overflow Interrupt Clear Register.

20.3.2.17 SSSP_icr Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**icr**	R	0	Clear Interrupts This register is set if any of the interrupts above are active. A read clears the SSI_TXO_INTR, SSI_RXU_INTR, SSI_RXO_ INTR, Transfer Done and Frame Counter Reach interrupts. Writing to this register has no effect.

Table 20-25. SSI / SPI Interrupt Clear Register.

20.3.2.18 SSSP_dmacr Register

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**tdmae**	RW	0	TX DMA request enable This field is the SSI TX DMA request enable control bit. When this field is set and the TX FIFO level is under the target value, SSI will issue a TX DMA request to the DMA controller. In non-DMA mode, this bit must be set to 1 before data can be pushed to the TX FIFO. 0 Disable 1 Enable

For Mekra Lang Only
Confidential
1.5 (08-15-15)

Bits	Name	Access	Reset	Description
0	**rdmae**	RW	0	RX DMA request enable This field is the SSI RX DMA request enable control bit. When this field is set and the RX FIFO level reaches the target value, SSI will issue an RX DMA request to the DMA controller. 0 Disable 1 Enable

Table 20-26. SSI / SPI Interrupt Clear Register.

20.3.2.19 SSSP_fault_inject Register

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**errchk_disable**	RW	0	Error output disable. Disable safety error output during registers programming phase. 0 Enable error output. 1 Disable error output.
0	**fault_inject**	RW	0	Fault injection. Writes “1” to the register to assert error output for SW error test.

Table 20-27. SSSP_fault_inject Register.

20.3.2.20 SSSP_idr Register

This read-only register is available for use to store a peripheral identification code.

Bits	Name	Access	Reset	Description
31:0	**id**	R	0x414D4241	The peripheral identification code

Table 20-28. SSI / SPI Identification Code Register.

20.3.2.21 SSSP_version Register

This read-only register is available for use to store the SSI design version.

Bits	Name	Access	Reset	Description
31:0	**version_id**	R	0x312E3030	The SSI design version.

Table 20-29. SSI / SPI Version Register.

20.3.2.22 SSSP_dr Register

This SSI / SPI data register is a 16-bit read/write buffer for the transmit/receive FIFOs. When the register is read, data in the receive FIFO buffer is accessed. When it is written to, data is moved into the transmit FIFO buffer.

This register can only be written when **SSSP_ssienr** bit **ssi_en** = 1. The FIFOs are reset when **ssi_en** = 0.

Note:

Setting 0x2 in address 0x4C (**tdmae**) arrows filling TX data into FIFO when **ssi_en** =0.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**dr**	RW	0	Data Register When writing to this register, it is necessary to right-justify the data. Read data is automatically right-justified. Read Receive FIFO buffer Write Transmit FIFO buffer

Table 20-30. SSI / SPI Data Register.

20.3.2.23 SSSP_ssienpolr Register

This register is used for selecting slave enable signal polarity.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**ssienpol**	RW	0	Slave Select Signal Polarity Each bit controls each SER bit. Real output depends on SSI mode and this register. 0 Use original polarity 1 Invert original polarity For example, if bit 0 is for a SPI interface device, then: SSIENPOL[0] = 0 - Active low SSIENPOL[0] = 1 - Active high

Table 20-31. SSI / SPI Slave Enable Signal Polarity Register.

20.3.2.24 SSSP_sclk_out_dly Register

This register is used for delaying **SSI0CLK** after slave select asserting.

Bits	Name	Access	Reset	Description
31:17				Reserved
16:0	**sclk_out_dly**	RW	0	Delay **SSI0CLK** cycles after slave select asserted

*Table 20-32. SSI / SPI **SSI0CLK** Delay Counter Register.*

20.3.2.25 SSSP_target_frame_count Register

This register is used for defining the target transferred frame count to assert an interrupt.

Bits	Name	Access	Reset	Description
31:17				Reserved
16:0	**target_ frame_ count**	RW	0	Target frame count. If target_frame_count=1, it means there is 1 data frame to be transferred.

Table 20-33. SSI / SPI Target Frame Count Register.

20.3.2.26 SSSP_frame_count Register

This register shows current transferred data frame count.

Bits	Name	Access	Reset	Description
31:17				Reserved
16:0	**frame_count**	R	0	Current frame count

Table 20-34. SSI / SPI Current Frame Count Register.

20.3.2.27 SSSP_fcricr Register

This register is used to read clear **fcrir** when **irq_mode** = 0.

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**fcricr**	R	0	Frame Counter Reaches Interrupt This register reflects the status of the interrupt. A read from this register clears the **fcris** interrupt; writing has no effect.

Table 20-35. SSI / SPI Frame Count Interrupt Clear Register.

20.3.2.28 SSSP_tx_frame_count Register

This register is used to define the total number of transmit (TX) frames to send.

Bits	Name	Access	Reset	Description
31:16				Reserved
15:0	**tx_frame_ count**	RW	0	Total TX frame count. If tx_frame_count=0, it means 1 frame will be transmitted.

Table 20-36. SSI / SPI Total TX Frame Count Register.

20.3.2.29 SSSP_transfer_doneicr Register

This register is used to read clear **transfer_doneir** when **irq_mode** = 0.

For Mekra Lang only (Confidential 1.5 (Oct 2025))

Bits	Name	Access	Reset	Description
31:1/ trans				Reserved
0	**transfer_ doneicr**	R	0	When **irq_mode** = 0, read this register to clear **transfer_doneir**.

Table 20-37. SSI / SPI Transfer Done Interrupt Clear Register.

20.4 SSI / SPI: SSI Safety Notes

To force an error:

1. Enable ECRU error mask bit-20 – bit-25 for SSI0 – SSI5 and bit-26 for SSIS0.
2. Enable CEHU error mask bit-154 amba_safety_error.
3. Write 0x1 at SSI 0x50.

To clear the error status:

1. Disable sm_error in SSI
   - Write 0x2 at SSI 0x50, then
   - Write 0x0 at SSI 0x50.
   - 0x50 bit-1: sm_error disable
     - 0: SM error detection logic is enabled.
     - 1: SM error detection logic is disabled.
2. Clear bit-20 – bit-26 for corresponding SSI sm_error in ECRU.

To disable error propagation from SSI to CEHU:

1. Disable sm_error in SSI
   - Write 0x2 at SSI 0x50.
   - 0x50 bit-1: sm_error disable
     - 0: SM error detection logic is enabled.
     - 1: SM error detection logic is disabled.
2. Disable ECRU mask bit-20 – bit-26 for corresponding SSI module. Software can read it from ECRU of AHB bus and check the error status.

Notes:

- SW should perform fault injection after boot up.
- Before any configuration changes, SW should first write 0x2 at SSI 0x50 (disable error detection). After configurations are complete, SW should write 0x0 at SSI 0x50 to enable error detection.

Confidential For Mekra Lang only 1.5 (Oct 2025)

20.5 SSI / SPI: SPI NOR Programming and Register Information

20.5.1 SPI-NOR: Controller Architecture

The following figure provides an illustration of the CV22FS SPI NOR controller architecture.

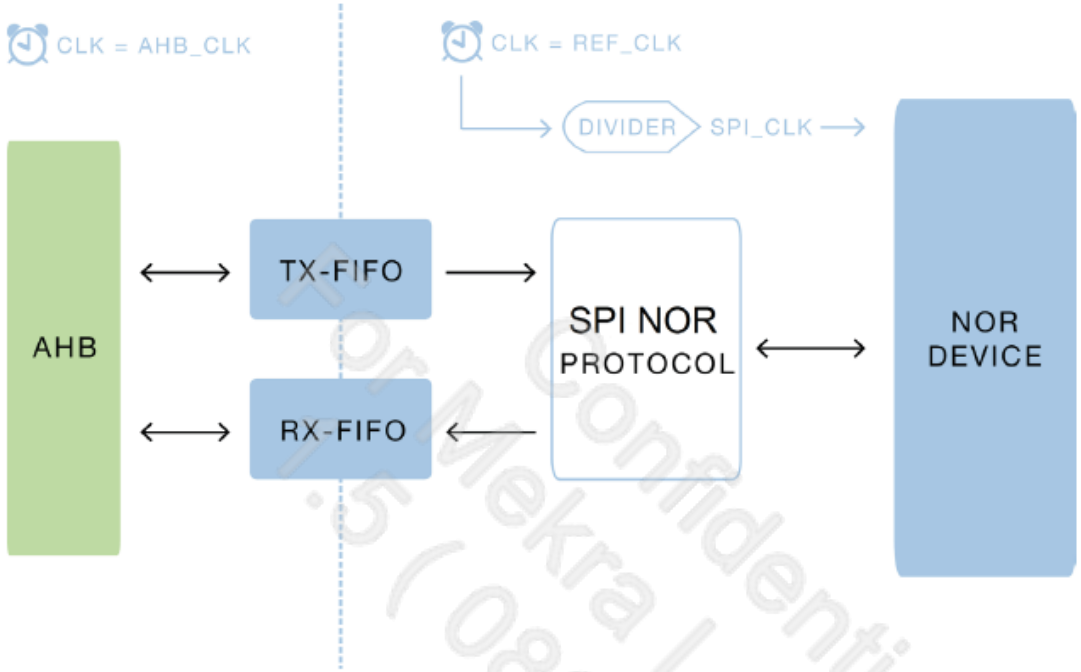


Figure 20-10. SPI NOR Controller Architecture.

Notes:

- The TX FIFO and RX FIFO modules are composed of asynchronous and synchronous FIFO.
- TX FIFO size is 265 bytes. RX FIFO size is 264 bytes.
- Based on the configuration register, the SPI NOR Protocol unit generates a NOR protocol waveform to access the NOR device.
- The clock divider is a register-based digital divider.
- The SPI NOR controller is located at AHB base address 0xE000.1000.

20.5.2 SPI NOR: Performance Specification Notes

- Ahb_clk: gclk_core
- Reference clock: gclk_ssi3
- Four clock source:
  - pll_out_enet

For Mekra Lang only Confidential 1.5 (08012025)

- gclk_core
- gclk_ssi3
- clk_ref

20.5.3 SPI NOR: Command Format

The figures below illustrate the basic structure and timing of a SPI NOR command. Note that the timing parameters used in this example are as follows:

- **cmdlength** = 1
- **addrlength** = 3
- **dummylength** = 2
- **datalength** = 1
- **numcmdlane** = 1
- **numaddrlane** = 1
- **numdatalane** = 1
- **dataread** = 1 (for read command)

Figure 20-11. SPI NOR Programming Command Format.

Confidential For Mekra Lang only 1.5 (08012025)

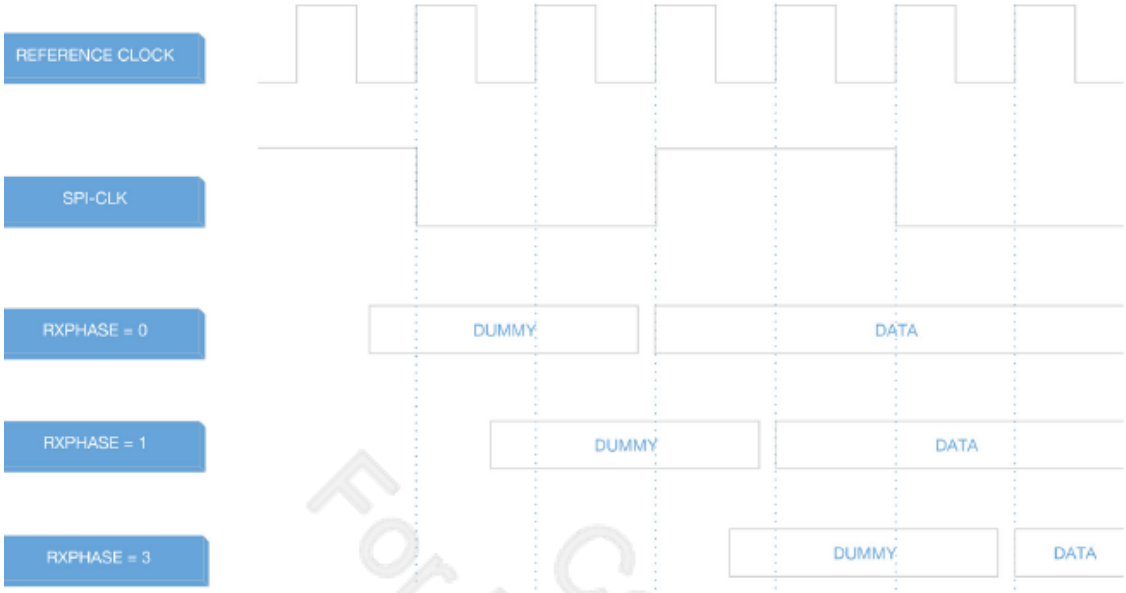


Figure 20-12. SPI NOR: Example of SPI RX Sampling Phase.

The following table shows the possible lane combinations for a commercially-available SPI NOR device. The CV22FS SPI NOR controller provides a degree of lane configuration flexibility; please contact an Ambarella representative for details.

Command Lane	Address Lane	Data Lane
1	1	01/2/4
1	2	2
2	2	2
1	4	4
4	4	4
1	8	8
8	8	8

Table 20-38. SPI NOR Lane Number Combinations.

Confidential For Weka Hang only 1.5 (08012025)

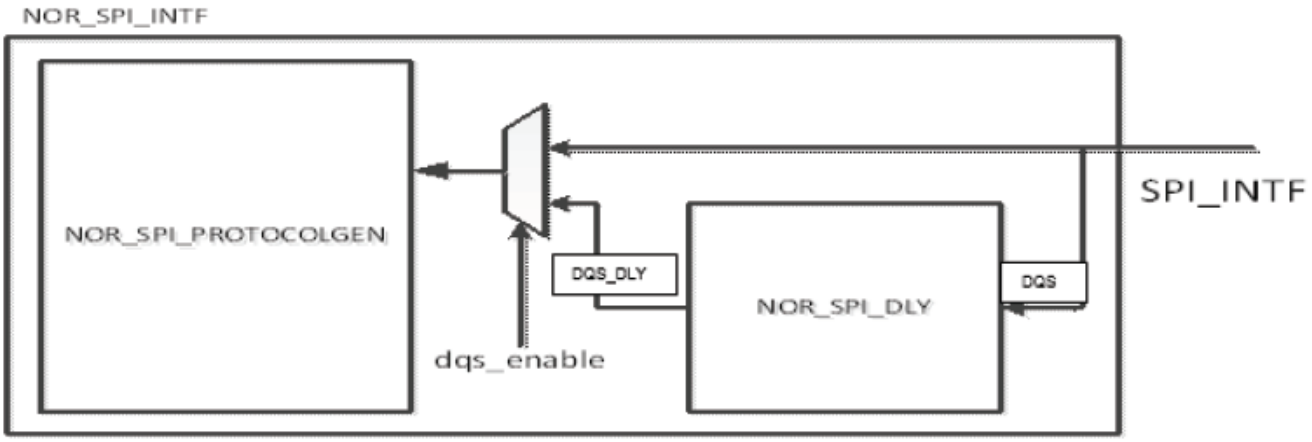


Figure 20-13. SPI NOR INTF Controller Architecture.

20.5.4 SPI NOR: SPI_NOR INTF

20.5.4.1 SPI_NOR Protocol Generator

Based on the configuration register, this unit generates the corresponding SPI protocol wave form to access the SPI NOR device.

20.5.4.2 SPI_NOR Delay line

The SPI_NOR delay module is used to delay the DQS signal in SPI INTF. Set 0x58 dqs_en is set to 1 when octa dqs read is used.

The following figure shows the octal DDR waveform.

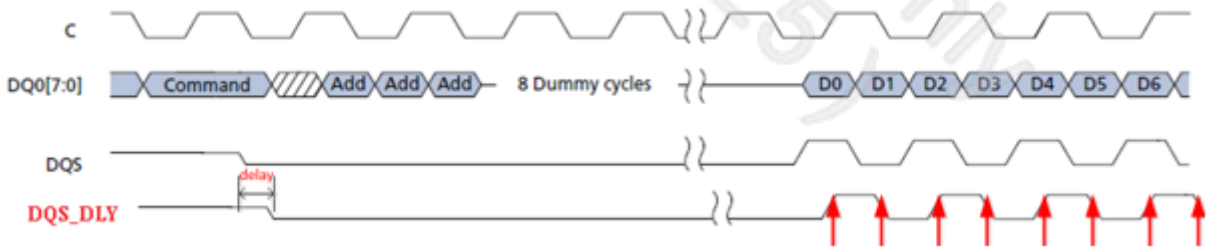


Figure 20-14. Octal DDR Waveform

DQS_DLY is delayed by “delay” ps by cascade delay lines.
delay = reg_dqs_dly_bin_in[13:7]*10+55+ reg_dqs_dly_bin_in[6:0]*10+55 ps.

The following figure displays the delay line.

Confidential
For Mekra Lang only
1 Oct 2025

Note:

Different corners of the chip will have different delay models.

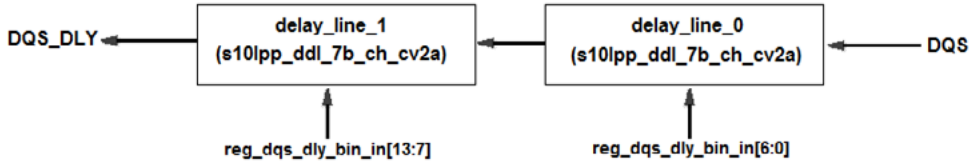


Figure 20-15. Delay Line

20.5.5 SPI NOR: Registers

When accessing SPI NOR controller registers, the following general guidelines apply:

1. Access to the SPI NOR device can be managed in one of two ways.
   - The device can be accessed via a DMA agent. In this case, the implementation of flow control measures can be useful for preventing TX/RX FIFO overflow/underflow when the DMA agent is otherwise occupied and fails to read or write data in a timely manner. This could be accomplished through the use of a commercially available device that supports flow control; however, the CV22FS SPI NOR controller can achieve a similar result by gating the SPI clock to hold the SPI NOR transaction. For more information on DMA access to the SPI NOR controller, please refer to Section 20.5.6.
   - Alternatively, the device can be directly accessed by the Cortex-A53 processor.
2. Program the configuration field according to the NOR device being used, then write the Start Transmission / Reception register **strttrx** to 1.
3. The data-write port range is between 0x100 and 0x1FF. The data-read port range is between 0x200 and 0x2FF. Once the programmed (received or transmitted) data length has been achieved, the **datalenthreachintr** interrupt will provide notification that the transaction is complete. The **rxalmostfullrintr** and **txunderflowintr** interrupts can be used to prevent a FIFO overflow or underflow.

The SPI NOR registers are listed in the following table.

For Confidential Use Only (2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x00	**format_0**	31:30	**CMDLENGTH**	RW	0x1	Command Length 0x0 0 byte 0x1 1 byte 0x2 2 bytes 0x3 3 bytes “disable the output on the interface” in case of 0 byte “
		29:27	**ADDRLENGTH**	RW	0x3	Address Length 0x0 0 byte 0x1 1 byte 0x2 2 bytes ... 0x7 7 bytes “disable the output on the interface” in case of 0 byte “
		26:22	**DUMMYLENGTH**	RW	0x0	Dummy Cycle Length 0x0 0 cycle 0x1 1 cycle 0x1 2 cycles ... 0x31 31 cycles “disable the output on the interface” in case of 0 byte “
		21:0	**DATALENGTH**	RW	0x18	Data Access Length 0x0 0 bytes 0x1 1 bytes 0x2 2 bytes ... 0x3FFFFF 3FFFFF bytes “disable the output on the interface” in case of 0 byte “

Confidential For Mekra Lang only 1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x04	**format_1**	31	**CMDDTR**	RW	0x0	DTR is only supported when reg_the clk divider >2 CMD Double Transfer Rate 0x0 NOT support DTR 0x1 Support DTR SW should program this register to match the device's mode
		30	**ADDRESSDTR**	RW	0x0	Address Double Transfer Rate 0x0 NOT support DTR 0x1 Support DTR SW should program this register to match the device's mode
		29	**DUMMYDTR**	RW	0x0	Dummy Double Transfer Rate 0x0 NOT support DTR 0x1 Support DTR SW should program this register to match the device's mode
		28	**DATADTR**	RW	0x0	Data Double Transfer Rate 0x0 NOT support DTR 0x1 Support DTR SW should program this register to match the device's mode If (dqs_en==1 && NUMDATALANE==3) DATADTR shall be set to 0
		27:25	-	-	-	Reserved
		24	**LSBFIRST**	RW	0x0	0x0 MSB first 0x1 LSB first
		15:14	**NUMCMDLANE**	RW	0x0	0x0 1 lane 0x1 2 lane 0x2 4 lane 0x3 8 lane
		13:12	**NUMADDRLANE**	RW	0x0	0x0 1 lane 0x1 2 lane 0x2 4 lane 0x3 Reserved
		11:10	**NUMDATALANE**	RW	0x0	0x0 1 lane 0x1 2 lane 0x2 4 lane 0x3 8 lane
		9	**RXLANE**	RW	0x1	0x0: numdatalane = 0 => uses lane#0 numdatalane = 1 => uses lane#0, #1 numdatalane = 2 => uses lane#0, #1, #2, #3 numdatalane = 3 => uses lane#0, #1, #2, #3, #4, #5, #6, #7

Confidential For Mekra Lang GmbH 1.5 (08/12/25)

Offset	Register	Bits	Field	Access	Reset	Description
0x04 (Cont'd)		9	**RXLANE**	RW	0x1	0x0: numdatalane = 0 => uses lane#0 numdatalane = 1 => uses lane#0, #1 numdatalane = 2 => uses lane#0, #1, #2, #3 numdatalane = 3 => uses lane#0, #1, #2, #3, #4, #5, #6, #7 0x1: numdatalane = 0 => uses lane#1 numdatalane = 1 => uses lane#1 numdatalane = 2 => uses lane#2, #3 numdatalane = 3 => uses lane#4, #5, #6, #7 Always set to 0 when TX only Always set to 1 when TRX
		1	**DATAWRITEEN**	RW	0x0	Data Part Write Mode 0x0 Write disable 0x1 Write enable Set to 1 when not only RX
		0	**DATAREADEN**	RW	0x1	Data Part Read Mode 0x0 Read disable 0x1 Read enable Set to 1 when not only TX
0x08	**config_0** **config_0**	31	**FLOWCONTROL**	RW	0x1	Flow Control Enable 0x0 No flow control 0x1 Flow control enable
		30:28	**HOLD**	RW	0x0	For flow control Takes effect when FLOWCONTROL=1 0x00 0 gate clock to perform flow control 0x01 Using dq_o[1] as hold to perform flow control 0x02 Using dq_o[2] as hold to perform flow control …. 0x06 Using dq_o[6] as hold to perform flow control 0x07 Using dq_o[7] as hold to perform flow control
		27	**SPICLKPOLARITY**	RW	0x0	Clock will remain 1 or 0 in standby mode. CPOL, CPHA only supports (0, 0) or (1, 1)
		26	-		-	Reserved
		25:18	**CHIPSELECT#**	RW	0xFE	CEN for multiple device SELPIN = CHIPSELECT | {8{SEL}}; 0 for select and vice versa

Confidential For Meka Liang only 1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x08 (Cont'd)		17:10	**CLKDIVIDER**	RW	0x1E	Divide reference clock 0x0 Reference clk (not allowed) 0x1 Reference clk (not allowed) 0x2 Reference clk/2 ... 0xFF Reference clk/255 Set CLKDIVIDER to an even number when using DTR mode
		9:5	-	-	-	Reserved
		4:0	**RXSAMPLEDELAY**	RW	0x0	Adjust RX Sampling Data Phase 0x0 0 reference clock cycle shift ... 0x1F 1F reference clock cycle shift 1 SPI cycle to adjust RX sampling phase
0x0C	**cmd_reg**	23:16	**CMD2**	RW	0x0	Command for SPI device
		15:8	**CMD1**	RW	0x0	Command for SPI device
		7:0	**CMD0**	RW	0x3	Command for SPI device
0x10	**addr_reg**	23:16	**ADDR6**	RW	0x0	Address field for SPI command
		15:8	**ADDR5**	RW	0x0	Address field for SPI command
		7:0	**ADDR4**	RW	0x0	Address field for SPI command
0x14	**addr2_reg**	31:24	**ADDR3**	RW	0x0	Address field for SPI command
		23:16	**ADDR2**	RW	0x0	Address field for SPI command
		15:8	**ADDR1**	RW	0x0	Address field for SPI command
		7:0	**ADDR0**	RW	0x0	Address field for SPI command
0x18	**dma_en**	1	**TXDMAENABLE**	RW	0x0	TX DMA enable
		0	**RXDMAENABLE**	RW	0x0	RX DMA enable
0x01C	**tx_fifo_th**	8:0	**TXFIFOTHLV**	RW	0x0	Transmit FIFO threshold level
0x20	**rx_fifo_th**	8:0	**RXFIFOTHLV**	RW	0x0	Receive FIFO threshold level
0x24	**tx_fifo_level**	8:0	**TXFIFOLV**	RO	0x0	Transmit FIFO level
0x028	**rx_fifo_level**	8:0	**RXFIFOLV**	RO	0x0	Receive FIFO level
0x02C	**fifo_status**	4	**RXFIFOFULL**	RO	0x0	RX FIFO full
		3	**RXFIFONOTEMP-TY**	RO	0x0	RX FIFO not empty
		2	**TXFIFOEMPTY**	RO	0x1	TX FIFO empty
		1	**TXFIFONOTFULL**	RO	0x1	TX FIFO not full
0x030	**interrupt_ mask**	6	**TXUNDER-FLOWMSK**	RW	0x0	TX underflow mask
		5	**DATALEN-THREACHMSK**	RW	0x0	Transaction done mask
		4	**RXALMOST-FULLMSK**	RW	0x0	RX FIFO almost full mask
		3	**RXOVERL-FOWMSK**	RW	0x0	RX overflow mask
		2	**RXUNDER-FLOWMSK**	RW	0x0	RX underflow mask
		1	**TXOVER-FLOWMSK**	RW	0x0	TX overflow mask
		0	**TXALMOSTEMP-TYMSK**	RW	0x0	TX FIFO almost empty mask

Offset	Register	Bits	Field	Access	Reset	Description
0x034	**interrupt**	6	**TXUNDERFLOW-INTR**	RO	0x0	Interrupt Status = Raw Interrupt Status & Interrupt Mask TX underflow interrupt
		5	**DATALEN-THREACHINTR**	RO	0x0	Transaction done interrupt
		4	**RXALMOSTFULL-INTR**	RO	0x0	RX FIFO almost full interrupt
		3	**RXOVERLFOW-INTR**	RO	0x0	RX FIFO overflow interrupt
		2	**RXUNDERFLOW-INTR**	RO	0x0	RX FIFO underflow interrupt
		1	**TXOVERFLOW-INTR**	RO	0x0	TX FIFO overflow interrupt
		0	**TXALMOSTEMP-TYINTR**	RO	0x0	TX almost empty interrupt
0x38	**raw_interrupt**	6	**TXUNDERFLOW-IRNTR**	RO	0x0	Raw Interrupt Status TX underflow raw interrupt
		5	**DATALEN-THREACHRINTR**	RO	0x0	Transaction done interrupt
		4	**RXALMOSTFULL-RINTR**	RO	0x0	RX FIFO almost full
		3	**RXOVERLFOW-RINTR**	RO	0x0	RX FIFO overflow interrupt
		2	**RXUNDERFLOW-RINTR**	RO	0x0	RX FIFO underflow interrupt
		1	**TXOVERFLOW-RINTR**	RO	0x0	TX FIFO overflow interrupt
		0	**TXALMOSTEMP-TYRINTR**	RO	0x0	TX almost empty interrupt
0x3C	**clear_inter-rupt**	6	**TXUNDERFLOW-INTRC**	W1C	-	Write Clear Interrupt Status Write 1 to clear TX underflow clear
		5	**DATALEN-THREACHRINTRC**	W1C	-	Transaction done clear
		4	**RXALMOSTFULL-RINTRC**	W1C	-	RX FIFO almost full clear
		3	**RXOVERLFOW-RINTRC**	W1C	-	RX overflow clear
		2	**RXUNDERFLOW-RINTRC**	W1C	-	RX underflow clear
		1	**TXOVERFLOW-RINTRC**	W1C	-	TX overflow clear
		0	**TXALMOSTEMP-TYRINTRC**	W1C	-	TX FIFO almost empty clear
0x40	**txfifo_reset**	0	**TXFIFORESET**	AC	0x0	Software reset the TX FIFO
0x44	**rxfifo_reset**	0	**RXFIFORESET**	AC	0x0	Software reset the RX FIFO
0x50	**start_trx**	0	**STRTTRX**	AC	0x0	Start TX or RX

For Confidential Ambarella Internal Use Only

Offset	Register	Bits	Field	Access	Reset	Description
0x54	**infinite_ length**	0	**INFINITE_DATA_ LENGTH**	RW	0x0	0 Normal mode: data_length is equal to what is programmed in 0x0 1 Data_length is not applicable as the data_length is infinite
0x58	**dqs_octa**	31:20	**reserved**	RW	0x0	Reserved
		19:16	**dqs_rx_select_ extent**	RW	0xF	0 No chip select extension 1 Delay chip select 1 reference cycle 2 Delay chip select 2 reference cycle … 15 Delay chip select 15 reference cycle If (STR mode && dqs_en && clk_divider==2) dqs_rx_select_extent should be set to non-zero value.
		15	**reserved**	RW	0x0	Reserved
		14:8	**dqs_dely_bin_hi**	RW	0xF	Cascade delay line to delay the DQS signal N*20+80
		7:1	**dqs_dely_bin_lo**	RW	0xF	Cascade Delay line to delay the DQS signal N*20+80
		0	**dqs_en**	RW	0x0	0 DQS is not used for octa device 1 DQS is used for octa device **Notes**: DQS is only used in RX mode. Set dqs_en=0 when the controller is in TX mode. 8-lane DTR DQS mode is not allowed. If(dqs_en==1 && NUMDATALANE==3) DATADTR shall be set to 0.
0x5C	**fault_inj**	31:2	**reserved**	RW	0x0	Reserved
		1	**sm_error_dis-able**	RW	0x0	0 The SM error detection logic is en abled 1 The SM error detection logic is en abled When SW must program the nor_spi control-ler, SW must set it to 1. After the programming is complete, then the SW should clear it to 0 to enable the SM er-ror detection logic.
		0	**fault_inj**	RW	0x0	0 No fault inject 1 Fault inject
0x100	**txdata**	31:0	**TXDATA**	WO	-	Data to be written to NOR device
...			**...**		-	Data to be written to NOR device
0x1FF	**txdata**	31:0	**TXDATA**	WO	-	Data to be written to NOR device
0x200	**rxdata**	31:0	**RXDATA**	RO	-	Data to read from NOR device
...					-	Data to read from NOR device
0x2FF	**rxdata**	31:0	**RXDATA**	RO	-	Data to read from NOR device

Table 20-39. SSI / SPI: SPI NOR Registers.

For Mekra Lang 1.5 (08/12/2025) Confidential

20.5.6 SPI NOR: Using DMA

Note that the use of DMA access with the CV22FS SPI NOR controller is subject to the requirements and constraints listed below.

- **tx_dma_req** will be asserted when the TX FIFO level (**txfifolv**) is less than or equal to **txfifothlv**.
- **rx_dma_req** will be asserted when the RX FIFO level (**rxfifolv**) exceeds **rxfifothlv**.
- RX DMA constraints:
  - **dma_blk_size**: **dma_ch[n]_control**[26:24]
    dma_byte_count: **dma_ch[n]_control**[21:0]
    - **rxfifothlv** == **dma_blk_size** - 1
    - (**datalength** % **dma_blk_size**) == 0
    - **dma_byte_count** == **datalength**
- For example: RX DMA 128 bytes of SPI NOR
  - **dma_blk_size** = 32 bytes (**dma_ch[n]_control**[26:24] = 2)
  - **dma_byte_count** = 128 bytes (**datalength** = 128)
  - **rxfifothlv** = 32 - 1 = 31

When using DMA:

1. Wait for the RX FIFO to accumulate 32 bytes, a value larger than the FIFO threshold level → DMA request → DMA AHB read → (128 - 32 bytes (**dma_blk_size** == 32 bytes) left to be read via DMA).
2. Wait for the RX FIFO to accumulate 32 bytes, a value larger than the FIFO threshold level → DMA request → DMA AHB read → (96 - 32 bytes (**dma_blk_size** == 32 bytes) left to be read via DMA).
3. Wait for the RX FIFO to accumulate 32 bytes, a value larger than the FIFO threshold level → DMA request → DMA AHB read → (64 - 32 bytes (**dma_blk_size** == 32 bytes) left to be read via DMA).
4. Wait for the RX FIFO to accumulate 32 bytes, a value larger than the FIFO threshold level → DMA request → DMA AHB read → (final DMA transaction).

Note that the steps above must be used to configure the RX DMA engine; otherwise, the DMA transaction will hang as a result of the residual data in the RX FIFO (i.e., the final DMA request will not be triggered).

20.6 SSI / SPI: SPI NOR Safety Notes

To force an error:

1. Enable ECRU error mask bit 30 nor_spi_par_failure.
2. Enable CEHU error mask bit 154 amba_safety_error.
3. Write NOR_SPI 0x5c 0x1.

To clear the error status:

1. Disable sm_error in NOR_SPI
   - Write NOR_SPI 0x5c 0x2, then
   - Write NOR_SPI 0x5c 0x0.

For Melfas Use Only Confidential 1.5 Oct 01 2025

- Bit [1] sm_error_disable
  - 0: SM error detection logic is enabled.
  - 1: SM error detection logic is disabled.

2. Clear bit 30 nor_spi_par_failure in ECRU.

To disable error propagation from NOR_SPI to CEHU:

1. Disable sm_error in NOR_SPI
   - Write NOR_SPI 0x5c 0x2.
   - Bit [1] sm_error_disable
     - 0: SM error detection logic is enabled.
     - 1: SM error detection logic is disabled
2. Disable ECRU mask: bit 30 nor_spi_par_failure. Software can read it from ECRU of AHB bus and check error status bit 30.

Notes:

- SW should perform fault injection after boot up.
- When performing fault injection, do not set sm_error_disable to 1; instead, write 0x5c to 1. Fault injection and real safety error are gated when sm_error_disable is set to 1.
- Before any configuration changes, SW should first set 0x5C to 0x2 (disable sm_error). After the configuration changes are complete, SW should set 0x5C to 0.

Confidential For Meka Lang only (08012025)

21. UART

This chapter discusses the CV22FS universal asynchronous receiver / transmitter (UART) interface. The chapter is organized as follows:

- (Section 21.1) UART: Overview
- (Section 21.2) UART: Clocking
- (Section 21.3) UART: Registers

21.1 UART: Overview

The CV22FS chip offers five industry-standard 16550 UART programming interfaces. One interface (uart0_apb) is under non-secure APB, and the other 4 (uart1_ahb - uart4_ahb) are under non-secure AHB. Features of the interfaces include:

- 32-bit data width and internal FIFO with a depth of 64
- Support for auto- and manual- flow control on UART interfaces uart1_ahb - uart4_ahb
- Loopback mode control support

The four UART interfaces located on the AHB bus (uart1_ahb - uart4_ahb) support flow control and DMA access by the DMA Engine. The fifth APB-based UART instance (uart0_apb) does not support flow control, and certain register fields are not applicable to this interface.

21.2 UART: Clocking

CV22FS system software generally sets and configures the source of the baud rate clock and the UART clock GCLK_UART. The register programming defined in this chapter is used for adjustment and refinement.

Notes:

- When using uart1_ahb ~ uart4_ahb, the AHB clock shall be faster than GCLK_UART / 8.
- When using uart0_apb, the APB clock shall be faster than GCLK_UART / 8.

Confidential For Mekra Lang only 1.5 (801)925

21.3 UART: Registers

21.3.1 UART Registers: Map

UART registers are multi-functional. Address offsets 0x00 and 0x04 use the value of a particular bit as a switch that triggers read and write operations to different registers at the same address. Address offset 0x08 also provides read or write access to different registers. The following map outlines these dependencies. Note that DMA-related register fields are not applicable to the **uart0_apb** interface.

Offset	Register	Field	Description
0x00	**UART_rbr**	Receive Buffer	Read access when **dlab** = 0
	UART_thr	Transmit Holding	Write access when **dlab** = 0
	UART_dll	Divisor Latch (Low)	Read and Write access when **dlab** = 1
0x04	**UART_ier**	Interrupt Enable	Read and Write access when **dlab** = 0
	UART_dlh	Divisor Latch (High)	Read and Write access when **dlab** = 1
0x08	**UART_iir**	Interrupt Identity	Read access only
	UART_fcr	FIFO Control	Write access only
0x0C	**UART_lcr**	Line Control	
0x10	**UART_mcr**	Modem Control	
0x14	**UART_lsr**	Line Status	
0x18	**UART_msr**	Modem Status	
0x1C	**UART_scr**	Scratchpad	
0x28	**UART_dmae**	DMA Enable in UART_AHB	
0x40 - 0x5F	**UART_thr (Remapped)**	Mapping to Transmit Holding - This allows the DMA Engine to access UART_AHB with AHB incrementing burst transaction.	Write access when **dlab** = 0 (Any write access to this section is equivalent to writing **UART_thr**)
	UART_rbr (Remapped)	Mapping to Receive Buffer - This allows the DMA Engine to access UART_AHB with AHB incrementing burst transaction.	Read access when **dlab** = 0 (Any read access to this section is equivalent to reading **UART_rbr**)
0x7C	**UART_usr**	UART Status	
0x80	**UART_tfl**	Transmit FIFO Level	
0x84	**UART_rfl**	Receive FIFO Level	
0x88	**UART_srr**	Software Reset	
0x8C - 0xA0		Reserved	
0xA4	**UART_htx**	Halt TX	
0xA8 - 0xAB		Reserved	
0xAC	**UART_rtr**	RX Threshold Register	
0xB0	**UART_ttr**	TX Threshold Register	
0xB4	**UART_state**	RX / TX State Register	Read only—for debug usage
0xB8	**UART_busy**	RX / TX Busy Register	Read only—for debug usage
0xC0 - 0xF3		Reserved	
0xF4	**UART_cpr**	Component Parameter	
0xF8	**UART_ucv**	UART Component Version	
0xFC	**UART_ctr**	Component Type	

Table 21-1. UART Interface Registers.

For Ambarella Confidential Use Only
1.5 (Feb 25, 2025)

21.3.2 UART Registers: Detail

21.3.2.1 UART_rbr Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**receive_buffer**	R	0	Receive Buffer The Receive Buffer register (**UART_rbr**) contains the data byte received on the serial input pin **UART_RX**. The data in **UART_rbr** is valid only if the Data Ready bit (**dr**) in the Line status register (**UART_lsr**) is set. In non-FIFO mode (the FIFOs are programmed OFF), the data in **UART_rbr** must be read before the next data arrives, otherwise it will be overwritten, resulting in an overrun error. In FIFO mode (the FIFOs are programmed ON), this register accesses the head of the receive FIFO. If the receive FIFO is full and this register is not read before the next data character arrives, then the data already in the FIFO will be preserved but any incoming data will be lost and an overrun error will occur.

Table 21-2. UART Receive Buffer Register (RBR).

21.3.2.2 UART_thr Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**transmit_holding**	W	0	Transmit Holding The Transmit Holding register (**UART_thr**) contains data to be transmitted on the serial output port (**UART_TX** pin). Data can be written to **UART_thr** any time the Transmit Holding Register Empty bit (**thre**) of the Line Status register (**UART_lsr**) is set. If FIFOs are not enabled and **thre** is set, writing a single character to **UART_thr** clears **thre**. Any additional writes to **UART_thr** before **thre** is set again causes the **UART_thr** data to be overwritten. If FIFOs are enabled and **thre** is set, x number of characters of data may be written to **UART_thr** before the FIFO is full. The number x (default = 16) is determined by the value of FIFO Depth that software sets during configuration. Any attempt to write data when the FIFO is full results in the loss of the write data.

Table 21-3. UART Transmit Holding Register (THR)

For Nekra Lane Only
1.5 (Confidential)

21.3.2.3 UART_dlh/ UART_dll Registers

The Divisor Latch High register (**UART_dlh**), in conjunction with Divisor Latch Low register (**UART_dll**), forms a 16-bit, read/write, Divisor Latch register that contains the baud rate divisor for the UART. It is accessed by first setting Line Control register (**UART_lcr**) bit 7 (**dlab**). The output baud rate is equal to the serial clock frequency divided by sixteen times the value of the baud rate divisor, as follows:

baud rate = GCLK_UART / (16 * divisor)

Note that when the Divisor Latch registers (**UART_dll** and **UART_dlh**) are set to zero, the baud clock is disabled and no serial communications occur.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**divisor_ latch_high**	RW	0	Divisor Latch (High) Upper 8 bits of a 16-bit, read/write, Divisor Latch register that contains the baud rate divisor for the UART.

Table 21-4. UART Divisor Latch High Register (DLH).

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**divisor_latch_ low**	RW	0	Divisor Latch (Low) Lower 8 bits of a 16-bit, read/write, Divisor Latch register that contains the baud rate divisor for the UART.

Table 21-5. UART Divisor Latch Low Register (DLL).

21.3.2.4 UART_ier Register

The Interrupt Enable register (**UART_ier**) is a read/write register that contains bits to enable the generation of interrupts. See Section 21.3.2.5 for details on the interrupt operation.

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**ptime**	RW	0	Programmable THRE Interrupt Mode Enable. Always readable. This is used to enable/disable the generation of THRE Interrupt. 0 - Disabled 1 - Enabled
6	**eretoi**	RW	0	Enable/Disable Receiver FIFO Empty and Timeout Interrupt. This is the second-highest priority interrupt. 0 - Disabled 1 - Enabled
5	**etoi**	RW	0	Enable/Disable Time Out Interrupt for a time out event 0 - Disabled 1 - Enabled
4	**ebdi**	RW	0	Enable/Disable Busy Detect Indication for writes to **UART_lcr** during a busy state 0 - Disabled 1 - Enabled

For Mass Production Lang only (2012-05-15) Confidential

Bits	Name	Access	Reset	Description
3	**edssi**	RW	NR	Enable Modem Status Interrupt This enables/disables generation of a Modem Status Interrupt. This is the fourth-highest priority interrupt. 0 - Disabled 1 - Enabled
2	**elsi**	RW	NR	Enable Receiver Line Status Interrupt This enables/disables generation of a Receiver Line Status Interrupt. This is the highest priority interrupt. 0 - Disabled 1 - Enabled
1	**etbei**	RW	NR	Enable Transmit Holding Register Empty Interrupt This enables/disables generation of a Transmitter Holding Register Empty Interrupt. This is the third-highest priority interrupt. 0 - Disabled 1 - Enabled
0	**erbfi**	RW	NR	Enable Received Data Available Interrupt This enables/disables generation of a Received Data Available Interrupt. This is the second-highest priority interrupt. 0 - Disabled 1 - Enabled

Table 21-6. UART Interrupt Enable Register (IER).

For Mekra Lang only
Confidential
15 (086 2025)

21.3.2.5 UART_iir Register

The Interrupt Identity register (**UART_iir**) is a read-only register that identifies the source of an interrupt. The upper two bits of the register are the FIFO enable bits. These bits will be 0x0 if the FIFOs are disabled, and 0x3 if they are enabled. The lower four bits identify the highest-priority pending interrupt as shown in Table 21-8 below.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:6	**fifos_enabled**	R	0	FIFOs Enabled 0x0 - FIFOs disabled 0x3 - FIFOs enabled
5:4				Reserved

Bits	Name	Access	Reset	Description
3:0	**interrupt_id**	R	1	Interrupt ID This indicates the highest-priority pending interrupt which can be one of the following types: 0x0 - Modem status changed 0x1 - No interrupt pending 0x2 - **UART_thr** empty 0x4 - Received data available 0x6 - Receiver line status 0x7 - Busy detect 0x12 - Character time out 0x13 - Character time out but RX FIFO empty Bit 3 indicates an interrupt can occur only when the FIFOs are enabled and used to distinguish a Character Timeout condition interrupt.

Table 21-7. UART Interrupt ID Register (IIR).

FIFO Mode Only	Interrupt Identification Register			Interrupt Set and Reset Functions			
Bit 3	**Bit 2**	**Bit 1**	**Bit 0**	**Priority Level**	**Interrupt Type**	**Interrupt Source**	**Interrupt Reset Control**
0	0	0	1		None	None	None
0	1	1	0	Highest	Receiver line status	Overrun/parity/framing errors or interrupt	Reading the Line Status register
0	1	0	0	Second	Received data available	Receiver data available (non-FIFO mode or FIFOs disabled) or RX FIFO trigger level reached (FIFO mode and FIFOs enabled)	Reading the Receiver Buffer register
1	1	0	0	Second	Character time out indication	No characters in or out of the RX FIFO during the last four character times and there is at least one character in it during this time	Reading the Receiver Buffer register
1	1	0	1	Second	Character time out but RX FIFO empty	No characters in or out of RX FIFO during the last four character times but RX FIFO is empty	Clear UART_ier[6] (eretoi)
0	0	1	0	Third	Transmit Holding Register empty	Transmitter Holding Register empty (Prog. THRE Mode disabled) or TX FIFO at or below threshold (Prog. THRE Mode enabled)	Reading the IIR register (if source of interrupt) or Writing to THR (FIFOs or THRE Mode not selected or disabled) or TX FIFO above threshold (FIFOs and THRE Mode selected and enabled).

Confidential
For Mekotronics
Freescale Semiconductor

FIFO Mode Only	Interrupt Identification Register			Interrupt Set and Reset Functions			
0	0	0	0	Fourth	Modem status	Clear to send or data set ready or ring indicator or data carrier detect. Note that if auto flow control mode is enabled, a change in **cts** (that is, setting **dcts**) does not cause an interrupt.	Reading the Modem Status register
0	1	1	1	Fifth	Busy detect indication	Master has tried to write to the Line Control register while the UART is busy (**UART_usr[0]** is 1).	Reading the UART status register

Table 21-8. UART Interrupt Control Functions.

For Mekra Lang only Confidential 1.5 (08012025)

21.3.2.6 UART_fcr Register

The FIFO Control register (**UART_fcr**) shares the address with **UART_iir** but is write-only. Note that bit 3 in this register is used with the UART_AHB interface only.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:6	**rcvr_trigger**	W	0	RX Trigger This is used to select the trigger level in the receiver FIFO at which the Received Data Available Interrupt is generated. In auto flow control mode it is used to determine when the **UART_AHB_RTS_N** signal is deasserted. It also determines when the external **dma_rx_req_n** signal is asserted in certain modes of operation. The following trigger levels are supported: 0x0 - One character in the FIFO 0x1 - 4 characters in the FIFO 0x2 - 8 characters in the FIFO 0x3 - Defined in UART_rtr
5:4	**tx_empty_trigger**	W	0	TX Empty Trigger This is used to select the empty threshold level at which the THRE Interrupts are generated when the mode is active. It also determines when the **dma_tx_req_n** signal is asserted when in certain modes of operation. The following trigger levels are supported: 0x0 - FIFO empty 0x1 - 2 characters in the FIFO 0x2 - 4 characters in the FIFO 0x3 - Defined in UART_ttr
3	**dma_mode**	W	0	DMA Mode (Note: UART_AHB only) This determines the DMA signaling mode used for the **DMA_TX_REQ_N** and **DMA_RX_REQ_N** output signals. 0 = Mode 0 1 = Mode 1 Note: When using Mode 1 with UART_AHB, the following constraints apply. For the TX FIFO: **tx_empty_trigger** can be 0, 1 or 2, and the maximum transfer length is 8 bytes. For the RX FIFO: **rcvr_trigger** can be 0, 1, 2 or 3.
2	**xmit_fifo_reset**	W_SC	0	TX FIFO Reset This resets the control portion of the transmit FIFO and treats the FIFO as empty. This also deasserts the DMA TX request and single signals. Note that this bit is self-clearing; it is not necessary to clear this bit.
1	**rcvr_fifo_reset**	W_SC	0	RX FIFO Reset This resets the control portion of the receive FIFO and treats the FIFO as empty. This also deasserts the DMA RX request and single signals. Note that this bit is self-clearing; it is not necessary to clear this bit.

For Mass Production 1.5 (Confidential)

Bits	Name	Access	Reset	Description
0	**fifo_enable**	W	0	FIFO Enable This enables/disables the transmit (TX) and receive (RX) FIFOs. Whenever the value of this bit is changed both the TX and RX controller portion of FIFOs is reset. Do not change this register when the UART is operating. The UART must be reset by the software (e.g., **UART_srr** fields **rfr**, **xfr**, and **ur**) after changing **fifo_enable**.

Table 21-9. FIFO Control Register (FCR).

21.3.2.7 UART_lcr Register

The Line Control register (**UART_lcr**) controls the format of the data that is transmitted and received by the UART.

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**dlab**	RW	0	Divisor Latch Access Bit Setting this bit enables reading and writing of the Divisor Latch register (**UART_dll** and **UART_dlh**) to set the baud rate of the UART. This bit must be cleared after initial baud rate setup in order to access other registers.
6	**break**	RW	0	Break Control Setting this bit sends a break signal by holding the SOUT line low (when not in Loopback mode, as determined by Modem Control egister (**UART_mcr**) bit 4, until **break** is cleared. When in Loopback mode, the break condition is internally looped back to the receiver.
5				Reserved
4	**eps**	RW	0	Even Parity Select This is used to select between even and odd parity. 0 - An odd number of logic 1s is transmitted or checked 1 - An even number of logic 1s is transmitted or checked
3	**pen**	RW	0	Parity Enable This bit is used to enable and disable parity generation and detection in transmitted and received serial character respectively. 0 - Disable 1 - Enable
2	**stop**	RW	0	Stop This is used to select the number of stop bits per character that the peripheral transmits and receives. If set to zero, one stop bit is transmitted in the serial data. If set to one and the data bits are set to 5 (LCR[1:0] set to zero) one and a half stop bits is transmitted. Otherwise, two stop bits are transmitted. Note that regardless of the number of stop bits selected, the receiver checks only the first stop bit. 0 - 1 stop bit 1 - 1.5 stop bits when **UART_lcr[1:0]** or **dls** is 0, else 2 stop bits

Bits	Name	Access	Reset	Description
1:0	**dls (cls)**	RW	0	Data Length Select (or previously CLS) This is used to select the number of data bits per character that the peripheral transmits and receives. The number of bits that may be selected are as follows: 0x0 - 5 bits 0x1 - 6 bits 0x2 - 7 bits 0x3 - 8 bits

Table 21-10. UART Line Control Register (LCR).

21.3.2.8 UART_mcr Register

Bits	Name	Access	Reset	Description
31:6				Reserved
5	**afce**	RW	0	0 - Auto Flow Control Mode disabled 1 - Auto Flow Control Mode enabled (UART_AHB only)
4	**loopback**	RW	0	LoopBack Bit This is used to put the UART into a diagnostic mode for test purposes. If operating in UART mode (SIR_MODE != Enabled or not active, **UART_mcr[6]** is 0), data on the SOUT line is held high, while serial data output is looped back to the sin line, internally. In this mode all the interrupts are fully functional. Also, in loopback mode, the modem control inputs (DSR_N, **UART_AHB_CTS_N**, RI_N, DCD_N) are disconnected and the modem control outputs (DTR_N, **UART_AHB_RTS_N**, OUT1_N, OUT2_N) are looped back to the inputs, internally.
3	**out2**	RW	0	OUT2 This is used to directly control the user-designated Output2 (OUT2_N) output. The value written to this location is inverted and driven out on OUT2_N, that is: 0 - OUT2_N deasserted (logic 1) 1 - OUT2_N asserted (logic 0) Note that in Loopback mode (**UART_mcr[4]** is 1), the OUT2_N output is held inactive high while the value of this location is internally looped back to an input.
2	**out1**	RW	0	OUT1 This is used to directly control the user-designated Output1 (OUT1_N) output. The value written to this location is inverted and driven out on OUT1_N, that is: 0 - OUT1_N deasserted (logic 1) 1 - OUT1_N asserted (logic 0) Note that in Loopback mode (**UART_mcr[4]** is 1), the OUT1_N output is held inactive high while the value of this location is internally looped back to an input.

For Preliminary Use Only

Bits	Name	Access	Reset	Description
1	**rts**	RW	0	Request to Send This is used to directly control the Request to Send (**UART_AHB_RTS_N**) output. The Request To Send (**UART_AHB_RTS_N**) output is used to inform the modem or dataset that the UART is ready to exchange data. When Auto RTS Flow Control is not enabled (**UART_mcr[5]** is 0), the **UART_AHB_RTS_N** signal is set low by programming MCR[1] (RTS) to a high. In Auto Flow Control, AFCE_MODE == Enabled and active (**UART_mcr[5]** is 1) and FIFOs enable (**UART_fcr[0]** is 1), the **UART_AHB_RTS_N** output is controlled in the same way, but is also gated with the receiver FIFO threshold trigger (**UART_AHB_RTS_N** is inactive high when above the threshold). The **UART_AHB_RTS_N** signal is deasserted when **UART_mcr[1]** is set low. Note that in Loopback mode (**UART_mcr[4]** is 1), the **UART_AHB_RTS_N** output is held inactive high while the value of this location is internally looped back to an input. Note that auto flow control mode is enabled when **afce** is 1. Do not program **UART_mcr** without turning off auto flow control (set **afce** to 0).
0	**dtr**	RW	0	Data Terminal Ready This bit directly controls the Data Terminal Ready (DTR_N) output. The value written to this location is inverted and driven out on DTR_N, that is: 0 - DTR_N deasserted (logic 1) 1 - DTR_N asserted (logic 0) The Data Terminal Ready output is used to inform the modem or data set that the UART is ready to establish communications. Note that in Loopback mode (**UART_mcr[4]** is 1), the DTR_N output is held inactive high while the value of this location is internally looped back to an input.

Table 21-11. UART Modem Control Register (MCR).

For Internal Use Only (1.5 (08/01/2025))

21.3.2.9 UART_lsr Register

The Line Status register (**UART_lsr**) contains status of the receiver and transmitter data transfers. This status register can be read by the programmer at any time.

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**rfe**	R	0	Receiver FIFO Error bit This bit is only relevant when FIFOs are enabled (UART_fcr[0] is 1). This indicates if there is at least one parity error, framing error, or break indication in the FIFO. 0 - No error in RX FIFO 1 - Error in RX FIFO This bit is cleared when **UART_lsr** is read and the character with the error is at the top of the receiver FIFO and there are no subsequent errors in the FIFO.
6	**temt**	R	1	Transmitter Empty If in FIFO mode and FIFOs enabled (UART_fcr[0] is 1), this bit is set whenever the Transmitter Shift register and the FIFO are both empty. If in non-FIFO mode or FIFOs are disabled, this bit is set whenever the Transmitter Holding register (**UART_thr**) and the Transmitter Shift register are both empty.
5	**thre**	R	1	Transmit Holding Register Empty If THRE mode is disabled (the UART_ier bit 7 (ptime) is set to zero) and regardless of FIFO's being enabled or not, this bit indicates that **UART_thr** or TX FIFO is empty. This bit is set whenever data is transferred from **UART_thr** or TX FIFO to the Transmitter Shift register and no new data has been written to **UART_thr** or TX FIFO. This also causes a THRE Interrupt to occur, if the THRE Interrupt is enabled (**UART_ier** bit **etbei**). If this mode is active (UART_ier bit 7 is set to one), the functionality is switched to indicate the transmitter FIFO is full, and no longer controls THRE interrupts, which are then controlled by the **UART_fcr** bits 5:4 threshold setting.
4	**bi**	R	0	Break Interrupt This is used to indicate the detection of a break sequence on the serial input data. If in UART mode, it is set whenever the serial input, sin, is held in a logic 0 state for longer than the sum of start time + data bits + parity + stop bits. In FIFO mode, the character associated with the break condition is carried through the FIFO and is revealed when the character is at the top of the FIFO. Reading **UART_lsr** clears **bi**. In the non-FIFO mode, the **bi** indication occurs immediately and persists until **UART_lsr** is read.

Bits	Name	Access	Reset	Description
3	**fe**	R	0	Framing Error This bit indicates a framing error has occurred when the receiver does not detect a valid stop bit in the received data. Because the framing error is associated with a character received, it is revealed when the character with the framing error is at the top of the FIFO. When a framing error occurs, the UART attempts to resynchronize. It does this by assuming that the error was due to the start bit of the next character and then continues receiving the other bit data, and/or parity and stop. The **fe** bit is set if a break interrupt **bi** has occurred (**UART_lsr[4]**). 0 - No framing error 1 - Framing error Note that **oe**, **pe** and **fe** are reset when **UART_lsr** is read.
2	**pe**	R	0	Parity Error This bit indicates a parity error in the receiver if the Parity Enable (**pen**) bit (**UART_lcr[3]**) is set. In the FIFO mode, since the parity error is associated with a character received, it is revealed when the character with the parity error arrives at the top of the FIFO. It should be noted that the Parity Error (**pe**) bit (**UART_lsr[2]**) is set if a break interrupt has occurred, as indicated by Break Interrupt (**bi**) bit (**UART_lsr[4]**). 0 - No parity error 1 - Parity error Note that **oe**, **pe** and **fe** are reset when **UART_lsr** is read.
1	**oe**	R	0	Overrun Error This bit indicates the occurrence of an overrun error. This error occurs if a new data character was received before the previous data was read. In non-FIFO mode, the **oe** bit is set when a new character arrives in the receiver before the previous character was read from **UART_rbr**. When this happens, the data in **UART_rbr** is overwritten. In the FIFO mode, an overrun error occurs when the FIFO is full and a new character arrives at the receiver. The data in the FIFO is retained and the data in the receive shift register is lost. 0 - No overrun error 1 - Overrun error Note that **oe**, **pe** and **fe** are reset when **UART_lsr** is read.
0	**dr**	R	0	Data Ready This is used to indicate that the receiver contains at least one character in **UART_rbr** or the receiver FIFO. 0 - No data ready 1 - Data ready This bit is cleared when the **UART_rbr** is read in non-FIFO mode, or when the receiver FIFO is empty, in FIFO mode.

Table 21-12. UART Line Status Register (LSR).

21.3.2.10 UART_msr Register

In UART0 this register is not used. The Modem Status register (**UART_msr**) contains the current status of the modem control input lines and indicates whether they have changed.

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**dcd**	R	0	Data Carrier Detect This is used to indicate the current state of the modem control line DCD_N. This bit is the complement of DCD_N. When the Data Carrier Detect input (DCD_N) is asserted it is an indication that the carrier has been detected by the modem or data set. 0 - DCD_N input is deasserted (logic 1) 1 - DCD_N input is asserted (logic 0) In Loopback Mode (**UART_mcr[4]** is 1), **dcd** is the same as **UART_mcr[3]** or **out2**.
6	**ri**	R	0	Ring Indicator This bit indicates the current state of the modem control line for Ring Indicator input or RI_N. This bit is the complement of RI_N. When RI_N is asserted it indicates that a telephone ringing signal has been received by the modem or data set. 0 - RI_N input is deasserted (logic 1) 1 - RI_N input is asserted (logic 0) In Loopback Mode (**UART_mcr[4]** is 1), **ri** is the same as **UART_mcr[2]** or **out1**.
5	**dsr**	R	0	Data Set Ready This is used to indicate the current state of the modem control line for Data Set Ready input or DSR_N. This bit is the complement of DSR_N. When DSR_N is asserted it indicates that the modem or data set is ready to establish communications with the UART. 0 - DSR_N input is deasserted (logic 1) 1 - DSR_N input is asserted (logic 0) In Loopback Mode (**UART_mcr[4]** is 1), DSR is the same as **UART_mcr[0]** or **dtr**.
4	**cts**	R	0	Clear to Send This is used to indicate the current state of the modem control line **UART_AHB_CTS_N**. This bit is the complement of **UART_AHB_CTS_N**. When the Clear to Send input (**UART_AHB_CTS_N**) is asserted it is an indication that the modem or data set is ready to exchange data with the UART. 0 - **UART_AHB_CTS_N** input is deasserted (logic 1) 1 - **UART_AHB_CTS_N** input is asserted (logic 0) In Loopback Mode (**UART_mcr[4]** is 1), **cts** is the same as **UART_mcr[1]** or **rts**.

For Marvell Internal Use Only 1.5 CONFIDENTIAL

Bits	Name	Access	Reset	Description
3	**ddcd**	R	0	Delta Data Carrier Detect This indicates that the modem control line DCD_N has changed since the last time the **UART_msr** was read. 0 - No change on DCD_N since last read of **UART_msr** 1 - Change on DCD_N since last read of **UART_msr** Reading the **UART_msr** clears the **ddcd** bit. In Loopback Mode (**UART_mcr[4]** is 1), **ddcd** reflects changes on **UART_mcr[3]** or **out2**. Note that if **ddcd** is not set and the DCD_N signal is asserted (low) and a reset occurs (software or otherwise), then the **ddcd** bit is set when the reset is removed if the DCD_N signal remains asserted.
2	**teri**	R	0	Trailing Edge of Ring Indicator This is used to indicate that a change on the input RI_N (from an active-low to an inactive-high state) has occurred since the last time the **UART_msr** was read. 0 - No change on RI_N since last read of **UART_msr** 1 - Change on RI_N since last read of **UART_msr** Reading the **UART_msr** clears the **teri** bit. In Loopback Mode (**UART_mcr[4]** is 1), **teri** reflects when MCR[2] or **out1** has changed state from high to a low.
1	**ddsr**	R	0	Delta Data Set Ready This is used to indicate that the modem control line DSR_N has changed since the last time the **UART_msr** was read. 0 - No change on DSR_N since last read of **UART_msr** 1 - Change on DSR_N since last read of **UART_msr** Reading the **UART_msr** clears the **ddsr** bit. In Loopback Mode (**UART_mcr[4]** is 1), **ddsr** reflects changes on MCR[0] or **dtr**. Note that if the **ddsr** bit is not set and the DSR_N signal is asserted (low) and a reset occurs (software or otherwise), then the **ddsr** bit is set when the reset is removed if the DSR_N signal remains asserted.
0	**dcts**	R	0	Delta Clear to Send This is used to indicate that the modem control line **UART_AHB_CTS_N** has changed since the last time the **UART_msr** was read. 0 - No change on CTSDSR_N since last read of **UART_msr** 1 - Change on CTSDSR_N since last read of **UART_msr** Reading the **UART_msr** clears the **dcts** bit. In Loopback Mode, (**UART_mcr[4]** is 1), **dcts** reflects changes on MCR[1] or RTS. Note that if the **dcts** bit is not set and the **UART_AHB_CTS_N** signal is asserted (low) and a reset occurs (software or otherwise), then the **dcts** bit is set when the reset is removed if the **UART_AHB_CTS_N** signal remains asserted.

Table 21-13. UART Modem Status Register (MSR).

21.3.2.11 UART_scr Register

The Scratchpad register (**UART_scr**) is an 8-bit read/write register for programmers to use as a temporary storage space. It has no defined purpose in the UART.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**scratchpad**	RW	0	Scratchpad Has no defined purpose in the UART. Use for temporary storage.

Table 21-14. UART Scratchpad Register (SCR).

21.3.2.12 UART_dmae Register

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**tdmae**	RW	0	Transmit DMA Enable (UART_AHB only). This enables/disables the transmit DMA function.
0	**rdmae**	RW	0	Receive DMA Enable (UART_AHB only). This enables/disables the receive DMA function.

Table 21-15. UART DMA Enable (UART1–UART4) Register (DMAE).

21.3.2.13 UART_usr Register

Bits	Name	Access	Reset	Description
31:5				Reserved
4	**rff**	R	0	Receive FIFO Full This bit is valid only when **fifo_stat** = YES. This is used to indicate that the receive FIFO is completely full. 0 - Receive FIFO not full 1 - Receive FIFO full This bit is cleared when the RX FIFO is no longer full.
3	**rfne**	R	0	Receive FIFO Not Empty This bit is used to indicate that the receive FIFO contains one or more data attempts. 0 - Receive FIFO is empty 1 - Receive FIFO is not empty This bit is cleared when the RX FIFO is empty.
2	**tfe**	R	1	Transmit FIFO Empty This bit is valid only when **fifo_stat** = YES. This is used to indicate that the transmit FIFO is completely empty. 0 - Transmit FIFO is not empty 1 - Transmit FIFO is empty This bit is cleared when the TX FIFO is no longer empty.
1	**tfnf**	R	1	Transmit FIFO Not Full This bit is used to indicate that the transmit FIFO in not full. 0 - Transmit FIFO is full 1 - Transmit FIFO is not full This bit is cleared when the TX FIFO is full.

For Mekro Logo only
1.5 Confidential

Bits	Name	Access	Reset	Description
0	**busy**	R	0	UART Busy Clearing this bit indicates that the UART is idle or inactive. 0 - UART is idle or inactive 1 - UART is busy (actively transferring data) Note that it is possible for **busy** to be cleared even though a new character may have been sent from another device. That is, if the UART has no data in **UART_thr** and **UART_rbr**; there is no transmission in progress; and a start bit of a new character has just reached the UART. This is because a valid start is not seen until the middle of the bit period, and this duration depends on the baud divisor which has been programmed. If a second system clock has been implemented, the assertion of this bit also is delayed by several cycles of the slower clock.

Table 21-16. UART Status Register (USR).

21.3.2.14 UART_tfl Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12:0	**tx_fifo**	R	0	Transmit FIFO Level This indicates the number of data data attempts in the transmit FIFO.

Table 21-17. UART Transmit FIFO Level Register (TFL).

21.3.2.15 UART_rfl Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12:0	**rx_fifo**	R	0	Receive FIFO Level This indicates the number of data data attempts in the receive FIFO.

Table 21-18. UART Receive FIFO Level Register (RFL).

Confidential For Mass Lang only 1 Feb 2026

21.3.2.16 UART_srr Register

Bits	Name	Access	Reset	Description
31:5				Reserved
2	**xfr**	W_SC	0	TX FIFO Reset This is a shadow register for the TX FIFO Reset bit (**UART_fcr[2]**). This can be used to remove the burden on the software to store previously written **UART_fcr** values (which are relatively static) to reset the transmit FIFO. This resets the control portion of the transmit FIFO and treats the FIFO as empty. This also deasserts the DMA TX request and single signals when additional DMA handshaking signals are selected. Note that this bit is self-clearing. It is not necessary to clear this bit. Dependencies are that writes have no effect when FIFO mode is not enabled.
1	**rfr**	W_SC	0	RX FIFO Reset This is a shadow register for the RX FIFO Reset bit (**UART_fcr[1]**). This can be used to remove the burden on the software to store previously written **UART_fcr** values (which are relatively static) to reset the receive FIFO This resets the control portion of the receive FIFO and treats the FIFO as empty. This also deasserts the DMA RX request and single signals when additional DMA handshaking signals are selected. Note that this bit is self-clearing. It is not necessary to clear this bit. Dependencies are that writes have no effect when FIFO mode is not enabled.
0	**ur**	W	0	UART Reset This asynchronously resets the UART and synchronously removes the reset assertion. For a two-clock implementation both PCLK and SCLK domains are reset. Note that this bit does not auto clear. When **ur** is asserted, the programmer should deassert **ur** as needed.

Table 21-19. UART Software Reset Register (SRR).

For Ambarella Confidential only
1.5 (Oct 2013)

21.3.2.17 UART_htx Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12:0	**halt_tx**	RW	0	This register is use to halt transmissions for testing, allowing the transmit FIFO to be filled by the master when FIFOs are implemented and enabled. 0 - Halt TX disabled 1 - Halt TX enabled Note that if FIFOs are implemented and not enabled, setting the halt TX register has no effect on operation. Dependencies are that writes have no effect when FIFO mode is not enabled.

Table 21-20. UART Halt TX Register (HTX).

21.3.2.18 UART_rtr Register

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	**rx_thd**	RW	0x0e	RX trigger Threshold. This is used to extend the value defined in rcvr_trigger of UART_fcr.

Table 21-21. UART RX Threshold Register (RTR).

21.3.2.19 UART_ttr Register

Bits	Name	Access	Reset	Description
31:6				Reserved
5:0	**tx_thd**	RW	0x08	TX Empty Trigger Threshold. This is used to extend the value defined in tx_empty_trigger of UART_fcr.

Table 21-22. UART TX Threshold Register (TTR).

21.3.2.20 UART_cpr Register

Bits	Name	Access	Reset	Description
31:24				Reserved
23:16	**fifo_ mode**	R	1	0x00 - 0 0x01 - 16 0x02 - 32 to 0x80 - 2048 0x81 - 0xFF - Reserved
15:14				Reserved

For Media Lang only (08012025) Confidential

Bits	Name	Access	Reset	Description
13	**dma_extra**	R	0	0 - False 1 - True
12	**uart_add_encoded_params**	R	1	0 - False 1 - True
11	**shadow**	R	1	0 - False 1 - True
10	**fifo_stat**	R	1	0 - False 1 - True
9	**fifo_access**	R	1	0 - False 1 - True
8	**additional_feat**	R	1	0 - False 1 - True
7	**sir_lp_mode**	R	0	0 - False 1 - True
6	**sir_mode**	R	0	0 - False 1 - True
5	**thre_mode**	R	1	0 - False 1 - True
4	**afce_mode**	R	1	0 - False 1 - True
3:2				Reserved
1:0	**apb_data_width**	R	2	0x0 - 8 bits 0x1 - 16 bits 0x2 - 32 bits 0x3 - Reserved

Table 21-23. UART Component Parameter Register (CPR).

21.3.2.21 UART_ucv Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12:0	**uart_component**	R	0x3130_302A	ASCII value for each number in the version. For example, 31_30_30_2A represents the version 1.00#

Table 21-24. UART Component Version Register (UCV).

For Meiya Zhang only (08/12/2025) Confidential 1.5

21.3.2.22 UART_ctr Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12:0	**peripheral_id**	R	0x414D_4241	This register contains the peripheral identification code.

Table 21-25. UART Component Type Register (CTR).

21.3.2.23 UART_state Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**tx_state_rdy**	R	0	TX state is ready. For debug usage.
6:4	**tx_state**	R	0	TX state. For debug usage. 0x00 – IDLE 0x01 – START 0x02 – DATA 0x03 – PARITY 0x04 – STOP
3	**rx_state_rdy**	R	0	RX state is ready. For debug usage.
2:0	**rx_state**	R	0	RX state. For debug usage. 0x00 – IDLE 0x01 – START 0x02 – DATA 0x03 – PARITY 0x04 – STOP

Table 21-26. UART_state Register.

21.3.2.24 UART_busy Register

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**tx_busy**	R	0	TX is busy. For debug usage.
0	**rx_busy**	R	0	RX is busy. For debug usage.

Table 21-27. UART Busy Register.

For Mediatek Confidential 1.5 / Oct 2025 / Hang only

22. I2C TWO-WIRE

This chapter covers the CV22FS I2C / IDC interface for two-wire communication as follows:

- (Section 22.1) I2C: Overview
- (Section 22.2) I2C: Interface Pins
- (Section 22.3) I2C: Master
- (Section 22.4) I2C: Slave

22.1 I2C: Overview

The I2C interface allows communication with external I2C devices. The CV22FS chip supports a bit rate of up to 374 Kbps. There are six I2C master controller interfaces (IDC0, IDC1, IDC2, IDC3, IDC4, and IDC5) and one I2C slave controller interface (IDCS).

IDC0, IDC1, IDC2, IDC3, IDC4, and IDC5, respectively, are located at base addresses 0xE400.8000, 0xE400.9000, 0xE400.A000, 0xE400.B000, 0xE401.9000 and 0xE401.A000 on the APB bus

IDCS, respectively, is located at base addresses 0xE400.7000.

22.2 I2C: Interface Pins

The interface consists of the pins in the following table. Refer to the CV22FS datasheet for a complete pin list.

Pin Name	Dir	Pad Type	Description
SSI0CLK	I/O	CMOS	First IDC serial port - clock (see GPIO3)
SSI0MOSI	I/O	CMOS	First IDC serial port - data (see GPIO4)
SSI0MISO	I/O	CMOS	Second IDC serial port - clock (see GPIO5)
SSI0EN0	I/O	CMOS	Second IDC serial port - data (see GPIO6)
SSI2CLK	I/O	CMOS	Third IDC serial port - clock (see GPIO8)
SSI2MOSI	I/O	CMOS	Third IDC serial port - data (see GPIO9)
SSI2MISO	I/O	CMOS	Fourth IDC serial port - clock (see GPIO10)
SSI2EN0	I/O	CMOS	Fourth IDC serial port - data (see GPIO11)
SSI3CLK	I/O	CMOS	Fifth IDC serial port - clock (see GPIO12)
SSI3MOSI	I/O	CMOS	Fifth IDC serial port - data (see GPIO13)
SSI3MISO	I/O	CMOS	Sixth IDC serial port - clock (see GPIO14)
SSI3EN0	I/O	CMOS	Sixth IDC serial port - data (see GPIO15)
IDCSCLK	I/O	CMOS	IDCS serial port - clock (see GPIO1)
IDCSDATA	I/O	CMOS	IDCS serial port - data (see GPIO2)

Table 22-1. IDC Interface (I2C-Compatible) Pins.

Confidential
For Sekra Lab Only

22.3 I2C: Master

22.3.1 I2C / IDC Master Turbo Mode Operation

The CV22FS I2C interface provides an operational mode which utilizes a 63-entry FIFO to buffer the commands and data bytes transmitted to a slave. This mode is referred to as “turbo mode” (and is also occasionally referred to as “burst mode”). Registers **IDC_fmcontrol** and **IDC_fmdata** are the write ports to the control portion and the data portion, respectively, of the FIFO. Super mode, an enhanced turbo mode, is only for write operations to reduce software configuration.

The **IDC_fmcontrol** and **IDC_fmdata** registers perform in a similar manner to the **IDC_control** and **IDC_data** registers, but with a difference: The **IDC_fmcontrol** and **IDC_fmdata** registers enable the software to write a batch of commands and data sequences rapidly and without the accompanying wait for each write onto the I2C bus to complete.

Note that when switching from turbo mode to non-turbo mode, the software must ensure that the FIFOs used during turbo mode are empty.

22.3.2 I2C / IDC Master Clock

The period for a given I2C controller is configured according to the following:

$$I^2C\ Clock\ Period = (gclk_apb) / ((4 + dutycycle[1:0]) * (prescale[15:0] + 1) + 2)$$

where the APB bus clock (gclk_apb) is used with a divider. The field prescale[15:0] equals the 16-bit value specified in the IDC_prescaleh and IDC_prescalel registers.

Additionally, when operating at high-speed mode, the register value of dutycycle[1:0] should set to 0 (it is equivalent to 2 based on I2C protocol). Then, the corresponding formula for high speed mode is as follows.

$$I^2C\ Clock\ Period = (gclk_apb) / ((6 * (prescale[15:0] + 1) + 2)$$

where the APB bus clock (gclk_apb) is used with a divider. The field prescalehs[7:0] equals to the 7-bit value specified in the IDC_prescalehs registers.

When operating at defined scl mode, SCL has defined N-phase, so the formula will be as follows:

$$I^2C\ Clock\ Period = (gclk_apb) / ((N * (prescale[15:0] + 1) + 2)$$

where the APB bus clock (gclk_apb) is used with a divider. N equals to the 4-bit value specified in the IDC_defined_scl_num registers.

Confidential For Mekra Lang only 1.5 (2021-02-25)

22.3.3 I2C Master Registers

22.3.3.1 I2C Master Registers: Map

Offset	Register	Description
0x00	**IDC_ENABLE**	Enable
0x04	**IDC_CONTROL**	Control
0x08	**IDC_DATA**	Data
0x0C	**IDC_STATUS**	Status
0x10	**IDC_PRESCALEL**	Prescaler (low byte)
0x14	**IDC_PRESCALEH**	Prescaler (high byte)
0x18	**IDC_FMCONTROL**	FIFO turbo mode control
0x1C	**IDC_FMDATA**	FIFO turbo mode data
0x20	**IDC_PRESCALEHS**	Prescaler for high-speed mode
0x24	**IDC_DUTYCYCLE**	Duty cycle setting for standard (non-high speed) mode
0x28	**IDC_STRETCH**	Extend SCL clock for START / STOP setup and hold timing
0x2C	**IDC_RX_FIFO**	RX FIFO for I2C turbo read
0x30	**IDC_DELAY_CNT**	Delay counter to delay the I2C protocol; used to synchronize near-end and far-end sensor enable time
0x34	**IDC_ST_BYP_LEN_LOW**	Stretch bypass region low byte
0x38	**IDC_ST_BYP_LEN_HIGH**	Stretch bypass region high byte
0x3C	**IDC_FM_NACK**	FIFO mode NACK log
0x40	**IDC_DEFINED_SCL_NUM**	I2C phase number for user defined mode
0x44	**IDC_DEFINED_SCL**	I2C non high-speed mode rising / falling phase configuration
0x48	**IDC_DEFINED_SCL_HS**	I2C high-speed mode rising / falling phase configuration
0x50	**IDC_FAULT_INJECT**	Fault injection test for I2C

Table 22-2. APB Registers of the I2C Interface.

Confidential
For Working Draft Only
(08 Jan 2025)

22.3.4 I2C Master Registers: Detail

22.3.4.1 IDC_enable Register

Bits	Name	Access	Reset	Description
31:4				Reserved
3	**irq_super_mask_en**	RW	0	0 - Super mode interrupt mask disabled 1 - Super mode interrupt mask enabled
2	**super_en**	RW	0	0 - Turbo write super mode disabled 1 - Turbo write super mode enabled
1	**scdc_en**	RW	0	0 - SCDC disabled 1 - SCDC enabled
0	**en**	RW	0	0 - I2C disabled (conserve power) 1 - I2C enabled

Table 22-3. I2C Enable Register.

22.3.4.2 IDC_control Register

Bits	Name	Access	Reset	Description
31:7				Reserved
6	**irq_scdc**	RW		Write 1 to clear
5	**clear**	WO		Write 1 to clear the 63-entry FIFO used during Turbo Mode
4	**hs_mode**	RW	0	1 - Enables High-Speed Mode
3	**stop**	RW	0	1 - Generates a STOP condition
2	**start**	RW	0	1 - Generates a START condition
1	**if**	RW	0	Interrupt flag. The software clears this flag by writing 0 0 - No interrupt 1 - Interrupt pending
0	**ack**	RW	0	Read: When receiving, 0/1 indicates ACK/NACK needs to be transmitted to the slave device. When transmitting, 0/1 indicates ACK/NACK received from slave. Write: Write 0/1 to transmit ACK/NACK to slave device.

Table 22-4. I2C Control Register.

22.3.4.3 IDC_data Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**data**	RW	0	I2C data register for single-byte operations only Write: Send data Read: Receive data

Table 22-5. I2C Data Register.

22.3.4.4 IDC_status Register

Bits	Name	Access	Reset	Description
31:14				Reserved
13:8	**fifoct**	R	0	FIFO count of empty entries
7:4				I2C Finite State Machine (FSM) status IDLE = 4'd0 START_COMMAND = 4'd1 TRANSMIT_DATA = 4'd2 RECEIVE_ACK = 4'd3 COMMAND_FINISH = 4'd4 STOP_COMMAND = 4'd5 RECEIVE_DATA = 4'd6 TRANSMIT_ACK = 4'd7 WAIT_DATA = 4'd8 READ_LENGTH = 4'd9
3				Reserved

Confidential
For Mekra Lang only
1.5 (Oct 2025)

Bits	Name	Access	Reset	Description
2	**fifoemp**	R	0	FIFO empty
1	**fifofull**	R	0	FIFO full
0	**mode**	R	0	0 - Master transmitter 1 - Master receiver

Table 22-6. I2C Status Register.

22.3.4.5 IDC_prescalel / IDC_prescaleh Registers

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**scale**	RW	0	Prescaler for non high-speed mode

Table 22-7. I2C Prescaler Registers (Low and High).

22.3.4.6 IDC_fmcontrol / IDC_fmdata Registers

The **IDC_fmcontrol** and **IDC_fmdata** registers are the write ports to the control and data portions of the FIFO when using Turbo Mode.

The following is an example of writing 61 data words to the I2C device using the **IDC_fmcontrol** and **IDC_fmdata** registers.

```
@ program base address of idc
  mov  r0, #APB_BASE_ADDRESS          @ load up idc base address
register
  orr  r0, r0, #IDC_DEVICE_OFFSET
  mov  r1, #0x01          @ configure prescale registers
  str  r1, [r0, #IDC_PRE_SCALE_LOW]
  mov r1, #0x01                 @ enable the module
  str  r1, [r0, #IDC_ENABLE]
  mov     r1, #0x04       @ push START command to FIFO
  str     r1, [r0, #IDC_FMCTRL] @ place slave address and r/w bit
into FIFO
  mov     r1, #0x2A       @ slave address = 0x15 / r/w bit = 0 (w)
  str     r1, [r0, #IDC_FMDATA]
  mov  r1, #0x0
fill_data_FIFO:                   @ place 61 data items in FIFO
  str     r1, [r0, #IDC_FMDATA]
  add r1, r1, #0x01
  cmp  r1, #0x3d
  bne   fill_data_FIFO
  mov     r1, #0x0A       @ push INT/STOP command to control FIFO
  str     r1, [r0, #IDC_FMCTRL]
```

Software uses the following two methods to verify that all data in FIFO has been transmitted.

1. Polling: Software polls the “stop” flag [1] in **IDC_control** register.
2. Wait for interrupt: When an interrupt asserts, software polls “STOP_COMMAND or IDLE” for its

state in the **IDC_status** register [7:4] to verify that the STOP command has been sent.

After verifying that the data has been transmitted, software writes 0x0 into **IDC_control** to clear IF. When the burst data is more than 61 bytes, the software splits the data into multiple 61-byte chunks, requiring that the software repeat the procedure described above. (Note that START / DATA/ STOP should be pushed into FIFO for each chunk.)

Turbo read mode is also applied. RX_FIFO can store up to 8 pieces of data, and send out continuously.

For super mode (compared to the original packet which includes START, the STOP command, and data sequences), the new packet type includes only the length of the packet and data sequences. If the length is equal to 0, then the corresponding packet includes one data sequence (slave address). If length is equal to 1, then the corresponding packet includes two data sequences (slave address and one piece of data). The length indicates the number of data without the corresponding address. Therefore, the total number of IDC_fmdata should be transmitted after the length information(N) is N plus one.

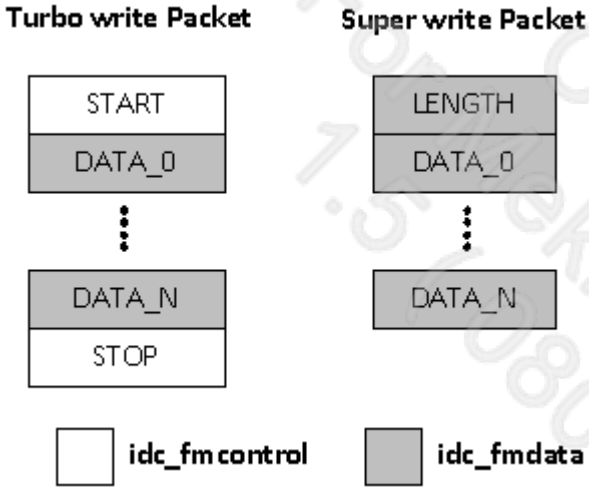


Figure 22-1. I2C Packet Type.

While FIFO contains data, the START signal activates. Alternatively, the STOP signal activates after the data transmission completes. Each packet transmitted includes its length, address, and data sequences.

When turbo mode is enabled, the last output of FIFO includes the **IDC_fmcontorl** signal, including interrupt information. However, if super mode is enabled, only **IDC_fmdata** is transmitted. Without the interrupt information, the software or high-level system cannot determine if the transmission is complete. As a result, when **irq_super_mask_en** is enabled, **IDC_fmdata** (which should be the length) is equal to ‘d255 (‘hff). Then, the internal FIFO interrupt is triggered and sets the IRQ at a high-level.

Note that if the hardware does not receive an acknowledgment from the device in executing a sequence of writes (e.g., the address is incorrect), the I2C hardware will hang. To avoid this occurrence, the software should implement an appropriate time-out and error recovery mechanism. In recovering the I2C hardware from a hang state, the software should clear the FIFO.

Confidential
For Ambarella Lang only
1.5 Rev 08012025

Bits	Name	Access	Reset	Description
31:8				Reserved
7:5	**rdcnt**	WO	0	Only for turbo read mode, after transmitting START and slave address but before STOP, read data count up to 8 and rfifo_mode will store in fmcontrol registers to control read behavior in idc_rfifo (0x2c)
4	**hs_mode**	RW	0	1 - Enables High-Speed Mode
3	**stop**	RW	0	1 - The next process generates a STOP condition
2	**start**	RW	0	1 - The next process generates a START condition
1	**is**	RW	0	Enable bit for FIFO mode Assert interrupt to ARM at the completion of this command
0	**rfifo_mode**	WO	0	Only for turbo read mode, after transmitting START and slave address but before STOP, read data count up to 8 and rfifo_mode will store in fmcontrol registers to control read behavior in idc_rfifo (0x2c)

Table 22-8. I2C Turbo-Mode Control Register.

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**fmdata**	RW	0	Data for turbo mode

Table 22-9. I2C Turbo-Mode Data Register.

22.3.4.7 IDC_prescaler_hs Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**prescalerhs**	RW	0	Prescaler for high-speed mode

Table 22-10. I2C Turbo-Mode Prescaler Register.

22.3.4.8 IDC_dutycycle Register

Bits	Name	Access	Reset	Description
31:5				Reserved
4	**defined_scl_mode**	RW	0	1 – Enable defined SCL mode
3	**sda**	RW	1	SDA (data line) pin current source enable
2	**scl**	RW	1	SCL (clock line) pin current source enable
1:0	**dutycycle**	RW	1	Duty cycle setting for standard (non-high speed) mode 0 - 1:1 1 - 2:3 2 - 1:2

Table 22-11. I2C Duty Cycle Register.

In order to meet the I2C timing specification and generate a variety of SCL pulses, the SW can configure **IDC_defined_scl_num** register (0x40) to determine the number of phases under high-speed mode, and **IDC_defined_scl**

For Ambarella Lang only Confidential 1.5 (Oct 2025)

register (0x44)/ **IDC_defined_scl_hs** register (0x48) to control the rising and falling phases of SCL. With divided N-phase of SCL, trise_phase (Nr), and tfall_phase (Nf), the SW can generate various and precise SCL to meet the I2C timing specification under a different mode. Note that the data change time is always the next phase to the falling phase with two APB clock cycle delay.

Moreover, defined SCL mode is also applied to high-speed mode, but limited by duty cycle equal to 33.3% (tHIGH : tLOW = 1:2). Setup / Hold time for START or STOP can be extended by configuring the **IDC_stretch** register (0x28) to meet timing specification.

$$t_{HIGH} \approx T_{SCL} \times \frac{N_f - N_r}{N_{phase_num}},$$

$$t_{LOW} \approx T_{SCL} \times \frac{N_{phase_num} + N_r - N_f}{N_{phase_num}},$$

$$t_{HD;DAT} \approx T_{SCL} \times \frac{1}{N_{phase_{num}}} + 2 \times T_{apb_clk},$$

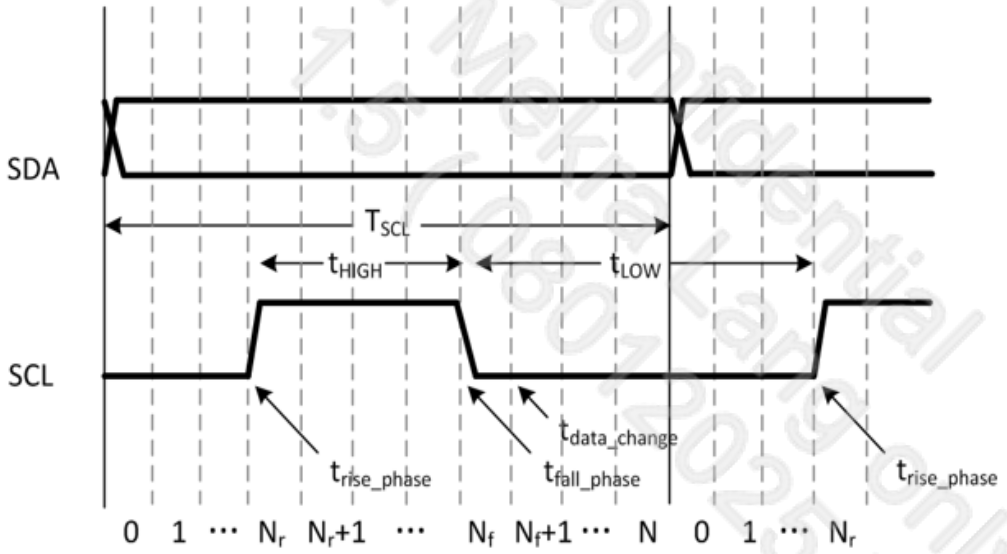


Figure 22-2. Defined SCL Clock and Timing.

22.3.4.9 IDC_stretch Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:4	**stretch_scl_hs**	RW	0	Extend the high-speed mode SCL clock START/STOP setup and hold timing
3:0	**stretch_scl**	RW	0	Extend the non high-speed mode SCL clock START/STOP setup and hold timing

Table 22-12. I2C SCL Clock Stretch Register.

22.3.4.10 IDC_rx_fifo Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**rfifo_data**	RW	0	Data for turbo read mode

Table 22-13. I2C RX FIFO Register.

The following is an example of reading 8 data bytes of I2C devices using **idc_rx_fifo**.

```
// Turbo Read (HS-mode)
// idcm disable
write $Hex{I2C1_DEVICE_BASE()+I2C_ENABLE()} 0x0
// start
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_CTRL()} 0x4
// HS master code (00001xxx)
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_DATA()} 0xF
// start, hs-mode
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_CTRL()} 0x14
// slave address = 0x42, r/w bit (r=1)
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_DATA()} 0x85
// data count = 8, hs, rfifo
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_CTRL()} 0xF1
// stop, interrupt, hs
write $Hex{I2C1_DEVICE_BASE()+I2C_FIFO_CTRL()} 0x1A
// idcm enable
write $Hex{I2C1_DEVICE_BASE()+I2C_ENABLE()} 0x1

wait_irq 0 $Hex{I2C1_IRQ()}
// read out data from RX FIFO
                :gen_ne_read(address=>I2C1_DEVICE_BASE()+I2C_RX_FIFO_DATA(),
               data=>\@rdata, incr=>0, hsize=>0);
```

22.3.4.11 IDC_delay_cnt Register

Bits	Name	Access	Reset	Description
31:22				Reserved
21:0	**delay_cnt**	RW	0	Delay counter to delay the I2C protocol. This is used to synchronize near-end and far-end sensor enable time. Controller will wait delay_cnt*apb_clock_period to send out the command.

Table 22-14. I2C Delay Counter Register.

22.3.4.12 IDC_st_byp_len_low / IDC_st_byp_len_high Register

To improve the accuracy of the I2C clock frequency formula, Ambarella has added two “stretching bypass” registers to the CV22FS I2C module: **IDC_st_byp_len_low / IDC_st_byp_len_high**.

The following figure illustrates the clock generation of the I2C controller, using the formula

I^2C Clock Frequency = (gclk_apb) / ((4 * (prescale[15:0] + 1) + 2).

Confidential Lang ong (08012023)

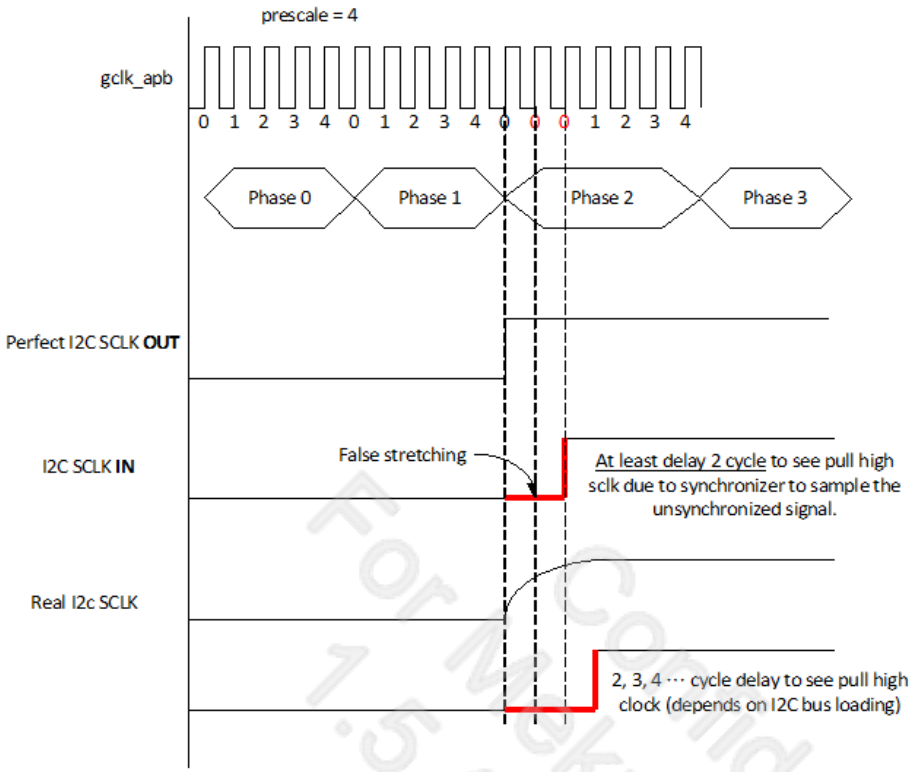


Figure 22-3. I2C Clock Generation.

To account for the “false stretching” shown in the diagram, the I2C clock frequency formula should be changed to

I²C Clock Frequency = (gclk_apb) / ((4 * (prescale[15:0] + 1) + 2 + X),

where X is dependent upon the I2C BUS RC delay.

To address this issue, Ambarella has added programmable stretching bypass length Z[15:0] (in units of gclk_apb) to effectively bypass the false stretching shown above.

Refer to the figure on the following page for an illustration of the stretch bypass region.

For Nekra Lang only
1.5 (08 01 2025)
Confidential

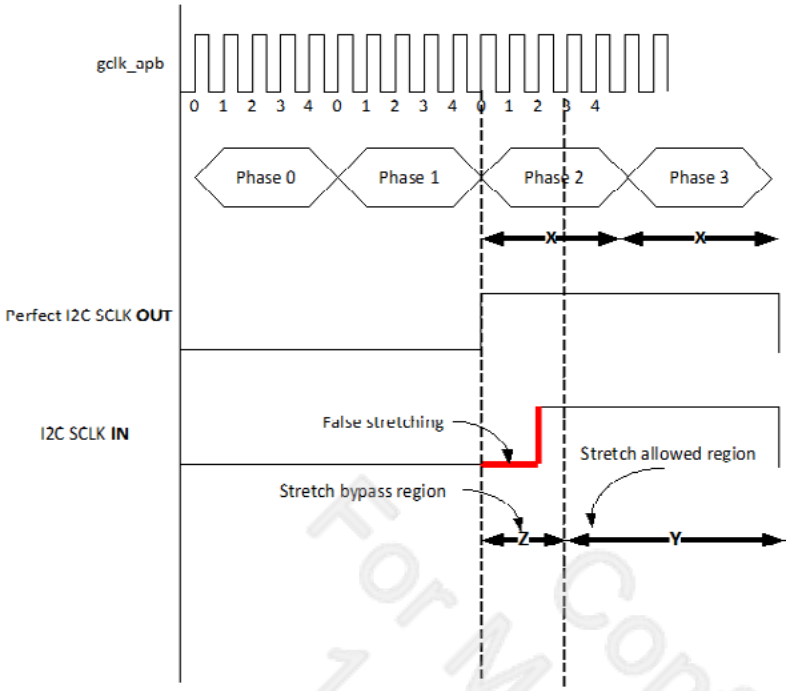


Figure 22-4. I2C Stretch Bypass Region.

If the SCLK IN signal equals 0 within the stretch bypass region Z, the I2C controller will not treat it as a stretch, and Phase 2 will not be extended.

Alternatively, if the SCLK IN signal equals 0 outside of the stretch bypass region Z, the I2C controller will treat this as an actual stretch event and extend Phase 2 accordingly. Ensure that Y = 2X-Z >= tHigh (High period of the SCL clock).

Using Standard Mode as an example: When SCLK = 100 kHz, tHigh = 4 us. The SCLK clock period = 1/100kHz = 10us, x = 2.5 us, Y = 2*2.5 – Z > 4 us. In this case, Z < 1 us; otherwise, the setting would result in a tHigh violation.

Z = {**stretch_bypass_high**[7:0], **stretch_bypass_low**[7:0]} * gclk_apb clock period.

Note that when Z is set to 0, the controller will behave as shown in the I2C Clock Generation figure.

The two additional registers (**IDC_st_byp_len_low** and **IDC_st_byp_len_high**) are described in the following tables.

22.3.4.13 IDC_st_byp_len_low Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**stretch_byp_low**	RW	0	Stretch bypass region low byte

Table 22-15. I2C Stretch Bypass Region Low Register.

22.3.4.14 IDC_st_byp_len_high

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**stretch_ byp_ high**	RW	0	Stretch bypass region high byte

Table 22-16. I2C Stretch Bypass Region High Register.

22.3.4.15 IDC_fm_nack Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**nack_log**	RW	0	0 - Transmitting/receiving without NACK in turbo/super mode 1 - Transmitting/receiving with NACK in turbo/super mode

Table 22-17. I2C FIFO Mode Nack Log Register.

22.3.4.16 IDC_defined_scl_num Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:4	**phase_num_ hs**	RW	0xB	Number of phase of SCL at HS mode Number = phase_num_hs + 1 Minimum of phase_num_hs should be 5. Number should be a multiple of 3.
3:0	**phase_num**	RW	0xE	Number of phase of SCL Number = phase_num + 1 Minimum of phase_num should be 3

Table 22-18. I2C Defined SCL Mode Phase Number Register.

22.3.4.17 IDC_defined_scl Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:4	**falling_phase**	RW	0x8	Falling phase of SCL
3:0	**rising_phase**	RW	0x1	Rising phase of SCL

Table 22-19. I2C Defined SCL Mode Phase Control Register.

For Mekra Lang only (2025-05-15)

22.3.4.18 IDC_defined_scl_hs Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:4	**falling_ phase_hs**	RW	0x8	Falling phase of SCL at HS mode
3:0	**rising_ phase_hs**	RW	0x4	Rising phase of SCL at HS mode

Table 22-20. I2C Defined SCL High Speed Mode Phase Control Register.

22.3.4.19 IDC_fault_inject Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**fault_inject**	RW	0	1 – Fault injection to IDC

Table 22-21. I2C Fault Injection Register.

22.4 I2C: Slave

22.4.1 IDC Slave (IDCS) Registers Map

Register Offset	Register Name	Description
0x00	**IDCS_enable**	Enable
0x04	**IDCS_control**	Control
0x08	**IDCS_data**	Data
0x0C	**IDCS_status**	Status
0x10	**IDCS_fifo_cnt**	FIFO data count
0x14	**IDCS_rx_fifo_th**	FIFO data count threshold for receiver
0x18	**IDCS_tx_fifo_th**	FIFO data count threshold for transmitter
0x1C	**IDCS_t_hd_data**	Data hold time
0x20	**IDCS_slave_addr**	Slave address
0x24	**IDCS_scl_timer**	Stretch clock timer
0x28	**IDCS_timeout_status**	Timeout status
0x2C	**IDCS_debug**	Debug information
0x30	**IDCS_ack_control**	ACK control
0x34	**IDCS_irq**	IRQ status
0x38	**IDCS_fault_inject**	Fault injection to IDCS

Table 22-22. Registers of the IDCS Interface.

For Mekra Lang only (080 only) 1.5 Confidential

22.4.2 IDC Slave (IDCS) Registers Detail

22.4.2.1 IDCS_enable Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**idcs_enable**	RW	0	0 - IDCS disabled 1 - IDCS enabled IDCS will complete the transfer in progress after it is disabled.

Table 22-23. IDCS Enable Register.

22.4.2.2 IDCS_control Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7	**idcs_reset**	W	0	IDCS software reset when 1'b1 is written to this field
6	**idcs_irq_ clear**	W	0	Clear the interrupt when 1'b1 is written to this field
5	**idcs_fifo_ clear**	W	0	Clear all the data in FIFO
4	**idcs_irq_ time- out**	RW	0	When set, the interrupt will be asserted if the internal timer reaches idcs_scl_timer, specified in the clock stretching timer (0x24)
3	**idcs_irq_ fifo_th**	RW	1	Slave-receiver mode: If the data count in FIFO is more than the thresh-old value, the interrupt will be asserted. Slave-transmitter mode: If the data count in FIFO is less than or equal to the threshold value, the interrupt will be asserted.
2	**idcs_irq_p_ sr**	RW	0	This bit is used only in slave-receiver mode. When set, the interrupt will be asserted if IDCS detects stop or repeated start condition.
1	**idcs_irq_ enable**	RW	1	An interrupt can be asserted only when idcs_irq_enable = 1'b1.
0	**idcs_ack**	RW	0	Slave-receiver mode: 1'b0 indicates an ACK has been received. Slave-transmitter mode: 1'b0 indicates an ACK needs to be transmit- ted.

Table 22-24. IDCS Control Register.

22.4.2.3 IDCS_data Register

Bits	Name	Access	Reset	Description
31:9				Reserved
8	**idcs_first_ byte**	R	0	This bit is used only in slave-receiver mode. 1'b1 indicates the data is the first byte after the start or the repeated start condition.
7:0	**idcs_data**	RW	0	Slave-receiver mode: Data received Slave-transmitter mode: Data to be transmitted

Table 22-25. IDCS Data Register.

22.4.2.4 IDCS_status Register

Bits	Name	Access	Reset	Description
31:9				Reserved
8	**idcs_time-out**	RW	0	1'b1 indicates time-out condition occurs for the clock stretching timer
7	**idcs_stop**	RW	0	1'b1 indicates that a STOP condition is received. The bit should be cleared by writing 1'b0.
6	**idcs_re-peated_ start**	RW	0	1'b1 indicates that a repeated START condition is received
5	**idcs_fifo_ th_vld**	R	0	Slave-receiver mode: 1'b1 if FIFO data count > **idcs_rx_fifo_th**
4	**idcs_sel**	R	0	1'b1 indicates IDCS is selected by a master
3	**idcs_gen-eral_ call**	R	0	1'b1 indicates the current transfer is a general call address
2	**idcs_fifo_ full**	R	0	1'b1 indicates the FIFO is full
1	**idcs_fifo_ emp- ty**	R	0	1'b1 indicates the FIFO is empty
0	**idcs_rx_tx_ state**	R	0	1'b1: Slave-receiver mode 1'b0: Slave-transmitter mode

Table 22-26. IDCS Status Register.

22.4.2.5 IDCS_fifo_cnt Register

Bits	Name	Access	Reset	Description
31:7				Reserved
6:0	**idcs_fifo_cnt**	R	0	The register shows the number of valid entries in the FIFO.

Table 22-27. IDCS FIFO Count Register.

22.4.2.6 IDCS_rx_fifo_th Register

Bits	Name	Access	Reset	Description
31:7				Reserved
6:0	**idcs_rx_ fifo_th**	RW	0	The register is valid in slave-receiver mode. If the number of data in FIFO is larger than **idcs_rx_fifo_th** and **idcs_ irq_fifo_th** = 1'b1, the interrupt will be asserted.

Table 22-28. IDCS FIFO Data Count Threshold for Receiver Register

For Makra Lang Only (08012025) Confidential 1.5

22.4.2.7 IDCS_tx_fifo_th Register

Bits	Name	Access	Reset	Description
31:7				Reserved
6:0	**idcs_tx_fifo_th**	RW	0	The register is valid in slave-transmitter mode. If the number of data in FIFO is less than or equal to **idcs_tx_fifo_th** and **idcs_irq_fifo_th** = 1'b1, the interrupt will be asserted.

Table 22-29. I2CS FIFO Data Count Threshold for Transmitter Register.

22.4.2.8 IDCS_t_hd_data Register

Bits	Name	Access	Reset	Description
31:16				Reserved
15:8	**idcs_hs_t_hd_dat**	RW	0	Data hold time setup for slave-transmitter at high speed mode. • tHD;DAT = idcs_hs_t_hd_dat x tapb_clk • tapb_clk is the APB clock period
7:0	**idcs_t_hd_dat**	RW	0	Data hold time setup for slave-transmitter. • tHD;DAT = idcs_t_hd_dat x tapb_clk • tapb_clk is the APB clock period

Table 22-30. IDCS Data Hold Time Register.

22.4.2.9 IDCS_slave_addr Register

Bits	Name	Access	Reset	Description
31:7				Reserved
6:0	**idcs_slave_addr**	RW	0	7-bit slave address

Table 22-31. IDCS Slave Address Register.

22.4.2.10 IDCS_scl_timer Register

Bits	Name	Access	Reset	Description
31:0	**idcs_scl_timer**	RW	0	32-bit timer for SCL stretching0x00000000: No SCL stretching. The slave will return NACK if it is busy or is not ready. 0xffffffff: SCL will be held low until the slave is ready. 0 < idcs_scl_timer < 0xffffffff: The slave will return NACK when the internal counter reaches **idcs_scl_timer.**

Table 22-32. IDCS Stretching Clock Timer Register.

For Mokra Lang only (08/12/2025) Proprietary and Confidential

22.4.2.11 IDCS_timeout_status Register

Bits	Name	Access	Reset	Description
31:4				Reserved
3	**idcs_time-out_ err_3**	RW	0	The interrupts from stop or repeated start condition are not cleared.
2	**idcs_time-out_ err_2**	RW	0	The received data in FIFO is not read, but IDCS is switched to trans-mitter mode.
1	**idcs_time-out_ err_1**	RW	0	FIFO is full in receiver mode
0	**idcs_time-our_ err_0**	RW	0	FIFO is empty in transmitter

Table 22-33. IDCS Time-out Status Register.

22.4.2.12 IDCS_debug Register

Bits	Name	Access	Reset	Description
31:13				Reserved
12	**idcs_time-out_ nack**	RO	0	The The slave will return NACK when the internal counter reaches idcs_scl_timer.
11	**idcs_err3_ int_busy**	RO	0	The interrupts from stop or repeated start condition are not cleared.
10	**idcs_err2_rx_ fifo_busy**	RO	0	The received data in FIFO is not read, but IDCS is switched to trans-mitter mode.
9	**idcs_err1_rx_ fifo_full**	RO	0	FIFO is full in receiver mode
8	**idcs_err0_tx_ fifo_empty**	RO	0	FIFO is empty in transmitter
7	**idcs_scl_ hold_low**	RO	0	SCL should keep low after transfer is done if error occurs.
6:3	**idcs_bit_cnt**	RO	0	Bit counter of SDA output
2:0	**idcs_state**	RO	0	IDCS finite state machine (FSM) status. ST_IDLE = 3'd0 ST_RECEIVE_ADDR = 3'd1 ST_RECEIVE_DATA = 3'd2 ST_TRANSMIT_DATA = 3'd3 ST_BYTE_END = 3'd4

Table 22-34. IDCS Debug Register.

22.4.2.13 IDCS_ack_control Register

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**idcs_ack_oen**	RW	0	

Confidential For Ambarella Only (c) 2020

Bits	Name	Access	Reset	Description
0	idcs_ack_value	RW	0	0 – ACK1 – NACK

Table 22-35. IDCS ACK Control Register.

22.4.2.14 IDCS_irq Register

Bits	Name	Access	Reset	Description
31:4				Reserved
3	timeout_int	RO	0	
2	tx_fifo_th_int	RO	0	
1	rx_fifo_th_int	RO	0	
0	p_sr_int	RO	0	

Table 22-36. IDCS IRQ Status Register.

22.4.2.15 IDCS_fault_inject Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	fault_inject	RW	0	Fault injection to IDCS

Table 22-37. IDCS Fault Injection Register.

Confidential For Mekra Lang only 1.5 (08012025)

23. CONTROLLER AREA NETWORK (CAN) BUS

This chapter overviews the CV22FS CAN bus controller, a module used by the DSP to communicate with other chips via a controller area network (CAN) bus. The CV22FS CAN bus controller implements a specific protocol that allows DSP configuration information to be formatted and transmitted through a CAN interface (composed of a digital TX pin and a digital RX pin) to a far-end CAN node. Ambarella's CAN bus controller supports CAN flexible data-rate (FD), a CAN data-link layer protocol developed by Bosch.

The proposed CAN controller provides up to 32 message buffers; the 32 message buffers can be configured for TX or RX usage, individually. For each instance, the message buffer can be used either as a TX buffer or an RX buffer only, not both simultaneously. When message buffer priority is not configured, message buffer #0 takes the highest priority, and message buffer #31 takes the lowest.

Notes:

- Programmers should use only classic CAN or CAN FD frame formats. Mixed format network configurations are *not* allowed.
- If the RX buffer overflow, the CAN controller will transmit a cyclic redundancy check (CRC) error packet. Although this not defined in the spec, programmers should prevent this occurrence.

The chapter is organized as follows.

- (Section 23.1) CAN Controller: Overview
- (Section 23.2) CAN Controller: Cluster Function Descriptions
- (Section 23.3) CAN Controller: DMA
- (Section 23.4) CAN Controller: Register Detail
- (Section 23.5) CAN Controller: Safety Notes

23.1 CAN Controller: Overview

23.1.1 Diagram of the Subsystem

The following two diagrams illustrate the functionality of the CV22FS CAN bus controller, including the principal interfaces involved in the data exchange. Note that CV22FS is equipped with six CAN controllers.

For Mekra Lang only (Confidential 1.5 (Jan 2025))

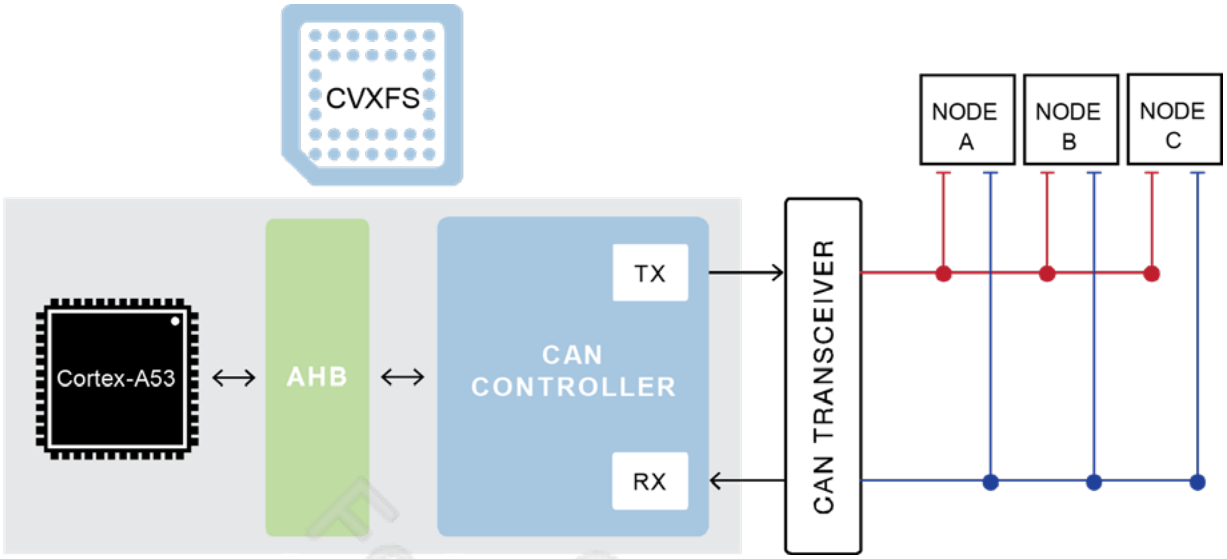


Figure 23-1. CAN Bus Controller Subsystem.

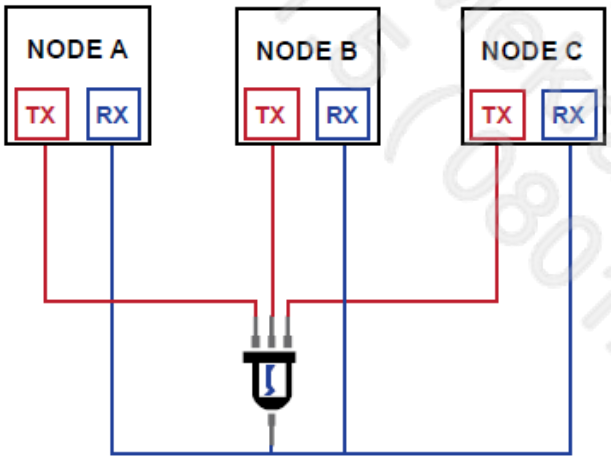


Figure 23-2. CAN Bus Controller: Logical View of the Digital Link Layer.

For Media Lang only
15 (08012025)
Confidential

23.1.2 Key Definitions

- **Message Buffer**: The buffer used to store the TX or RX packet.
- **TT CAN**: Time-Triggered CAN Protocol.

23.2 CAN Controller: Cluster Function Descriptions

This section provides diagrams and descriptions of the principal functional blocks of the CV22FS CAN bus controllers.

23.2.1 Functional Overview

The following figure provides an illustration of the CAN bus controller data flow.

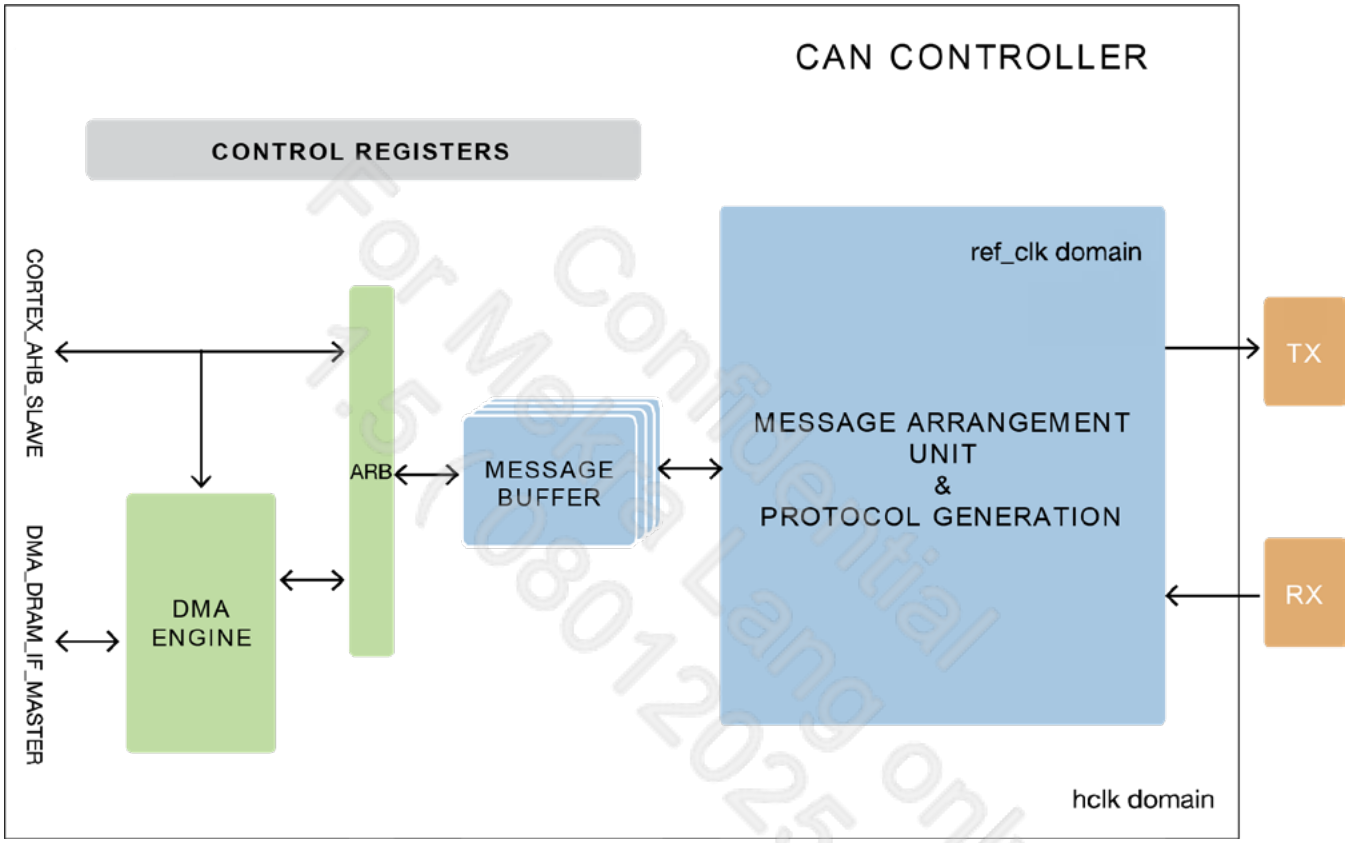


Figure 23-3. CAN Bus Controller: Functional Overview.

Notes:

- There are two clock domains in the CAN controller: hclk and ref_clk. The message buffer is accessed through AHB synchronously in hclk domain; ref_clk is used to generate the CAN bus signal to TX and sample the CAN bus signal from RX.
- Both transmitted and received data are placed in message buffers via the AHB bus.
- There are two clock domains in the CAN bus controller. The **ref_clk** domain is used to generate the CAN bus signal to the TX block and to sample the CAN bus signal from the RX block.
- The message buffer is accessed by the AHB, the TX block, and the RX block. The access priority ranking is as follows: TX > RX > AHB.

For Media Lang only 1.5 (08012025) Confidential

23.3 CAN Controller: DMA

23.3.1 DMA: One Message Per Descriptor

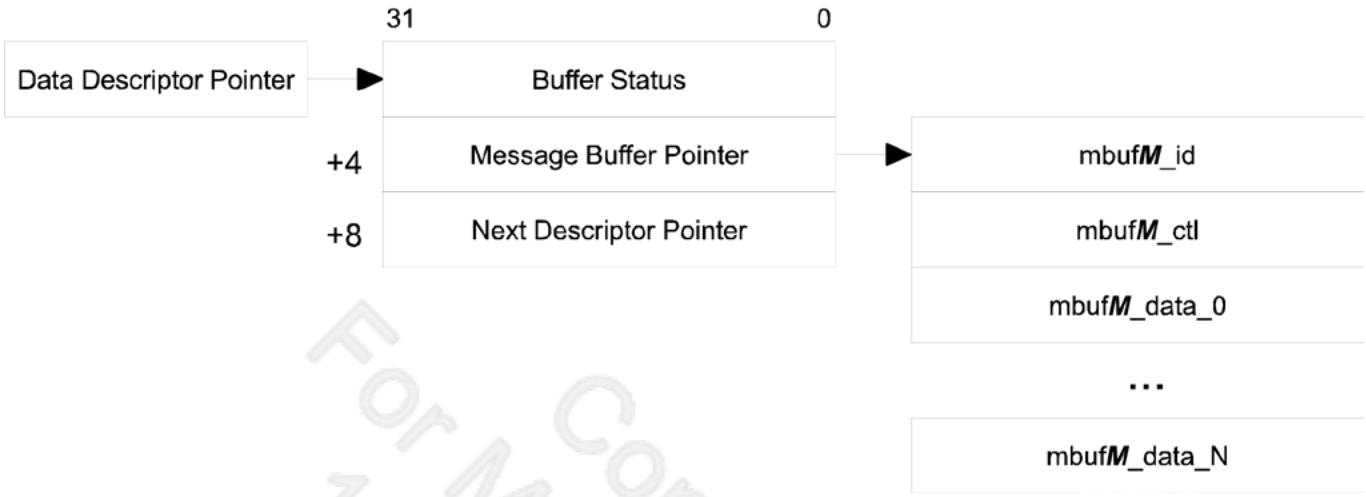


Figure 23-4. Block Diagram of RX DMA Descriptor.

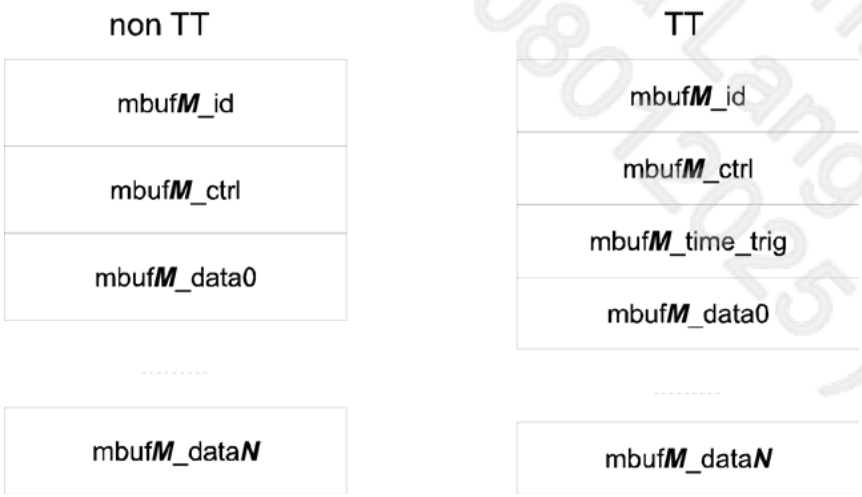


Figure 23-5. Data Part of TTCAN and NON-TTCAN.

Confidential
For Mekra Lang only
1 5 (0 8 0 2 2 0 2 5)

23.3.2 DMA: RX Buffer Status

Register	Field	Bits	Access	Reset	Description
Buffer Status	**DMA Status**	31:30	RW	0	2'b00: Host Ready 2'b01: DMA Busy 2'b10: DMA Done 2'b11: Host Busy
	Received Status	29:28	RW	-	2'b00: Success 2'b01: DESERR 2'b10: Reserved 2'b11: BUFERR
	Last	27	RW	0	0: 1: Indicate that this descriptor is the last one of the chain
	Reserved	26:8	RW		
	Received Number Of Bytes	7:0	RW	0	Record number of bytes received from CANC

Table 23-1. Buffer Status.

23.3.3 DMA Usage Flow

1. Configure CANC controller's message buffer's attribute (TX or RX, **id_filter** for RX).
2. Program **msg_buf_config_done** (0x80). Set **canc_en** (0x38) to 1.
3. Program RX Data descriptor pointer (0x504, 0x508) if the DMA is going to be enabled.
4. Program 0x50C, 0x510, and 0x514 for RX DMA.
5. Program **rx_dma_en** to 1.

For Mekra Lang only Confidential 1.5 (Oct 2025)

23.3.4 RX DMA Flow

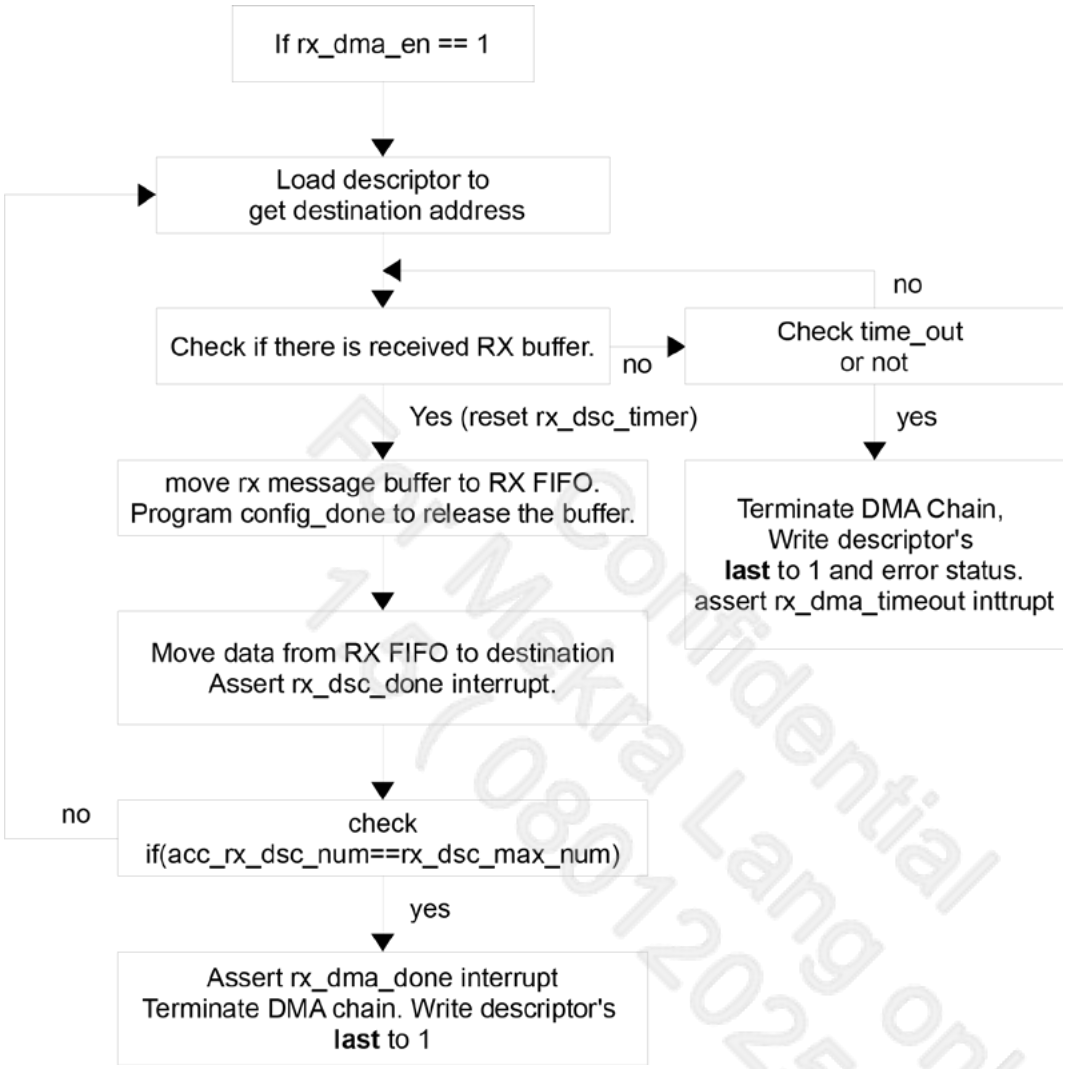


Figure 23-6. RX DMA Flow.

23.4 CAN Controller: Register Detail

This section describes the register interface for the six CV22FS CAN bus controllers.

The following table provides a summary of the CAN bus controller registers.

Offset	Register	Field	Bits	Access	Reset	Description
0x00	ctrl_reg	can_fd	31	RW	0	0: CAN 2.0 1: CAN FD (FD and 2.0)
		Reserved	30:4	-	-	Reserved
		auto_resp_mode	3	RW	0	0: Disable the auto-response feature 1: The controller will automatically compare the remote request frame's ID with the TX buffer's ID. If they match, TX would automatically transmit the TX messages. The corresponding received remote message would not be put in the RX message buffer.
		listen_mode	2	RW	0	0:normal mode 1:rx listening only and don't transmit any message
		loopback_mode	1:0	RW	0	0: Normal mode 1: Loopback mode inside the chip for self testing. 2. Loopback mode outside the chip for self testing. 3: Reserved
0x04	cfg_ttcan	Reserved	31:1	-	-	
		tt_can_en	0	RW	0	0: TT CAN disable 1: TT CAN enable
0x08	rst_reg	Reserved	31:1	-	-	
		rst_canc_dma	2	RW	0	Software resets the CAN DMA engine only
		rst_canc	1	RW	0	Software resets the CAN controller only
		rst_canc_sys	0	RW	0	Software resets the CAN controller and DMA engine.
0x0C	fd_ssp_reg	Reserved	31:24	-	-	Reserved
		ssp_offset_m1	23:16	RW	0	ssp_offset_m1 is used when ssp_position mode is set to 0. The SSP position = delay_measurement (in TQ) + ssp_offset (in reference clock period) 0: 1 reference clock period 1: 2 reference clock period … 255: 256 reference clock period

For Mikra Langenbach 1st (08.12.2025) Confidential

Offset	Register	Field	Bits	Access	Reset	Description
0x0C (Cont'd)	fd_ssp_reg	ssp_position	15:8	RW	0	0: 0 minimum time quanta 1: 1 minimum time quanta .. 255: 255 minimum time quanta The position of SSP shall be specified by its distance from the start of the bit time. if SSP position value is an odd number and the value of the prescaler for the data time quantum is 2, then SSP position may be determined by dividing the SSP position value by 2 and rounding down the result.
		Reserved	7:2	-	-	Reserved
		ssp_position_mode	1	RW	0	0: Delay Measurement mode 1: Fixed Value mode When the Delay Measurement mode is used, the SSP position shall be the sum of the measured transmitter delay value and the configurable SSP offset. When the Fixed Value mode is used, ssp_position should be programmed to a fixed value.
		tx_delay_compensate_enable	0	RW	0	0: Disable 1: Enable This mechanism shall only be used by transmitters in the data phase of the FD frame, when BRS is recessive.
0x10	tq_reg	sample_times	31	RW	0	0: 1-time sampling 1: 3-times sampling
		sample_delay	30:23	RW	0	Delay the sample point in the unit of ref_clk cycle
		sync_mode	22	RW	0	0: Only falling edge is used for synchronization 1: Both edges are used for synchronization
		tq_pre_m1	21:14	RW	0	Time quanta prescaler for nominal bit time 0:1x 1:2x ... 255:256x

Confidential For Mekra Lang (0801205) 1.5

Offset	Register	Field	Bits	Access	Reset	Description
0x10 (Cont'd)	**tq_reg**	**sjw_m1**	13:10	RW	0	SJW minus one for nominal bit time SJW = sjw_m1 +1 0: 1 … 15:16
		tseg1_m1	9:4	RW	0	Prop_seg + Phase_seg1 for nominal bit time (Prop_seg + Phase_seg1) = tseg1_m1 +1 0: 1 … 63: 64
		tseg2_m1	3:0	RW	0	Phase_seg2 for nominal bit time Phase_seg2 = tseg2_m1+1 0:1 … 15:16
0x14	**tq_fd_reg**	**Reserved**	31:21	-	-	Reserved
		tq_pre_fd_m1	20:13	RW	0	Time quanta prescaler for FD data bit time 0:1x 1:2x ... 255:256x
		sjw_fd_m1	12:9	RW	0	SJW minus one for FD data bit time SJW = sjw_fd_m1 +1 0: 1 … 15:16
		tseg1_fd_m1	8:4	RW	0	Prop_seg + Phase_seg1 for FD data bit time (Prop_seg + Phase_seg1) = tseg1_fd_m1 +1 0: 1 … 31: 32
		tseg2_fd_m1	3:0	RW	0	Phase_seg2 for FD data bit time Phase_seg2 = tseg2_fd_m1+1 0:1 … 15:16
0x18	**tt_timer_reg**	**tt_timer**	31:0	RW	0	Timer in TT CAN mode Users can monitor the timer's value. Users should only modify the timer's value when the timer is disabled.
0x1C	**tt_timer_en_reg**	**Reserved**	31:1	-	-	Reserved
		tt_timer_en	0	RW	0	1: **tt_timer** is free running 0: Freeze **tt_timer** value

For Mekra Langenly Confidential 1.5 (08012025)

Offset	Register	Field	Bits	Access	Reset	Description
0x20	**err_status_reg**	**Reserved**	31:27	-	-	Reserved Please see ISO 11898-1 Figure 16 and 13.1.4.2
		error_state	26:25	R	0	0: Idle state 1: error_active 2: error_passive 3: bus_off
		rx_error_cnt	24:16	R	0	RX error counter Maximum **rx_error_cnt** value is 511
		Reserved	15:9	-	-	Reserved
		tx_error_cnt	8:0	R	0	TX error counter Maximum **tx_error_cnt** maximum value is 511.
0x24	**rbuf_ovflow_reg**	**Reserved**	31:29	-	-	Reserved
		rbuf_ovflow_id	28:0	R	0	When RX overflow happen, the first dropped message's ID is recorded here. This field is invalid when **rbuf_oveflow_ritr** is 0.
0x28	**rx_cnt**	**Reserved**	31:9	-	-	Reserved
		rx_cnt	8:0	RW	0	**rx_cnt** is only used for non-TT CAN mode. **rx_cnt** will increase when the controller receives a valid message. **rx_cnt** value will be stored into **mbuf_time_trigger** for recording the order in sequence. The counter will wrap around and not saturate.
0x2C	**global_op_itr**	**Reserved**	31:22	-	-	Reserved
		new_fd_error_itr	21	R	0	New types of error
		rx_dma_rtr_itr	20	R	0	When rx_dma writes a RTR packet, write last to the corresponding descriptor
		rx_dma_id_itr	19	R	0	When rx_dma receives special ID, assert **rx_dma_id_ritr** and write last to the correspond descriptor.
		rx_dma_timeout_itr	18	R	0	When total number of descriptors have been processed
		rx_dma_done_itr	17	R	0	When **acc_rx_dsc_num** equals **rx_dsc_max_num**, the RX DMA chain ends, and the bit asserts.
		rx_dsc_done_itr	16	R	0	After 1 RX descriptor has been processed by DMA engine, the bit asserts.
		tx_done_time_out_itr	15	R	0	**tx_done** does not increment for a long period of time. Only asserts when **tx_done_num_th**!=0

Confidential
For Mokra Lang (08012025)

Offset	Register	Field	Bits	Access	Reset	Description
0x2C (Cont'd)	**global_op_itr**	**rx_done_time_out_itr**	14	R	0	rx_done does not increment for a long period of time. Only asserts when **rx_done_num_th**!=0
		rp_fail_itr	13	R	0	Replay failed interrupt
		wake_up_itr	12	R	0	Wake up interrupt (entering active state from bus off state)
		timer_wrap_itr	11	R	0	TT timer wrap interrupt
		tx_done_itr	10	R	0	CAN bus packet has been transmitted
		rx_done_itr	9	R	0	CAN bus packet has been received
		rbuf_ovflow_itr	8	R	0	Any one of rx message buffer has overflowed
		timeout_itr	7	R	0	Any one of message buffer has timed out in RX or TX mode
		bit_error_itr	6	R	0	
		stuff_error_itr	5	R	0	
		crc_error_itr	4	R	0	
		form_error_itr	3	R	0	
		ack_error_itr	2	R	0	
		enter_error_psv_itr	1	R	0	Entering error passive mode
		enter_bus_off_itr	0	R	0	Entering bus off state
0x30	**global_op_ritr**	**Reserved**	31:22	-	-	Reserved
		new_fd_error_ritr	21	RW1C	0	New types of error
		rx_dma_rtr_ritr	20	RW1C	0	When **rx_dma** writes a rtr packet, write last to the correspond descriptor
		rx_dma_id_ritr	19	RW1C	0	When **rx_dma** receives special ID, assert **rx_dma_id_ritr**
		rx_dma_timeout_ritr	18	RW1C	0	When total number of descriptors have been processed
		rx_dma_done_ritr	17	RW1C	0	When **acc_rx_dsc_num** equals **rx_dsc_max_num**, the RX DMA chain ends, and the bit asserts.
		rx_dsc_done_ritr	16	RW1C	0	After 1 RX descriptor has been processed by DMA engine, the bit asserts.
		tx_done_time_out_ritr	15	RW1C	0	**tx_done** does not increment for a long period of time

Ambarella Confidential
Version 1.5 Dec 12, 2025

Offset	Register	Field	Bits	Access	Reset	Description
0x30 (Cont'd)	global_op_ritr	rx_done_time_out_ritr	14	RW1C	0	**rx_done** does not increment for a period of time
		rp_fail_ritr	13	RW1C	0	Replay failed raw interrupt
		wake_up_ritr	12	RW1C	0	Wakeup interrupt
		timer_wrap_ritr	11	RW1C	0	TT timer wrap interrupt
		tx_done_ritr	10	RW1C	0	Any one CAN bus packet has been transmitted
		rx_done_ritr	9	RW1C	0	Any one CAN bus packet has been received
		rbuf_ovflow_ritr	8	RW1C	0	Any one RX message buffer has overflowed
		timeout_ritr	7	RW1C	0	Any one message buffer has timed out in RX or TX mode
		bit_error_ritr	6	RW1C	0	
		stuff_error_ritr	5	RW1C	0	
		crc_error_ritr	4	RW1C	0	
		form_error_ritr	3	RW1C	0	
		ack_error_ritr	2	RW1C	0	
		enter_error_psv_ritr	1	RW1C	0	Entering error passive mode
		enter_bus_off_ritr	0	RW1C	0	Entering bus off
0x34	global_op_itr_msk	Reserved	31:22	-	-	Reserved
		new_fd_error_msk	21	RW	0	New types of errors
		rx_dma_rtr_msk	20	RW	0	**rx_dma_rtr** mask
		rx_dma_id_msk	19	RW	0	**rx_dma_id** mask
		rx_dma_time-out_msk	18	RW	0	When total number of descriptors have been processed
		rx_dma_done_msk	17	RW	0	When **acc_rx_dsc_num** equals **rx_dsc_max_num**, the RX DMA chain ends, and the bit asserts
		rx_dsc_done_msk	16	RW	0	After 1 RX descriptor has been processed by DMA engine, the bit asserts
		tx_done_time_out_msk	15	RW	0	**tx_done** does not increment for a long time.
		rx_done_time_out_msk	14	RW	0	**rx_done** does not increment for a long time.

Confidential Ambarella Inc. 2023

Offset	Register	Field	Bits	Access	Reset	Description
0x34 (Cont'd)	**global_op_itr_msk**	**rp_fail_msk**	13	RW	0	Replay failed mask
		wake_up_msk	12	RW	0	Wakeup interrupt
		timer_wrap_msk	11	RW	0	**timer_wrap_ritr** interrupt mask
		tx_done_msk	10	RW	0	**tx_done_ritr** interrupt mask
		rx_done_msk	9	RW	0	**rx_done_ritr** interrupt mask
		rbuf_ovflow_msk	8	RW	0	**rbuf_ovflow_ritr** interrupt mask
		timeout_msk	7	RW	0	**timeout_ritr** interrupt mask
		bit_error_msk	6	RW	0	**bit_error_ritr** interrupt mask
		stuff_error_msk	5	RW	0	**stuff_error_ritr** interrupt mask
		crc_error_msk	4	RW	0	**crc_error_ritr** interrupt mask
		form_error_msk	3	RW	0	**form_error_ritr** interrupt mask
		ack_error_msk	2	RW	0	**ack_error_ritr** interrupt mask
		enter_error_psv_msk	1	RW	0	**enter_error_psv_ritr** interrupt mask
		enter_bus_off_msk	0	RW	0	**enter_bus_off_ritr** interrupt mask
0x38	**canc_en_reg**	**Reserved**	31:1	-	-	Reserved
		canc_en	0	RW	0	After basic configuration is complete (similar to time quanta and prescaler value). Write **canc_en** to 1 to enable the CAN controller
0x3C	**mbuf_tx_busy_reg**	**msg_tx_buf_busy**	31:0	R	0	**msg_buf_tx_busy**[i] = 1: i-th TX message buffer is during transaction **msg_buf_tx_busy**[i] = 0: i-th TX message buffer has not been processed.
0x40	**mbuf_rx_busy_reg**	**msg_rx_buf_busy**	31:0	R	-	**msg_buf_rx_busy**[i] = 1: i-th RX message buffer is during transaction **msg_buf_rx_busy**[i] = 0: i-th RX message buffer has not been processed
0x80	**mbuf_cfg_done_reg**	**msg_buf_config_done**	31:0	RWSC	0	**msg_buf_config_done**[i] is set to 1 when the message buffer i has been configured and would be automatically cleared when i-th message is buffer has been released
0x84	**msg_req_reg1**	**msg_request**	31:0	RWSC	0	Used to revise the configuration of i-th message buffer. Write **msg_request[**i] to 1 and poll **msg_request**[i]. When **msg_request**[i] ==0, **msg_request_status**[i] indicates whether the request is granted or not

For Weka Language Confidential 2025-08-01

Offset	Register	Field	Bits	Access	Reset	Description
0x88	**msg_req_reg2**	**msg_request_status**	31:0	R	0	**msg_request_status**[i]=1: Grant the request **msg_request_status**[i]=0: Do not grant the request
0x8C	**tx_mbuf_done_reg**	**tx_msg_buf_done**	31:0	R	0	**msg_buf_tx_done**[i] = 1: i-th TX message has been transmitted. **msg_buf_tx_done**[i] = 0: i-th TX message has not been transmitted.
0x90	**rx_mbuf_done_reg**	**rx_msg_buf_done**	31:0	R	0	**msg_buf_rx_done**[i] = 1: i-th RX message has been received **msg_buf_rx_done**[i] = 0: i-th RX message has not been recieved
0x94	**mbuf_rp_fail_reg**	**mbuf_rp_fail**	31:0	R		**mbuf_rp_fail[i]**= 1: i-th message when retry failure
0x98	**mbuf_timeout_reg**	**msg_buf_timeout**	31:0	R	0	Message buffer time out. (For TX or RX) **timeout_itr** = 1 and see which buffer asserts interrupt
0x9C	**mbuf_tx_reg**	**msg_buf_tx**	31:0	RW	0	**msg_buf_tx**[i] specify i-th message buffer's type **msg_buf_tx**[i] = 1'b0: RX message **msg_buf_tx**[i] = 1'b1: TX message
0x100[1]	**id_fltr_reg0**	**id_fltr0_en**	31	RW	0	0: Disable ID filter #0 1: Enable ID filter #0
		Reserved	30:29	-	-	Reserved
		id_fltr0	28:0	RW	0	ID filter for RX data would be received if the ID matches
0x104	**id_fltr_mask_reg0**	**Reserved**	31:29	-	-	Reserved
		id_fltr0_mask	28:0	RW	0	ID filter for RX data would be received if the ID matches
...	**...**	**...**	...	...	...	
0x1F8	**id_fltr_reg31**	**id_fltr31_en**	31	RW	0	Manual
		Reserved	30:29	-	-	Reserved changed
		id_fltr31	28:0	RW	0	
0x1FC	**id_fltr_mask_reg31**	**Reserved**	31:29	-	-	Manual
		id_fltr31_mask	28:0	RW	0	
0x200	**mbuf0_id**	**Reserved**	31:30	-	-	Reserved (PUT)
		msg_0_ide	29	RW	0	Used to distinguish between base format and extended format. Moving IDE to the ID field is more conducive to hardware implementation.
		msg_0_id_field	28:0	RW	0	

1 0x100~0x104: N-th RX filter message buffer will receive the packet if ((msg_N_id_filter_mask & msg_N_id_filter) == (msg_N_id_filter_mask & rx_data_id[28:0]) & msg_N_id_filter_en) == 1

Confidential
For Mekra Lang only
1.5 (2021-05-25)

Offset	Register	Field	Bits	Access	Reset	Description
0x204[2]	**mbuf0_ctl**	**msg_0_tx_retry_num**	31:24	RW	0	0: No TX re-transmission … 255: At most 255 times TX re-transmission RX will reset to 0!
		msg_0_tx_retry_num_en	23	RW	0	0:Infinite TX re-transmit times. 1:Limit TX re-transmit times up to **tx_retry_num**. RX will reset to 0!
		Reserved	22:21	-	-	Reserved
		msg_0_tx_enable	20:17	RW	0	For TTCAN only, data in TX buffer should be transmitted in **tx_enable**+1 bit time window. ie. (1...16) Otherwise, assert timeout interrupt. RX will reset to 0!
		Reserved	16:8	-	-	Reserved
		msg_0_rtr	7	RW	0	
		msg_0_edl	6	RW	0	FDF flag. When 0x0 **can_fd** is set to 0 (classical CAN 2.0), software can simply ignore the flag. In CAN 2.0 protocol-wise, the fields set to nominal or non-nominal bit are considered classical CAN packets. Hardware passes the field's value from the bus.
		msg_0_brs	5	RW	0	Bitrate-switch
		msg_0_esi	4	RW	0	
		msg_0_dlc	3:0	RW	0	
0x208	**mbuf0_time_trig**	**msg_0__time_trig**	31:0	RW	0	This field has no effect when **tt_can_en**=0. When this message buffer is for TX, this field indicates the time to trigger transmission when **tt_timer** == **msg_i_time_trig**. When this message buffer is for RX, this field indicates the time to trigger message reception when **tt_timer**==**msg_i_time_trig**. After a message is successfully received, this field is overwritten with the time stamp that SOF is received.

For Mekra Lang GmbH & Co. KG Confidential 1.5 (08-2025)

2 "RX will reset to 0!" Indicates that the value will be 0 if the i-th message buffer is programmed as RX and not TX.

Offset	Register	Field	Bits	Access	Reset	Description
0x20C	**mbuf0_time-out_th**	**mbuf0_time-out_th**	31:0	RW	0	This field takes no effect when **tt_can_en**=0. If this message buffer is for TX, **time-out_ritr** will assert when **tt_timer** reaches **mbuf_timeout_th** and message is not transmitted yet. In this case, the transaction will be aborted. If this message buffer is for RX, **timeout_ritr** will assert when **tt_timer** reaches **mbuf_timeout_th** and message is not received yet. In this case, the message buffer will be invalidated.
....	**....**	**....**				
0x3f0	**mbuf31_id**	**Reserved**	31:29	-	-	Reserved (PUT)
		msg_31_ide	29	RW	0	Used to distinguish between base format and extended format. Moving IDE to ID field is easier for hardware implementation.
		msg_31_id_field	28:0	RW	0	Message identification field. Base Identifier should be filled in [28:18 for base format. Base ID+ ID extension should be filled in [28:0]
0x3f4	**mbuf31_ctl**	**msg_31_tx_re-try_num**	31:24	RW	0	0: No TX re-transmission … 255: At most 255 times TX re-transmission
		msg_31_tx_re-try_num_en	23	RW	0	0:Infinite TX re-transmit times. 1:Limit TX re-transmit times up to **tx_re-try_num**.
		Reserved	22:21	-	-	Reserved
		msg_31_tx_en-able	20:17	RW	0	For ttcan only, data in TX buffer should be transmitted in **tx_enable**+1 bit time window. ie. (1...16) Otherwise, assert timeout interrupt
		Reserved	16:8	-	-	Reserved
		msg_31_rtr	7	RW	0	
		msg_31_edl	6	RW	0	
		msg_31_brs	5	RW	0	Bitrate-switch
		msg_31_esi	4	RW	0	Error state (Should be filled by hardware)
		msg_31_dlc	3:0	RW	0	FD Basic should perform hardware protection during non-FD

For Mekra Lang (Suzhou) Co., Ltd. 2025 Confidential

Offset	Register	Field	Bits	Access	Reset	Description
0x3f8	**mbuf31_ time_trig**	**msg_31__time_ trig**	31:0	RW	0	This field takes no effect when **tt_can_ en**=0. When this message buffer is for TX, this field indicates the time to trigger transmission when **tt_timer** == **msg_i_ time_trig**. When this message buffer is for RX, this field indicates the time to trigger message reception when **tt_timer**==**msg_i_time_trig**. After a message is successfully received, this field is overwritten with the time stamp that SOF is received.
0x3fC	**mbuf31_time-out_th**	**mbuf31_time-out_th**	31:0	RW	0	This field takes no effect when **tt_can_ en**=0. If this message buffer is for TX, **time-out_ritr** will assert when **tt_timer** reaches **mbuf_timeout_th** and message is not transmitted yet. In this case, the transaction will be aborted. If this message buffer is for RX, **timeout_ritr** will assert when **tt_timer** reaches **mbuf_timeout_th** and message is not received yet. In this case, the message buffer will be invalidated.
0x400[3]	**msg_priority0**	**Reserved**	31:30	-	-	Reserved for message
		msg_0_priority	29:24	RW	0	msg_0_priority
		Reserved	23:22	-	-	Reserved for message
		msg_1_priority	21:16	RW	0	msg_1_priority
		Reserved	15:14	-	-	Reserved for message
		msg_2_priority	13:8	RW	0	msg_2_priority
		Reserved	7:6	-	-	Reserved for message
		msg_3_priority	5:0	RW	0	msg_3_priority
...	**...**	**...**	...	...	...	...
0x41C	**msg_priority7**	**Reserved**	31:30	-	-	Reserved for message
		msg_28_prior-ity	29:24	RW	0	msg_28_priority
		Reserved	23:22	-	-	Reserved for message

Confidential For Mekra Lang GmbH 1.5 (as of 2025-)

3 0x400~0x41C: The range for priority is 0~31 (32 message buffers); small value means high priority.

Offset	Register	Field	Bits	Access	Reset	Description
0x41C (Cont'd)	**msg_priority7**	**msg_29_priority**	21:16	RW	0	msg_29_priority
		Reserved	15:14	-	-	Reserved for message
		msg_30_priority	13:8	RW	0	msg_30_priority
		Reserved	7:6	-	-	Reserved for message
		msg_31_priority	5:0	RW	0	msg_31_priority
0x420	**tx_done_th**	**Reserved**	31:22	-	-	Reserved
		tx_done_num	21:16	RW	0	Accumulate **tx_done** number. 0: 0 **tx_done** … 32:
		Reserved	31:22	-	-	Reserved
		tx_done_num_th	4:0	RW	0	Accumulate **tx_done** number threshold. When **tx_done_num**== **tx_done_num_th**, **tx_done_ritr** asserts. When timeout occurs, the **tx_done_timer** will not increment--user should reset it manually after timeout interrupt asserts. 0: 1 … 31: 32
0x424	**tx_done_timer**	**tx_done_timer**	31:0	RW	0	Reset when **tx_done**. Increment only when **tx_done_num_th**!=0 When the accumulated interrupt does not assert for a period of time (**tx_done_timer** == **tx_done_timer_th**), assert **tx_done timeout_ritr**.
0x428	**tx_done_timer_th**	**tx_done_timer_th**	31:0	RW	0xffffffff	
0x42C	**rx_done_th**	**Reserved**	31:22	-	-	Reserved
		rx_done_num	21:16	RW	0	Accumulated **rx_done** number. 0: 0 **tx_done** … 32:
		Reserved	31:22	-	-	Reserved
		rx_done_num_th	4:0	RW	0	Accumulated **tx_done** number threshold. When **rx_done_num**== **rx_done_num_th**, **tx_done_ritr** asserts. When timeout occurs, the **rx_done_timer** will not increment and user should reset it manually after timeout interrupt asserts. 0: 1 … 31: 32

Confidential only
For Mekra Lang only
1.5 (20120215)

Offset	Register	Field	Bits	Access	Reset	Description
0x430	**rx_done_timer**	**rx_done_timer**	31:0	RW	0	Reset when **rx_done**. Increment only when **rx_done_num_th**!=0 When the accumulated interrupt does not assert for a long time (**rx_done_timer** == **rx_done_timer_th**) assert **rx_done timeout_ritr**.
0x434	**rx_done_timer_th**	**rx_done_timer_th**	31:0	RW	0xffffffff	
0x500	**dma_ctrl_reg**	**Reserved**	30:2	-	-	Reserved
		rx_dma_en	0	RW	0	Receive DMA is enabled
0x504	**rx_dma_dsc_ptr**	**rx_data_dsc_ptr**	31:0	-	-	RX Data descriptor pointer
0x508	**rx_dma_num**	**rx_dsc_timer_en**	31	-	1	**rx_dsc_timer_en** default is 1. If we do not want **rx_dsc** timeout scheme, we can write this bit to 0.
		Reserved	30:25	-	-	Reserved
		acc_rx_dsc_num	24:16	RW	0	Accumulated number of descriptors stored by DMA engine. If the **acc_dsc_num** equal **rx_dsc_max_num**, then hardware will auto-clear **acc_dsc_num** to 0. Timeout of descriptor, if **acc_dsc_num** does not increment for a long time (**rx_dsc_timer** > **rx_dsc_time_th**), **rx_dma_timeout_ritr** will assert. DMA engine will also write descriptor **buffer_status**27] ("last") to 1 (refer to "Table 23-1. Buffer Status.", and auto-clear **rx_dma_en**. Software should re-enable RX DMA to receive the remaining messages. 0: Stored 0 messages … 256: Stored 128 messages 39~255: Not possible
		Reserved	15:8	-	-	Reserved
		rx_dsc_max_num	7:0	RW	0	Receive descriptor number threshold. When number of received descriptors stored by **dma_engine** equals **rx_dsc_max_num**, **dma_engine** assert **rx_dma_done_ritr** (in 0x30). DMA engine will also write descriptor [**buffer_status**27] ("last") to 1 (refer to "Table 23-1. Buffer Status.", and auto-clear **rx_dma_en**. Software should re-enable RX DMA to receive the remaining messages. 0: 1 message … 255: 256 messages

Confidential Langley Mekra 1.5 (08012025) only

Offset	Register	Field	Bits	Access	Reset	Description
0x50C	**rx_dsc_timer**	**rx_dsc_timer**	31:0	RW	0	When **rx_dma_en** equal 1, **rx_dsc_timer** increments in ahb_clk clock frequency. The timer will also be reset when **acc_dsc_num** increments. Increment only when **rx_dsc_max_num**!=0 Timeout of descriptor, if **acc_dsc_num** does not increment for a long time (**rx_dsc_timer** == **rx_dsc_time_th**), **rx_dma_timeout_ritr** will assert, and there is no timeout for first descriptor.
0x510	**rx_dsc_timer_th**	**rx_dsc_timer_th**	31:0	RW	0xFFFFFFFF	rx_dsc_timer_th
0x514	**rx_special_id**	**Reserved**	31:17	-	-	Reserved
		rtr_terminate_dma_en	16	RW	0x1	To prevent RTR packets from terminating DMA transaction. When **rtr_terminate_dma_en** == “0”, DMA engine will not be terminated when receiving RTR packet.
		rx_special_id_en	15	RW	0x0	Write 'last' in descriptor status when the DMA engine writes special ID message
		Reserved	14:5	-	-	Reserved
		rx_special_id_mbuf_idx	4:0	RW	0x1F	Similar to **rx_id_filter**
0x518	**canc_fault_inj**	**Reserved**	31:2	-	-	Reserved
		sm_error_disableReserved	1	RW	0	0: SM error detection logic is enabled. 1: Disable SM error logic. During programming, SW should set it to 1. When programming is complete, SW should set it to 0.
		Reservedcanc_fault_inj	0	RW	0x0	0: No fault inject 1: Fault inject
0x800	**mbuf0_data0**	**mbuf0_data0**	31:0	RW	0	64-byte data field of message buffer 0,
0x804	**mbuf0_data1**	**mbuf0_data1**	31:0	RW	0	64-byte data field of message buffer 0
0x808	**mbuf0_data2**	**mbuf0_data2**	31:0	RW	0	64-byte data field of message buffer 0
0x80c	**mbuf0_data3**	**mbuf0_data3**	31:0	RW	0	64-byte data field of message buffer 0
0x810	**mbuf0_data4**	**mbuf0_data4**	31:0	RW	0	64-byte data field of message buffer 0
0x814	**mbuf0_data5**	**mbuf0_data5**	31:0	RW	0	64-byte data field of message buffer 0
0x818	**mbuf0_data6**	**mbuf0_data6**	31:0	RW	0	64-byte data field of message buffer 0
0x81c	**mbuf0_data7**	**mbuf0_data7**	31:0	RW	0	64-byte data field of message buffer 0
0x820	**mbuf0_data8**	**mbuf0_data8**	31:0	RW	0	64-byte data field of message buffer 0
0x824	**mbuf0_data9**	**mbuf0_data9**	31:0	RW	0	64-byte data field of message buffer 0
0x828	**mbuf0_data10**	**mbuf0_data10**	31:0	RW	0	64-byte data field of message buffer 0

Confidential For Meta Lang 15 Dec 2025

Offset	Register	Field	Bits	Access	Reset	Description
0x82c	**mbuf0_data11**	**mbuf0_data11**	31:0	RW	0	64-byte data field of message buffer 0
0x830	**mbuf0_data12**	**mbuf0_data12**	31:0	RW	0	64-byte data field of message buffer 0
0x834	**mbuf0_data13**	**mbuf0_data13**	31:0	RW	0	64-byte data field of message buffer 0
0x838	**mbuf0_data14**	**mbuf0_data14**	31:0	RW	0	64-byte data field of message buffer 0
0x83c	**mbuf0_data15**	**mbuf0_data15**	31:0	RW	0	64-byte data field of message buffer 0
...	**...**	**...**	...	...	...	
0xfc0	**mbuf31_data0**	**mbuf31_data0**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfc4	**mbuf31_data1**	**mbuf31_data1**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfc8	**mbuf31_data2**	**mbuf31_data2**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfcc	**mbuf31_data3**	**mbuf31_data3**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfd0	**mbuf31_data4**	**mbuf31_data4**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfd4	**mbuf31_data5**	**mbuf31_data5**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfd8	**mbuf31_data6**	**mbuf31_data6**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfdc	**mbuf31_data7**	**mbuf31_data7**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfe0	**mbuf31_data8**	**mbuf31_data8**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfe4	**mbuf31_data9**	**mbuf31_data9**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfe8	**mbuf31_ data10**	**mbuf31_data10**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xfec	**mbuf31_ data11**	**mbuf31_data11**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xff0	**mbuf31_ data12**	**mbuf31_data12**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xff4	**mbuf31_ data13**	**mbuf31_data13**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xff8	**mbuf31_ data14**	**mbuf31_data14**	31:0	RW	0	Manual 64-byte data field of message buffer 31
0xffc	**mbuf31_ data15**	**mbuf31_data15**	31:0	RW	0	Manual 64-byte data field of message buffer 31

Table 23-2. CAN Bus Controller: Control Registers.

For Ambarella Lang Confidential Only

23.4.1 Register Map

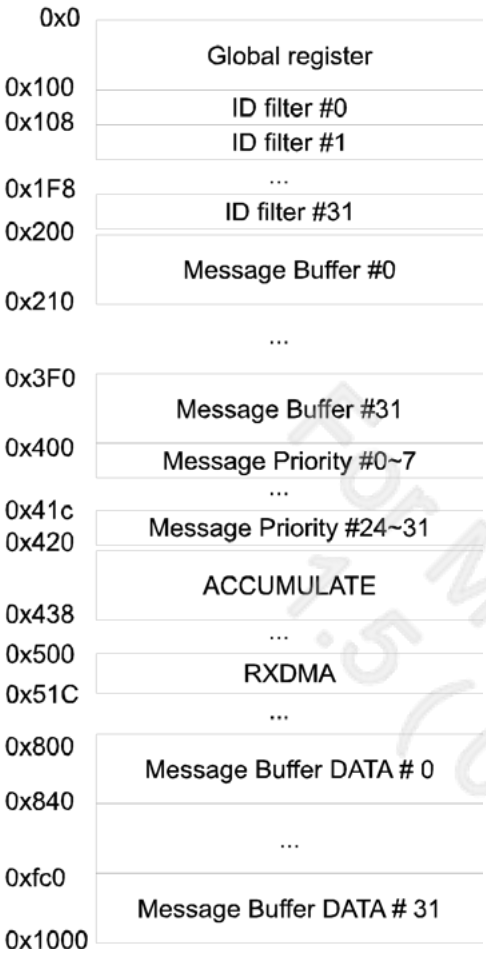


Figure 23-7. CAN Bus Controller: Register Map.

Confidential
For Mekra Lang only
1.5 (08012025)

23.4.2 Register Programming Notes

The six CAN controllers include CAN0, CAN1, CAN2, CAN3, CAN4 and CAN5, respectively.

23.4.2.1 Global Register Configuration

- After configuring global registers, write **canc_en_reg** = 0x1 to complete configuration.
- To reprogram global registers, write **can_en** to 0x0 and poll **canc_pg_status** == 1.

23.4.2.2 Reconfigure the i-th Message Buffer

- Monitor **msg_tx_status** and **msg_rx_status** to determine whether or not the i-th message buffer has been processed.
- Write **msg_request[i]** to 1; polling write **msg_request[i]** to 0
- Read **msg_request_status[i]** to determine whether or not the CAN bus controller has been successfully configured
- After reconfiguration of the i-th message buffer, write **msg_buf_config_done[i]** to 1.

23.4.2.3 Interrupts

- Monitor **global_op_itr** to identify the interrupt types.
- If the interrupt is message buffer-related, verify whether or not the corresponding interrupt occurred on the i-th message buffer

23.4.2.4 Software Abort Flow

The software abort feature can be included in a bus guard which observes error flags; if error flags are detected on the CAN bus, the system can remove the frame in error and proceed uninterrupted.

The following describes the software abort flow and setup options for terminating messages. (Messages are terminated after two unsuccessful transmission attempts.)

Software abort flow:

1. Set msg_N_tx_retry_num_en = 1
2. msg_N_tx_retry_num = 1. (system attempts to send message once after the initial unsuccessful attempt)
3. Software checks the TX buffer transmission (sent-out) status

CAN bus behavior (based on user settings) under three conditions / cases:

- Case 1: No Error, Normal Communication

  All Messages are sent without affecting the CAN bus.

- Case 2: Arbitration Lost

  The first two messages result in an arbitration lost condition on the bus. With the threshold of two (msg_N_tx_retry_num = 1), the CAN core discontinues the retransmission. There will be no further CAN frame on the bus, and the CAN bus becomes recessive. Software receives a “rp_fail_itr interrupt + No error flag status, indicating that arbitration is lost. Additionally, the TX buffer does not transmit as the retry number is set to 1.

Confidential
For Mekra Lang only
1.5 (08/01/2025)

- Case 3: On Error with Different Error Positions

  The first two attempts of the TX message result in a CAN bus error, and transmission is discontinued. There will be no further CAN frame on the bus, and the CAN bus becomes recessive. Software receives a “rp_fail_itr interrupt + error flag status,” indicating an error on the bus. Additionally, the TX buffer does not transmit as the retry number is set to 1.

23.4.2.5 Message Buffer RX Filter Flow

The following lists the uses cases and corresponding settings for the message buffer RX filter flow.

Case 1: All message buffers are used as normal message buffers, and no RX packet will be filtered.

- Set all 32 message buffers’ filter_en to 0.

Case 2: All message buffers enable the filter feature, and no normal message buffer is enabled.

- Set all 32 message buffers’ filter_en to 1.

Case 3: Neither all 32 message buffer filters are enabled, nor all 32 message buffer filters are disabled.

SW must implement the flow as follows:

a) For the expected unfiltered message buffers, set the corresponding filter_en to 1, and set their idFIlterMask to 0. This setting implies the message buffer is not filtering any message, as the idFIlterMask is set to 0.

b) SW shall allocate filter_en = 1 buffers with a lower buffer index compared to those expected unfiltered buffers (shown below). When multiple filtered buffers match the RX packet at the same time, the lower index (buf_idx, idx=0~31) takes higher priority in receiving the packet.

The following figure displays the buffer allocation for filtered and unfiltered packets.

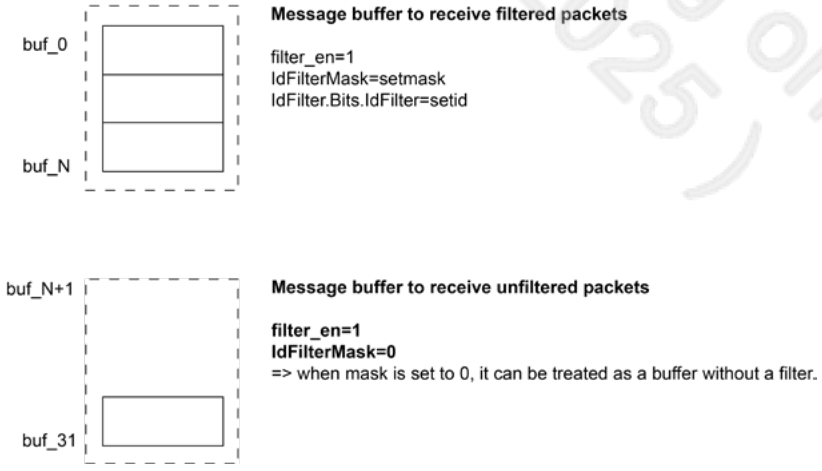


Figure 23-8. Buffer Allocation for Unfiltered and Filtered Buffers.

23.5 CAN Controller: Safety Notes

To force an error for CANCN unit, **N**=0~5:

1. Enable ECRU error mask bit 39+**N** for canc**N**_sm_error.
2. Enable CEHU error mask bit 154 amba_safety_error.
3. Write CANC**N** 0x518 0x1.

To clear the error status CANC**N** unit, **N**=0~5:

1. Disable sm_error in CANC**N**
   - Write CANC**N** 0x518 0x2, then
   - Write CANC**N** 0x518 0x0.
   - Bit [1] sm_error_disable
     - 0: SM error detection logic is enabled.
     - 1: SM error detection logic is disabled.
2. Clear bit 39+**N** for the corresponding canc**N**_sm_error in ECRU.

To disable error propagation from CANC**N** unit to CEHU, **N**=0~5:

1. Disable sm_error in CANCN
   - Write CANC**N** 0x518 0x2.
   - Bit [1] sm_error_disable
     - 0: SM error detection logic is enabled.
     - 1: SM error detection logic is disabled.
2. Disable ECRU mask: bit 39+**N** for corresponding canc**N**_sm_error. Software can read it from ECRU of AHB bus and check the error status bit 39+**N.**

For Confidential Mekra Lang only (15 Feb 8012025)

24. RTC

This chapter details the interfaces for the CV22FS real-time clock (RTC).

The chapter is organized as shown below.

- (Section 24.1) RTC: Overview
- (Section 24.2) RTC: Registers
- (Section 24.3) RTC: Programming Notes

24.1 RTC: Overview

The CV22FS real-time clock (RTC) modules share a register space located on the AHB bus.

24.2 RTC: Registers

The RTC instances on the AHB bus use 32-bit registers as shown below.

24.2.1 RTC Registers: Map

Offset	Register	Bits	Access	Reset	Description
0x30	**RTC_CURT**	31:0	RW	0x0	ahb_rtc_curt
0x34	**RTC_REG_CURT**	31:0	RO	0x0	ahb_reg_curt
0x40	**RTC_PCRST**	31:1	-	-	Reserved
		0	RW	0x0	ahb_rtc_pcrst
0x54	**RTC_REG_STA**	31:8	-	-	Reserved
		7:0	RO	0x0	rtc_ahb_reg_sta_reg
0x58	**RTC_STA**	31:8	-	-	Reserved
		7:0	RW	0x0	ahb_rtc_sta
0x90	**RTC_STROBE**	31:1	-	-	Reserved
		0	RW	0x0	ahb_rtc_strobe

Table 24-1. RTC Registers on the AHB Bus.

24.2.2 RTC Registers: Detail

24.2.2.1 rtc_curt Register

Bits	Name	Access	Reset	Description
31:0	rtc_curt	RW	0	Current time programming in RTC Writing to this register sets the current wall clock time

Table 24-2. RTC Current Time Programming Register.

Confidential
For Mekra use only
1.5 (2022-05-25)

24.2.2.2 rtc_reg_curt Register

Bits	Name	Access	Reset	Description
31:0	**rtc_reg_curt**	R	0	Current time value in RTC. This register should be read to retrieve the current wall clock time.

Table 24-3. RTC Current Time Value Register.

24.2.2.3 rtc_pcrst Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**rtc_pcrst**	RW	0	Use PCRST strobe for current time setting. All bits must be re-freshed at the same time.

Table 24-4. RTC pcrst Register.

24.2.2.4 RTC_reg_sta Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**reg_sta**	**R**	0	Current value of status bits

Table 24-5. RTC Status Read Register.

24.2.2.5 RTC_sta Register

Bits	Name	Access	Reset	Description
31:8				Reserved
7:0	**sta**	**RW**	0	Sets the status bits

Table 24-6. RTC Status Set Register.

24.3 RTC: Programming Notes

This section provides programming notes related to the load registers of the RTC interfaces.

- A positive PCRST pulse causes the RTC to load the values of RTC_sta (bits 0:7) and RTC_curt (0x30, bits 0:31 to corresponding registers. Ensure that all register data is ready when programming specific registers.
- The data must be valid on these inputs at least 3 ms before the assertion of PCRST.
- Both the data and PCRST signals should be maintained for at least 4 ms.
- At least one programming task is required after PWC_AVDD18 is on or recovered above the mini- mum voltage (see the CV22FS chip datasheet for electrical details).
- After programming is completed, the RTC controller should reset both inputs and PCRST to 0.

Confidential For Meike Lang only 15:08 08012025)

25. PULSE WIDTH MODULATOR (PWM)

This chapter overviews the CV22FS pulse width modulator (PWM) controller interface as follows:

- (Section 25.1) PWM: Overview
- (Section 25.2) PWM: External Pin Muxing
- (Section 25.3) PWM: Interface
- (Section 25.4) PWM: Register Map and Details

25.1 PWM: Overview

The CV22FS pulse width modulator consists of three interfaces, each with four pins:

- Three pulse width modulation interfaces. Each interface has four pins:
  - PWM0 (E400.3000): **PWM0:3**
  - PWM1 (E400.C000): **PWM4:7**
  - PWM2 (E400.D000): **PWM8:11**

25.2 PWM: External Pin Muxing

The PWM functions are multiplexed on GPIO pins. Refer to the GPIO programming chapter in this manual for information regarding function selection when using GPIO pins.

25.3 PWM: Interface

The PWM interface outputs are shared across multiple pins. When enabled, a given PWM emits a repeated sequence of logic-high and logic-low. The duration of each logic-high and logic-low is programmable (**xon** and **xoff**).

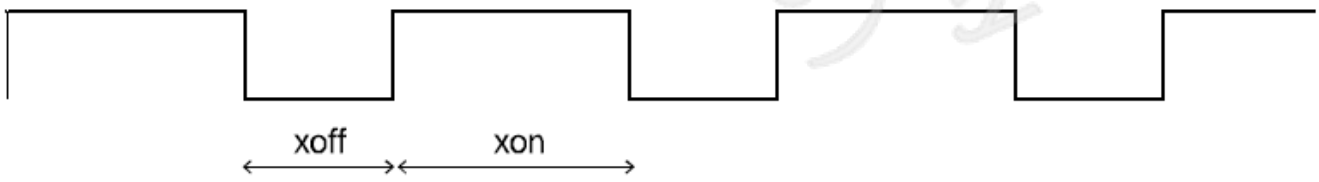


Figure 25-1. PWM Timing Diagram.

The PWM interface generally uses the PWM clock (GCLK_PWM) as a source clock, and PWM_CLK = SOURCE_CLK / (**divider**+1). Refer to the next section for an overview of this register programming.

Confidential
For Mekra Lang only
(2012-05-25)

25.3.1 PWM: Clocking

The pulse width modulators generally uses GCLK_PWM, which is set by the system software. The register programming information provided in this chapter can be used for adjustment and refinement. Contact an Ambarella representative for additional details.

The PWM pulse rates are calculated using GCLK_PWM with a divider defined on PWM registers (Section 25.4.1, Section 25.4.2 and Section 25.4.3). The relationship is illustrated below for SOURCE_CLK = GCLK_PWM:

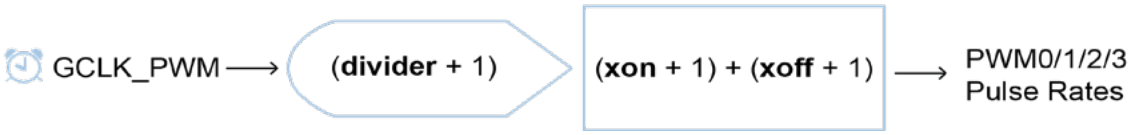


Figure 25-2. PWM Clock Settings.

- The PWM pulse rates are calculated with the **PWM_n_enable** bit **divider** and **PWM_n_data** bits **xon** and **xoff**. The calculation for one of the pulse rates is:

$$\text{PWM[n] Pulse Rate} = \frac{\text{gclk_pwm} / (\textbf{divider} + 1)}{[(\textbf{xon} + 1) + (\textbf{xoff} + 1)]}$$

25.4 PWM: Register Map and Details

This section includes a map of the visible registers as well as details of the registers used by the PWM software interface.

Offset	Register	Description
0x000		Reserved
0x004	**PWM_0_enable**	PWM0 Enable
0x008		Reserved
0x00C	**PWM_1_enable**	PWM1 Enable
0x010		Reserved
0x014	**PWM_2_enable**	PWM2 Enable
0x018		Reserved
0x01C	**PWM_3_enable**	PWM3_Enable
0x020	**PWM_0_data1**	Register Bank for PWM0
0x024	**PWM_1_data1**	Register Bank for PWM1
0x028	**PWM_2_data1**	Register Bank for PWM2
0x02C	**PWM_3_data1**	Register Bank for PWM3
0x030 - 0xFFC		Reserved

Table 25-1. PWM Interface Register Map.

Confidential Ambarella Language only (08/12/2025)

25.4.1 PWM_0/2_enable Register

Bits	Name	Access	Reset	Description
31				Reserved
30:1	**divider**	RW	0	Clock divider of PWM (PWM_CLK = SOURCE_CLK/(**divider**+1))
0	**pwm_en**	RW	0	PWM enable

Table 25-2. PWM Enable Register for PWM0 and PWM2.

Note that system software generally sets / configures the PWM source clock, and register programming is used for adjustment. See Section 25.3.1 for details.

25.4.2 PWM_1/3_enable Register

Bits	Name	Access	Reset	Description
31	**inv_en**	RW	0	When 1, set PWM1/3 becomes an inverted version of PWM0/2, and the register settings for PWM1/3 are ignored.
30:1	**divider**	RW	0	Clock divider of PWM (PWM_CLK = SOURCE_CLK/(**divider**+1))
0	**pwm_en**	RW	0	PWM enable

Table 25-3. PWM Enable Register for PWM1 and PWM3.

Note that system software generally sets / configures the PWM source clock, and register programming is used for adjustment. See Section 25.3.1 for details.

25.4.3 PWM_xn_data1 Register

Bits	Name	Access	Reset	Description
31:16	**xon**	RW	0	**xon** data pattern for PWM
15:0	**xoff**	RW	0	**xoff** data pattern for PWM

Table 25-4. PWM Register Bank for PWM0/1/2/3.

Note that system software generally sets / configures the PWM source clock, and register programming is used for adjustment. See Section 25.3.1 for details.

For Wakra Lab only (Confidential 15:(080 2025)

26. INTERVAL TIMERS

This chapter covers the CV22FS interval timers as follows:

- (Section 26.1) Interval Timer: Overview
- (Section 26.2) Interval Timer: Clocking
- (Section 26.3) Interval Timer: Registers
- (Section 26.4) Interval Timer: Safety Notes

26.1 Interval Timer: Overview

Features of the interval timers includes the following:

- Programmable 32-bit timer / counters
- Three instances
  - Each has 10 sets of independent timers / counters
- Interrupts generated at underflow (i.e., reached limit) or regular intervals
- Two match registers
- Current value of each count register can be read at any time
- Sensitive to both edges when driven by an external clock source
- Sensitive to positive edge when driven by an internal clock source (reference clock: 24 MHz)

26.2 Interval Timer: Clocking

The interval timers can be independently configured to be clocked by three types of clock sources: APB, INT, and EXT. When clocked by APB, the timer runs at the same frequency as the APB bus clock. When clocked internally, the timer is sensitive to positive edges of the internal clock. The internal clock is typically tied to reference clock (INTCLK = REFCLK = 24 MHz).

The timers can also be clocked by an external source. When clocked by an external clock source, the interval timer is sensitive to both edges of the clock. More specifically, the timer decrements on a rising edge as well as on a falling edge of the external clock.

Refer to the chip CV22FS datasheet for details on the external pins used in CV22FS programming.

Confidential For Mekra Lang only (Jan 2025)

26.3 Interval Timer: Registers

26.3.1 Interval Timer Registers: Map

Offset	Register	Description
0x00	**TM_cnt1_sts**	Counter 1 status
0x04	**TM_cnt1_reload**	Counter 1 reload value
0x08	**TM_cnt1_match1**	Counter 1 first match
0x0C	**TM_cnt1_match2**	Counter 1 second match
0x10	**TM_cnt2_sts**	Counter 2 status
0x14	**TM_cnt2_reload**	Counter 2 reload value
0x18	**TM_cnt2_match1**	Counter 2 first match
0x1C	**TM_cnt2_match2**	Counter 2 second match
0x20	**TM_cnt3_sts**	Counter 3 status
0x24	**TM_cnt3_reload**	Counter 3 reload value
0x28	**TM_cnt3_match1**	Counter 3 first match
0x2C	**TM_cnt3_match2**	Counter 3 second match
0x30	**TM_ctrl**	Control
0x34	**TM_cnt4_sts**	Counter 4 status
0x38	**TM_cnt4_reload**	Counter 4 reload value
0x3C	**TM_cnt4_match1**	Counter 4 first match
0x40	**TM_cnt4_match2**	Counter 4 second match
0x44	**TM_cnt5_sts**	Counter 5 status
0x48	**TM_cnt5_reload**	Counter 5 reload value
0x4C	**TM_cnt5_match1**	Counter 5 first match
0x50	**TM_cnt5_match2**	Counter 5 second match
0x54	**TM_cnt6_sts**	Counter 6 status
0x58	**TM_cnt6_reload**	Counter 6 reload value
0x5C	**TM_cnt6_match1**	Counter 6 first match
0x60	**TM_cnt6_match2**	Counter 6 second match
0x64	**TM_cnt7_sts**	Counter 7 status
0x68	**TM_cnt7_reload**	Counter 7 reload value
0x6C	**TM_cnt7_match1**	Counter 7 first match
0x70	**TM_cnt7_match2**	Counter 7 second match
0x74	**TM_cnt8_sts**	Counter 8 status
0x78	**TM_cnt8_reload**	Counter 8 reload value
0x7C	**TM_cnt8_match1**	Counter 8 first match
0x80	**TM_cnt8_match2**	Counter 8 second match
0x84	**TM_ctrl1**	Control 1
0x88	**TM_cnt9_sts**	Counter 9 status
0x8C	**TM_cnt9_reload**	Counter 9 reload value
0x90	**TM_cnt9_match1**	Counter 9 first match
0x94	**TM_cnt9_match2**	Counter 9 second match
0x98	**TM_cnt10_sts**	Counter 10 status
0x9C	**TM_cnt10_reload**	Counter 10 reload value
0xA0	**TM_cnt10_match1**	Counter 10 first match
0xA4	**TM_cnt10_match2**	Counter 10 second match

Confidential
For Ambarella Internal Use Only

Offset	Register	Description
0XA8	**TM_fault_inject**	Timer fault inject, bit[0] to inject fault

Table 26-1. Interval Timer Registers.

26.3.2 Interval Timer Registers: Details

26.3.2.1 TM_cnt[n]_sts Register

Bits	Name	Access	Reset	Description
31:0	**cnt[n]_sts**	RW	0	Counter status **TM_cnt[n]_sts** register stores the current status of counter[n]. When the enable bit of **TM_ctrl** register is set, the counter will start to decrement. Software can modify the register value at any time.

Table 26-2. Interval Timer Counter Status Register.

26.3.2.2 TM_cnt[n]_reload Register

Bits	Name	Access	Reset	Description
31:0	**reload[n]**	RW	0	Counter[n] reload value register. The reload value is the interval. When the counter is decremented to 0, the reload value will be reloaded to the counter value automatically. For example, if **reload** is set to 2 the counter sequence will be 2,1,0,2,1,… Trigger one-shot mode by setting the **TM_cnt[n]_reload** bit **reload**=0, the **TM_cnt[n]_match1** bit **match1**!=0, the **TM_cnt[n]_match2** bit **match2**!=0, and **TM_ctrl** bit **of[n]** =0.

Table 26-3. Interval Timer Reload Value Register.

26.3.2.3 TM_cnt[n]_match1 Register

Bits	Name	Access	Reset	Description
31:0	**match1[n]**	RW	0	First match register When counter[n] matches this register, the timer will generate an edge trigger interrupt to the Arm core. Trigger one-shot mode by setting the **TM_cnt[n]_reload** bit **reload**=0, the **TM_cnt[n]_match1** bit **match1**!=0, the **TM_cnt[n]_match2** bit **match2**!=0, and **TM_ctrl** bit **of [n]** =0.

Table 26-4. Interval Timer Match 1 Register.

For Confidential Use Only
1.5 (08/25/2025)

26.3.2.4 TM_cnt[n]_match2 Register

Bits	Name	Access	Reset	Description
31:0	**match2[n]**	RW	0	Second set match register When counter[n] matches this register, the timer will generate an edge trigger interrupt to the Arm. Trigger one-shot mode by setting the **TM_cnt[n]_reload** bit **reload**=0, the **TM_cnt[n]_match1** bit **match1**!=0, the **TM_cnt[n]_match2** bit **match2**!=0, and **TM_ctrl** bit **of[n]**=0.

Table 26-5. Interval Timer Match 2 Register.

26.3.2.5 TM_ctrl Register

Trigger one-shot mode by setting the **TM_cnt[n]_reload** bit **reload**=0, the **TM_cnt[n]_match1** bit **match1**!=0, the **TM_cnt[n]_match2** bit **match2**!=0, and **TM_ctrl** bit **of[n]**=0. Three clock sources (extclk1, extclk2 and extclk3) can be selected to reduce power consumption for 1st, 2nd and 3rd timer.

Bits	Name	Access	Reset	Description
31	**Intclksel8**	RW	0	Clock selection for Timer/Counter 8d 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
30	**of8**	RW	0	Overflow for Timer/Counter 8 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
29	**Reserved**	-	-	Reserved
28	**enable8**	RW	0	Timer enable for Timer/Counter 8 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
27	**Intclksel7**	RW	0	Clock selection for Timer/Counter 7 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
26	**Of7**	RW	0	Overflow for Timer/Counter 7 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
25	**Reserved**	-	-	Reserved
24	**Enable7**	RW	0	Timer enable for Timer/Counter 7 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
23	**Intclksel6**	RW	0	Clock selection for Timer/Counter 6 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
22	**Of6**	RW	0	Overflow for Timer/Counter 6 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
21	**Reserved**	-	-	Reserved

For Internal Use Only

Bits	Name	Access	Reset	Description
20	**enable6**	RW	0	Timer enable for Timer/Counter 6 0 – Disable 1 – EnableNotes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
19	**Intclksel5**	RW	0	Clock selection for Timer/Counter 5 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
18	**Of5**	RW	0	Overflow for Timer/Counter 5 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
17	**Reserved**	-	-	Reserved
16	**Enable5**	RW	0	Timer enable for Timer/Counter 5 0 – Disable 1 – EnableNotes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
15	**Intclksel4**	RW	0	Clock selection for Timer/Counter 4 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
14	**of4**	RW	0	Overflow for Timer/Counter 4 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
13	**Reserved**	-	-	Reserved
12	**Enable4**	RW	0	Timer enable for Timer/Counter 4 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
11	**Intclksel3**	RW	0	Clock selection for Timer/Counter 3 Only valid when Clksel3 is set to 0. 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
10	**Of3**	RW	0	Overflow for Timer/Counter 3 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
9	**ExtClksel3**	RW	0	Clock selection for Timer/Counter 3 0 – Check Intclksel3 1 – External clock (EXTCLK)
8	**Enable3**	RW	0	Timer enable for Timer/Counter 3 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
7	**Intclksel2**	RW	0	Clock selection for Timer/Counter 2 Only valid when Clksel2 is set to 0. 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
6	**Of2**	RW	0	Overflow for Timer/Counter 2 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow

For Ambarella Confidential
1.5 Only (Preliminary)

Bits	Name	Access	Reset	Description
5	**ExtClksel2**	RW	0	Clock selection for Timer/Counter 2 0 – Check Intclksel2 1 – External clock (EXTCLK)
4	**Enable2**	RW	0	Timer enable for Timer/Counter 2 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
3	**Intclksel1**	RW	0	Clock selection for Timer/Counter 1 Only valid when Clksel1 is set to 0. 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
2	**Of1**	RW	0	Overflow for Timer/Counter 1 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
1	**ExtClksel1**	RW	0	Clock selection for Timer/Counter 1 0 – Check Intclksel1 1 – External clock (EXTCLK)
0	**Enable1**	RW	0	Timer enable for Timer/Counter 1 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.

Table 26-6. Interval Timer Control Register.

26.3.2.6 TM_ctrl1 Register

Bits	Name	Access	Reset	Description
31:8	**Reserved**	-	-	Reserved
7	**Intclksel10**	RW	0	Clock selection for Timer/Counter 10 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
6	**Of10**	RW	0	Overflow for Timer/Counter 10 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
5	**Reserved**	-	-	Reserved
4	**Enable10**	RW	0	Timer enable for Timer/Counter 10 0 – Disable 1 – Enable Notes: When timer is disabled, all actions for counter, reload, and interrupt will be halted.
3	**Intclksel9**	RW	0	Clock selection for Timer/Counter 9 0 – APB clock (PCLK) 1 – Internal clock (INTCLK)
2	**Of9**	RW	0	Overflow for Timer/Counter 9 0 – No interrupt on counter underflow 1 – Generate interrupt on counter underflow
1	**Reserved**	-	-	Reserved

For Media Lang only 1.5 (08012021) Confidential

Bits	Name	Access	Reset	Description
0	**Enable9**	RW	0	Timer enable for Timer/Counter 9 0 – Disable 1 – Enable Notes: When the timer is disabled, all actions for counter, reload, and interrupt will be halted.

Table 26-7. Interval Timer Control 1 Register.

26.3.2.7 TM_ctrl1 Register

Bits	Name	Access	Reset	Description
31:1	**Reserved**	-	-	Reserved
0	**fault_inj**	RW	0	Fault inject register 0 – Not inject fault 1 – Inject fault

Table 26-8. Timer Fault Injection Control Register.

26.4 Interval Timer: Safety Notes

The following provides safety notes, including a diagram of the fault detection flow of SM_SW_RT.

To force an error to the CEHU on the <TIMER0/1/2> unit:

1. Enable the ECRU error mask bit 17, 18 and 19 timerN_par_failure. N=0~2. .
2. Enable the CEHU error mask bit 154 amba_safety_error.
3. Write Timer0~2 0x5c 0x1.

To to clear the error status:

- Clear bit 17,18, and 19 timerN_par_failure in ECRU. N=0~2.

To disable error propagation from <TIMER> to CEHU:

- Disable the ECRU mask: bit 17,18 and 19 timerN_par_failure.

  Software can read from the ECRU of AHB bus and check error status bits 17,18, and 19 to determine if an error occurred.

Notes:

- SW should perform fault injection after boot up.
- Before any configuration changes, SW should first set 0x5C to 1. After configuration changes are complete, SW should set 0x5C to 0.

For confidential 1.5 Mekra tang only (09012025)

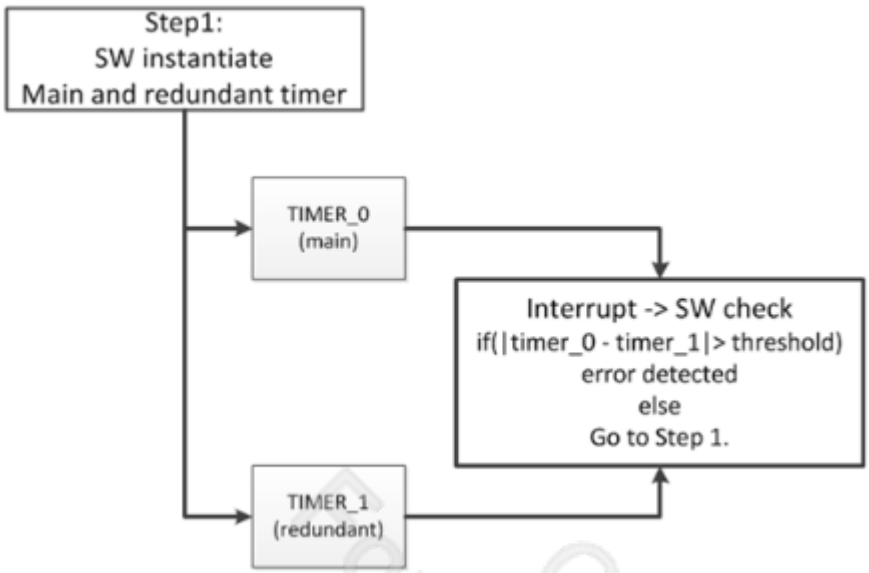


Figure 26-1. Fault Detection Flow of SM_SW_RT.

- When SW must use a timer, SW must instantiate an additional redundant timer with the same configuration. When the main timer interrupt asserts, the redundant timer's value shall be compared with the main timer to determine if the value is reasonable.

  The flow shown in the diagram above shall be implemented in ISR, when the difference is not as expected. SW shall program AXI configuration 0x158 to assert the safety error to CEHU.

  SW shall enable error source bit 7 (bit 7 in 0x3C) and corresponding safety_mode_0 to propagate the error to Arm and safety error pins.

Confidential
For Mokra Lang only
1.5 (08012025)

27. WATCHDOG TIMER (WDT)

This chapter covers the CV22FS watchdog timer (WDT) as follows:

- (Section 27.1) WDT: Overview
- (Section 27.2) WDT: Registers
- (Section 27.3) WDT: Software Programming Flow

27.1 WDT: Overview

The WDT is used to prevent system deadlock. When enabled, the software must reload the WDT periodically to prevent the watchdog counter from underflow (decrementing to zero). The WDT runs at the same rate as the APB bus clock and can be programmed to perform the following:

- Generate a system reset signal
- Generate an interrupt to the system interrupt controller
- Perform none of the above

27.2 WDT: Registers

27.2.1 WDT Registers: Map

Offset	Register	Description
0x00	**WDT_cnt_sts**	Counter status
0x04	**WDT_reload**	Counter reload value
0x08	**WDT_restart**	Counter restart
0x0C	**WDT_ctrl**	Control
0x10	**WDT_timeout**	Timeout status
0x14	**WDT_clr**	Clear timeout status
0x18	**WDT_rst_wd**	Reset width
0x1C	**WDT_fault_inject**	Fault injection

Table 27-1. APB Registers of the Watchdog Timer (WDT)

27.2.2 WDT Registers: Details

27.2.2.1 WDT_cnt_sts Register

When enabled, the watchdog counter decrements by 1 every APB bus clock period. The current counter value is in this Counter Status register. Decrementing this register below 0 will cause an underflow condition to occur. Software can prevent this underflow by writing a new value into the Watchdog Restart register.

Confidential For Mekra Lang only 1.5 (08/05/2025)

Bits	Name	Access	Reset	Description
31:0	**cnt_sts**	R	0x3EF1480	Current counter value of the watchdog timer The **cnt_sts** starts to decrement once the **WDT_ctrl** bit 0 (**enable**) is set.

Table 27-2. WDT Counter Status Register.

27.2.2.2 WDT_reload Register

Bits	Name	Access	Reset	Description
31:0	**reload**	RW	0x3EF1480	This value reloads to the **WDT_cnt_sts** register when it underflows or software writes “0x4755” to restart the register.

Table 27-3. WDT Counter Reload Value Register.

27.2.2.3 WDT_restart Register

To prevent the watchdog timer from underflowing, software must write to this register periodically and in a timely manner.

Bits	Name	Access	Reset	Description
15:0	**restart**	W	0	When software writes “0x4755” to this register, values in “reload” register will be loaded to the **WDT_cnt_sts** register

Table 27-4. WDT Counter Restart Register.

27.2.2.4 WDT_ctrl Register

Bits	Name	Access	Reset	Description
31:4				Reserved
3	**wdt_ext_en**	RW	0	Reset system after timeout for external used 0 - Disable system reset on counter underflow 1 - Enable system reset on counter underflow
2	**intenable**	RW	0	Interrupt enable 0 - Disable interrupt on counter underflow 1 - Enable interrupt on counter underflow
1	**rstenable**	RW	0	Reset system after timeout 0 - Disable system reset on counter underflow 1 - Enable system reset on counter underflow
0	**enable**	RW	0	WDT enable signal 0 - Disable watchdog timer 1 - Enable watchdog timer

Table 27-5. WDT Control Register.

Confidential
For Mektec Lang only
1.5 (08012025)

27.2.2.5 WDT_timeout Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**timeout**	R	0	Indicate timeout Only valid when **WDT_ctrl[2]** is 1 and **WDT_ctrl[1]** is 0. If the WDT has reached the timeout condition, this bit will be set automatically. 0 - Timeout has not occurred 1 - Timeout has occurred

Table 27-6. WDT Timeout Status Register.

27.2.2.6 WDT_clr Register

Bits	Name	Access	Reset	Description
31:1				Reserved
0	**clr**	W	0	Clear timeout status Write 1 value into this bit to clear timeout register

Table 27-7. WDT Clear Timeout Status Register.

27.2.2.7 WDT_rst_wd Register

The watchdog timer can generate an interrupt to the GIC as well as a reset signal to the RCT. The pulse width of the reset signal and the reset interrupt is controlled by this register.

Bits	Name	Access	Reset	Description
31:0	**rstwd**	RW	0xFF	Reset width

Table 27-8. WDT Reset Width Register.

27.2.2.8 WDT_fault_inject Register

The fault injection register is used to check if the safety error output can assert successfully in order to be detected by CEHU.

Bits	Name	Access	Reset	Description
31:2				Reserved
1	**fault_inject_ext**	RW	0	Software fault injection for external reset Write “1” to assert the external reset.
0	**fault_inject**	RW	0	Software fault injection Write “1” to assert the error output.

Table 27-9. WDT_fault_inject Register.

For Mokra Lang only (08/12/2025) Confidential

27.3 WDT: Software Programming Flow

The following describes the software programming flow of the watchdog.

1. Reset WDT.
2. Set up the WDT control registers in order.
   A. Write the WDT_reload register for the WDT_cnt_sts register reloaded value.
   B. Write the WDT_rst_wd register to set up the reset pulse width.
   C. Write the WDT_ctrl register to set up the external reset enable, interrupt enable, internal reset enable, and WDT enable. The countdown to WDT begins when WDT enable is set.
3. Poll the WDT_cnt_sts register to see the current counter value of the watchdog timer and poll the WDT_timeout register to see if timeout has occurred.
4. Write the WDT_clr register to clear the timeout status.
5. Write the WDT_restart register to reload value to the WDT_cnt_sts register.

For Mekra Lang only Confidential 1.5 (08012025)

28. CHIP ERROR HANDLING UNIT (CEHU)

This chapter includes the following sections:

- (Section 28.1) CEHU Overview
  - (Section 28.1.1) Safety Pin Functionality
- (Section 28.2) CEHU Error Sources
- (Section 28.3) Software Interface
  - (Section 28.3.1) Register Descriptions
- (Section 28.4) Software Programming Flow
  - (Section 28.4.1) From Detecting Latent Fault to Normal Operation

28.1 CEHU Overview

The purpose of the chip error handling unit (CEHU), also known as the “central error handling unit,” is as follows:

- Alert the external system that the chip is in error and entering a safe-state via a safety pin.
- Record the ID of the device that first reported the error (for debug purposes).

The system includes two copies of the CEHU with two separate safety pins. If both safety pins indicate safety, then the chip is functioning correctly.

28.1.1 Safety Pin Functionality

The following figure illustrates how the safety pin functions. Immediately following reset, the external safety pin is active low, indicating an error, until the boot process completes; then the software completes all safety checks, including checking safety monitors for latent faults, and determines that the CEHU and chip function is correct.

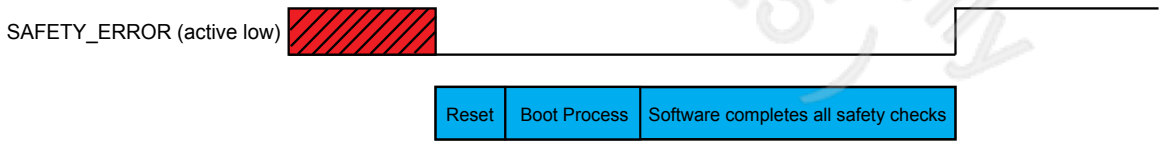


Figure 28-1. Safety Pin Functionality.

Confidential
For Ambarella Lanonly
1.5 (08012025)

28.2 CEHU Error Sources

The following table shows the CEHU input sources map of 155 used error bits.

ID	Source	Not Maskable(1)	Not Resettable by Warm Reset(2)	Description
154	amba_safety_error			Safety error from ECRU
153	biasgen_safety_error	V	V	Safety error from Band Gap
152:142	reserved_ckmon_failure			Reserved. 11 bits clock monitor failure is not used.
141	mipi_dphy_dbg_slave_par_fail			Safety error from mipi_dphy_ dbg
140	pm_errorb15	V		Safety error from power moni- tor.
139	ddrhost0_dram_ckmon_error			Safety error from clock monitor at ddrhost0
138	ddrhost0_dram_div2_ckmon_error			Safety error from clock monitor at ddrhost0 div2
137	ddrhoste_dram_ckmon_error			Safety error from clock monitor at ddrhost
136	ddrhoste_dram_div2_ckmon_error			Safety error from clock monitor at ddrhost div2
135	fex_cehu_fex_ck_mon_error			Safety error from clock monitor at FEX
134	storc_vorc_ckmon_safety_error			Safety error from clock monitor at STORC
133	storc_l2c_ckmon_safety_error			Safety error from clock monitor at L2C
132	clk_ref_err	V	V	Safety error from clock monitor for ref_clk
131	vpa_cehu_dbg_ckmon_error			Safety error from clock monitor for dbg clock at VP
130	vpa_cehu_vision_ckmon_error			Safety error from clock monitor for vision clock at VPA
129	vpb_cehu_vision_ckmon_error			Safety error from clock monitor for vision clock at VPB
128	vpc_cehu_vision_ckmon_error			Safety error from clock monitor for vision clock at VPC
127	vpd_cehu_vision_ckmon_error			Safety error from clock monitor for vision clock at VPD
126	vpf_cehu_vision_ckmon_error			Safety error from clock monitor for vision clock at VPF
125	vmem_cehu_gclk_vision_ck_mon_ error			Safety error from clock monitor for vision clock at VMEM
124	vdsp_cehu_clk_dbg_ck_mon_error			Safety error from clock monitor for dbg clock at VDSP
123	vdsp_cehu_clk_core_ck_mon_error			Safety error from clock monitor for core clock at VDSP
122	smem_cehu_core_ck_mon_error			Safety error from clock monitor for core clock at SMEM

Confidential For Mekra Lang only (08012025)

ID	Source	Not Maskable[1]	Not Resettable by Warm Reset[2]	Description
121	**orc_cehu_clk_dbg_ck_mon_error**			Safety error from clock monitor for dbg clock at ORC
120	**orc_cehu_clk_core_ck_mon_error**			Safety error from clock monitor for core clock at ORC
119	**code_cehu_clk_dbg_ck_mon_error**			Safety error from clock monitor for dbg clock at CODE
118	**code_cehu_clk_core_ck_mon_error**			Safety error from clock monitor for core clock at CODE
117:99	**rct_ckm_err_flag[18:0]**		V	Safety error from clock monitor at RCT
98	**idsp1_cehu_idsp_ck_mon_error**			Safety error from clock monitor for IDSP clock at IDSP1
97	**idsp1_cehu_core_ck_mon_error**			Safety error from clock monitor for core clock at IDSP1
96	**idsp1_cehu_dbg_ck_mon_error**			Safety error from clock monitor for dbg clock at IDSP1
95	**idsp0_cehu_idsp_ck_mon_error**			Safety error from clock monitor for idsp clock at IDSP0
94	**idsp0_cehu_core_ck_mon_error**			Safety error from clock monitor for core clock at IDSP0
93	**idsp0_cehu_dbg_ck_mon_error**			Safety error from clock monitor for dbg clock at IDSP0
92:86	**ckm_err_flag_amba1[9:3]**			Safety error from clock monitor for {gclk_axi, gclk_ahb, gclk_core, gclk_core_ahb, gclk_dbg2dbg, gclk_fex, combophy_clkb} at AMBA1
85	**ckm_err_flag_amba1[2]**			Safety error from clock monitor for combophy_clka at AMBA1
84:83	**ckm_err_flag_amba1[1:0]**			Safety error from clock monitor for { gclk_dram_div2, gclk_vision} at AMBA1
82:80	**ckm_err_flag_amba0[24:22]**			Safety error from clock monitor for { gclk_vo_b, gclk_vo_a, gclk_axi1} at AMBA0
79	**ckm_err_flag_amba0[21]**			Safety error from clock monitor for gclk_gtx at AMBA0
78:58	**ckm_err_flag_amba0[20:0]**			Safety error from clock monitor for {gclk_ssi_dft, gclk_ssi2_dft, gclk_ssi3_dft, gclk_uart0_dft, gclk_uart1_dft, gclk_uart2_dft, gclk_uart3_dft, gclk_uart_apb_dft, gclk_core_ahb, gclk_core_dft, gclk_ahb_dft, gclk_apb_dft, gclk_dbg_dft, gclk_sdio0_dft, gclk_sd48_dft, gclk_nand, n0_gclk_sys_cnt_dft, n1_gclk_sys_cnt_dft, gclk_pwm_dft, gclk_can_dft, gclk_au_dft,} in AMBA0~

For Mekra Lang only Confidential (08012025)

ID	Source	Not Maskable[1]	Not Resettable by Warm Reset[2]	Description
57	**ckm_err_flag_axi[4]**			Safety error from clock monitor clock at cortex A53
56	**ckm_err_flag_axi[3]**			Safety error from clock monitor clock at cortex R52
55:53	**ckm_err_flag_axi[2:0]**			Safety error from clock monitor for { n1_pclkdbg_clk, n0_pclk-dbg_clk, gclk_axi1} at AXI
52	**code_cehu_safety_error**			Safety error from CODE
51	**cpu_monitor_safety_error**			Safety error from CPU_MON
50	**ddrhost0_ddrc_safety_error**		V	Safety error from DDRHOST0_ DDRC
49	**ddrhost0_dtte_safety_error**		V	Safety error from DDRHOST0_ DTTE
48	**ddrhoste_ddrc_safety_error**		V	Safety error from DDRHOST_ DDRC
47	**ddrhoste_dtte_safety_error**		V	Safety error from DDRHOST_ DTTE
46	**dorc_cehu_safety_error**			Safety error from DORC
45	**dram_controller_safety_error**			Safety error from DRAMC
44	**eorc_cehu_safety_error**			Safety error from EORC
43	**fex_cehu_safety_error**			Safety error from FEX
42	**idsp_safety_error**			Safety error from IDSP
41	**ijtag_reset_cehu_monitor_safety_ error**	V		Safety error from IJTAG
40	**Reserved**			Reserved
39	**mddec_cehu_safety_error**			Safety error from MDDEC
38	**mdenc_cehu_safety_error**			Safety error from MDENC
37	**mipi_dphy_safety_error**		V	Safety error from MIPI_DPHY
36:31	**mipi_dsi_safety_error**			Safety error from MIPI_DSI
30	**ol2c_safety_error**			Safety error from OL2C
29	**osc_monitor_safety_error**	V	V	Safety error from clock monitor clock for rtc clock at amba
28	**otp_safety_error**		V	Safety error from OPT
27:11	**power_monitor_safety_error**	V		Safety error from POWER_ MON
10	**rct_safety_error**		V	Safety error from RCT
9	**rtc_dbg_sm_safety_error**			Safety error from RCT_DBG
8	**smem_safety_error**			Safety error from SMEM
7	**sw_arm_safety_error**			Safety error from AHB_ SCRATCH_PAD. SW shall set it to 1 when using SW safety mechanism. For more detail, please see AHB_SCRATCH_ PAD.
6	**test_mode_cehu_monitor_safety_ error**	V		Safety error from BIST

For Mekra Lang only (08012025) Confidential

ID	Source	Not Maskable(1)	Not Resettable by Warm Reset(2)	Description
5	**tsfm_cehu_safety_error**			Safety error from TSFM
4	**vmem_safety_error**			Safety error from VMEM
3	**vorc_safety_error**			Safety error from VORC
2	**vp_safety_error**			Safety error from VP
1	**vrom_safety_error**			Safety error from VROM
0	**dummy**			

Table 28-1. Sources to ID Mapping Table.

Notes:

(1) The safety error bits (marked “V”) cannot be masked out by CEHU **error_bit_mask_0~4** (0x3C~0x4C).

(2) After warm reset, the safety error bits (marked “V”) keep their error status from before warm reset (instead of the default value). Software must determine whether or not to clear these bits after warm reset is complete.

The following table describes the **error_bit_vector** which holds the state of the error signals at input to the CEHU.

Bit value 0 = No error at corresponding input error signal; bit value 1 = error at corresponding input error signal.

Register	Bits	Access	Power-On Reset	Warm_Reset_Mask (corresponds to Reset column in the above table)	Description
error_bit_vector_0	31:0	RW1C	0xffff_ff3e	0x3000400	[0] = source_id 0 [1] = source_id 1 ... [31] = source_id 31
error_bit_vector_1	31:0	RW1C	0xffff_fcff	0x0078020	[0] = source_id 32 [1] = source_id 33 ... [31] = source_id 63
error_bit_vector_2	31:0	RW1C	0xffff_ffff	0x0000000	[0] = source_id 64 [1] = source_id 65 ... [31] = source_id 95
error_bit_vector_3	31:0	RW1C	0xffff_ffff	0x3FFFF8	[0] = source_id 96 [1] = source_id 97 ... [31] = source_id 127
error_bit_vector_4	31:0	RW1C	0x0600_3fff	0x2000010	[0] = source_id 128 [1] = source_id 129 ... [31] = source_id 160

Table 28-2. Description of Error_Bit_Vec_N

Note:

Error_bit_vector_0_0 is reserved; error_id = 0 means no error. Error pins from other clusters begin with **error_bit_vector_0_1**.

The following figure illustrates the error_bit_vector in CEHU @ warm reset (**error_bit_vector_warm_reset**). During this process, SW performs/check the following:

1. error_bit_vector_warm_reset and ~WARM_RESET_MASK == ~error_bit_mask and error_bit_vector_power_on and ~WARM_RESET_MASK.
2. error_bit_vector_warm_reset and WARM_RESET_MASK log the error status before warm reset.

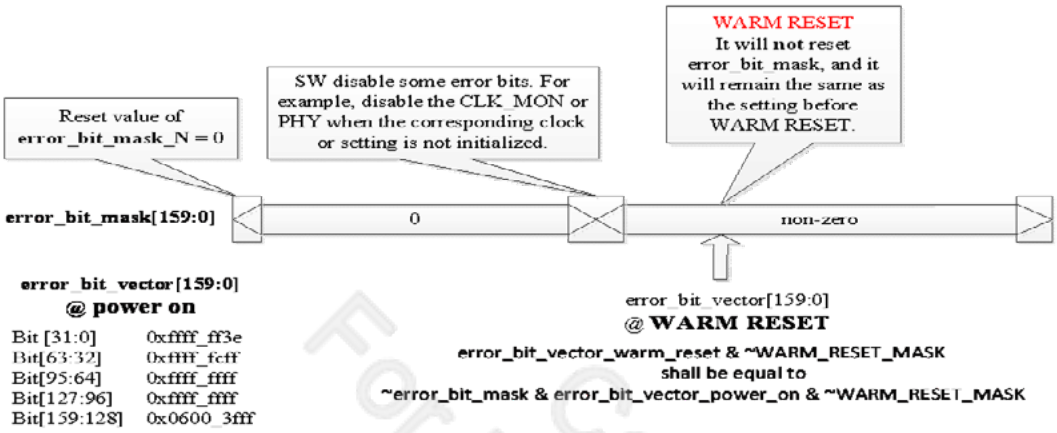


Figure 28-2. Error_bit_vector in CEHU.

IThe following table describes **safety_error_inj** register.

Bit value 0 = No error at corresponding inject error signal; bit value 1 = error at corresponding inject error signal.

Register	Bits	Access	Reset	Description
safety_error_inj_0	31:0	RW	0	[0] = source_id 0 [1] = source_id 1 ... [31] = source_id 31
safety_error_inj_1	31:0	RW	0	[0] = source_id 32 [1] = source_id 33 ... [31] = source_id 63
safety_error_inj_2	31:0	RW	0	[0] = source_id 64 [1] = source_id 65 ... [31] = source_id 95
safety_error_inj_3	31:0	RW	0	[0] = source_id 96 [1] = source_id 97 ... [31] = source_id 127
safety_error_inj_4	31:0	RW	0	[0] = source_id 128 [1] = source_id 129 ... [31] = source_id 160

Table 28-3. Description of Safety_Error_Inj_N .

For Mekatang only (Confidential)

28.3 Software Interface

28.3.1 Register Descriptions

The tables in this section include

- CEHU Registers
- Safety_mode_0~15 Register
- Safety_mode_t Description

The following describes the access types used in the tables.

- **RW**: Read-Write register. The register bits are read-write and can be either set or cleared by software.
- **RW1C**: Read-Only Status, Write-1 to clear Status. The register bits indicate status when read, and can be cleared by writing a 1. Writing a 0 to RW1C bits has no effect.

For Confidential
Fan Mekra Lang only
1.5 (08012025)

28.3.1.1 CEHU Registers

The following lists the CEHU registers.

Offset	Register	Bits	Field	Access	Reset	Description
0x000	**safety_mode_0**	31:0	**mode_0**	RW	0xAAAAAAAE	Refer to Table 28-5 for more detail. Reset value of safety mode is 2' safety_mode_0[3:2] = 2'b11 for VROM_ERROR
0x004	**safety_mode_1**	31:0	**mode_1**	RW	0xABAAAAAA	Reserved safety_mode_1[25:24] = 2'b11 for OTP_ERROR
0x008	**safety_mode_2**	31:0	**mode_2**	RW	0xAAAAAAAA	Reserved
0x00C	**safety_mode_3**	31:0	**mode_3**	RW	0xAAAAAAAA	Reserved
0x010	**safety_mode_4**	31:0	**mode_4**	RW	0xAAAAAAAA	Reserved
0x014	**safety_mode_5**	31:0	**mode_5**	RW	0xAAAAAAAA	Reserved
0x018	**safety_mode_6**	31:0	**mode_6**	RW	0xAAAAAAAA	Reserved
0x01C	**safety_mode_7**	31:0	**mode_7**	RW	0xAAAAAAAA	Reserved
0x020	**safety_mode_8**	31:0	**mode_8**	RW	0xAAAAAAAA	Reserved
0x024	**safety_mode_9**	31:0	**mode_9**	RW	0xAAAAAAAA	Reserved
0x28	**safety_error_inj_0**	31:0	**injection_0**	RW	0	Refer to Table 28-3 for more detail
...						
0x038	**safety_error_inj_4**	31:0	**injection_4**	RW	0	Refer to Table 28-3 for more detail
0x03C	**error_bit_mask_0**	31:0	**error_bit_mask_0**	RW	0	error_bit_mask_N[i] is used to mask out i-th input error signals. error_bit_mask_N[i]==0, i-th input is enabled error_bit_mask_N[i]==1, i-th input is disabled
...						
0x04C	**error_bit_mask_4**	31:0	**error_bit_mask_4**	RW	0	error_bit_mask_N[i] is used to mask out i-th input error signals. error_bit_mask_N[i]==0, i-th input is enabled error_bit_mask_N[i]==1, i-th input is disabled
0x050	**error_bit_por_0**	31:0	**error_bit_por_0**	RW	1	error_bit_por_N[i] is used to choose i-th input porlarity. error_bit_por_N [i]==1, i-th input is active high error_bit_por_N [i]==0, i-th input is active low
...						

For Ambarella Confidential use only (Oct 2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x060	**error_bit_por_4**	31:0	**error_bit_por_4**	RW	1	error_bit_por_N[i] is used to choose i-th input porlarity. error_bit_por_N [i]==1, i-th input is active high error_bit_por_N [i]==0, i-th input is active low
0x64	**error_bit_vector_0**	31:0	**err_bit_vec_0**	RW1C		Refer to Table 28-2 for more detail
...						
0x74	**error_bit_vector_4**	31:0	**err_bit_vec_4**	RW1C		Refer to Table 28-2 for more detail
0x78	**reserved**	31:8	**reserved**	-		Reserved
	error_id	7:0	**error_id**	RW		The error ID provides the device that first encountered the error A value of '0' indicates the chip is executing normal operation Software should clear it after safety injection
0x7c	**safety_check_done**	31:1	**reserved**	-	-	Reserved
		0	**safety_check_done**	RW	0	0 No safety check 1 Safety_check

Table 28-4. CEHU Registers.

For Media Lang only Confidential 1.5 (08012025)

28.3.1.2 Safety_Mode_0~9 Register and Safety_Mode_T Description

Note that each register **safety_mode_N** corresponds to 16 error bit's **safety_mode_t**. Refer to Table 28-6 in this section for the description of **safety_mode_t[1:0]**.

Register	Bits	Field	Access	Reset	Description
safety_mode_N	31:30	**mode_N_15**	RW	0	Safety_mode_t*
	29:28	**mode_N_14**	RW	0	
	…	**…**	…		…
	5:4	**mode_N_2**	RW	0	
	3:2	**mode_N_1**	RW	0	
	1:0	**mode_N_0**	RW	0	

Table 28-5. Safety_Mode_0~9 Register.

*The following table provides the description of **safety_mode_t**.

	Bits	Description
safety_mode_t	**1**	For corresponding the error pin to the safety pin 0 Do not propagate the error to safety pin 1 Propagate the error to the safety pin (default)
	0	For corresponding the error pin to the GIC 0 Do not generate an interrupt to the Arm's generic interrupt controller (GIC). (This is the default, with the exception of the VORC and OTP errors.) 1 Generate an interrupt to the Arm's generic interrupt controller (GIC)

Table 28-6. Description of Safety_Mode_T.

28.4 Software Programming Flow

28.4.1 From Detecting Latent Fault to Normal Operation

Safety_error_injection and **safety_mode** registers are used to determine that the CEHU is free of latent faults prior to normal operation. After the software completes checking the CEHU, it resets any error signals set during testing and configures the CEHU for normal operation (program 0xC4 **safety_check_done** to 1).

The following lists the flow for MIPI_DSI-related safety errors (ID 31:36 requires specific treatment--before using bit mask input and initialize source driver mipi_dsi_phy).

1. SW masks MIPI_DSI's safety_err in CEHU.
2. SW reprograms MIPI_DSI control registers in RCT.
3. SW unmasks MIPI_DSI's safety_err in CEHU.

Confidential For Mona Lang only (Feb 12 2025)

29. CPU ERROR MONITORING UNIT (CEMU)

This chapter includes the following sections:

- (Section 29.1) CEMU Overview
- (Section 29.2) Performance Specification
- (Section 29.3) Software Interface
- (Section 29.4) Software Programming Flow Example
- (Section 29.5) Cluster Function Descriptions

29.1 CEMU Overview

Although the Arm Cortex A53 cores provide ECC checking, they otherwise lack general safety-checking mechanisms. To detect if the Arm cores deviate from normal operation, CV22FS includes a CPU monitor that checks all read / write requests from Cortex A53.

The CPU monitor includes ten sets of configurable address ranges. Access outside of these configured address ranges is considered invalid and triggers an error signal to CEHU. For further details, refer to the software interface table and the software programming flow example in the following sections.

29.2 Performance Specification

Target clock frequency of clk_axi1 is 400 MHz.

29.3 Software Interface

The following registers are located in axi_config. For further details, refer to Chapter 17 “AXI Configuration”

Offset	Register	Bits	Field	Access	Reset	Description
0XC8	**addr_start_0**	31:0	**addr_start_0**	RW	0X0	Starting address 0 (reserved for DRAM)
0XCC	**addr_end_0**	31:0	**addr_end_0**	RW	0XDFFDFFFF	Ending address 0 **(reserved for DRAM)**
0XD0	**addr_start_1**	31:0	**addr_start_1**	RW	0XDFFE0000	Starting address 1 (Reserved for DRAM)
0XD4	**addr_end_1**	31:0	**addr_end_1**	RW	0XDFFE7FFF	Ending address 1 (reserved for DRAM)
0XD8	**addr_start_2**	31:0	**addr_start_2**	RW	0XF2000000	Starting address 2 (reserved for DRAM)
0XDC	**addr_end_2**	31:0	**addr_end_2**	RW	0XF200FFFF	Ending address 2 (reserved for DRAM)
0XE0	**addr_start_3**	31:0	**addr_start_3**	RW	0XE0000000	Other starting address (default value is AHB starting address)
0XE4	**addr_end_3**	31:0	**addr_end_3**	RW	0XE3FFFFFF	Other ending address (default value is AHB ending address)

Offset	Register	Bits	Field	Access	Reset	Description
0XE8	addr_start_4	31:0	addr_start_4	RW	0XE4000000	Other starting address (default value is APB starting address)
0XEC	addr_end_4	31:0	addr_end_4	RW	0XE7FFFFFF	Other ending address (default value is APB ending address)
0XF0	addr_start_5	31:0	addr_start_5	RW	0XED000000	Other starting address (default value is APB_DBG starting address)
0XF4	addr_end_5	31:0	addr_end_5	RW	0XEFFFFFFF	Other ending address (default value is APB_DBG ending address)
0XF8	addr_start_6	31:0	addr_start_6	RW	0XF3000000	Other starting address (default value is GIC starting address)
0XFC	addr_end_6	31:0	addr_end_6	RW	0XF3FFFFFF	Other ending address (default value is GIC ending address)
0X100	addr_start_7	31:0	addr_start_7	RW	0XF1000000	Other starting address
0X104	addr_end_7	31:0	addr_end_7	RW	0XF10FFFFF	Other ending address
0X108	addr_start_8	31:0	addr_start_8	RW	0X0	Other starting address
0X10C	addr_end_8	31:0	addr_end_8	RW	0X0	Other ending address
0X110	addr_start_9	31:0	addr_start_9	RW	0X0	Other starting address
0X114	addr_end_9	31:0	addr_end_9	RW	0X0	Other ending address
0x118	range_ck_en	31:10	Reserved	-	-	
		9	range_ck_en_9	RW	0X0	Address range checking enable
		8	range_ck_en_8	RW	0X0	Address range checking enable
		7	range_ck_en_7	RW	0X1	Address range checking enable
		6	range_ck_en_6	RW	0X1	Address range checking enable
		5	range_ck_en_5	RW	0X1	Address range checking enable
		4	range_ck_en_4	RW	0X1	Address range checking enable
		3	range_ck_en_3	RW	0X1	Address range checking enable
		2	range_ck_en_2	RW	0x1	Address range checking enable
		1	range_ck_en_1	RW	0x1	Address range checking enable
		0	range_ck_en_0	RW	0x1	Address range checking enable

For Internal Use Only Confidential

Offset	Register	Bits	Field	Access	Reset	Description
0X11C	error_output_mask	31:3	Reserved	-	-	
		2	arc_mask	RW	0x0	Set 1 to mask arc invalid from output
		1	nexterrirq_mask	RW	0x0	Set 1 to mask nEXTER-RIRQ from output
		0	ninterrirq_mask	RW	0x0	Set 1 to mask nINTER-RIRQ from output
0X120	invalid_addr	31:0	invalid_addr	R	0x0	The invalid address which causes an address range checking error
0X124	invalid_core	31:2	Reserved	-	-	
		1:0	invalid_core	R	0x0	The CPU core which attempts to access an invalid address. 00: core0 01: core1 10: core2 11: core3

Table 29-1. CEMU Software Registers.

29.4 Software Programming Flow Example

29.4.1 Configuration for Address Range Checking

1. Set the starting and ending address of DRAM if the actual address range is narrower or different than the default value.

   Set addr_start_0 / addr_end_0 ~ addr_start_2 / addr_end_2 to 0x0000_0000 / 0x7FFE_FFFF, 0x8000_0000 / 0xDFFD_FFFF and 0xDFFE_8000 / 0xDFFF_FFFF. (Refer to the figure on the next page.)
2. Set the starting and ending addresses of all the desired IPs / blocks, such as VOUT, UART, and scratchpad. (Maximum 7 sets. addr_start_3 / addr_end_3 ~ addr_start_9 / addr_end_9.)

   To disable this feature, set addr_start_7 / addr_end_7 to GPV starting / ending address which is 0xF000_0000 / 0xF00F_FFFF.
3. Set range_ck_en accordingly to enable range checking. range_ck_en_0 controls addr_start_0 / addr_end_0, range_ck_en_1 controls addr_start_1 / addr_end_1, and so on.
4. Perform steps 2 and 3 to access different desired IPs / blocks.

For Mekra Lang only
Confidential
1.5 (osh1cdg)

The following figure shows the mapping for checking address ranges of CV22FS main CPU.

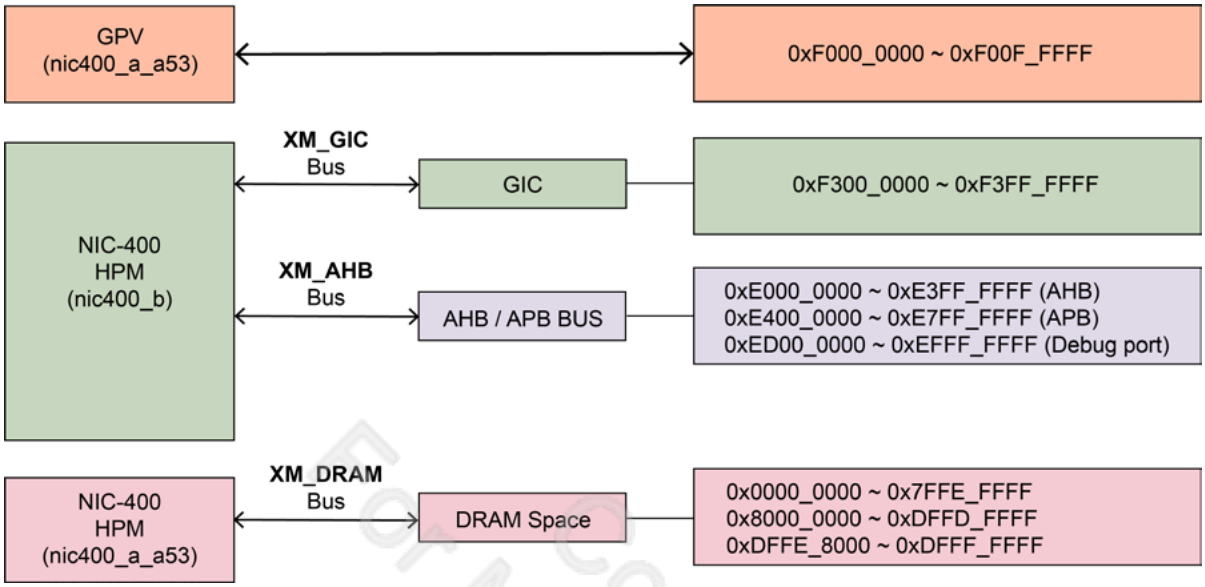


Figure 29-1. Address Mapping for Address Range Checking.

29.5 Cluster Function Descriptions

The figure below shows the top-level block diagram of the cluster functions. Descriptions of the blocks within the clusters follow.

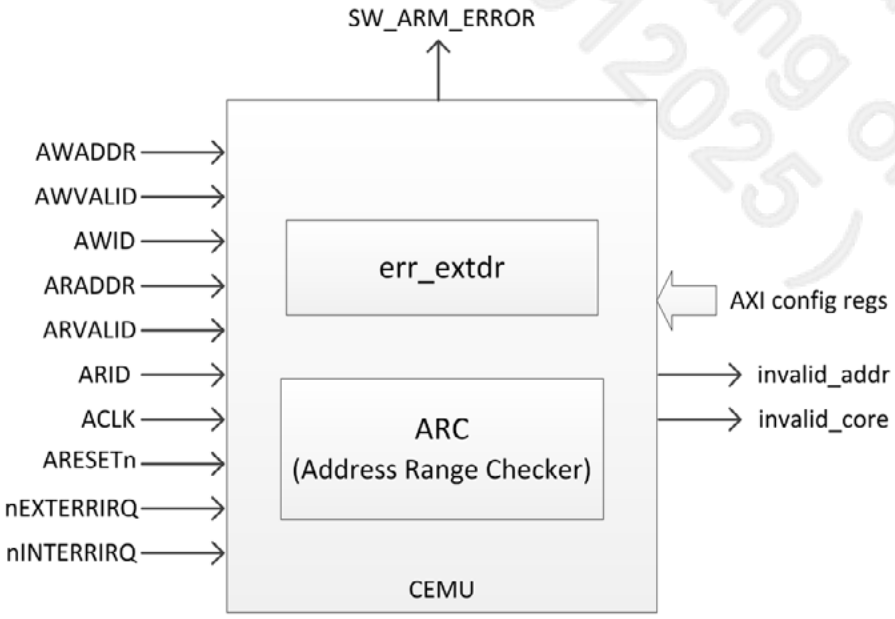


Figure 29-2. Block Diagram.

Confidential
For Mekra Lang only
1.5 (08012025)

29.5.1 Address Range Checker (ARC)

The primary function of this block is to check if the request address from the CPU is within the valid range. There are ten sets of SW programmable ranges. The ARC sends an error signal to the CEHU if the CPU attempts to access non pre-defined address ranges.

29.5.2 Error Extender (err_extdr)

The primary function of this block is to extend the error signal so that the CEHU (running in the slower clock domain) can detect it.

Confidential
For Mekra Lang only
1.5 (08012025)

30. AMBA ECRU

This chapter provides information for the AMBA error collection and reporting unit (AMBA ECRU) and includes the following sections:

- (Section 30.1) AMBA ECRU Overview
- (Section 30.2) Software Interface
- (Section 30.3) Software Programming Flow

30.1 AMBA ECRU Overview

The purpose of the AMBA error reporting and collection unit (AMBA ECRU) is as follows:

- Collect and record the safety mechanism errors from each IP in the AMBA cluster
- Generate one consolidated error signal to the CEHU

30.2 Software Interface

The tables in this section include

- Registers used in AMBA ECRU
- Error assignments for AMBA ECRU

30.2.1 AMBA ECRU Registers

The following lists the AMBA ECRU registers. Registers with the suffix "_0" correspond to bits 0 ~ 31 in Table 30-2 AMBA ECRU Error Assignments.; registers with suffix "_1" correspond to bits 32 ~ 63 in the table, etc.

Offset	Register	Bits	Field	Access	Reset	Description
0x0	**error_mask_0**	31:0	error_ mask_0	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error output
0x4	**error_mask_1**	31:0	error_ mask_1	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error output

Confidential For Mekra Lang only 1.5 (0801-2023)

Offset	Register	Bits	Field	Access	Reset	Description
0x8	**error_mask_2**	31:0	error_ mask_2	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error output
0xC	**error_mask_3**	31:19	Reserved	-	-	
		18:0	error_ mask_3	**RW**	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error output
0x10	**inserted_error_0**	31:0	inserted_error_0	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to insert an error to the corresponding error signal
0x14	**inserted_error_1**	31:0	inserted_error_1	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to insert an error to the corresponding error signal
0x18	**inserted_error_2**	31:0	inserted_error_2	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to insert an error to the corresponding error signal
0x1C	**inserted_error_3**	31:19	Reserved	-	-	
		18:0	inserted_error_3	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to insert an error to the corresponding error signal
0x20	**clear_error_0**	31:0	clear_error_0	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to clear the corresponding error signal
0x24	**clear_error_1**		clear_error_1			
0x28	**clear_error_2**		clear_error_2			

Confidential
For Mojra Lang only
1.5 (08/12/2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x2C	**clear_error_3**	31:19	Reserved	-	-	
		18:0	clear_error_3	RW	0x0	Each bit of this field correspond to a specific error (see Table 30-2) Set the bit to 1 to clear the corresponding error signal
0x30	**input_mask_0**	31:0	input_mask_0	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error input
0x34	**input_mask_1**	31:0	input_mask_1	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error input
0x38	**input_mask_2**	31:0	input_mask_2	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error input
0x3C	**input_mask_3**	31:19	Reserved	-	-	
		18:0	input_mask_3	RW	0x0	Each bit of this field controls a specific error (see Table 30-2) When the bit is 1, the corresponding error will be masked from amba_ecru error input
0x40	**error_log_0**	31:0	error_log_0	R	0xffffffff	Log register for recording error bit 0~31 (see Table 30-2)
0x44	**error_log_1**	31:0	error_log_1	R	0xfff7ffc0	Log register for recording error bit 32~63 (see Table 30-2)
0x48	**error_log_2**	31:0	error_log_2	R	0xffffffff	Log register for recording error bit 64~95 (see Table 2)
0x4C	**error_log_3**	31:19	Reserved	-	-	
		18:0	error_log_3	R	0x7ffff	Log register for recording error bit 96~114 (see Table 30-2)

Confidential
For Media Lang only
1.5 (08 2025)

Offset	Register	Bits	Field	Access	Reset	Description
0x50	**pri_mem_err_idx0**	31:25	Reserved	-	-	
		24:0	pri_mem_ err_idx0	R	0x0	R52 primary memory error reporting interface index
0x54	**pri_mem_err_mem0**	31:15	Reserved	-	-	
		14:0	pri_mem_ err_mem0	R	0x0	R52 primary memory error reporting interface memory identifier
0x58	**sec_mem_err_idx0**	31:25	Reserved	-	-	
		24:0	sec_mem_ err_idx0	R	0x0	R52 secondary memory error reporting interface index
0x5C	**sec_mem_err_mem0**	31:15	Reserved	-	-	
		14:0	sec_mem_ err_mem0	R	0x0	R52 secondary memory error reporting interface memory identifier

Table 30-1. AMBA ECRU Registers.

For Mekra Lang only
1.5 (Oct 25)
Confidential

30.2.2 Error Assignments for AMBA ECRU

The following table lists the error assignments for AMBA ECRU.

Bits	Error Source
114	ecch_sm_error
113	dbse_sm_error
112	vsdelay_valid_clk_sm_error
111	axi_cfg_wdata_par_failure
110	gic_wdata_par_failure
109	r52_rdata_par_failure
108	a53_rdata_par_failure
107	pmuevent3
106	pmuevent2
105	pmuevent1
104	pmuevent0
103	r52_remap_err
102	axi_config_par_failure
101	system_timer_a53_lockstep_error
100	system_timer_r52_lockstep_error
99:82	dclscompout[17:0]
81:56	errevent0[25:0]
55	secmemerrv0
54	primemerrv0
53	vout_sm_error

Bits	Error Source
52	vsdelay_valid_clk_config_sm_error
51	apb_dbg_time_out
50	ahb_wdata_par_failure
49	gdma_cehu_error_detected
48	iomux_ahb_reg_par_failure
47	apbbrg_sm_error
46	ahb_scratchpad_sec_par_failure
45	ahb_scratchpad_nsec_par_failure
44	canc5_sm_error
43	canc4_sm_error
42	canc3_sm_error
41	canc2_sm_error
40	canc1_sm_error
39	canc0_sm_error
38	ahb_bus_sm_error
37	wdt_r52_ext
36	wdt_a53_ext
35	wdt_core3_ext
34	wdt_core2_ext
33	wdt_core1_ext
32	wdt_core0_ext
31	apb_dbg_sm_error
30	nor_spi_par_failure
29	dma1_sm_error
28	dma0_sm_error
27	dma_ahb_sm_error
26	ssis0_par_failure
25	ssi5_par_failure
24	ssi4_par_failure
23	ssi3_par_failure
22	ssi2_par_failure
21	ssi1_par_failure
20	ssi0_par_failure
19	timer2_par_failure
18	timer1_par_failure
17	timer0_par_failure
16	idcs_par_failure
15	idc5_par_failure
14	idc4_par_failure
13	idc3_par_failure
12	idc2_par_failure

For Confidential Lang only
1.5 (Mar 2025)

Bits	Error Source
11	idc1_par_failure
10	idc0_par_failure
9	gpio3_par_failure
8	gpio2_par_failure
7	gpio1_par_failure
6	gpio0_par_failure
5	wdt_r52_sm_failure
4	wdt_core3_sm_failure
3	wdt_core2_sm_failure
2	wdt_core1_sm_failure
1	wdt_core0_sm_failure
0	wdt_sm_failure

Table 30-2. AMBA ECRU Error Assignments.

30.3 Software Programming Flow

The following provides the recommended software programming flow for AMBA ECRU.

1. To block specific error sources listed in Table 30-2 from propagating to CEHU, set error_mask_0~3 or input_mask_0~3 to 1.

   Error_mask_0~3 blocks errors from outputting, but still records each error error_log_0~3 coming into amba_ecru.

   In contrast, input_mask_0~3 block errors from coming into amba_ecru; as a result, no error is recorded in error_log_0~3.
2. Set clear_error_0~3 to 1 to clear corresponding bits of error_log_0~3. Ensure that clear_error_0~3 is reset to 0 to continue recording error input.
3. Set inserted_error_0~3 to insert errors to specify bits. This method enables identifying faults that exist on the path between amba_ecru and the CEHU.

Confidential For Meera Lang only (080125)

31. DRAM

The DRAM module is a traffic controller or arbitration module for clients which access the DDR controller.

This chapter includes the following sections:

- (Section 31.1) DRAM Registers
  - (Section 31.1.1) Global Physical Memory Map
  - (Section 31.1.2) Safety-Related Debug Registers
  - (Section 31.1.3) Client Interface (Clientif)
- (Section 31.2) DRAM BIST and Scrubbing Engine (DBSE)

31.1 DRAM Registers

Configuration space must be accessed by AXI0 only; all other clients have access to DRAM only.

This section includes DRAM registers and a table for the global physical memory.

Offset	Register	Bits	Field	Access	Reset	Description
0x0	**REG_DRAM_MODE**	2:0		R	0x0	Reserved
		4:3	**intlv_bdry**	RW		DDR's burst size 0 = Reserved 1=32 B 2=64 B
		7:5		R		Reserved
		9:8	Bank_mode	RW		0: 8 banks 1: 16 banks LPDDR4 dual die 2: DDR4 BG1x 4Bank
0x4	**REG_DRAM_INTERRUPT**	0		WC R	0x0	Interrupt status
0x8	**REG_DRAM_BW_THROTTLE_DLN**	31:0		RW	X	Contains the time window in which the BW throttle heuristic is evaluated. Must be set before throttling is enabled and before USAGE_TARGET is set.
0x10 – 0x7C	**REG_DRAM_CREDIT address = 0x10 + 4*client_id (Refer to** Table 31-4 "Clientif ID and Name.**")**	2:0		**If writes to any other client's request, credit registers are unaffected**	0x3	Req credit for the following: AXI0, AXI1, L2C

Confidential
For Wuxia Langdon
Tue Aug 08 2023

Offset	Register	Bits	Field	Access	Reset	Description
0x10 – 0x7C (Cont'd)		1:0		**If writes to any other client's request, credit registers are unaffected**	0x2	Req credit for the following: FDMA, ARMDMA0, ARMDMA1, ENET0, ENET1
0x80 – 0xF8	**REG_DRAM_PRIORITY address = 0x80 + 4*client_id (Refer to Table 31-4 "Clientif ID and Name.")**	3:0	Normal mode priority	RW	0x0	**Can be dynamically changed**
		7:4	BW throttle mode priority	RW	0x0	**Can be dynamically changed**
		11:8	RW throttle mode priority	RW	0x0	**Can be dynamically changed**
		15:12	Bank throttle mode priority	RW	0x0	**Can be dynamically changed**
		19:16	Granularity high priority	RW	0x0	**Can be dynamically changed**
		22:20	Granularity	RW	0x0	0: disabled 1: 512B 2: 1KB 3: 2KB 4: 4KB **Can be dynamically changed**
		23	last_bit disabled	RW	0x0	only for SMEM type of clients **Can be dynamically changed**
		27:24	reserve_cycles	RW	0x0	Number of arbiter cycles reserved after the client's grant for higher priority clients. [Actual value minus 1 is written to this register; at least 1 cycle is reserved, corresponding to a programmed value of 0x0]. *Cannot* be dynamically changed
		28	allow_rw_switch_disable_cycles	RW	0x0	**Can be dynamically changed**
		29	allow_row_switch_disable_cycles	RW	0x0	**Can be dynamically changed**

For Confidential Use Only
1.5 (08/12/25)

Offset	Register	Bits	Field	Access	Reset	Description
0xFC	**REG_DRAM_PRIORITY_SMEM_ HIPRI**	3:0	Normal mode priority	RW	0x0	**Can be dynamically changed**
		11:8	RW throttle mode priority	RW	0x0	**Can be dynamically changed**
		15:12	Bank throttle mode priority	RW	0x0	**Can be dynamically changed**
		19:16	Granularity high priority	RW	0x0	**Can be dynamically changed**
		27:20	**Reserved**	R	0x0	
		28	allow_rw_switch_disable_cycles	RW	0x0	**Can be dynamically changed**
		29	allow_row_switch_disable_cycles	RW	0x0	**Can be dynamically changed**
0x100 – 0x17C	**REG_DRAM_BW_THROTTLE address = 0x100 + 4*client_id (Refer to** Table 31-4 "Clientif ID and Name."**)**	8	BW_Throttle enable	RW	0x0	BW_Throttle enable (exception ENET0/1, rd throttle en is bit 17, wr throttle en is bit 8). Clients: ENET0/1, SDXC0/1, FDMA, SMEM
		7:0	Target <client> usage	RW	0x0	Target <client> usage *256 (exception ENET0/1, rd throttle usage is bits 16:9, wr throttle en is bits 7:0) Clients: ENET0/1, SDXC0/1, FDMA, SMEM **Can be dynamically changed**
0x190	**REG_DRAM_RW_THROTTLE**	5:0	post_rw_switch_opp_type_disable_cycles	RW	0x0	
		12:6	post_rw_switch_opp_type_disable_cycles_if_allowed	RW	0x0	
		21:13	post_rw_switch_opp_type_throttle_cycles	RW	0x0	
		25:22	post_grant_opp_req_type_throttle_cycles	RW	0x0	

For Mekro Lang only (08012025) Confidential

Offset	Register	Bits	Field	Access	Reset	Description
0x194	**REG_DRAM_BANK_ THROTTLE**	5:0	post_row_ switch_dif- ferent_row_ disable_cy- cles	RW	0x0	
		12:6	post_row_ switch_dif- ferent_row_ disable_cy- cles_if_al- lowed	RW	0x0	
		21:13	post_row_ switch_dif- ferent_row_ throttle_cy- cles	RW	0x0	
		25:22	post_grant_ different_ row_req_ throttle_cy- cles	RW	0x0	
		29:26	post_grant_ different_ die_dis- able_cycles	RW	0x0	When bank_mode=1, disable the other die request after a request is granted.
0x210	**REG_DRAM_ ACCESS_ PRIV(P)***	25:0		RW	0x0	1 in client bit position gives secure mode privileged access Only available following clients: AXI0, AXI1, CANC0, CANC1, CANC2, CANC3, CANC4, CANC5, VMEM0 , SMEM For others, unused
0x214	**REG_DRAM_ACCESS_ VIRTUAL(P)***	25:0		RW	0x0	1 in client bit position indicates virtual client
0x218	**REG_DRAM_SECMEM_ CTRL(P)***	0		RW	0x0	Enable secure memory check
		1		RW	0x0	Enable data scrambling
		2		RW	0x0	Enable no ECC for PPN
0x230- 0x240	**REG_DRAM_SEC_ KEY(P)***			WO	X	128+32 bit secure key for scrambler
0x248	**REG_DRAM_SECMEM_ BASE(P)***	13:0		RW	X	Page number of start of secure memory for a privileged client (inclusive)
0x24C	**REG_DRAM_SECMEM_ LIMIT(P)***	13:0		RW	X	Page number of end of secure memory for a privileged client (inclusive)
0x250	**REG_DRAM_SECVPN_ BASE(P)**	13:0		RW	X	Virtual page number of start of secure VPN region in ATT
0x254	**REG_DRAM_SECVPN_ LIMIT(P)***	13:0		RW	X	Virtual page number of end of secure VPN region in ATT
0x300	**REG_DRAM_VPN_ BASE[client_ID](P)***	13:0 base		RW	X	32 addresses reserved for segment table entries
0x380	**REG_DRAM_VPN_ BOUND[client_ID](P)***	13:0 bound		RW	X	32 addresses reserved for segment table entries

Confidential For Mass Lang only 1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x410	**REG_DRAM_SEG_FAULT**	25:0		R W1C	0x0	This register must be cleared in order for FAULT_ADDR and FAULT_CLIENT to be updated. 1 in client position indicates segmentation fault by virtual client or access to reserved memory by physical client.
0x414	**REG_DRAM_SECMEM_FAULT**	25:0		R W1C	0x0	This register must be cleared in order for FAULT_ADDR and FAULT_CLIENT to be updated. 1 in client position indicates an attempt by this client to access a protected location of memory / protected config registers.
0x418	**REG_DRAM_ATTWR_FAULT**	25:0		R W1C	0x0	This register must be cleared in order for FAULT_ADDR and FAULT_CLIENT to be updated. 1 in client position indicates an attempt by this client to write a secure PPN or VPN in the ATT.
0x41C	**REG_DRAM_FAULT_ADDR**	31:0		R	X	PA or VA of first error If ATTWR Fault is first error
		27:14		R	X	ATT VPN
		13:0		R	X	ATT PPN
0x420	**REG_DRAM_FAULT_CLIENT**	4:0	**client**	R	X	First client to cause an error ADDR type
		5	**addr type**	R	X	0: PHY 1: Virtual
0x8000	**REG_DRAM_ATT[vpn]**	13:0	**PPN even**	RW	X	VPN range is [0, 16k) 0x1_0000 to 0x1_fffc reserved for ATT entries Even / odd read_only is a read / writable register which inhibits write access for the virtual access. The write request to the virtual address with the bit 1 will be dropped. Note that its value after reset is not guaranteed. Can be dynamically changed.

Confidential
For Mekra Lang only
1 5108125)

Offset	Register	Bits	Field	Access	Reset	Description
0x8000 (Cont'd)		14	**even no_ ecc**	RW	X	
		15	**even read_ only**	RW	X	
		29:16	**PPN odd**	RW	X	
		30	**odd no_ ecc**	RW	X	
		31	**odd read_ only**	RW	X	
0x800-0xBFF	**REG_DRAM_DDRC_ BROADCAST_CONFIG (in DDRC)**	31:0		WO	X	Broadcast registers to all available DDRC This is only for writing DDRC's DRAM_CONTRL register, which includes DRAM_AUTO_REF_EN bit to synchronize refresh between multiple DDRCs Refer to *Table 2-1 Register Map in the CV22FS DDR Controller Programming Guide.*
0x1000-0x13FF	**REG_DRAM_DDRC0_ CONFIG**	31:0			X	Address is DRAM_CFG_BASE_ADDR + REG_DRAM_DDRCx_CONFIG + <DDRC internal address> Refer to *Table 2-1 Register Map in the CV22FS DDR Controller Programming Guide.*
0x1400-0x177F	**REG_DRAM_DDRCe_ CONFIG**				X	
0x1780-0x17FF	**REG_DRAM_ECCH0_ CONFIG**				X	
0x6000	**STAT_CLIENT_REQ[client id]**	31:0		R	X	Statistics for each client's number of requests Readable only when STAT_CONTROL is OFF

For Mekro Lang only
Confidential
1.5 (08012025)

Offset	Register	Bits	Field	Access	Reset	Description
0x6080	**STAT_CLIENT_BURSTS [client id]**	31:0		R	X	Statistics for each client's number of bursts Readable only when STAT_CONTROL is OFF
0x6100	**STAT_CLIENT_MASK [client id]**	31:0		R	X	Statistics for each client's number of masked write bursts Readable only when STAT_CONTROL is OFF
0x6180	**STAT_GLOBAL_BANK [bank id] where 0<= bank_id<=15**	31:0		R	X	Statistics for global numbers of bank opens Readable only when STAT_CONTROL is OFF
0x61C0	**STAT_GLOBAL_RW**	31:0		R	X	Statistics for global numbers of read/ write toggles Readable only when STAT_CONTROL is OFF
0x61C4	**STAT_GLOBAL_DIE**	31:0		R	X	Statistics for global numbers of die toggles when bank_mode=1 Readable only when STAT_CONTROL is OFF
0x61D0	**STAT_CONTROL**	1:0		RW	X	0: OFF 1: ON 2: Reset R: When reset is done, the read value will be changed to 0 (OFF)
0X7000	**DRAM_DBG_MMU_ STATE**	31:0		R	X	MMU internal state (cfg_cnt, buff valid and mmu_state)
0x7008-0x7024	**DRAM_DBG_EX_MON**	31:0		R	X	Exclusive monitors

Table 31-1. DRAM Registers.

* (P) indicates privileged access only

31.1.1 Global Physical Memory Map

Start	End	Description
0x0	0xDFFD_FFFF	Actual physical memory (4 GB max)
0xDFFE_0000	0xDFFE_FFFF	DRAM config space (for AXI0 only). For other clients' access to physical DRAM memory.
0xDFFF_0000	0XFFFF_FFFF	Actual physical memory (4 GB max)

Table 31-2. Global Physical Memory Map.

For Mekra Lang only Confidential 1.5 (08012025)

31.1.2 Safety-Related Debug Registers

Offset	Field	Bits	Access	Reset	Description
0x7F00	**ERROR_STATUS_REG_Error**	0	RW	0x1	Fatal error reporting status for read Writing with 1, reset error
0x7F00	**ERROR_STATUS_REG_injection**	1	W	X	Fatal error reporting injection Writing with 1, inject error
0x7F00	**ERROR_STATUS_REG_INTF**	9:2	R	X	Synchronizer and input interface monitor per clk domain [2] gclk_dram [3] clk_core [4] clk_core_2x [5] clk_ahb_core [6] clk_axi0 [7] clk_axi1 [8] clk_vision [9] clk_fec
0x7F00	**ERROR_STATUS_REG_LOCKSTEP**	17:10	R	X	Lockstep monitor per clk domain [10] gclk_dram [11] clk_core [12] clk_core_2x [13] clk_ahb_core [14] clk_axi0 [15] clk_axi1 [16] clk_vision [17] clk_fec
0x7F00	**ERROR_STATUS_REG_MEM**	25:18	R	X	Memory monitor per clk domain [18] gclk_dram [19] clk_core [20] clk_core_2x [21] clk_ahb_core [22] clk_axi0 [23] clk_axi1 [24] clk_vision [25] clk_fec
0x7F00	**ERROR_STATUS_REG_WatchDog**	28:26	R	X	Watchdog timer monitor [26] write data watchdog timer [27] read data & mmu watchdog timer
0x7F04	**ERROR_MASK_REG_Error**	0	RW	0x1	Fatal error reporting Mask bit
0x7F04	**ERROR_MASK_REG_INTF**	9:2	RW	8'hff	Synchronizer and input interface mask per clk domain [2] gclk_dram [3] clk_core [4] clk_core_2x [5] clk_ahb_core [6] clk_axi0 [7] clk_axi1 [8] clk_vision [9] clk_fec

For Mekra Lang (08012025)
1.5
Confidential

Offset	Field	Bits	Access	Reset	Description
0x7F04	**ERROR_MASK_REG_LOCKSTEP**	17:10	RW	8'hff	Lockstep mask per clk domain [10] gclk_dram [11] clk_core [12] clk_core_2x [13] clk_ahb_core [14] clk_axi0 [15] clk_axi1 [16] clk_vision [17] clk_fec
0x7F04	**ERROR_MASK_REG_MEM**	25:18	RW	8'hff	Memory mask per clk domain [18] gclk_dram [19] clk_core [20] clk_core_2x [21] clk_ahb_core [22] clk_axi0 [23] clk_axi1 [24] clk_vision [25] clk_fec
0x7F04	**ERROR_MASK_REG_Watch-Dog**	28:26	RW	3'h7	Watchdog timer mask [26] write data watchdog timer [27] read data watchdog timer [28] mmu watchdog timer
0x7F08	**WATCH DOG_TIMER_0_CON-FIG**	15:0	RW	0x0	The number of cycles/16 to expire wdata Watchdog timer (1 ms max) Writing 0 turns off the watchdog timer. The watchdog timers will start counting with write pending the queue available flag and reset by dram_ecch_wdata_valid
0x7F0c	**WATCH DOG_TIMER_1_CON-FIG**	15:0	RW	0x0	The number of cycles/16 to expire rdata & mmu Watchdog timer (1 ms max) Writing 0 turns off the watchdog timer. The rdata watchdog timers will start counting with read pending the queue available flag and reset by ecch_dram_rdata_valid The mmu watchdog timers will start counting with arb_mmu_req_valid and reset by arb_mmu_req_next

Table 31-3. Safety-Related Debug Registers.

31.1.3 Client Interface (Clientif)

client_id	Client Name
0	AXI0 (for programming priorities)
1	AXI1
2	ARM_DMA0
3	ARM_DMA1
4	ENET0

For Mekra Lang (China) Co. 2012 2023 Confidential

client_id	Client Name
5	ENET1
6	Flash DMA (FDMA)
7-12	CANC0-5
13	GDMA
14-15	SDXC0/1
16	USB20 (Device)
17	Usb_host
18	OrcME
19	OrcCode
20	Orcvp
21	OrcL2 Cache
22	SMEM
23	VMEM0
24	FEX
25	DBSE

Table 31-4. Clientif ID and Name.

31.2 DRAM BIST and Scrubbing Engine (DBSE)

This section details the software interface of the DBSE. DBSE sits behind DRAM as one of its clients with 64-bit read / write interface to DDR (similar to other clients).

DBSE contains a BIST engine and a scrubbing engine. The BIST engine sends out multiple test patterns for comprehensive memory testing, while the scrubbing engine performs continuous, periodic reads with exclusive locks and write the same data back. This enables removal of any latent 1-bit, ECC errors in the memory.

The following table lists the architectural registers for control / debug. The base address is 0xED260000.

Offset	Register	Bits	Access	Description
0x000- 0x003	**DBSE_BIST_ERROR**	31:0	R	Debug information for BIST [0] = Memory Address 0 [1] = Memory Address 1 [2] = Read Data from Address 0 [3] = Read Data from Address 1
0x004	**DBSE_START_ADDR**	31:0	RW	Start Address for DBSE Engine(*1)
0x005	**DBSE_END_ADDR**	31:0	RW	End Address for DBSE Engine(*1)
0x006	**DBSE_WT_DELAY**	15:0	RW	Controls the frequency of scrubbing atomic operations. Hardware will wait for these many clocks before initiating another atomic operation. Atomic operation is defined as one read with EX-OR and one write operation.
0x006	**DBSE_RANDOM_SEED**	31:16	RW	Random seed used for pattern generation
0x007		31:0		
0x008		15:0		

Confidential
For Mekta Lab only
1.5 Oct 2020

Offset	Register	Bits	Access	Description
0x008	**DBSE_BIST_DONE**	16	R	It is set by hardware when BIST is done. It is reset by hardware on the rising edge of DBSE_EN and (DBSE_MODE == 0 || DBSE_MODE==1)
0x008	**DBSE_BIST_STATUS**	17	R	0 = Pass 1 = Fail Valid only when DBSE_BIST_DONE == 1
0x008	**DBSE_BURST_SIZE**	19:18	RW	00:1 Burst 01: 2 Burst 10: 4 Burst 11: Not used
0x008	**DBSE_BIST_FAIL_MODE**	24:20	R	Records fail mode information 5'd15 will be used as reset value
0x008	**Reserved**	31:25		
0x009	**DBSE_EN**	0	RW	0 = DBSE is disabled 1 = DBSE is enabled(*2)
0x009	**ENABLE_CODE**	8:1	RW	0x2D to enable the DBSE
0x009	**DBSE_MODE**	10:9	RW	0 = Run BIST 1 = Run BIST and then scrubbing engine 2 = Unused 3 = Run scrubbing engine only (software's responsibility that memory is initialized with ECC) **Note**: Format DRAM is replaced by DBSE_MODE = 0 & DBSE_BIST_PAT-TERN = 'h1.
0x009	**DBSE_BIST_PATTERN**	23:11	RW	[0] = Random [1] = Sequential increment [2] = Solid bits [3] = Block sequential [4] = Checkerboard [5] = Bit spread [6] = Bit flip [7] = Walking ones [8] = Walking zeros [9] = Compare XOR [10] = Compare SUB [11] = Compare OR [12] = Compare AND One or more patterns can be selected at the same time
0x009	**Reserved**	25:24		
0x009	**DBSE_BIST_LOOP**	31:26	RW	Loop control register for mode 2–8

Table 31-5. Registers for Control / Debug.

Notes:

*1: For BIST, ADDR[8:0] must be 0 because DBSE does not handle page crossing. The minimum size, per checking, is 1K bytes. For scrubbing, ADDR[7:0] must be 0 to avoid page crossing.

*2: Before DBSE is enabled, users must write a non-zero pattern and read it back to ensure that the data bus is not corrupted, and avoid the DBSE potentially falsely reporting a pass status.

Confidential For Mekra Lang 1.5 (08012025)

32. ECC HANDLER (ECCH)

The CV22FS chip supports single-error correction and double-error detection (SECDED) ECC to protect data in the main memory. The ECC bits are stored in a separate DRAM device, away from the main memory. The ECCH module computes the ECC bits on writes and then stores them on separate ECC memory. The module will read ECC bits from ECC memory on reads and then apply SECDED on read data, using an in-house developed error correction code of n = 298, r = 16, where m is the codeword block length, and n is the number of parity bits.

The codeword bits include 256 bits of data, 26 bits of address, and 16 bits of parity. Based on a random simulation, ECCH has a 98.82% probability of detecting three bit errors. To improve the ECC memory bandwidth, ECC bits are locally cached. The hardware is responsible for handling read-modify-write accesses on partial writes. The main memory burst size is 64 bytes, covered by two blocks of ECC, which enables single-bit error correction in each half of the burst.

Notes:

- Software must initialize the main memory region that includes ECC protection in order to avoid reading from uninitialized memory. The DBSE hardware engine can be used to initialize memory.
- The ECC has a SECDED capability for each half burst (32 bytes).

This chapter provides the following information about the CV22FS ECC Handler (ECCH):

- (Section 32.1) ECCH: Programmer Visible Registers
- (Section 32.2) ECCH: Safety-Related Debug Registers

32.1 ECCH: Programmer Visible Registers

The following lists the programmer visible register for the software interface (architectural registers for control / debug).

Offset	Register	Bits	Reset Value	Description
0x0	**REG_ECCH_MODE**	0	0x0	0: ECC disabled 1: ECC enabled
		2:1		Main DDRHOST Bank_mode 0: LPDDR4 - 8 banks 1: LPDDR4 - 16 banks dual-die 2: DDR4 - 2BG x 4Bank / BG
		31:3		Reserved
0x8	**REG_ECCH_ARB**	4:0	0x0	Arb_read_reserve Cache Arbiter read / write switching reserve cycle for performance tuning The ECC cache arbitrates the eviction writes and read miss fills. It will reserve **Arb_read_reserve** number of DRAM controller clock cycles for reads once a read miss fill request is granted, before switching the grant to eviction writes

Confidential For Mekra Lang only 1.5 Use 01/2021

Offset	Register	Bits	Reset Value	Description
0x8 (Cont'd)		11:8	0x0	Arb_write_bank_reserve Cache Arbiter write bank reserve cycle for performance tuning Because the ECC DDRHOST adopts an open-page policy, ECC Cache reserves **Arb_write_bank_reserve** number of DRAM controller clock cycles to prevent successive eviction writes to different pages (rows) of the same DDR bank

Table 32-1. Programmer Visible Registers.

32.2 ECCH: Safety-Related Debug Registers

Offset	Register	Bits	Access	Reset Value	Description
0x60	**SAFETY_STATUS**	0	RW	0x1	Fatal error reporting status Write this bit with 1 to clear all errors on offset 0x60-0x6c
		1	W	X	Fatal error reporting injection Writing this bit with 1 to inject error on bit [0] of this offset
		7:2	R	X	Fatal error reporting status error module [2] MMU [3] cfg [4] hamming [5] cache [6] asyncfifos ddrc0 [7] asyncfifos ddrce
		10:8	R	X	[8] Main correctable error status [9] ECC correctable error status [10] uncorrectable error status
		31:11		X	Reserved
0x64	**SAFETY_MASK**	0	RW	32'hffff_ffff	Reserved
		1	RW		Reserved
		10:2	RW		Mask bit for position 0x60[10:2] 0: Error disabled 1: Error enabled
		31:11	RW		Reserved
0x68	**MAIN_CORRECTABLE_COUNTER**	15:0	R	0x0	Main correctable error counter Reset by writing 1 to 0x60[0]
		31:16	RW	16'hffff	Main correctable error threshold
0x6c	**ECC_CORRECTABLE_COUNTER**	15:0	R	0x0	ECC correctable error counter Reset by writing 1 to 0x60[0]
		31:16	RW	16'hffff	ECC correctable error threshold

Table 32-2. Safety-Related Debug Registers.

Confidential For Wekta Lang 1.5 (08012025)

33. TEMPERATURE SENSOR

This chapter provides information about the temperature sensor module (temperature sensor and its controller) for the CV22FS chip.

The chapter is organized as follows:

- (Section 33.1) Temperature Sensor: Overview
- (Section 33.2) Temperature Sensor: Controller Registers
- (Section 33.3) Temperature Sensor: Programming Notes

33.1 Temperature Sensor: Overview

The temperature sensor module includes the temperature sensor, tem1501x, and its controller. Users send the configuration information to the controller and temperature sensor; which then returns the temperature information to the specific registers.

33.2 Temperature Sensor: Controller Registers

There are two independent temperature sensor controllers, placed closely to their four probes.

33.2.1 Temperature Sensor Controller Registers: Map

The following includes a map and details of the registers for the temperature sensor controller.

The base address is 0xED0D0000.

Offset	Register	Description
0x80	**ts_en**	Enable
0x84	**ts0_ctl0**	Control
0x88	**ts0_ctl1**	Control
0x8C	**ts0_sample_num**	Sample numbers
0x90	**ts0_probe0**	Sample probe ID
0x94	**ts0_probe1**	Sample probe ID
0x98	**ts0_data0**	Temperature code output
0x9C	**ts0_data1**	Temperature code output
0xA0	**ts0_data2**	Temperature code output
0xA4	**ts0_data3**	Temperature code output
0xA8	**ts0_data4**	Temperature code output
0xAC	**ts0_data5**	Temperature code output
0xB0	**ts0_data6**	Temperature code output
0xB4	**ts0_data7**	Temperature code output
0xB8	**ts1_ctl0**	Control
0xBC	**ts1_ctl1**	Control
0xC0	**ts1_sample_num**	Sample numbers
0xC4	**ts1_probe0**	Sample probe ID

Confidential For Mekra Lang only 1.5 (2012-05-25)

Offset	Register	Description
0xC8	**ts1_probe1**	Sample probe ID
0xCC	**ts1_data0**	Temperature code output
0xD0	**ts1_data1**	Temperature code output
0xD4	**ts1_data2**	Temperature code output
0xD8	**ts1_data3**	Temperature code output
0xDC	**ts1_data4**	Temperature code output
0xE0	**ts1_data5**	Temperature code output
0xE4	**ts1_data6**	Temperature code output
0xE8	**ts1_data7**	Temperature code output

Table 33-1. Temperature Sensor Controller Registers of the RTC_DBG Interface.

33.2.2 Temperature Sensor Registers: Detail

33.2.2.1 TS_EN Register

Bits	Name	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	**rtc_en_ts1**	RW	0	Temperature sensor 1 enable port 0 Disabled 1 Enabled
0	**rtc_en_ts0**	RW	0	Temperature sensor 0 enable port 0 Disabled 1 Enabled

Table 33-2. Temperature Sensor Controller Enable Register.

33.2.2.2 TS0/1_CTL0 Register

Bits	Name	Access	Reset	Description
31:28	**bgr_r_trim_ts0/1**	RW	0x0	BGR control ports in the reference generator
27:24	**bgr_i_trim_ts0/1**	RW	0x0	BJT emitter current control ports in the reference generator
23		-	-	Reserved
22:20	**avg_mode_ts0/1**	RW	0x6	Average mode control ports for the digital filter
19:18	**Reserved**	-	-	Reserved
17	**en_dem_ts0/1**	RW	1	Dynamic element matching enable port
16	**en_ch_ts0/1**	RW	1	Digital offset cancellation enable port
15	**Reserved**	-	-	Reserved
14:12	**mux_addr_ts0/1**	RW	0x0	MUX address control port for internal node monitoring Use for testing purpose only
11:9	**Reserved**	-	-	Reserved
8:4	**buf_vref_sel_ts0/1**	RW	0x00	Offset control ports for temperature output code
3:0	**buf_slope_sel_ts0/1**	RW	0x5	Slope control ports for temperature output code

Table 33-3. Temperature Sensor Controller Control_0 Register.

Confidential
For Meirang only
Feb 08 2025

33.2.2.3 TS0/1_CTL1 Register

Bits	Name	Access	Reset	Description
31:29		-	-	Reserved
28	**en_read_ts0/1**	RW	0	DAC test mode selection Use for testing purpose only Normal operation mode—DAC mode
27:25		-	-	Reserved
24:16	**vref_vbe_sel_ts0/1**	RW	0x000	DAC input ports for DAC test
15:10		-	-	Reserved
9:8	**comp_i_trim_ts0/1**	RW	0x0	Comparator current trimming ports
7:4	**vbe_i_trim_ts0/1**	RW	0x0	Current trimming ports for VBE in PTAT generator
3:0	**vref_trim_ts0/1**	RW	0x0	Reference voltage trimming ports for reference generator

Table 33-4. Temperature Sensor Controller Control_1 Register.

33.2.3 TS0/1_SAMPLE_NUMBER Register

Bits	Name	Access	Reset	Description
31:4		-	-	Reserved
3:0	**sample_num_m1_ts0/1**	RW	0xF	Sample times of temperature sensor (= register value+1)

Table 33-5. Temperature Sensor Controller Sample Number Register.

33.2.3.1 TS0/1_PROBE0 Register

Bits	Name	Access	Reset	Description
31		-	-	Reserved
30:28	**sample_probe_7_ts0/1**	RW	0x3	Probe_7 point ID (0~4)
27		-	-	Reserved
26:24	**sample_probe_6_ts0/1**	RW	0x3	Probe_6 point ID (0~4)
23		-	-	Reserved
22:20	**sample_probe_5_ts0/1**	RW	0x2	Probe_5 point ID (0~4)
19		-	-	Reserved
18:16	**sample_probe_4_ts0/1**	RW	0x2	Probe_4 point ID (0~4)
15		-	-	Reserved
14:12	**sample_probe_3_ts0/1**	RW	0x1	Probe_3 point ID (0~4)
11		-	-	Reserved
10:8	**sample_probe_2_ts0/1**	RW	0x1	Probe_2 point ID (0~4)
7		-	-	Reserved
6:4	**sample_probe_1_ts0/1**	RW	0x0	Probe_1 point ID (0~4)
3		-	-	Reserved
2:0	**sample_probe_0_ts0/1**	RW	0x0	Probe_0 point ID (0~4)

Table 33-6. Temperature Sensor Controller Probe_0 Register.

For Mikai Lang only Confidential 1.5 20180120

33.2.3.2 TS0/1_PROBE1 Register

Bits	Name	Access	Reset	Description
31		-	-	Reserved
30:28	**sample_probe_15_ts0/1**	RW	0x0	Probe_15 point ID (0~4)
27		-	-	Reserved
26:24	**sample_probe_14_ts0/1**	RW	0x3	Probe_14 point ID (0~4)
23		-	-	Reserved
22:20	**sample_probe_13_ts0/1**	RW	0x1	Probe_13 point ID (0~4)
19		-	-	Reserved
18:16	**sample_probe_12_ts0/1**	RW	0x4	Probe_12 point ID (0~4)
15		-	-	Reserved
14:12	**sample_probe_11_ts0/1**	RW	0x2	Probe_11 point ID (0~4)
11		-	-	Reserved
10:8	**sample_probe_10_ts0/1**	RW	0x0	Probe_10 point ID (0~4)
7		-	-	Reserved
6:4	**sample_probe_9_ts0/1**	RW	0x4	Probe_9 point ID (0~4)
3		-	-	Reserved
2:0	**sample_probe_8_ts0/1**	RW	0x4	Probe_8 point ID (0~4)

Table 33-7. Temperature Sensor Controller Probe_1 Register.

33.2.3.3 TS0/1_DATA0 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_1_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_0_ts0/1**	RO	0	Temperature code output

Table 33-8. Temperature Sensor Controller Data_0 Register.

33.2.3.4 TS0/1_DATA1 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_3_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_2_ts0/1**	RO	0	Temperature code output

Table 33-9. Temperature Sensor Controller Data_1 Register.

For Ambarella Internal Confidential Only (1.5 08/01/25)

33.2.4 TS0/1_DATA2 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_5_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_4_ts0/1**	RO	0	Temperature code output

Table 33-10. Temperature Sensor Controller Data_2 Register.

33.2.4.1 TS0/1_DATA3 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_7_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_6_ts0/1**	RO	0	Temperature code output

Table 33-11. Temperature Sensor Controller Data_3 Register.

33.2.4.2 TS0/1_DATA4 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_9_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_8_ts0/1**	RO	0	Temperature code output

Table 33-12. Temperature Sensor Controller Data_4 Register.

33.2.4.3 TS0/1_DATA5 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_11_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_10_ts0/1**	RO	0	Temperature code output

Table 33-13. Temperature Sensor Controller Data_5 Register.

Confidential only For Hekra use only 1.5 (09/12/25)

33.2.4.4 TS0/1_DATA6 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_13_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_12_ts0/1**	RO	0	Temperature code output

Table 33-14. Temperature Sensor Controller Data_6 Register.

33.2.4.5 TS0/1_DATA7 Register

Bits	Name	Access	Reset	Description
31:25		-	-	Reserved
24:16	**sample_data_15_ts0/1**	RO	0	Temperature code output
15:9		-	-	Reserved
8:0	**sample_data_14_ts0/1**	RO	0	Temperature code output

Table 33-15. Temperature Sensor Controller Data_7 Register.

33.3 Temperature Sensor: Programming Notes

33.3.1 Overview

There are two operation modes: DOC (digital offset cancellation) and NON-DOC. Users should set a configuration based on the selected operation mode prior to activating the sensor. Ambarella supports up to 5 points (sensor and 4 probe nodes) and 16 times in succession. Users can configure TS*_SAMPLE_NUM and TS*_PROBE* to check or monitor the desired temperature.

Temperature code is stored in the buffer for both modes. After users enable the temperature sensor and its controller, the temperature code output is stored in the corresponding buffer at the expected time.

Note:

If the temperature sensor module is functioning in DOC mode (refer to the following section) with 2-sample average mode, the first data in buffer is invalid.

33.3.2 DOC Mode

The following table shows the temperature sensor 0 (TS0) operating in DOC mode with 2-sample average mode. In this mode, the temperature code averages a value of two places. Users can poll rtc_en_ts0 to check if the temperature sensing is complete, and then read the temperature code in the buffer to perform calibration.

Command	Description
Write 0x8c 0xf	Set sample times
Write 0x90 0x33221100	Set select probe point
Write 0x94 0x01342044	Set select probe point
Write 0x80 0x1	Enable temperature sensor 0

Command	Description
Polling	Wait rtc_en_ts0 = 0
Read 0x98	Read temperature code
Read 0x9c	Read temperature code

Table 33-16. Sensor 0 Operation Under DOC Mode.

33.3.3 NON-DOC Mode

The following table shows the temperature sensor 1 (TS1) operating in NON-DOC mode. In NON-DOC mode, the temperature code would be one probe or the sensor temperature code value. Users should disable en_ch_ts/ en_dem_ts/avg_mode_ts with zero. Users can poll rtc_en_ts1 to check if the temperature sensing is com-plete, and then read the temperature code in the buffer to perform calibration.

Command	Description
Write 0xb8 0x00200005	Disable en_ch_ts/en_dem_ts/avg_mode_ts
Write 0xc0 0xf	Set sample times
Write 0xc4 0x33221100	Set select probe point
Write 0xc8 0x01342044	Set select probe point
Write 0x80 0x2	Enable temperature sensor 1
Polling	Wait rtc_en_ts1 = 0
Read 0xcc	Read temperature code
Read 0xd0	Read temperature code

Table 33-17. Sensor 1 Operation Under NON-DOC Mode.

For Internal Confidential Use Only (1.5 20250801)

34. IMPORTANT NOTICE

All Ambarella design specifications, datasheets, drawings, files, and other documents (together and separately, "materials") are provided on an "*as is*" basis, and Ambarella makes no warranties, expressed, implied, statutory, or otherwise with respect to the materials, and expressly disclaims all implied warranties of noninfringement, merchantability, and fitness for a particular purpose. The information contained herein is believed to be accurate and reliable. However, Ambarella assumes no responsibility for the consequences of use of such information.

Ambarella reserves the right to correct, modify, enhance, improve, and otherwise change its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

All products are sold subject to Ambarella's terms and conditions of sale supplied at the time of order acknowledgment. Ambarella warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with its standard warranty. Testing and other quality control techniques are used to the extent Ambarella deems necessary to support this warranty.

Ambarella assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using Ambarella components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards and ensure their products and applications meet applicable standards and safety requirements.

Ambarella does not warrant or represent that any license, either expressed or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right relating to any combination, machine, or process in which Ambarella products or services are used. Information published or provided by Ambarella regarding third-party products or services does not constitute a license from Ambarella to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from Ambarella under the patents or other intellectual property of Ambarella.

Reproduction of information from Ambarella documents is not permissible without prior approval from Ambarella.

Ambarella products are not authorized for use in safety-critical applications (such as life support) where a failure of the product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Customers acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of Ambarella products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by Ambarella. Further, customers must fully indemnify Ambarella and its representatives against any damages arising out of the use of Ambarella products in such safety-critical applications.

Ambarella products are neither designed nor intended for use in military/aerospace applications or environments. Customers acknowledge and agree that any such use of Ambarella products is solely at the customers' risk, and they are solely responsible for compliance with all legal and regulatory requirements in connection with such use..

Confidential
For Mikrotik only
(c) 2025

35. REVISION HISTORY

Our goal is to provide our customers with the highest-quality documentation possible, and to continuously improve our publications to ensure that your experience with Ambarella's products is a positive one. If you have any questions or comments regarding this document, please contact the Technical Writing team at docs@ambarella.com. Your feedback is welcomed and appreciated.

NOTE: Page numbers for previous drafts may differ from page numbers in the current version.

Version	Date	Comments
0.1	9 March 2020	Initial Version
0.2	11 June 2020	Updated Figure 1-1 Functional Block Diagram Updated Table 1-3 APB Address Ranges Updated Table 2-5 SPI NOR Boot Options for SPI mode selection Updated Table 5-1 Graphics DMA Register Map for fatal error, error detected registers Added Table 5-15 Graphics DMA Fatal Error Register Added Table 5-16 Graphics DMA Error Detected Register Updated Section 6.5 VIN: Programming Details for MIPI mode Updated Section 7.9.7 VOUTD_digital_output_mode to add note Added Section 7.13 VOUT: Safety Notes Updated Section 8.1 Delayed VSync: Overview for source selection Updated Table 13-2 Flash IO Registers Added Section 13.5 Flash: Safety Notes Updated Table 14-1 SD PHY Registers for register at 0x4c4; descriptions Added Section 14-3 SD: Safety Notes Updated Tables 14-15, 14-19, 14-20 to remove vendor_specfic_error Updated Table 14-21 SD Controller Auto CMD12 Error Status and Host Control 2 Register Updated Section 14.2.6.4 SD_transfer_mode_and_command Register Updated Table 15-1 Non-Secure Scratchpad Registers for VSDELAY registers; updated bits 7, 8 description for offset 0xd; added 0x148 (CTRL26) Updated Table 16-6 Security: OTP Data Format Updated Figure 16-1 Security Overview: System Bus Architecture for comments Updated Section 16.2 Security: VROM and OTP for notes Updated Table 16-4 DRAM Access: DRAM Clients to remove sorc, add enet1 Updated Section 16.7.1 Security: OTP Data Format Updated Section 16.7.3 Security: Public Key Management Updated Section 16.8.2 Boot: Secure Boot to remove secure boot process diagrams Updated Section 16.8.4 Boot: Secure Warm Boot to remove AES Updated Chapter 17 AXI Configuration for introduction Updated Section 18.3 GIC: Interrupt Vectors Updated Table 19-1 Address Offsets for Each Set of GPIO Registers for GPIO fault inject Added Table 19-14 GPIO Fault Inject Register Added Section 20.4 SSI / SPI: SSI Safety Notes Added Section 20.6 SSI / SPI: SPI NOR Safety Notes Updated Section 22.1 I2C: Overview for 374 Kbps bit rate Updated Section 23.4.2 Register Programming Notes to add description Added Section 23.4.2.4 Software Abort Flow

Version	Date	Comments
0.2 (Cont'd)		Added Section 23.4.2.5 Message Buffer RX Filter Flow Updated Table 20-39 SSI / SPI: SPI NOR Registers for CLKDIVIDER field Added Section 26.4 Interval Timer: Safety Notes Updated Table 30-1 AMBA ECRU Registers for error log reset values Updated Table 31-1 DRAM Registers Updated Table 32-1 Programmer Visible Registers Updated Table 32-2 Safety-Related Debug Registers
0.3	20 November 2020	Updated Table 2-2 Power On Clock Rates (MHz) Updated Table 2-5 SPI NOR Boot Options to remove SCPL Updated Table 7-8 Vout_Enable_SmemIf_Clock_gating Updated Section 13.13.1 NAND Flash: Overview to remove command confirmation Incorporated A1 change: Removed references to FPDLINK
0.4	30 June 2021	Updated Figure 3-1 The DSP Subsystem to remove SLVS, LVDS, LVCMOS, FPD, stereo Updated Section 6.1 VIN: Overview Updated Section 6.5 VIN: Programming Details Updated Table 6-6 VIN: Registers Updated Table 7-54 VOUTD_background for digital_output_ mode Added Section 7.13 VOUT: BTA Programming Flow
0.5	1 November 2021	Updated Chapter 1 Introduction for third party documents Updated Section 2.2.8 POC: Boot Bypass Updated Section 2.2.3 POC: Boot Mode Summary for note Updated Table 2-1 Power On Configuration Pin Assignments for bit 24 function Updated Table 2-4 Shared Boot Options Matrix to remove hs_boot Updated Table 2-6 SPI NAND Boot Options for pin 20 description Updated Table 2-7 MMC Boot Options to remove hs_boot; updated pin 20 description Updated Chapter 4 DMA to remove dual-space mode (spare-area) information Updated Section 10.2.2 Ethernet: MAC Register Map Updated Table 10-4 Ethernet PBL Values for FIFO Depth Added Section 10.5 Ethernet: Programming Guidelines Updated Table 10-18 Ethernet Controller MAC Register1 for description of saf register Updated Table 10-31 Ethernet Controller MAC Register18 is the MAC Address1 High Register to update mbc description Updated Table 10-53 Ethernet Transmit Descriptor 0 Register for bit 11 Updated Table 14-28 SD Controller Boot Control Register for boot_size bits and reset values Updated Section 13.3.4 SPI NAND Flash: Initialization to add notes Updated Table 15-1 Non-Secure Scratchpad Registers for 0xb4 to add bit 1/0; add 0x10c Updated Table 17-1 AXI Registers for descriptions Updated Section 13.3.4 SPI NAND Flash: Initialization to add notes Updated Section 18-1 GIC: Overview to add reference to GIC-400 manual Updated Table 19-1 Address Offsets for Each Set of GPIO Registers to remove GPIO~3_AFSEL

For Ambarella Internal Use Only 1.5 Confidential

Version	Date	Comments
0.5 (Cont'd)		Removed Table 19-8 GPIO Mode Select Register Updated Table 19-15 GPIO Function Selection: Assignment Table for GPIO24 Alt1, GPIO25 Alt1, GPIO58 Alt5 Updated Figure 20-1 SPI Interface with the SSI / SPI as the Serial Master and Two Slave Devices on the Serial Bus Updated Section 23.4.2 Register Programming Notes Added Section 23.5 CAN Controller: Safety Notes Updated Section 24.2.1 RTC Registers: Map to add RTC_Strobe Updated Table 28-1 Sources to ID Mapping Table and added notes Updated Table 32-1 Programmer Visible Registers Updated Table 32-2 Safety-Related Debug Registers
0.6	12 December 2021	Updated Section 2.2.7 POC: MMC Boot to add note Updated Section 19.4.2 GPIO Function Selection: Programming Procedures Updated Table 19-12 GPIO Enable Register for attr Updated Table 23-2 CAN Bus Controller: Control Registers to remove constraint for tx_delay_compensate_enable Added Section 31.2 DRAM BIST and Scrubbing Engine (DBSE)
0.7	28 June 2022	Added Section 11-8 USB Device: DMA Operation Updated Table 14-32 Read Latency control Register for ncr_reg description Updated Table 19-12 GPIO Enable Register Updated Chapter 23 Controller Area Network (CAN) Bus to add note about allowed formats
0.8	17 January 2023	Updated Table 2-7 MMC Boot Options for description of POC pin 17 Updated Table 20-39 SSI / SPI: SPI NOR Registers for note at 0x58
0.9	5 September 2023	Updated Table 20-39 SSI SPI: SPI NOR Registers for description of offset 0x04 bits 31 through 28 Updated Table 20-39 SSI SPI: SPI NOR Registers for description of offset 0x58 bits 31 through 20; bit 0 Updated Chapter 32 ECC Handler (ECCH) for introduction Updated Table 32-1 Programmer Visible Registers for offset 0x8 description
1.0	22 March 2024	Updated Table 11-1 USB CSR Memory Map for USB_end2, USB_end5 register offsets Updated Section 15.3 Scratchpad: Secure Registers to add note Updated Section 18.8 GIC: GIC Virtual CPU Interface Register Summary for description of GICV_AIAR
1.1	6 April 2024	Updated Table 11-1 USB CSR Memory Map
1.2	10 July 2024	Updated Chapter 1 Introduction for the functional block diagram Updated Table 2-3 Power On Boot Mode for note Updated Section 16.1 Security: Overview for the functional block diagram Added Section 7.15 VOUT: MIPI DSI-PHY Protocol Timing
1.3	12 November 2024	Updated Table 2-3 Power On Boot Mode for SPI NOR Boot, POC Bit 20 Updated Table 2-4 Shared Boot Options Matrix for SPI NOR, boot_option[7] Updated Table 2-5 SPI NOR Boot Options to add "Signed Image Size"
1.4	27 February 2025	Updated Table 21-1 UART Interface Registers Updated Section 21.3.2.6 UART_fcr Register Updated Table 21-11 UART Modem Control Register (MCR) Updated Table 21-15 UART DMA Enable (UART_AHB) Register (DMAE)
1.5	1 August 2025	Updated Section 15.3 Scratchpad: Secure Registers for note Updated Table 27-8 WDT Reset Width Register

Table 35-1. Revision History.

For Internal Use Only 1.5 Confidential